



500 Problemas Reales de Lógica de Programación

De Junior a Senior, en cualquier lenguaje

10 niveles · 50 ejercicios por nivel · 30 desafíos integradores

Situaciones reales · Salidas esperadas verificadas · Sin soluciones: el código lo escribís vos

Cómo usar este libro

Este libro tiene 500 problemas originales, ordenados de menor a mayor dificultad en 10 niveles de 50. Todos se pueden resolver en cualquier lenguaje de programación: ningún enunciado depende de una herramienta, biblioteca o sintaxis particular.

Cómo está armado cada ejercicio

- **Situación:** un problema real con personajes y contexto. Leela como si te lo pidiera un cliente.
- **Datos de entrada:** todo lo que necesitás, sin buscar nada afuera. Desde el nivel 6 incluye formato, ejemplo y restricciones de tamaño.
- **Tu programa debe:** los pasos concretos, en orden.
- **Salida esperada:** el resultado exacto para los datos dados, verificado con una solución de referencia. Tu programa tiene que producir exactamente eso, carácter por carácter.
- **Reglas:** redondeos, límites, complejidad esperada y casos borde. Desde el nivel 8 también hay requisitos no funcionales (latencia, memoria, auditoría, tolerancia a fallas).
- **Pista:** leela solo si te trabaste de verdad. **Bonus:** extensiones para cuando ya lo resolviste. **Cierre:** dónde se usa esto en la industria.

Convenciones

- En las **salidas esperadas** los decimales van con punto y sin separador de miles (así las compara cualquier programa). En las tablas de datos se usa el formato local: \$12.000 o 7,2 km.
- Salvo que el ejercicio diga otra cosa, se redondea al final y "0,5 hacia arriba".
- Los generadores de números aleatorios están especificados con su fórmula y su semilla: con la misma semilla, tu salida tiene que ser idéntica.
- Los últimos 3 ejercicios de cada nivel son **desafíos de nivel:** integran varias técnicas y son más largos.

Cómo recorrerlo

- **Si estás empezando:** niveles 1 a 3 en orden. No saltees: cada nivel supone el anterior.
- **Si ya programás:** hacé los desafíos de los niveles 1 a 3 como diagnóstico y arrancá en el nivel donde te trabes.
- **Para entrevistas técnicas:** niveles 4 a 7, cuidando la complejidad pedida en las reglas.
- **Para roles senior y staff:** niveles 8 a 10, donde importan tanto el algoritmo como el diseño del sistema.

Resolvé primero en papel, después programá, y recién al final compará con la salida esperada. Si no coincide, el error casi siempre está en un caso borde o en un redondeo: releé las reglas.

Índice

Cómo usar este libro	2
-----------------------------	----------

Nivel 1: Junior I • Fundamentos	16
--	-----------

1. La propina de la mesa 4	17
2. ¿Cuánto tarda en llegar el colectivo?	18
3. Termómetro de dos escalas	19
4. El kiosco que cobra con descuento	20
5. ¿Aprobé o voy a recuperatorio?	21
6. Plazo fijo a 30 días	22
7. El semáforo del IMC	23
8. Horas extra del turno noche	24
9. Cuenta regresiva para el lanzamiento del globo sonda	25
10. El precio del envío según el peso	27
11. ¿Me alcanza para la cuota?	28
12. Megas que quedan en el plan	29
13. Goles por partido del torneo barrial	30
14. La factura de luz escalonada	31
15. Vidas y escudo del personaje	33
16. El metrónomo del ensayo	34
17. Cambio de moneda en el aeropuerto	35
18. Seguidores ganados en la semana	36
19. Tres intentos para entrar	37
20. La alcancía con meta de ahorro	38
21. Pasos diarios hacia los 10.000	39
22. La billetera virtual que rinde todos los días	40
23. Kilómetros por litro del camión de reparto	41
24. La receta para 12 comensales	42
25. Tablas de multiplicar para la seño	43
26. El año bisiesto del cumpleaños	44
27. Estacionamiento por fracción de hora	45
28. Préstamo entre amigos con interés simple	46
29. El vuelto con billetes y monedas	47
30. Paneles solares para el techo	48
31. Racha de victorias del equipo	49
32. Monedas de oro del cofre	50
33. ¿Cuotas sin interés o contado con descuento?	52
34. La helada de la madrugada	53
35. Duración total de la playlist	55
36. Presupuesto de las vacaciones en la costa	56
37. ¿Es primo el número de la rifa?	57

38. Del sueldo bruto al neto	58
39. Recargo por pago atrasado	60
40. El plan de datos que conviene	61
41. Dosis según el peso del paciente	62
42. Pedidos que entran en la camioneta	63
43. Likes que se duplican cada hora	64
44. El dígito que delata una tarjeta mal tipeada	66
45. Cuánto debo después de cada cuota	67
46. La cantina escolar y sus tickets	68
47. Rendimiento de la cosecha por hectárea	69
48. Desafío de nivel: La caja registradora del almacén	70
49. Desafío de nivel: El cajero automático que no da monedas	72
50. Desafío de nivel: La liga de fútbol 5 de los jueves	73

Nivel 2: Junior II · Colecciones y funciones 75

51. Notas del curso: la mejor y la peor	76
52. Días del mes por encima del promedio	77
53. Gastos del mes día por día	78
54. Patentes capicúa	79
55. Vocales y consonantes de la canción	80
56. Stock bajo en el depósito	81
57. Cuentas claras en el asado	82
58. Butacas libres en el teatro	83
59. Frecuencia cardíaca en reposo	84
60. El hashtag más largo del posteo	85
61. Conversor de monedas en funciones	86
62. Ta-te-ti: ¿quién ganó?	88
63. Nombres prolijos en la planilla de personal	90
64. Lluvia acumulada por semana en el campo	91
65. El producto estrella del día	92
66. ¿Contraseña fuerte o débil?	94
67. El mensaje secreto del club de ajedrez	96
68. Movimientos de la cuenta y el saldo más bajo	97
69. La vuelta más rápida del karting	99
70. Cobertura de antenas en la grilla	100
71. Menú del día que entra en el presupuesto	101
72. Vuelos demorados del día	103
73. Espejo de palabras para el cartel	104
74. El paciente con la fiebre más alta	105
75. Rotar la foto 90 grados	106
76. La tasa real de las cuotas	108
77. Consumo eléctrico por franja horaria	109
78. Contador de palabras para el ensayo	110

79. La parada de colectivo más cercana	111
80. Inventario de pociones	112
81. El CBU que no cierra	114
82. Planilla de asistencia del mes	116
83. Transportar acordes un tono arriba	117
84. Pedidos atrasados del día	118
85. El comentario con más reacciones	119
86. Glucemia de la semana	120
87. Tabla de posiciones sin ordenar	122
88. Cuánto rinde cada alternativa de ahorro	123
89. Sopa de letras de la escuela	124
90. Calorías del plato armado	125
91. SMS sospechosos en la bandeja	126
92. Parcelas con sequía	128
93. El robot del depósito sigue instrucciones	129
94. La acción que más subió en la semana	130
95. Horario de la medicación	131
96. Sensores que marcan valores imposibles	133
97. Una generación del juego de la vida	134
98. Desafío de nivel: El resumen de la tarjeta de crédito	136
99. Desafío de nivel: Batalla naval en la terminal	138
100. Desafío de nivel: Boletín de calificaciones del curso	140

Nivel 3: Junior III · Datos del mundo real 142

101. Invitados repetidos del casamiento	143
102. Palabras más repetidas en las reseñas	144
103. Gastos agrupados por categoría	145
104. La fila de la guardia por hora de llegada	146
105. Días hasta el vencimiento	148
106. Alumnos que cursan los dos talleres	149
107. Leer el log del servidor	150
108. Ranking de goleadores del campeonato	151
109. Turnos médicos que se pisan	152
110. Canciones favoritas en común	153
111. Postulantes con email inválido	154
112. ¿De cuántas formas sirvo los platos?	156
113. Stock consolidado de sucursales	157
114. Horas trabajadas por semana del año	159
115. Anagramas del juego de palabras	161
116. Tarifas de roaming por país	162
117. El extracto bancario desordenado	164
118. Viaje con escalas en distintos husos horarios	166
119. Hashtags en tendencia	167

120. Descendientes de la vaca campeona	168
121. Consumo por electrodoméstico y por mes	169
122. Validar un CUIT	171
123. Reservas duplicadas del hotel	172
124. La carta ordenada por precio y nombre	173
125. Traductor de código Morse	175
126. Pacientes por obra social	176
127. Suscripciones que vencen este mes	177
128. Planilla de jugadas del partido de básquet	179
129. Capítulos en números romanos	181
130. Los que no te siguen de vuelta	182
131. Direcciones mal cargadas	183
132. Las torres del museo de ciencias	185
133. Puntos de fidelidad por categoría	186
134. Cumpleaños del mes en la oficina	188
135. Intercalar dos listas ordenadas de transacciones	189
136. Lecturas dudosas de la estación meteorológica	191
137. Logros desbloqueados	193
138. Setlist sin repetir	194
139. Tareas de la chacra según el día de la semana	195
140. IPs con demasiados intentos	197
141. Pagos contra facturas	198
142. Tiempos de entrega por zona	200
143. Buscador de recetas por ingredientes	202
144. Todas las formaciones posibles del equipo	203
145. Fechas en tres formatos distintos	204
146. Aguinaldo proporcional	206
147. Calendario de vacunación	207
148. Desafío de nivel: El extracto de dos bancos en uno	208
149. Desafío de nivel: Padrón de socios del club	210
150. Desafío de nivel: Farmacia con vencimientos	212

Nivel 4: Semi-Senior I • Eficiencia 214

151. La calculadora que se queja de los paréntesis	215
152. Un DNI en un padrón de 10 millones	216
153. La cola de la sucursal bancaria	217
154. Dos productos que suman el cupón	219
155. Los 7 días de más ventas en un millón de registros	220
156. Ctrl+Z del editor de publicaciones	221
157. Saldo acumulado por rango de días	223
158. ¿Cuántos días hasta que haga más calor?	224
159. El mejor día para comprar y vender	225
160. El lado del potrero cuadrado	226

161. Semáforo y fila de autos	227
162. Reservas que se pueden fusionar	229
163. El tramo de ADN sin bases repetidas	230
164. Capacidad mínima del camión para cumplir el plazo	231
165. Lluvia atrapada entre silos	232
166. La versión que rompió todo	233
167. La ventana de mayor pulso promedio	234
168. Atrás y adelante en el navegador	235
169. Turnos de la impresora de la facultad	237
170. Tramos de gastos que suman exacto	238
171. Canciones que completan justo una hora	239
172. La papa caliente	240
173. La calculadora de la base antártica	241
174. Stock mínimo en cada ventana de días	243
175. Parejas de tenis con nivel parecido	244
176. Logs entre dos horarios	245
177. La racha más larga de días en verde	246
178. El ascensor del edificio de oficinas	247
179. Vistas únicas en la última hora	248
180. Hornos compartidos en la pastelería	249
181. La tasa que hace cerrar la cuota	251
182. Ventas por región y rango de meses	252
183. Ráfagas de intentos de login	253
184. El rectángulo más grande de la tribuna	254
185. Cola de pedidos armada con dos pilas	255
186. Pulsos del marcapasos	257
187. Llamadas simultáneas en la central	259
188. Dosis máxima en cualquier ventana de 24 horas	260
189. El dólar más caro de cada semana	262
190. El mazo de la escoba	263
191. Tres productos para el presupuesto exacto	265
192. Radar de velocidad en la ruta	266
193. ¿Está la melodía escondida en la canción?	267
194. Baterías de la red domiciliaria	268
195. Buscar un precio en la grilla ordenada	270
196. El catálogo ordenado que alguien rotó	271
197. Calendario de guardias rotativas	272
198. Desafío de nivel: La caja de ahorro con débitos automáticos	273
199. Desafío de nivel: Monitor de tráfico de la autopista	275
200. Desafío de nivel: La cocina del restaurante en hora pico	276

Nivel 5: Semi-Senior II · Estrategia 278

201. Vuelto con la menor cantidad de piezas	279
---	-----

202. El sudoku del diario del domingo	280
203. Estados de un pedido online	282
204. La calculadora del presupuesto de obra	284
205. Máximo de charlas en el auditorio	286
206. Archivos duplicados en el servidor	288
207. Ocho damas en el tablero del club	290
208. El semáforo inteligente	291
209. Descuentos que se acumulan (o no)	293
210. Todas las formas de pagar justo	295
211. El barco con carga fraccionable	296
212. La marca de la tarjeta según su número	297
213. El fixture con sedes y descansos	299
214. La máquina expendedora	300
215. Contraseñas débiles por diccionario	302
216. Turnos de las bombas de riego	304
217. Casilleros del gimnasio	305
218. Mesas de examen sin choques	307
219. Estados de un turno médico	308
220. Tarifa de taxi con recargo nocturno	310
221. Fotos casi iguales	311
222. Cartones de bingo que suman justo	313
223. Liquidación de sueldos según convenio	314
224. Comandos del chatbot de atención	316
225. Cargar primero lo que más rinde	317
226. Reglas del firewall	318
227. Dos equipos parejos	320
228. El crucigrama de la revista escolar	321
229. Impuesto progresivo por tramos	322
230. El mismo tema cargado dos veces	324
231. El cajero automático como máquina de estados	325
232. Menú de pasos sin ingredientes repetidos	327
233. Paradas de carga del auto eléctrico	328
234. Aprobación de créditos por puntaje	329
235. Validar fórmulas químicas	331
236. Tareas del sprint con fecha límite	332
237. El saludo de tres pasos entre módems	333
238. Aulas mínimas para la cursada	334
239. Política de cancelación de reservas	335
240. El ahorcado con estados	336
241. Cupones incompatibles	337
242. Filtros con comodines	338
243. Rotación de cultivos con restricciones	340
244. Saltos entre antenas repetidoras	341

245. Fórmulas de precios con variables	342
246. El city tour con presupuesto	343
247. Transacciones duplicadas por reintento	344
248. Desafío de nivel: Motor de promociones del supermercado	346
249. Desafío de nivel: El intérprete de la hoja de cálculo	348
250. Desafío de nivel: Agenda del congreso médico	350

Nivel 6: Semi-Senior III · Estructuras clásicas 352

251. Niveles del organigrama	353
252. Las 5 transacciones más grandes del día	355
253. El camino más corto en el depósito	357
254. Islas de cultivo en la foto satelital	359
255. Materias correlativas	361
256. Amigos de amigos	363
257. La mochila del mochilero	365
258. El mismo cliente escrito distinto	367
259. Árbol de categorías de la tienda	369
260. Triage en la guardia	371
261. Barrios conectados a la red eléctrica	373
262. Reparto con peajes y horarios de corte	374
263. Qué cambió en el reglamento	376
264. Precios entre dos valores al instante	378
265. De cuántas formas se junta un monto	379
266. Orden de carga de los niveles del juego	380
267. El antepasado común de dos especies	382
268. Mediana de tiempos de respuesta en vivo	384
269. Fibra óptica mínima entre edificios	385
270. La racha creciente de precios más larga	387
271. Cuánto tarda en viajar un rumor	388
272. Cadena de contagio en el hospital	389
273. Dos cosechas del mismo peso	390
274. Combinar vuelos de varias aerolíneas	391
275. Cortes de tela en la fábrica	392
276. El árbol de decisión del crédito	393
277. Pagos circulares entre cuentas	395
278. Palabras escondidas en el tablero	397
279. Escalera de palabras	398
280. El planificador del router	399
281. Cuentas del mismo titular	401
282. Tendido eléctrico sobre terreno irregular	402
283. La sala más profunda del museo	404
284. ¿Se pueden armar dos bandos rivales?	405
285. Guardar y recuperar el menú anidado	407

286. Monedas con denominaciones raras	408
287. Trenes con horarios fijos	410
288. Reposición con prioridades que cambian	411
289. Mutaciones con costos distintos	413
290. Estaciones críticas de la red de subte	414
291. Campañas publicitarias con cupos	415
292. Árbol de géneros musicales	416
293. El proveedor más confiable	418
294. El camino crítico del proyecto	419
295. La frase palíndroma más larga de la melodía	421
296. Camas disponibles en la clínica	422
297. Conversiones encadenadas de monedas	424
298. Desafío de nivel: Centro de distribución con prioridades y rutas	425
299. Desafío de nivel: El planificador de la cursada	427
300. Desafío de nivel: Presupuesto de inversión con dependencias	429

Nivel 7: Senior I - Algoritmia avanzada 431

301. Autocompletado del buscador de la tienda	432
302. Permisos que caben en un número	433
303. Primos para rifas gigantes	435
304. Colectivos que coinciden en la parada	436
305. Formas de armar el podio	437
306. Interés compuesto a plazos astronómicos	438
307. Orden óptimo para fusionar archivos de logs	440
308. Visitar 15 sucursales en un solo día	441
309. Guardias en el árbol de la empresa	443
310. Mínimo de temperatura por rango	444
311. Palabras prohibidas en el chat	445
312. Saldos por rango de fechas en tiempo real	447
313. Cortar la barra de chocolate en la fábrica	448
314. XOR de canales en un rango	449
315. Repartir premios módulo un primo	450
316. Cada empleado con su tarea ideal	452
317. Inversiones en el ranking	453
318. Divisores del número de lote	454
319. Caminos en la grilla con obstáculos	455
320. Ajustes de stock por rango	456
321. La transacción huérfana	457
322. Formas de agrupar una expresión	458
323. Máscaras de subred de los dispositivos	459
324. Números de serie primos entre dos billones	460
325. Distancias en el árbol de la red eléctrica	461
326. Prefijos telefónicos más comunes	463

327. Subconjuntos de cartas que suman el objetivo	465
328. Contraseñas de largo N sin patrones prohibidos	466
329. Parcelas que no pueden sembrarse juntas	468
330. Clientes distintos en un rango de días	469
331. Qué día cae el vencimiento dentro de 10^{18} días	470
332. La canción con más energía tras los cambios	471
333. La k-ésima transacción más chica	473
334. Mezclar pociones sin que exploten	475
335. Boletos capicúa en un rango	476
336. Relojes que vuelven a coincidir	477
337. Productos de una categoría con altas y bajas	478
338. Caminos posibles en la red de rutas	480
339. El par de claves con XOR máximo	482
340. Tabla de posiciones con actualizaciones en vivo	483
341. Repartir caramelos entre chicos	485
342. MCD de rangos de lotes	486
343. Fusionar lotes de pedidos al menor costo	487
344. Combinaciones de vacunas compatibles	489
345. Ocupación de camas por rango de días	490
346. Cuotas coprimas con el período	492
347. El juego de las fichas del recreo	493
348. Desafío de nivel: Libro mayor con correcciones retroactivas	494
349. Desafío de nivel: El teclado predictivo	496
350. Desafío de nivel: Plan de inversión en la red de riego	498

Nivel 8: Senior II · Algoritmos de sistemas 500

351. El catálogo que no entra en memoria	501
352. Mil pedidos por segundo al procesador de pagos	503
353. Probabilidad de lluvia el fin de semana	505
354. Reintentos con espera exponencial	507
355. Pagos que no se cobran dos veces	509
356. Los temas que siempre vuelven	511
357. Jugadores que no cambian de servidor	513
358. Reconstruir el carrito desde sus eventos	515
359. Reglas antifraude de tarjetas	517
360. Entradas vendidas de más	519
361. Deadlock en la reserva de camas	521
362. El bot que prueba contraseñas de a poco	523
363. Motor de matching de órdenes de compra y venta	525
364. Servidores nuevos sin mudanza masiva	527
365. Mensajes entregados al menos una vez	529
366. ¿Cuándo chocan dos códigos cortos?	530
367. El saldo de diez años en un segundo	532

368. El proveedor de envíos que se cae	534
369. Caché con vencimiento para el pronóstico	536
370. Velocidad de transacciones sospechosa	538
371. Lectores y escritores en la historia clínica	540
372. Vuelo, hotel y auto: todo o nada	542
373. Cajas de botín y valor esperado	544
374. Órdenes stop y cancelaciones	546
375. Repartir usuarios entre particiones	548
376. La nómina que se procesa de noche	550
377. Si hoy llueve, ¿mañana también?	552
378. Stock reservado por 10 minutos	554
379. Doble gasto en la billetera virtual	556
380. Límites por usuario y por país	558
381. Cinco cocineros y cinco sartenes	560
382. El feed armado desde eventos	561
383. La racha que funde la caja chica	563
384. Escritura directa o diferida en el caché de stock	564
385. Dos recepcionistas, un mismo turno	566
386. Viajes compartidos que se emparejan solos	568
387. ¿El botón nuevo vende más?	570
388. Webhooks de pago que llegan desordenados	571
389. El caché de dos niveles del CDN	573
390. Motor de reglas para becas	575
391. Canciones al azar de un catálogo infinito	577
392. Órdenes iceberg en el libro	578
393. Anomalías en la telemetría del satélite	580
394. Registro de accesos que no se puede editar	581
395. La espera en los boxes de atención	583
396. Riego limitado por sensores	585
397. Servidores de partidas que se caen	587
398. Desafío de nivel: Plataforma de pagos con reintentos y antifraude	589
399. Desafío de nivel: La ticketera del recital masivo	591
400. Desafío de nivel: Un exchange de criptomonedas en miniatura	593

Nivel 9: Senior III · Matemática aplicada 595

401. Una cláusula en miles de contratos	596
402. La tasa que esconde el crédito	598
403. Cuántos camiones soporta la red de rutas	600
404. Médicos para cada guardia	602
405. El alambrado más corto del campo	604
406. Arbitraje de divisas	606
407. Distancias entre todas las ciudades del tour	608
408. Plagio en los trabajos prácticos	610

409. La TIR de un préstamo con flujos irregulares	612
410. La superficie de la laguna	614
411. Patrones repetidos en el genoma	616
412. Cortar la red con el menor daño	617
413. El rebaño en el año 10^{18}	619
414. Balancear reacciones químicas	620
415. ¿El repartidor está dentro de la zona?	621
416. Lo que tienen en común dos logs gigantes	623
417. Habitaciones según preferencias	624
418. Corrientes del circuito eléctrico	626
419. Rutas aéreas que se cruzan	628
420. El precio de un bono según su rendimiento	630
421. Evacuación del estadio	632
422. Publicaciones casi idénticas	634
423. Descuentos que se pueden explotar en fletes	636
424. Las dos antenas más cercanas	638
425. Subdominios distintos	640
426. Mora que se propaga mes a mes	641
427. Proyectos a equipos con capacidades	643
428. El punto de equilibrio del bar	645
429. La melodía en otra tonalidad	647
430. Superficie vigilada por cámaras	648
431. Tasas forward a partir de la curva	649
432. ¿Se puede llegar en tren?	651
433. Tutores y alumnos	653
434. El candado de combinación circular	655
435. Trayectoria del proyectil del juego	657
436. Parcelas superpuestas	659
437. Precios de la canasta a partir de tickets	660
438. El palíndromo más largo de los posteos	661
439. Bicicletas públicas con demandas por estación	663
440. Visibilidad desde la torre forestal	665
441. Sensibilidad de la cartera de bonos	667
442. Un logo escondido en la imagen	669
443. El mejor lugar para el hospital	671
444. Vuelos con límite de escalas	673
445. Proyectos rentables que dependen de otros	675
446. Pacientes y donantes compatibles	677
447. Pronóstico de demanda eléctrica	679
448. Desafío de nivel: La mesa de dinero	681
449. Desafío de nivel: Plataforma de logística urbana	683
450. Desafío de nivel: Detector de fraude documental	685

Nivel 10: Staff / Experto	687
451. Mil millones de sitios peligrosos en poca memoria	688
452. ¿Cuánta gente distinta entró hoy a la tienda?	690
453. Las canciones más escuchadas con memoria mínima	692
454. Comprimir los mensajes de la flota	694
455. Recibos que nadie puede falsificar	696
456. ¿Quién escribió primero en el grupo?	698
457. Votos del jugador del partido en varios servidores	700
458. Probar que una transacción está en el lote	702
459. Historias clínicas en tres centros de datos	704
460. Comprimir los logs de los sensores	706
461. Todas las cosechas posibles de la cooperativa	708
462. Alimento balanceado al menor costo	710
463. Cuánto podemos perder en un mal día	712
464. El recorrido de 300 entregas	714
465. El precio justo de una opción	716
466. La cartera óptima con presupuesto entero	718
467. Elección de líder entre servidores	720
468. El libro de actas inalterable	722
469. Rentabilidad sin perder el sueño	724
470. La receta colaborativa sin conflictos	726
471. Reseñas casi duplicadas a escala	728
472. La vibración del motor del colectivo	730
473. El cronograma de enfermería que conforma a todos	731
474. Inventario sincronizado entre depósitos	733
475. Dos réplicas, ¿en qué difieren?	735
476. Escaneos de puertos con memoria mínima	737
477. Despacho eléctrico al menor costo	739
478. Cien dados sobre la mesa	741
479. Comprimir partituras	742
480. Prueba de trabajo de juguete	743
481. Turistas únicos por región	745
482. Choferes para cada servicio de larga distancia	746
483. La opción que se puede ejercer antes	748
484. Tabla de posiciones replicada con particiones	749
485. Productos dados de baja en memoria mínima	751
486. Sensores que mienten	753
487. Intercambio de claves de juguete	754
488. Riesgo de crédito con incumplimientos correlacionados	755
489. Recorrido de las cosechadoras	757
490. Contratos de energía entre vecinos productores	758
491. Índice comprimido del buscador de recetas	760
492. Turnos de fábrica que respetan a todos	762

493. El feed que llega desordenado 764

494. Telemetría de un país entero en poca memoria 766

495. Rendición de cuentas de un fondo verificable 768

496. Radiografías más nítidas 770

497. Qué avión vuela cada ruta 771

498. Desafío de nivel: Ledger distribuido que sobrevive a cortes de red 773

499. Desafío de nivel: Antifraude a escala planetaria 775

500. Desafío de nivel: La mesa de riesgo del banco 777

Apéndice A: ejercicios y conceptos 779

Apéndice B: glosario 789

Nivel 1

Junior I · Fundamentos

Ejercicios 1 a 50

Qué vas a practicar

Variables, condicionales, bucles y operaciones básicas con números y texto.

Qué se espera al terminar

Cálculos correctos, redondeos bien aplicados y salidas con el formato exacto.

Conceptos del nivel

variables, aritmética básica, porcentajes, aritmética, conversión de unidades, fórmulas, números decimales, condicionales simples, condicionales, promedio simple, redondeo, condicionales encadenados, bucles simples, contadores, condicionales por rangos, comparación de montos, división entera y resto, acumuladores, promedio, condicionales por tramos, conversión de tiempo, comisiones, bucles, bucle con corte, bucle mientras, interés diario, división, proporciones, bucles anidados simples, condicionales compuestos, operador resto, redondeo hacia arriba, división entera, resto, contadores con reinicio, comparación de alternativas, máximo y mínimo con bucle, conversión de segundos a minutos, divisibilidad, acumulación de descuentos....

Desafíos de nivel

- 48. La caja registradora del almacén
- 49. El cajero automático que no da monedas
- 50. La liga de fútbol 5 de los jueves

Ejercicio 1: La propina de la mesa 4

Nivel 1: Junior I · Fundamentos · Temática: Gastronomía · Dificultad 1/100

Situación: En el bodegón de la esquina, la mesa 4 pide la cuenta. Son tres amigas, quieren dejar un 10 % de propina y pagar en partes iguales. La moza está a full y te pide un programa que haga la cuenta sin errores.

Datos de entrada

Dato	Valor
Consumo de la mesa	\$18.750
Propina	10 %
Comensales	3

Tu programa debe

1. Calcular el monto de la propina.
2. Calcular el total a pagar (consumo + propina).
3. Calcular cuánto paga cada comensal.

Salida esperada

```
Consumo: $18750
Propina (10%): $1875
Total: $20625
Cada uno paga: $6875
```

Reglas

- Alcanza con variables y aritmética.
- Los datos pueden estar fijos en el código.
- Si el reparto no da exacto, cada uno paga redondeando hacia arriba al peso siguiente (mejor que sobre a que falte).
- El programa tiene que funcionar si cambiás el consumo, el porcentaje o la cantidad de comensales.

Pista (leela solo si te trabás): calculá primero el total y recién después dividí. Para redondear hacia arriba sin funciones especiales, pensá qué pasa si al dividendo le sumás (divisor – 1) antes de hacer la división entera.

Bonus (opcional)

- Mostrá también cuánto sobra en total por el redondeo hacia arriba.
- Permití que cada comensal elija su propio porcentaje de propina.

Cierre: Es la misma cuenta que hace cualquier sistema de punto de venta de un restaurante cuando alguien pide "dividir la cuenta".

Ejercicio 2: ¿Cuánto tarda en llegar el colectivo?

Nivel 1: Junior I · Fundamentos · Temática: Transporte · Dificultad 1/100

Situación: Martina toma el colectivo todas las mañanas para ir a la facultad y siempre llega tarde a la primera clase. Quiere saber a qué hora llega realmente si sale a las 8:15. El colectivo recorre 7,2 km a una velocidad promedio de 24 km/h y se detiene en 12 paradas, 30 segundos en cada una.

Datos de entrada

Dato	Valor
Distancia	7,2 km
Velocidad promedio	24 km/h
Paradas	12
Tiempo por parada	30 segundos
Hora de salida	08:15

Tu programa debe

1. Calcular los minutos que el colectivo está en movimiento.
2. Calcular los minutos que pierde en las paradas.
3. Sumar ambos tiempos y mostrar el total.
4. Mostrar la hora de llegada en formato HH:MM (con dos dígitos).

Salida esperada

```
Tiempo en movimiento: 18 minutos  
Tiempo en paradas: 6 minutos  
Tiempo total: 24 minutos  
Llega a las 08:39
```

Reglas

- Alcanza con aritmética, división entera y resto.
- Los tiempos de este caso dan minutos exactos; si no fuera así, redondea hacia abajo.
- Si la llegada pasa de las 23:59, la hora tiene que volver a empezar desde 00 (usá el resto de la división por 24).
- La hora se muestra siempre con dos dígitos: 08:39, no 8:39.

Pista (leela solo si te trabás): convertí todo a la misma unidad (minutos) antes de sumar. Para la hora de llegada, pasá la hora de salida a "minutos desde la medianoche", sumá y después volvé a separar horas y minutos con división entera y resto.

Bonus (opcional)

- Si la primera clase empieza a las 8:45, decile a Martina cuántos minutos antes o después llega.
- Calculá a qué hora tendría que salir para llegar exactamente 5 minutos antes de la clase.

Cierre: Es la misma lógica básica que usan las apps de transporte para estimar el horario de llegada, antes de sumarle datos del tránsito en vivo.

Ejercicio 3: Termómetro de dos escalas

Nivel 1: Junior I · Fundamentos · Temática: Ciencia · Dificultad 1/100

Situación: Un grupo de estudiantes de un secundario técnico comparte datos con una escuela de Estados Unidos y con un laboratorio que trabaja en kelvin. Cada vez que mandan una lectura tienen que convertirla a mano y ya se equivocaron dos veces. Necesitan un conversor confiable.

Datos de entrada

Lectura	Lugar	Temperatura
1	Estación de Ushuaia	-3,5 °C
2	Aula del laboratorio	22 °C
3	Incubadora	37,8 °C

Fórmulas: $F = C \times 9 / 5 + 32$ · $K = C + 273,15$

Tu programa debe

1. Para cada lectura, convertir de Celsius a Fahrenheit.
2. Convertir de Celsius a Kelvin.
3. Mostrar las tres escalas en una línea y marcar "(bajo cero)" si la temperatura en Celsius es negativa.

Salida esperada

```
-3.5 °C = 25.7 °F = 269.65 K (bajo cero)
22.0 °C = 71.6 °F = 295.15 K
37.8 °C = 100.0 °F = 310.95 K
```

Reglas

- Alcanza con variables decimales, fórmulas y un condicional.
- Celsius y Fahrenheit se muestran con 1 decimal; Kelvin con 2 decimales. Redondeo hacia arriba desde 0,5.
- Los decimales se muestran con punto y sin separador de miles en la salida.
- Cuidado con el orden de las operaciones: $9 / 5$ no es lo mismo que $9 / (5 + 32)$.
- 0 °C no es bajo cero.

Pista (leela solo si te trabás): escribí la fórmula una sola vez dentro de un bucle que recorra las tres lecturas. Si tu resultado para 37,8 °C da 100.04, recordá que con un decimal se muestra como 100.0.

Bonus (opcional)

- Agregá la conversión inversa: de Fahrenheit a Celsius.
- Detectá lecturas imposibles: nada puede estar por debajo de 0 K (−273,15 °C).

Cierre: Es la misma conversión que hacen las estaciones meteorológicas y los equipos de laboratorio cuando exportan datos para otros países.

Ejercicio 4: El kiosco que cobra con descuento

Nivel 1: Junior I · Fundamentos · Temática: E-commerce · Dificultad 2/100

Situación: Don Rubén tiene un kiosco frente a una escuela y lanzó dos promociones: 10 % de descuento en compras desde \$5.000 y un 5 % extra si pagan en efectivo. Los chicos le discuten cada ticket y él termina haciendo cuentas en un papel. Tu programa tiene que calcular el precio final sin discusiones.

Datos de entrada

Cliente	Subtotal	Medio de pago
Cliente 1	\$4.200	Efectivo
Cliente 2	\$6.350	Tarjeta

Tu programa debe

1. Aplicar un 10 % de descuento si el subtotal es de \$5.000 o más.
2. Aplicar un 5 % adicional si paga en efectivo, calculado sobre el monto que queda después del primer descuento.
3. Mostrar cada descuento por separado (aunque sea \$0) y el monto final a pagar.

Salida esperada

```
Cliente 1
Subtotal: $4200
Descuento por monto: $0
Descuento por efectivo: $210
A pagar: $3990
Cliente 2
Subtotal: $6350
Descuento por monto: $635
Descuento por efectivo: $0
A pagar: $5715
```

Reglas

- Alcanza con condicionales simples y porcentajes.
- Los descuentos se redondean al peso más cercano (0,5 hacia arriba).
- Exactamente \$5.000 sí tiene descuento por monto.
- Los dos descuentos son independientes: un cliente puede tener ninguno, uno o los dos.

Pista (leela solo si te trabás): usá una variable para el "monto actual" y andá descontándole en orden. Que el segundo descuento se calcule sobre el monto ya rebajado es lo que suele salir mal.

Bonus (opcional)

- Agregá un tope: el descuento total nunca puede superar los \$1.500.
- Calculá cuánto dinero dejó de cobrar Don Rubén en el día sumando los descuentos de todos los clientes.

Cierre: Es la base de cualquier motor de promociones de una caja registradora: reglas que se aplican en un orden definido.

Ejercicio 5: ¿Aprobé o voy a recuperatorio?

Nivel 1: Junior I · Fundamentos · Temática: Educación · Dificultad 2/100

Situación: En un instituto terciario, la preceptoría arma a mano la lista de quién promociona, quién va a final y quién va a recuperatorio. Cada cuatrimestre hay reclamos por errores de cálculo. Te piden un programa que aplique el reglamento al pie de la letra.

Datos de entrada

Estudiante	Parcial 1	Parcial 2	Parcial 3
Lucía	8	6	9
Tomás	7	3	10
Sofía	5	6	7

Reglamento:

- Si alguna nota es menor que 4 → Recuperatorio.
- Si no, y el promedio es 7 o más → Promociona.
- En cualquier otro caso → Va a final.

Tu programa debe

1. Calcular el promedio de los tres parciales de cada estudiante.
2. Decidir su situación aplicando el reglamento en el orden indicado.
3. Mostrar nombre, promedio y situación; si va a recuperatorio, aclarar el motivo.

Salida esperada

```
Lucía: promedio 7.67 -> Promociona
Tomás: promedio 6.67 -> Recuperatorio (tiene una nota menor a 4)
Sofía: promedio 6.00 -> Va a final
```

Reglas

- Alcanza con condicionales encadenados.
- El promedio se muestra con 2 decimales, redondeo hacia arriba desde 0,5. Los decimales se muestran con punto y sin separador de miles en la salida.
- La decisión se toma con el promedio sin redondear.
- Una nota exactamente igual a 4 no manda a recuperatorio; un promedio exactamente 7 promociona.

Pista (leela solo si te trabás): el orden de los condicionales importa: Tomás tiene promedio 6.67, pero lo primero que hay que revisar es si tiene alguna nota menor que 4.

Bonus (opcional)

- Agregá la regla: "si tiene un 10, promociona aunque el promedio sea 6,5 o más" y fijate dónde conviene ubicarla.
- Contá cuántos estudiantes quedaron en cada situación.

Cierre: Es el mismo tipo de reglas que aplican los sistemas de gestión académica de universidades e institutos.

Ejercicio 6: Plazo fijo a 30 días

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 2/100

Situación: Gabriela cobró un bono y lo quiere poner en un plazo fijo a 30 días. El banco le informa una tasa nominal anual (TNA) del 36 %, pero ella no sabe cuánta plata va a tener al final ni si le conviene contra la inflación esperada del mes. Tu programa se lo explica en números.

Datos de entrada

Dato	Valor
Capital	\$500.000
TNA	36 %
Plazo	30 días
Inflación esperada del mes	2,5 %

Fórmula: $\text{interés} = \text{capital} \times \text{TNA} \times \text{días} / 365$

Tu programa debe

1. Calcular el interés ganado en el plazo.
2. Calcular el total que cobra al vencimiento.
3. Calcular el rendimiento del período como porcentaje del capital.
4. Indicar si el rendimiento supera la inflación esperada del mes.

Salida esperada

```
Interés ganado: $14794.52
Total al vencimiento: $514794.52
Rendimiento del período: 2.96%
Le ganó a la inflación del mes (2.5%): sí
```

Reglas

- Alcanza con aritmética con decimales y un condicional.
- El interés se redondea a 2 decimales (0,5 hacia arriba) y ese valor redondeado es el que se usa en el total y en el rendimiento.
- El rendimiento se muestra con 2 decimales. Los decimales se muestran con punto y sin separador de miles en la salida.
- Usá el año de 365 días, como indica la fórmula.

Pista (leela solo si te trabás): la TNA viene como porcentaje: 36 % equivale a 0,36. Si te da un interés enorme, seguramente multiplicaste por 36 en lugar de por 0,36.

Bonus (opcional)

- Calculá qué pasa si Gabriela renueva el plazo fijo 12 veces seguidas reinvertiendo capital + interés.
- Calculá qué TNA mínima necesitaría para empatarle a la inflación del mes.

Cierre: Es exactamente la cuenta que muestran los simuladores de plazo fijo de los bancos.

Ejercicio 7: El semáforo del IMC

Nivel 1: Junior I · Fundamentos · Temática: Salud · Dificultad 2/100

Situación: En un centro de atención primaria, la enfermera de triage calcula el índice de masa corporal (IMC) de cada paciente con la calculadora del celular. Quieren un programa que lo calcule y lo muestre como un semáforo, para que el médico vea de un vistazo a quién prestarle más atención. Es solo un indicador orientativo.

Datos de entrada

Paciente	Peso (kg)	Altura (m)
Marta	58	1,65
Jorge	92	1,78
Nicolás	50	1,72

$IMC = \text{peso} / (\text{altura} \times \text{altura})$

IMC	Categoría	Semáforo
menor que 18,5	Bajo peso	AMARILLO
de 18,5 a menos de 25	Normal	VERDE
de 25 a menos de 30	Sobrepeso	AMARILLO
30 o más	Obesidad	ROJO

Tu programa debe

1. Calcular el IMC de cada paciente.
2. Clasificarlo según la tabla.
3. Mostrar nombre, IMC, categoría y color del semáforo.

Salida esperada

```
Marta: IMC 21.30 -> Normal [VERDE]
Jorge: IMC 29.04 -> Sobrepeso [AMARILLO]
Nicolás: IMC 16.90 -> Bajo peso [AMARILLO]
```

Reglas

- Alcanza con fórmulas y condicionales encadenados.
- El IMC se muestra con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- La clasificación se hace con el valor sin redondear.
- Los límites importan: un IMC de 25,00 exacto es Sobrepeso, no Normal.

Pista (leela solo si te trabás): ordená los condicionales de menor a mayor; así cada condición solo necesita comparar contra un límite, porque los casos anteriores ya quedaron descartados.

Bonus (opcional)

- Validá los datos: rechazá alturas menores a 0,5 m o mayores a 2,5 m con un mensaje claro.
- Mostrá cuántos kilos le faltan o le sobran a cada paciente para entrar en la franja Normal.

Cierre: Es el mismo cálculo que hacen las historias clínicas electrónicas y muchas balanzas inteligentes.

Ejercicio 8: Horas extra del turno noche

Nivel 1: Junior I · Fundamentos · Temática: Recursos humanos · Dificultad 2/100

Situación: Una planta de envasado trabaja las 24 horas y el área de Recursos Humanos liquida los jornales a mano con una planilla que nadie entiende. El convenio dice que las horas después de la octava se pagan 50 % más y que el turno noche tiene un recargo del 20 % sobre todo lo cobrado ese día. Tu programa tiene que liquidar cada jornada.

Datos de entrada

Empleado	Horas trabajadas	Turno
Rocío	10	noche
Damián	7	día

- Valor de la hora normal: \$4.800
- Jornada normal: 8 horas
- Hora extra: valor normal $\times$ 1,5
- Recargo nocturno: 20 % sobre (horas normales + horas extra)

Tu programa debe

1. Separar las horas normales (hasta 8) de las horas extra.
2. Calcular cuánto se paga por cada grupo de horas.
3. Si el turno es noche, calcular el recargo del 20 % sobre la suma anterior.
4. Mostrar el detalle y el total del día de cada empleado.

Salida esperada

```
Rocío (10 h, turno noche)
  Horas normales: 8 -> $38400
  Horas extra: 2 -> $14400
  Recargo nocturno: $10560
  Total del día: $63360
Damián (7 h, turno día)
  Horas normales: 7 -> $33600
  Horas extra: 0 -> $0
  Recargo nocturno: $0
  Total del día: $33600
```

Reglas

- Alcanza con condicionales y aritmética.
- Todos los montos de este caso dan enteros; si no, redondea al peso más cercano.
- Si alguien trabaja 8 horas o menos, las horas extra son 0 (nunca negativas).
- El recargo nocturno es \$0 en turno día, pero igual se muestra.

Pista (leela solo si te trabás): calculá las horas extra como "horas trabajadas menos 8, pero nunca menos de cero". Con un condicional alcanza.

Bonus (opcional)

- Agregá una regla: a partir de la hora 12, las horas extra se pagan al doble.
- Liquidá la semana completa de un empleado leyendo 7 jornadas seguidas.

Cierre: Es la lógica central de cualquier sistema de liquidación de sueldos y jornales.

Ejercicio 9: Cuenta regresiva para el lanzamiento del globo sonda

Nivel 1: Junior I · Fundamentos · Temática: Ciencia · Dificultad 2/100

Situación: Un club de ciencia de una escuela rural va a lanzar un globo sonda para medir temperatura en altura. La maestra quiere proyectar en la pantalla una cuenta regresiva con los avisos de cada paso y, después del despegue, la altura estimada minuto a minuto hasta superar los 1.400 m.

Datos de entrada

Dato	Valor
Cuenta regresiva desde	10
Aviso en T-5	Encender la radiosonda
Aviso en T-2	Soltar amarras
Velocidad de ascenso	5 metros por segundo
Altura objetivo	superar 1.400 m

Tu programa debe

1. Mostrar la cuenta regresiva de 10 a 1, agregando el aviso en los segundos que corresponde.
2. Mostrar "¡Despegue!" al llegar a cero.
3. Mostrar la altura al final de cada minuto mientras no se haya superado el objetivo.
4. Informar en qué minuto se superó la altura objetivo.

Salida esperada

```
T-10
T-9
T-8
T-7
T-6
T-5 Encender la radiosonda
T-4
T-3
T-2 Soltar amarras
T-1
¡Despegue!
Minuto 1: 300 m
Minuto 2: 600 m
Minuto 3: 900 m
Minuto 4: 1200 m
Minuto 5: 1500 m
Superó los 1400 m en el minuto 5
```

Reglas

- Alcanza con un bucle descendente, un bucle con condición y un contador.
- "Superar" significa estrictamente mayor: si la altura fuera exactamente 1.400 m, todavía no la superó.
- El programa tiene que funcionar si cambiás el número de inicio o la velocidad de ascenso.

Pista (leela solo si te trabás): la primera parte es un bucle que cuenta hacia atrás; la segunda es un bucle que se repite "mientras" no llegues al objetivo. Pensá bien cuál de los dos conviene para cada parte.

Bonus (opcional)

- Mostrá también la temperatura estimada: baja 6,5 °C cada 1.000 m, partiendo de 18 °C en el suelo.
- Agregá un aviso cada vez que el globo supera un múltiplo de 500 m.

Cierre: Es el mismo esquema de "secuencia con eventos" que usan los scripts de lanzamiento de cohetes y los paneles de control de misiones.

Ejercicio 10: El precio del envío según el peso

Nivel 1: Junior I · Fundamentos · Temática: Logística y envíos · Dificultad 3/100

Situación: Una pyme que vende yerba y mate por internet despacha decenas de paquetes por día y el correo le cobra según el peso. La encargada de depósito quiere saber cuánto va a pagar antes de llevar el lote a la sucursal, para no llevarse sorpresas.

Datos de entrada

Peso del paquete	Precio
hasta 1 kg	\$2.500
más de 1 kg y hasta 5 kg	\$4.200
más de 5 kg y hasta 20 kg	\$7.800
más de 20 kg	\$7.800 + \$450 por cada kg o fracción que pase de 20

Paquete	Peso
A-101	0,8 kg
A-102	1,0 kg
A-103	5,3 kg
A-104	23,4 kg

Tu programa debe

1. Calcular el precio de envío de cada paquete según su peso.
2. Para los paquetes de más de 20 kg, cobrar \$450 por cada kilo o fracción excedente.
3. Mostrar el precio de cada paquete y el total del lote.

Salida esperada

```
A-101 (0.8 kg): $2500
A-102 (1.0 kg): $2500
A-103 (5.3 kg): $7800
A-104 (23.4 kg): $9600
Total del lote: $22400
```

Reglas

- Alcanza con condicionales por rangos y un acumulador.
- Los límites son inclusivos: un paquete de 1,0 kg paga \$2.500 y uno de 20,0 kg paga \$7.800.
- "Fracción" significa que 3,4 kg de exceso se cobran como 4 kg.
- Los pesos se muestran con 1 decimal. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): para "kilo o fracción" necesitás redondear hacia arriba el excedente. Una forma: tomá la parte entera y, si quedó algo de decimal, sumale uno.

Bonus (opcional)

- Agregá un recargo del 30 % para envíos al interior.
- Si un paquete pesa más de 50 kg, rechazalo con un mensaje: el correo no lo acepta.

Cierre: Es la misma tabla de tarifas escalonadas que aplican los correos y las empresas de mensajería.

Ejercicio 11: ¿Me alcanza para la cuota?

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 3/100

Situación: Una fintech que da préstamos personales quiere una primera regla automática antes de pasar cada pedido a un analista. La política dice que la cuota no puede superar el 30 % del ingreso y que, después de pagar gastos fijos y cuota, a la persona le tienen que quedar al menos \$100.000. Tu programa hace ese primer filtro.

Datos de entrada

Solicitante	Ingreso mensual	Gastos fijos	Cuota pedida
Carla	\$850.000	\$420.000	\$240.000
Pablo	\$620.000	\$350.000	\$190.000

Tu programa debe

1. Calcular qué porcentaje del ingreso representa la cuota.
2. Calcular cuánto le queda después de gastos fijos y cuota.
3. Aprobar solo si se cumplen las dos condiciones; si no, rechazar indicando todos los motivos que fallan.

Salida esperada

Carla: cuota 28.24% del ingreso, le quedan \$190000 -> Aprobado
Pablo: cuota 30.65% del ingreso, le quedan \$80000 -> Rechazado (supera el 30%; le quedan menos de \$100000)

Reglas

- Alcanza con condicionales y comparaciones.
- El porcentaje se muestra con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Exactamente 30 % o exactamente \$100.000 cumplen la condición.
- Si fallan las dos condiciones, se muestran las dos, en el orden del enunciado y separadas por "; ".

Pista (leela solo si te trabás): evalúa cada condición por separado y guardá el resultado en su propia variable. Recién al final decidí el veredicto y armá el texto de los motivos.

Bonus (opcional)

- Calculá la cuota máxima que se le podría aprobar a Pablo cumpliendo ambas reglas.
- Agregá una tercera regla: la antigüedad laboral tiene que ser de al menos 6 meses.

Cierre: Es la primera capa de los motores de decisión crediticia que usan bancos y fintechs antes del análisis humano.

Ejercicio 12: Megas que quedan en el plan

Nivel 1: Junior I · Fundamentos · Temática: Telecomunicaciones · Dificultad 3/100

Situación: Lautaro tiene un plan de 15 GB por mes y siempre se queda sin datos antes de fin de mes. La app de la compañía solo le muestra el consumo por aplicación en MB. Tu programa tiene que decirle cuánto le queda y cuánto puede gastar por día para llegar al final del ciclo.

Datos de entrada

Aplicación	Consumo (MB)
WhatsApp	1.350
Videos	7.420
Música	2.210
Redes	750

- Plan: 15 GB (1 GB = 1.024 MB)
- Días que faltan para que se renueve: 9

Tu programa debe

1. Pasar el plan a MB y sumar el consumo de todas las aplicaciones.
2. Calcular el porcentaje consumido.
3. Mostrar lo que queda en MB y también como "X GB y Y MB".
4. Calcular cuántos MB por día puede usar sin pasarse.

Salida esperada

```
Plan: 15360 MB
Consumido: 11730 MB (76.37%)
Quedan: 3630 MB (3 GB y 558 MB)
Podés usar 403 MB por día durante 9 días
```

Reglas

- Alcanza con aritmética, división entera y resto.
- El porcentaje se muestra con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Los MB por día se calculan con división entera (redondeo hacia abajo), para no pasarse.
- Si el consumo supera el plan, mostrará cuántos MB se pasó en lugar de lo que queda.

Pista (leela solo si te trabás): división entera y resto son tus aliados: el cociente de dividir por 1.024 te da los GB completos y el resto, los MB sueltos.

Bonus (opcional)

- Si le quedan menos de 100 MB por día, mostrará un aviso sugiriendo comprar un paquete adicional.
- Calculá qué día del ciclo se va a quedar sin datos si sigue con el ritmo de consumo promedio de los 21 días que ya pasaron.

Cierre: Es la misma cuenta que hacen las apps de las compañías telefónicas cuando te avisan "te quedan X GB".

Ejercicio 13: Goles por partido del torneo barrial

Nivel 1: Junior I · Fundamentos · Temática: Deportes · Dificultad 3/100

Situación: El delegado de "Los Pibes del Oeste", un equipo de un torneo barrial, anota los goles de cada fecha en un cuaderno. Para la reunión de fin de temporada quiere llevar las estadísticas del equipo, pero siempre se equivoca al sumar.

Datos de entrada

Fecha	1	2	3	4	5	6	7	8
Goles	3	1	0	4	2	2	0	5

Tu programa debe

1. Recorrer las 8 fechas sumando los goles.
2. Calcular el promedio de goles por partido.
3. Contar en cuántos partidos no hicieron goles.
4. Encontrar la fecha con más goles.

Salida esperada

```
Goles totales: 17
Promedio por partido: 2.13
Partidos sin goles: 2
Mejor fecha: 8 (5 goles)
```

Reglas

- Alcanza con un bucle, acumuladores y contadores. Podés ingresar los goles uno por uno.
- El promedio se muestra con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Si dos fechas empatan en máximo de goles, se informa la primera.
- El programa tiene que funcionar con cualquier cantidad de fechas.

Pista (leela solo si te trabás): necesitás tres variables que se actualizan en cada vuelta del bucle: la suma, el contador de ceros y el "mejor hasta ahora" (con su fecha).

Bonus (opcional)

- Calculá la racha más larga de partidos consecutivos con al menos un gol.
- Agregá los goles recibidos y mostrá la diferencia de gol.

Cierre: Es la base de las estadísticas que muestran los apps de resultados deportivos.

Ejercicio 14: La factura de luz escalonada

Nivel 1: Junior I · Fundamentos · Temática: Clima y energía · Dificultad 3/100

Situación: La distribuidora eléctrica cobra la energía por tramos: los primeros kWh son baratos y, cuanto más consumís, más caro sale cada kWh adicional. Muchos usuarios no entienden por qué la factura se dispara en verano. Tu programa tiene que desarmar la factura tramo por tramo.

Datos de entrada

Tramo	Precio por kWh
los primeros 150 kWh	\$95
del kWh 151 al 400	\$130
del kWh 401 en adelante	\$185

- Cargo fijo mensual: \$2.300
- IVA: 21 % sobre (energía + cargo fijo)

Usuario	Consumo del mes
Casa de Ana	520 kWh
Depto de Luis	130 kWh

Tu programa debe

1. Repartir el consumo en los tramos que correspondan.
2. Calcular el importe de cada tramo usado y mostrarlo.
3. Sumar el cargo fijo y calcular el IVA.
4. Mostrar el total de la factura.

Salida esperada

```
Casa de Ana (520 kWh)
 150 kWh x $95 = $14250
 250 kWh x $130 = $32500
 120 kWh x $185 = $22200
Cargo fijo: $2300
Subtotal: $71250
IVA 21%: $14962.50
Total: $86212.50
Depto de Luis (130 kWh)
 130 kWh x $95 = $12350
Cargo fijo: $2300
Subtotal: $14650
IVA 21%: $3076.50
Total: $17726.50
```

Reglas

- Alcanza con condicionales por tramos y acumuladores.
- Solo se muestran los tramos que tienen consumo.
- IVA y total se muestran con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Ojo: 520 kWh NO se cobran todos a \$185. Cada tramo cobra solo los kWh que caen dentro de él.

Pista (leela solo si te trabás): pensá en "llenar" cada tramo en orden: el primero se llena con hasta 150 kWh, el segundo con hasta 250 más, y el tercero con lo que sobre.

Bonus (opcional)

- Agregá un subsidio: los usuarios con tarifa social no pagan el cargo fijo y tienen 10 % de descuento en el primer tramo.

- Calculá cuántos kWh podría consumir un usuario con un presupuesto de \$50.000 (IVA incluido).

Cierre: Es el mismo esquema de tarifa escalonada que usan las distribuidoras de energía, gas y agua.

Ejercicio 15: Vidas y escudo del personaje

Nivel 1: Junior I · Fundamentos · Temática: Videojuegos · Dificultad 4/100

Situación: Un estudio indie está haciendo un juego de plataformas y el sistema de daño tiene un bug: a veces el personaje pierde dos vidas con un solo golpe. Te piden reescribir la lógica desde cero y probarla con una secuencia de golpes conocida.

Datos de entrada

Dato	Valor inicial
Vidas	3
Salud	100
Escudo	40

Golpes recibidos, en orden: 35, 50, 80, 20, 90, 60

Tu programa debe

1. Para cada golpe, descontar primero del escudo; lo que el escudo no absorba se descuenta de la salud.
2. Si la salud llega a 0 o menos, el personaje pierde una vida y la salud vuelve a 100 (el daño sobrante se pierde; el escudo no se recarga).
3. Si se queda sin vidas, mostrar GAME OVER y marcar como ignorados los golpes restantes.
4. Mostrar el estado después de cada golpe y el estado final.

Salida esperada

```
Golpe 1 (35): escudo 5, salud 100, vidas 3
Golpe 2 (50): escudo 0, salud 55, vidas 3
Golpe 3 (80): ¡perdió una vida! escudo 0, salud 100, vidas 2
Golpe 4 (20): escudo 0, salud 80, vidas 2
Golpe 5 (90): ¡perdió una vida! escudo 0, salud 100, vidas 1
Golpe 6 (60): escudo 0, salud 40, vidas 1
Estado final: vidas 1, salud 40, escudo 0
```

Reglas

- Alcanza con variables de estado, condicionales y un bucle.
- Un golpe nunca puede sacar más de una vida.
- Con 0 vidas la salud se muestra en 0.
- El texto "¡perdió una vida!" o "GAME OVER" va justo antes de "escudo" solo en el golpe en que ocurre.

Pista (leela solo si te trabás): separa cada golpe en dos momentos: cuánto absorbe el escudo (el mínimo entre el escudo y el golpe) y cuánto pasa a la salud. Recién después revisá si la salud llegó a cero.

Bonus (opcional)

- Agregá pociones: si en la secuencia aparece un valor negativo, significa curación (sin superar 100 de salud).
- Agregá invulnerabilidad: después de perder una vida, el siguiente golpe no hace daño.

Cierre: Es el mismo tipo de lógica de estado que tiene cualquier videojuego con vidas, salud y escudos.

Ejercicio 16: El metrónomo del ensayo

Nivel 1: Junior I · Fundamentos · Temática: Música · Dificultad 4/100

Situación: Una banda de rock prepara su primer show y el baterista quiere cargar el metrónomo digital con el tiempo exacto de cada tema. Además necesitan saber cuánto dura el bloque de tres canciones, porque tienen solo 6 minutos en la grilla del festival.

Datos de entrada

Tema	Tempo (BPM)	Compases	Pulsos por compás
1	120	64	4
2	90	48	3
3	160	80	4

Tu programa debe

1. Calcular cuántos milisegundos dura cada pulso ($60.000 / \text{BPM}$).
2. Calcular la cantidad total de pulsos de cada tema.
3. Calcular la duración de cada tema en minutos y segundos (M:SS).
4. Mostrar la duración total del bloque.

Salida esperada

```
Tema 1: 120 BPM -> 500.00 ms por pulso | 256 pulsos | 2:08
Tema 2: 90 BPM -> 666.67 ms por pulso | 144 pulsos | 1:36
Tema 3: 160 BPM -> 375.00 ms por pulso | 320 pulsos | 2:00
Duración total del ensayo: 5:44
```

Reglas

- Alcanza con aritmética y conversión de unidades.
- Los milisegundos se muestran con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- La duración de cada tema se redondea al segundo más cercano antes de sumarla al total.
- Los segundos se muestran siempre con dos dígitos: 1:05, no 1:5.

Pista (leela solo si te trabás): la duración en segundos es $\text{pulsos} \times 60 / \text{BPM}$. Después, minutos = división entera por 60 y segundos = resto.

Bonus (opcional)

- Indicá si el bloque entra en los 6 minutos y cuánto sobra o falta.
- Calculá a qué BPM tendría que tocar el tema 1 para que el bloque dure exactamente 6:00.

Cierre: Es la misma cuenta que hacen los programas de grabación de audio para ubicar compases en la línea de tiempo.

Ejercicio 17: Cambio de moneda en el aeropuerto

Nivel 1: Junior I · Fundamentos · Temática: Turismo · Dificultad 4/100

Situación: Emma llega de viaje con USD 400 y necesita pesos. En el aeropuerto hay una casa de cambio con cotización más baja, y en el centro un local que paga mejor pero cobra un cargo fijo alto. Quiere saber dónde recibe más pesos, en números y no a ojo.

Datos de entrada

Lugar	Cotización (pesos por dólar)	Comisión	Cargo fijo
Aeropuerto	\$1.180	3 % del bruto	\$2.500
Centro	\$1.210	0 %	\$9.800

Monto a cambiar: USD 400

Tu programa debe

1. Calcular el bruto en pesos de cada lugar (dólares × cotización).
2. Descontar la comisión y el cargo fijo.
3. Mostrar el detalle y el neto de cada lugar.
4. Indicar cuál conviene y cuántos pesos más se reciben.

Salida esperada

Aeropuerto: bruto \$472000 - comisión \$14160 - cargo fijo \$2500 = \$455340
Centro: bruto \$484000 - comisión \$0 - cargo fijo \$9800 = \$474200
Conviene: Centro (recibís \$18860 más)

Reglas

- Alcanza con aritmética, porcentajes y una comparación.
- La comisión se redondea al peso más cercano (0,5 hacia arriba).
- Si los dos lugares dan lo mismo, mostrará "Da lo mismo".
- El programa tiene que servir para cualquier monto en dólares.

Pista (leela solo si te trabás): calculá el neto de los dos lugares con la misma fórmula y recién al final compará.

Bonus (opcional)

- Calculá a partir de qué monto en dólares empieza a convenir el centro (el cargo fijo pesa menos cuanto más cambiás).
- Agregá un tercer lugar con cotización \$1.195, comisión 1 % y sin cargo fijo.

Cierre: Es la misma comparación que hacen los sitios que muestran cotizaciones de distintas casas de cambio.

Ejercicio 18: Seguidores ganados en la semana

Nivel 1: Junior I · Fundamentos · Temática: Redes sociales · Dificultad 4/100

Situación: Juli maneja las redes de una cafetería de especialidad y todos los lunes le presenta un reporte a la dueña. La plataforma le muestra seguidores ganados y perdidos por día, pero no el neto ni cuál fue el mejor día. Tu programa arma el reporte semanal.

Datos de entrada

Día	Ganados	Perdidos
Lunes	120	30
Martes	85	40
Miércoles	240	25
Jueves	60	70
Viernes	310	20
Sábado	190	35
Domingo	45	50

Seguidores al comenzar la semana: 4.870

Tu programa debe

1. Calcular el neto de cada día (ganados – perdidos) y mostrarlo con signo.
2. Calcular el neto semanal y la cantidad de seguidores al final.
3. Encontrar el día con mejor neto.
4. Contar cuántos días tuvieron neto negativo.

Salida esperada

```
Lunes: +90
Martes: +45
Miércoles: +215
Jueves: -10
Viernes: +290
Sábado: +155
Domingo: -5
Neto de la semana: +780
Seguidores al domingo: 5650
Mejor día: Viernes (+290)
Días con saldo negativo: 2
```

Reglas

- Alcanza con un bucle, acumuladores, contadores y un máximo.
- Los netos positivos o cero se muestran con "+" adelante (+0 incluido); los negativos, con "-".
- Si dos días empatan como mejores, se informa el primero.
- Los datos se pueden ingresar día por día.

Pista (leela solo si te trabás): el "mejor día" se resuelve guardando el mejor neto visto hasta ahora y actualizándolo solo cuando aparece uno estrictamente mayor.

Bonus (opcional)

- Calculá la tasa de crecimiento semanal en porcentaje respecto de los seguidores iniciales.
- Proyectá cuántas semanas harían falta para llegar a 10.000 seguidores si todas las semanas fueran iguales a esta.

Cierre: Es el mismo cálculo que muestran los paneles de estadísticas de las redes sociales para cuentas de empresa.

Ejercicio 19: Tres intentos para entrar

Nivel 1: Junior I · Fundamentos · Temática: Seguridad informática · Dificultad 4/100

Situación: La app de una cooperativa de crédito pide un PIN de 4 dígitos para entrar. Detectaron que alguien probaba PINs al azar sin ningún límite, así que ahora quieren bloquear la cuenta después de 3 intentos fallidos. Tu programa simula dos sesiones para probar la regla.

Datos de entrada

PIN correcto de Julia: 4821

Sesión	Intentos, en orden
A	1234, 4812, 4821
B	0000, 1111, 4280, 4821

Tu programa debe

1. Procesar los intentos de cada sesión en orden, comparando contra el PIN correcto.
2. Ante un error, avisar cuántos intentos quedan (cuidando singular y plural).
3. Al tercer error, bloquear la cuenta; los intentos siguientes se ignoran aunque sean correctos.
4. Si acierta, dar la bienvenida e ignorar los intentos que vengan después.

Salida esperada

```
Sesión A
Intento 1: incorrecto (te quedan 2)
Intento 2: incorrecto (te queda 1)
Intento 3: correcto. ¡Bienvenida, Julia!
Sesión B
Intento 1: incorrecto (te quedan 2)
Intento 2: incorrecto (te queda 1)
Intento 3: incorrecto. Cuenta bloqueada por 15 minutos.
Intento 4: ignorado (cuenta bloqueada)
```

Reglas

- Alcanza con un bucle, un contador y variables de estado.
- Cada sesión empieza con el contador de errores en cero.
- "Te quedan 2" pero "te queda 1".
- El PIN 0000 tiene que tratarse como un PIN válido de 4 dígitos (si lo guardás como número, es 0).

Pista (leela solo si te trabás): usá dos variables de estado además del contador: una que diga si la cuenta está bloqueada y otra que diga si ya ingresó. Consultalas al principio de cada vuelta.

Bonus (opcional)

- Hací que el bloqueo dure 15 minutos simulando la hora de cada intento.
- Si se bloquea tres veces en el día, la cuenta queda bloqueada hasta llamar a atención al cliente.

Cierre: Es la protección básica contra ataques de fuerza bruta que tienen todos los home banking y cajeros automáticos.

Ejercicio 20: La alcancía con meta de ahorro

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 4/100

Situación: Bauti tiene 12 años y quiere comprarse una bici de \$180.000. Ya juntó \$20.000, ahorra \$12.500 por semana de lo que gana cortando el pasto de los vecinos, y cada 4 semanas su abuela le suma \$8.000. Quiere saber en cuántas semanas llega.

Datos de entrada

Dato	Valor
Meta	\$180.000
Ahorro inicial	\$20.000
Ahorro semanal	\$12.500
Regalo de la abuela	\$8.000 cada 4 semanas (semanas 4, 8, 12...)

Tu programa debe

1. Simular semana a semana cómo crece el ahorro.
2. Sumar el regalo de la abuela en las semanas múltiplo de 4 y marcarlo.
3. Detenerse cuando el ahorro alcance o supere la meta.
4. Mostrar en cuántas semanas llegó y cuánto le sobra.

Salida esperada

```
Semana 1: $32500
Semana 2: $45000
Semana 3: $57500
Semana 4: $78000 (incluye regalo de la abuela)
Semana 5: $90500
Semana 6: $103000
Semana 7: $115500
Semana 8: $136000 (incluye regalo de la abuela)
Semana 9: $148500
Semana 10: $161000
Semana 11: $173500
Semana 12: $194000 (incluye regalo de la abuela)
Llega a la meta en 12 semanas y le sobran $14000
```

Reglas

- Alcanza con un bucle "mientras", un acumulador y el operador resto.
- Si el ahorro inicial ya alcanza la meta, el programa no muestra ninguna semana y dice que llega en 0 semanas.
- El regalo se suma en la misma semana en que corresponde, después del ahorro semanal.

Pista (leela solo si te trabás): el bucle se repite mientras el ahorro sea menor que la meta. Para saber si una semana es múltiplo de 4, mirá el resto de dividirla por 4.

Bonus (opcional)

- La bici aumenta un 2 % cada 4 semanas. ¿Llega igual? ¿En cuántas semanas?
- Calculá cuánto tendría que ahorrar por semana para llegar en exactamente 8 semanas.

Cierre: Es la misma simulación que hacen las apps de ahorro con metas, esas que te muestran una barra de progreso hasta el objetivo.

Ejercicio 21: Pasos diarios hacia los 10.000

Nivel 1: Junior I · Fundamentos · Temática: Salud · Dificultad 5/100

Situación: El médico le recomendó a Alicia caminar 10.000 pasos por día. Su reloj cuenta los pasos pero no le arma un resumen semanal, y ella quiere mostrárselo en el próximo control. Tu programa arma ese resumen.

Datos de entrada

Día	Pasos
Lunes	8.450
Martes	12.300
Miércoles	10.000
Jueves	6.720
Viernes	9.980
Sábado	15.400
Domingo	11.250

- Meta diaria: 10.000 pasos
- Largo promedio del paso de Alicia: 0,75 m

Tu programa debe

1. Para cada día, indicar si cumplió la meta o cuántos pasos le faltaron.
2. Calcular el total de pasos y la distancia total en kilómetros.
3. Calcular el promedio diario.
4. Contar los días con meta cumplida.

Salida esperada

```
Lunes: 8450 pasos - faltaron 1550
Martes: 12300 pasos - meta cumplida
Miércoles: 10000 pasos - meta cumplida
Jueves: 6720 pasos - faltaron 3280
Viernes: 9980 pasos - faltaron 20
Sábado: 15400 pasos - meta cumplida
Domingo: 11250 pasos - meta cumplida
Total: 74100 pasos (55.58 km)
Promedio diario: 10585.71
Días con meta cumplida: 4 de 7
```

Reglas

- Alcanza con un bucle, acumuladores y condicionales.
- Llegar exactamente a 10.000 cuenta como meta cumplida.
- Kilómetros y promedio se muestran con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): los kilómetros salen de $\text{pasos} \times 0,75 / 1.000$. Hacé la cuenta con el total, no sumando kilómetros redondeados día por día.

Bonus (opcional)

- Calculá la racha más larga de días consecutivos con meta cumplida.
- Estimá las calorías: 0,04 kcal por paso.

Cierre: Es el mismo resumen que muestran los relojes inteligentes y las apps de actividad física.

Ejercicio 22: La billetera virtual que rinde todos los días

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 5/100

Situación: La billetera virtual de Ramiro paga intereses todos los días sobre el saldo, y los intereses de ayer también generan intereses hoy. Él escuchó que eso "rinde más" que cobrar todo junto al final, pero no sabe cuánto más. Tu programa simula una semana.

Datos de entrada

Dato	Valor
Saldo inicial	\$250.000
TNA	32 %
Días	7

- Interés de cada día = saldo del día $\times$ TNA / 365, redondeado a 2 decimales.
- Interés simple de comparación = saldo inicial $\times$ TNA $\times$ 7 / 365, redondeado a 2 decimales.

Tu programa debe

1. Simular día por día: calcular el interés, redondearlo y sumarlo al saldo.
2. Mostrar el interés acreditado y el saldo al final de cada día.
3. Calcular la ganancia total de la semana.
4. Comparar con el interés simple y mostrar la diferencia.

Salida esperada

```
Día 1: +$219.18 -> $250219.18
Día 2: +$219.37 -> $250438.55
Día 3: +$219.56 -> $250658.11
Día 4: +$219.76 -> $250877.87
Día 5: +$219.95 -> $251097.82
Día 6: +$220.14 -> $251317.96
Día 7: +$220.33 -> $251538.29
Ganancia con interés diario: $1538.29
Ganancia con interés simple: $1534.25
Diferencia a favor del interés diario: $4.04
```

Reglas

- Alcanza con un bucle y un acumulador con decimales.
- Todos los montos van con 2 decimales, redondeo hacia arriba desde 0,5. Los decimales se muestran con punto y sin separador de miles en la salida.
- El redondeo se hace cada día, antes de sumar al saldo (así lo hacen las billeteras reales).
- Si trabajás con números decimales binarios podés tener errores de centavos: redondeá en cada paso o trabajá en centavos enteros.

Pista (leela solo si te trabás): trabajar en centavos (enteros) evita sorpresas: 250.000 pesos son 25.000.000 centavos. Aun así, el redondeo diario tiene que ser explícito.

Bonus (opcional)

- Extendé la simulación a 365 días y calculá la tasa efectiva anual real.
- Agregá un retiro de \$50.000 el día 4 y mirá cuánto afecta el resultado.

Cierre: Es el cálculo que hacen todas las noches las billeteras virtuales y cuentas remuneradas.

Ejercicio 23: Kilómetros por litro del camión de reparto

Nivel 1: Junior I · Fundamentos · Temática: Logística y envíos · Dificultad 5/100

Situación: Una empresa de reparto quiere detectar si el camión del recorrido sur está consumiendo de más (puede ser un problema mecánico o un desvío de combustible). El chofer anota el odómetro y los litros cada vez que llena el tanque. Tu programa calcula el rendimiento entre cargas.

Datos de entrada

Odómetro al llenar el tanque por primera vez: 120.000 km

Carga	Odómetro (km)	Litros cargados
1	120.480	62
2	120.910	55
3	121.450	71
4	121.800	48

Alerta si un tramo rinde menos de 7,5 km/L.

Tu programa debe

1. Calcular, para cada tramo, los kilómetros recorridos desde la carga anterior.
2. Calcular el rendimiento del tramo: km recorridos / litros de la carga que cierra el tramo.
3. Marcar con "REVISAR" los tramos que rinden menos de 7,5 km/L.
4. Mostrar el rendimiento general y el peor tramo.

Salida esperada

```
Tramo 1: 480 km / 62 L = 7.74 km/L
Tramo 2: 430 km / 55 L = 7.82 km/L
Tramo 3: 540 km / 71 L = 7.61 km/L
Tramo 4: 350 km / 48 L = 7.29 km/L - REVISAR
Rendimiento general: 7.63 km/L (1800 km, 236 L)
Peor tramo: 4 (7.29 km/L)
```

Reglas

- Alcanza con un bucle, acumuladores y un mínimo.
- Los rendimientos se muestran con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- El rendimiento general es km totales / litros totales, NO el promedio de los rendimientos de cada tramo (dan distinto).
- La alerta se evalúa con el valor sin redondear.

Pista (leela solo si te trabás): guardá el odómetro de la carga anterior en una variable; en cada vuelta, la resta con el actual te da los km del tramo y después actualizás la variable.

Bonus (opcional)

- Calculá el costo por kilómetro con un precio de \$1.150 por litro.
- Si el odómetro de una carga es menor que el de la anterior, detectá el error de carga y saltealo.

Cierre: Es la misma lógica que usan los sistemas de gestión de flotas para detectar consumo anómalo.

Ejercicio 24: La receta para 12 comensales

Nivel 1: Junior I · Fundamentos · Temática: Gastronomía · Dificultad 5/100

Situación: Una rotisería vende panqueques con dulce de leche y la receta de la abuela está pensada para 8 porciones. Hoy tienen un pedido para 12 personas y el cocinero no quiere hacer las cuentas a ojo porque la última vez la masa quedó líquida.

Datos de entrada

Receta base (8 porciones):

Ingrediente	Cantidad
Harina	400 g
Huevos	3 unidades
Leche	750 ml
Azúcar	30 g
Manteca	50 g
Sal	4 g

Porciones pedidas: 12. La leche se compra en sachets de 1 L.

Tu programa debe

1. Calcular el factor de escala (porciones pedidas / porciones de la receta).
2. Multiplicar cada ingrediente por el factor.
3. Mostrar cada cantidad con la unidad adecuada: los huevos, redondeados hacia arriba; los líquidos de 1.000 ml o más, en litros.
4. Calcular cuántos sachets de leche hay que comprar.

Salida esperada

```
Receta para 12 (factor 1.50)
Harina: 600 g
Huevos: 5 unidades
Leche: 1.13 L
Azúcar: 45 g
Manteca: 75 g
Sal: 6 g
Sachets de leche de 1 L a comprar: 2
```

Reglas

- Alcanza con proporciones, decimales y condicionales.
- Gramos y mililitros se muestran sin decimales; litros con 2 decimales. Redondeo hacia arriba desde 0,5. Los decimales se muestran con punto y sin separador de miles en la salida.
- No existen medios huevos: siempre se redondea hacia arriba.
- El factor se muestra con 2 decimales.

Pista (leela solo si te trabás): calculá el factor una sola vez y aplicalo a todos los ingredientes. La unidad de cada ingrediente decide cómo se redondea y cómo se muestra.

Bonus (opcional)

- Hacé que el programa también sirva para achicar la receta (por ejemplo, para 3 porciones).
- Calculá el costo total del pedido con una lista de precios por kilo, litro o unidad.

Cierre: Es la misma función de "ajustar porciones" que tienen las apps y sitios de recetas.

Ejercicio 25: Tablas de multiplicar para la seño

Nivel 1: Junior I · Fundamentos · Temática: Educación · Dificultad 5/100

Situación: La seño de cuarto grado arma fichas de repaso con las tablas que más les cuestan a los chicos: la del 6 a la del 9. Quiere cada tabla en una sola línea para ahorrar papel, con la suma al final para un juego de cálculo mental, y saber cuántos resultados pasan de 50.

Datos de entrada

Dato	Valor
Tablas	del 6 al 9
Multiplicadores	del 1 al 10
Umbral	50

Tu programa debe

1. Generar cada tabla en una sola línea, con los resultados separados por un espacio.
2. Mostrar al final de cada línea la suma de sus resultados.
3. Contar cuántos resultados, entre todas las tablas, son mayores que 50.

Salida esperada

```
Tabla del 6: 6 12 18 24 30 36 42 48 54 60 | suma 330
Tabla del 7: 7 14 21 28 35 42 49 56 63 70 | suma 385
Tabla del 8: 8 16 24 32 40 48 56 64 72 80 | suma 440
Tabla del 9: 9 18 27 36 45 54 63 72 81 90 | suma 495
Resultados mayores que 50: 14
```

Reglas

- Alcanza con dos bucles anidados, un acumulador y un contador.
- El valor 50 no cuenta como "mayor que 50".
- El programa tiene que funcionar si la seño cambia el rango de tablas o el umbral.

Pista (leela solo si te trabás): el bucle de afuera recorre las tablas y el de adentro los multiplicadores. La suma de cada tabla se reinicia en cada vuelta del bucle de afuera; el contador de mayores que 50 no.

Bonus (opcional)

- Generará una ficha de ejercicios "salteados": solo los multiplicadores impares.
- Mostrará la tabla completa del 1 al 10 como una grilla alineada en columnas.

Cierre: Es el mismo patrón de bucles anidados con el que se generan planillas, grillas y calendarios en cualquier sistema.

Lote 02: ejercicios 26 a 50

Ejercicio 26: El año bisiesto del cumpleaños

Nivel 1: Junior I · Fundamentos · Temática: Ciencia · Dificultad 6/100

Situación: Valentina nació el 29 de febrero de 2004 y su familia bromea con que solo cumple años "de verdad" cada cuatro. Un compañero le dijo que el año 2100 no va a ser bisiesto y ella no le creyó. Tu programa tiene que aplicar la regla completa del calendario y contar sus cumpleaños reales.

Datos de entrada

Regla del calendario: un año es bisiesto si es divisible por 4, excepto los divisibles por 100, salvo que también sean divisibles por 400.

Dato	Valor
Años a verificar	1900, 2000, 2024, 2100, 2023
Rango para contar cumpleaños	de 2004 a 2030, inclusive

Tu programa debe

1. Decir, para cada año a verificar, si es bisiesto o no.
2. Recorrer los años de 2004 a 2030 y contar cuántos son bisiestos.
3. Mostrar la cantidad y los años, separados por un espacio.

Salida esperada

```
1900: no bisiesto
2000: bisiesto
2024: bisiesto
2100: no bisiesto
2023: no bisiesto
Cumpleaños reales de Valentina entre 2004 y 2030: 7
2004 2008 2012 2016 2020 2024 2028
```

Reglas

- Alcanza con condicionales compuestos (y, o, no), el operador resto y un bucle.
- Los extremos del rango se incluyen.
- La condición tiene que resolverse en una sola expresión o en condicionales anidados, pero sin tablas de años precargados.

Pista (leela solo si te trabás): escribí la regla tal como la dirías en voz alta: "divisible por 4 y no divisible por 100, o divisible por 400". Probala primero con 1900 y 2000, que son los casos que suelen fallar.

Bonus (opcional)

- Calculá cuántos cumpleaños "reales" tendría Valentina si llega a los 100 años.
- Decí qué día de la semana cae el 29 de febrero de 2028 sabiendo que el 29 de febrero de 2024 fue jueves.

Cierre: Es la misma regla que implementa cualquier calendario digital, desde el del celular hasta el de un sistema bancario que calcula vencimientos.

Ejercicio 27: Estacionamiento por fracción de hora

Nivel 1: Junior I · Fundamentos · Temática: Transporte · Dificultad 6/100

Situación: Un estacionamiento del centro cobra la primera hora completa y después por fracciones de 15 minutos, con un tope diario. El playero hace las cuentas con un reloj y un papel, y los clientes reclaman todo el tiempo. Tu programa calcula cada ticket.

Datos de entrada

Concepto	Precio
Primera hora (o fracción)	\$1.800
Cada 15 minutos o fracción después de la primera hora	\$500
Tope diario	\$9.000

Patente	Entrada	Salida
AB 123 CD	09:10	09:55
AC 456 EF	08:00	10:20
AD 789 GH	07:30	19:05
AE 012 IJ	12:00	13:00
AF 345 KL	12:00	13:01

Tu programa debe

1. Calcular los minutos de estadía de cada auto.
2. Cobrar \$1.800 por la primera hora y \$500 por cada fracción de 15 minutos adicional.
3. Aplicar el tope diario si corresponde.
4. Mostrar el cobro de cada auto y la recaudación total.

Salida esperada

```
AB 123 CD: 45 min -> $1800
AC 456 EF: 140 min -> $4800
AD 789 GH: 695 min -> $9000 (tope diario)
AE 012 IJ: 60 min -> $1800
AF 345 KL: 61 min -> $2300
Recaudación: $19700
```

Reglas

- Alcanza con aritmética, condicionales y redondeo hacia arriba.
- Todos los autos entran y salen el mismo día.
- Exactamente 60 minutos es solo la primera hora; 61 minutos ya suma una fracción.
- Cuando se aplica el tope, se aclara con "(tope diario)".

Pista (leela solo si te trabás): convertí cada horario a minutos desde la medianoche. Para las fracciones: minutos extra divididos por 15, redondeando hacia arriba.

Bonus (opcional)

- Agregá la tarifa nocturna: entre las 22:00 y las 06:00 cada fracción cuesta \$300.
- Soportá autos que salen al día siguiente de haber entrado.

Cierre: Es la misma lógica de tarifa por fracción que usan los sistemas de cobro de estacionamientos y de alquiler de bicicletas o monopatines.

Ejercicio 28: Préstamo entre amigos con interés simple

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 6/100

Situación: Nacho le presta \$300.000 a su amigo Fede para arreglar el auto. Acuerdan un 4 % de interés mensual simple (siempre sobre el capital original) y que Fede devuelva todo en 6 cuotas iguales. Para que no haya malentendidos, quieren un cronograma claro.

Datos de entrada

Dato	Valor
Capital prestado	\$300.000
Interés simple mensual	4 % sobre el capital
Cantidad de cuotas	6 mensuales, iguales

Tu programa debe

1. Calcular el interés total del préstamo.
2. Calcular el total a devolver y el valor de cada cuota.
3. Mostrar, mes a mes, cuánto lleva pagado Fede y cuánto le falta.

Salida esperada

```
Interés total: $72000
Total a devolver: $372000
Cuota mensual: $62000
Mes 1: pagado $62000 | resta $310000
Mes 2: pagado $124000 | resta $248000
Mes 3: pagado $186000 | resta $186000
Mes 4: pagado $248000 | resta $124000
Mes 5: pagado $310000 | resta $62000
Mes 6: pagado $372000 | resta $0
```

Reglas

- Alcanza con fórmulas, porcentajes y un bucle.
- Interés simple: cada mes genera el mismo interés, calculado sobre el capital original.
- En este caso la cuota da exacta; si no, la última cuota tendría que ajustar la diferencia (ver Bonus).

Pista (leela solo si te trabás): con interés simple, el interés total es $\text{capital} \times \text{tasa} \times \text{meses}$. Después, todo lo demás es dividir y acumular.

Bonus (opcional)

- Hacé que funcione con montos que no dividen exacto: las primeras cuotas se redondean hacia abajo y la última absorbe la diferencia.
- Compará con un préstamo al 4 % mensual sobre saldo (interés compuesto) y mostrá cuál le conviene a Fede.

Cierre: Es la misma cuenta que hacen los simuladores de préstamos más simples y los planes de pago en cuotas fijas.

Ejercicio 29: El vuelto con billetes y monedas

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 6/100

Situación: En la caja de una ferretería, el cajero tarda en armar el vuelto y a veces entrega de más. El dueño quiere que la pantalla le diga exactamente qué billetes y monedas devolver, usando siempre la menor cantidad de piezas posible con las denominaciones disponibles.

Datos de entrada

Billetes: \$10.000, \$2.000, \$1.000, \$500, \$200, \$100

Monedas: \$50, \$10, \$5, \$1

Venta	Monto de la compra	Paga con
1	\$13.476	\$20.000
2	\$8.750	\$10.000
3	\$5.300	\$5.000

Tu programa debe

1. Calcular el vuelto de cada venta.
2. Si el pago no alcanza, informar cuánto falta y no calcular vuelto.
3. Desglosar el vuelto empezando por la denominación más grande.
4. Mostrar billetes y monedas por separado, solo con las denominaciones que se usan.

Salida esperada

```
Venta 1: compra $13476, paga $20000
Vuelto: $6524
Billetes: 3 x $2000, 1 x $500
Monedas: 2 x $10, 4 x $1
Venta 2: compra $8750, paga $10000
Vuelto: $1250
Billetes: 1 x $1000, 1 x $200
Monedas: 1 x $50
Venta 3: compra $5300, paga $5000
Pago insuficiente: faltan $300
```

Reglas

- Alcanza con división entera, resto y un bucle (o una secuencia de condicionales).
- Si en un grupo (billetes o monedas) no se usa ninguna denominación, esa línea no se muestra.
- Con estas denominaciones, empezar siempre por la más grande da la menor cantidad de piezas.
- Si paga justo, mostrará "Sin vuelto".

Pista (leela solo si te trabás): para cada denominación, la división entera te dice cuántas piezas usar y el resto te dice cuánto falta devolver. Repetí con la siguiente denominación usando ese resto.

Bonus (opcional)

- La caja tiene una cantidad limitada de cada billete y moneda: si no alcanza, usá la denominación siguiente.
- Mostrá cuántas piezas en total se entregan en cada vuelto.

Cierre: Es exactamente lo que hacen las cajas registradoras con cajón automático y las máquinas de autoservicio que devuelven vuelto.

Ejercicio 30: Paneles solares para el techo

Nivel 1: Junior I · Fundamentos · Temática: Clima y energía · Dificultad 6/100

Situación: La familia Ortiz quiere instalar paneles solares para dejar de depender de la red. Un instalador les dio datos técnicos y precios, pero no les dijo cuántos paneles necesitan, si entran en el techo ni en cuánto tiempo recuperan la inversión. Tu programa hace el estudio.

Datos de entrada

Dato	Valor
Consumo mensual de la casa	420 kWh
Generación promedio por panel	1,35 kWh por día
Días por mes (para el cálculo)	30
Superficie útil del techo	22 m ²
Superficie por panel	2 m ²
Precio por panel	\$185.000
Instalación (costo fijo)	\$350.000
Precio del kWh de la red	\$140

Tu programa debe

1. Calcular cuántos kWh genera un panel por mes y cuántos paneles hacen falta para cubrir el consumo.
2. Verificar si esa cantidad entra en el techo.
3. Calcular la inversión total y el ahorro mensual (lo que dejarían de pagarle a la red).
4. Calcular en cuántos meses se recupera la inversión y expresarlo en años y meses.

Salida esperada

```

Generación por panel: 40.5 kWh/mes
Paneles necesarios: 11
Entran en el techo: sí (cabén 11)
Inversión total: $2385000
Ahorro mensual: $58800
Recupero de la inversión: 41 meses (3 años y 5 meses)
  
```

Reglas

- Alcanza con aritmética, redondeo hacia arriba y condicionales.
- Los paneles se redondean hacia arriba (no existe medio panel), igual que los meses de recupero.
- La generación por panel se muestra con 1 decimal. Los decimales se muestran con punto y sin separador de miles en la salida.
- Suponé que el precio del kWh no cambia.

Pista (leela solo si te trabás): tres cuentas del tipo "cuántos necesito para cubrir X" aparecen en este ejercicio y todas se resuelven con una división redondeada hacia arriba.

Bonus (opcional)

- Si no entran todos los paneles, calculá qué porcentaje del consumo cubren los que sí entran.
- Suponé que el kWh aumenta 4 % por mes y recalculá el recupero mes a mes.

Cierre: Es el cálculo preliminar que hacen los simuladores de energía solar antes de una visita técnica.

Ejercicio 31: Racha de victorias del equipo

Nivel 1: Junior I · Fundamentos · Temática: Deportes · Dificultad 7/100

Situación: El club de hockey del barrio quiere premiar a su equipo de primera con una placa por la mejor racha de la temporada. El secretario tiene los resultados en orden, partido por partido, pero no sabe cuál fue la racha más larga de victorias ni la racha invicta (sin perder).

Datos de entrada

Resultados de los 15 partidos, en orden (G = ganó, E = empató, P = perdió):

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
G	G	E	G	G	G	P	G	G	G	G	E	P	G	G

Tu programa debe

1. Contar partidos ganados, empatados y perdidos.
2. Calcular los puntos (3 por victoria, 1 por empate).
3. Encontrar la racha más larga de victorias consecutivas.
4. Encontrar la racha invicta más larga (partidos consecutivos sin perder).

Salida esperada

Ganados: 11 | Empatados: 2 | Perdidos: 2
Puntos: 35
Racha más larga de victorias: 4
Racha invicta más larga: 6

Reglas

- Alcanza con un bucle, contadores, condicionales y contadores que se reinician.
- Un empate corta la racha de victorias pero no la racha invicta.
- Los resultados se pueden procesar de a uno, en orden.

Pista (leela solo si te trabás): para cada racha usá dos variables: la racha actual (que se reinicia) y la mejor racha vista hasta ahora (que solo crece).

Bonus (opcional)

- Informá también en qué partido empezó y terminó la racha invicta más larga.
- Calculá la racha más larga sin ganar (empates y derrotas seguidas).

Cierre: Es el mismo cálculo de rachas que muestran las transmisiones deportivas y los sitios de estadísticas.

Ejercicio 32: Monedas de oro del cofre

Nivel 1: Junior I · Fundamentos · Temática: Videojuegos · Dificultad 7/100

Situación: En un juego de rol para celular, los jugadores abren cofres al terminar cada nivel. El diseñador quiere balancear la economía: saber cuántas monedas junta un jugador típico en 10 cofres y en qué momento puede comprar la espada legendaria. Tu programa simula esa partida.

Datos de entrada

Cofre	Monedas
Bronce	15
Plata	40
Oro	100

- El jugador consigue un amuleto al abrir el cofre 4: desde ese cofre (inclusive), los cofres de plata dan el doble.
- Cada 5 cofres (el 5, el 10...) hay un bonus de 25 monedas.
- La espada legendaria cuesta 500 monedas.

Cofres abiertos, en orden: bronce, plata, oro, plata, bronce, bronce, oro, plata, bronce, oro

Tu programa debe

- Calcular las monedas de cada cofre aplicando amuleto y bonus.
- Mostrar el detalle de cada cofre y el acumulado.
- Mostrar el total final.
- Informar desde qué cofre le alcanza para la espada.

Salida esperada

```
Cofre 1 (bronce): +15 -> 15
Cofre 2 (plata): +40 -> 55
Cofre 3 (oro): +100 -> 155
Cofre 4 (plata, amuleto x2): +80 -> 235
Cofre 5 (bronce, bonus +25): +40 -> 275
Cofre 6 (bronce): +15 -> 290
Cofre 7 (oro): +100 -> 390
Cofre 8 (plata, amuleto x2): +80 -> 470
Cofre 9 (bronce): +15 -> 485
Cofre 10 (oro, bonus +25): +125 -> 610
Total: 610 monedas
Espada (500): la puede comprar desde el cofre 10
```

Reglas

- Alcanza con un bucle, acumuladores y condicionales.
- Primero se aplica el amuleto y después se suma el bonus.
- En el detalle se muestra el tipo de cofre y, si aplican, "amuleto x2" y "bonus +25", separados por coma.
- La espada se puede comprar cuando el acumulado es mayor o igual a 500.

Pista (leela solo si te trabás): el momento en que alcanza para la espada se detecta una sola vez: guardá el número de cofre la primera vez que el acumulado llega a 500 y no lo pises después.

Bonus (opcional)

- Agregá cofres malditos que restan 30 monedas, sin que el total pueda quedar negativo.
- Calculá cuántos cofres más necesitaría para comprar también un escudo de 400 monedas.

Cierre: Es la misma simulación que hacen los diseñadores de videojuegos para balancear la economía interna.

Ejercicio 33: ¿Cuotas sin interés o contado con descuento?

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 7/100

Situación: Mónica quiere comprar una heladera de \$1.200.000. La tienda le ofrece 15 % de descuento pagando de contado o 12 cuotas sin interés. Con inflación, pagar en cuotas puede convenir aunque se pierda el descuento, porque cada cuota "vale menos" con el paso de los meses. Tu programa hace la comparación justa.

Datos de entrada

Dato	Valor
Precio de lista	\$1.200.000
Descuento por contado	15 %
Cuotas sin interés	12, mensuales, iguales
Inflación mensual estimada	3 %

Valor presente de la cuota del mes k = cuota / $(1,03)^k$. La primera cuota se paga en el mes 1.

Tu programa debe

1. Calcular el precio de contado y el valor de cada cuota.
2. Calcular el valor presente de cada cuota y sumarlos.
3. Comparar con el precio de contado y decir qué conviene y por cuánto.

Salida esperada

```
Contado: $1020000.00
Cuota: $100000.00 x 12
Valor presente de las cuotas: $995400.40
Conviene: cuotas (ahorrás $24599.60 en valor presente)
```

Reglas

- Alcanza con un bucle, un acumulador con decimales y un condicional.
- Los montos se muestran con 2 decimales, redondeo hacia arriba desde 0,5. Los decimales se muestran con punto y sin separador de miles en la salida.
- El valor presente se redondea una sola vez, al final de la suma.
- No uses una función de potencia: calculá $(1,03)^k$ multiplicando dentro del bucle.

Pista (leela solo si te trabás): Llevá una variable "factor" que empiece en 1 y en cada vuelta se multiplique por 1,03. Dividí la cuota por ese factor y acumulá.

Bonus (opcional)

- Calculá con qué inflación mensual da lo mismo pagar de contado o en cuotas (probá valores de a 0,01 %).
- Agregá la opción de 6 cuotas con 5 % de descuento y compará las tres alternativas.

Cierre: Es la misma comparación que hacen las calculadoras de "¿cuotas o contado?" que publican los sitios de finanzas personales.

Ejercicio 34: La helada de la madrugada

Nivel 1: Junior I · Fundamentos · Temática: Agricultura · Dificultad 7/100

Situación: En una chacra de frutales del Alto Valle, una helada puede arruinar la floración de toda la temporada. Un sensor registra la temperatura cada hora durante la noche y el productor necesita, a primera hora, un resumen para decidir si activa el riego antiheladas.

Datos de entrada

Hora	Temperatura (°C)
20:00	8,2
21:00	6,5
22:00	5,1
23:00	3,8
00:00	2,4
01:00	1,1
02:00	0,3
03:00	-0,8
04:00	-1,9
05:00	-2,6
06:00	-1,4
07:00	0,9

Criterio de alerta: mínima menor que -2 °C → helada fuerte; mínima menor que 0 °C → helada leve; si no, sin helada.

Tu programa debe

1. Encontrar la temperatura mínima y a qué hora ocurrió.
2. Encontrar la temperatura máxima y calcular la amplitud térmica (máxima – mínima).
3. Contar cuántas horas estuvo bajo cero.
4. Emitir la alerta según el criterio.

Salida esperada

```
Mínima: -2.6 °C a las 05:00
Máxima: 8.2 °C
Amplitud térmica: 10.8 °C
Horas bajo cero: 4
Alerta: HELADA FUERTE - activar riego antiheladas
```

Reglas

- Alcanza con un bucle, variables de máximo y mínimo, un contador y condicionales.
- Las temperaturas se muestran con 1 decimal. Los decimales se muestran con punto y sin separador de miles en la salida.
- Si la mínima se repite, se informa la primera hora en que ocurrió.
- 0 °C no cuenta como bajo cero.

Pista (leela solo si te trabás): inicializá mínimo y máximo con la primera lectura, no con 0: si inicializás el máximo en 0, una noche entera bajo cero te daría un resultado falso.

Bonus (opcional)

- Calculá cuántas horas seguidas estuvo bajo cero (la racha más larga).
- Estimá la hora en que la temperatura cruzó los 0 °C interpolando entre las dos lecturas vecinas.

Cierre: Es la lógica básica de las estaciones agrometeorológicas que mandan alertas de helada a los productores.

Ejercicio 35: Duración total de la playlist

Nivel 1: Junior I · Fundamentos · Temática: Música · Dificultad 7/100

Situación: Una banda independiente arma la playlist de su nuevo disco y quiere mostrar en la contratapa la duración total. Además, el cantante tarda 30 minutos en llegar al ensayo y quiere saber cuántos temas completos alcanza a escuchar en orden.

Datos de entrada

N.º	Tema	Minutos	Segundos
1	Madrugada	3	45
2	Ruta 40	4	12
3	Café con leche	2	58
4	Tormenta	5	30
5	Faro	3	3
6	Cordillera	6	41
7	Siesta	4	0
8	Humo	3	27
9	Mate amargo	2	15
10	Vuelta a casa	4	49

Tu programa debe

1. Calcular la duración total del disco en formato H:MM:SS.
2. Calcular la duración promedio por tema en formato M:SS.
3. Encontrar el tema más largo.
4. Contar cuántos temas completos, en orden desde el primero, entran en 30 minutos.

Salida esperada

```
Duración total: 0:40:40
Promedio por tema: 4:04
Tema más largo: Cordillera (6:41)
Temas completos en un viaje de 30 minutos: 6 (26:09)
```

Reglas

- Alcanza con conversión de unidades, acumuladores, un máximo y un bucle con corte.
- El promedio se redondea al segundo más cercano (0,5 hacia arriba).
- Para el viaje, si un tema no entra completo, se corta ahí: no se saltea para probar con el siguiente.
- Minutos y segundos se muestran siempre con dos dígitos, salvo el primer número (horas o minutos).

Pista (leela solo si te trabás): trabajá todo en segundos y convertí a H:MM:SS solo al mostrar. Las horas son segundos / 3600; los minutos, el resto de eso dividido por 60.

Bonus (opcional)

- Permití saltar temas que no entran y seguir probando con los siguientes: ¿cuántos entran ahora?
- Encontrá la combinación de temas consecutivos (desde cualquier posición) que más se acerque a 30 minutos sin pasarse.

Cierre: Es el mismo cálculo que hacen los reproductores de música cuando muestran la duración total de una lista.

Ejercicio 36: Presupuesto de las vacaciones en la costa

Nivel 1: Junior I · Fundamentos · Temática: Turismo · Dificultad 7/100

Situación: La familia Giménez (cuatro personas) planea una semana en la costa y tiene ahorrado \$1.200.000. Quieren saber si les alcanza y, si sobra, cuántas noches más podrían quedarse. Tu programa arma el presupuesto completo.

Datos de entrada

Concepto	Valor
Personas	4
Noches	7
Alojamiento	\$58.000 por noche
Distancia a la costa	400 km (solo ida)
Rendimiento del auto	12 km por litro
Precio de la nafta	\$1.150 por litro
Comida	\$9.500 por persona por día
Extras (excursiones, regalos)	\$80.000
Presupuesto disponible	\$1.200.000

Tu programa debe

1. Calcular el costo de alojamiento, nafta (ida y vuelta), comida (un día por noche) y extras.
2. Sumar el total y compararlo con el presupuesto.
3. Si alcanza, calcular cuántas noches extra podrían pagar con lo que sobra (cada noche extra suma alojamiento + un día de comida para todos).
4. Si no alcanza, informar cuánto falta.

Salida esperada

```
Alojamiento: $406000
Nafta (67 L): $77050
Comida: $266000
Extras: $80000
Total: $829050
Alcanza: sobran $370950
Noches extra posibles: 3 (sobrarían $82950)
```

Reglas

- Alcanza con aritmética, redondeo hacia arriba, división entera y condicionales.
- Los litros de nafta se redondean hacia arriba a litros enteros antes de multiplicar por el precio.
- Las noches extra no generan nafta ni extras adicionales.

Pista (leela solo si te trabás): separá el problema en subtotales con nombres claros (alojamiento, nafta, comida). Así es fácil revisar cada uno por separado si el total no te coincide.

Bonus (opcional)

- Agregá peajes: \$3.200 por tramo, 3 tramos por viaje.
- Si no alcanza, calculá cuál es la mayor cantidad de noches que sí pueden pagar.

Cierre: Es la misma lógica que usan los cotizadores de paquetes turísticos para armar un presupuesto por partes.

Ejercicio 37: ¿Es primo el número de la rifa?

Nivel 1: Junior I · Fundamentos · Temática: Ciencia · Dificultad 8/100

Situación: La cooperadora de una escuela organiza una rifa con una regla simpática para la clase de matemática: los números ganadores que sean primos cobran el doble. El problema es que nadie sabe a simple vista si 7.919 es primo. Tu programa lo decide.

Datos de entrada

Números ganadores: 7919, 1001, 2, 1, 997, 4096

Tu programa debe

1. Para cada número, decidir si es primo.
2. Si no es primo, mostrar su menor divisor mayor que 1.
3. Contar cuántos números primos hay.

Salida esperada

```
7919: primo
1001: no es primo (divisible por 7)
2: primo
1: no es primo (por definición)
997: primo
4096: no es primo (divisible por 2)
Números primos (ganan doble): 3
```

Reglas

- Alcanza con bucles, condicionales y el operador resto.
- El 1 no es primo por definición (y tampoco el 0).
- Solo hace falta probar divisores d mientras $d \times d \leq \text{número}$: si no encontraste ninguno hasta ahí, es primo.
- El bucle de búsqueda tiene que cortarse apenas encuentra un divisor.

Pista (leela solo si te trabás): probá dividir desde 2 en adelante. Si un número tiene un divisor más grande que su raíz cuadrada, también tiene uno más chico, así que no hace falta seguir más allá.

Bonus (opcional)

- Contá cuántas divisiones hizo tu programa para cada número y compará con probar todos los divisores hasta el número.
- Mostrá la descomposición completa en factores primos de los que no son primos ($1001 = 7 \times 11 \times 13$).

Cierre: Es la misma prueba básica que está en el corazón de muchos algoritmos de criptografía, aunque ahí los números tienen cientos de dígitos.

Ejercicio 38: Del sueldo bruto al neto

Nivel 1: Junior I · Fundamentos · Temática: Recursos humanos · Dificultad 8/100

Situación: El estudio contable que liquida los sueldos de una pyme textil recibe siempre la misma consulta: "¿por qué cobro menos de lo que dice mi sueldo?". Quieren imprimir un detalle claro que explique cómo se pasa del básico al neto.

Datos de entrada

Empleado	Sueldo básico	Faltas en el mes	Afiliado al sindicato
Ana	\$1.150.000	0	Sí
Marcos	\$980.000	2	No

- Presentismo: 8,33 % del básico, solo si no tuvo faltas.
- Bruto = básico + presentismo.
- Descuentos sobre el bruto: jubilación 11 %, obra social 3 %, PAMI 3 % y, solo si está afiliado, sindicato 2 %.

Tu programa debe

1. Calcular el presentismo (si corresponde) y el sueldo bruto.
2. Calcular cada descuento sobre el bruto.
3. Calcular el neto (bruto – descuentos).
4. Mostrar el detalle completo de cada empleado.

Salida esperada

```
Ana
  Básico: $1150000.00
  Presentismo: $95795.00
  Bruto: $1245795.00
  Jubilación (11%): -$137037.45
  Obra social (3%): -$37373.85
  PAMI (3%): -$37373.85
  Sindicato (2%): -$24915.90
  Neto: $1009093.95
Marcos
  Básico: $980000.00
  Presentismo: $0.00
  Bruto: $980000.00
  Jubilación (11%): -$107800.00
  Obra social (3%): -$29400.00
  PAMI (3%): -$29400.00
  Neto: $813400.00
```

Reglas

- Alcanza con porcentajes, acumuladores y condicionales.
- Cada importe se redondea a 2 decimales (0,5 hacia arriba) antes de sumarlo o restarlo. Los decimales se muestran con punto y sin separador de miles en la salida.
- El descuento del sindicato solo aparece en el detalle si está afiliado.
- Los porcentajes en el detalle se muestran sin decimales.

Pista (leela solo si te trabás): calculá primero el bruto y recién después los descuentos: todos se aplican sobre el bruto, no sobre el básico ni en cascada.

Bonus (opcional)

- Agregá horas extra al 50 % que suman al bruto.
- Agregá un tope a la base de cálculo de la jubilación (por ejemplo, \$1.800.000).

Cierre: Es la estructura básica del recibo de sueldo que genera cualquier sistema de liquidación.

Ejercicio 39: Recargo por pago atrasado

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 8/100

Situación: Una cooperativa de servicios cobra recargos a quienes pagan la factura después del vencimiento. El reglamento tiene interés diario, un cargo fijo si el atraso es largo y un tope para que la deuda no se vuelva impagable. El área de cobranzas quiere automatizar el cálculo.

Datos de entrada

Regla	Valor
Monto de la factura	\$85.000
Interés punitorio	0,15 % diario simple sobre el monto
Cargo fijo	\$1.500 si el atraso es de más de 5 días
Tope	el recargo total (interés + cargo fijo) no puede superar el 20 % del monto

Cliente	Días de atraso
Andrea	0
Benjamín	4
Celeste	15
Darío	160

Tu programa debe

1. Calcular el interés punitorio según los días de atraso.
2. Sumar el cargo fijo si corresponde.
3. Aplicar el tope y marcarlo si se usó.
4. Mostrar el recargo y el total a pagar de cada cliente.

Salida esperada

```
Andrea: 0 días de atraso -> recargo $0.00 -> paga $85000.00
Benjamín: 4 días de atraso -> recargo $510.00 -> paga $85510.00
Celeste: 15 días de atraso -> recargo $3412.50 -> paga $88412.50
Darío: 160 días de atraso -> recargo $17000.00 (tope 20%) -> paga $102000.00
```

Reglas

- Alcanza con porcentajes, condicionales y comparaciones.
- Montos con 2 decimales, redondeo hacia arriba desde 0,5. Los decimales se muestran con punto y sin separador de miles en la salida.
- Con 0 días de atraso el recargo es \$0.00.
- Exactamente 5 días de atraso NO genera cargo fijo.

Pista (leela solo si te trabás): calculá el recargo "sin tope" completo y recién al final comparalo contra el 20 %. Si aplicás el tope antes de sumar el cargo fijo, te va a dar mal.

Bonus (opcional)

- Hacé que el interés sea compuesto (cada día sobre el monto más los intereses acumulados) y compará.
- Calculá a partir de cuántos días de atraso se alcanza el tope.

Cierre: Es la misma lógica de punitorios con tope que aplican las empresas de servicios y las tarjetas de crédito.

Ejercicio 40: El plan de datos que conviene

Nivel 1: Junior I · Fundamentos · Temática: Telecomunicaciones · Dificultad 8/100

Situación: Una compañía de telefonía tiene tres planes y los clientes eligen casi siempre mal: pagan de más por un plan grande o se comen excedentes carísimos. El área de atención quiere una herramienta que, dado el consumo típico de un cliente, le recomiende el plan más barato.

Datos de entrada

Plan	Precio mensual	GB incluidos	Excedente
Chico	\$9.000	5	\$1.200 por GB o fracción
Mediano	\$14.500	12	\$900 por GB o fracción
Libre	\$21.000	ilimitado	—

Consumos típicos a evaluar: 4 GB, 9,5 GB y 20 GB

Tu programa debe

1. Para cada consumo, calcular cuánto pagaría con cada plan (precio + excedentes).
2. Elegir el plan más barato.
3. Mostrar el costo de los tres planes y la recomendación.

Salida esperada

```
Consumo 4.0 GB -> Chico: $9000 | Mediano: $14500 | Libre: $21000 -> conviene Chico
Consumo 9.5 GB -> Chico: $15000 | Mediano: $14500 | Libre: $21000 -> conviene Mediano
Consumo 20.0 GB -> Chico: $27000 | Mediano: $21700 | Libre: $21000 -> conviene Libre
```

Reglas

- Alcanza con condicionales, redondeo hacia arriba y comparaciones.
- El excedente se cobra por GB o fracción: 4,5 GB de exceso se cobran como 5 GB.
- Si dos planes empatan en precio, se recomienda el que aparece primero en la tabla.
- Los consumos se muestran con 1 decimal. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): calculá los tres costos con la misma lógica y recién después buscá el mínimo. El plan ilimitado es simplemente un plan sin excedente.

Bonus (opcional)

- Calculá los puntos de corte: a partir de cuántos GB conviene pasar del plan Chico al Mediano y del Mediano al Libre.
- Agregá un descuento del 20 % en el plan Libre para clientes con más de 2 años de antigüedad.

Cierre: Es la misma comparación que hacen los simuladores de planes en los sitios de las compañías telefónicas.

Ejercicio 41: Dosis según el peso del paciente

Nivel 1: Junior I · Fundamentos · Temática: Salud · Dificultad 8/100

Situación: En la guardia pediátrica de un hospital, las enfermeras calculan a mano la dosis de un jarabe según el peso de cada chico, y un error puede ser grave. Quieren un programa que haga la cuenta, respete la dosis máxima y avise cuando el peso está fuera del rango seguro. (Medicamento y valores ficticios, con fines didácticos.)

Datos de entrada

Parámetro	Valor
Dosis	12 mg por kg de peso
Dosis máxima por toma	600 mg
Concentración del jarabe	40 mg por ml
Rango de peso válido	de 3 kg a 60 kg, inclusive

Paciente	Peso
Paciente 1	8,5 kg
Paciente 2	22 kg
Paciente 3	55 kg
Paciente 4	2,1 kg

Tu programa debe

1. Validar que el peso esté dentro del rango; si no, avisar y no calcular.
2. Calcular la dosis en mg y aplicar la dosis máxima si corresponde.
3. Convertir la dosis a ml, redondeando hacia abajo a 1 decimal.
4. Mostrar mg, ml y si se aplicó la dosis máxima.

Salida esperada

Paciente 1 (8.5 kg): 102.0 mg = 2.5 ml
 Paciente 2 (22.0 kg): 264.0 mg = 6.6 ml
 Paciente 3 (55.0 kg): 600.0 mg = 15.0 ml (dosis máxima)
 Paciente 4 (2.1 kg): fuera de rango - consultar al médico

Reglas

- Alcanza con fórmulas, condicionales y redondeo.
- Los ml se redondean SIEMPRE hacia abajo a 1 decimal (2,55 ml → 2,5 ml): nunca se da de más.
- Pesos y mg se muestran con 1 decimal. Los decimales se muestran con punto y sin separador de miles en la salida.
- Los límites del rango son válidos: 3 kg y 60 kg se calculan normalmente.

Pista (leela solo si te trabás): para redondear hacia abajo a 1 decimal: multiplicá por 10, quedate con la parte entera y dividí por 10.

Bonus (opcional)

- Calculá la dosis diaria total si se toma cada 6 horas y avisá si supera un máximo diario de 2.400 mg.
- Mostrá la dosis en gotas, sabiendo que 1 ml equivale a 20 gotas.

Cierre: Es el mismo tipo de cálculo que hacen los sistemas de prescripción electrónica para reducir errores de medicación.

Ejercicio 42: Pedidos que entran en la camioneta

Nivel 1: Junior I · Fundamentos · Temática: Logística y envíos · Dificultad 9/100

Situación: Un corralón despacha pedidos con una sola camioneta que soporta 800 kg. Los pedidos se cargan estrictamente en el orden en que llegaron, porque los clientes tienen turno. Cuando un pedido no entra, la camioneta sale y ese pedido encabeza el viaje siguiente.

Datos de entrada

Capacidad de la camioneta: 800 kg

Pedido	1	2	3	4	5	6	7
Peso (kg)	120	250	90	310	60	45	200

Tu programa debe

1. Cargar pedidos en orden mientras entren en la camioneta.
2. Cuando el siguiente pedido no entra, cerrar el viaje y empezar uno nuevo.
3. Mostrar, por viaje, qué pedidos lleva, el peso total y la capacidad libre.
4. Mostrar la cantidad de viajes necesarios.

Salida esperada

Viaje 1: pedidos 1, 2, 3, 4 | 770 kg | libre 30 kg
 Viaje 2: pedidos 5, 6, 7 | 305 kg | libre 495 kg
 Viajes necesarios: 2

Reglas

- Alcanza con bucles con condición de corte y acumuladores.
- Los pedidos no se reordenan: aunque un pedido más chico entraría, se respeta el turno.
- Llenar exactamente 800 kg está permitido.
- Todos los pedidos pesan menos que la capacidad (si alguno pesara más, el programa entraría en un bucle infinito: ver Bonus).

Pista (leela solo si te trabás): usá un bucle para los viajes y, adentro, otro bucle que siga cargando "mientras quede algún pedido y el siguiente entre".

Bonus (opcional)

- Detectá pedidos que pesan más que la capacidad y marcalos como "requiere camión" en lugar de colgarse.
- Compará cuántos viajes harían falta si sí se pudiera saltar pedidos para aprovechar el espacio.

Cierre: Es la versión más simple del problema de carga de vehículos que resuelven los sistemas de ruteo de las empresas de logística.

Ejercicio 43: Likes que se duplican cada hora

Nivel 1: Junior I · Fundamentos · Temática: Redes sociales · Dificultad 9/100

Situación: Un video de un perro que "canta" empieza a volverse viral. El community manager de la marca de alimento que aparece en el video quiere estimar a qué hora va a superar el millón de likes para lanzar una promo justo en ese momento.

Datos de entrada

Dato	Valor
Likes a las 12:00	150
Crecimiento mientras tenga hasta 100.000 likes	se duplican cada hora
Crecimiento cuando ya pasó los 100.000 likes	30 % por hora (se satura)
Objetivo	superar 1.000.000 de likes

Tu programa debe

1. Simular hora por hora el crecimiento de los likes según la regla vigente.
2. Mostrar la hora del reloj y los likes al final de cada hora.
3. Detenerse al superar el millón e informar cuántas horas pasaron.

Salida esperada

```
12:00 - 150 likes
13:00 - 300 likes
14:00 - 600 likes
15:00 - 1200 likes
16:00 - 2400 likes
17:00 - 4800 likes
18:00 - 9600 likes
19:00 - 19200 likes
20:00 - 38400 likes
21:00 - 76800 likes
22:00 - 153600 likes
23:00 - 199680 likes
00:00 - 259584 likes
01:00 - 337459 likes
02:00 - 438696 likes
03:00 - 570304 likes
04:00 - 741395 likes
05:00 - 963813 likes
06:00 - 1252956 likes
Superó el millón en 18 horas
```

Reglas

- Alcanza con un bucle con condición, multiplicaciones y condicionales.
- La regla se decide con los likes al comienzo de cada hora: si al empezar la hora tiene más de 100.000, crece 30 %.
- Con el 30 %, los likes se redondean hacia abajo al entero.
- La hora del reloj vuelve a 00 después de las 23.

Pista (leela solo si te trabás): el crecimiento exponencial explota rápido: en pocas vueltas pasás de cientos a cientos de miles. Revisá que el cambio de regla ocurra en la hora correcta.

Bonus (opcional)

- Calculá cuántas horas habría tardado si se duplicara siempre (sin saturación).
- Hacé que la saturación sea gradual: 30 % desde 100.000, 15 % desde 500.000.

Cierre: Es el mismo modelo de crecimiento con saturación que se usa para estimar la viralización de contenidos (y también la propagación de epidemias).

Ejercicio 44: El dígito que delata una tarjeta mal tipeada

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 9/100

Situación: En el checkout de una tienda online, muchos pagos se rechazan porque la gente tipea mal el número de tarjeta y recién se entera después de esperar la respuesta del procesador. El equipo quiere detectar el error al instante, antes de enviar nada, con el dígito verificador que todas las tarjetas traen.

Datos de entrada

Números a verificar (son números de prueba públicos, no tarjetas reales):

N.º	Número
1	4111 1111 1111 1111
2	4111 1111 1111 1121
3	5555 5555 5555 4444
4	5555 5555 5554 5444

Algoritmo de verificación:

1. Recorrer los dígitos de derecha a izquierda.
2. Duplicar uno sí y uno no, empezando por el segundo desde la derecha. Si al duplicar da más de 9, restarle 9.
3. Sumar todos los dígitos (los duplicados y los que no).
4. El número es válido si la suma es múltiplo de 10.

Tu programa debe

1. Extraer los dígitos de derecha a izquierda.
2. Aplicar la duplicación alternada y calcular la suma.
3. Mostrar el número en grupos de 4, la suma y si es válido o inválido.

Salida esperada

```
4111 1111 1111 1111: suma 30 -> válida
4111 1111 1111 1121: suma 32 -> INVÁLIDA (revisá lo tipeado)
5555 5555 5555 4444: suma 60 -> válida
5555 5555 5554 5444: suma 52 -> INVÁLIDA (revisá lo tipeado)
```

Reglas

- Alcanza con un bucle, división entera, resto y un contador de posición.
- Podés tratar el número como un entero grande (16 dígitos entran en un entero de 64 bits) o como texto; con la división entera y el resto no necesitás recorrer texto.
- La posición se cuenta desde la derecha empezando en 0: se duplican las posiciones impares.

Pista (leela solo si te trabás): $n \% 10$ te da el último dígito y n división entera 10 lo "saca". Repetí mientras n sea mayor que 0 y llevá un contador para saber si te toca duplicar.

Bonus (opcional)

- Dado un número de 15 dígitos, calculá qué dígito verificador hay que agregarle al final para que sea válido.
- Detectá la marca por los primeros dígitos: empieza con 4 → Visa; empieza con 51 a 55 → Mastercard.

Cierre: Es el algoritmo de Luhn, el mismo que validan los formularios de pago de cualquier tienda online antes de enviar la tarjeta.

Ejercicio 45: Cuánto debo después de cada cuota

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 9/100

Situación: Germán sacó un préstamo de \$600.000 en una financiera que cobra 5 % mensual sobre el saldo adeudado. Él paga \$150.000 por mes y quiere saber cuántos meses le lleva cancelarlo y cuánto termina pagando en intereses, porque sospecha que es mucho.

Datos de entrada

Dato	Valor
Capital	\$600.000
Interés	5 % mensual sobre el saldo
Pago mensual	\$150.000 (el último puede ser menor)

Tu programa debe

1. Cada mes, sumar al saldo el interés del 5 %.
2. Descontar el pago del mes (o lo que falte, si es menos que la cuota).
3. Mostrar interés, pago y saldo de cada mes hasta cancelar.
4. Mostrar el total pagado y el total de intereses.

Salida esperada

```
Mes 1: interés $30000.00 | paga $150000.00 | debe $480000.00
Mes 2: interés $24000.00 | paga $150000.00 | debe $354000.00
Mes 3: interés $17700.00 | paga $150000.00 | debe $221700.00
Mes 4: interés $11085.00 | paga $150000.00 | debe $82785.00
Mes 5: interés $4139.25 | paga $86924.25 | debe $0.00
Total pagado: $686924.25
Total de intereses: $86924.25
```

Reglas

- Alcanza con un bucle "mientras", acumuladores y condicionales.
- El interés de cada mes se redondea a 2 decimales (0,5 hacia arriba) antes de sumarlo. Los decimales se muestran con punto y sin separador de miles en la salida.
- El último pago es exactamente lo que falta: el saldo final tiene que ser 0.00, nunca negativo.
- Si la cuota no alcanza a cubrir el interés del mes, la deuda nunca se cancela: detectalo y cortá con un mensaje (ver Bonus).

Pista (leela solo si te trabás): el orden dentro del mes importa: primero se suma el interés sobre el saldo que había, después se paga.

Bonus (opcional)

- Detectá el caso de deuda impagable (cuota menor o igual al interés del primer mes) y mostrá un aviso.
- Calculá cuánto se ahorraría Germán pagando \$200.000 por mes.

Cierre: Es la misma simulación de amortización que hacen los bancos para armar el cronograma de pagos de un préstamo.

Ejercicio 46: La cantina escolar y sus tickets

Nivel 1: Junior I · Fundamentos · Temática: Educación · Dificultad 9/100

Situación: La cantina de la escuela anota cada venta del recreo como un código de producto en una planilla. Al final del día, la cooperadora quiere saber cuánto se vendió de cada cosa, cuánto se recaudó y si hubo códigos mal cargados.

Datos de entrada

Código	Producto	Precio
1	Alfajor	\$900
2	Jugo	\$700
3	Sándwich	\$1.800
4	Fruta	\$400

Códigos cargados en el recreo, en orden: 1, 3, 2, 1, 4, 4, 3, 1, 2, 2, 1, 5, 3, 4, 1

Tu programa debe

1. Contar cuántas unidades se vendieron de cada producto.
2. Detectar y contar los códigos inválidos.
3. Calcular la recaudación por producto y total.
4. Informar el producto más vendido.

Salida esperada

```
Alfajor: 5 vendidos -> $4500
Jugo: 3 vendidos -> $2100
Sándwich: 3 vendidos -> $5400
Fruta: 3 vendidos -> $1200
Total recaudado: $13200
Tickets inválidos: 1
Más vendido: Alfajor
```

Reglas

- Alcanza con un bucle, un contador por producto y condicionales.
- Los códigos inválidos no suman dinero.
- Si hay empate en el más vendido, se informa el de menor código.
- Los productos se muestran en orden de código.

Pista (leela solo si te trabás): con cuatro productos alcanza con cuatro contadores y un condicional por código. En el nivel siguiente vas a ver cómo hacerlo con estructuras que escalan a cien productos.

Bonus (opcional)

- Calculá qué porcentaje de la recaudación aporta cada producto.
- Agregá un combo: cada alfajor + jugo vendidos en tickets consecutivos cuesta \$1.400 en lugar de \$1.600.

Cierre: Es el mismo conteo por categoría que hace cualquier sistema de punto de venta al cerrar la caja del día.

Ejercicio 47: Rendimiento de la cosecha por hectárea

Nivel 1: Junior I · Fundamentos · Temática: Agricultura · Dificultad 9/100

Situación: Un productor de soja del sur de Córdoba cosechó cuatro lotes de distinto tamaño. El agrónomo le pide el rendimiento promedio del campo, pero el productor promedió los rendimientos de cada lote y le dio un número que no cierra. Tu programa calcula ambos y muestra la diferencia.

Datos de entrada

Lote	Hectáreas	Toneladas cosechadas
Lote 1	45	162
Lote 2	30	96
Lote 3	60	234
Lote 4	25	70

Precio de la tonelada: \$210.000

Tu programa debe

1. Calcular el rendimiento (t/ha) de cada lote.
2. Calcular el rendimiento promedio ponderado (toneladas totales / hectáreas totales) y marcar los lotes que quedan debajo.
3. Calcular también el promedio simple de los rendimientos, para comparar.
4. Mostrar la producción total y el ingreso estimado.

Salida esperada

```
Lote 1: 3.60 t/ha
Lote 2: 3.20 t/ha - debajo del promedio
Lote 3: 3.90 t/ha
Lote 4: 2.80 t/ha - debajo del promedio
Rendimiento promedio ponderado: 3.51 t/ha
Promedio simple de los lotes: 3.38 t/ha
Producción total: 562 t | Ingreso estimado: $118020000
```

Reglas

- Alcanza con acumuladores, divisiones y condicionales (vas a necesitar recorrer los datos dos veces).
- Rendimientos con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- La marca "debajo del promedio" se decide contra el promedio ponderado, sin redondear.

Pista (leela solo si te trabás): el promedio simple trata igual a un lote de 25 ha y a uno de 60 ha; el ponderado no. Por eso el correcto para el campo entero es el ponderado.

Bonus (opcional)

- Calculá cuántas toneladas más se habrían cosechado si todos los lotes rindieran como el mejor.
- Descontá un costo de flete de \$18.000 por tonelada y mostrá el ingreso neto.

Cierre: Es el mismo cálculo que hacen los monitores de rendimiento de las cosechadoras y los sistemas de gestión agrícola.

Ejercicio 48: Desafío de nivel: La caja registradora del almacén

Nivel 1: Junior I · Fundamentos · Temática: E-commerce · Dificultad 10/100

Situación: El almacén de Doña Rosa todavía cobra con calculadora y quiere pasar a una caja registradora propia. Necesita que el programa arme el ticket, aplique el mejor descuento disponible (no se acumulan) y le diga qué billetes y monedas dar de vuelto. Este es el desafío que integra todo el nivel.

Datos de entrada

Producto	Cantidad	Precio unitario
Yerba 1 kg	2	\$4.300
Azúcar 1 kg	1	\$1.250
Fideos 500 g	3	\$980
Leche 1 L	4	\$1.150
Galletitas	2	\$1.890

- Descuentos (no acumulables, se aplica el mayor): 10 % en compras de \$20.000 o más; 15 % para jubilados los miércoles.
- Cliente: jubilada, hoy es miércoles.
- Paga con: \$20.000
- Billetes y monedas disponibles: \$10.000, \$2.000, \$1.000, \$500, \$200, \$100, \$50, \$10, \$5, \$1

Tu programa debe

1. Mostrar cada línea del ticket (cantidad × precio = importe) y el subtotal.
2. Calcular los descuentos que aplican y quedarse con el mayor.
3. Mostrar el descuento aplicado y el total.
4. Calcular el vuelto y desglosarlo en la menor cantidad de billetes y monedas.

Salida esperada

```

Yerba 1 kg      2 x $4300 = $8600
Azúcar 1 kg     1 x $1250 = $1250
Fideos 500 g    3 x $980 = $2940
Leche 1 L       4 x $1150 = $4600
Galletitas      2 x $1890 = $3780
Subtotal: $21170
Descuento (jubilados miércoles 15%): -$3176
TOTAL: $17994
Paga con: $20000
Vuelto: $2006
  1 x $2000
  1 x $5
  1 x $1
  
```

Reglas

- Alcanza con bucles, acumuladores, condicionales, división entera y resto: todo lo del nivel 1.
- El descuento se redondea al peso más cercano (0,5 hacia arriba).
- El nombre del producto ocupa 14 caracteres alineados a la izquierda en el ticket (se completa con espacios).
- Si ningún descuento aplica, se muestra "(sin descuento): -\$0".
- Si el pago no alcanza, en lugar de vuelto se informa cuánto falta.

Pista (leela solo si te trabás): resuelvelo en tres bloques independientes y probá cada uno por separado: ticket y subtotal, descuento, vuelto. El bloque del vuelto es el mismo del ejercicio "El vuelto con billetes y monedas" (si lo hiciste, reutilizá la idea).

Bonus (opcional)

- Permití pagar una parte con tarjeta y el resto en efectivo.
- Guardá el total de cada venta y, al final del día, mostrá la recaudación y el ticket promedio.

Cierre: Es, en miniatura, lo que hace el software de cualquier caja registradora de supermercado.

Ejercicio 49: Desafío de nivel: El cajero automático que no da monedas

Nivel 1: Junior I · Fundamentos · Temática: Finanzas y FinTech · Dificultad 10/100

Situación: Un cajero automático de un pueblo solo tiene billetes de \$10.000, \$2.000 y \$1.000, y en cantidades limitadas hasta que llegue el camión de caudales. El banco quiere simular un día de extracciones de una cuenta para verificar que el cajero nunca entregue mal ni se quede "a mitad de camino".

Datos de entrada

Billete	Cantidad en el cajero
\$10.000	5
\$2.000	20
\$1.000	10

- Saldo inicial de la cuenta: \$150.000
- Máximo por extracción: \$80.000
- Pedidos de extracción, en orden: \$27.000, \$1.500, \$94.000, \$53.000, \$45.000, \$14.000, \$80.000

Tu programa debe

1. Validar cada pedido en este orden: múltiplo de \$1.000, máximo por extracción, saldo suficiente.
2. Armar la entrega empezando por el billete más grande, sin usar más billetes de los que hay.
3. Si no se puede armar el monto exacto, rechazar sin tocar el stock ni el saldo.
4. Si se puede, actualizar stock y saldo y mostrar el detalle; al final, mostrar los billetes que quedan.

Salida esperada

```
Extracción 1 ($27000): entrega 2 x $10000, 3 x $2000, 1 x $1000 | saldo $123000
Extracción 2 ($1500): rechazada - el monto debe ser múltiplo de $1000
Extracción 3 ($94000): rechazada - supera el máximo por extracción ($80000)
Extracción 4 ($53000): entrega 3 x $10000, 11 x $2000, 1 x $1000 | saldo $70000
Extracción 5 ($45000): rechazada - el cajero no tiene billetes para ese monto
Extracción 6 ($14000): entrega 6 x $2000, 2 x $1000 | saldo $56000
Extracción 7 ($80000): rechazada - saldo insuficiente (saldo $56000)
Billetes que quedan: $10000: 0, $2000: 0, $1000: 6
```

Reglas

- Alcanza con condicionales, división entera, bucles y variables de estado: es el desafío integrador del nivel 1.
- Una extracción rechazada no modifica nada (ni saldo ni billetes).
- Se informa solo el primer motivo de rechazo, según el orden de validación.
- En el detalle de la entrega se muestran solo los billetes usados.

Pista (leela solo si te trabás): calculá primero la entrega en variables auxiliares (cuántos de cada billete usarías) y recién si el monto cierra exacto, restá del stock. Ese "probar antes de confirmar" es la clave.

Bonus (opcional)

- Agregá un límite diario de \$100.000 por cuenta.
- Si no puede entregar el monto exacto, sugerí el monto más cercano menor que sí puede entregar.

Cierre: Es la misma lógica de validación y dispensado que ejecuta cada cajero automático, donde entregar un billete de más es un error carísimo.

Ejercicio 50: Desafío de nivel: La liga de fútbol 5 de los jueves

Nivel 1: Junior I · Fundamentos · Temática: Deportes · Dificultad 10/100

Situación: Cuatro equipos de amigos juegan una liga de fútbol 5 todos los jueves y ya terminaron el todos contra todos. Hubo discusiones por quién salió primero, porque dos equipos creen que ganaron. Tu programa arma la tabla oficial con los criterios de desempate que acordaron.

Datos de entrada

Partido	Local	Goles	Goles	Visitante
1	Halcones	3	1	Leones
2	Toros	2	2	Pumas
3	Halcones	0	2	Toros
4	Leones	4	1	Pumas
5	Halcones	2	2	Pumas
6	Leones	1	0	Toros

- Puntos: 3 por victoria, 1 por empate, 0 por derrota.
- Líder: más puntos; si empatan, mejor diferencia de gol; si siguen empatados, más goles a favor; si todo empata, el que aparece primero en la lista de equipos.
- Orden de los equipos en la tabla: Halcones, Leones, Toros, Pumas (el orden de inscripción, no se ordena).

Tu programa debe

1. Procesar cada partido y actualizar ganados, empatados, perdidos, goles a favor, goles en contra y puntos de los dos equipos.
2. Mostrar la tabla con diferencia de gol (con signo) y puntos, en el orden de inscripción.
3. Determinar el líder aplicando los criterios de desempate.
4. Informar la valla menos vencida (menos goles recibidos; si empatan, el primero de la lista).

Salida esperada

```

Equipo  G E P GF GC DIF PTS
Halcones 1 1 1 5 5 +0 4
Leones   2 0 1 6 4 +2 6
Toros    1 1 1 4 3 +1 4
Pumas    0 2 1 5 8 -3 2
Líder: Leones
Valla menos vencida: Toros (3 goles recibidos)

```

Reglas

- Alcanza con variables por equipo, bucles y condicionales: es el desafío integrador del nivel 1 (las listas llegan en el nivel 2).
- Formato de la tabla: nombre alineado a la izquierda en 9 caracteres; GF y GC en 2; DIF en 3 con signo (+2, -3, +0); PTS en 3.
- Cada partido afecta a los dos equipos: no te olvides del visitante.

Pista (leela solo si te trabás): escribí la actualización de estadísticas pensando en "un equipo con sus goles a favor y en contra" y aplícala dos veces por partido, una desde el punto de vista de cada equipo.

Bonus (opcional)

- Mostrá la tabla ordenada por posición (cuando llegues al nivel 3, vas a poder hacerlo con un ordenamiento).
- Agregá una segunda rueda y la regla de que en caso de empate total se mira el resultado entre ellos.

Cierre: Es la misma lógica que calculan las apps y sitios de resultados deportivos para armar las tablas de posiciones.

Lote 03: ejercicios 51 a 75

Nivel 2

Junior II · Colecciones y funciones

Ejercicios 51 a 100

Qué vas a practicar

Listas, diccionarios, funciones y recorridos sobre colecciones.

Qué se espera al terminar

Organizar el código en funciones y elegir la colección adecuada.

Conceptos del nivel

listas, máximo, mínimo, promedio, filtrado, acumuladores, recorrido de texto, comparación de extremos, contadores, búsqueda lineal, saldos, emparejamiento de deudores y acreedores, matrices, recorrido, rangos, separar palabras, funciones, parámetros, valor de retorno, filas, columnas y diagonales, sumas por fila y columna, listas paralelas, clasificación de caracteres, desplazamiento de caracteres, saldo acumulado, conversión de tiempo, vecinos de una celda, inversión, listas de registros, transposición, porcentajes, sumas por rango, distancia absoluta, búsqueda, actualización, dígitos verificadores, texto, contadores por fila y columna, aritmética circular....

Desafíos de nivel

- 98. El resumen de la tarjeta de crédito
- 99. Batalla naval en la terminal
- 100. Boletín de calificaciones del curso

Ejercicio 51: Notas del curso: la mejor y la peor

Nivel 2: Junior II · Colecciones y funciones · Temática: Educación · Dificultad 11/100

Situación: El profesor de Historia de un secundario tiene las notas finales en dos listas: una con los nombres y otra con las notas. Para la reunión de profesores quiere un resumen rápido con el promedio del curso, quién sacó la mejor y la peor nota, y quiénes quedaron por encima del promedio.

Datos de entrada

Estudiante	Nota
Ana	8
Bruno	5
Camila	9
Diego	7
Elena	10
Franco	4
Gala	6
Hernán	8
Inés	3
Joaquín	7
Karen	9
Lucas	6

Tu programa debe

1. Guardar los nombres y las notas en listas.
2. Calcular el promedio del curso.
3. Encontrar la mejor y la peor nota, con el nombre de quien la sacó.
4. Listar, en el orden original, a quienes están estrictamente por encima del promedio.

Salida esperada

```
Promedio del curso: 6.83
Mejor nota: Elena (10)
Peor nota: Inés (3)
Por encima del promedio (7): Ana, Camila, Diego, Elena, Hernán, Joaquín, Karen
```

Reglas

- Alcanza con listas, bucles, acumuladores y la búsqueda de máximo y mínimo.
- Implementá vos la búsqueda del máximo y del mínimo (sin operaciones ya hechas que lo resuelvan).
- Si hay empate en la mejor o la peor nota, se informa quien aparece primero.
- El promedio se muestra con 2 decimales (0,5 hacia arriba), pero la comparación se hace con el valor sin redondear. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): recordá la posición del máximo, no solo su valor: con la posición recuperarás el nombre desde la otra lista.

Bonus (opcional)

- Mostrá la distribución de notas: cuántos 1, cuántos 2... hasta 10, usando una lista de 10 contadores.
- Calculá la mediana del curso.

Cierre: Es el mismo resumen que generan las plataformas educativas al cerrar un trimestre.

Ejercicio 52: Días del mes por encima del promedio

Nivel 2: Junior II · Colecciones y funciones · Temática: Clima y energía · Dificultad 11/100

Situación: Una cooperativa eléctrica sabe que los días de calor disparan el consumo por los aires acondicionados. Para planificar la compra de energía del próximo verano, quiere saber cuántos días de noviembre la temperatura máxima superó el promedio del mes y cuál fue el más caluroso.

Datos de entrada

Temperaturas máximas de noviembre (°C), del día 1 al 30:

Días	Temperaturas
1-10	24,1 26,3 27,8 25,0 22,4 21,9 23,5 28,7 30,2 31,4
11-20	29,8 26,1 24,6 23,3 22,0 20,8 21,5 25,9 27,4 29,1
21-30	32,0 33,6 31,2 27,7 24,4 23,0 22,6 24,9 26,5 28,3

Tu programa debe

1. Guardar las 30 temperaturas en una lista.
2. Calcular el promedio de las máximas.
3. Listar los días (número de día) cuya máxima supera el promedio y contarlos.
4. Informar el día más caluroso.

Salida esperada

```
Promedio de máximas: 26.20 °C
Días por encima del promedio (14): 2 3 8 9 10 11 19 20 21 22 23 24 29 30
Día más caluroso: 22 (33.6 °C)
```

Reglas

- Alcanza con listas, un acumulador, un filtrado y un máximo.
- El promedio se muestra con 2 decimales y las temperaturas con 1 (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- "Superar" es estrictamente mayor. Si dos días empatan como más calurosos, se informa el primero.
- El número de día es la posición en la lista + 1.

Pista (leela solo si te trabás): necesitás dos recorridos: uno para calcular el promedio y otro para comparar cada día contra ese promedio. No se puede hacer bien en uno solo, porque el promedio recién se conoce al final.

Bonus (opcional)

- Encontrá la racha más larga de días consecutivos por encima del promedio.
- Calculá la desviación de cada día respecto del promedio y mostrá los 3 días más atípicos.

Cierre: Es el tipo de análisis que hacen las empresas de energía para anticipar picos de demanda.

Ejercicio 53: Gastos del mes día por día

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 11/100

Situación: Sabrina exportó de su billetera virtual los gastos de cada día de septiembre. Se había propuesto no gastar más de \$450.000 en el mes y quiere saber si lo cumplió, qué día gastó más y en qué día exacto se pasó del presupuesto (si es que se pasó).

Datos de entrada

Gasto de cada día de septiembre, del 1 al 30 (en pesos):

Días	Gastos
1-10	12.500 · 0 · 8.400 · 32.000 · 4.100 · 0 · 55.000 · 7.300 · 2.600 · 0
11-20	18.900 · 9.400 · 0 · 64.000 · 3.800 · 0 · 11.200 · 27.500 · 5.600 · 0
21-30	42.000 · 8.800 · 0 · 15.300 · 96.000 · 6.100 · 0 · 22.400 · 4.700 · 38.000

Presupuesto mensual: \$450.000

Tu programa debe

1. Calcular el total gastado y el promedio diario.
2. Encontrar el día de mayor gasto.
3. Contar los días sin gastos.
4. Recorrer los días acumulando el gasto e informar el primer día en que el acumulado supera el presupuesto.

Salida esperada

```
Total del mes: $495600
Promedio diario: $16520.00
Día de mayor gasto: 25 ($96000)
Días sin gastos: 8
Superó el presupuesto de $450000 el día 28
```

Reglas

- Alcanza con listas, acumuladores, contadores y un máximo.
- El promedio se muestra con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Superar es estrictamente mayor: llegar a exactamente \$450.000 no es pasarse.
- Si nunca lo supera, se informa "No superó el presupuesto de \$450000".

Pista (leela solo si te trabás): el "día en que se pasó" se detecta con un acumulado que avanza día por día, y se registra solo la primera vez que cruza el límite.

Bonus (opcional)

- Calculá el gasto promedio de los días hábiles y de los fines de semana, sabiendo que el 1 de septiembre fue martes.
- Mostrá cuánto le quedaba de presupuesto al final de cada semana.

Cierre: Es el mismo análisis que muestran las apps de finanzas personales en sus gráficos de gasto mensual.

Ejercicio 54: Patentes capicúa

Nivel 2: Junior II · Colecciones y funciones · Temática: Transporte · Dificultad 12/100

Situación: Un grupo de fanáticos de las patentes capicúa organiza un concurso de fotos en una red social. Llegan cientos de fotos y algunas patentes están mal cargadas, con minúsculas o con espacios de más. Te piden un verificador que no se deje engañar por el formato.

Datos de entrada

Patentes cargadas (tal cual las escribió la gente):

- AB 121 BA
- AC 454 CA
- AD 123 DA
- ae 797 EA
- XY 101 YZ
- AF 3 3 3 FA

Tu programa debe

1. Normalizar cada patente: quitar los espacios y pasar las letras a mayúsculas.
2. Verificar si la patente normalizada se lee igual al derecho y al revés, comparando sus extremos hacia el centro.
3. Mostrar la patente original, la normalizada y el resultado; al final, cuántas son capicúa.

Salida esperada

```
AB 121 BA -> AB121BA -> capicúa
AC 454 CA -> AC454CA -> capicúa
AD 123 DA -> AD123DA -> no es capicúa
ae 797 EA -> AE797EA -> capicúa
XY 101 YZ -> XY101YZ -> no es capicúa
AF 3 3 3 FA -> AF333FA -> capicúa
Patentes capicúa: 4
```

Reglas

- Alcanza con recorrido de texto, dos índices y condicionales.
- No vale invertir el texto con una operación ya hecha: comparalo carácter a carácter con un índice que avanza y otro que retrocede.
- La comparación se corta apenas encuentra una diferencia.
- Un texto de un solo carácter o vacío es capicúa.

Pista (leela solo si te trabás): usá un índice *i* que arranca al principio y otro *j* que arranca al final. Mientras $i < j$, compará y acercalos.

Bonus (opcional)

- Contá cuántas comparaciones hizo tu programa para cada patente.
- Para las que no son capicúa, decí cuál es el primer par de caracteres que no coincide.

Cierre: Es el mismo tipo de normalización y comparación que se usa para validar códigos, patentes y números de documento en sistemas de registro.

Ejercicio 55: Vocales y consonantes de la canción

Nivel 2: Junior II · Colecciones y funciones · Temática: Música · Dificultad 12/100

Situación: Una cantautora está estudiando la musicalidad de sus letras: dice que las frases con muchas vocales abiertas "cantan mejor". Quiere un programa que cuente vocales, consonantes y demás caracteres de cada verso que escribe, empezando por el primero de su nueva canción.

Datos de entrada

Verso (una sola línea):

```
Bajo la lluvia de junio, el tren se aleja sin mí; guardo tu nombre en el bolsillo, ¿quién sabe si  
vuelvo aquí?
```

Tu programa debe

1. Recorrer el texto carácter por carácter.
2. Clasificar cada carácter como vocal, consonante, espacio u otro signo.
3. Contar cada vocal por separado, sumando las acentuadas a su vocal base (á cuenta como a; ü cuenta como u).
4. Mostrar los totales, el conteo por vocal y la vocal más frecuente.

Salida esperada

```
Vocales: 41  
Consonantes: 43  
Espacios: 21  
Otros signos: 5  
Total de caracteres: 110  
Por vocal: a=8 e=11 i=8 o=7 u=7  
Vocal más frecuente: e
```

Reglas

- Alcanza con recorrido de texto, contadores y condicionales.
- Mayúsculas y minúsculas cuentan igual.
- La ñ es consonante. Los signos ¿ ? ; , son "otros signos".
- Si dos vocales empatan como más frecuentes, se informa la primera en el orden a, e, i, o, u.

Pista (leela solo si te trabás): definí un texto con todas las vocales posibles (incluidas las acentuadas) y preguntá si cada carácter pertenece a él. Lo que sea letra y no esté ahí, es consonante.

Bonus (opcional)

- Contá sílabas aproximadas: cada grupo de vocales seguidas cuenta como una.
- Encontrá la palabra con más vocales del verso.

Cierre: Es el mismo tipo de recorrido que usan los correctores ortográficos y las herramientas de análisis de texto.

Ejercicio 56: Stock bajo en el depósito

Nivel 2: Junior II · Colecciones y funciones · Temática: Logística y envíos · Dificultad 12/100

Situación: El depósito de una distribuidora de ferretería se queda sin productos clave porque nadie revisa a tiempo el stock mínimo. El jefe de depósito quiere un listado diario de lo que hay que reponer y, además, poder consultar un producto por código cuando lo llaman del mostrador.

Datos de entrada

Código	Producto	Stock	Stock mínimo
A-101	Tornillo 6 mm	1200	500
A-102	Tarugo 8 mm	180	400
B-204	Cinta aisladora	35	60
B-310	Pegamento	90	50
G-015	Guantes	12	40
G-022	Barbijo	300	300
D-400	Lija 120	55	100

Códigos a consultar: B-204, Z-999

Tu programa debe

1. Listar los productos con stock estrictamente menor que el mínimo.
2. Para cada uno, calcular cuánto pedir para llegar al doble del mínimo.
3. Mostrar cuántos productos hay que reponer.
4. Buscar cada código consultado recorriendo la lista y mostrar sus datos y su posición, o "no existe".

Salida esperada

```
Stock bajo:
A-102 Tarugo 8 mm: 180 (mínimo 400) -> pedir 620
B-204 Cinta aisladora: 35 (mínimo 60) -> pedir 85
G-015 Guantes: 12 (mínimo 40) -> pedir 68
D-400 Lija 120: 55 (mínimo 100) -> pedir 145
Productos a reponer: 4
Búsqueda B-204: Cinta aisladora, stock 35 (posición 3 de 7)
Búsqueda Z-999: no existe
```

Reglas

- Alcanza con listas (o listas paralelas), un filtrado y una búsqueda lineal.
- Un stock igual al mínimo no se repone.
- La búsqueda se corta apenas encuentra el código (no sigue recorriendo).
- Los datos pueden estar fijos en el código, pero el programa tiene que funcionar con cualquier cantidad de elementos.

Pista (leela solo si te trabás): escribí la búsqueda como una función que recibe el código y devuelve la posición, o -1 si no lo encuentra. Así la reutilizás para cada consulta.

Bonus (opcional)

- Contá cuántas comparaciones hizo cada búsqueda y pensá cuántas haría con 100.000 productos.
- Ordená el listado de reposición por urgencia (proporción stock / mínimo).

Cierre: Es la misma alerta de "punto de pedido" que tienen los sistemas de gestión de inventario.

Ejercicio 57: Cuentas claras en el asado

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 12/100

Situación: Cuatro amigos hicieron un asado y cada uno pagó una parte distinta. Al final nadie sabe cuánto le debe a quién, y el grupo de WhatsApp es un caos. Tu programa tiene que ordenar las cuentas.

Datos de entrada

Persona	Pagó	Qué compró
Ana	\$12.000	Carne
Beto	\$3.000	Pan
Caro	\$0	Nada
Dani	\$5.000	Bebidas

Tu programa debe

1. Calcular el total gastado y cuánto le corresponde pagar a cada uno (se divide en partes iguales).
2. Mostrar el **saldo** de cada persona: si le deben plata (positivo) o si debe (negativo).
3. Decir **quién le transfiere a quién y cuánto** para quedar todos a mano.

Salida esperada

Total: \$20000 | Cada uno paga: \$5000

Ana: +7000 (le deben)
Beto: -2000 (debe)
Caro: -5000 (debe)
Dani: 0 (está a mano)
Transferencias:
Caro -> Ana: \$5000
Beto -> Ana: \$2000

Reglas

- Solo necesitás variables, arreglos o listas, bucles y condicionales.
- Los nombres y montos pueden estar fijos en el código.
- El programa tiene que funcionar si cambiás los montos o agregás una persona más.
- Formato del saldo: nombre con dos puntos en 5 caracteres alineado a la izquierda, y el saldo con signo alineado a la derecha en 6. Primero se empareja al que más debe con el que más tiene que cobrar.

Pista (leela solo si te trabás): primero separá a la gente en dos grupos, los que deben y los que tienen que cobrar. Después andá emparejando a un deudor con un acreedor y restando lo que se transfiere, hasta que todos queden en cero.

Bonus (opcional)

- ¿Qué pasa si el total no se divide exacto, por ejemplo \$20.001 entre 3? Decidí cómo manejar los centavos.
- Intentá que la cantidad de transferencias sea la menor posible.

Cierre: Es el mismo problema que resuelven las apps para dividir gastos entre amigos.

Ejercicio 58: Butacas libres en el teatro

Nivel 2: Junior II · Colecciones y funciones · Temática: Turismo · Dificultad 12/100

Situación: Un teatro de la calle Corrientes vende entradas por internet y en boletería. Una familia de cuatro llega a último momento y quiere sentarse junta. El boletero necesita ver cuántas butacas libres hay y dónde está el primer bloque de 4 butacas contiguas libres.

Datos de entrada

Mapa de la sala (X = ocupada, . = libre). Filas 1 a 6, butacas numeradas del 1 al 10 de izquierda a derecha:

```
1 XX..XXXX..
2 XXXXXXXXXX
3 X...XX.XX
4 ..XXX....
5 XXXX..XXX
6 .....
```

Tu programa debe

1. Guardar el mapa en una matriz (o en una lista de textos).
2. Contar las butacas libres de cada fila y del total.
3. Calcular el porcentaje de ocupación.
4. Encontrar el primer bloque de 4 butacas libres contiguas en una misma fila, buscando fila por fila de la 1 a la 6 y de izquierda a derecha.

Salida esperada

```
Fila 1: 4 libres
Fila 2: 0 libres
Fila 3: 5 libres
Fila 4: 7 libres
Fila 5: 2 libres
Fila 6: 10 libres
Libres: 28 de 60 | Ocupación: 53.3%
Primeras 4 butacas juntas: fila 3, butacas 2 a 5
```

Reglas

- Alcanza con matrices, recorridos anidados y contadores.
- Las butacas contiguas tienen que estar en la misma fila.
- La ocupación se muestra con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Si no hay bloque de 4, mostrará "No hay 4 butacas juntas".

Pista (leela solo si te trabás): para el bloque, llevá un contador de "libres seguidas" que se reinicia en cada X y en cada fila nueva. Cuando llega a 4, calculá la butaca inicial a partir de la posición actual.

Bonus (opcional)

- Generalizá para un grupo de N personas.
- Si no hay N butacas juntas en una fila, buscá la mejor alternativa en dos filas consecutivas con las mismas columnas.

Cierre: Es la misma búsqueda que hacen los sistemas de venta de entradas cuando te sugieren "los mejores asientos disponibles juntos".

Ejercicio 59: Frecuencia cardíaca en reposo

Nivel 2: Junior II · Colecciones y funciones · Temática: Salud · Dificultad 12/100

Situación: Horacio empezó a correr hace un mes y su médica le pidió que se mida el pulso en reposo cada mañana durante dos semanas. Un pulso en reposo que baja suele indicar mejor estado físico, pero valores fuera de 60-100 latidos por minuto hay que comentarlos en la consulta.

Datos de entrada

Pulso en reposo (latidos por minuto), del día 1 al 14:

1	2	3	4	5	6	7	8	9	10	11	12	13	14
72	70	74	69	71	68	73	66	67	64	58	65	63	62

Tu programa debe

1. Calcular el promedio, la mínima y la máxima.
2. Listar los días fuera del rango de 60 a 100 lpm.
3. Comparar el promedio de la primera semana (días 1 a 7) con el de la segunda (días 8 a 14) e indicar la tendencia.

Salida esperada

```
Promedio: 67.3 lpm | Mínima: 58 | Máxima: 74
Fuera del rango 60-100: día 11 (58)
Semana 1: 71.0 lpm | Semana 2: 63.6 lpm -> tendencia bajando
```

Reglas

- Alcanza con listas, promedios, máximo, mínimo y filtrado.
- Promedios con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- 60 y 100 están dentro del rango.
- La tendencia es "bajando", "subiendo" o "estable" según los promedios sin redondear.

Pista (leela solo si te trabás): las dos semanas son dos tramos de la misma lista: días 1 a 7 y 8 a 14. Escribí una función que calcule el promedio de un tramo dado por su inicio y su fin.

Bonus (opcional)

- Calculá el promedio móvil de 3 días y mostralo para cada día desde el 3.
- Detectá el día con la mayor caída respecto del día anterior.

Cierre: Es el mismo seguimiento que hacen los relojes deportivos para estimar la recuperación y el estado físico.

Ejercicio 60: El hashtag más largo del posteo

Nivel 2: Junior II · Colecciones y funciones · Temática: Redes sociales · Dificultad 13/100

Situación: Una agencia de turismo analiza los posteos de sus clientes para saber qué hashtags usan. El sistema de la red social les da el texto crudo, y los hashtags aparecen pegados a comas, signos de exclamación o escritos con mayúsculas distintas. Tu programa tiene que extraerlos bien.

Datos de entrada

Texto del posteo:

```
Arrancamos la temporada #Verano2026 en la costa con amigos #playa #MarDelPlata #atardecer, y el
finde vuelve #PLAYA! #NoTeLoPodésPerder #
```

Tu programa debe

1. Recorrer el texto y extraer cada hashtag: empieza en # y sigue mientras haya letras, dígitos o guion bajo.
2. Descartar los # que no van seguidos de ningún carácter válido.
3. Quedarse con los hashtags distintos (sin distinguir mayúsculas), conservando la primera forma en que aparecen.
4. Mostrar los distintos con su largo (sin contar el #) y el más largo.

Salida esperada

```
Hashtags encontrados: 6
Distintos: 5
#Verano2026 (10)
#playa (5)
#MarDelPlata (11)
#atardecer (9)
#NoTeLoPodésPerder (17)
Más largo: #NoTeLoPodésPerder (17 caracteres)
```

Reglas

- Alcanza con recorrido de texto, dos índices y una lista.
- Las letras con tilde cuentan como letras.
- #playa y #PLAYA son el mismo hashtag.
- Si dos hashtags empatan en largo, se informa el primero en aparecer.

Pista (leela solo si te trabás): cuando encontrás un #, usá un segundo índice que avance mientras el carácter sea válido. El hashtag es el tramo entre los dos índices. Después seguí desde donde terminó.

Bonus (opcional)

- Contá cuántas veces aparece cada hashtag (sin distinguir mayúsculas).
- Separá los hashtags escritos en formato "camello" en palabras: NoTeLoPodésPerder → No Te Lo Podés Perder.

Cierre: Es el mismo tipo de extracción que hacen las redes sociales para calcular tendencias.

Ejercicio 61: Conversor de monedas en funciones

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 13/100

Situación: Una fintech de remesas quiere reescribir su conversor de monedas porque el actual repite la misma fórmula en veinte lugares y cada tanto alguien cambia una y se olvida de las otras. La idea es armarlo con funciones pequeñas y reutilizables, con el peso argentino como moneda puente.

Datos de entrada

Moneda	Pesos por unidad
ARS	1
USD	1200
EUR	1300
BRL	215
CLP	1,25

- Comisión de la plataforma: 1,5 % sobre el monto convertido.

Operación	Monto	De	A
1	100	USD	EUR
2	5000	BRL	USD
3	250000	ARS	CLP
4	80	EUR	ARS
5	10	JPY	ARS

Tu programa debe

- Escribir una función que pase un monto de cualquier moneda a pesos.
- Escribir una función que pase un monto en pesos a cualquier moneda.
- Escribir una función convertir(monto, origen, destino) que use las dos anteriores, y otra que aplique la comisión.
- Procesar las operaciones mostrando el monto convertido con y sin comisión, o un error si alguna moneda no está soportada.

Salida esperada

```
100.00 USD = 92.31 EUR | con comisión 1.5%: 90.92 EUR
5000.00 BRL = 895.83 USD | con comisión 1.5%: 882.40 USD
250000.00 ARS = 200000.00 CLP | con comisión 1.5%: 197000.00 CLP
80.00 EUR = 104000.00 ARS | con comisión 1.5%: 102440.00 ARS
10.00 JPY -> ARS: moneda no soportada (JPY)
```

Reglas

- Alcanza con funciones con parámetros y valor de retorno, y una búsqueda en la tabla de monedas.
- Todos los montos se muestran con 2 decimales (0,5 hacia arriba), redondeando solo al mostrar. Los decimales se muestran con punto y sin separador de miles en la salida.
- La fórmula de conversión tiene que aparecer una sola vez en todo el programa.
- Si la moneda de origen o de destino no existe, se informa cuál es la que falta (primero se revisa la de origen).

Pista (leela solo si te trabás): convertir de A a B es "pasar A a pesos" y después "pasar pesos a B". Si lo escribís así, sumar una moneda nueva es agregar una fila a la tabla y nada más.

Bonus (opcional)

- Agregá cotizaciones distintas de compra y de venta.

- Hacé que la comisión tenga un mínimo equivalente a 2 USD.

Cierre: Es el mismo diseño de "moneda puente" que usan las plataformas de pagos internacionales y remesas.

Ejercicio 62: Ta-te-ti: ¿quién ganó?

Nivel 2: Junior II · Colecciones y funciones · Temática: Videojuegos · Dificultad 13/100

Situación: Una app de juegos de mesa para chicos tiene un modo ta-te-ti multijugador, y el árbitro automático declaró ganadores equivocados en varias partidas. Te piden un árbitro nuevo que reciba un tablero y diga si alguien ganó, si fue empate, si la partida sigue o si el tablero es imposible.

Datos de entrada

Tableros (X y O son fichas, . es casilla vacía). Siempre empieza X.

Tablero 1:

```
X00
0XX
..X
```

Tablero 2:

```
X0X
X0.
.0.
```

Tablero 3:

```
X0X
X00
0XX
```

Tablero 4:

```
X.0
.X.
0..
```

Tablero 5:

```
XXX
000
...
```

Tablero 6:

```
00.
0..
X..
```

Tu programa debe

1. Guardar cada tablero en una matriz de 3×3 .
2. Revisar las 3 filas, las 3 columnas y las 2 diagonales buscando tres fichas iguales.
3. Validar el tablero: como empieza X, la cantidad de X menos la de O tiene que ser 0 o 1, y no pueden ganar los dos.
4. Informar: "ganó X", "ganó O", "empate" (tablero lleno sin ganador), "partida en curso" o "tablero inválido".

Salida esperada

```
Tablero 1: ganó X
Tablero 2: ganó O
Tablero 3: empate
Tablero 4: partida en curso
Tablero 5: tablero inválido
Tablero 6: tablero inválido
```


Reglas

- Alcanza con matrices, recorrido de filas, columnas y diagonales, y contadores.
- La validación se hace antes de declarar un ganador.
- El programa tiene que servir para cualquier tablero de 3×3 .

Pista (leela solo si te trabás): las 8 líneas ganadoras se pueden recorrer con el mismo código si antes armás una lista con las 8 líneas (cada una, sus 3 casillas). Así no escribís 8 condicionales casi iguales.

Bonus (opcional)

- Detectá otro caso inválido: si ganó X, tiene que haber una X más que O; si ganó O, la misma cantidad.
- Generalizá a un tablero de $N \times N$ donde gana quien complete una línea entera.

Cierre: Es el mismo tipo de verificación de estado que tienen todos los juegos de tablero digitales, desde el ta-te-ti hasta el ajedrez online.

Ejercicio 63: Nombres prolijos en la planilla de personal

Nivel 2: Junior II · Colecciones y funciones · Temática: Recursos humanos · Dificultad 13/100

Situación: En la textil "La Costurera", la planilla de personal se cargó durante años a mano: hay nombres en mayúsculas, con espacios de más y apellidos mal escritos. Recursos Humanos necesita normalizar los nombres y generar el email corporativo de cada persona.

Datos de entrada

Nombres tal como están en la planilla (entre comillas para que se vean los espacios):

- " mARÍA JOSÉ pérez "
- "juan DE la CRUZ"
- " ana belén del río"
- "PEDRO"
- "luis o'donnell"

Reglas de formato:

- Cada palabra con la primera letra en mayúscula y el resto en minúscula.
- Las partículas de, del, la, las, los, y van en minúscula, salvo que sean la primera palabra.
- Un solo espacio entre palabras, sin espacios al principio ni al final.
- Email: inicial del primer nombre + última palabra del nombre, todo en minúscula, sin tildes (ñ → n) y sin caracteres que no sean letras, seguido de @lacosturera.com.ar.

Tu programa debe

1. Escribir una función que separe un texto en palabras ignorando los espacios repetidos.
2. Escribir una función que capitalice una palabra, y aplicarla respetando las partículas.
3. Escribir una función que arme el email a partir del nombre ya normalizado.
4. Mostrar, para cada persona, el texto original, el nombre normalizado y el email.

Salida esperada

```
" mARÍA JOSÉ pérez " -> María José Pérez | mperez@lacosturera.com.ar
"juan DE la CRUZ" -> Juan de la Cruz | jcruz@lacosturera.com.ar
" ana belén del río" -> Ana Belén del Río | ario@lacosturera.com.ar
"PEDRO" -> Pedro | ppedro@lacosturera.com.ar
"luis o'donnell" -> Luis O'donnell | lodonnell@lacosturera.com.ar
```

Reglas

- Alcanza con recorrido de texto y funciones (separar, capitalizar y quitar tildes las escribís vos).
- Las letras con tilde se capitalizan correctamente: "ángel" → "Ángel".
- En el email se eliminan apóstrofes y cualquier carácter que no sea letra.
- Si el nombre tiene una sola palabra, el email usa su inicial y la misma palabra.

Pista (leela solo si te trabás): separar en palabras es recorrer el texto acumulando caracteres en una "palabra actual" y cerrarla cada vez que aparece un espacio, solo si no está vacía.

Bonus (opcional)

- Detectá emails duplicados y agregales un número: mperez, mperez2...
- Soportá apellidos compuestos con guion ("García-Márquez" → cada parte capitalizada).

Cierre: Es la limpieza de datos que hace cualquier área de sistemas antes de migrar una base de personas a un sistema nuevo.

Ejercicio 64: Lluvia acumulada por semana en el campo

Nivel 2: Junior II · Colecciones y funciones · Temática: Agricultura · Dificultad 13/100

Situación: Un productor tambero de la cuenca lechera registra con un pluviómetro los milímetros de lluvia de cada día. Quiere saber qué semana llovió más para ajustar la siembra de pasturas, y si hay algún día de la semana en que "siempre" llueve (su vecino jura que son los lunes).

Datos de entrada

Lluvia diaria en mm (filas = semanas, columnas = días):

Semana	Lun	Mar	Mié	Jue	Vie	Sáb	Dom
1	0	12	0	0	25	3	0
2	0	0	0	0	0	8	41
3	15	2	0	0	0	0	0
4	0	0	33	18	0	0	6

Tu programa debe

1. Guardar los datos en una matriz de 4×7 .
2. Calcular la lluvia total de cada semana (suma por fila).
3. Calcular la lluvia total de cada día de la semana (suma por columna).
4. Mostrar el total del mes, la semana más lluviosa y la cantidad de días sin lluvia.

Salida esperada

```
Semana 1: 40 mm
Semana 2: 49 mm
Semana 3: 17 mm
Semana 4: 57 mm
Por día de la semana: Lun=15 Mar=14 Mié=33 Jue=18 Vie=25 Sáb=11 Dom=47
Total del mes: 163 mm
Semana más lluviosa: 4 (57 mm)
Días sin lluvia: 18 de 28
```

Reglas

- Alcanza con matrices y recorridos por filas y por columnas.
- Si dos semanas empatan como más lluviosas, se informa la primera.
- El programa tiene que funcionar con cualquier cantidad de semanas.

Pista (leela solo si te trabás): sumar una fila es recorrer sus columnas; sumar una columna es fijar la columna y recorrer las filas. Es el mismo bucle anidado con los índices intercambiados.

Bonus (opcional)

- Calculá la racha más larga de días seguidos sin lluvia, cruzando de una semana a la otra.
- Marcá los días con más de 30 mm como "lluvia intensa" y mostrá su ubicación (semana y día).

Cierre: Es el mismo tipo de resumen que muestran los servicios agroclimáticos para planificar siembras y cosechas.

Ejercicio 65: El producto estrella del día

Nivel 2: Junior II · Colecciones y funciones · Temática: E-commerce · Dificultad 14/100

Situación: Una tienda online de accesorios de tecnología quiere destacar en la portada al "producto estrella" del día. El equipo de marketing discute si la estrella es el que más unidades vendió o el que más plata facturó, así que les vas a mostrar las dos cosas.

Datos de entrada

Producto	Unidades vendidas	Precio
Auriculares	14	\$28.500
Funda	52	\$4.900
Cargador	31	\$12.800
Cable USB	60	\$3.500
Parlante	9	\$41.000
Soporte	23	\$6.700
Mouse	18	\$15.900
Teclado	11	\$33.800

Tu programa debe

1. Guardar productos, unidades y precios en tres listas paralelas.
2. Calcular la facturación de cada producto y el total.
3. Encontrar el producto con más unidades vendidas y el de mayor facturación.
4. Calcular qué porcentaje del total representa el de mayor facturación.

Salida esperada

```
Auriculares: 14 u x $28500 = $399000
Funda: 52 u x $4900 = $254800
Cargador: 31 u x $12800 = $396800
Cable USB: 60 u x $3500 = $210000
Parlante: 9 u x $41000 = $369000
Soporte: 23 u x $6700 = $154100
Mouse: 18 u x $15900 = $286200
Teclado: 11 u x $33800 = $371800
Total vendido: $2441700
Más unidades: Cable USB (60)
Más facturación: Auriculares ($399000)
Porcentaje de la facturación del producto estrella: 16.34%
```

Reglas

- Alcanza con listas paralelas, acumuladores y máximos.
- El porcentaje se muestra con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Si hay empate, se informa el producto que aparece primero.
- Las tres listas tienen siempre el mismo largo; la posición *i* de cada una habla del mismo producto.

Pista (leela solo si te trabás): con listas paralelas, el índice es lo que une los datos: si encontrás el máximo en la posición 4 de la lista de facturación, el nombre está en la posición 4 de la lista de productos.

Bonus (opcional)

- Mostrá el ranking de los 3 productos con más facturación.
- Calculá el precio promedio ponderado por unidades vendidas.

Cierre: Es el mismo cálculo que hacen los paneles de ventas de las plataformas de comercio electrónico.

Ejercicio 66: ¿Contraseña fuerte o débil?

Nivel 2: Junior II · Colecciones y funciones · Temática: Seguridad informática · Dificultad 14/100

Situación: El área de seguridad de una obra social detectó que muchos afiliados usan contraseñas como "123456" en la app. Quieren un medidor que, mientras la persona escribe, le diga qué tan fuerte es su contraseña y qué le falta para mejorarla.

Datos de entrada

Contraseñas de prueba:

- hola123
- Mate2026!
- CaféConLeche#77
- PASSWORD 2026
- zX9\$kL2@pQ

Criterios (cada uno suma 1 punto):

1. Al menos 10 caracteres.
2. Al menos una mayúscula.
3. Al menos una minúscula.
4. Al menos un dígito.
5. Al menos un símbolo (cualquier carácter que no sea letra, dígito ni espacio).
6. Sin espacios.

Clasificación: 6 puntos → FUERTE; 4 o 5 → MEDIA; 3 o menos → DÉBIL.

Tu programa debe

1. Recorrer cada contraseña y clasificar sus caracteres.
2. Evaluar los 6 criterios y calcular el puntaje.
3. Clasificarla como FUERTE, MEDIA o DÉBIL.
4. Mostrar el puntaje, la clasificación y los criterios que no cumple, en el orden de la lista.

Salida esperada

```
hola123: 3/6 DÉBIL | falta: al menos 10 caracteres, una mayúscula, un símbolo
Mate2026!: 5/6 MEDIA | falta: al menos 10 caracteres
CaféConLeche#77: 6/6 FUERTE
PASSWORD 2026: 3/6 DÉBIL | falta: una minúscula, un símbolo, sin espacios
zX9$kL2@pQ: 6/6 FUERTE
```

Reglas

- Alcanza con recorrido de texto, clasificación de caracteres y contadores o indicadores.
- Las letras con tilde cuentan como letras (é es minúscula, É es mayúscula).
- Si cumple todo, no se muestra la parte "falta".
- Recorré cada contraseña una sola vez, activando indicadores a medida que encontrás cada tipo de carácter.

Pista (leela solo si te trabás): usá un indicador por tipo ("tiene mayúscula", "tiene dígito"...) que arranca en falso y se enciende cuando encontrás un carácter de ese tipo. Al final, contá los indicadores encendidos.

Bonus (opcional)

- Restá un punto si la contraseña contiene el nombre de usuario o la palabra "password" (sin importar mayúsculas).

- Penalizá secuencias como "123" o "abc" y caracteres repetidos tres veces seguidas.

Cierre: Es el medidor de fortaleza que muestran los formularios de registro de bancos, redes sociales y servicios online.

Ejercicio 67: El mensaje secreto del club de ajedrez

Nivel 2: Junior II · Colecciones y funciones · Temática: Seguridad informática · Dificultad 14/100

Situación: En el club de ajedrez del barrio juegan un torneo por correspondencia y a los chicos les encanta mandarse las jugadas "en código" para que el rival no las vea en el pizarrón. Usan un cifrado por desplazamiento sobre el alfabeto español y te piden un programa que cifre y descifre.

Datos de entrada

Alfabeto (27 letras, con Ñ): ABCDEFGHIJKLMNOPQRSTUVWXYZ

- Desplazamiento: 4 (A → E, K → Ñ, X → B)
- Mensaje a enviar: Alfil a la casilla E5 al amanecer
- Mensaje recibido: HEPE E OE GSOYPQE L PEREQE XIPTVEQS

Tu programa debe

1. Escribir una función que reciba un texto y un desplazamiento y devuelva el texto cifrado.
2. Cifrar el mensaje a enviar (en mayúsculas) y verificar que al descifrarlo vuelve al original.
3. Descifrar el mensaje recibido usando la misma función con el desplazamiento negativo.

Salida esperada

```
Original:  ALFIL A LA CASILLA E5 AL AMANECER
Cifrado:  EOJMO E OE GEWM00E I5 EO EPEQIGIV
Verificación: ALFIL A LA CASILLA E5 AL AMANECER
Recibido:  HEPE E OE GSOYPQE L PEREQE XIPTVEQS
Descifrado: DAMA A LA COLUMNA H MAÑANA TEMPRANO
```

Reglas

- Alcanza con recorrido de texto, búsqueda de la posición de una letra y aritmética circular (resto).
- Las letras se pasan a mayúsculas; espacios, dígitos y signos quedan igual.
- El alfabeto es circular: después de la Z viene la A.
- Ojo con el resto de números negativos: según cómo lo calcules, puede darte negativo. Sumale el largo del alfabeto antes de tomar el resto.

Pista (leela solo si te trabás): la posición de la letra cifrada es (posición original + desplazamiento) resto 27. Descifrar es lo mismo con el desplazamiento negativo.

Bonus (opcional)

- Descifrá un mensaje sin conocer el desplazamiento probando los 27 posibles y eligiendo el que tenga más palabras de una lista de palabras comunes.
- Implementá una variante con clave (cifrado de Vigenère): cada letra usa un desplazamiento distinto según una palabra clave.

Cierre: Es el cifrado de César, el abuelo de la criptografía: hoy no protege nada, pero es la puerta de entrada para entender cómo funcionan los cifrados modernos.

Ejercicio 68: Movimientos de la cuenta y el saldo más bajo

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 14/100

Situación: Leandro tiene una cuenta con descubierto autorizado y el banco le cobra intereses cada vez que queda en rojo. Quiere revisar los movimientos de octubre para ver cuál fue su saldo más bajo y qué movimientos lo dejaron en negativo.

Datos de entrada

Saldo al 1 de octubre: \$45.000

Día	Concepto	Importe
2	Sueldo	+\$820.000
3	Alquiler	-\$380.000
5	Supermercado	-\$96.500
8	Tarjeta	-\$312.000
11	Transferencia de Juan	+\$25.000
14	Servicios	-\$48.700
17	Farmacia	-\$22.300
20	Nafta	-\$61.000
24	Venta de bici	+\$150.000
27	Cuota del préstamo	-\$140.000
29	Cine	-\$18.500

Tu programa debe

1. Recorrer los movimientos actualizando el saldo y mostrar una línea por movimiento.
2. Sumar ingresos y egresos por separado.
3. Encontrar el saldo más bajo y el día en que ocurrió.
4. Contar los movimientos después de los cuales el saldo quedó negativo e informar sus días.

Salida esperada

```

Día 2 Sueldo          +820000 -> 865000
Día 3 Alquiler        -380000 -> 485000
Día 5 Supermercado    -96500 -> 388500
Día 8 Tarjeta         -312000 -> 76500
Día 11 Transferencia de Juan +25000 -> 101500
Día 14 Servicios      -48700 -> 52800
Día 17 Farmacia       -22300 -> 30500
Día 20 Nafta          -61000 -> -30500
Día 24 Venta de bici   +150000 -> 119500
Día 27 Cuota del préstamo -140000 -> -20500
Día 29 Cine           -18500 -> -39000
Ingresos: $995000 | Egresos: $1079000 | Saldo final: $-39000
Saldo más bajo: $-39000 (día 29)
Movimientos que dejaron la cuenta en rojo: 3 (días 20, 27, 29)

```

Reglas

- Alcanza con listas, un saldo acumulado, un mínimo y un filtrado.
- Formato de cada línea: día en 2 caracteres, concepto en 22 alineado a la izquierda, importe con signo en 8 y saldo en 8 (alineados a la derecha).
- Si el saldo más bajo se repite, se informa el primer día.
- El saldo inicial cuenta para el mínimo (si nunca bajó, el mínimo es el inicial y se informa el día 0).

Pista (leela solo si te trabás): el saldo mínimo se busca sobre los saldos acumulados, no sobre los importes: un egreso chico puede dejarte en rojo si venías justo.

Bonus (opcional)

- Calculá los intereses del descubierto: 0,2 % diario sobre el saldo negativo, por cada día que la cuenta estuvo en rojo.
- Detectá el período más largo (en días) con saldo negativo.

Cierre: Es el mismo cálculo de saldo corrido que muestra el homebanking en la pantalla de movimientos.

Ejercicio 69: La vuelta más rápida del karting

Nivel 2: Junior II · Colecciones y funciones · Temática: Deportes · Dificultad 14/100

Situación: En un kartódromo, el cronometraje registra cada vuelta en milisegundos, pero la pantalla del público quiere mostrar los tiempos como en la Fórmula 1: minutos, segundos y milésimas. Además hay que anunciar la vuelta rápida y el ganador de una carrera de 8 vueltas.

Datos de entrada

Piloto	V1	V2	V3	V4	V5	V6	V7	V8
Vera	62415	61880	61702	61955	61610	61799	62040	61655
Iñaki	63120	61590	61488	61730	61905	61512	61840	61777
Tamara	61950	61875	61910	61802	61720	61699	61690	61705

(Tiempos en milisegundos.)

Tu programa debe

1. Escribir una función que convierta milisegundos al formato M:SS.mmm.
2. Para cada piloto, encontrar su mejor vuelta (y en qué vuelta la hizo) y su tiempo total.
3. Anunciar la vuelta rápida de la carrera y el ganador (menor tiempo total).
4. Mostrar la diferencia de cada uno de los demás pilotos con el ganador, en segundos con 3 decimales.

Salida esperada

```
Vera: mejor vuelta 1:01.610 (vuelta 5) | total 8:15.056
Iñaki: mejor vuelta 1:01.488 (vuelta 3) | total 8:14.962
Tamara: mejor vuelta 1:01.690 (vuelta 7) | total 8:14.351
Vuelta rápida de la carrera: Iñaki (1:01.488)
Ganador: Tamara
Vera a +0.705 s
Iñaki a +0.611 s
```

Reglas

- Alcanza con listas (una por piloto o una matriz), mínimos, acumuladores y conversión de unidades.
- Trabaja en milisegundos enteros y conviértelo solo al mostrar: así no hay errores de redondeo.
- Segundos con 2 dígitos y milésimas con 3: 1:01.702, nunca 1:1.702.
- Las diferencias se muestran en el orden de la lista de pilotos, con 3 decimales (en este caso son exactas). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabas): 61.702 ms son 1 minuto (61.702 / 60.000), 1 segundo ((61.702 resto 60.000) / 1.000) y 702 milésimas (61.702 resto 1.000).

Bonus (opcional)

- Muestra la posición de cada piloto al final de cada vuelta (quién iba primero en la vuelta 3, por ejemplo).
- Detecta vueltas anómalas: más de 1,5 % más lentas que la mejor vuelta del mismo piloto.

Cierre: Es exactamente lo que hacen los sistemas de cronometraje de las carreras de automovilismo y atletismo.

Ejercicio 70: Cobertura de antenas en la grilla

Nivel 2: Junior II · Colecciones y funciones · Temática: Telecomunicaciones · Dificultad 14/100

Situación: Una cooperativa de internet rural divide la zona que atiende en una grilla de parcelas y ubicó tres antenas. Cada antena cubre su propia parcela y las 8 que la rodean. Antes de salir a vender el servicio, quieren un mapa de qué parcelas tienen cobertura.

Datos de entrada

Mapa de la zona (A = antena, . = parcela), 6 filas × 8 columnas:

```
.....
.A.....
.....
.....A.
.....
.....
..A.....
```

Tu programa debe

1. Guardar el mapa en una matriz.
2. Marcar como cubiertas la parcela de cada antena y sus 8 vecinas, sin salirse de los bordes.
3. Dibujar el mapa de cobertura: A para las antenas, # para las parcelas cubiertas y . para las que no.
4. Mostrar cuántas parcelas están cubiertas (y el porcentaje) y cuántas no.

Salida esperada

```
###.....
#A#.....
###.###
....#A#
.###.###
.#A#....
Celdas cubiertas: 24 de 48 (50.0%)
Sin cobertura: 24
```

Reglas

- Alcanza con matrices, bucles anidados y control de bordes.
- Una parcela cubierta por dos antenas se cuenta una sola vez.
- El porcentaje se muestra con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Antes de acceder a una vecina, verificá que sus índices estén dentro de la matriz.

Pista (leela solo si te trabás): usá una segunda matriz de verdadero/falso para la cobertura. Para las 8 vecinas, recorré los desplazamientos -1, 0 y +1 en filas y columnas.

Bonus (opcional)

- Proponé dónde poner una cuarta antena para cubrir la mayor cantidad de parcelas nuevas (probá todas las posiciones).
- Hacé que cada antena tenga su propio radio de alcance (1, 2 o 3 parcelas).

Cierre: Es la versión simplificada de los mapas de cobertura que calculan las empresas de telecomunicaciones.

Ejercicio 71: Menú del día que entra en el presupuesto

Nivel 2: Junior II · Colecciones y funciones · Temática: Gastronomía · Dificultad 15/100

Situación: En un bodegón porteño, una clienta vegetariana pregunta qué puede comer con \$14.000. El mozo tiene que recorrer la carta de memoria y siempre se olvida de algo. El dueño quiere un programa que filtre la carta y arme el menú completo (entrada, principal y postre) más barato.

Datos de entrada

Plato	Categoría	Precio	Vegetariano
Empanadas de verdura	entrada	\$3.200	Sí
Provoleta	entrada	\$4.800	Sí
Rabas	entrada	\$6.900	No
Milanesa napolitana	principal	\$9.800	No
Ñoquis con fileto	principal	\$7.400	Sí
Risotto de hongos	principal	\$10.200	Sí
Bife de chorizo	principal	\$13.500	No
Flan casero	postre	\$2.900	Sí
Panqueque con dulce de leche	postre	\$3.100	Sí
Helado	postre	\$3.600	Sí

Presupuesto de la clienta: \$14.000

Tu programa debe

1. Filtrar los platos vegetarianos cuyo precio no supere el presupuesto y mostrarlos en el orden de la carta.
2. Probar todas las combinaciones de una entrada, un principal y un postre vegetarianos.
3. Encontrar la combinación más barata e informar si entra en el presupuesto y cuánto sobra.

Salida esperada

Platos vegetarianos dentro del presupuesto:

Empanadas de verdura (entrada) \$3200

Provoleta (entrada) \$4800

Ñoquis con fileto (principal) \$7400

Risotto de hongos (principal) \$10200

Flan casero (postre) \$2900

Panqueque con dulce de leche (postre) \$3100

Helado (postre) \$3600

Menú completo más barato: Empanadas de verdura + Ñoquis con fileto + Flan casero = \$13500 (sobran \$500)

Reglas

- Alcanza con listas, filtrado, búsqueda lineal y bucles anidados.
- Si hay empate en precio, se queda la primera combinación encontrada (recorriendo entradas, después principales, después postres en el orden de la carta).
- Si no hay ningún menú completo que entre, se informa "No hay menú completo vegetariano dentro del presupuesto".

Pista (leela solo si te trabás): separará primero las tres categorías en tres listas filtradas. Después, tres bucles anidados prueban todas las combinaciones.

Bonus (opcional)

- Buscá el menú completo más caro que todavía entre en el presupuesto (la clienta quiere darse un gusto).

- Agregá la opción "sin TACC" como segundo filtro combinable con el vegetariano.

Cierre: Es el mismo filtrado que hacen las apps de delivery cuando elegís "vegetariano" y un rango de precios.

Ejercicio 72: Vuelos demorados del día

Nivel 2: Junior II · Colecciones y funciones · Temática: Turismo · Dificultad 15/100

Situación: El área de atención al pasajero de un aeropuerto tiene que publicar un reporte de puntualidad de la mañana. Por norma, un vuelo se considera demorado si despegua más de 15 minutos después de lo programado. Tu programa calcula el reporte a partir de los horarios reales.

Datos de entrada

Vuelo	Destino	Programado	Real
AR1302	Bariloche	06:40	06:52
FO5401	Mendoza	07:15	08:05
AR1528	Salta	08:00	08:00
JA3210	Córdoba	09:30	09:41
AR1712	Ushuaia	10:05	11:47
FO5033	Iguazú	11:20	11:36

Tu programa debe

1. Calcular la demora de cada vuelo en minutos.
2. Clasificar cada vuelo como DEMORADO (más de 15 minutos) o a horario, y mostrar una línea por vuelo.
3. Calcular el porcentaje de puntualidad y la demora promedio de los demorados.
4. Informar el vuelo con mayor demora.

Salida esperada

```
AR1302 Bariloche 06:40 -> 06:52 | 12 min | a horario
FO5401 Mendoza 07:15 -> 08:05 | 50 min | DEMORADO
AR1528 Salta 08:00 -> 08:00 | 0 min | a horario
JA3210 Córdoba 09:30 -> 09:41 | 11 min | a horario
AR1712 Ushuaia 10:05 -> 11:47 | 102 min | DEMORADO
FO5033 Iguazú 11:20 -> 11:36 | 16 min | DEMORADO
Vuelos demorados: 3 de 6 | Puntualidad: 50.0%
Demora promedio de los demorados: 56.0 min
Mayor demora: AR1712 (102 min)
```

Reglas

- Alcanza con listas de registros, conversión de horas a minutos, filtrado y promedio.
- Exactamente 15 minutos de demora sigue siendo "a horario".
- Porcentajes y promedios con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Formato: destino en 10 caracteres alineado a la izquierda y la demora en 3 alineada a la derecha. Todos los vuelos salen el mismo día.

Pista (leela solo si te trabás): pasá cada horario a "minutos desde la medianoche" con una función; la demora es la resta. Separar la hora del texto "HH:MM" es tomar los caracteres antes y después de los dos puntos.

Bonus (opcional)

- Soportá vuelos que tenían que salir a las 23:50 y salieron a las 00:20 del día siguiente.
- Agrupá la puntualidad por aerolínea (las dos primeras letras del código del vuelo).

Cierre: Es el mismo indicador de puntualidad que publican los aeropuertos y las aerolíneas.

Ejercicio 73: Espejo de palabras para el cartel

Nivel 2: Junior II · Colecciones y funciones · Temática: Educación · Dificultad 15/100

Situación: Una maestra de Lengua prepara un cartel para el aula sobre palabras que se leen igual al derecho y al revés. Quiere mostrar la misma frase dada vuelta de tres formas distintas y que el programa descubra cuáles palabras son palíndromas.

Datos de entrada

Frase: Ana visitó Neuquén con su oso Otto (las palabras están separadas por un único espacio)

Tu programa debe

1. Mostrar la frase completa invertida, carácter por carácter.
2. Mostrar cada palabra invertida, pero manteniendo el orden de las palabras.
3. Mostrar las palabras en orden inverso (sin invertir sus letras).
4. Contar y listar las palabras palíndromas de más de una letra, ignorando mayúsculas y tildes.

Salida esperada

```
Texto al revés:      ott0 oso us noc néuqueN ótisiv anA
Cada palabra al revés: anA ótisiv néuqueN noc us oso ott0
Palabras en orden inverso: Otto oso su con Neuquén visitó Ana
Palabras palíndromas: 4 (Ana, Neuquén, oso, Otto)
```

Reglas

- Alcanza con recorrido de texto, separación en palabras y listas.
- Las inversiones las implementás vos, recorriendo los caracteres o las palabras.
- "Neuquén" es palíndroma: sin tilde y en minúscula es "neuquen".
- Las palabras se listan en el orden en que aparecen en la frase.

Pista (leela solo si te trabás): las tres inversiones son el mismo algoritmo aplicado a cosas distintas: a los caracteres de la frase, a los caracteres de cada palabra y a la lista de palabras.

Bonus (opcional)

- Verificá si la frase entera es palíndroma ignorando espacios, tildes y mayúsculas (probá con "Anita lava la tina").
- Soportá signos de puntuación pegados a las palabras sin que arruinen la detección.

Cierre: Invertir texto es una de las operaciones más básicas del procesamiento de cadenas, y aparece desde en editores de texto hasta en validadores de códigos.

Ejercicio 74: El paciente con la fiebre más alta

Nivel 2: Junior II · Colecciones y funciones · Temática: Salud · Dificultad 15/100

Situación: En la sala de internación de una clínica, las enfermeras toman la temperatura a cada paciente cuatro veces por turno. El médico de guardia, al llegar, quiere ver en un solo resumen quién tuvo fiebre y quién tuvo la temperatura más alta, para empezar la recorrida por ahí.

Datos de entrada

Paciente	Habitación	Toma 1	Toma 2	Toma 3	Toma 4
Rosa Medina	201	37,2	37,8	38,4	37,9
Julián Paz	202	36,8	36,9	37,1	36,7
Estela Ruiz	205	38,1	38,9	39,2	38,6
Omar Vidal	207	37,5	38,0	37,6	37,4
Lía Sosa	210	39,2	38,7	38,3	37,9

Tu programa debe

1. Guardar cada paciente como un registro (nombre, habitación y su lista de temperaturas).
2. Calcular para cada paciente su máxima (con el número de toma) y su promedio.
3. Marcar con FIEBRE a quienes tuvieron al menos una toma de 38,0 °C o más, y contarlos.
4. Informar la temperatura más alta de toda la guardia, con paciente, habitación y toma.

Salida esperada

Rosa Medina (hab. 201): máx 38.4 °C en la toma 3 | promedio 37.83 | FIEBRE
 Julián Paz (hab. 202): máx 37.1 °C en la toma 3 | promedio 36.88
 Estela Ruiz (hab. 205): máx 39.2 °C en la toma 3 | promedio 38.70 | FIEBRE
 Omar Vidal (hab. 207): máx 38.0 °C en la toma 2 | promedio 37.63 | FIEBRE
 Lía Sosa (hab. 210): máx 39.2 °C en la toma 1 | promedio 38.53 | FIEBRE
 Pacientes con fiebre: 4 de 5
 Temperatura más alta de la guardia: 39.2 °C - Estela Ruiz (hab. 205, toma 3)

Reglas

- Alcanza con listas de registros, máximos y promedios.
- Máximas con 1 decimal y promedios con 2, redondeo hacia arriba desde 0,5. Los decimales se muestran con punto y sin separador de miles en la salida.
- En los empates (dentro de un paciente o entre pacientes), gana el que aparece primero.
- 38,0 °C exactos ya es fiebre.

Pista (leela solo si te trabás): la máxima de la guardia se puede calcular mientras recorrés a los pacientes: compará la máxima de cada uno contra la mejor vista hasta ahora.

Bonus (opcional)

- Detectá pacientes con fiebre en ascenso: cada toma más alta que la anterior.
- Ordená el resumen de mayor a menor temperatura máxima para armar la recorrida.

Cierre: Es el mismo tipo de tablero de alertas que tienen los sistemas de enfermería de los hospitales.

Ejercicio 75: Rotar la foto 90 grados

Nivel 2: Junior II · Colecciones y funciones · Temática: Redes sociales · Dificultad 15/100

Situación: Una red social recibe fotos sacadas con el celular de costado y tiene que girarlas antes de mostrarlas. Una imagen es una matriz de píxeles, así que girarla es reacomodar esa matriz. Probalo con un ícono en blanco y negro de 5×7 píxeles.

Datos de entrada

Ícono (1 = píxel negro, 0 = blanco), 5 filas $\times$ 7 columnas:

```
0011100
0100010
1000001
1111111
1000001
```

Tu programa debe

1. Guardar el ícono en una matriz de 5×7 .
2. Generar una nueva matriz con el ícono rotado 90° en sentido horario (queda de 7×5).
3. Generar otra con el ícono rotado 90° en sentido antihorario.
4. Mostrar las tres versiones dibujando # para el 1 y . para el 0, y verificar que cuatro rotaciones horarias devuelven el original.

Salida esperada

```
Original (5x7):
..###..
.###..
#....#
#####
#....#
Rotada 90° horario (7x5):
###..
.##.
.##.
.##.
.##.
.##.
###..
Rotada 90° antihorario (7x5):
..###
.##.
#..#
#..#
#..#
#..#
..###
Cuatro rotaciones horarias vuelven al original: sí
```

Reglas

- Alcanza con matrices y recorridos con índices calculados.
- La rotación va a una matriz nueva: no se modifica la original.
- Tiene que funcionar para matrices no cuadradas (las filas y columnas se intercambian).
- Entre cada dibujo va una línea en blanco, con los títulos tal como aparecen en la salida esperada.

Pista (leela solo si te trabás): en la rotación horaria, la fila i de la nueva matriz es la columna i de la original leída de abajo hacia arriba. Anotá dónde termina cada esquina antes de escribir los índices.

Bonus (opcional)

- Implementá el espejo horizontal y el vertical.
- Rotá una matriz cuadrada "en el lugar" (sin matriz auxiliar), intercambiando de a cuatro elementos.

Cierre: Es la misma operación que hace tu celular cuando gira una foto, solo que con millones de píxeles.

Lote 04: ejercicios 76 a 100

Ejercicio 76: La tasa real de las cuotas

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 16/100

Situación: Un televisor cuesta \$900.000 de contado y la casa de electrodomésticos ofrece cuatro planes de cuotas "fijas". Los vendedores dicen que todos "son lo mismo", pero el recargo real cambia bastante. Tu programa compara los planes con una función reutilizable.

Datos de entrada

Plan	Cuotas	Valor de la cuota
1	3	\$320.000
2	6	\$168.000
3	12	\$96.000
4	18	\$67.500

- Precio de contado: \$900.000
- Sueldo del comprador: \$800.000

Tu programa debe

1. Escribir una función que reciba precio de contado, cantidad de cuotas y valor de la cuota, y devuelva el total financiado, el recargo total en % y el recargo mensual promedio (recargo total / cuotas).
2. Aplicarla a los cuatro planes.
3. Calcular qué porcentaje del sueldo representa cada cuota.
4. Informar el plan con menor recargo mensual promedio.

Salida esperada

```
3 cuotas de $320000: total $960000 | recargo 6.67% | 2.22% por mes | cuota = 40.0% del sueldo
6 cuotas de $168000: total $1008000 | recargo 12.00% | 2.00% por mes | cuota = 21.0% del sueldo
12 cuotas de $96000: total $1152000 | recargo 28.00% | 2.33% por mes | cuota = 12.0% del sueldo
18 cuotas de $67500: total $1215000 | recargo 35.00% | 1.94% por mes | cuota = 8.4% del sueldo
Menor recargo mensual promedio: 18 cuotas
```

Reglas

- Alcanza con funciones que devuelven varios valores (o una estructura), porcentajes y un mínimo.
- Recargos con 2 decimales y porcentaje del sueldo con 1 (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- El recargo mensual promedio es una simplificación (no es la tasa real); en el nivel 9 vas a calcular la tasa verdadera.
- Formato: la cantidad de cuotas se alinea a la derecha en 2 caracteres.

Pista (leela solo si te trabás): que la función no imprima nada: que devuelva los tres números y que el programa principal decida cómo mostrarlos. Eso la hace reutilizable.

Bonus (opcional)

- Agregá la inflación esperada (3 % mensual) y calculá el valor presente de cada plan.
- Rechazá los planes cuya cuota supere el 30 % del sueldo.

Cierre: Es el mismo cálculo de costo financiero total que las tiendas están obligadas a informar en sus publicidades de cuotas.

Ejercicio 77: Consumo eléctrico por franja horaria

Nivel 2: Junior II · Colecciones y funciones · Temática: Clima y energía · Dificultad 16/100

Situación: La distribuidora eléctrica ofrece una tarifa por franja horaria: la energía es barata de madrugada y carísima entre las 18 y las 23. La familia Ferreyra instaló un medidor inteligente y quiere saber cuánto gasta en cada franja y cuánto ahorraría si pone el lavarropas de noche.

Datos de entrada

Consumo por hora (kWh). La posición 0 es de 00:00 a 01:00, la 23 es de 23:00 a 24:00:

Horas	Consumos
00-05	0,4 0,3 0,3 0,3 0,3 0,4
06-11	0,9 1,2 0,8 0,6 0,5 0,7
12-17	1,1 0,9 0,6 0,5 0,7 1,0
18-23	1,8 2,4 2,6 2,1 1,5 0,8

Franja	Horas	Precio por kWh
Valle	0 a 5 y 23	\$80
Resto	6 a 17	\$120
Pico	18 a 22	\$190

Tu programa debe

1. Sumar el consumo de cada franja recorriendo solo las posiciones que le corresponden.
2. Calcular el costo de cada franja y el total del día.
3. Encontrar la hora de mayor consumo.
4. Calcular cuánto se ahorra por lavado al mover un lavarropas de 1,2 kWh de la franja pico a la valle.

Salida esperada

```
Valle: 2.8 kWh x $80 = $224.00
Resto: 9.5 kWh x $120 = $1140.00
Pico: 10.4 kWh x $190 = $1976.00
Total: 22.7 kWh | $3340.00
Hora de mayor consumo: 20:00 (2.6 kWh)
Mover el lavarropas (1.2 kWh) de pico a valle ahorra $132.00 por lavado
```

Reglas

- Alcanza con listas, sumas por rango de posiciones y un máximo.
- kWh con 1 decimal y pesos con 2 (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Cuidado: la franja valle no es un rango continuo (incluye la hora 23).
- Si dos horas empatan en el máximo, se informa la primera.

Pista (leela solo si te trabás): escribí una función que sume la lista entre dos posiciones (inicio y fin inclusive). La franja valle es la suma de dos llamadas.

Bonus (opcional)

- Proponé a qué hora poner el lavarropas (1,2 kWh durante una hora) para que el costo total sea mínimo sin superar 2 kWh en ninguna hora.
- Compará el costo del día con una tarifa plana de \$125 por kWh.

Cierre: Es la misma cuenta que hacen los medidores inteligentes y las apps de las distribuidoras con tarifa horaria.

Ejercicio 78: Contador de palabras para el ensayo

Nivel 2: Junior II · Colecciones y funciones · Temática: Educación · Dificultad 16/100

Situación: En un profesorado, los ensayos tienen un límite de palabras y el sistema del campus cuenta mal: suma los signos de puntuación como palabras. La coordinación quiere un contador propio que además informe cuántas oraciones hay y el largo promedio de las oraciones, un indicador de claridad.

Datos de entrada

Fragmento del ensayo:

La inteligencia artificial cambió la forma en que estudiamos. Hace diez años, buscar una fuente llevaba horas en la biblioteca; hoy alcanza con una pregunta bien hecha. ¿Eso nos vuelve más curiosos o más cómodos? No hay una respuesta única. Lo que sí sabemos es que pensar sigue siendo trabajo nuestro, y que ninguna herramienta lo hace por nosotros.

Tu programa debe

1. Separar el texto en palabras: una palabra es una secuencia de letras o dígitos (los signos y espacios separan).
2. Contar las oraciones: cada . ? o ! cierra una.
3. Calcular las palabras por oración.
4. Encontrar la palabra más larga y estimar el tiempo de lectura a 200 palabras por minuto.

Salida esperada

Palabras: 59
Oraciones: 5
Palabras por oración: 11.8
Palabra más larga: inteligencia (12 letras)
Tiempo de lectura: 1 min

Reglas

- Alcanza con recorrido de texto, contadores y una lista de palabras.
- Las letras con tilde y la ñ son letras. "¿" no cierra oración, "?" sí.
- Palabras por oración con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- El tiempo de lectura se redondea hacia arriba a minutos enteros. En empates de largo, gana la primera palabra.

Pista (leela solo si te trabás): acumulá caracteres válidos en una palabra actual y, cuando aparece un separador, guardala si no está vacía. No te olvides de la última palabra cuando termina el texto.

Bonus (opcional)

- Calculá la frecuencia de cada palabra (sin distinguir mayúsculas) y mostrá las 3 más usadas ignorando artículos y preposiciones.
- Implementá un índice de legibilidad simple: promedio de sílabas por palabra aproximado por grupos de vocales.

Cierre: Es el mismo tipo de conteo que hacen los procesadores de texto y las herramientas de análisis de legibilidad.

Ejercicio 79: La parada de colectivo más cercana

Nivel 2: Junior II · Colecciones y funciones · Temática: Transporte · Dificultad 16/100

Situación: La municipalidad de una ciudad mediana está armando un bot que responde "¿cuál es la parada más cercana?". Las paradas de la línea principal están todas sobre la misma avenida, identificadas por su altura. El usuario manda la altura donde está y el bot responde.

Datos de entrada

Parada	Altura
Plaza Moreno	300
Hospital	1150
Escuela 12	1900
Mercado	2600
Club Atlético	3350
Terminal	4200

Usuario	Altura donde está
Clara	2347
Mateo	2250
Rita	5000

Radio de caminata aceptable: 400 m (1 unidad de altura = 1 m).

Tu programa debe

1. Para cada usuario, recorrer las paradas y calcular la distancia como la diferencia absoluta de alturas.
2. Encontrar la parada más cercana y su distancia.
3. Listar las paradas a 400 m o menos, en el orden de la lista.

Salida esperada

Clara (altura 2347): más cercana Mercado a 253 m | a menos de 400 m: Mercado
Mateo (altura 2250): más cercana Escuela 12 a 350 m | a menos de 400 m: Escuela 12, Mercado
Rita (altura 5000): más cercana Terminal a 800 m | a menos de 400 m: ninguna

Reglas

- Alcanza con búsqueda lineal, valor absoluto y filtrado.
- Si dos paradas están a la misma distancia, se elige la primera de la lista.
- Si ninguna está a 400 m o menos, se muestra "ninguna".
- No asumas que el usuario está entre dos paradas: puede estar antes de la primera o después de la última.

Pista (leela solo si te trabás): el valor absoluto lo podés calcular vos: si la resta es negativa, cambiale el signo.

Bonus (opcional)

- Como las paradas están ordenadas por altura, encontrá la más cercana sin recorrer toda la lista (vas a ver la técnica en el nivel 4).
- Estimá el tiempo de caminata a 80 m por minuto y el tiempo de espera según la frecuencia de la línea.

Cierre: Es la versión en una dimensión de lo que hacen las apps de mapas cuando te sugieren la parada más cercana.

Ejercicio 80: Inventario de pociones

Nivel 2: Junior II · Colecciones y funciones · Temática: Videojuegos · Dificultad 16/100

Situación: En un juego de rol, el inventario tiene 6 casilleros y cada casillero apila hasta 10 pociones iguales. Los jugadores reportan que a veces las pociones "desaparecen" o se duplican. Te toca reescribir la lógica del inventario y probarla con una secuencia de acciones conocida.

Datos de entrada

Inventario inicial (en orden de casilleros): Vida 8 | Maná 10 | Veneno 2

Paso	Acción	Poción	Cantidad
1	agregar	Vida	5
2	usar	Maná	4
3	agregar	Fuerza	12
4	agregar	Maná	7
5	usar	Vida	12
6	usar	Veneno	5

Reglas del inventario:

- Al agregar: primero se completan, en orden, los casilleros existentes de esa poción; lo que sobra ocupa casilleros nuevos al final (de a 10 como máximo). Si no hay más casilleros libres, lo que no entra se pierde.
- Al usar: si entre todos los casilleros de esa poción no alcanza, no se usa nada. Si alcanza, se consume desde el último casillero de esa poción hacia el primero, y los casilleros que quedan en 0 desaparecen.

Tu programa debe

1. Guardar el inventario como una lista de casilleros (poción y cantidad).
2. Implementar la acción agregar según las reglas e informar si se perdió algo.
3. Implementar la acción usar según las reglas e informar si no alcanza.
4. Procesar la secuencia y mostrar el inventario final.

Salida esperada

```
agregar Vida x5 -> ok
usar Maná x4 -> ok
agregar Fuerza x12 -> ok
agregar Maná x7 -> se perdieron 3 (inventario lleno)
usar Vida x12 -> ok
usar Veneno x5 -> no alcanza (hay 2)
Inventario final: Vida 1 | Maná 10 | Veneno 2 | Fuerza 10 | Fuerza 2 (5/6 casilleros)
```

Reglas

- Alcanza con listas, búsqueda lineal y actualización de elementos.
- La cantidad total de pociones nunca puede cambiar salvo por las acciones (ni aparecer ni desaparecer de más).
- El orden de los casilleros se conserva cuando se elimina uno vacío.
- Formato de cada línea: acción, poción y cantidad ("x5"), seguido del resultado.

Pista (leela solo si te trabás): separá cada acción en una función. Para usar, primero calculá cuánto hay en total de esa poción: si no alcanza, salí sin tocar nada.

Bonus (opcional)

- Agregá la acción "ordenar": juntar todas las pilas incompletas de la misma poción para liberar casilleros.
- Agregá un peso por poción y un peso máximo del inventario.

Cierre: Es la misma lógica de pilas y casilleros que tienen los inventarios de la mayoría de los juegos de rol y supervivencia.

Ejercicio 81: El CBU que no cierra

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 17/100

Situación: Una billetera virtual recibe miles de pedidos de transferencia por día y muchos fallan porque el CBU está mal tipeado. En vez de mandar la transferencia y esperar el rechazo del banco, quieren validar el CBU apenas el usuario lo escribe, usando sus dos dígitos verificadores.

Datos de entrada

CBU a validar (algunos tienen errores a propósito; son números ficticios):

1. 0720415388000012345678
2. 2850590940090412345608
3. 0720415388100012345678
4. 2855090940090412345608
5. 072041538800001234567
6. 07204A5388000012345678

Estructura y verificación:

- El CBU tiene 22 dígitos, en dos bloques: el bloque 1 son los dígitos 1 a 8 y el bloque 2, los dígitos 9 a 22.
- Bloque 1: los primeros 7 dígitos se multiplican, uno a uno, por los pesos 7, 1, 3, 9, 7, 1, 3 y se suman los productos. Dígito verificador = $(10 - (\text{suma resto } 10)) \text{ resto } 10$. Tiene que coincidir con el dígito 8.
- Bloque 2: los primeros 13 dígitos se multiplican por los pesos 3, 9, 7, 1, 3, 9, 7, 1, 3, 9, 7, 1, 3, con la misma fórmula. Tiene que coincidir con el dígito 22.

Tu programa debe

1. Escribir una función que calcule el dígito verificador de una secuencia de dígitos con una lista de pesos dada.
2. Escribir una función que valide un CBU: largo, que sean todos dígitos y los dos verificadores.
3. Mostrar el resultado de cada CBU indicando el motivo cuando es inválido.

Salida esperada

```
0720415388000012345678: válido
2850590940090412345608: válido
0720415388100012345678: inválido (no cierra bloque 2)
2855090940090412345608: válido
072041538800001234567: inválido (tiene 21 dígitos, deben ser 22)
07204A5388000012345678: inválido (contiene caracteres que no son dígitos)
```

Reglas

- Alcanza con funciones, recorrido de texto y listas de pesos.
- El CBU se trata como texto (un entero perdería los ceros a la izquierda).
- Orden de validación: primero el largo, después los caracteres y al final los verificadores. Si fallan los dos bloques, se informan los dos ("no cierra bloque 1 ni bloque 2").
- La misma función de dígito verificador sirve para los dos bloques: solo cambian los dígitos y los pesos.

Pista (leela solo si te trabás): el truco del $(10 - \text{s resto } 10) \text{ resto } 10$ es para que, cuando la suma ya es múltiplo de 10, el dígito sea 0 y no 10.

Bonus (opcional)

- Extraé el código de banco (primeros 3 dígitos) y mostrá el nombre del banco a partir de una tabla que armes vos.
- El CBU 4 es el CBU 2 con dos dígitos intercambiados y aun así da válido: explicá por qué el dígito verificador no detecta ese error y qué pesos lo habrían detectado.

Cierre: Es la misma validación que hacen los homebanking y las billeteras virtuales antes de mandar una transferencia.

Ejercicio 82: Planilla de asistencia del mes

Nivel 2: Junior II · Colecciones y funciones · Temática: Recursos humanos · Dificultad 17/100

Situación: En una planta de logística, el presentismo se paga solo si el empleado no faltó y llegó tarde como mucho dos veces en la quincena. La encargada tiene la planilla de asistencia y necesita el cálculo para mandar a liquidación, además de saber qué día hubo más faltas.

Datos de entrada

Asistencia de 10 días hábiles (P = presente, A = ausente, T = tarde):

Empleado	1	2	3	4	5	6	7	8	9	10
Aguirre	P	P	P	P	P	T	P	P	P	P
Benítez	P	A	P	P	P	P	T	T	P	P
Castro	P	P	P	P	P	P	P	P	P	P
Díaz	T	P	A	P	P	P	T	T	P	P
Escobar	P	P	A	A	P	P	P	P	P	P

Tu programa debe

1. Guardar la planilla en una matriz (empleados × días).
2. Para cada empleado, contar ausencias y tardanzas y calcular el porcentaje de asistencia (días no ausentes / días).
3. Contar los ausentes de cada día y encontrar el día con más ausencias.
4. Listar a quienes cobran presentismo (0 ausencias y como máximo 2 tardanzas).

Salida esperada

```
Aguirre  ausencias 0 | tardanzas 1 | asistencia 100.0%
Benítez  ausencias 1 | tardanzas 2 | asistencia 90.0%
Castro   ausencias 0 | tardanzas 0 | asistencia 100.0%
Díaz     ausencias 1 | tardanzas 3 | asistencia 90.0%
Escobar  ausencias 2 | tardanzas 0 | asistencia 80.0%
Ausentes por día: 0 1 2 1 0 0 0 0 0 0
Día con más ausencias: 3 (2)
Cobran presentismo: Aguirre, Castro
```

Reglas

- Alcanza con matrices y contadores por fila y por columna.
- Llegar tarde cuenta como día asistido para el porcentaje.
- Porcentaje con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Nombre alineado a la izquierda en 8 caracteres; en empates de días, el primero.

Pista (leela solo si te trabás): las ausencias por empleado son conteos por fila; los ausentes por día son conteos por columna. El mismo recorrido anidado sirve para ambos si llevás dos juegos de contadores.

Bonus (opcional)

- Detectá empleados con 2 o más ausencias en días consecutivos.
- Calculá el ranking de puntualidad (menos tardanzas primero).

Cierre: Es la misma lógica que aplican los sistemas de control de asistencia y fichaje.

Ejercicio 83: Transportar acordes un tono arriba

Nivel 2: Junior II · Colecciones y funciones · Temática: Música · Dificultad 17/100

Situación: Una cantante del coro de la parroquia no llega a las notas altas de una canción y el guitarrista necesita bajar... en realidad subir, la tonalidad un tono entero para que quede cómoda para ella. Transportar a mano una canción larga es tedioso y fácil de errar, así que te piden un transportador de acordes.

Datos de entrada

Escala cromática (12 semitonos, circular): Do, Do#, Re, Re#, Mi, Fa, Fa#, Sol, Sol#, La, La#, Si

Equivalencias de bemoles: Reb = Do#, Mib = Re#, Solb = Fa#, Lab = Sol#, Sib = La#

Acordes de la canción, en orden: Do Lam Fa Sol7 Mim Re/Fa# Sib Mib Si7

Transporte: +2 semitonos (un tono). Un acorde tiene una nota raíz (por ejemplo Sol o Fa#), seguida de un sufijo que no cambia (m, 7, m7...). Si tiene barra, como Re/Fa#, la nota después de la barra también se transporta.

Tu programa debe

1. Separar cada acorde en nota raíz y sufijo (y el bajo, si tiene barra).
2. Convertir los bemoles a su equivalente con sostenido.
3. Transportar cada nota sumando semitonos de forma circular sobre la escala.
4. Mostrar los acordes transportados y verificar que al transportar -2 se vuelve a los originales (expresados con sostenidos).

Salida esperada

```
Original:      Do Lam Fa Sol7 Mim Re/Fa# Sib Mib Si7
Un tono arriba: Re Sim Sol La7 Fa#m Mi/Sol# Do Fa Do#7
Vuelta atrás:  Do Lam Fa Sol7 Mim Re/Fa# La# Re# Si7
```

Reglas

- Alcanza con listas, búsqueda de posición y aritmética circular (resto 12).
- Cuidado al reconocer la raíz: "Sol" tiene tres letras y "Si" dos; "Sib" es Si bemol, no Si con sufijo "b".
- El resultado se expresa siempre con sostenidos.
- Tiene que funcionar con cualquier cantidad de semitonos, positiva o negativa.

Pista (leela solo si te trabás): buscá la raíz probando los nombres de notas contra el principio del acorde, y después mirá si el carácter siguiente es # o b.

Bonus (opcional)

- Elegí entre sostenidos y bemoles según la tonalidad de destino (por ejemplo, en Fa se prefiere Sib antes que La#).
- Detectá automáticamente la tonalidad más probable de la canción según qué acordes aparecen.

Cierre: Es la misma función de "transportar" que tienen los sitios y apps de acordes para músicos.

Ejercicio 84: Pedidos atrasados del día

Nivel 2: Junior II · Colecciones y funciones · Temática: Logística y envíos · Dificultad 17/100

Situación: Una dietética con envíos a domicilio promete una franja horaria a cada cliente. Son las 18:30 y la dueña quiere ver qué pedidos llegaron tarde, cuáles siguen pendientes y ya están atrasados, y qué repartidor acumula más atrasos para hablar con él.

Datos de entrada

Hora actual: 18:30

Pedido	Repartidor	Hora comprometida	Hora de entrega
P-501	Lucas	11:00	10:45
P-502	Mora	12:30	13:10
P-503	Lucas	14:00	pendiente
P-504	Nadia	15:00	15:00
P-505	Mora	16:30	17:25
P-506	Nadia	19:00	pendiente
P-507	Lucas	17:45	18:20

Tu programa debe

1. Para cada pedido entregado, calcular los minutos de atraso (si los hay).
2. Para cada pedido pendiente, determinar si ya está atrasado respecto de la hora actual.
3. Contar los pedidos atrasados (entregados tarde + pendientes atrasados).
4. Informar el repartidor con más atrasos.

Salida esperada

```
P-501 (Lucas): entregado a tiempo
P-502 (Mora): entregado con 40 min de atraso
P-503 (Lucas): PENDIENTE y atrasado 270 min
P-504 (Nadia): entregado a tiempo
P-505 (Mora): entregado con 55 min de atraso
P-506 (Nadia): pendiente, en término
P-507 (Lucas): entregado con 35 min de atraso
Pedidos atrasados: 4 de 7
Repartidor con más atrasos: Lucas (2)
```

Reglas

- Alcanza con listas de registros, comparación de horas y contadores.
- Entregar exactamente a la hora comprometida es "a tiempo".
- Un pendiente cuya hora comprometida es posterior a la actual está "en término".
- Si dos repartidores empatan, se informa el que aparece primero en la lista de pedidos.

Pista (leela solo si te trabás): pasá todas las horas a minutos desde la medianoche con una función; así "llegó tarde" es solo comparar dos números.

Bonus (opcional)

- Calculá el atraso promedio por repartidor.
- Avisá qué pendientes van a atrasarse en los próximos 30 minutos si no se entregan.

Cierre: Es el mismo tablero de control de SLA (nivel de servicio comprometido) que usan las empresas de delivery y logística.

Ejercicio 85: El comentario con más reacciones

Nivel 2: Junior II · Colecciones y funciones · Temática: Redes sociales · Dificultad 17/100

Situación: Una cuenta de recetas quiere fijar arriba el mejor comentario de cada publicación. Discuten si es el que tiene más reacciones o el que genera reacciones más positivas, porque un comentario polémico junta muchos "enojos". Tu programa calcula las dos cosas y, de paso, quién comenta más.

Datos de entrada

Autor	Comentario	Me gusta	Corazones	Risas	Enojos
@lucho_cocina	¡Qué buena receta!	120	45	3	0
@marce.viajes	No me salió igual...	34	2	18	6
@tati_fit	La hago todos los domingos	210	88	1	2
@lucho_cocina	Probala con nuez	56	30	0	0
@nico.ok	Me parece carísima	15	0	5	41
@tati_fit	Gracias por compartir	98	120	4	0

Puntaje = me gusta + 2 × corazones + risas – 3 × enojos

Tu programa debe

1. Guardar los comentarios como una lista de registros.
2. Calcular para cada uno el total de reacciones y el puntaje.
3. Encontrar el comentario con más reacciones y el de mejor puntaje.
4. Encontrar el autor con más comentarios.

Salida esperada

```
@lucho_cocina total 168 | puntaje 213 | "¡Qué buena receta!"
@marce.viajes total 60 | puntaje 38 | "No me salió igual..."
@tati_fit total 301 | puntaje 381 | "La hago todos los domingos"
@lucho_cocina total 86 | puntaje 116 | "Probala con nuez"
@nico.ok total 61 | puntaje -103 | "Me parece carísima"
@tati_fit total 222 | puntaje 342 | "Gracias por compartir"
Más reacciones: @tati_fit - "La hago todos los domingos"
Mejor puntaje: @tati_fit - "La hago todos los domingos"
Autor más activo: @lucho_cocina (2 comentarios)
```

Reglas

- Alcanza con listas de registros, máximos y conteo.
- En empates, gana el que aparece primero.
- Formato: autor en 14 caracteres alineado a la izquierda, total en 3 y puntaje en 4 alineados a la derecha.

Pista (leela solo si te trabás): para el autor más activo, primero armá la lista de autores sin repetir y después contá cuántas veces aparece cada uno.

Bonus (opcional)

- Calculá el porcentaje de reacciones negativas de cada comentario y marcá los polémicos (más del 30 %).
- Agregá una penalización por antigüedad: el puntaje baja 5 % por cada hora desde que se publicó.

Cierre: Es una versión simple de los algoritmos que deciden qué comentario ves primero en las redes sociales.

Ejercicio 86: Glucemia de la semana

Nivel 2: Junior II · Colecciones y funciones · Temática: Salud · Dificultad 17/100

Situación: Graciela usa un glucómetro que guarda las mediciones pero no las resume. Su diabetóloga le pidió que lleve a la consulta cuántas mediciones de la semana quedaron fuera de los objetivos acordados y que marque cualquier valor bajo, que es lo más urgente. (Valores de objetivo ilustrativos; cada persona tiene los suyos.)

Datos de entrada

Día	Momento	Glucemia (mg/dl)
Lun	ayunas	112
Lun	post	168
Mar	ayunas	125
Mar	post	192
Mié	ayunas	98
Mié	post	154
Jue	ayunas	66
Jue	post	141
Vie	ayunas	134
Vie	post	205
Sáb	ayunas	118
Sáb	post	176
Dom	ayunas	104
Dom	post	149

Objetivos acordados: en ayunas, de 70 a 130; post comida (2 horas después), menos de 180. Cualquier valor menor que 70 es hipoglucemia.

Tu programa debe

1. Recorrer las mediciones y marcar las alertas de hipoglucemia (menos de 70), sea cual sea el momento.
2. Marcar las que están fuera de objetivo según su momento.
3. Calcular el promedio de las mediciones en ayunas y el de las post comida.
4. Calcular cuántas mediciones están en objetivo y su porcentaje.

Salida esperada

```
Fuera de objetivo: Mar post (192 mg/dl)
ALERTA HIPOGLUCEMIA: Jue ayunas (66 mg/dl)
Fuera de objetivo: Vie ayunas (134 mg/dl)
Fuera de objetivo: Vie post (205 mg/dl)
Promedio en ayunas: 108.1 mg/dl
Promedio post comida: 169.3 mg/dl
Mediciones en objetivo: 10 de 14 (71.4%)
```

Reglas

- Alcanza con listas, filtrado, promedios y alertas por umbral.
- La alerta de hipoglucemia tiene prioridad: una medición marcada así no se vuelve a marcar como "fuera de objetivo".
- Promedios y porcentajes con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Los avisos se muestran en el orden de las mediciones.

Pista (leela solo si te trabás): el orden de los condicionales resuelve la prioridad: primero preguntá por hipoglucemia y solo si no lo es, evaluá el objetivo según el momento.

Bonus (opcional)

- Calculá el tiempo en rango: porcentaje de mediciones entre 70 y 180 sin importar el momento.
- Detectá patrones: si la medición post comida supera en más de 60 a la de ayunas del mismo día, avisalo.

Cierre: Es el mismo tipo de resumen que generan las apps de los glucómetros y los monitores continuos de glucosa.

Ejercicio 87: Tabla de posiciones sin ordenar

Nivel 2: Junior II · Colecciones y funciones · Temática: Deportes · Dificultad 18/100

Situación: Un diario del interior publica la tabla de la liga regional, pero el sistema que la arma a veces declara un solo puntero cuando hay empate. El editor quiere un programa que detecte empates en la punta y los resuelva con los criterios del reglamento, sin ordenar la tabla.

Datos de entrada

Equipo	Puntos	Diferencia de gol	Goles a favor
Atlético Sur	31	+8	29
Unión Norte	34	+11	33
Deportivo Río	34	+11	30
Sportivo Este	22	-2	20
Juventud	18	-9	15
Ferro Oeste	25	+3	24
Racing Club Local	18	-5	17
Independiente del Valle	27	+6	26

Criterios: más puntos; si empatan, mejor diferencia de gol; si siguen empatados, más goles a favor.

Tu programa debe

1. Encontrar el máximo de puntos y todos los equipos que lo tienen.
2. Si hay más de uno, filtrar por mejor diferencia de gol; si sigue el empate, por goles a favor.
3. Mostrar cada paso del desempate que haga falta y el puntero (o el empate total).
4. Encontrar el mínimo de puntos y todos los equipos que están últimos.

Salida esperada

Máximo de puntos: 34 (Unión Norte, Deportivo Río)
 Desempate por diferencia de gol (+11): Unión Norte, Deportivo Río
 Desempate por goles a favor (33): Unión Norte
 Puntero: Unión Norte
 Último con 18 puntos: Juventud, Racing Club Local

Reglas

- Alcanza con listas paralelas, máximos, mínimos y filtrado.
- No ordenes la tabla: la idea es resolverlo con recorridos.
- Los equipos empatados se listan en el orden de la tabla original.
- Solo se muestran las líneas de desempate que se usaron.

Pista (leela solo si te trabás): cada criterio de desempate es el mismo patrón: buscá el máximo del criterio entre los candidatos y quedate solo con los que lo alcanzan.

Bonus (opcional)

- Agregá un cuarto criterio: resultado entre los equipos empatados, a partir de una lista de partidos.
- Calculá cuántos puntos le faltan a cada equipo para alcanzar al puntero.

Cierre: Es la misma lógica de desempate que aplican los sistemas que generan tablas de posiciones oficiales.

Ejercicio 88: Cuánto rinde cada alternativa de ahorro

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 18/100

Situación: Paula ganó \$1.000.000 en un concurso y quiere invertirlo por un año. Tiene cuatro alternativas y cada una informa el rendimiento de forma distinta, así que no se pueden comparar a simple vista. Tu programa lleva todas a la misma pregunta: ¿cuánta plata tengo dentro de un año?

Datos de entrada

Alternativa	Cómo rinde
Plazo fijo	TNA 33 %, a 30 días, renovando capital + interés 12 veces (tasa mensual = TNA / 12)
Billetera virtual	TNA 32 %, interés diario compuesto durante 365 días (tasa diaria = TNA / 365)
Fondo común	TNA 30 %, interés diario compuesto durante 365 días
Dólares	comprar a \$1.200, vender dentro de un año a \$1.500 estimado; 1 % de comisión al comprar y 1 % al vender

Tu programa debe

1. Escribir una función de interés compuesto: capital, tasa por período y cantidad de períodos.
2. Usarla para calcular el monto final de las tres alternativas en pesos.
3. Calcular el monto final de la alternativa en dólares.
4. Mostrar el monto final y el rendimiento porcentual de cada una, y cuál es la mejor.

Salida esperada

Plazo fijo renovado (TNA 33%, 12 renovaciones mensuales): \$1384783.78 (+38.48%)
 Billetera (TNA 32%, interés diario, 365 días): \$1376934.72 (+37.69%)
 Fondo común (TNA 30%, interés diario, 365 días): \$1349692.49 (+34.97%)
 Dólares (compra \$1200, venta estimada \$1500, 1% de comisión en cada operación): \$1225125.00 (+22.51%)
 Mejor alternativa: Plazo fijo renovado

Reglas

- Alcanza con funciones, bucles y porcentajes.
- Implementá la potencia con un bucle dentro de la función (sin funciones de potencia ya hechas).
- Montos y porcentajes con 2 decimales, redondeando solo al mostrar (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Usá números decimales con buena precisión: 365 multiplicaciones acumulan error si trabajás con poca precisión.

Pista (leela solo si te trabás): las tres primeras alternativas son la misma función con distintos argumentos. La de dólares es: pesos → dólares (menos comisión) → pesos (menos comisión).

Bonus (opcional)

- Calculá a qué precio del dólar dentro de un año da lo mismo que la mejor alternativa en pesos.
- Agregá una inflación anual del 25 % y mostrá el rendimiento real de cada alternativa.

Cierre: Es la comparación que hacen los simuladores de inversión de los brokers y las apps de finanzas personales.

Ejercicio 89: Sopa de letras de la escuela

Nivel 2: Junior II · Colecciones y funciones · Temática: Educación · Dificultad 18/100

Situación: Una maestra de matemática arma sopas de letras con vocabulario de geometría para sus alumnos de quinto. Antes de imprimirlas, quiere confirmar que todas las palabras estén de verdad en la grilla (una vez imprimió una sopa con una palabra que no estaba y fue un escándalo).

Datos de entrada

Grilla de 8 × 8:

```
N U M E R O Q T
A L K S U M A R
Z R E S T A P A
A N G U L O W D
F E X O Y L J I
R E G L A A B O
O V I S U D M U
P L T K Q O Z L
```

Palabras a buscar: NUMERO, SUMA, RESTA, ANGULO, REGLA, RADIO, LADO, COMPAS

Tu programa debe

1. Guardar la grilla en una matriz de caracteres.
2. Para cada palabra, buscarla horizontalmente (de izquierda a derecha) y verticalmente (de arriba hacia abajo).
3. Informar la fila, la columna de la primera letra y la dirección, o "no está".
4. Mostrar cuántas palabras se encontraron.

Salida esperada

```
NUMERO: fila 1, columna 1, horizontal
SUMA: fila 2, columna 4, horizontal
RESTA: fila 3, columna 2, horizontal
ANGULO: fila 4, columna 1, horizontal
REGLA: fila 6, columna 1, horizontal
RADIO: fila 2, columna 8, vertical
LADO: fila 5, columna 6, vertical
COMPAS: no está
Encontradas: 7 de 8
```

Reglas

- Alcanza con matrices, bucles anidados y comparación carácter a carácter.
- Filas y columnas se informan empezando en 1.
- Se recorren las posiciones de inicio fila por fila y, dentro de cada fila, columna por columna; en cada posición se prueba primero horizontal y después vertical. Se informa la primera coincidencia.
- Antes de comparar, verificá que la palabra entre sin salirse de la grilla.

Pista (leela solo si te trabás): escribí una función que reciba la grilla, una posición de inicio, una dirección (cuánto avanzar en fila y en columna por letra) y la palabra, y que diga si coincide. Así sumar diagonales es trivial.

Bonus (opcional)

- Buscá también en diagonal y en las direcciones inversas (8 direcciones en total).
- Generá la sopa de letras: ubicá las palabras al azar y completá el resto con letras aleatorias.

Cierre: Es el mismo tipo de búsqueda en grilla que usan los juegos de palabras y los algoritmos de reconocimiento de patrones en imágenes.

Ejercicio 90: Calorías del plato armado

Nivel 2: Junior II · Colecciones y funciones · Temática: Gastronomía · Dificultad 18/100

Situación: Una vianda saludable que entrega almuerzos a oficinas quiere mostrar en cada etiqueta las calorías y las proteínas del plato. La nutricionista tiene una tabla por cada 100 g de ingrediente, pero hacer la cuenta plato por plato le lleva horas.

Datos de entrada

Ingrediente	kcal cada 100 g	Proteína cada 100 g
arroz	130	2,7 g
pollo	165	31 g
brócoli	34	2,8 g
palta	160	2 g
aceite de oliva	884	0 g
huevo	155	13 g

Plato "Bowl de pollo": arroz 180 g, pollo 150 g, brócoli 120 g, aceite de oliva 10 g, queso 30 g

Tu programa debe

1. Escribir una función que, dado un ingrediente y sus gramos, devuelva calorías y proteínas buscando en la tabla.
2. Recorrer el plato acumulando calorías y proteínas; si un ingrediente no está en la tabla, avisar y seguir.
3. Mostrar el detalle por ingrediente y los totales.
4. Calcular qué porcentaje de una dieta de 2.000 kcal representa el plato.

Salida esperada

```
arroz (180 g): 234.0 kcal | 4.9 g de proteína
pollo (150 g): 247.5 kcal | 46.5 g de proteína
brócoli (120 g): 40.8 kcal | 3.4 g de proteína
aceite de oliva (10 g): 88.4 kcal | 0.0 g de proteína
queso (30 g): no está en la tabla, no se suma
Total: 610.7 kcal | 54.7 g de proteína
Representa el 30.5% de una dieta de 2000 kcal
```

Reglas

- Alcanza con listas, búsqueda lineal (o en una tabla de dos columnas), acumuladores y funciones.
- Todos los valores se muestran con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Los ingredientes que no están en la tabla no frenan el programa: se informan y no suman.
- La búsqueda de ingredientes distingue el nombre exacto.

Pista (leela solo si te trabás): la regla de tres es la misma para calorías y proteínas: valor cada 100 g × gramos / 100.

Bonus (opcional)

- Agregá hidratos y grasas y mostrá qué porcentaje de las calorías aporta cada macronutriente (4, 4 y 9 kcal por gramo).
- Dado un objetivo de 600 kcal, calculá cuántos gramos de arroz agregar o sacar para llegar exacto.

Cierre: Es el mismo cálculo que hacen las apps de nutrición y los sistemas de rotulado de alimentos.

Ejercicio 91: SMS sospechosos en la bandeja

Nivel 2: Junior II · Colecciones y funciones · Temática: Telecomunicaciones · Dificultad 18/100

Situación: Una compañía telefónica recibe denuncias por SMS de estafa: falsos premios, cuentas "bloqueadas" y el clásico "cambié de número". Quieren un filtro simple que puntúe cada mensaje según palabras clave y marque los sospechosos antes de entregarlos.

Datos de entrada

Palabra clave	Puntos
urgente	2
premio	3
clave	3
http	3
bloqueada	2
transferí	2
ganaste	3
token	3

Mensajes:

1. Hola ma, llego a las 8. Besos
2. URGENTE: tu cuenta fue bloqueada. Ingresá tu clave en <http://bit.ly/x9>
3. Ganaste un premio de \$500.000! Reclamalo en <http://premios-ya.net>
4. Tu código de verificación es 482913. No lo compartas.
5. Soy tu hijo, cambié de número. Transferí urgente que después te explico
6. Tu token es 7731. Dictáselo a nuestro operador

Un mensaje es sospechoso si suma 4 puntos o más.

Tu programa debe

1. Pasar cada mensaje a minúsculas.
2. Buscar cada palabra clave dentro del mensaje y sumar sus puntos (cada palabra clave suma una sola vez por mensaje).
3. Marcar como SOSPECHOSO los que sumen 4 o más, mostrando las palabras encontradas en el orden de la tabla.
4. Contar cuántos mensajes son sospechosos.

Salida esperada

```
SMS 1: 0 puntos -> ok
SMS 2: 10 puntos [urgente, clave, http, bloqueada] -> SOSPECHOSO
SMS 3: 9 puntos [premio, http, ganaste] -> SOSPECHOSO
SMS 4: 0 puntos -> ok
SMS 5: 4 puntos [urgente, transferí] -> SOSPECHOSO
SMS 6: 3 puntos [token] -> ok
Sospechosos: 3 de 6
```

Reglas

- Alcanza con recorrido de texto, búsqueda de subcadenas y acumuladores.
- La búsqueda de una palabra clave dentro del texto la implementás vos, comparando carácter a carácter desde cada posición posible.
- La coincidencia puede estar dentro de otra palabra ("clave" aparece en "claves").

Pista (leela solo si te trabás): buscar una subcadena es probar cada posición de inicio del texto y comparar letra por letra con la palabra clave. Si todas coinciden, la encontraste.

Bonus (opcional)

- Normalizá las tildes antes de buscar ("transferi" sin tilde también tiene que detectarse).
- Detectá números de 6 dígitos o más y enlaces aunque no empiecen con "http" (por ejemplo, "bit.ly").

Cierre: Es la versión más simple de los filtros antispam y antifraude que aplican las operadoras y los servicios de mensajería.

Ejercicio 92: Parcelas con sequía

Nivel 2: Junior II · Colecciones y funciones · Temática: Agricultura · Dificultad 19/100

Situación: Un viñedo de Mendoza instaló sensores de humedad del suelo en una grilla de 5 hileras x 6 parcelas. El encargado de riego necesita saber qué parcelas están en sequía, cuáles en riesgo y qué hilera priorizar, porque el agua del turno de riego no alcanza para todas.

Datos de entrada

Humedad del suelo (%), filas = hileras, columnas = parcelas:

Hilera	1	2	3	4	5	6
1	34	28	19	15	22	40
2	31	18	12	17	26	38
3	36	33	21	19	30	41
4	42	37	29	24	35	44
5	45	39	32	27	38	47

Clasificación: menos de 20 → sequía; de 20 a 35 inclusive → riesgo; más de 35 → óptima.

Tu programa debe

1. Guardar las mediciones en una matriz.
2. Clasificar cada parcela y contar cuántas hay de cada tipo.
3. Listar las coordenadas (hilera, parcela) de las parcelas en sequía, recorriendo por hileras.
4. Informar la hilera con más parcelas en sequía.

Salida esperada

Sequía: 6 | Riesgo: 13 | Óptima: 11
Parcelas en sequía: (1,3) (1,4) (2,2) (2,3) (2,4) (3,4)
Hilera más afectada: 2 (3 parcelas en sequía)

Reglas

- Alcanza con matrices, umbrales y contadores.
- Las coordenadas empiezan en 1 y se muestran como (hilera,parcela) separadas por un espacio.
- Si dos hileras empatan, se informa la primera.

Pista (leela solo si te trabás): un solo recorrido anidado alcanza para todo: clasificás, contás y, al terminar cada fila, comparás su contador de sequía con el mejor visto.

Bonus (opcional)

- Calculá cuántos litros hacen falta para llevar cada parcela en sequía al 30 %, si cada punto de humedad requiere 120 litros.
- Detectá manchas de sequía: parcelas en sequía con al menos una vecina (arriba, abajo, izquierda o derecha) también en sequía.

Cierre: Es la base de los sistemas de riego de precisión que usan sensores distribuidos en el campo.

Ejercicio 93: El robot del depósito sigue instrucciones

Nivel 2: Junior II · Colecciones y funciones · Temática: Logística y envíos · Dificultad 19/100

Situación: Un depósito de repuestos está probando un robot que lleva cajas entre estanterías. Recibe una cadena de instrucciones simples y, si una instrucción lo haría chocar con una estantería o salirse del depósito, se queda quieto y registra el incidente. Antes de probarlo en el depósito real, quieren simular el recorrido.

Datos de entrada

Mapa (# = estantería, . = pasillo). El robot arranca en la esquina superior izquierda (fila 1, columna 1) mirando al Este:

```
.....
.##...
....#.
#.....
...##.
```

Instrucciones: AADAAAIAADAAAIIA

- A: avanzar una celda en la dirección en que mira.
- D: girar 90° a la derecha (sin moverse).
- I: girar 90° a la izquierda (sin moverse).

Tu programa debe

1. Guardar el mapa en una matriz y el estado del robot (fila, columna, dirección).
2. Ejecutar cada instrucción; si avanzar lo sacaría del mapa o lo metería en una estantería, no se mueve y se cuenta un choque evitado.
3. Marcar con * cada celda por la que pasó (incluida la inicial) y con @ su posición final.
4. Mostrar el mapa del recorrido, la posición y dirección final y los choques evitados.

Salida esperada

```
****@.
.##.*.
....#.
#.....
...##.
Posición final: fila 1, columna 5, mirando al Norte
Choques evitados: 6
```

Reglas

- Alcanza con matrices, variables de estado y un bucle sobre el texto de instrucciones.
- Las direcciones rotan en el orden Este → Sur → Oeste → Norte → Este al girar a la derecha.
- Posiciones empezando en 1 al mostrarlas.

Pista (leela solo si te trabás): guardá las 4 direcciones en una lista de desplazamientos (cuánto cambia la fila y la columna). Girar es sumar o restar 1 al índice de dirección, con resto 4.

Bonus (opcional)

- Agregá la instrucción "Rn": repetir la instrucción siguiente n veces.
- Calculá la secuencia de instrucciones más corta para llevar el robot a una celda destino (lo vas a poder resolver bien en el nivel 6).

Cierre: Es el mismo tipo de simulación que se usa para probar robots de almacenes automatizados antes de ponerlos a andar.

Ejercicio 94: La acción que más subió en la semana

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 19/100

Situación: Un newsletter financiero sale todos los viernes con el resumen de la semana de un índice de cinco empresas (ficticias). El redactor quiere automatizar los números: cuánto subió o bajó cada acción, cuál fue la mejor y la peor, y cuál tuvo el salto diario más grande.

Datos de entrada

Acción	Lun	Mar	Mié	Jue	Vie
AGRO	1520	1548	1533	1590	1612
BANK	842	830	815	861	858
ENRG	2310	2295	2400	2388	2455
TECH	5100	5320	5210	5050	5185
TELC	612	605	598	590	601

(Precios de cierre en pesos.)

Tu programa debe

1. Calcular la variación porcentual semanal de cada acción: $(\text{viernes} - \text{lunes}) / \text{lunes} \times 100$.
2. Encontrar la que más subió y la que más bajó.
3. Calcular la variación de cada día respecto del anterior y encontrar el mayor salto diario de toda la tabla (acción y día, con lunes = día 1).

Salida esperada

```
AGRO: 1520 -> 1612 (+6.05%)
BANK: 842 -> 858 (+1.90%)
ENRG: 2310 -> 2455 (+6.28%)
TECH: 5100 -> 5185 (+1.67%)
TELC: 612 -> 601 (-1.80%)
Más subió: ENRG (6.28%)
Más bajó: TELC (-1.80%)
Mayor salto diario: BANK el día 4 (+5.64%)
```

Reglas

- Alcanza con listas o matrices, variación porcentual, máximos y mínimos.
- Porcentajes con 2 decimales (0,5 hacia arriba), con signo + si son positivos o cero. Los decimales se muestran con punto y sin separador de miles en la salida.
- Las comparaciones se hacen con los valores sin redondear. En empates, gana el primero.

Pista (leela solo si te trabás): el salto diario es la misma fórmula que la variación semanal, pero entre días consecutivos: recorré cada fila desde la segunda columna.

Bonus (opcional)

- Calculá la volatilidad de cada acción como el promedio de los valores absolutos de sus variaciones diarias.
- Armá un índice ponderado (con pesos por empresa) y calculá su variación semanal.

Cierre: Es el mismo resumen que publican los portales financieros al cierre de cada semana bursátil.

Ejercicio 95: Horario de la medicación

Nivel 2: Junior II · Colecciones y funciones · Temática: Salud · Dificultad 19/100

Situación: Don Aldo toma cuatro medicamentos con horarios distintos y su hija le quiere pegar en la heladera una sola lista con todas las tomas del día en orden, desde la medianoche. Cada medicamento se toma cada cierta cantidad de horas a partir de una primera toma. (Nombres y esquemas ficticios.)

Datos de entrada

Medicamento	Primera toma	Cada (horas)	Tomas por día
Comprimido azul	08:00	8	3
Jarabe	07:30	12	2
Gotas	06:00	6	4
Comprimido blanco	22:00	24	1

Tu programa debe

1. Escribir una función que genere los horarios de un medicamento a partir de la primera toma, el intervalo y la cantidad de tomas.
2. Juntar todas las tomas del día.
3. Mostrarlas en orden horario desde las 00:00, agrupando en una misma línea los medicamentos que coinciden a la misma hora.
4. Mostrar la cantidad total de tomas.

Salida esperada

```
00:00 - Comprimido azul + Gotas
06:00 - Gotas
07:30 - Jarabe
08:00 - Comprimido azul
12:00 - Gotas
16:00 - Comprimido azul
18:00 - Gotas
19:30 - Jarabe
22:00 - Comprimido blanco
Tomas en el día: 10
```

Reglas

- Alcanza con funciones, listas y conversión de horas.
- Si un horario pasa de las 24:00, vuelve a empezar desde 00:00 (resto 1.440 minutos).
- Todavía no hace falta ordenar: podés recorrer los minutos del día de 0 a 1.439 y mostrar lo que corresponde a cada uno.
- Los medicamentos que coinciden se unen con " + " en el orden de la tabla.

Pista (leela solo si te trabás): trabajá con minutos desde la medianoche. Para mostrar en orden sin ordenar, el bucle de afuera recorre los minutos del día y el de adentro busca qué tomas caen en ese minuto.

Bonus (opcional)

- Detectá pares de medicamentos que no se pueden tomar con menos de 1 hora de diferencia y avisá si ocurre.
- Resolvé el orden con un algoritmo de ordenamiento en lugar de recorrer los 1.440 minutos, y compará cuántas operaciones hace cada enfoque.

Cierre: Es la misma lógica que usan las apps recordatorias de medicación y los pastilleros electrónicos.

Ejercicio 96: Sensores que marcan valores imposibles

Nivel 2: Junior II · Colecciones y funciones · Temática: Ciencia · Dificultad 19/100

Situación: En el laboratorio de una facultad de química, un sensor de temperatura de un reactor a veces devuelve basura: -127 cuando pierde conexión o picos absurdos por interferencias. Antes de calcular el promedio del experimento, hay que descartar las lecturas imposibles y las sospechosas.

Datos de entrada

Lecturas ($^{\circ}\text{C}$), en orden: 21,4, 21,9, 22,3, -127,0, 22,8, 23,1, 85,5, 23,6, 36,2, 24,0, 24,3, 24,1

- Rango físico del sensor: de -40°C a 85°C , inclusive.
- Una lectura dentro del rango es sospechosa si difiere en más de 10°C de la última lectura válida.

Tu programa debe

1. Recorrer las lecturas descartando las que están fuera del rango del sensor.
2. Descartar las que saltan más de 10°C respecto de la última lectura válida (no de la anterior cruda).
3. Mostrar cada lectura descartada con su motivo.
4. Mostrar cuántas lecturas quedaron y su promedio.

Salida esperada

```
Lectura 4: -127.0 -> descartada (fuera del rango del sensor)
Lectura 7: 85.5 -> descartada (fuera del rango del sensor)
Lectura 9: 36.2 -> descartada (salto de 12.6 °C)
Lecturas válidas: 9 de 12
Promedio de las válidas: 23.06 °C
```

Reglas

- Alcanza con listas, validación y filtrado.
- Temperaturas y saltos con 1 decimal; promedio con 2 (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- 85,0 exacto es válido; 85,5 no.
- La primera lectura válida no tiene con qué compararse: solo se valida su rango.

Pista (leela solo si te trabás): guardá la última lectura válida en una variable y actualizala solo cuando aceptás una lectura. Así un pico absurdo no arrastra a las siguientes.

Bonus (opcional)

- Reemplazá cada lectura descartada por el promedio de la válida anterior y la válida siguiente (interpolación).
- Si hay 3 lecturas descartadas seguidas, informá "posible falla del sensor".

Cierre: Es el mismo filtrado de datos que hacen los sistemas de adquisición en laboratorios y plantas industriales.

Ejercicio 97: Una generación del juego de la vida

Nivel 2: Junior II · Colecciones y funciones · Temática: Ciencia · Dificultad 19/100

Situación: En la clase de biología de un secundario simulan el crecimiento de una colonia de bacterias sobre una grilla, con reglas muy simples que igual producen patrones sorprendentes. La profe quiere un programa que calcule las generaciones siguientes para mostrarlas en el proyector.

Datos de entrada

Grilla inicial (# = viva, . = muerta), 6 × 6:

```
.....
.##...
.##...
....#.
....#.
....#.
```

Reglas (cada celda mira a sus 8 vecinas; fuera de la grilla cuenta como muerta):

- Una celda viva con 2 o 3 vecinas vivas sigue viva; si no, muere.
- Una celda muerta con exactamente 3 vecinas vivas nace.

Tu programa debe

1. Contar las vecinas vivas de cada celda, cuidando los bordes.
2. Calcular la generación 1 en una matriz nueva, aplicando las reglas a todas las celdas al mismo tiempo.
3. Calcular la generación 2 a partir de la 1.
4. Mostrar las tres generaciones con su cantidad de celdas vivas e indicar si la generación 2 es igual a la 0.

Salida esperada

Generación 0 (7 vivas):

```
.....
.##...
.##...
....#.
....#.
....#.
```

Generación 1 (9 vivas):

```
.....
.##...
.###..
...#..
...###
.....
```

Generación 2 (7 vivas):

```
.....
.#.#..
.#.#..
.....
...##.
....#.
```

La generación 2 es igual a la 0: no

Reglas

- Alcanza con matrices, conteo de vecinos y copia de matrices.
- No modifies la matriz mientras la recorres: los cambios tienen que ir a una matriz nueva (si no, una celda que nace afecta a sus vecinas en la misma generación).

- Entre cada generación va una línea en blanco.

Pista (leela solo si te trabás): escribí una función `vecinas(matriz, fila, columna)` y otra `siguiente(matriz)` que devuelva una matriz nueva. Llamando dos veces a `siguiente`, obtenés la generación 2.

Bonus (opcional)

- Hacé que la grilla sea "toroidal": el borde derecho se conecta con el izquierdo y el de arriba con el de abajo.
- Detectá cuándo la colonia entra en un ciclo (vuelve a un estado anterior) y de qué período.

Cierre: Es el Juego de la Vida de Conway, un clásico de la computación que se usa para estudiar sistemas complejos y autómatas celulares.

Ejercicio 98: Desafío de nivel: El resumen de la tarjeta de crédito

Nivel 2: Junior II · Colecciones y funciones · Temática: Finanzas y FinTech · Dificultad 20/100

Situación: Una fintech que emite tarjetas de crédito quiere un resumen que la gente entienda: muchos clientes pagan solo el mínimo sin saber que eso genera intereses. Tu programa arma el resumen del mes a partir de los movimientos y explica cada número. Es el desafío que integra todo el nivel 2.

Datos de entrada

Fecha	Descripción	Categoría	Importe
03/10	Supermercado	Supermercado	\$64.300
05/10	Nafta	Transporte	\$38.000
08/10	Heladera cuota 3/12	Hogar	\$95.000
12/10	Streaming	Entretenimiento	\$8.900
15/10	Farmacia	Salud	\$21.400
19/10	Supermercado	Supermercado	\$72.850
22/10	Cine	Entretenimiento	\$17.600
27/10	Peaje	Transporte	\$4.600

- Saldo del resumen anterior: \$250.000. Pago realizado: \$180.000.
- Saldo financiado = saldo anterior – pago. Intereses: 7 % del saldo financiado. IVA: 21 % de los intereses.
- Total a pagar = saldo financiado + intereses + IVA + consumos del mes.
- Pago mínimo = el mayor entre el 5 % del total y \$5.000.

Tu programa debe

1. Escribir funciones para calcular el saldo financiado con intereses, el total y el pago mínimo.
2. Calcular el total de consumos del mes.
3. Agrupar el gasto por categoría usando una lista de categorías distintas (en orden de aparición) y otra de totales.
4. Mostrar el resumen completo, la categoría con más gasto y el consumo más alto.

Salida esperada

```
Saldo anterior: $250000.00
Su pago: -$180000.00
Saldo financiado: $70000.00
Intereses (7%): $4900.00
IVA sobre intereses: $1029.00
Consumos del mes: $322650.00
TOTAL A PAGAR: $398579.00
PAGO MÍNIMO: $19928.95
Gasto por categoría:
  Supermercado    $137150
  Transporte       $42600
  Hogar            $95000
  Entretenimiento $26500
  Salud            $21400
Categoría con más gasto: Supermercado
Consumo más alto: Heladera cuota 3/12 ($95000)
```

Reglas

- Alcanza con listas, funciones, acumuladores y máximos: es el desafío integrador del nivel 2.

- Montos con 2 decimales. Intereses, IVA y mínimo se redondean (0,5 hacia arriba) al calcularlos. Los decimales se muestran con punto y sin separador de miles en la salida.
- Formato de categorías: nombre alineado a la izquierda en 16 caracteres; los totales por categoría se muestran como enteros.
- En empates de máximos, gana el que aparece primero.

Pista (leela solo si te trabás): la agrupación por categoría sin estructuras clave-valor sale con dos listas paralelas: si la categoría ya está, sumás en su posición; si no, la agregás al final con su primer importe.

Bonus (opcional)

- Calculá cuánto pagaría de intereses el mes siguiente si paga solo el mínimo.
- Separá los consumos en cuotas ("3/12") y proyectá cuánto de cada compra queda por pagar en los próximos meses.

Cierre: Es, simplificado, el cálculo que hace cada mes el sistema de cualquier emisor de tarjetas de crédito.

Ejercicio 99: Desafío de nivel: Batalla naval en la terminal

Nivel 2: Junior II · Colecciones y funciones · Temática: Videojuegos · Dificultad 20/100

Situación: Un grupo de estudiantes está programando una batalla naval para jugar en la terminal durante los recreos. Ya tienen el tablero con los barcos ubicados; falta el motor que procesa los disparos y dice agua, tocado o hundido. Es el desafío de matrices y texto del nivel 2.

Datos de entrada

Tablero oculto, 8 × 8 (P = portaaviones, B = buque, S = submarino, L = lancha, . = agua):

```
PPPP....
.....
.B.....
.B....S.
.B....S.
.....
...LL...
.....
```

Disparos, en orden, como (fila, columna) empezando en 1:

(1,1), (3,2), (4,2), (5,2), (4,7), (4,7), (8,8), (7,4), (7,5), (5,7), (1,2), (1,3), (1,4), (9,3)

Tu programa debe

1. Guardar el tablero y contar cuántas casillas tiene cada barco.
2. Procesar cada disparo: fuera del tablero, repetido, agua, tocado o hundido (cuando se toca la última casilla de ese barco).
3. Marcar el tablero: x para tocado y o para agua.
4. Mostrar el tablero final como lo ve el rival (~ para lo no disparado, x y o para los disparos), el resumen y si la flota fue destruida.

Salida esperada

```
(1,1): tocado
(3,2): tocado
(4,2): tocado
(5,2): ¡hundiste el buque!
(4,7): tocado
(4,7): repetido
(8,8): agua
(7,4): tocado
(7,5): ¡hundiste la lancha!
(5,7): ¡hundiste el submarino!
(1,2): tocado
(1,3): tocado
(1,4): ¡hundiste el portaaviones!
(9,3): fuera del tablero
Tablero final:
XXXX~...
~...~...
~X~...~
~X~...X~
~X~...X~
~...~...
~~~XX~~
~...~O
Aguas: 1 | Tocados: 11 | Barcos hundidos: 4 de 4
¡Flota destruida!
```

Reglas

- Alcanza con matrices, texto, contadores y variables de estado: es el desafío integrador de matrices del nivel 2.
- Un disparo repetido (a una casilla ya marcada con x u o) no cuenta como agua ni como tocado.
- Un disparo fuera del tablero se informa y no afecta nada.
- El mensaje de hundido nombra el barco con su artículo: "¡hundiste el submarino!", "¡hundiste la lancha!".

Pista (leela solo si te trabás): llevá un contador de casillas vivas por barco (una lista de 4 contadores, o una por letra). Cuando un disparo lo deja en 0, ese barco está hundido.

Bonus (opcional)

- Hacé que la computadora dispare contra vos con una estrategia: después de un tocado, probar las casillas vecinas.
- Validá el tablero inicial: los barcos tienen que ser rectos (horizontales o verticales) y no pueden tocarse.

Cierre: Es la misma lógica de estado de juego, con tablero oculto y visible, que tienen los juegos de estrategia por turnos.

Ejercicio 100: Desafío de nivel: Boletín de calificaciones del curso

Nivel 2: Junior II · Colecciones y funciones · Temática: Educación · Dificultad 20/100

Situación: La secretaría de una escuela secundaria arma los boletines a mano, copiando notas de una planilla con tres trimestres por materia. Todos los años hay errores de promedio que terminan en reclamos de las familias. Tu programa genera los boletines y un resumen del curso. Es el desafío que integra el nivel 2.

Datos de entrada

Abril:

Materia	T1	T2	T3
Matemática	8	7	9
Lengua	6	7	7
Historia	9	8	9
Inglés	7	7	8

Benicio:

Materia	T1	T2	T3
Matemática	4	5	6
Lengua	7	6	8
Historia	5	4	6
Inglés	6	6	5

Delfina:

Materia	T1	T2	T3
Matemática	9	10	9
Lengua	9	9	10
Historia	8	9	9
Inglés	10	9	10

Emilio:

Materia	T1	T2	T3
Matemática	6	6	7
Lengua	5	5	4
Historia	7	7	6
Inglés	8	7	7

Una materia se aprueba con promedio 6 o más; si no, va a diciembre.

Tu programa debe

1. Guardar las notas en una estructura de tres niveles: alumno → materia → trimestre (una matriz de listas).
2. Generar el boletín de cada alumno: notas, promedio y estado de cada materia, promedio general y materias a diciembre.
3. Calcular el promedio del curso en cada materia.
4. Informar el alumno con mejor promedio general.

Salida esperada

```

BOLETÍN - Abril
Matemática  8 7 9 | 8.00 | Aprobada
Lengua      6 7 7 | 6.67 | Aprobada
Historia    9 8 9 | 8.67 | Aprobada
Inglés      7 7 8 | 7.33 | Aprobada
Promedio general: 7.67
BOLETÍN - Benicio
Matemática  4 5 6 | 5.00 | A diciembre
Lengua      7 6 8 | 7.00 | Aprobada
Historia    5 4 6 | 5.00 | A diciembre
Inglés      6 6 5 | 5.67 | A diciembre
Promedio general: 5.67 | Materias a diciembre: Matemática, Historia, Inglés
BOLETÍN - Delfina
Matemática  9 10 9 | 9.33 | Aprobada
Lengua      9 9 10 | 9.33 | Aprobada
Historia    8 9 9 | 8.67 | Aprobada
Inglés     10 9 10 | 9.67 | Aprobada
Promedio general: 9.25
BOLETÍN - Emilio
Matemática  6 6 7 | 6.33 | Aprobada
Lengua      5 5 4 | 4.67 | A diciembre
Historia    7 7 6 | 6.67 | Aprobada
Inglés      8 7 7 | 7.33 | Aprobada
Promedio general: 6.25 | Materias a diciembre: Lengua
PROMEDIO DEL CURSO POR MATERIA
Matemática  7.17
Lengua      6.92
Historia    7.25
Inglés      7.50
Mejor promedio: Delfina (9.25)

```

Reglas

- Alcanza con matrices, promedios, funciones y formato de texto: es el desafío integrador del nivel 2.
- Promedios con 2 decimales (0,5 hacia arriba), redondeando solo al mostrar; las decisiones se toman con el valor sin redondear. Los decimales se muestran con punto y sin separador de miles en la salida.
- Formato de cada materia: nombre en 11 caracteres a la izquierda, cada nota en 2 a la derecha, promedio en 5 a la derecha.
- El promedio general es el promedio de los promedios por materia. En empates, gana el primero de la lista.

Pista (leela solo si te trabás): escribí una función promedio(lista) y usala en todos lados: para cada materia, para el general y para el curso. Casi todo el ejercicio es recorrer la misma estructura en distinto orden.

Bonus (opcional)

- Agregá la condición de repitencia: 3 o más materias a diciembre.
- Generá un ranking del curso ordenado por promedio (cuando llegues al nivel 3, vas a poder ordenarlo).

Cierre: Es la misma lógica que tienen los sistemas de gestión escolar que emiten boletines digitales.

Lote 05: ejercicios 101 a 125

Nivel 3

Junior III · Datos del mundo real

Ejercicios 101 a 150

Qué vas a practicar

Datos reales: textos sucios, fechas, validaciones, archivos y registros.

Qué se espera al terminar

Validar entradas, limpiar datos y manejar casos borde sin romperse.

Conceptos del nivel

conjuntos, eliminación de duplicados, mapas clave-valor, parseo de texto, acumulación por clave, implementar un ordenamiento simple, fechas, años bisiestos, intersección, ordenamiento, fechas y horas, comparación de intervalos, intersección y unión, validación de datos, recursión básica, factorial, fusión de datos, agrupación, mapas de frecuencias, ordenamiento de caracteres, búsqueda por clave, validación, agrupación por categoría y mes, husos horarios, conteo, top-k por ordenamiento, parseo de texto separado por comas, mapas anidados, dígito verificador, parseo, claves compuestas, ordenamiento con criterio múltiple, filtrado, conversión, diferencia, normalización de texto, recursión, reglas simples, intercalación, orden de inserción....

Desafíos de nivel

- 148. El extracto de dos bancos en uno
- 149. Padrón de socios del club
- 150. Farmacia con vencimientos

Ejercicio 101: Invitados repetidos del casamiento

Nivel 3: Junior III · Datos del mundo real · Temática: Gastronomía · Dificultad 21/100

Situación: Para un casamiento, cada familia armó su lista de invitados por separado y el salón cobra \$45.000 por cubierto. Al juntar las listas aparecen repetidos, escritos con mayúsculas distintas o con espacios de más. Los novios necesitan la lista final sin duplicados y saber cuánta plata se ahorran.

Datos de entrada

Lista del novio: Martín Ruiz · laura gómez · Pablo Sanz · MARTÍN RUIZ · Sofía Lema · Tío Carlos

Lista de la novia: Laura Gómez · Inés Paz · tío carlos · Ramiro Díaz · Inés Paz · Pablo Sanz · Belén Soto

Tu programa debe

1. Normalizar cada nombre: minúsculas y un solo espacio entre palabras.
2. Recorrer primero la lista del novio y después la de la novia, usando un conjunto para detectar los ya vistos.
3. Armar la lista final en orden de primera aparición, con cada palabra capitalizada.
4. Mostrar cuántos nombres había, cuántos invitados únicos quedan, los duplicados descartados (con la lista en la que aparecieron) y el ahorro.

Salida esperada

```
Nombres cargados: 13
Invitados únicos: 8
Duplicados descartados (5): Martín Ruiz (novio), Laura Gómez (novia), Tío Carlos (novia), Inés Paz (novia), Pablo Sanz (novia)
Ahorro en catering: $225000
Lista final: Martín Ruiz, Laura Gómez, Pablo Sanz, Sofía Lema, Tío Carlos, Inés Paz, Ramiro Díaz, Belén Soto
```

Reglas

- Alcanza con conjuntos y normalización de texto.
- Dos nombres son el mismo invitado si coinciden después de normalizar ("Inés Paz" = "Inés Paz").
- Las tildes se respetan: "Martin" y "Martín" serían personas distintas (no hay casos así en los datos).
- La consulta "¿ya lo vi?" tiene que ser de tiempo constante: usá un conjunto, no una búsqueda lineal.

Pista (leela solo si te trabás): el conjunto guarda las formas normalizadas y la lista final guarda las formas lindas. Consultás siempre el conjunto antes de agregar a la lista.

Bonus (opcional)

- Detectá nombres parecidos que podrían ser la misma persona (por ejemplo, que difieren solo en una letra) y listalos para revisión manual.
- Asigná mesas de 10 personas manteniendo juntos a los invitados de cada lista.

Cierre: Es la misma deduplicación que hacen los sistemas de CRM al unificar bases de contactos de distintas fuentes.

Ejercicio 102: Palabras más repetidas en las reseñas

Nivel 3: Junior III · Datos del mundo real · Temática: E-commerce · Dificultad 21/100

Situación: Una tienda online de bazar tiene cientos de reseñas y el equipo de producto quiere saber de qué habla la gente sin leerlas una por una. El primer paso es contar las palabras más repetidas, ignorando las palabras "vacías" que no aportan significado.

Datos de entrada

Reseñas:

1. Llegó rápido y el producto es excelente. Muy recomendable.
2. El envío fue rápido, pero la caja llegó rota.
3. Excelente calidad, el producto es tal cual la foto.
4. Muy mala atención, el envío tardó una semana.
5. Producto excelente y envío rápido. Volvería a comprar.

Palabras vacías a ignorar: a, cual, de, el, es, fue, la, muy, pero, tal, una, y

Tu programa debe

1. Separar cada reseña en palabras (secuencias de letras) y pasarlas a minúsculas.
2. Contar la frecuencia de cada palabra con un mapa, ignorando las palabras vacías.
3. Ordenar por frecuencia descendente y, en empate, alfabéticamente (ignorando tildes).
4. Mostrar la cantidad de palabras distintas y el top 5.

Salida esperada

```
Palabras distintas (sin vacías): 16
Top 5:
1. envío (3)
2. excelente (3)
3. producto (3)
4. rápido (3)
5. llegó (2)
```

Reglas

- Alcanza con mapas clave-valor, parseo de texto y ordenamiento (acá podés usar el ordenamiento que tengas disponible, con un criterio propio).
- Los signos de puntuación separan palabras y no se cuentan.
- "rápido" y "rapido" serían palabras distintas para el conteo; el orden alfabético sí ignora tildes.

Pista (leela solo si te trabás): el criterio de orden es un par: primero la frecuencia (de mayor a menor) y después la palabra (de menor a mayor). Muchas herramientas de ordenamiento aceptan una función que compare dos elementos o que genere una clave compuesta.

Bonus (opcional)

- Contá pares de palabras consecutivas ("envío rápido") en lugar de palabras sueltas.
- Clasificá cada reseña como positiva o negativa según una lista de palabras positivas y negativas.

Cierre: Es el primer paso de cualquier análisis de opiniones de clientes, antes de técnicas más sofisticadas de procesamiento de lenguaje.

Ejercicio 103: Gastos agrupados por categoría

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 21/100

Situación: Joaquín exporta los movimientos de su tarjeta de débito como texto separado por punto y coma. Quiere saber en qué se le va la plata: el total por categoría, cuántos movimientos tuvo cada una y qué porcentaje representa del gasto del mes.

Datos de entrada

Formato de cada línea: fecha;comercio;categoría;monto (el monto usa punto decimal)

```
2026-05-02;Coto;Supermercado;18450.50
2026-05-03;YPF;Transporte;32000
2026-05-05;Farmacity;Salud;9870.25
2026-05-07;Día;Supermercado;22310
2026-05-09;Cinemark;Entretenimiento;14500
2026-05-12;SUBE;Transporte;8000
2026-05-15;Coto;Supermercado;41200.75
2026-05-18;Spotify;Entretenimiento;4299
2026-05-21;Médico;Salud;25000
2026-05-26;YPF;Transporte;29500
```

Tu programa debe

1. Separar cada línea en sus cuatro campos.
2. Acumular el monto y la cantidad de movimientos por categoría en mapas.
3. Calcular el total general y el porcentaje de cada categoría.
4. Mostrar las categorías ordenadas de mayor a menor gasto (en empate, alfabéticamente) y el total.

Salida esperada

Supermercado	\$ 81961.25	3 mov.	40.0%
Transporte	\$ 69500.00	3 mov.	33.9%
Salud	\$ 34870.25	2 mov.	17.0%
Entretenimiento	\$ 18799.00	2 mov.	9.2%
Total	\$205130.50		

Reglas

- Alcanza con parseo de texto, mapas clave-valor y ordenamiento.
- Montos con 2 decimales y porcentajes con 1 (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Formato: categoría en 16 caracteres a la izquierda, monto en 9 a la derecha, porcentaje en 4 a la derecha.
- Usá números decimales exactos o centavos enteros: sumar montos con decimales binarios puede dar diferencias de centavos.

Pista (leela solo si te trabás): el mapa tiene como clave la categoría y como valor el acumulado. La primera vez que aparece una categoría, su acumulado arranca en cero.

Bonus (opcional)

- Agrupá además por comercio dentro de cada categoría (un mapa de mapas).
- Mostrá un gráfico de barras en texto: un # cada 5 % del gasto.

Cierre: Es la misma agrupación que muestran las apps de bancos en la sección "¿en qué gastaste este mes?".

Ejercicio 104: La fila de la guardia por hora de llegada

Nivel 3: Junior III · Datos del mundo real · Temática: Salud · Dificultad 22/100

Situación: En la guardia de un hospital municipal, la planilla de ingreso se llena a mano y los papeles se mezclan. La administrativa carga los datos en el orden en que agarra las hojas, pero la atención tiene que ser por orden de llegada. Tu programa ordena la fila, implementando vos el algoritmo.

Datos de entrada

Paciente	Hora de llegada
Ruiz, Ana	08:42
Toledo, Raúl	08:15
Mendez, Sol	09:03
Paz, Irma	08:15
Gil, Ciro	07:58
Luna, Dora	08:50
Vera, Íker	08:31

Tu programa debe

1. Implementar el ordenamiento por inserción: tomar cada elemento desde el segundo y correrlo hacia la izquierda mientras el anterior tenga una hora mayor.
2. Contar cuántos desplazamientos (corrimientos de un elemento una posición a la derecha) hizo el algoritmo.
3. Mostrar la fila ordenada y la cantidad de desplazamientos.

Salida esperada

```
1. 07:58 Gil, Ciro
2. 08:15 Toledo, Raúl
3. 08:15 Paz, Irma
4. 08:31 Vera, Íker
5. 08:42 Ruiz, Ana
6. 08:50 Luna, Dora
7. 09:03 Mendez, Sol
Desplazamientos realizados: 11
```

Reglas

- Alcanza con listas y un ordenamiento implementado por vos: acá NO vale usar un ordenamiento ya hecho.
- Si dos pacientes llegaron a la misma hora, se mantiene el orden en que estaban en la planilla (el algoritmo tiene que ser estable: solo se corre si la hora anterior es estrictamente mayor).
- Las horas en formato HH:MM con dos dígitos se pueden comparar como texto; si no, pasalas a minutos.

Pista (leela solo si te trabás): guardá el elemento que estás insertando en una variable auxiliar, corré los mayores una posición a la derecha y, cuando encuentres uno menor o igual (o llegues al principio), ubicalo en el hueco.

Bonus (opcional)

- Implementá también el ordenamiento de burbuja y compará la cantidad de intercambios con los mismos datos.
- Agregá la prioridad de triage (1 a 5): se atiende primero por prioridad y, a igual prioridad, por hora de llegada.

Cierre: Es la base de cualquier sistema de turnos y colas de atención; el ordenamiento por inserción, además, es el que usan muchas librerías para listas cortas.

Ejercicio 105: Días hasta el vencimiento

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 22/100

Situación: Una app de pagos de servicios quiere mostrar cuántos días faltan para cada vencimiento y resaltar los urgentes. Algunos vencimientos están lejos en el tiempo y cruzan años bisiestos, así que no alcanza con restar días del mes. Hoy es 20/02/2026.

Datos de entrada

Fecha de hoy: 20/02/2026

Factura	Vencimiento
Luz	18/02/2026
Gas	26/02/2026
Internet	05/03/2026
Patente	15/01/2027
Seguro del auto	01/03/2028
Expensas	20/02/2026

Tu programa debe

1. Escribir una función que convierta una fecha en la cantidad de días transcurridos desde una fecha fija (por ejemplo, el 01/01 del año 1).
2. Calcular los días entre hoy y cada vencimiento como la resta de esos números.
3. Clasificar: vencida (negativo), vence hoy (0), urgente (1 a 7 días) o normal.
4. Mostrar cada factura con su vencimiento y su estado.

Salida esperada

```
Luz          18/02/2026 -> VENCIDA hace 2 días
Gas          26/02/2026 -> urgente: vence en 6 días
Internet     05/03/2026 -> vence en 13 días
Patente      15/01/2027 -> vence en 329 días
Seguro del auto 01/03/2028 -> vence en 740 días
Expensas     20/02/2026 -> VENCE HOY
```

Reglas

- Alcanza con fechas, años bisiestos y condicionales. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos (es un buen ejercicio).
- Regla de bisiesto: divisible por 4, salvo los divisibles por 100 que no lo son por 400.
- Formato: nombre en 16 caracteres a la izquierda y fecha DD/MM/AAAA.
- El 29/02/2028 existe: el seguro del auto cruza ese día.

Pista (leela solo si te trabás): pasar cada fecha a un número de días absoluto convierte un problema de calendario en una resta. La función suma los días de los años completos, los de los meses completos del año y el día.

Bonus (opcional)

- Calculá el día de la semana de cada vencimiento sabiendo que el 20/02/2026 es viernes.
- Si el vencimiento cae sábado o domingo, se corre al lunes siguiente: recalculá los días.

Cierre: Es el mismo cálculo de días entre fechas que hacen todos los sistemas de cobranzas, préstamos y vencimientos.

Ejercicio 106: Alumnos que cursan los dos talleres

Nivel 3: Junior III · Datos del mundo real · Temática: Educación · Dificultad 22/100

Situación: Una escuela ofrece dos talleres extracurriculares a contraturno: robótica y teatro. Los que cursan los dos necesitan una autorización especial para quedarse a almorzar. La secretaria tiene las dos listas (cargadas con mayúsculas desprolijas) y necesita cruzarlas.

Datos de entrada

Robótica: Agustina, bruno, Camilo, Dana, Ezequiel, Flor, gael

Teatro: Bruno, Dana, Hilda, Ivo, Gael, Julieta

Tu programa debe

1. Normalizar los nombres (primera letra mayúscula y el resto minúscula) y guardar cada lista en un conjunto.
2. Calcular la intersección (los que cursan los dos), las diferencias (solo uno) y la unión.
3. Mostrar cada grupo ordenado alfabéticamente, con su cantidad, y el total de chicos distintos.

Salida esperada

```
En los dos talleres (3): Bruno, Dana, Gael
Solo robótica (4): Agustina, Camilo, Ezequiel, Flor
Solo teatro (3): Hilda, Ivo, Julieta
Total de chicos distintos: 10
```

Reglas

- Alcanza con conjuntos y sus operaciones (intersección, diferencia, unión).
- Si implementás las operaciones vos, que sean eficientes: nada de recorrer una lista dentro de otra.
- Los grupos vacíos se muestran igual, con cantidad 0.

Pista (leela solo si te trabás): los conjuntos resuelven esto casi solos: la intersección son los que están en ambos, la diferencia $A - B$ son los de A que no están en B.

Bonus (opcional)

- Agregá un tercer taller (ajedrez) y mostrá quiénes están en exactamente dos de los tres.
- Generá la planilla de almuerzos: una línea por chico que cursa los dos, con los días que se queda.

Cierre: Es el mismo cruce de listas que hacen los sistemas de gestión para detectar clientes en común entre dos servicios.

Ejercicio 107: Leer el log del servidor

Nivel 3: Junior III · Datos del mundo real · Temática: Seguridad informática · Dificultad 22/100

Situación: La API de una fintech tuvo una caída de unos minutos y el equipo de seguridad tiene que entender qué pasó revisando el log del servidor. El log es texto plano, una línea por pedido. Tu programa lo procesa y arma un resumen para el informe del incidente.

Datos de entrada

Formato de cada línea: fecha hora método ruta código_de_estado milisegundos

```
2026-03-14 10:22:31 GET /api/pagos 200 123
2026-03-14 10:22:35 POST /api/pagos 201 340
2026-03-14 10:22:40 GET /api/usuarios 200 88
2026-03-14 10:23:02 POST /api/pagos 500 1250
2026-03-14 10:23:10 GET /api/pagos 200 97
2026-03-14 10:23:15 GET /api/saldo 404 12
2026-03-14 10:23:44 POST /api/login 401 45
2026-03-14 10:24:01 POST /api/pagos 503 3000
2026-03-14 10:24:20 GET /api/usuarios 200 102
2026-03-14 10:24:33 POST /api/login 200 60
```

Tu programa debe

1. Separar cada línea en sus campos.
2. Contar las respuestas por código de estado.
3. Calcular la latencia promedio de cada endpoint (método + ruta), con la cantidad de pedidos.
4. Listar los errores del servidor (códigos que empiezan con 5) con su hora.

Salida esperada

```
Respuestas por código: 200=5 201=1 401=1 404=1 500=1 503=1
Latencia promedio por endpoint:
GET /api/pagos          110.0 ms (2)
GET /api/saldo          12.0 ms (1)
GET /api/usuarios       95.0 ms (2)
POST /api/login         52.5 ms (2)
POST /api/pagos        1530.0 ms (3)
Errores del servidor (2):
10:23:02 POST /api/pagos -> 500
10:24:01 POST /api/pagos -> 503
```

Reglas

- Alcanza con parseo de texto y mapas clave-valor.
- Los códigos se muestran ordenados de menor a mayor y los endpoints alfabéticamente.
- Latencia con 1 decimal (0,5 hacia arriba). Formato: endpoint en 22 caracteres a la izquierda, promedio en 7 a la derecha. Los decimales se muestran con punto y sin separador de miles en la salida.
- Los errores se listan en el orden del log.

Pista (leela solo si te trabás): usá la combinación "método ruta" como clave: GET /api/pagos y POST /api/pagos son endpoints distintos.

Bonus (opcional)

- Calculá la latencia en percentil 90 de cada endpoint.
- Detectá ventanas de 1 minuto con más de un error 5xx y marcalas como posible incidente.

Cierre: Es el primer paso de cualquier análisis de logs que hacen las herramientas de observabilidad y monitoreo.

Ejercicio 108: Ranking de goleadores del campeonato

Nivel 3: Junior III · Datos del mundo real · Temática: Deportes · Dificultad 22/100

Situación: La liga amateur del departamento carga cada gol en una planilla compartida, una línea por gol. Al final de la primera rueda quieren publicar la tabla de goleadores con posiciones compartidas cuando hay empate, como en los diarios.

Datos de entrada

Formato: fecha;jugador;equipo

```
F1;Gómez;Tiro Federal
F1;Acosta;Sportivo
F1;Gómez;Tiro Federal
F2;Benítez;Unión
F2;Acosta;Sportivo
F3;Ríos;Unión
F3;Gómez;Tiro Federal
F3;Benítez;Unión
F4;Acosta;Sportivo
F4;Ábalos;Juventud
F5;Ríos;Unión
F5;Ábalos;Juventud
F5;Benítez;Unión
F6;Ábalos;Juventud
F6;Díaz;Tiro Federal
```

Tu programa debe

1. Contar los goles de cada jugador con un mapa (y guardar su equipo).
2. Ordenar por goles de mayor a menor y, en empate, alfabéticamente ignorando tildes.
3. Asignar posiciones compartidas: los empatados comparten posición y la siguiente salta (1, 1, 3...).
4. Mostrar la tabla completa.

Salida esperada

```
1. Ábalos (Juventud) - 3 goles
1. Acosta (Sportivo) - 3 goles
1. Benítez (Unión) - 3 goles
1. Gómez (Tiro Federal) - 3 goles
5. Ríos (Unión) - 2 goles
6. Díaz (Tiro Federal) - 1 gol
```

Reglas

- Alcanza con mapas clave-valor y ordenamiento con criterio compuesto.
- "Ábalos" va antes que "Acosta" (la tilde no cuenta para el orden).
- La posición de un jugador es 1 + la cantidad de jugadores con más goles que él.

Pista (leela solo si te trabás): al recorrer la lista ya ordenada, la posición solo cambia cuando cambia la cantidad de goles, y en ese caso pasa a ser el índice actual (empezando en 1).

Bonus (opcional)

- Mostrá el goleador de cada equipo.
- Agregá la cantidad de fechas en las que convirtió cada jugador como segundo criterio de desempate.

Cierre: Es la misma tabla que arman los sitios deportivos y las apps de ligas amateurs.

Ejercicio 109: Turnos médicos que se pisan

Nivel 3: Junior III · Datos del mundo real · Temática: Salud · Dificultad 22/100

Situación: El sistema de turnos de un consultorio permite que los médicos carguen turnos a mano y, a veces, un turno arranca antes de que termine el anterior. Los pacientes terminan esperando en la sala. La administración quiere detectar todas las superposiciones antes de confirmar la agenda.

Datos de entrada

Turno	Médico	Fecha	Hora de inicio	Duración (min)
1	Dra. Ferro	2026-06-10	09:00	30
2	Dra. Ferro	2026-06-10	09:20	20
3	Dr. Quiroga	2026-06-10	09:00	45
4	Dra. Ferro	2026-06-10	09:40	20
5	Dr. Quiroga	2026-06-10	09:45	30
6	Dra. Ferro	2026-06-11	09:00	30
7	Dr. Quiroga	2026-06-10	10:00	20
8	Dra. Ferro	2026-06-10	10:30	15

Tu programa debe

1. Calcular el inicio y el fin de cada turno.
2. Comparar cada par de turnos del mismo médico en la misma fecha.
3. Detectar si se superponen y cuántos minutos se pisan.
4. Listar todas las superposiciones y su cantidad.

Salida esperada

Dra. Ferro 2026-06-10: turno 1 (09:00-09:30) se pisa con turno 2 (09:20-09:40) por 10 min
 Dr. Quiroga 2026-06-10: turno 5 (09:45-10:15) se pisa con turno 7 (10:00-10:20) por 15 min
 Superposiciones: 2

Reglas

- Alcanza con fechas y horas, comparación de intervalos y bucles anidados.
- Dos turnos se superponen si uno empieza antes de que termine el otro, y viceversa. Si uno termina justo cuando empieza el otro, NO se superponen.
- Cada par se informa una sola vez, en el orden (turno i, turno j) con $i < j$, recorriendo i y después j en orden creciente.

Pista (leela solo si te trabás): la condición de superposición de dos intervalos $[a, b)$ y $[c, d)$ es: $a < d$ y $c < b$. Los minutos superpuestos son $\min(b, d) - \max(a, c)$.

Bonus (opcional)

- Con n turnos, comparar todos los pares cuesta del orden de n^2 . Ordená los turnos por médico, fecha e inicio y detectá las superposiciones comparando solo turnos consecutivos.
- Proponé una reprogramación automática que corra cada turno superpuesto al primer horario libre.

Cierre: Es la misma validación que hacen los sistemas de agenda y reserva de salas para evitar dobles reservas.

Ejercicio 110: Canciones favoritas en común

Nivel 3: Junior III · Datos del mundo real · Temática: Música · Dificultad 23/100

Situación: Tres amigos arman la playlist para un viaje en auto de 8 horas y cada uno trajo sus cinco temas favoritos. Quieren empezar con los que les gustan a los tres, y además que la app le recomiende a cada uno lo que les gusta a los otros y él todavía no escuchó.

Datos de entrada

Mica: Luz de Neón, Ruta Sur, Café Frío, Otoño, Marea

Joaco: Marea, Otoño, Tren de Noche, Luz de Neón, Brasa

Vale: Otoño, Brasa, Luz de Neón, Faro, Ruta Sur

Tu programa debe

1. Guardar los favoritos de cada uno en un conjunto.
2. Calcular las canciones que les gustan a los tres (intersección) y las que les gustan a exactamente dos.
3. Contar las canciones distintas (unión).
4. Para cada amigo, recomendar las canciones de los otros dos que no están en su lista.

Salida esperada

Les gustan a los tres: Luz de Neón, Otoño
Les gustan a exactamente dos: Brasa, Marea, Ruta Sur
Canciones distintas: 8
Recomendaciones:
Mica: Brasa, Faro, Tren de Noche
Joaco: Café Frío, Faro, Ruta Sur
Vale: Café Frío, Marea, Tren de Noche

Reglas

- Alcanza con conjuntos: intersección, unión y diferencia.
- Todas las listas de salida van ordenadas alfabéticamente, ignorando tildes.
- El programa tiene que funcionar con cualquier cantidad de amigos (acá son 3).

Pista (leela solo si te trabás): "exactamente dos" no sale de una sola operación: recorré la unión y contá en cuántos conjuntos está cada canción.

Bonus (opcional)

- Calculá la similitud entre cada par de amigos: tamaño de la intersección / tamaño de la unión.
- Armá la playlist final: primero las de los tres, después las de dos, después el resto, sin repetir.

Cierre: Es la base de los sistemas de recomendación colaborativa que usan las plataformas de música y video.

Ejercicio 111: Postulantes con email inválido

Nivel 3: Junior III · Datos del mundo real · Temática: Recursos humanos · Dificultad 23/100

Situación: Una consultora de Recursos Humanos recibió 3.000 postulaciones por un formulario viejo que no validaba nada. Antes de mandar los mails de convocatoria, necesita separar las direcciones mal escritas para contactar a esas personas por teléfono.

Datos de entrada

Direcciones a validar:

- ana.paz@gmail.com
- juan..perez@correo.com
- sofi@empresa
- @hotmail.com
- martin_g@mail.com.ar
- lu cia@gmail.com
- pedro@@gmail.com
- .rocio@mail.com
- tomas@mail.c
- carla-ok@sub.dominio.org

Reglas (se verifican en este orden y se informa la primera que falla):

1. Exactamente una @.
2. La parte local (antes de la @) no está vacía.
3. La parte local solo tiene letras, dígitos, punto, guion bajo o guion.
4. La parte local no empieza ni termina con punto.
5. La parte local no tiene dos puntos seguidos.
6. El dominio (después de la @) tiene al menos un punto.
7. Ninguna parte del dominio separada por puntos está vacía.
8. Cada parte del dominio solo tiene letras, dígitos o guion.
9. La última parte del dominio tiene al menos 2 caracteres y son todas letras.

Tu programa debe

1. Escribir una función que valide una dirección y devuelva el motivo del primer error, o un valor ausente si es válida.
2. Aplicarla a todas las direcciones.
3. Mostrar el resultado de cada una y la cantidad de válidas.

Salida esperada

```
ana.paz@gmail.com: válido
juan..perez@correo.com: INVÁLIDO (no puede tener dos puntos seguidos)
sofi@empresa: INVÁLIDO (el dominio no tiene punto)
@hotmail.com: INVÁLIDO (falta la parte antes de la @)
martin_g@mail.com.ar: válido
lu cia@gmail.com: INVÁLIDO (carácter no permitido ' ')
pedro@@gmail.com: INVÁLIDO (debe tener exactamente una @)
.rocio@mail.com: INVÁLIDO (no puede empezar ni terminar con punto)
tomas@mail.c: INVÁLIDO (la terminación debe tener al menos 2 letras)
carla-ok@sub.dominio.org: válido
Válidos: 3 de 10
```

Reglas

- Alcanza con validación de datos y parseo de texto.

- Implementá las reglas con recorridos y comparaciones (sin patrones de expresiones regulares), para practicar el parseo.
- Los mensajes de error se escriben exactamente como en la salida esperada; cuando se nombra un carácter, va entre comillas simples.

Pista (leela solo si te trabás): separá en dos partes por la @ y validá cada parte con su propia función. El orden de las verificaciones define qué motivo se informa.

Bonus (opcional)

- Detectá errores de tipeo comunes en dominios conocidos ("gmial.com", "hotmial.com") y sugerí la corrección.
- Resolvé la misma validación con expresiones regulares y compará la legibilidad.

Cierre: Es la validación que hace cualquier formulario de registro serio antes de guardar un email.

Ejercicio 112: ¿De cuántas formas sirvo los platos?

Nivel 3: Junior III · Datos del mundo real · Temática: Gastronomía · Dificultad 23/100

Situación: Un restaurante de autor quiere cambiar el orden de su menú degustación cada noche sin repetir nunca una secuencia. El chef pregunta cuántos órdenes distintos existen y quiere ver, para un caso chico, todos los órdenes listados. Tu programa lo resuelve con recursión.

Datos de entrada

Caso chico: Entrada, Principal, Postre

Menú degustación: 6 pasos. Variante corta: elegir 4 de los 6 pasos y ordenarlos.

Tu programa debe

1. Escribir una función recursiva que calcule el factorial de n .
2. Escribir una función recursiva que genere todas las permutaciones de una lista, y listar las del caso chico.
3. Calcular la cantidad de órdenes del menú de 6 pasos y la de la variante de 4 de 6: $n! / (n - k)!$.
4. Mostrar también $20!$, para dimensionar el crecimiento.

Salida esperada

```
Órdenes posibles de 3 platos: 6
1. Entrada -> Principal -> Postre
2. Entrada -> Postre -> Principal
3. Principal -> Entrada -> Postre
4. Principal -> Postre -> Entrada
5. Postre -> Entrada -> Principal
6. Postre -> Principal -> Entrada
6! = 720 órdenes para los 6 pasos del menú degustación
Elegir y ordenar 4 de 6 pasos: 360
20! = 2432902008176640000
```

Reglas

- Alcanza con recursión básica.
- Las permutaciones se generan eligiendo, en orden, cada elemento como primero y permutando el resto recursivamente.
- $20! = 2.432.902.008.176.640.000$ entra en un entero de 64 bits con signo; $21!$ ya no.
- Toda función recursiva necesita un caso base: definí cuál es en cada una.

Pista (leela solo si te trabás): para permutar $[a, b, c]$: poné a primero y permutá $[b, c]$; después b primero y permutá $[a, c]$; después c primero y permutá $[a, b]$.

Bonus (opcional)

- Calculá cuántos días de apertura hacen falta para agotar todos los órdenes del menú de 6 pasos si se cambia uno por noche.
- Generá las permutaciones sin recursión, usando el algoritmo del siguiente orden lexicográfico.

Cierre: El factorial y las permutaciones están detrás de todo cálculo de combinaciones: desde contraseñas posibles hasta rutas de reparto.

Ejercicio 113: Stock consolidado de sucursales

Nivel 3: Junior III · Datos del mundo real · Temática: Logística y envíos · Dificultad 23/100

Situación: Una cadena de pinturerías tiene tres sucursales y cada una exporta su stock en un archivo de texto con formato propio. El área de compras quiere un stock consolidado por código de producto, y saber qué productos se manejan en todas las sucursales y cuáles en una sola.

Datos de entrada

Formato: código:cantidad, una línea por producto.

Sucursal Centro:

```
A10:25
B22:8
C05:40
D18:3
```

Sucursal Norte:

```
A10:12
C05:10
E30:7
B22:0
```

Sucursal Sur:

```
B22:15
C05:5
D18:9
F01:20
```

Tu programa debe

1. Leer las tres listas y sumar las cantidades por código en un mapa.
2. Registrar, para cada código, en qué sucursales aparece (un mapa de código a conjunto de sucursales).
3. Mostrar el stock consolidado ordenado por código, con las sucursales donde está.
4. Listar los productos presentes en todas las sucursales y los presentes en una sola.

Salida esperada

```
Stock consolidado:
A10: 37 (Centro, Norte)
B22: 23 (Centro, Norte, Sur)
C05: 55 (Centro, Norte, Sur)
D18: 12 (Centro, Sur)
E30: 7 (Norte)
F01: 20 (Sur)
En todas las sucursales: B22, C05
En una sola sucursal: E30, F01
```

Reglas

- Alcanza con mapas clave-valor, conjuntos y fusión de datos.
- Un producto con cantidad 0 cuenta como presente en la sucursal (está dado de alta, aunque sin stock).
- Las sucursales se listan en orden alfabético.

Pista (leela solo si te trabás): son dos mapas que se llenan en el mismo recorrido: uno acumula cantidades, el otro acumula sucursales.

Bonus (opcional)

- Sugeri transferencias: si una sucursal tiene 0 de un producto y otra tiene más de 10, proponé mover la mitad.
- Detecta códigos duplicados dentro de una misma sucursal y avisá.

Cierre: Es la misma consolidación que hacen los sistemas de gestión de cadenas con varias sucursales o depósitos.

Ejercicio 114: Horas trabajadas por semana del año

Nivel 3: Junior III · Datos del mundo real · Temática: Recursos humanos · Dificultad 23/100

Situación: Una empresa de mantenimiento paga horas extra cuando un técnico supera las 44 horas semanales. Las fichadas se registran por día, pero la liquidación es por semana ISO (de lunes a domingo). Enero es complicado porque los primeros días del año pueden pertenecer a la última semana del año anterior.

Datos de entrada

Fecha	Horas
2026-01-02	9
2026-01-05	9
2026-01-06	10
2026-01-07	9
2026-01-08	11
2026-01-09	8
2026-01-10	4
2026-01-12	8
2026-01-13	8
2026-01-14	9
2026-01-16	8
2026-01-19	10
2026-01-20	10
2026-01-21	9
2026-01-22	10
2026-01-23	9
2026-01-24	5

Tu programa debe

1. Calcular la semana ISO (año y número de semana) de cada fecha.
2. Agrupar las horas por semana con un mapa cuya clave sea (año, semana).
3. Calcular las horas extra de cada semana (lo que pasa de 44).
4. Mostrar las semanas en orden y el total de horas extra.

Salida esperada

```
Semana 01/2026: 9 h
Semana 02/2026: 51 h -> 7 h extra
Semana 03/2026: 33 h
Semana 04/2026: 53 h -> 9 h extra
Total de horas extra: 16
```

Reglas

- Alcanza con fechas, agrupación en mapas y ordenamiento. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos (es un buen ejercicio).
- Semana ISO: empieza el lunes; la semana 1 de un año es la que contiene el primer jueves de ese año. Por eso el 02/01/2026 (viernes) pertenece a la semana 1 de 2026, porque el 01/01/2026 fue jueves.
- Formato: número de semana con 2 dígitos y el año con 4.

Pista (leela solo si te trabás): si lo implementás a mano: calculá el día de la semana de la fecha, buscá el jueves de esa misma semana y fijate en qué año cae ese jueves; la semana es $(\text{día del año de ese jueves} - 1) / 7 + 1$.

Bonus (opcional)

- Pagá distinto las horas extra en sábado (50 %) y domingo (100 %).
- Probá con fechas de fines de diciembre de 2026 y comienzos de 2027, donde la semana cruza el año.

Cierre: Es la misma agrupación semanal que hacen los sistemas de liquidación y de control horario.

Ejercicio 115: Anagramas del juego de palabras

Nivel 3: Junior III · Datos del mundo real · Temática: Videojuegos · Dificultad 24/100

Situación: Un juego de palabras para celular muestra un conjunto de letras y el jugador tiene que encontrar todas las palabras que se forman con ellas. Para armar los niveles, el diseñador necesita agrupar su diccionario en familias de anagramas (palabras con exactamente las mismas letras).

Datos de entrada

Palabras del diccionario (en mayúsculas, sin tildes): ROMA, AMOR, PERRO, MORA, OMAR, SALTO, LOSTA, RAMO, PIEDRA, TALOS, PERRA, ARPE

Tu programa debe

1. Calcular una "firma" para cada palabra: sus letras ordenadas alfabéticamente (ROMA → AMOR).
2. Agrupar las palabras por firma en un mapa firma → lista de palabras.
3. Mostrar los grupos con 2 o más palabras, ordenados por tamaño descendente y, a igual tamaño, por su primera palabra.
4. Listar las palabras que no tienen anagrama, en el orden original.

Salida esperada

Grupo 1: ROMA, AMOR, MORA, OMAR, RAMO
Grupo 2: SALTO, LOSTA, TALOS
Sin anagrama: PERRO, PIEDRA, PERRA, ARPE

Reglas

- Alcanza con mapas clave-valor y ordenamiento de caracteres.
- Dentro de cada grupo, las palabras mantienen el orden en que aparecen en el diccionario.
- Dos palabras con distinta cantidad de alguna letra no son anagramas (PERRO y PERRA no lo son).

Pista (leela solo si te trabás): dos palabras son anagramas si y solo si tienen la misma firma. Así, en vez de comparar cada par de palabras, las agrupás en una sola pasada.

Bonus (opcional)

- Usá como firma el conteo de cada letra (26 números) en lugar de ordenar: es más rápido para palabras largas.
- Dado un conjunto de letras, encontrará todas las palabras del diccionario que se pueden formar usando cada letra como máximo una vez.

Cierre: Es la misma técnica de "firma canónica" que se usa para agrupar datos equivalentes en bases de datos y buscadores.

Ejercicio 116: Tarifas de roaming por país

Nivel 3: Junior III · Datos del mundo real · Temática: Telecomunicaciones · Dificultad 24/100

Situación: Una operadora móvil cobra el roaming según la zona del país visitado, y las tarifas cambian por zona, no por país. Un cliente volvió de un viaje largo, le llegó una factura enorme y pidió el detalle. Tu programa lo calcula con dos tablas de búsqueda.

Datos de entrada

País	Zona
Uruguay	Mercosur
Brasil	Mercosur
Chile	Mercosur
España	Europa
Italia	Europa
Estados Unidos	América del Norte

Zona	Minuto de voz	MB de datos	SMS
Mercosur	\$25	\$3	\$10
Europa	\$90	\$12	\$30
América del Norte	\$70	\$9	\$25

País visitado	Minutos	MB	SMS
Brasil	35	800	4
España	20	1500	10
Italia	12	900	2
Japón	5	300	0
Chile	0	2200	1

Tu programa debe

1. Guardar las dos tablas en mapas: país → zona y zona → tarifas.
2. Para cada país visitado, buscar su zona y sus tarifas.
3. Calcular el costo del uso en ese país; si el país no tiene zona, informarlo y no cobrar.
4. Mostrar el detalle y el total.

Salida esperada

```
Brasil (Mercosur): 35 min + 800 MB + 4 SMS = $3315.00
España (Europa): 20 min + 1500 MB + 10 SMS = $20100.00
Italia (Europa): 12 min + 900 MB + 2 SMS = $11940.00
Japón: país sin acuerdo de roaming, no se tarifa
Chile (Mercosur): 0 min + 2200 MB + 1 SMS = $6610.00
Total del viaje: $41965.00
```

Reglas

- Alcanza con mapas clave-valor y búsqueda por clave.
- Montos con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Buscar una clave que no existe no tiene que cortar el programa: detectá el caso y seguí.

Pista (leela solo si te trabás): la búsqueda en dos pasos (país → zona → tarifa) evita repetir las tarifas en cada país: si cambia el precio de una zona, se cambia en un solo lugar.

Bonus (opcional)

- Agregá paquetes: con el "Pase Mercosur" (\$8.000) los primeros 1.000 MB de esa zona son gratis. Decí si le convenía comprarlo.
- Sumá un tope diario por zona y distribuí el uso por días.

Cierre: Es la misma lógica de tarificación por zonas que usan las operadoras para roaming y llamadas internacionales.

Ejercicio 117: El extracto bancario desordenado

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 24/100

Situación: Un banco digital permite descargar el extracto, pero el archivo viene desordenado, con montos en formato argentino (punto de miles y coma decimal) y algunas líneas corruptas por un error de exportación. Rocío quiere un resumen limpio por mes y por categoría, sin que las líneas rotas arruinen los totales.

Datos de entrada

Formato: DD/MM/AAAA;categoría;monto (el monto tiene signo, punto de miles y coma decimal)

```
05/03/2026;Supermercado;-18.450,50
07/03/2026;Sueldo;+950.000,00
12/03/2026;Transporte;-8.000,00
31/02/2026;Supermercado;-5.000,00
18/03/2026;Servicios;-42.310,75
02/04/2026;Supermercado;-22.100,00
02/04/2026;Transporte;-7.500,00
10/04/2026;Servicios;-39.990,00
15/04/2026;Supermercado;-mil pesos
21/04/2026;Sueldo;+950.000,00
28/04/2026;Supermercado;-31.004,25
03/05/2026;Transporte;-9.200,00
```

Tu programa debe

1. Separar cada línea y validar la fecha (tiene que existir en el calendario) y el monto (solo dígitos, puntos y una coma decimal).
2. Convertir el monto al formato numérico: sacar los puntos de miles y cambiar la coma por punto.
3. Agrupar los montos válidos por mes y, dentro del mes, por categoría.
4. Mostrar los meses en orden cronológico con sus categorías en orden alfabético y el neto del mes, y listar las líneas descartadas con su motivo.

Salida esperada

```
03/2026
  Servicios      -42310.75
  Sueldo         950000.00
  Supermercado   -18450.50
  Transporte      -8000.00
  Neto del mes    881238.75
04/2026
  Servicios      -39990.00
  Sueldo         950000.00
  Supermercado   -53104.25
  Transporte      -7500.00
  Neto del mes    849405.75
05/2026
  Transporte      -9200.00
  Neto del mes    -9200.00
Líneas descartadas (2):
línea 4: fecha inválida (31/02/2026)
línea 9: monto inválido (-mil pesos)
```

Reglas

- Alcanza con validación, fechas, parseo de texto y mapas anidados. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos (es un buen ejercicio).
- Montos con 2 decimales, con signo menos cuando son negativos. Formato: categoría en 13 caracteres a la izquierda, monto en 11 a la derecha. Los decimales se muestran con punto y sin separador de miles en la salida.
- El 31/02/2026 no existe: esa línea se descarta.

- Una línea descartada no suma en ningún lado y el programa sigue con la siguiente.

Pista (leela solo si te trabás): validá primero y convertí después. Separar "qué está mal" de "cuánto suma" hace que los errores no contaminen los totales.

Bonus (opcional)

- Detectá líneas duplicadas (misma fecha, categoría y monto) y preguntá si deben contarse dos veces.
- Calculá el promedio de gasto mensual por categoría y marcá los meses que lo superan en más de 20 %.

Cierre: Es el mismo trabajo de limpieza y agrupación que hacen las apps de finanzas personales al importar extractos bancarios.

Ejercicio 118: Viaje con escalas en distintos husos horarios

Nivel 3: Junior III · Datos del mundo real · Temática: Turismo · Dificultad 24/100

Situación: Una agencia de viajes arma itinerarios con escalas y los clientes se confunden con las horas: los horarios de salida y llegada están en la hora local de cada ciudad. Tu programa tiene que calcular la hora local de llegada de cada tramo, el tiempo de escala y el tiempo total real de viaje.

Datos de entrada

Tra mo	Origen (huso)	Destino (huso)	Salida (hora local del origen)	Duración del vuelo
1	Buenos Aires (UTC-3)	Madrid (UTC+2)	10/07/2026 22:35	12 h 20 min
2	Madrid (UTC+2)	Doha (UTC+3)	11/07/2026 18:10	6 h 35 min
3	Doha (UTC+3)	Tokio (UTC+9)	12/07/2026 02:05	10 h 50 min

Tu programa debe

1. Pasar la hora de salida de cada tramo a UTC restando el huso del origen.
2. Sumar la duración del vuelo y pasar la llegada a hora local del destino.
3. Calcular la escala entre tramos como la diferencia (en UTC) entre la salida de un tramo y la llegada del anterior.
4. Mostrar el itinerario y el tiempo total (desde la primera salida hasta la última llegada).

Salida esperada

```
Buenos Aires -> Madrid: sale 10/07 22:35 (UTC-3), llega 11/07 15:55 (UTC+2)
Escala en Madrid: 2 h 15 min
Madrid -> Doha: sale 11/07 18:10 (UTC+2), llega 12/07 01:45 (UTC+3)
Escala en Doha: 0 h 20 min
Doha -> Tokio: sale 12/07 02:05 (UTC+3), llega 12/07 18:55 (UTC+9)
Tiempo total de viaje: 32 h 20 min
```

Reglas

- Alcanza con fechas y horas, husos horarios y aritmética de minutos. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos (es un buen ejercicio).
- Los husos son fijos y están dados (no hay cambios de horario de verano durante el viaje).
- Formato de horas: DD/MM HH:MM; duraciones como "H h MM min".
- Tené en cuenta que la llegada puede caer en otro día.

Pista (leela solo si te trabás): convertí todo a UTC para calcular y volvé a hora local solo para mostrar. Mezclar horas locales de distintas ciudades es la fuente de todos los errores.

Bonus (opcional)

- Marcá las escalas de menos de 1 h 30 min como "conexión ajustada".
- Calculá cuántas horas de diferencia tiene el pasajero al llegar (jet lag) y sugerí a qué hora dormir el primer día.

Cierre: Es la misma conversión que hacen los sistemas de reservas de aerolíneas, donde todos los horarios se guardan en UTC.

Ejercicio 119: Hashtags en tendencia

Nivel 3: Junior III · Datos del mundo real · Temática: Redes sociales · Dificultad 24/100

Situación: Un medio digital deportivo quiere un widget de "tendencias" en su portada. Toma los posts del día, cuenta los hashtags y muestra el top 3. El editor pidió además marcar los hashtags "en ascenso": los que se mencionan más en la segunda mitad del período que en la primera.

Datos de entrada

Posteo	Hora	Hashtags
1	09	#Mundial #Messi #futbol
2	09	#futbol #asado
3	10	#mundial #Scaloni
4	10	#Asado #domingo #FUTBOL
5	11	#Messi #mundial
6	11	#futbol #mundial #Messi
7	12	#domingo #asado
8	12	#mundial #Messi #scaloni

Primera mitad: posts 1 a 4. Segunda mitad: posts 5 a 8.

Tu programa debe

1. Contar las menciones de cada hashtag sin distinguir mayúsculas (se muestran en minúscula).
2. Ordenar por cantidad descendente y, en empate, alfabéticamente.
3. Armar el top 3 incluyendo a todos los que empatan con el tercero.
4. Listar los hashtags en ascenso, en el orden del ranking.

Salida esperada

```
Ranking:
#mundial 5
#futbol 4
#messi 4
#asado 3
#domingo 2
#scaloni 2
Top 3 (con empates): #mundial, #futbol, #messi
En ascenso (más menciones en la segunda mitad): #mundial, #messi
```

Reglas

- Alcanza con mapas clave-valor, conteo y ordenamiento (top-k).
- #Mundial, #mundial y #MUNDIAL son el mismo hashtag.
- Si hay empate en el tercer puesto, el top 3 puede tener más de 3 elementos.

Pista (leela solo si te trabás): usá tres mapas: el total, el de la primera mitad y el de la segunda. Un hashtag que no aparece en un mapa cuenta 0 en esa mitad.

Bonus (opcional)

- Calculá una puntuación de tendencia que pese más las menciones recientes (por ejemplo, peso = hora - 8).
- Si hubiera millones de posts, ¿cómo harías el top 3 sin ordenar todo el ranking? (Lo vas a ver en el nivel 6.)

Cierre: Es la versión simplificada de los algoritmos de "tendencias" de las redes sociales.

Ejercicio 120: Descendientes de la vaca campeona

Nivel 3: Junior III · Datos del mundo real · Temática: Agricultura · Dificultad 24/100

Situación: Una cabaña ganadera de la provincia de Buenos Aires quiere demostrar el valor genético de "Estrella", una vaca campeona de exposición. Tiene los registros de qué vaca es madre de cuál y necesita contar todas sus descendientes y listarlas por generación.

Datos de entrada

Madre	Hijas
Estrella	Luna, Brisa
Luna	Perla, Canela, Nube
Brisa	Aurora
Perla	Dalia
Canela	—
Nube	Flor, Hebra
Aurora	—
Dalia	Iris
Flor	—
Hebra	—
Iris	—

Tu programa debe

1. Guardar el registro en un mapa madre → lista de hijas.
2. Escribir una función recursiva que cuente todas las descendientes de una vaca.
3. Escribir una función recursiva que agrupe a las descendientes por generación (hijas = 1, nietas = 2...).
4. Mostrar el total y cada generación de Estrella, y el total de descendientes de Luna.

Salida esperada

```
Descendientes de Estrella: 10
Generación 1: Luna, Brisa
Generación 2: Perla, Canela, Nube, Aurora
Generación 3: Dalia, Flor, Hebra
Generación 4: Iris
Descendientes de Luna: 7
```

Reglas

- Alcanza con recursión básica y mapas clave-valor.
- Dentro de cada generación, las vacas se listan en el orden en que aparecen al recorrer: primero toda la línea de la primera hija, después la de la segunda, y así.
- Una vaca sin hijas es el caso base: aporta 0 descendientes.

Pista (leela solo si te trabás): las descendientes de una vaca son sus hijas más las descendientes de cada hija. Eso ya es la definición recursiva.

Bonus (opcional)

- Encontrá la generación más numerosa y la descendiente más lejana.
- Dadas dos vacas, encontrá su antepasada común más cercana (vas a volver a este problema en el nivel 6).

Cierre: Es la misma estructura de árbol que usan los registros genealógicos de razas y los sistemas de trazabilidad ganadera.

Ejercicio 121: Consumo por electrodoméstico y por mes

Nivel 3: Junior III · Datos del mundo real · Temática: Clima y energía · Dificultad 25/100

Situación: Una familia instaló enchufes inteligentes que exportan el consumo mensual de cada electrodoméstico en un archivo separado por comas. Quieren ver cuánto consumen por mes, cuánto consume cada aparato en total y cuál tuvo el salto más grande de un mes al siguiente (sospechan del termotanque).

Datos de entrada

Archivo (la primera línea es el encabezado):

```
mes,electrodomestico,kWh
ene,Aire acondicionado,210
ene,Heladera,45
ene,Termotanque,60
feb,Aire acondicionado,185
feb,Heladera,44
feb,Termotanque,58
mar,Aire acondicionado,95
mar,Heladera,42
mar,Termotanque,71
mar,Estufa eléctrica,20
abr,Heladera,40
abr,Termotanque,88
abr,Estufa eléctrica,64
```

Tu programa debe

1. Leer el archivo salteando el encabezado y separar cada línea por comas.
2. Guardar los datos en un mapa anidado: mes → (electrodoméstico → kWh).
3. Calcular el total por mes (en el orden del archivo) y el total por electrodoméstico (en orden alfabético).
4. Encontrar el mayor aumento de un electrodoméstico entre dos meses consecutivos.

Salida esperada

```
Total por mes: ene=315 feb=287 mar=228 abr=192
Total por electrodoméstico:
  Aire acondicionado  490
  Estufa eléctrica    84
  Heladera            171
  Termotanque         277
Mayor aumento entre meses consecutivos: Estufa eléctrica (+44 kWh de mar a abr)
```

Reglas

- Alcanza con parseo de texto separado por comas y mapas anidados.
- Si un electrodoméstico no aparece en un mes, su consumo ese mes es 0.
- Formato: nombre del electrodoméstico en 19 caracteres a la izquierda.
- En empate de aumentos, gana el primero que se encuentra recorriendo los pares de meses en orden y, dentro de cada par, los electrodomésticos en orden de aparición en el archivo.

Pista (leela solo si te trabás): para comparar meses consecutivos necesitás el orden de los meses: guardalo en una lista aparte a medida que los vas viendo, porque el mapa no garantiza orden.

Bonus (opcional)

- Calculá el costo mensual con una tarifa escalonada y mostrá cuánto aporta cada aparato a la factura.

- Detectá aparatos que aparecen o desaparecen de un mes al otro y marcalos como "estacionales".

Cierre: Es el mismo tipo de agregación que hacen las plataformas de hogar inteligente y de eficiencia energética.

Ejercicio 122: Validar un CUIT

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 25/100

Situación: Un sistema de facturación electrónica rechaza comprobantes cuando el CUIT del cliente está mal cargado, y el error aparece recién al enviar la factura. El contador de una pyme quiere validar los CUIT al darlos de alta, con el dígito verificador oficial.

Datos de entrada

CUIT a validar (números ficticios):

- 20-31456789-8
- 30712345671
- 27-38111222-3
- 20-1234567-8
- 45-12345678-9
- 30.71234567.1

Reglas:

- Formato aceptado: 11 dígitos, con o sin guiones (no se aceptan otros separadores).
- Prefijos válidos: 20, 23, 24, 27 (personas) y 30, 33, 34 (empresas).
- Verificador: multiplicar los primeros 10 dígitos por 5, 4, 3, 2, 7, 6, 5, 4, 3, 2 y sumar. Calcular 11 – (suma resto 11). Si da 11, el verificador es 0; si da 10, el CUIT es inválido; si no, es ese número. Tiene que coincidir con el último dígito.

Tu programa debe

1. Quitar los guiones y validar que queden exactamente 11 dígitos.
2. Validar el prefijo e identificar si es persona o empresa.
3. Calcular el dígito verificador y compararlo con el último dígito.
4. Mostrar el resultado de cada CUIT con el primer motivo de rechazo, si lo hay.

Salida esperada

```
20-31456789-8: válido (persona)
30712345671: válido (empresa)
27-38111222-3: dígito verificador incorrecto
20-1234567-8: formato inválido
45-12345678-9: prefijo 45 desconocido
30.71234567.1: formato inválido
```

Reglas

- Alcanza con validación, parseo y cálculo de dígito verificador.
- Orden de validación: formato, prefijo, verificador.
- Escribí la función que calcula el verificador por separado: vas a reutilizarla en el bonus.

Pista (leela solo si te trabás): después de quitar los guiones, el texto tiene que ser de 11 caracteres y todos dígitos. Un CUIT con puntos no pasa esa prueba.

Bonus (opcional)

- Dado un DNI y el sexo registrado, generá el CUIT correspondiente, incluyendo el caso en que el verificador da 10 y el prefijo cambia a 23.
- Normalizá la salida: mostrá siempre los CUIT válidos con el formato XX-XXXXXXXX-X.

Cierre: Es la misma validación de módulo 11 que aplican los sistemas de facturación y los organismos de recaudación.

Ejercicio 123: Reservas duplicadas del hotel

Nivel 3: Junior III · Datos del mundo real · Temática: Turismo · Dificultad 25/100

Situación: Un hotel de Bariloche vende por su web, por agencias y por teléfono, y en temporada alta aparecen reservas duplicadas (la misma persona reserva por dos canales) y habitaciones vendidas dos veces para las mismas noches. La recepción necesita detectar ambos problemas antes de que lleguen los huéspedes.

Datos de entrada

Reserva	Huésped	DNI	Habitación	Entrada	Salida	Canal
R1	Gómez, Paula	30111222	12	2026-01-10	2026-01-15	web
R2	Suárez, Leo	28999000	14	2026-01-12	2026-01-14	agencia
R3	Gómez, Paula	30111222	12	2026-01-10	2026-01-15	agencia
R4	Ortiz, Ema	35123456	14	2026-01-13	2026-01-16	web
R5	Luna, Tito	27555444	21	2026-01-11	2026-01-12	teléfono
R6	Suárez, Leo	28999000	15	2026-01-12	2026-01-14	web
R7	Ríos, Ada	33222111	21	2026-01-12	2026-01-13	web

Tu programa debe

1. Detectar reservas duplicadas: mismo DNI, misma fecha de entrada y misma fecha de salida (aunque cambien el canal o la habitación). Se conserva la primera.
2. Con las reservas no duplicadas, detectar sobreventa: misma habitación con estadías que se superponen.
3. Mostrar las duplicadas, la sobreventa y la cantidad de reservas válidas.

Salida esperada

Duplicadas:
 R3 duplica a R1 (Gómez, Paula, 2026-01-10 a 2026-01-15, por agencia)
 R6 duplica a R2 (Suárez, Leo, 2026-01-12 a 2026-01-14, por web)
 Sobreventa:
 Habitación 14: R2 y R4 se superponen
 Reservas válidas: 5 de 7

Reglas

- Alcanza con conjuntos o mapas con claves compuestas y comparación de intervalos de fechas.
- La fecha de salida no es una noche ocupada: una estadía que sale el 14 no se superpone con otra que entra el 14.
- Las fechas en formato AAAA-MM-DD se pueden comparar como texto.
- Si no hay sobreventa, se muestra "ninguna".

Pista (leela solo si te trabás): una clave compuesta es simplemente la tupla (DNI, entrada, salida). Usarla en un mapa te dice en tiempo constante si ya viste esa reserva y cuál fue la primera.

Bonus (opcional)

- Para la sobreventa, sugerí una habitación alternativa libre en esas fechas.
- Detectá posibles duplicados con DNI mal tipeado: mismo nombre y mismas fechas, DNI distinto.

Cierre: Es el mismo control que hacen los channel managers hoteleros que sincronizan varias plataformas de reservas.

Ejercicio 124: La carta ordenada por precio y nombre

Nivel 3: Junior III · Datos del mundo real · Temática: Gastronomía · Dificultad 25/100

Situación: Un restaurante imprime su carta todas las semanas y el diseñador pidió un orden fijo: primero por sección (entradas, principales, postres), dentro de cada sección de más barato a más caro y, a igual precio, alfabéticamente. Hasta ahora lo hacían a mano y siempre quedaba algo desordenado.

Datos de entrada

Plato	Sección	Precio
Flan con dulce de leche	Postres	\$4.200
Milanesa a caballo	Principales	\$11.800
Empanadas (2)	Entradas	\$3.600
Ñoquis del 29	Principales	\$8.900
Ensalada rusa	Entradas	\$3.600
Bife de chorizo	Principales	\$15.200
Arroz con leche	Postres	\$3.900
Provoleta	Entradas	\$5.100
Ávila vegana	Principales	\$8.900
Almendrado	Postres	\$4.200

Tu programa debe

1. Definir el orden de las secciones: Entradas, Principales, Postres.
2. Ordenar los platos con un criterio compuesto: sección, precio ascendente y nombre alfabético.
3. Imprimir la carta con el título de cada sección en mayúsculas y los platos debajo.

Salida esperada

ENTRADAS		
Empanadas (2)	\$	3600
Ensalada rusa	\$	3600
Provoleta	\$	5100
PRINCIPALES		
Ávila vegana	\$	8900
Ñoquis del 29	\$	8900
Milanesa a caballo	\$	11800
Bife de chorizo	\$	15200
POSTRES		
Arroz con leche	\$	3900
Almendrado	\$	4200
Flan con dulce de leche	\$	4200

Reglas

- Alcanza con ordenamiento con criterio múltiple.
- El orden alfabético ignora tildes y mayúsculas: "Ávila" va antes que "Ñoquis", y la Ñ va después de la N.
- Formato: nombre en 26 caracteres a la izquierda y precio en 6 a la derecha precedido de \$.

Pista (leela solo si te trabás): la clave de orden es una tupla (posición de la sección, precio, nombre normalizado). Comparar tuplas compara el primer elemento y, si empatan, el siguiente.

Bonus (opcional)

- Implementá vos un ordenamiento (por ejemplo, por selección) que use una función de comparación con los tres criterios.

- Agregá un cuarto criterio: los platos marcados como "del día" van primero dentro de su sección.

Cierre: Es el mismo ordenamiento con varios criterios que aplica cualquier tienda online cuando elegís "ordenar por categoría y precio".

Ejercicio 125: Traductor de código Morse

Nivel 3: Junior III · Datos del mundo real · Temática: Telecomunicaciones · Dificultad 25/100

Situación: Un club de radioaficionados de la costa practica emergencias marítimas usando código Morse. Para entrenar a los socios nuevos, quieren un traductor en las dos direcciones que además marque los símbolos que no reconoce, en lugar de inventar letras.

Datos de entrada

Tabla de código (letra y su código):

A .-	B -...	C -.-.	D -..	E .	F ..-
G --.	H	I ..	J .---	K -.-	L .-..
M --	N -.	Ñ --.-	O ---	P .---	Q --.-
R .-.	S ...	T -	U ...	V ...-	W .--
X -.-.	Y -.-.	Z ---.	0 -----	1 .----	2 ..----
3 ...--	4-	5	6 -.....	7 -.....	8 ---..
9 ----.					

- Formato: los símbolos de una misma palabra se separan con un espacio y las palabras se separan con " / ".
- Texto a enviar: SOS barco en zona 3
- Mensaje recibido: -.-. --- .-. .-.-. --- /-.-. --- / .---- .---- / .----

Tu programa debe

1. Guardar la tabla en un mapa letra → código y construir el mapa inverso código → letra.
2. Traducir el texto a Morse (en mayúsculas); los caracteres sin código se traducen como "?".
3. Traducir el mensaje recibido a texto; los códigos desconocidos se traducen como "?".
4. Verificar que traducir de ida y vuelta devuelve el texto en mayúsculas.

Salida esperada

```

Texto: SOS barco en zona 3
Morse: ... --- .-.-. --- / -.-. .-.-. --- / .-.-. / --- --- .-.-. / ----
Recibido: -.-. --- .-. .-.-. --- / ... .-.-. --- / .---- .---- / .----
Traducido: COPIADO FARO 12 ?
Ida y vuelta: SOS BARCO EN ZONA 3

```

Reglas

- Alcanza con mapas clave-valor y parseo de texto.
- El mapa inverso se construye recorriendo el original, no se escribe a mano.
- Los espacios múltiples en el texto de entrada se tratan como uno solo.

Pista (leela solo si te trabás): para decodificar, separá primero por " / " (palabras) y después cada palabra por espacios (letras). Son dos niveles de separación.

Bonus (opcional)

- Convertí el Morse en una secuencia de tiempos: punto = 1 unidad, raya = 3, entre símbolos 1, entre letras 3 y entre palabras 7. Calculá la duración total del mensaje.
- Decodificá un mensaje recibido sin espacios entre letras encontrando todas las interpretaciones posibles (vas a necesitar recursión).

Cierre: Es la misma idea de tabla de codificación que está detrás de cualquier protocolo de comunicación, desde el telégrafo hasta los códigos de caracteres de las computadoras.

Lote 06: ejercicios 126 a 150

Ejercicio 126: Pacientes por obra social

Nivel 3: Junior III · Datos del mundo real · Temática: Salud · Dificultad 26/100

Situación: Un centro médico factura a cada obra social por separado y el área administrativa necesita, a fin de mes, cuántas consultas y cuántos pacientes distintos atendió de cada una, y qué especialidad le piden más. Hoy lo cuentan a mano desde la planilla de turnos.

Datos de entrada

Formato: paciente;obra social;especialidad;fecha

```
Pérez, Lía;OSDE;Clínica;2026-08-01
Gómez, Iván;PAMI;Cardiología;2026-08-01
Pérez, Lía;OSDE;Dermatología;2026-08-03
Ruiz, Ema;IOMA;Clínica;2026-08-04
Gómez, Iván;PAMI;Cardiología;2026-08-08
Sosa, Leo;OSDE;Clínica;2026-08-09
Díaz, Ana;PAMI;Traumatología;2026-08-10
Ruiz, Ema;IOMA;Pediatría;2026-08-11
Vera, Tomás;OSDE;Dermatología;2026-08-12
Díaz, Ana;PAMI;Cardiología;2026-08-15
Luna, Paz;Particular;Clínica;2026-08-16
```

Tu programa debe

1. Contar las consultas por obra social.
2. Contar los pacientes distintos por obra social usando un conjunto por cada una.
3. Encontrar la especialidad más pedida de cada obra social (mapa de mapas).
4. Mostrar las obras sociales de más a menos consultas (en empate, alfabéticamente).

Salida esperada

```
OSDE      4 consultas | 3 pacientes | más pedida: Clínica (2)
PAMI      4 consultas | 2 pacientes | más pedida: Cardiología (3)
IOMA      2 consultas | 1 paciente | más pedida: Clínica (1)
Particular 1 consulta  | 1 paciente | más pedida: Clínica (1)
```

Reglas

- Alcanza con mapas clave-valor, conjuntos y ordenamiento.
- En empate de especialidades, gana la primera en orden alfabético.
- Formato: nombre de la obra social en 11 caracteres a la izquierda.

Pista (leela solo si te trabás): la diferencia entre "consultas" y "pacientes" es la diferencia entre contar y contar distintos. Lo segundo es exactamente lo que resuelve un conjunto.

Bonus (opcional)

- Calculá cuánto facturar a cada obra social con una tabla de valores por especialidad.
- Detectá pacientes que consultaron la misma especialidad dos veces en menos de 10 días (posible control).

Cierre: Es el mismo reporte que generan los sistemas de gestión de clínicas para facturar a las prepagas y obras sociales.

Ejercicio 127: Suscripciones que vencen este mes

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 26/100

Situación: Cami tiene seis suscripciones con ciclos distintos y siempre la sorprende un débito que no esperaba. Quiere una lista con la próxima fecha de renovación de cada una y el total que se le va a debitar en lo que queda del mes. Hoy es 10/06/2026.

Datos de entrada

Hoy: 10/06/2026

Servicio	Fecha de alta	Ciclo	Precio
Streaming	31/03/2025	mensual	\$8.900
Nube 2 TB	15/06/2025	anual	\$52.000
Gimnasio	31/03/2026	trimestral	\$45.000
Diario	05/12/2024	mensual	\$6.500
Antivirus	02/07/2025	anual	\$30.000
Música	20/01/2026	mensual	\$4.300

Las renovaciones caen cada 1, 3 o 12 meses contados desde la fecha de alta, siempre con el día de la fecha de alta. Si ese día no existe en el mes (por ejemplo, 31 en junio), la renovación cae el último día de ese mes.

Tu programa debe

1. Escribir una función que sume n meses a una fecha manteniendo el día original y ajustando al último día del mes cuando haga falta.
2. Para cada suscripción, encontrar la primera renovación igual o posterior a hoy.
3. Marcar las que caen en el mes actual y sumar sus precios.
4. Mostrar la lista y el total.

Salida esperada

```
Streaming    (mensual) próxima renovación 30/06/2026 -> ESTE MES $8900
Nube 2 TB    (anual) próxima renovación 15/06/2026 -> ESTE MES $52000
Gimnasio     (trimestral) próxima renovación 30/06/2026 -> ESTE MES $45000
Diario       (mensual) próxima renovación 05/07/2026
Antivirus    (anual) próxima renovación 02/07/2026
Música       (mensual) próxima renovación 20/06/2026 -> ESTE MES $4300
A pagar en lo que queda de junio: $110200
```

Reglas

- Alcanza con fechas y filtrado. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- Cada renovación se calcula desde la fecha de alta (alta + k ciclos), no sumando meses a la renovación anterior: así el día 31 no se "pierde" después de pasar por un mes corto.
- Una renovación que cae hoy cuenta como de este mes.
- Formato: nombre en 11 caracteres a la izquierda y fechas DD/MM/AAAA.

Pista (leela solo si te trabás): sumar meses es sumar al número de mes, ajustar el año cuando pasa de 12 y recortar el día al máximo del mes destino (que depende de si el año es bisiesto).

Bonus (opcional)

- Calculá el gasto anual total en suscripciones.
- Mostrá un calendario de débitos de los próximos 6 meses, agrupado por mes.

Cierre: Es la lógica de facturación recurrente que usan las plataformas de suscripción y los procesadores de pagos.

Ejercicio 128: Planilla de jugadas del partido de básquet

Nivel 3: Junior III · Datos del mundo real · Temática: Deportes · Dificultad 26/100

Situación: La liga de básquet del interior no tiene estadísticos oficiales y un voluntario carga cada jugada en una planilla de texto durante el partido. Al terminar, quieren publicar la planilla de estadísticas (box score) de cada equipo: puntos, tiros de campo y rebotes por jugador.

Datos de entrada

Formato: cuarto reloj equipo número tipo [resultado]. Tipos: 1P (tiro libre), 2P, 3P (tiros de campo, con resultado anotado o errado) y REB (rebote, sin resultado).

```
Q1 09:40 LOC 7 2P anotado
Q1 09:12 VIS 11 3P errado
Q1 09:10 LOC 12 REB
Q1 08:55 LOC 12 2P anotado
Q1 08:30 VIS 4 2P anotado
Q1 08:02 LOC 7 3P anotado
Q2 09:15 VIS 11 3P anotado
Q2 08:50 LOC 5 1P anotado
Q2 08:50 LOC 5 1P errado
Q2 08:31 VIS 4 REB
Q2 08:10 VIS 4 2P errado
Q2 07:45 LOC 7 2P errado
Q2 07:44 LOC 12 REB
Q2 07:40 LOC 12 2P anotado
Q3 06:20 VIS 11 3P anotado
Q3 05:58 VIS 20 2P anotado
```

Tu programa debe

1. Separar cada línea en sus campos (la cantidad de campos varía según el tipo).
2. Acumular por jugador (equipo + número): puntos, tiros de campo intentados y convertidos, y rebotes.
3. Calcular el porcentaje de campo de cada jugador (los tiros libres no cuentan para el campo).
4. Mostrar cada equipo con su total y sus jugadores ordenados por puntos (en empate, por número).

Salida esperada

```
LOC: 10 puntos
#7 5 pts | campo 2/3 (66.7%) | 0 reb
#12 4 pts | campo 2/2 (100.0%) | 2 reb
#5 1 pts | campo 0/0 (-) | 0 reb
VIS: 10 puntos
#11 6 pts | campo 2/3 (66.7%) | 0 reb
#4 2 pts | campo 1/2 (50.0%) | 1 reb
#20 2 pts | campo 1/1 (100.0%) | 0 reb
```

Reglas

- Alcanza con parseo de texto y mapas clave-valor con clave compuesta.
- Porcentaje con 1 decimal (0,5 hacia arriba); si el jugador no tiró al campo, se muestra "-". Los decimales se muestran con punto y sin separador de miles en la salida.
- Formato: número de camiseta en 3 caracteres a la izquierda (después de #), puntos en 2 a la derecha.
- Primero se muestra LOC y después VIS.

Pista (leela solo si te trabás): el jugador 7 del local y el 7 del visitante son personas distintas: la clave tiene que ser el par (equipo, número).

Bonus (opcional)

- Calculá los puntos de cada equipo por cuarto.
- Detectá la racha de puntos seguidos más larga de cada equipo (parcial sin que el rival anote).

Cierre: Es el mismo procesamiento de "play by play" que alimenta las estadísticas de las ligas profesionales.

Ejercicio 129: Capítulos en números romanos

Nivel 3: Junior III · Datos del mundo real · Temática: Educación · Dificultad 26/100

Situación: Una editorial numera los capítulos y los tomos de sus colecciones en números romanos, y los correctores encontraron errores como "IIII" o "IC" en las pruebas de imprenta. Quieren un conversor en las dos direcciones que además rechace los números romanos mal escritos.

Datos de entrada

Números a convertir a romano: 4, 9, 14, 40, 90, 400, 1994, 2026

Romanos a convertir a número: XLII, MCMXCIX, IIII, IC, MMXXVI, XM, VL

Valores y formas restadas válidas:

```
I=1  V=5  X=10  L=50  C=100  D=500  M=1000
IV=4  IX=9  XL=40  XC=90  CD=400  CM=900
```

Tu programa debe

1. Convertir de número a romano usando una tabla ordenada de valores que incluya las formas restadas.
2. Convertir de romano a número: si un símbolo vale menos que el siguiente, se resta; si no, se suma.
3. Validar que el romano sea canónico: convertirlo a número y de vuelta a romano tiene que dar el mismo texto.
4. Mostrar los resultados de ambas conversiones.

Salida esperada

```
A romano: 4=IV, 9=IX, 14=XIV, 40=XL, 90=XC, 400=CD, 1994=MCMXCIV, 2026=MMXXVI
XLII: 42
MCMXCIX: 1999
IIII: no es un número romano válido
IC: no es un número romano válido
MMXXVI: 2026
XM: no es un número romano válido
VL: no es un número romano válido
```

Reglas

- Alcanza con mapas clave-valor y conversión.
- El rango válido es de 1 a 3999.
- Un romano no canónico (IIII, IC, XM, VL) se rechaza aunque la regla de suma y resta le dé un valor.

Pista (leela solo si te trabás): la validación por ida y vuelta es una técnica muy potente: en lugar de programar todas las reglas de escritura, usás la conversión correcta como referencia.

Bonus (opcional)

- Convertí fechas: 28/09/2026 → XXVIII.IX.MMXXVI.
- Sumá dos números romanos sin pasar por los números arábigos, manipulando solo los símbolos.

Cierre: Es el mismo tipo de conversión de sistemas de numeración que usan los programas de maquetación y los relojes.

Ejercicio 130: Los que no te siguen de vuelta

Nivel 3: Junior III · Datos del mundo real · Temática: Redes sociales · Dificultad 26/100

Situación: Ro maneja la cuenta de su emprendimiento de velas y quiere ordenar a quién sigue: dejar de seguir a quienes no le devuelven el follow y devolverle el follow a quienes la siguen. La red social muestra las dos listas, pero con los nombres de usuario escritos de forma inconsistente.

Datos de entrada

Cuentas que Ro sigue: @mati.r, @Cande_ok, @juli, @FERNI, @laura.p, @tomas_22, @nico, @ro.ok

Seguidores de Ro: @cande_ok, @juli, @pau_m, @ferni, @nico, @vale.v, @laura.p

Tu programa debe

1. Normalizar los usuarios a minúsculas y guardar cada lista en un conjunto.
2. Calcular los mutuos, los que no la siguen de vuelta y los que ella no sigue de vuelta.
3. Mostrar cada grupo ordenado alfabéticamente con su cantidad.

Salida esperada

```
Mutuos (5): @cande_ok, @ferni, @juli, @laura.p, @nico
No te siguen de vuelta (3): @mati.r, @ro.ok, @tomas_22
No seguís de vuelta (2): @pau_m, @vale.v
```

Reglas

- Alcanza con conjuntos y la operación diferencia.
- Los nombres de usuario no distinguen mayúsculas: @FERNI y @ferni son la misma cuenta.
- Tiene que funcionar en tiempo razonable con 100.000 cuentas en cada lista: nada de recorrer una lista dentro de otra.

Pista (leela solo si te trabás): "los que sigo pero no me siguen" es exactamente la diferencia entre el conjunto de seguidos y el de seguidores.

Bonus (opcional)

- Guardá una foto de los seguidores de cada semana y calculá quién te dejó de seguir entre dos fotos.
- Ordená los que no te siguen por última interacción, a partir de un mapa usuario → fecha.

Cierre: Es exactamente lo que hacen las apps de terceros que analizan seguidores en redes sociales.

Ejercicio 131: Direcciones mal cargadas

Nivel 3: Junior III · Datos del mundo real · Temática: Logística y envíos · Dificultad 27/100

Situación: Una empresa de mensajería pierde plata con los envíos que vuelven por "dirección inexistente". Las direcciones las cargan los vendedores de distintas tiendas, cada uno a su manera. Antes de despachar, quieren normalizarlas y rechazar las que no tienen lo mínimo indispensable.

Datos de entrada

Direcciones cargadas:

- av. corrientes 1234 piso 3 dto b
- GRAL PAZ 5400 cp 1419
- calle san martín
- Avda de Mayo 760
- pje. la rioja 22 cp 50000
- belgrano 1050 dto 4a cp 5000

Reglas:

- Se pasa todo a minúsculas y se separa por espacios (los espacios repetidos no importan).
- La calle son las palabras antes del primer número. Abreviaturas: av, av., avda, avenida → Av.; gral, gral. → General; pje, pje., pasaje → Pasaje; la palabra calle se elimina. Las palabras de, la y del quedan en minúscula; el resto se capitaliza.
- La primera palabra formada solo por dígitos es la altura. Sin altura → ERROR: falta la altura.
- Después pueden venir, en cualquier orden: piso N, dto X o depto X (el depto va en mayúsculas), cp NNNN (4 dígitos exactos; si no, ERROR: código postal inválido). Cualquier otra cosa → ERROR: dato no reconocido.
- Salida: Calle altura[, piso N][, depto X][(CP NNNN)].

Tu programa debe

1. Escribir una función que reciba una dirección y devuelva la versión normalizada o un mensaje de error.
2. Procesar la calle con el mapa de abreviaturas.
3. Extraer la altura y los datos opcionales, validando el código postal.
4. Mostrar cada dirección original junto a su resultado.

Salida esperada

```
"av. corrientes 1234 piso 3 dto b" -> Av. Corrientes 1234, piso 3, depto B
"GRAL PAZ 5400 cp 1419" -> General Paz 5400 (CP 1419)
"calle san martín" -> ERROR: falta la altura
"Avda de Mayo 760" -> Av. de Mayo 760
"pje. la rioja 22 cp 50000" -> ERROR: código postal inválido (50000)
"belgrano 1050 dto 4a cp 5000" -> Belgrano 1050, depto 4A (CP 5000)
```

Reglas

- Alcanza con validación, parseo y normalización de texto, y un mapa de abreviaturas.
- El primer error encontrado corta el procesamiento de esa dirección.
- Las abreviaturas con punto final ("pje.") se buscan también sin el punto.

Pista (leela solo si te trabás): recorré las palabras con un índice y consumilas por etapas: primero todas las de la calle, después la altura, después los pares clave-valor (piso, depto, cp).

Bonus (opcional)

- Validá el código postal contra un rango por provincia (por ejemplo, 1000–1499 CABA).

- Soportá el formato de código postal argentino de 8 caracteres (una letra, 4 dígitos y 3 letras).

Cierre: Es la normalización de direcciones que hacen los sistemas de logística antes de geolocalizar un envío.

Ejercicio 132: Las torres del museo de ciencias

Nivel 3: Junior III · Datos del mundo real · Temática: Ciencia · Dificultad 27/100

Situación: El museo de ciencias de la ciudad tiene una instalación con tres postes y discos de madera: hay que pasar la torre de un poste a otro moviendo un disco por vez y sin poner nunca un disco grande sobre uno chico. Quieren una pantalla que muestre la solución paso a paso y cuánto tardaría con muchos discos.

Datos de entrada

Postes: Izquierda (origen), Centro (auxiliar), Derecha (destino).

Discos numerados del 1 (el más chico) al n (el más grande). Cantidades a analizar: 3 (con todos los movimientos), 10, 20 y 64 (solo la cantidad).

Tu programa debe

1. Escribir una función recursiva que resuelva la torre de n discos: mover $n - 1$ discos al poste auxiliar, mover el disco n al destino y mover los $n - 1$ discos del auxiliar al destino.
2. Listar los movimientos para 3 discos.
3. Calcular la cantidad de movimientos para 10, 20 y 64 discos ($2^n - 1$).
4. Calcular cuántos años tardaría con 64 discos a un movimiento por segundo (años de 365 días, redondeo hacia abajo).

Salida esperada

```
Movimientos para 3 discos (7):
1. Disco 1: Izquierda -> Derecha
2. Disco 2: Izquierda -> Centro
3. Disco 1: Derecha -> Centro
4. Disco 3: Izquierda -> Derecha
5. Disco 1: Centro -> Izquierda
6. Disco 2: Centro -> Derecha
7. Disco 1: Izquierda -> Derecha
10 discos: 1023 movimientos
20 discos: 1048575 movimientos
64 discos: 18446744073709551615 movimientos
Con 64 discos a un movimiento por segundo: 584942417355 años
```

Reglas

- Alcanza con recursión.
- Para 64 discos no generes los movimientos: calculá la cantidad.
- $2^{64} - 1 = 18.446.744.073.709.551.615$ no entra en un entero de 64 bits con signo: usá enteros de precisión arbitraria o sin signo.
- El caso base es 0 discos: no hay nada que mover.

Pista (leela solo si te trabás): confiá en la recursión: si sabés mover $n - 1$ discos (aunque todavía no sepas cómo), mover n es un paso más. La función se llama a sí misma con los postes en otro orden.

Bonus (opcional)

- Demostrá (o verificá con tu programa) que la cantidad mínima de movimientos es $2^n - 1$.
- Resolvé la torre sin recursión, alternando entre mover el disco 1 y el único otro movimiento válido.

Cierre: Las Torres de Hanói son el ejemplo clásico para entender recursión, y la misma idea de "dividir en un problema más chico" está en cientos de algoritmos reales.

Ejercicio 133: Puntos de fidelidad por categoría

Nivel 3: Junior III · Datos del mundo real · Temática: E-commerce · Dificultad 27/100

Situación: Un supermercado de barrio lanzó un programa de puntos: cada categoría de producto suma distinto, y los clientes que ya son "oro" duplican sus puntos. El cajero no tiene idea de cuántos puntos lleva cada cliente, así que te piden calcularlo y asignar el nivel de cada uno.

Datos de entrada

Categoría	Puntos por cada \$100 completos
Electro	1
Almacén	2
Perfumería	3

Clientes que ya son oro (duplican puntos): Maru

Niveles por puntos acumulados: 3.000 o más → Oro; 1.500 o más → Plata; menos → Bronce.

Cliente	Categoría	Monto
Maru	Almacén	\$45.000
Tito	Electro	\$380.000
Maru	Perfumería	\$22.000
Sil	Almacén	\$61.500
Tito	Almacén	\$12.999
Sil	Juguetería	\$30.000
Sil	Perfumería	\$18.000
Maru	Electro	\$99.000

Tu programa debe

1. Calcular los puntos de cada compra: \$100 completos del monto × puntos de la categoría (× 2 si el cliente es oro).
2. Acumular los puntos por cliente en un mapa.
3. Asignar el nivel según el total.
4. Mostrar los clientes de mayor a menor puntaje y avisar las compras de categorías que no suman.

Salida esperada

```
Maru  5100 puntos -> Oro
Tito  4058 puntos -> Oro
Sil   1770 puntos -> Plata
Aviso: Juguetería no suma puntos (Sil)
```

Reglas

- Alcanza con mapas clave-valor y reglas simples.
- "\$100 completos": \$12.999 son 129 veces \$100 (división entera).
- Formato: nombre en 5 caracteres a la izquierda, puntos en 5 a la derecha. Empates por nombre alfabético.
- Los avisos van al final, en el orden de las compras.

Pista (leela solo si te trabás): las reglas de puntos por categoría son un mapa: si mañana agregan "Bazar", se agrega una entrada al mapa y el programa no cambia.

Bonus (opcional)

- Agregá un tope de 500 puntos por compra.

- Hacé que el nivel se recalcule compra a compra: si un cliente pasa a Oro a mitad de mes, sus compras siguientes ya duplican.

Cierre: Es la lógica de los programas de fidelización de supermercados, aerolíneas y tarjetas de crédito.

Ejercicio 134: Cumpleaños del mes en la oficina

Nivel 3: Junior III · Datos del mundo real · Temática: Recursos humanos · Dificultad 27/100

Situación: En una oficina de 40 personas, la encargada de "cultura" compra una torta para cada cumpleaños y siempre se entera tarde. Quiere la lista del mes que viene ordenada por día, con la edad que cumple cada uno, y saber cuál es el próximo cumpleaños desde hoy (15/06/2026).

Datos de entrada

Hoy: 15/06/2026. Mes a listar: julio.

Empleado	Fecha de nacimiento
Lorena	22/07/1988
Hugo	28/02/1975
Nadia	03/07/1996
Brian	29/02/2000
Pilar	20/06/1991
Mauro	22/07/1983
Elsa	14/06/1969

Quien nació un 29/02 festeja el 28/02 en los años no bisiestos.

Tu programa debe

1. Filtrar a quienes cumplen en julio y ordenarlos por día (en empate, por nombre).
2. Calcular la edad que cumple cada uno en 2026.
3. Calcular el próximo cumpleaños de cada empleado a partir de hoy (si ya pasó este año, el del año siguiente) y encontrar el más cercano.
4. Informar en qué fecha festeja Brian su próximo cumpleaños.

Salida esperada

```
Cumpleaños de julio:
03/07 Nadia (cumple 30)
22/07 Lorena (cumple 38)
22/07 Mauro (cumple 43)
Próximo cumpleaños desde hoy: Pilar el 20/06/2026 (faltan 5 días)
Brian (29/02) festeja el 28/02/2027
```

Reglas

- Alcanza con fechas y ordenamiento. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- Si alguien cumple hoy, su próximo cumpleaños es hoy (faltan 0 días).
- Si varios empatan como próximos, se listan todos, en orden alfabético y separados por coma.

Pista (leela solo si te trabás): para el próximo cumpleaños, armá la fecha con el día y mes de nacimiento y el año actual; si quedó antes de hoy, usá el año siguiente. El 29/02 necesita un caso especial.

Bonus (opcional)

- Agrupá los cumpleaños del año por trimestre para planificar el presupuesto de tortas.
- Calculá la edad exacta en años, meses y días de cada empleado a la fecha de hoy.

Cierre: Es la misma lógica de "próximas fechas" que usan los recordatorios de cumpleaños de calendarios y redes sociales.

Ejercicio 135: Intercalar dos listas ordenadas de transacciones

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 27/100

Situación: Una billetera virtual procesa cobros por dos procesadores de pago distintos y cada uno entrega su propio listado ya ordenado por hora. El equipo de conciliación necesita un único listado cronológico, sin transacciones duplicadas (a veces una transacción viene informada por los dos procesadores).

Datos de entrada

Procesador A:

ID	Hora	Monto
T01	10:00:05	\$1.500
T03	10:00:40	\$8.200
T04	10:01:10	\$450
T07	10:03:00	\$12.000

Procesador B:

ID	Hora	Monto
T02	10:00:20	\$3.100
T03	10:00:40	\$8.200
T05	10:01:55	\$700
T06	10:02:30	\$2.400
T08	10:04:10	\$990

Tu programa debe

1. Intercalar las dos listas en una sola ordenada por hora, recorriendo ambas con un índice cada una (sin reordenar todo).
2. Si las horas empatan, va primero la de A.
3. Descartar las transacciones cuyo ID ya fue agregado, informando de qué procesador venía el duplicado.
4. Mostrar el listado final, los duplicados y cuántas comparaciones de horas se hicieron.

Salida esperada

```
10:00:05 T01 $ 1500 (A)
10:00:20 T02 $ 3100 (B)
10:00:40 T03 $ 8200 (A)
10:01:10 T04 $ 450 (A)
10:01:55 T05 $ 700 (B)
10:02:30 T06 $ 2400 (B)
10:03:00 T07 $ 12000 (A)
10:04:10 T08 $ 990 (B)
Duplicadas descartadas: T03 (repetido en B)
Comparaciones: 8
```

Reglas

- Alcanza con intercalación de listas ordenadas y un conjunto para los duplicados.
- No vale concatenar y ordenar: la idea es aprovechar que cada lista ya viene ordenada (costo proporcional a la suma de los largos).
- Una comparación es cada vez que se comparan las horas de A y B; cuando una lista se termina, el resto de la otra se copia sin comparar.
- Formato: hora, ID, monto en 6 caracteres a la derecha y la fuente entre paréntesis.

Pista (leela solo si te trabás): en cada paso mirás el primer elemento pendiente de cada lista y tomás el menor. Es la mitad del algoritmo de ordenamiento por mezcla (merge sort).

Bonus (opcional)

- Generalizá para k procesadores (vas a ver cómo hacerlo eficientemente con un heap en el nivel 6).
- Detectá transacciones con el mismo ID pero distinto monto y marcalas para revisión.

Cierre: Es el paso de "merge" que hacen los sistemas de conciliación y también el corazón del algoritmo merge sort.

Ejercicio 136: Lecturas dudosas de la estación meteorológica

Nivel 3: Junior III · Datos del mundo real · Temática: Clima y energía · Dificultad 27/100

Situación: Una estación meteorológica automática de la Patagonia manda una lectura cada 10 minutos, pero con viento fuerte el módem se cae, se reconecta y a veces reenvía datos o los manda con formato raro. El servicio de pronóstico necesita un reporte de calidad antes de usar los datos.

Datos de entrada

Formato: AAAA-MM-DD HH:MM, temperatura, humedad

```
2026-07-01 06:00,4.2,88
2026-07-01 06:10,4.0,89
2026-07-01 06:20,3.9,91
2026-07-01 06:20,3.9,91
2026-07-01 06:40,3.1,93
2026-07-01 06:50,2.8,104
2026-07-01 07:00,2.5,95
2026-07-01 7:10,2.4,95
2026-07-01 07:20,2.6,94
2026-07-01 07:50,3.4,90
```

Tu programa debe

1. Validar el formato exacto de fecha y hora (dos dígitos en horas y minutos).
2. Validar que la humedad esté entre 0 y 100.
3. Comparar cada lectura con la última lectura con fecha válida: si es la misma hora, es un duplicado; si pasaron más de 10 minutos, faltan lecturas.
4. Mostrar los problemas en orden y un resumen con la cantidad de lecturas válidas y la temperatura mínima entre ellas.

Salida esperada

```
Línea 4: registro duplicado (2026-07-01 06:20)
Línea 5: faltan 1 lectura(s) antes de 06:40
Línea 6: humedad imposible (104%)
Línea 8: formato de fecha y hora inválido (2026-07-01 7:10)
Línea 9: faltan 1 lectura(s) antes de 07:20
Línea 10: faltan 2 lectura(s) antes de 07:50
Lecturas válidas: 7 | Temperatura mínima válida: 2.5 °C
```

Reglas

- Alcanza con validación, parseo y fechas. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- Una línea con fecha inválida se descarta entera y no cuenta como "última lectura".
- Un duplicado se informa y se descarta (no se evalúa su humedad ni cuenta como válida).
- Una lectura con humedad imposible se informa, no cuenta como válida, pero su hora sí sirve para detectar huecos.
- Cantidad de lecturas faltantes = minutos de diferencia / 10 - 1. Se escribe "lectura(s)" siempre igual.

Pista (leela solo si te trabás): guardá la fecha de la última lectura con hora válida en una variable. Todos los chequeos de secuencia se hacen contra ella.

Bonus (opcional)

- Completá los huecos por interpolación lineal y generá la serie completa cada 10 minutos.
- Detectá saltos de temperatura de más de 3 °C en 10 minutos como posibles errores del sensor.

Cierre: Es el control de calidad que aplican los servicios meteorológicos a los datos de estaciones automáticas.

Ejercicio 137: Logros desbloqueados

Nivel 3: Junior III · Datos del mundo real · Temática: Videojuegos · Dificultad 28/100

Situación: Un juego de aventura tiene logros que se desbloquean al completar un conjunto de acciones, en cualquier orden. El sistema actual revisa todos los logros contra toda la historia del jugador en cada evento y se vuelve lento. Te piden rehacerlo con conjuntos y mostrar además qué logros están "casi".

Datos de entrada

Logro	Requisitos
Explorador	visitar_bosque, visitar_cueva, visitar_volcan
Coleccionista	gema_azul, gema_roja, gema_verde
Pacifista	cero_combates, terminar_nivel_3
Madrugador	jugar_6am
Leyenda	derrotar_jefe, gema_roja, terminar_nivel_3

Eventos del jugador, en orden: visitar_bosque, gema_roja, visitar_cueva, gema_roja, terminar_nivel_3, derrotar_jefe, gema_azul, visitar_volcan

Tu programa debe

1. Guardar los requisitos de cada logro como un conjunto y los eventos realizados en otro conjunto que crece.
2. Después de cada evento, desbloquear los logros cuyos requisitos sean un subconjunto de lo realizado (revisando en orden alfabético) y avisarlo.
3. Al final, para cada logro pendiente (en orden alfabético), mostrar qué le falta y marcar "¡casi!" si falta uno solo.
4. Mostrar cuántos logros se desbloquearon.

Salida esperada

```
Evento 6 (derrotar_jefe): ¡logro desbloqueado: Leyenda!
Evento 8 (visitar_volcan): ¡logro desbloqueado: Explorador!
Coleccionista: falta gema_verde (¡casi!)
Madrugador: falta jugar_6am (¡casi!)
Pacifista: falta cero_combates (¡casi!)
Logros: 2 de 5
```

Reglas

- Alcanza con conjuntos (inclusión y diferencia) y mapas clave-valor.
- Un evento repetido no cambia nada (ya estaba en el conjunto).
- Un logro se desbloquea una sola vez.
- Los requisitos faltantes se listan en orden alfabético.

Pista (leela solo si te trabás): "todos los requisitos cumplidos" es "requisitos $\subseteq$ realizados". Lo que falta es la diferencia requisitos – realizados.

Bonus (opcional)

- Para no revisar todos los logros en cada evento, armá un mapa evento → logros que lo requieren y revisá solo esos.
- Agregá logros con contador: "juntar 5 gemas" (cualquier color, repetidas valen).

Cierre: Es el mismo sistema que usan las plataformas de videojuegos para los logros y trofeos.

Ejercicio 138: Setlist sin repetir

Nivel 3: Junior III · Datos del mundo real · Temática: Música · Dificultad 28/100

Situación: Una banda que gira por el interior no quiere repetir canciones de los dos shows anteriores, porque mucha gente los sigue de ciudad en ciudad. La excepción son los hits, que la gente pide siempre. Te piden armar automáticamente la setlist de 8 temas del próximo show.

Datos de entrada

Catálogo (en orden de preferencia de la banda): Humo, Faro, Cordillera, Ruta 40, Siesta, Marea, Brasa, Café Frío, Tormenta, Otoño, Tren de Noche, Luz de Neón, Madrugada, Vuelta a Casa

Hits (van siempre): Luz de Neón, Marea

Show	Setlist
1	Faro, Marea, Brasa, Humo, Otoño
2	Siesta, Luz de Neón, Faro, Tormenta
3	Ruta 40, Marea, Humo, Cordillera

Tu programa debe

1. Armar el conjunto de canciones tocadas en los dos últimos shows (el 2 y el 3).
2. Agregar primero los hits, en el orden del catálogo.
3. Completar hasta 8 canciones recorriendo el catálogo en orden y saltando las ya elegidas y las tocadas recientemente (informando estas últimas).
4. Mostrar la setlist, las descartadas por repetidas y cuántas canciones distintas se tocaron en los dos últimos shows.

Salida esperada

Setlist (8): Marea, Luz de Neón, Brasa, Café Frío, Otoño, Tren de Noche, Madrugada, Vuelta a Casa
Descartadas por repetidas: Humo, Faro, Cordillera, Ruta 40, Siesta, Tormenta
Canciones de los últimos dos shows: 8

Reglas

- Alcanza con conjuntos y listas que conservan el orden de inserción.
- Los hits no se descartan aunque se hayan tocado recientemente.
- Solo se informan como descartadas las canciones que se revisaron antes de completar la setlist.

Pista (leela solo si te trabás): usá el conjunto para preguntar rápido "¿se tocó hace poco?" y la lista para mantener el orden. Son dos estructuras con dos trabajos distintos.

Bonus (opcional)

- Agregá una restricción de duración total: la setlist no puede pasar de 40 minutos.
- Alterná canciones rápidas y lentas usando una etiqueta de ritmo por canción.

Cierre: Es la misma lógica de "no repetir lo reciente" que usan los reproductores en modo aleatorio y las radios automatizadas.

Ejercicio 139: Tareas de la chacra según el día de la semana

Nivel 3: Junior III · Datos del mundo real · Temática: Agricultura · Dificultad 28/100

Situación: En un tambo de la cuenca lechera, las tareas tienen frecuencias distintas: algunas son semanales, otras según el día del mes y otras cada tantos días. El encargado quiere imprimir la agenda de las próximas dos semanas para pegarla en el galpón.

Datos de entrada

Tarea	Regla
Ordeñe con control lechero	cada lunes
Revisar alambrado	días pares del mes
Rotar potreros	cada 3 días desde el 01/09/2026
Feria de hacienda	primer sábado del mes
Vacunar terneros	el 10 de cada mes

Período: del 01/09/2026 (martes) al 14/09/2026.

Tu programa debe

1. Escribir una función que diga si una regla aplica a una fecha dada.
2. Recorrer las fechas del período día por día.
3. Para cada día, listar las tareas que aplican en el orden de la tabla.
4. Mostrar la agenda con el nombre del día de la semana.

Salida esperada

```

martes    01/09/2026: Rotar potreros
miércoles 02/09/2026: Revisar alambrado
jueves    03/09/2026: —
viernes   04/09/2026: Revisar alambrado, Rotar potreros
sábado    05/09/2026: Feria de hacienda
domingo   06/09/2026: Revisar alambrado
lunes     07/09/2026: Ordeñe con control lechero, Rotar potreros
martes    08/09/2026: Revisar alambrado
miércoles 09/09/2026: —
jueves    10/09/2026: Revisar alambrado, Rotar potreros, Vacunar terneros
viernes   11/09/2026: —
sábado    12/09/2026: Revisar alambrado
domingo   13/09/2026: Rotar potreros
lunes     14/09/2026: Ordeñe con control lechero, Revisar alambrado
  
```

Reglas

- Alcanza con fechas, día de la semana y aritmética de días. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- "Primer sábado del mes" es el sábado cuyo día del mes está entre 1 y 7.
- "Cada 3 días desde el 01/09" incluye el 01/09.
- Si un día no tiene tareas, se muestra "—". El nombre del día va en 9 caracteres a la izquierda.

Pista (leela solo si te trabás): si implementás el día de la semana vos, sabiendo que el 01/09/2026 es martes, el día de la semana de cualquier fecha posterior es (días transcurridos + 1) resto 7, contando lunes = 0.

Bonus (opcional)

- Agregá la regla "último viernes del mes".
- Agregá feriados: si una tarea cae feriado, se corre al día hábil siguiente.

Cierre: Es la misma lógica de reglas de recurrencia que tienen los calendarios cuando elegís "repetir cada...".

Ejercicio 140: IPs con demasiados intentos

Nivel 3: Junior III · Datos del mundo real · Temática: Seguridad informática · Dificultad 28/100

Situación: El servidor de una municipalidad recibe intentos de login de bots que prueban usuarios comunes. El área de sistemas quiere bloquear una IP cuando acumula 3 intentos fallidos en menos de 60 segundos, y dejar en paz a los usuarios que se equivocan una vez y entran.

Datos de entrada

Formato: hora ip usuario resultado

```
10:00:01 190.2.1.10 ana FALLO
10:00:05 200.45.7.3 root FALLO
10:00:09 200.45.7.3 admin FALLO
10:00:30 190.2.1.10 ana OK
10:00:41 200.45.7.3 test FALLO
10:01:02 181.1.9.77 luis FALLO
10:01:15 200.45.7.3 root FALLO
10:01:50 181.1.9.77 luis FALLO
10:02:30 181.1.9.77 luis FALLO
10:02:35 181.1.9.77 luis FALLO
10:02:58 181.1.9.77 luis FALLO
```

Tu programa debe

1. Guardar, por IP, las horas (en segundos) de sus intentos fallidos.
2. En cada fallo, descartar los fallos de esa IP con 60 segundos o más de antigüedad y contar los que quedan.
3. Si quedan 3 o más, bloquear la IP en ese momento; los eventos posteriores de una IP bloqueada se ignoran.
4. Mostrar los bloqueos, las IPs bloqueadas y cuántos fallos quedaban en la ventana de cada IP al final.

Salida esperada

```
10:00:41 BLOQUEO 200.45.7.3 (3 fallos en menos de 60 s)
10:02:35 BLOQUEO 181.1.9.77 (3 fallos en menos de 60 s)
IPs bloqueadas: 200.45.7.3, 181.1.9.77
Fallos por IP (ventana vigente al final): 190.2.1.10=1, 200.45.7.3=3, 181.1.9.77=3
```

Reglas

- Alcanza con mapas clave-valor y ventanas de tiempo simples.
- "Menos de 60 segundos" es estricto: un fallo de hace exactamente 60 s ya no cuenta.
- Un login OK no reinicia los fallos de esa IP (el bot puede acertar una contraseña de otro usuario).
- Las IPs se listan en el orden en que aparecieron por primera vez con un fallo.

Pista (leela solo si te trabás): la ventana se mantiene "limpiando" la lista de fallos de la IP cada vez que llega uno nuevo: te quedás solo con los que están a menos de 60 s del actual.

Bonus (opcional)

- Desbloquea automáticamente una IP después de 5 minutos sin intentos.
- Detecta ataques distribuidos: más de 5 fallos contra el mismo usuario desde IPs distintas en 1 minuto.

Cierre: Es el mismo mecanismo de protección contra fuerza bruta que implementan los firewalls y los servidores de autenticación.

Ejercicio 141: Pagos contra facturas

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 28/100

Situación: Una distribuidora de bebidas cobra por transferencia y los clientes ponen el número de factura en el concepto, a veces mal. A fin de mes, la administración tiene que conciliar: qué facturas están pagas, cuáles parciales, dónde hay pagos de más y qué transferencias no corresponden a ninguna factura.

Datos de entrada

Facturas:

Factura	Cliente	Monto
F-1001	Almacén Tito	\$48.000
F-1002	Kiosco Lu	\$12.500
F-1003	Bar El Faro	\$96.300
F-1004	Almacén Tito	\$31.000
F-1005	Verdulería Sol	\$18.750

Pagos recibidos (factura informada, monto):

Factura	Monto
F-1001	\$48.000
F-1003	\$50.000
F-1003	\$46.300
F-1002	\$13.000
F-1009	\$7.000
F-1004	\$20.000

Tu programa debe

1. Guardar las facturas en un mapa por número.
2. Acumular los pagos por factura; los pagos a facturas inexistentes van a una lista aparte.
3. Clasificar cada factura: pagada, parcial, sin pago o con pago de más.
4. Mostrar la conciliación, los pagos sin factura y el total adeudado.

Salida esperada

```
F-1001 Almacén Tito    $ 48000 | pagado $ 48000 | pagada
F-1002 Kiosco Lu      $ 12500 | pagado $ 13000 | PAGO DE MÁS: $500 a favor del cliente
F-1003 Bar El Faro    $ 96300 | pagado $ 96300 | pagada
F-1004 Almacén Tito   $ 31000 | pagado $ 20000 | PARCIAL, falta $11000
F-1005 Verdulería Sol $ 18750 | pagado $      0 | SIN PAGO
Pagos sin factura: F-1009 ($7000)
Total adeudado: $29750
```

Reglas

- Alcanza con mapas clave-valor, conjuntos y validación.
- Una factura puede pagarse en varios pagos.
- El total adeudado suma solo lo que falta pagar (los pagos de más no se descuentan de otras facturas).
- Formato: cliente en 14 caracteres a la izquierda, montos en 6 a la derecha. Las facturas se muestran en el orden original.

Pista (leela solo si te trabás): la conciliación es un cruce entre dos colecciones por una clave común (el número de factura). El mapa hace que cada búsqueda sea directa.

Bonus (opcional)

- Aplica los pagos de más como crédito a otras facturas pendientes del mismo cliente.
- Sugiere a qué factura podría corresponder un pago huérfano: la del mismo monto pendiente, o la de número más parecido.

Cierre: Es el trabajo diario de conciliación de cobranzas que automatizan los sistemas contables y de tesorería.

Ejercicio 142: Tiempos de entrega por zona

Nivel 3: Junior III · Datos del mundo real · Temática: Logística y envíos · Dificultad 29/100

Situación: Una app de delivery promete entregar en 45 minutos, pero hay zonas donde casi nunca llega a tiempo. El equipo de operaciones quiere un ranking de zonas por tiempo promedio de entrega para decidir dónde sumar repartidores.

Datos de entrada

Envío	Zona	Minutos
1	Centro	32
2	Norte	55
3	Centro	28
4	Sur	41
5	Norte	62
6	Oeste	38
7	Sur	47
8	Centro	35
9	Norte	44
10	Oeste	52
11	Sur	39
12	Centro	49
13	Oeste	36
14	Norte	71

Tu programa debe

1. Agrupar los tiempos por zona en un mapa zona → lista de tiempos.
2. Calcular para cada zona el promedio, el máximo y el porcentaje de envíos a tiempo (45 minutos o menos).
3. Ordenar las zonas por promedio ascendente.
4. Mostrar el ranking y la zona más lenta.

Salida esperada

```
Centro promedio 36.0 min | máximo 49 | a tiempo 75.0% (4 envíos)
Oeste promedio 42.0 min | máximo 52 | a tiempo 66.7% (3 envíos)
Sur promedio 42.3 min | máximo 47 | a tiempo 66.7% (3 envíos)
Norte promedio 58.0 min | máximo 71 | a tiempo 25.0% (4 envíos)
Zona más lenta: Norte
```

Reglas

- Alcanza con mapas clave-valor, promedios y ordenamiento.
- Promedio y porcentaje con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- El orden usa el promedio sin redondear; en empate, alfabéticamente.
- Formato: zona en 7 caracteres a la izquierda y promedio en 5 a la derecha.

Pista (leela solo si te trabás): primero agrupá (un recorrido) y después calculá las métricas de cada grupo (otro recorrido, sobre el mapa).

Bonus (opcional)

- Calculá la mediana de cada zona: con tiempos muy dispersos representa mejor que el promedio.

- Estimá cuántos repartidores extra necesita cada zona para bajar su promedio a 40 minutos, suponiendo que el tiempo baja proporcionalmente.

Cierre: Es el mismo análisis de nivel de servicio por zona que hacen las plataformas de delivery y las empresas de logística.

Ejercicio 143: Buscador de recetas por ingredientes

Nivel 3: Junior III · Datos del mundo real · Temática: Gastronomía · Dificultad 29/100

Situación: Una app de cocina quiere sugerir recetas según lo que la persona tiene en la heladera. Primero las que se pueden hacer ya, después las que requieren comprar pocas cosas, y siempre diciendo qué falta.

Datos de entrada

Receta	Ingredientes
Tortilla de papas	aceite, cebolla, huevo, papa
Guiso de lentejas	cebolla, chorizo, lenteja, papa, zanahoria
Panqueques	harina, huevo, leche
Puré	leche, manteca, papa
Omelette	huevo, queso
Fideos con manteca	fideos, manteca, queso

En la heladera: aceite, cebolla, harina, huevo, leche, papa, queso

Tu programa debe

1. Guardar los ingredientes de cada receta y los de la heladera como conjuntos.
2. Calcular, para cada receta, qué ingredientes faltan y qué porcentaje de sus ingredientes ya tenés (cobertura).
3. Ordenar por cantidad de faltantes ascendente, después por cobertura descendente y después por nombre.
4. Mostrar cada receta con lo que falta.

Salida esperada

```
Omelette: ¡la podés hacer! (100%)
Panqueques: ¡la podés hacer! (100%)
Tortilla de papas: ¡la podés hacer! (100%)
Puré: faltan 1 (manteca) - cobertura 67%
Fideos con manteca: faltan 2 (fideos, manteca) - cobertura 33%
Guiso de lentejas: faltan 3 (chorizo, lenteja, zanahoria) - cobertura 40%
```

Reglas

- Alcanza con parseo, conjuntos (intersección y diferencia) y ordenamiento con criterio compuesto.
- Cobertura sin decimales (0,5 hacia arriba).
- Los faltantes se listan en orden alfabético.

Pista (leela solo si te trabás): lo que falta es receta – heladera; lo que ya tenés es receta n heladera. La cobertura es $|receta \cap heladera| / |receta|$.

Bonus (opcional)

- Considerá ingredientes "básicos" (sal, agua, aceite) como siempre disponibles.
- Sugerí la compra mínima: el ingrediente que, si lo comprás, habilita más recetas.

Cierre: Es la lógica de los buscadores de recetas por ingredientes y, en general, de cualquier sistema que compara lo que tenés contra lo que se requiere.

Ejercicio 144: Todas las formaciones posibles del equipo

Nivel 3: Junior III · Datos del mundo real · Temática: Deportes · Dificultad 29/100

Situación: El técnico de un equipo de fútbol 5 amateur quiere probar todas las formaciones posibles en los amistosos de pretemporada. Tiene 2 arqueros, 3 defensores y 3 delanteros, pero hay dos condiciones: Bruno y Eze no se llevan bien en la cancha, y Fede solo ataja si juega Ciro (que es el que ordena la defensa).

Datos de entrada

Posición	Jugadores	Titulares
Arquero	Tomi, Fede	1
Defensor	Ale, Bruno, Ciro	2
Delantero	Dani, Eze, Facu	2

Restricciones: Bruno y Eze no juegan juntos. Si ataja Fede, Ciro tiene que estar entre los defensores.

Tu programa debe

1. Escribir una función recursiva que genere todas las combinaciones de k elementos de una lista, respetando el orden original.
2. Generar todas las formaciones: cada arquero con cada par de defensores y cada par de delanteros.
3. Filtrar las que cumplen las restricciones.
4. Mostrar la cantidad total, la cantidad de válidas y la lista de válidas numerada.

Salida esperada

```
Formaciones posibles: 18
Formaciones válidas: 9
1. Tomi | Ale, Bruno | Dani, Facu
2. Tomi | Ale, Ciro | Dani, Eze
3. Tomi | Ale, Ciro | Dani, Facu
4. Tomi | Ale, Ciro | Eze, Facu
5. Tomi | Bruno, Ciro | Dani, Facu
6. Fede | Ale, Ciro | Dani, Eze
7. Fede | Ale, Ciro | Dani, Facu
8. Fede | Ale, Ciro | Eze, Facu
9. Fede | Bruno, Ciro | Dani, Facu
```

Reglas

- Alcanza con recursión y generación de combinaciones.
- Las combinaciones se generan así: primero las que incluyen al primer elemento (combinado con las de $k - 1$ del resto) y después las que no lo incluyen (las de k del resto).
- El orden de las formaciones: arquero (en el orden de la tabla), después defensores, después delanteros, cada uno en el orden en que los genera la función.

Pista (leela solo si te trabás): $\text{combinaciones}([a, b, c], 2) = [a] + \text{cada combinación de 1 de } [b, c], \text{ más las combinaciones de 2 de } [b, c]$. El caso base: elegir 0 elementos tiene una única forma (la lista vacía).

Bonus (opcional)

- Contá las formaciones con la fórmula de combinaciones ($n! / (k! (n - k)!)$) y verificá que coincide.
- Agregá un puntaje por jugador y encontrá la formación válida de mayor puntaje.

Cierre: Es la misma generación de combinaciones que usan los sistemas de fantasy fútbol, de armado de equipos y de planificación de turnos.

Ejercicio 145: Fechas en tres formatos distintos

Nivel 3: Junior III · Datos del mundo real · Temática: Educación · Dificultad 29/100

Situación: Un instituto recibe inscripciones por tres canales distintos (web, planilla y mail) y cada uno anota la fecha de nacimiento a su manera. Antes de cargar todo en el sistema, necesitan unificar las fechas en un solo formato y detectar las que son imposibles.

Datos de entrada

Fechas recibidas:

- 2026-03-05
- 05/03/2026
- 5 de marzo de 2026
- 31/04/2026
- 2026-13-01
- 29 de febrero de 2027
- 1 de enero de 2027
- 15-08-2026

Formatos aceptados: AAAA-MM-DD, DD/MM/AAAA y D de mes de AAAA (mes en español, en minúsculas).

Tu programa debe

1. Detectar el formato de cada fecha por su estructura (separadores y posiciones).
2. Extraer día, mes y año y validar que la fecha exista en el calendario.
3. Mostrar cada fecha válida en formato ISO (AAAA-MM-DD), en formato DD/MM/AAAA y con su día de la semana.
4. Mostrar el error de las inválidas: "formato no reconocido", "mes desconocido" o "la fecha no existe".

Salida esperada

```
"2026-03-05" -> 2026-03-05 | 05/03/2026 | jueves
"05/03/2026" -> 2026-03-05 | 05/03/2026 | jueves
"5 de marzo de 2026" -> 2026-03-05 | 05/03/2026 | jueves
"31/04/2026" -> ERROR: la fecha no existe
"2026-13-01" -> ERROR: la fecha no existe
"29 de febrero de 2027" -> ERROR: la fecha no existe
"1 de enero de 2027" -> 2027-01-01 | 01/01/2027 | viernes
"15-08-2026" -> ERROR: formato no reconocido
```

Reglas

- Alcanza con parseo de fechas y validación. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- El formato con guiones solo se acepta como AAAA-MM-DD (15-08-2026 no es un formato aceptado).
- El 29/02/2027 no existe (2027 no es bisiesto).
- Los días de la semana se muestran en minúscula.

Pista (leela solo si te trabás): decidí el formato mirando el largo y la posición de los separadores antes de separar. Así evitás confundir 2026-03-05 con 05-03-2026.

Bonus (opcional)

- Aceptá también meses abreviados ("5 mar 2026") y meses con mayúscula.
- Detectá fechas ambiguas como 03/05/2026 cuando el canal podría estar usando el formato estadounidense (mes/día).

Cierre: Es la normalización de fechas que hace cualquier proceso de importación de datos entre sistemas.

Ejercicio 146: Aguinaldo proporcional

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 29/100

Situación: En junio hay que pagar la primera cuota del aguinaldo (sueldo anual complementario) y en una pyme hay tres casos distintos: alguien que trabajó todo el semestre, alguien que entró en marzo y alguien que se fue en mayo. El estudio contable quiere el cálculo proporcional exacto por días.

Datos de entrada

Semestre: del 01/01/2026 al 30/06/2026.

Empleado	Ene	Feb	Mar	Abr	May	Jun	Ingreso	Egreso
Carolina	980.000	980.000	1.050.000	1.050.000	1.120.000	1.120.000	—	—
Matías	0	0	760.000	760.000	815.000	815.000	16/03/2026	—
Raquel	1.400.000	1.400.000	1.450.000	1.450.000	700.000	0	—	20/05/2026

Fórmula: $\text{aguinaldo} = (\text{mejor sueldo mensual del semestre} / 2) \times \text{días trabajados en el semestre} / \text{días del semestre}$.

Tu programa debe

1. Calcular la cantidad de días del semestre (inclusive).
2. Calcular los días trabajados de cada empleado dentro del semestre, según su fecha de ingreso o egreso (ambas inclusive).
3. Encontrar el mejor sueldo mensual del semestre.
4. Calcular el aguinaldo proporcional.

Salida esperada

Días del semestre: 181
 Carolina: mejor sueldo \$1120000 | 181 días trabajados | aguinaldo \$560000.00
 Matías: mejor sueldo \$815000 | 107 días trabajados | aguinaldo \$240897.79
 Raquel: mejor sueldo \$1450000 | 140 días trabajados | aguinaldo \$560773.48

Reglas

- Alcanza con fechas y cálculo proporcional. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- El aguinaldo se redondea a 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Los días se cuentan inclusive: del 01/01 al 30/06 hay 181 días en 2026.
- El sueldo parcial del mes de egreso cuenta como sueldo del mes (acá no es el mejor, así que no cambia nada).

Pista (leela solo si te trabás): días entre dos fechas inclusive = diferencia en días + 1. Si usás la función de "días desde una fecha fija" de ejercicios anteriores, sale directo.

Bonus (opcional)

- Calculá también la segunda cuota (julio a diciembre) para Matías, suponiendo sueldos de \$850.000 fijos.
- Descontá los aportes del empleado (17 %) y mostrá el neto a cobrar.

Cierre: Es el cálculo que hace cada sistema de liquidación de sueldos en junio y diciembre.

Ejercicio 147: Calendario de vacunación

Nivel 3: Junior III · Datos del mundo real · Temática: Salud · Dificultad 29/100

Situación: Un centro de salud quiere mandar recordatorios a las familias antes de cada vacuna de sus bebés. El calendario se define en meses de vida, y a los administrativos les cuesta calcular fechas cuando el bebé nació un día 31. (Esquema ilustrativo, no reemplaza el calendario oficial.)

Datos de entrada

Bebé nacido el 31/01/2026. Hoy es 20/07/2026.

Dosis	Edad (meses)
Dosis de nacimiento	0
Primera dosis combinada	2
Segunda dosis combinada	4
Tercera dosis combinada	6
Refuerzo del año	12
Refuerzo de los 15 meses	15

Si el día de nacimiento no existe en el mes de la dosis, la fecha es el último día de ese mes.

Tu programa debe

1. Calcular la fecha de cada dosis sumando meses a la fecha de nacimiento (con el ajuste de fin de mes).
2. Calcular el día de la semana de cada fecha.
3. Clasificar cada dosis respecto de hoy: ya pasó, hoy, próxima (en 30 días o menos) o pendiente.
4. Mostrar el calendario completo.

Salida esperada

Dosis de nacimiento	31/01/2026 (sábado) -> fecha pasada (verificar que se aplicó)
Primera dosis combinada	31/03/2026 (martes) -> fecha pasada (verificar que se aplicó)
Segunda dosis combinada	31/05/2026 (domingo) -> fecha pasada (verificar que se aplicó)
Tercera dosis combinada	31/07/2026 (viernes) -> PRÓXIMA: faltan 11 días
Refuerzo del año	31/01/2027 (domingo) -> pendiente: faltan 195 días
Refuerzo de los 15 meses	30/04/2027 (viernes) -> pendiente: faltan 284 días

Reglas

- Alcanza con fechas y suma de meses y días. Podés usar las herramientas de fechas que tengas disponibles o implementar el cálculo vos.
- Cada fecha se calcula desde la fecha de nacimiento (no desde la dosis anterior).
- Formato: nombre de la dosis en 25 caracteres a la izquierda y fechas DD/MM/AAAA.

Pista (leela solo si te trabás): reutilizá la función de sumar meses del ejercicio de suscripciones: $31/01 + 1 \text{ mes} = 28/02$ (o $29/02$ si es bisiesto), pero $31/01 + 2 \text{ meses} = 31/03$.

Bonus (opcional)

- Si una fecha cae domingo, correla al lunes y avisá del cambio.
- Generá el texto del recordatorio de cada dosis para enviarlo 7 días antes.

Cierre: Es la misma lógica de agenda que usan los sistemas de salud para los calendarios de vacunación y controles.

Ejercicio 148: Desafío de nivel: El extracto de dos bancos en uno

Nivel 3: Junior III · Datos del mundo real · Temática: Finanzas y FinTech · Dificultad 30/100

Situación: Valeria tiene cuentas en dos bancos y cada uno exporta los movimientos con un formato distinto. Quiere una sola vista cronológica, con cada movimiento categorizado, y un resumen mensual. El problema: las transferencias entre sus propias cuentas aparecen dos veces (salida en un banco y entrada en el otro) e inflan los totales. Es el desafío que integra el nivel 3.

Datos de entrada

Banco A (AAAA-MM-DD, descripción, monto con punto decimal):

```
2026-04-02,Supermercado Día,-24500.00
2026-04-03,Sueldo,950000.00
2026-04-05,Transferencia a cuenta propia,-200000.00
2026-04-11,Nafta YPF,-38200.50
2026-05-02,Supermercado Coto,-31990.00
```

Banco B (DD/MM/AAAA; monto; descripción con punto de miles y coma decimal):

```
05/04/2026;+200.000,00;Transferencia desde cuenta propia
08/04/2026;-15.300,00;Farmacity
20/04/2026;-9.999,99;Netflix
02/05/2026;-4.500,00;Cine
30/04/2026;-61.000,00;Nafta Shell
```

Reglas de categoría (se busca el texto en la descripción en minúsculas; gana la primera regla que coincide): "supermercado" → Supermercado, "sueldo" → Ingresos, "nafta" → Transporte, "farmac" → Salud, "netflix" → Entretenimiento, "cine" → Entretenimiento. Si ninguna coincide: Otros.

Movimiento interno: misma fecha, montos opuestos, bancos distintos y ambas descripciones contienen "propia".

Tu programa debe

1. Parsear los dos formatos y llevarlos a una misma estructura (fecha, monto, descripción, banco).
2. Ordenar todos los movimientos por fecha y, a igual fecha, por monto ascendente.
3. Detectar los pares de movimientos internos y marcarlos con la categoría "Interno"; categorizar el resto con las reglas.
4. Mostrar el listado unificado y el resumen por mes y categoría, excluyendo los internos.

Salida esperada

```
Movimientos unificados:
02/04/2026 A -24500.00 Supermercado Supermercado Día
03/04/2026 A 950000.00 Ingresos Sueldo
05/04/2026 A -200000.00 Interno Transferencia a cuenta propia
05/04/2026 B 200000.00 Interno Transferencia desde cuenta propia
08/04/2026 B -15300.00 Salud Farmacity
11/04/2026 A -38200.50 Transporte Nafta YPF
20/04/2026 B -9999.99 Entretenimiento Netflix
30/04/2026 B -61000.00 Transporte Nafta Shell
02/05/2026 A -31990.00 Supermercado Supermercado Coto
02/05/2026 B -4500.00 Entretenimiento Cine
Resumen por mes (sin movimientos internos):
04/2026: Entretenimiento -9999.99 | Ingresos 950000.00 | Salud -15300.00 | Supermercado
-24500.00 | Transporte -99200.50
05/2026: Entretenimiento -4500.00 | Supermercado -31990.00
```

Reglas

- Alcanza con parseo, fechas, mapas y ordenamiento: es el desafío integrador del nivel 3.

- Montos con 2 decimales. Formato del listado: fecha, banco, monto en 10 caracteres a la derecha, categoría en 15 a la izquierda y descripción. Los decimales se muestran con punto y sin separador de miles en la salida.
- En el resumen, los meses van en orden cronológico y las categorías en orden alfabético, separadas por " | ".
- Usá números decimales exactos: estos montos no pueden tener errores de centavos.

Pista (leela solo si te trabás): el truco es normalizar lo antes posible: apenas lees una línea de cualquier banco, convertila a la estructura común. De ahí en adelante, el programa no sabe (ni le importa) de qué banco vino.

Bonus (opcional)

- Detectá movimientos internos aunque la acreditación llegue un día hábil después.
- Agregá un tercer banco con formato de ancho fijo (cada campo ocupa una cantidad fija de caracteres).

Cierre: Es exactamente lo que hacen los agregadores financieros que conectan varias cuentas bancarias en una sola app.

Ejercicio 149: Desafío de nivel: Padrón de socios del club

Nivel 3: Junior III · Datos del mundo real · Temática: Deportes · Dificultad 30/100

Situación: El club social y deportivo del barrio tiene que presentar su padrón de socios en la asamblea anual. La base se cargó durante años sin controles: hay DNI mal escritos, socios duplicados y categorías desactualizadas (la categoría depende de la edad al 31 de diciembre). Es el desafío de validación del nivel 3.

Datos de entrada

DNI	Socio	Nacimiento	Categoría cargada
30111222	Sosa, Martina	2014-05-10	Infantil
28.555.999	Ibarra, Carlos	1959-11-02	Mayor
4512345	Álvarez, Juana	2010-12-31	Cadete
30111222	Sosa, Martina	2014-05-10	Infantil
40123	Nieto, Pablo	2001-03-15	Mayor
33444555	Benítez, Sol	2008-06-30	Cadete
27888111	Zapata, Hugo	1961-01-01	Vitalicio
44222333	Acuña, Leo	2013-12-31	Infantil

Categoría según la edad cumplida al 31/12/2026: menos de 13 → Infantil; de 13 a 17 → Cadete; de 18 a 64 → Mayor; 65 o más → Vitalicio.

Tu programa debe

1. Validar el DNI: se quitan los puntos y tienen que quedar 7 u 8 dígitos.
2. Descartar duplicados por DNI, conservando el primero.
3. Calcular la edad al 31/12/2026 y la categoría que corresponde; marcar las categorías mal cargadas.
4. Mostrar el padrón ordenado por categoría (Infantil, Cadete, Mayor, Vitalicio) y, dentro de cada una, por apellido; después, los rechazados en orden de aparición.

Salida esperada

Padrón válido:

```

Infantil  Sosa, Martina    DNI 30111222 | 12 años
Cadete    Acuña, Leo      DNI 44222333 | 13 años | CORREGIR: figuraba como Infantil
Cadete    Álvarez, Juana  DNI 4512345  | 16 años
Mayor     Benítez, Sol     DNI 33444555 | 18 años | CORREGIR: figuraba como Cadete
Vitalicio Ibarra, Carlos  DNI 28555999 | 67 años | CORREGIR: figuraba como Mayor
Vitalicio Zapata, Hugo   DNI 27888111 | 65 años
  
```

Rechazados:

```

Sosa, Martina: duplicado (DNI 30111222)
Nieto, Pablo: DNI inválido (40123)
  
```

Reglas

- Alcanza con validación, conjuntos, fechas y ordenamiento: es el desafío integrador de validación del nivel 3.
- La edad se calcula con cumpleaños exacto: alguien nacido el 31/12/2013 cumple 13 justo el 31/12/2026, así que ya es Cadete.
- El orden alfabético ignora tildes. Formato: categoría en 9 caracteres, nombre en 16, DNI en 8 a la derecha.

Pista (leela solo si te trabás): la edad es la diferencia de años, menos 1 si todavía no llegó el cumpleaños en la fecha de corte. Comparar (mes, día) resuelve "todavía no llegó".

Bonus (opcional)

- Calculá la recaudación mensual con una cuota distinta por categoría.
- Detectá socios que cambian de categoría durante 2027 y cuándo.

Cierre: Es la misma validación y normalización que hacen los sistemas de gestión de clubes, obras sociales y cualquier padrón de afiliados.

Ejercicio 150: Desafío de nivel: Farmacia con vencimientos

Nivel 3: Junior III · Datos del mundo real · Temática: Salud · Dificultad 30/100

Situación: Una farmacia de barrio perdió plata tirando medicamentos vencidos porque despachaba siempre del lote que tenía más a mano. Ahora quieren aplicar la regla "primero vence, primero sale", retirar lo vencido y recibir alertas de lo que vence pronto. Es el desafío que cierra el nivel 3.

Datos de entrada

Hoy: 28/09/2026. Un vencimiento MM/AAAA significa "vence el último día de ese mes".

Producto	Lote	Vencimiento	Unidades
Ibuprofeno 400	L-A12	10/2026	40
Ibuprofeno 400	L-A07	08/2026	15
Ibuprofeno 400	L-B03	03/2027	100
Amoxicilina 500	L-C21	12/2026	30
Amoxicilina 500	L-C30	06/2027	60
Enalapril 10	L-D02	13/2026	50
Enalapril 10	L-D05	01/2028	80

Pedidos a despachar, en orden: Ibuprofeno 400 × 30, Amoxicilina 500 × 20, Enalapril 10 × 90

Tu programa debe

1. Validar los vencimientos (mes entre 1 y 12) y excluir los inválidos; retirar los lotes ya vencidos.
2. Despachar cada pedido tomando unidades de los lotes del producto en orden de vencimiento (el más próximo primero). Si no alcanza el stock total, no despachar nada.
3. Mostrar el stock restante por producto.
4. Alertar los lotes con stock que vencen dentro de los próximos 90 días.

Salida esperada

```
Lote L-A07 (Ibuprofeno 400): VENCIDO el 31/08/2026, retirar 15 unidades
Lote L-D02 (Enalapril 10): vencimiento inválido (13/2026), se excluye
Despachos (primero vence, primero sale):
  Ibuprofeno 400 x30: L-A12 x30
  Amoxicilina 500 x20: L-C21 x20
  Enalapril 10 x90: stock insuficiente (hay 80)
Stock restante por producto:
  Amoxicilina 500   70
  Enalapril 10      80
  Ibuprofeno 400   110
Alerta, vencen en 90 días o menos: L-A12 (Ibuprofeno 400) vence 31/10/2026
```

Reglas

- Alcanza con fechas, mapas, validación y ordenamiento: es el desafío integrador del nivel 3.
- Los avisos de lotes inválidos o vencidos se muestran primero, en el orden de la tabla.
- Formato de despacho: lote y unidades tomadas ("L-A12 x40"), separados por coma. Stock: producto en 16 caracteres a la izquierda.
- Las alertas se ordenan por fecha de vencimiento y se separan con "; ".

Pista (leela solo si te trabás): para cada pedido, filtrá los lotes del producto, ordenalos por vencimiento y consumí de a uno hasta completar. Antes, verificá que la suma alcance.

Bonus (opcional)

- Proponé un descuento del 30 % para los lotes que vencen en menos de 60 días, para venderlos antes.
- Registrá cada movimiento de stock (ingreso, despacho, retiro) y reconstruí el stock de cualquier día pasado.

Cierre: Es la política FEFO (first expired, first out) que aplican los sistemas de gestión de farmacias, supermercados y depósitos de alimentos.

Lote 07: ejercicios 151 a 175

Nivel 4

Semi-Senior I · Eficiencia

Ejercicios 151 a 200

Qué vas a practicar

Complejidad, búsqueda binaria, ordenamiento, dos punteros, ventanas y sumas prefijas.

Qué se espera al terminar

Pasar de soluciones cuadráticas a lineales o logarítmicas.

Conceptos del nivel

pilas, búsqueda binaria, complejidad, colas, simulación con estado, dos punteros, ordenamiento, ventana deslizante, sumas prefijas, Big-O, pila monótona, una sola pasada, mínimo acumulado, búsqueda binaria sobre la respuesta, fusión de intervalos, conjuntos, ventana deslizante de tamaño fijo, dos pilas, simulación, sumas prefijas con mapa, cola circular, notación posfija, ventana deslizante con cola doble, búsqueda binaria de límites inferior y superior, una pasada, estado, mapas, búsqueda binaria sobre números reales, sumas prefijas en dos dimensiones, ventana deslizante con cola, cola con dos pilas, costo amortizado, ordenamiento de eventos, barrido, ventana deslizante sobre tiempo, cola doble monótona, simulación con reglas, complejidad cuadrática, consultas por rango, subsecuencia....

Desafíos de nivel

- 198. La caja de ahorro con débitos automáticos
- 199. Monitor de tráfico de la autopista
- 200. La cocina del restaurante en hora pico

Ejercicio 151: La calculadora que se queja de los paréntesis

Nivel 4: Semi-Senior I · Eficiencia · Temática: Educación · Dificultad 31/100

Situación: Una plataforma educativa de matemática deja que los alumnos escriban expresiones con paréntesis, corchetes y llaves. Cuando la expresión está mal armada, el mensaje actual es "error de sintaxis" y los chicos no entienden qué pasó. Quieren un verificador que diga exactamente dónde está el problema.

Datos de entrada

Expresiones:

1. $(3 + 4) * [2 - (1 / 5)]$
2. $\{[(a + b) * c] - d\}$
3. $(2 + 3]$
4. $((5 * 2) + 1$
5. $3 + 4) * 2$
6. $\{[(())()]\}()$
7. (expresión vacía)

Tu programa debe

1. Recorrer la expresión una sola vez usando una pila para los símbolos de apertura, guardando su posición.
2. Ante un cierre, verificar que haya algo abierto y que corresponda al último símbolo abierto.
3. Al terminar, verificar que no quede nada sin cerrar.
4. Informar OK o el primer error encontrado, con su posición (contando desde 1).

Salida esperada

```
"(3 + 4) * [2 - (1 / 5)]" -> OK
"{[(a + b) * c] - d}" -> OK
"(2 + 3]" -> ERROR en la posición 7: ']' no corresponde con '(' abierto en la posición 1
"((5 * 2) + 1" -> ERROR: '(' abierto en la posición 1 nunca se cerró
"3 + 4) * 2" -> ERROR en la posición 6: ')' cierra algo que nunca se abrió
"{[(())()]\}()" -> OK
"" -> OK
```

Reglas

- Límites: el largo de la expresión puede llegar a 1.000.000 de caracteres.
- Complejidad esperada: $O(n)$ en tiempo; $O(n)$ de memoria en el peor caso.
- Si quedan varios símbolos sin cerrar, se informa el más reciente (el del tope de la pila).
- Los caracteres que no son paréntesis, corchetes ni llaves se ignoran. La expresión vacía es válida.
- Casos borde: expresión vacía, solo cierres, solo aperturas, anidamiento de 500.000 niveles.

Pista (leela solo si te trabás): el último que se abrió es el primero que se tiene que cerrar: eso es exactamente una pila.

Bonus (opcional)

- Informá todos los errores en lugar del primero, intentando "recuperarse" después de cada uno.
- Sugerí una corrección automática con la menor cantidad de símbolos agregados.

Cierre: Es el mismo chequeo que hacen los editores de código y los compiladores cuando te marcan un paréntesis sin cerrar.

Ejercicio 152: Un DNI en un padrón de 10 millones

Nivel 4: Semi-Senior I · Eficiencia · Temática: Recursos humanos · Dificultad 31/100

Situación: El padrón electoral de una provincia tiene 10 millones de DNI ordenados de menor a mayor. La app de consulta "¿dónde voto?" recorre el padrón de principio a fin y tarda segundos por consulta en el día de la elección. Te piden reescribir la búsqueda para que responda al instante.

Datos de entrada

Padrón de ejemplo (ordenado, 15 registros): 4512873, 8031245, 12004589, 17882310, 20113456, 22567001, 25998012, 28400332, 30111222, 31778450, 33004125, 35642009, 38110754, 41233087, 44901562

DNI a buscar: 30111222, 4512873, 44901562, 29000000, 50000000

Tu programa debe

1. Implementar la búsqueda binaria: comparar con el elemento del medio y descartar la mitad donde el DNI no puede estar.
2. Contar las comparaciones (iteraciones) de cada búsqueda.
3. Informar la posición (desde 1) o que no está.
4. Calcular el peor caso de comparaciones para 10.000.000 de registros y compararlo con la búsqueda lineal.

Salida esperada

```
DNI 30111222: posición 9 (4 comparaciones)
DNI 4512873: posición 1 (4 comparaciones)
DNI 44901562: posición 15 (4 comparaciones)
DNI 29000000: no está en el padrón (4 comparaciones)
DNI 50000000: no está en el padrón (4 comparaciones)
Peor caso con 10000000 registros: búsqueda binaria 24 comparaciones | búsqueda lineal 10000000
```

Reglas

- Límites: $1 \leq n \leq 10.000.000$ registros y hasta 1.000.000 de consultas.
- Complejidad esperada: $O(\log n)$ por consulta. Una solución $O(n)$ por consulta no pasa.
- Implementación exacta (para que las comparaciones coincidan): inicio = 0, fin = $n - 1$; mientras inicio $\leq$ fin, medio = (inicio + fin) / 2 con división entera.
- El peor caso son las veces que se puede dividir n por 2 (división entera) hasta llegar a 0.
- Casos borde: el primero y el último del padrón, un DNI menor que todos y uno mayor que todos.

Pista (leela solo si te trabás): cada comparación descarta la mitad de lo que queda. Por eso con 10 millones alcanzan unas 24 comparaciones: 2^{24} ya supera los 16 millones.

Bonus (opcional)

- Si el padrón tuviera DNI repetidos, encontrá la primera aparición.
- Si el DNI no está, devolvé la posición donde debería insertarse para mantener el orden.

Cierre: La búsqueda binaria es la base de los índices de las bases de datos: por eso encontrar un registro entre millones tarda milisegundos.

Ejercicio 153: La cola de la sucursal bancaria

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 31/100

Situación: Una sucursal bancaria tiene dos cajas y una sola fila. La gerencia quiere saber cuánto esperan los clientes en hora pico para decidir si abre una tercera caja. Tu programa simula la fila con los horarios de llegada y la duración de cada atención.

Datos de entrada

Cajas: 2. Tiempos en minutos desde la apertura.

Cliente	Llega	Duración de la atención
C1	0	7
C2	1	4
C3	2	6
C4	3	3
C5	5	2
C6	6	5
C7	12	4
C8	13	1

Tu programa debe

1. Mantener, para cada caja, el minuto en que queda libre.
2. Atender a los clientes en orden de llegada: cada uno va a la caja que le permite empezar antes (en empate, la de menor número).
3. Calcular el inicio, el fin y la espera de cada cliente.
4. Mostrar la espera promedio, la máxima y el máximo de personas esperando en la fila (medido en el instante en que llega cada cliente).

Salida esperada

```
C1: llega 0 | caja 1 | atiende 0 a 7 | espera 0
C2: llega 1 | caja 2 | atiende 1 a 5 | espera 0
C3: llega 2 | caja 2 | atiende 5 a 11 | espera 3
C4: llega 3 | caja 1 | atiende 7 a 10 | espera 4
C5: llega 5 | caja 1 | atiende 10 a 12 | espera 5
C6: llega 6 | caja 2 | atiende 11 a 16 | espera 5
C7: llega 12 | caja 1 | atiende 12 a 16 | espera 0
C8: llega 13 | caja 1 | atiende 16 a 17 | espera 3
Espera promedio: 2.50 min | Espera máxima: 5 min
Máximo de personas esperando: 3
```

Reglas

- Límites: hasta 100.000 clientes y hasta 50 cajas.
- Complejidad esperada: $O(n \cdot c)$ con c cajas es suficiente acá; en el nivel 6 vas a hacerlo en $O(n \log c)$ con un heap.
- Un cliente que llega justo cuando una caja se libera no espera.
- Personas esperando cuando llega un cliente = clientes que ya llegaron (incluido él) y todavía no empezaron a ser atendidos.
- Espera promedio con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): no hace falta simular minuto a minuto: alcanza con saber cuándo se libera cada caja. El inicio de un cliente es el máximo entre su llegada y el momento en que se libera la caja elegida.

Bonus (opcional)

- Simulá con 3 cajas y decí cuánto baja la espera promedio.
- Agregá clientes prioritarios (jubilados, embarazadas) que pasan adelante de la fila.

Cierre: Es la misma simulación de colas que se usa para dimensionar cajas de supermercado, centros de atención telefónica y servidores.

Ejercicio 154: Dos productos que suman el cupón

Nivel 4: Semi-Senior I · Eficiencia · Temática: E-commerce · Dificultad 32/100

Situación: Una tienda online regala un cupón de \$20.000 para usar en exactamente dos productos distintos, y el equipo de marketing quiere mostrar "combos perfectos": pares cuyo precio suma justo el cupón. El catálogo ya viene ordenado por precio, así que se puede hacer mucho mejor que probar todos los pares.

Datos de entrada

Precios del catálogo (ordenados, todos distintos): \$2.500, \$3.100, \$4.800, \$6.200, \$7.400, \$8.900, \$11.100, \$12.600, \$13.800, \$15.200, \$16.900, \$17.500

Cupón: \$20.000

Tu programa debe

1. Recorrer la lista con dos índices: uno desde el más barato y otro desde el más caro.
2. Si la suma es menor que el cupón, avanzar el de la izquierda; si es mayor, retroceder el de la derecha; si es igual, registrar el par y mover ambos.
3. Registrar, durante el mismo recorrido, el par con mayor suma sin pasarse.
4. Mostrar los pares exactos y el mejor par sin pasarse.

Salida esperada

```
Pares que suman exactamente $20000: 6
$2500 + $17500
$3100 + $16900
$4800 + $15200
$6200 + $13800
$7400 + $12600
$8900 + $11100
Mejor par sin pasarse: $2500 + $17500 = $20000
```

Reglas

- Límites: $2 \leq n \leq 200.000$ productos; precios hasta 10.000.000.
- Complejidad esperada: $O(n)$ si la lista ya está ordenada ($O(n \log n)$ si hubiera que ordenarla). Probar todos los pares, $O(n^2)$, no pasa con 200.000 productos.
- Los precios son todos distintos y un producto no se combina consigo mismo.
- Los pares se muestran en el orden en que los encuentra el recorrido.
- Casos borde: ningún par entra en el cupón; todos los pares se pasan.

Pista (leela solo si te trabás): si la suma del más barato y el más caro se pasa, el más caro no puede formar par exacto con ninguno más caro que el más barato: se descarta. Cada paso descarta un producto.

Bonus (opcional)

- Si los precios se pueden repetir, contá cuántos pares ($i < j$) suman exacto.
- Resolvé la versión sin ordenar en $O(n)$ usando un conjunto de precios vistos.

Cierre: Es la técnica de dos punteros, que usan desde los motores de promociones hasta los algoritmos de conciliación de montos.

Ejercicio 155: Los 7 días de más ventas en un millón de registros

Nivel 4: Semi-Senior I · Eficiencia · Temática: E-commerce · Dificultad 32/100

Situación: Una cadena de electrodomésticos quiere repetir su mejor semana de ventas del año y necesita saber cuál fue: los 7 días consecutivos con mayor venta total. El problema es que el histórico de todas las sucursales tiene un millón de registros diarios y la solución obvia tarda demasiado.

Datos de entrada

Ejemplo reducido: ventas diarias (en miles de pesos) de 20 días:

Días	1	2	3	4	5	6	7	8	9	10
1-10	182	205	197	240	310	422	390	188	176	201

Días	11	12	13	14	15	16	17	18	19	20
11-20	233	298	405	451	367	170	158	214	260	305

Largo de la ventana: 7 días.

Tu programa debe

1. Calcular la suma de los primeros 7 días.
2. Deslizar la ventana un día por vez: sumar el día que entra y restar el que sale.
3. Quedarse con la ventana de mayor suma (en empate, la primera).
4. Mostrar los días, el total, el promedio diario y cuántas ventanas se evaluaron.

Salida esperada

```
Mejor semana: días 9 a 15
Ventas: 2131 miles de pesos
Promedio diario: 304.43
Ventanas evaluadas: 14
```

Reglas

- Límites: $7 \leq n \leq 1.000.000$ días; ventas diarias de hasta 1.000.000.000 cada una.
- Complejidad esperada: $O(n)$. La fuerza bruta $O(n \cdot k)$ recalcula cada ventana desde cero.
- Las sumas pueden superar lo que entra en un entero de 32 bits: usá enteros de 64 bits.
- Promedio con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Casos borde: exactamente 7 días (una sola ventana); todos los días iguales.

Pista (leela solo si te trabás): dos ventanas consecutivas comparten 6 de sus 7 días. No tiene sentido volver a sumarlos: la nueva suma es la anterior + el día que entra – el que sale.

Bonus (opcional)

- Resuélvelo con sumas prefijas: la suma de los días $a..b$ es $P[b] - P[a - 1]$. Compará ambos enfoques.
- Encontrá la mejor ventana de cualquier largo entre 5 y 10 días, con el mayor promedio diario.

Cierre: Es la técnica de ventana deslizante, la misma que usan los sistemas de monitoreo para calcular métricas "de los últimos N minutos".

Ejercicio 156: Ctrl+Z del editor de publicaciones

Nivel 4: Semi-Senior I · Eficiencia · Temática: Redes sociales · Dificultad 32/100

Situación: El editor de publicaciones de una red social para comercios tiene un botón de deshacer que a veces borra más de la cuenta y un rehacer que no funciona. Te piden reimplementar ambos con la semántica estándar de los editores de texto.

Datos de entrada

Paso	Operación	Argumento
1	escribir	"Hoy "
2	escribir	"inauguramos "
3	escribir	"el local"
4	deshacer	
5	escribir	"la tienda"
6	borrar	6
7	deshacer	
8	deshacer	
9	rehacer	
10	escribir	"!"
11	rehacer	

- escribir: agrega texto al final.
- borrar k: borra los últimos k caracteres (o todos si hay menos).
- deshacer: vuelve al estado anterior a la última operación de edición.
- rehacer: vuelve a aplicar lo último deshecho. Cualquier edición nueva (escribir o borrar) vacía el historial de rehacer.

Tu programa debe

1. Mantener el texto actual y dos pilas: una de estados para deshacer y otra para rehacer.
2. Antes de cada edición, guardar el estado actual en la pila de deshacer y vaciar la de rehacer.
3. Implementar deshacer y rehacer moviendo estados entre las pilas; si la pila correspondiente está vacía, informar que no hay nada que hacer.
4. Mostrar el texto después de cada operación.

Salida esperada

```
escribir 'Hoy ': "Hoy "
escribir 'inauguramos ': "Hoy inauguramos "
escribir 'el local': "Hoy inauguramos el local"
deshacer: "Hoy inauguramos "
escribir 'la tienda': "Hoy inauguramos la tienda"
borrar 6: "Hoy inauguramos la "
deshacer: "Hoy inauguramos la tienda"
deshacer: "Hoy inauguramos "
rehacer: "Hoy inauguramos la tienda"
escribir '!': "Hoy inauguramos la tienda!"
(nada para rehacer)
rehacer: "Hoy inauguramos la tienda!"
```

Reglas

- Límites: hasta 100.000 operaciones; el texto puede llegar a 1.000.000 de caracteres.
- Complejidad esperada: $O(1)$ por deshacer y rehacer, sin contar la copia del texto. (En el bonus vas a evitar copiar el texto entero.)
- El aviso "(nada para ...)" se muestra en una línea propia, antes de la línea de esa operación.

- Formato: la operación, su argumento (el texto entre comillas simples) y el texto actual entre comillas dobles.

Pista (leela solo si te trabás): deshacer no "calcula" nada: solo vuelve al estado guardado. El error típico es olvidarse de vaciar la pila de rehacer cuando se hace una edición nueva.

Bonus (opcional)

- En lugar de guardar el texto completo, guardá la operación inversa (qué agregar o qué borrar) para no copiar un millón de caracteres en cada paso.
- Agrupá escrituras consecutivas en un solo paso de deshacer, como hacen los editores reales.

Cierre: Es el mismo mecanismo de dos pilas que implementan todos los editores de texto, de imágenes y de planillas.

Ejercicio 157: Saldo acumulado por rango de días

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 32/100

Situación: La app de un banco digital deja consultar "¿cuánto gasté o ingresé entre tal día y tal día?". Con años de historial y miles de consultas por segundo, sumar el rango en cada consulta es demasiado lento. Te piden precalcular algo que permita responder cada consulta en tiempo constante.

Datos de entrada

Movimiento neto de cada día (días 1 a 14): 12000, -3500, -8200, 45000, -1200, -9900, -300, 18000, -15600, -2400, -7000, 30000, -500, -11800

Consultas (desde el día a hasta el día b, inclusive): (1, 7), (3, 3), (8, 14), (1, 14), (5, 10)

Tu programa debe

1. Construir el arreglo de sumas prefijas: $P[0] = 0$ y $P[i] = P[i - 1] + \text{movimiento del día } i$.
2. Responder cada consulta (a, b) como $P[b] - P[a - 1]$.
3. Mostrar las sumas prefijas (de $P[1]$ a $P[n]$) y el resultado de cada consulta con signo.

Salida esperada

```
Sumas prefijas: 12000 8500 300 45300 44100 34200 33900 51900 36300 33900 26900 56900 56400 44600
Días 1 a 7: +33900
Días 3 a 3: -8200
Días 8 a 14: +10700
Días 1 a 14: +44600
Días 5 a 10: -11400
```

Reglas

- Límites: $1 \leq n \leq 1.000.000$ días; hasta 1.000.000 de consultas; movimientos entre -10^9 y 10^9 .
- Complejidad esperada: $O(n)$ de preprocesamiento y $O(1)$ por consulta. Sumar el rango en cada consulta, $O(n \cdot q)$, no pasa.
- Las sumas pueden superar 2^{31} : usá enteros de 64 bits.
- Casos borde: $a = b$ (un solo día), $a = 1$, $b = n$.

Pista (leela solo si te trabás): el truco del $P[0] = 0$ evita un caso especial cuando $a = 1$: la suma de los días 1 a b es $P[b] - P[0]$.

Bonus (opcional)

- Si además hay que actualizar movimientos (corregir un día), las sumas prefijas dejan de servir: pensá qué estructura permite actualizar y consultar en $O(\log n)$ (la vas a ver en el nivel 7).
- Encontrá el rango de días con el saldo neto más negativo (el período de mayor gasto).

Cierre: Es la técnica de sumas prefijas, que usan los sistemas de reportes para responder consultas por rango sin recalcularlo todo.

Ejercicio 158: ¿Cuántos días hasta que haga más calor?

Nivel 4: Semi-Senior I · Eficiencia · Temática: Clima y energía · Dificultad 32/100

Situación: Un servicio de pronóstico extendido quiere mostrar, para cada día, cuántos días faltan hasta que haga más calor. Es útil para quienes planifican cosechas, eventos al aire libre o compras de energía. La serie histórica tiene décadas de datos diarios, así que la solución tiene que ser lineal.

Datos de entrada

Temperaturas máximas de 12 días consecutivos (°C): 18, 21, 19, 17, 23, 25, 22, 20, 24, 26, 26, 21

Tu programa debe

1. Para cada día, encontrar el primer día posterior con una temperatura estrictamente mayor.
2. Calcular cuántos días hay que esperar (0 si no hay ninguno más caluroso después).
3. Resolverlo con una pila que guarde los días que todavía no encontraron un día más caluroso.
4. Mostrar las temperaturas y los días de espera alineados.

Salida esperada

```
Temperaturas: 18 21 19 17 23 25 22 20 24 26 26 21
Días a esperar: 1 3 2 1 1 4 2 1 1 0 0 0
```

Reglas

- Límites: $1 \leq n \leq 1.000.000$ días; temperaturas entre -60 y 60 .
- Complejidad esperada: $O(n)$. Buscar hacia adelante desde cada día es $O(n^2)$ y no pasa.
- Una temperatura igual no cuenta como "más calor".
- Formato: cada número en 2 caracteres alineados a la derecha, separados por un espacio.
- Casos borde: temperaturas siempre decrecientes (todos 0), todas iguales.

Pista (leela solo si te trabás): la pila guarda índices de días con temperaturas decrecientes. Cuando llega un día más caluroso que el del tope, ese día "resuelve" al del tope, y puede seguir resolviendo a los de abajo.

Bonus (opcional)

- Calculá también, para cada día, cuántos días atrás fue el último día más caluroso.
- Resolvé la variante circular: después del último día, se vuelve a empezar desde el primero.

Cierre: Es la técnica de pila monótona, que se usa en análisis de series de precios, alturas y señales.

Ejercicio 159: El mejor día para comprar y vender

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 32/100

Situación: Una plataforma de educación financiera tiene un simulador: "si hubieras sabido los precios de antemano, ¿cuánto podrías haber ganado con una sola compra y una sola venta?". Sirve para mostrar lo difícil que es acertar. El simulador tiene que responder al instante incluso con años de precios diarios.

Datos de entrada

Acción AGRO (precio de cierre por día): 1520, 1480, 1395, 1440, 1610, 1575, 1702, 1650, 1590

Acción TELC (precio de cierre por día): 980, 955, 930, 910, 902

Tu programa debe

1. Recorrer los precios una sola vez manteniendo el precio mínimo visto hasta el momento (y su día).
2. En cada día, calcular la ganancia de vender ese día habiendo comprado en el mínimo anterior, y quedarse con la mejor.
3. Si ninguna ganancia es positiva, informar que no conviene operar.
4. Mostrar los días de compra y venta, la ganancia por acción y el porcentaje.

Salida esperada

Acción AGRO: comprar el día 3 a \$1395 y vender el día 7 a \$1702 -> ganancia \$307 por acción (22.01%)

Acción TELC: no hay forma de ganar, mejor no operar

Reglas

- Límites: $1 \leq n \leq 1.000.000$ días; precios hasta 10^9 .
- Complejidad esperada: $O(n)$ en tiempo y $O(1)$ de memoria. Probar todos los pares (compra, venta) es $O(n^2)$.
- Hay que comprar antes de vender (no se puede vender en corto).
- Si hay varias opciones con la misma ganancia máxima, se informa la primera que encuentra el recorrido.
- Porcentaje con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): en cada día, la mejor compra posible para vender hoy es la del mínimo anterior. Actualizá la ganancia antes de actualizar el mínimo.

Bonus (opcional)

- Permití hasta dos operaciones (dos compras y dos ventas que no se superpongan).
- Permití operaciones ilimitadas pagando una comisión fija por operación.

Cierre: Es un clásico de entrevistas técnicas en empresas financieras: mide si sabés reducir un problema cuadrático a una sola pasada.

Ejercicio 160: El lado del potrero cuadrado

Nivel 4: Semi-Senior I · Eficiencia · Temática: Agricultura · Dificultad 33/100

Situación: Un agrimensor tiene que marcar el potrero cuadrado más grande que entre en una cierta superficie, con lados de metros enteros. Sus planillas tienen superficies enormes (hasta 10^{18} m^2) para ejercicios de estancias gigantes en modo "simulación" y la raíz cuadrada con decimales le da errores de redondeo. Te pide un método exacto.

Datos de entrada

Superficies (m^2): 400, 10000, 20000, 999999999999, 1000000000000000000

Tu programa debe

1. Buscar con búsqueda binaria el mayor lado L entero tal que $L \times L \leq \text{área}$.
2. Indicar si el cuadrado es exacto o cuántos m^2 sobran.
3. Calcular los metros de alambre para el perímetro ($4 \times L$).
4. Mostrar cuántas iteraciones hizo la búsqueda.

Salida esperada

```
Área 400 m²: lado 20 m (exacto) | alambre 80 m | 9 iteraciones
Área 10000 m²: lado 100 m (exacto) | alambre 400 m | 14 iteraciones
Área 20000 m²: lado 141 m (sobran 119 m²) | alambre 564 m | 14 iteraciones
Área 999999999999 m²: lado 999999 m (sobran 1999998 m²) | alambre 3999996 m | 30 iteraciones
Área 1000000000000000000 m²: lado 1000000000 m (exacto) | alambre 4000000000 m | 30 iteraciones
```

Reglas

- Límites: $1 \leq \text{área} \leq 10^{18}$.
- Complejidad esperada: $O(\log(\text{área}))$ iteraciones.
- No uses raíz cuadrada con números decimales: con 10^{18} , la precisión de un decimal de 64 bits no alcanza para garantizar el entero exacto.
- Implementación exacta (para que coincidan las iteraciones): inicio = 0, fin = mínimo(área, 2.000.000.000); mientras inicio < fin: medio = (inicio + fin + 1) / 2 con división entera; si medio $\times$ medio $\leq$ área, inicio = medio; si no, fin = medio - 1.
- Cuidado: medio $\times$ medio puede llegar a $4 \cdot 10^{18}$, que todavía entra en un entero de 64 bits con signo (máximo $\approx 9,2 \cdot 10^{18}$).

Pista (leela solo si te trabás): la pregunta " $L \times L \leq \text{área}$?" es verdadera para todos los L chicos y falsa para todos los grandes. Buscás el último verdadero: eso es búsqueda binaria sobre la respuesta.

Bonus (opcional)

- Generalizá a la raíz cúbica entera (el silo cúbico más grande).
- Resolvé el potrero rectangular de lados enteros con proporción 2:1 de mayor superficie posible.

Cierre: La búsqueda binaria sobre la respuesta aparece en todos lados: desde calcular raíces exactas hasta ajustar capacidades de servidores.

Ejercicio 161: Semáforo y fila de autos

Nivel 4: Semi-Senior I · Eficiencia · Temática: Transporte · Dificultad 33/100

Situación: La municipalidad quiere evaluar si el semáforo de una esquina muy transitada tiene bien configurados los tiempos. Tu programa simula la fila de autos: cuándo llega cada uno, cuándo cruza y cuánto espera, con un ciclo fijo de verde y rojo.

Datos de entrada

Ciclo: verde de 0 a 19 s, rojo de 20 a 39 s, y se repite (período de 40 s). Cada auto tarda 3 s en cruzar.

Auto	Llega (s)
1	0
2	1
3	2
4	8
5	15
6	16
7	17
8	30
9	41

Tu programa debe

1. Procesar los autos en orden de llegada (es una sola fila: nadie se adelanta).
2. Un auto empieza a cruzar en el primer instante $t \geq$ su llegada en que el auto anterior ya terminó de cruzar, el semáforo está en verde y le alcanza el verde para cruzar completo ($t \text{ resto } 40 + 3 \leq 20$).
3. Si no le alcanza, espera al comienzo del siguiente verde.
4. Mostrar llegada, inicio del cruce y espera de cada auto, y la espera promedio y máxima.

Salida esperada

```
Auto 1: llega 0 | cruza en 0 | espera 0
Auto 2: llega 1 | cruza en 3 | espera 2
Auto 3: llega 2 | cruza en 6 | espera 4
Auto 4: llega 8 | cruza en 9 | espera 1
Auto 5: llega 15 | cruza en 15 | espera 0
Auto 6: llega 16 | cruza en 40 | espera 24
Auto 7: llega 17 | cruza en 43 | espera 26
Auto 8: llega 30 | cruza en 46 | espera 16
Auto 9: llega 41 | cruza en 49 | espera 8
Espera promedio: 9.00 s | Máxima: 26 s
```

Reglas

- Límites: hasta 1.000.000 de autos; tiempos hasta 10^9 segundos.
- Complejidad esperada: $O(n)$. No simules segundo a segundo: calculá el próximo verde con aritmética.
- Un auto que termina de cruzar justo en el segundo 20 del ciclo llegó a tiempo (" ≤ 20 ").
- Espera promedio con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): el próximo verde después de un instante t en rojo empieza en $(t / 40 + 1) \times 40$, con división entera. Con eso saltás directo sin simular los segundos intermedios.

Bonus (opcional)

- Calculá qué duración de verde (con el mismo período de 40 s) minimiza la espera promedio.
- Agregá un segundo carril que se habilita cuando hay más de 5 autos esperando.

Cierre: Es el mismo tipo de simulación que usan los ingenieros de tránsito para configurar las "ondas verdes".

Ejercicio 162: Reservas que se pueden fusionar

Nivel 4: Semi-Senior I · Eficiencia · Temática: Turismo · Dificultad 33/100

Situación: Un complejo de cabañas recibe reservas por varios canales para la misma cabaña, y el calendario se llena de fragmentos. El dueño quiere ver los bloques continuos de ocupación (juntando reservas que se superponen o que se tocan) y los huecos libres para ofrecer promociones.

Datos de entrada

Reservas como [día de entrada, día de salida) — la salida no es una noche ocupada:

[3, 7), [1, 4), [10, 12), [7, 9), [15, 18), [12, 13), [20, 21), [16, 17)

Tu programa debe

1. Ordenar las reservas por día de entrada.
2. Recorrerlas fusionando cada una con el bloque actual si empieza antes o justo cuando termina ese bloque.
3. Calcular las noches ocupadas en total.
4. Mostrar los bloques y los huecos libres entre bloques consecutivos.

Salida esperada

Bloques ocupados: [1, 9), [10, 13), [15, 18), [20, 21)
Noches ocupadas: 15
Huecos libres entre bloques: [9, 10), [13, 15), [18, 20)

Reglas

- Límites: hasta 200.000 reservas; días hasta 10^9 .
- Complejidad esperada: $O(n \log n)$ por el ordenamiento; la fusión es $O(n)$.
- [3, 7) y [7, 9) se fusionan (se tocan: salida y entrada el mismo día).
- Casos borde: una sola reserva; una reserva contenida dentro de otra ([15, 18) contiene a [16, 17)).

Pista (leela solo si te trabás): después de ordenar por entrada, alcanza con comparar cada reserva con el último bloque armado: si empieza a más tardar donde termina el bloque, lo extiende (tomando el máximo de los finales).

Bonus (opcional)

- Con varias cabañas, calculá para cada día cuántas están ocupadas y el día de máxima ocupación.
- Dada una nueva reserva, decí en $O(\log n)$ si se puede aceptar sin superponerse (con los bloques ordenados).

Cierre: Es la fusión de intervalos que usan los calendarios, los sistemas de reservas y los planificadores de recursos.

Ejercicio 163: El tramo de ADN sin bases repetidas

Nivel 4: Semi-Senior I · Eficiencia · Temática: Ciencia · Dificultad 33/100

Situación: Un laboratorio de biología molecular estudia proteínas representadas como cadenas de letras (cada letra es un aminoácido). Para un análisis de diversidad, necesitan encontrar el tramo continuo más largo en el que ningún aminoácido se repite. Las cadenas reales tienen decenas de miles de letras.

Datos de entrada

Secuencia (una sola línea):

```
MKTAYIAKQRQISFVKSHFSRQLEERLGLIEVQAPILSRVGDGTQDNLSGAEKAVQVKVK  
ALPDAQFEVVHSLAKWKRQTLGQHDFSAGEGLYTHMKALRPDEDRLSPLHSVYVDQWDWE  
RVMGDGERQFSTLKSTVEAIWAGIKATEAAVSEEFGLAPFLPDQIHVHSQELLSRYPD  
LDAKGRERAIKDLGAVFLVGIGKLSGDGHRHVRAPDYDDW
```

(Se muestra en líneas de 60 letras; es una única secuencia.)

Tu programa debe

1. Recorrer la secuencia con una ventana [inicio, i] que no tenga letras repetidas.
2. Guardar la última posición en que se vio cada letra.
3. Cuando la letra actual ya está dentro de la ventana, mover el inicio justo después de su aparición anterior.
4. Mostrar el tramo más largo, sus posiciones (desde 1) y su largo.

Salida esperada

```
Largo de la secuencia: 221  
Tramo más largo sin repetir: YTHMKALRPDE  
Posiciones: 93 a 103 (largo 11)
```

Reglas

- Límites: largo hasta 1.000.000; alfabeto de hasta 26 letras.
- Complejidad esperada: $O(n)$. Revisar todos los tramos es $O(n^2)$ o peor.
- Si hay varios tramos del largo máximo, se informa el primero.
- Casos borde: todas las letras iguales (largo 1), todas distintas (toda la cadena).

Pista (leela solo si te trabás): el inicio de la ventana nunca retrocede. Si la letra actual se vio antes pero antes del inicio actual, no molesta.

Bonus (opcional)

- Permití como máximo una repetición dentro del tramo.
- Encontrá el tramo más corto que contiene las 20 letras distintas de aminoácidos al menos una vez.

Cierre: Es la técnica de ventana deslizante con conjunto (o mapa de posiciones), muy usada en bioinformática y en análisis de texto.

Ejercicio 164: Capacidad mínima del camión para cumplir el plazo

Nivel 4: Semi-Senior I · Eficiencia · Temática: Logística y envíos · Dificultad 33/100

Situación: Una distribuidora tiene que entregar una lista de pedidos en un orden fijo (la ruta ya está armada) en 4 días. Cada día el camión carga pedidos consecutivos de la lista hasta donde le da la capacidad. Quieren alquilar el camión más chico posible que permita terminar en plazo.

Datos de entrada

Pesos de los pedidos, en orden (kg): 320, 150, 480, 260, 90, 410, 380, 200, 170, 530, 240, 300

Plazo: 4 días.

Tu programa debe

1. Escribir una función que, dada una capacidad, calcule cuántos días hacen falta cargando pedidos en orden hasta llenar el camión.
2. Hacer una búsqueda binaria sobre la capacidad entre el pedido más pesado y la suma de todos.
3. Encontrar la menor capacidad que cumple el plazo.
4. Mostrar la capacidad, las iteraciones y el reparto de cada día con esa capacidad.

Salida esperada

```
Capacidad mínima: 1070 kg (11 iteraciones de búsqueda)
Día 1: 320 + 150 + 480 = 950 kg
Día 2: 260 + 90 + 410 = 760 kg
Día 3: 380 + 200 + 170 = 750 kg
Día 4: 530 + 240 + 300 = 1070 kg
```

Reglas

- Límites: hasta 100.000 pedidos de hasta 10^6 kg; plazo entre 1 y n días.
- Complejidad esperada: $O(n \cdot \log(\text{suma de pesos}))$.
- Implementación exacta: inicio = máximo peso, fin = suma; mientras inicio < fin: medio = (inicio + fin) / 2 con división entera; si alcanza con medio, fin = medio; si no, inicio = medio + 1.
- Los pedidos no se dividen ni se reordenan. Terminar antes del plazo está permitido.
- Casos borde: plazo de 1 día (la suma total); plazo igual a la cantidad de pedidos (el máximo).

Pista (leela solo si te trabás): la función "¿alcanza con esta capacidad?" es monótona: si alcanza con una capacidad, también alcanza con cualquier capacidad mayor. Eso habilita la búsqueda binaria sobre la respuesta.

Bonus (opcional)

- Cada día de alquiler cuesta distinto según la capacidad: encontrá la combinación de capacidad y cantidad de días más barata.
- Permití dos camiones en paralelo, cada uno con su parte de la ruta.

Cierre: Es la búsqueda binaria sobre la respuesta, que se usa para dimensionar flotas, servidores y cualquier recurso con un plazo fijo.

Ejercicio 165: Lluvia atrapada entre silos

Nivel 4: Semi-Senior I · Eficiencia · Temática: Agricultura · Dificultad 34/100

Situación: En un acopio de granos, los silos están en fila y tienen alturas distintas. Después de una tormenta, el agua queda atrapada en los huecos entre silos y humedece los granos que están en el piso. El encargado quiere calcular cuánta agua se acumula para dimensionar el drenaje.

Datos de entrada

Alturas de las columnas (silos y huecos), de izquierda a derecha: 4, 1, 3, 0, 2, 5, 1, 1, 4, 2, 0, 3

Cada posición tiene ancho 1.

Tu programa debe

1. Para cada posición, el agua que acumula es el mínimo entre la altura máxima a su izquierda y a su derecha, menos su propia altura (si es positivo).
2. Calcularlo con dos punteros que avanzan desde los extremos, llevando el máximo de cada lado, sin arreglos auxiliares.
3. Mostrar un dibujo (# = silo, ~ = agua, . = aire), el agua por posición y el total.

Salida esperada

```

.....#.....
#~...#~#...
#~...#~#~#
#~...#~#~#
###~#####~#
Agua acumulada por posición: 0 3 1 4 2 0 3 3 0 1 3 0
Total: 20 unidades

```

Reglas

- Límites: hasta 1.000.000 de posiciones; alturas hasta 10^9 .
- Complejidad esperada: $O(n)$ en tiempo y $O(1)$ de memoria extra (sin contar el dibujo, que es solo para este ejemplo chico).
- El total puede superar 2^{31} : usá enteros de 64 bits.
- Casos borde: alturas crecientes o decrecientes (no se acumula nada), menos de 3 posiciones.

Pista (leela solo si te trabás): si el máximo de la izquierda es menor que la altura del extremo derecho, el agua de la posición izquierda ya está definida por el máximo de la izquierda: podés calcularla y avanzar.

Bonus (opcional)

- Resolvé la versión en dos dimensiones: una grilla de alturas y el agua que queda atrapada (vas a necesitar un heap).
- Calculá cuánta agua se va por el drenaje si hay una cañería en la posición 4.

Cierre: Es un clásico de entrevistas técnicas que muestra cómo pasar de $O(n^2)$ a $O(n)$ con dos punteros bien pensados.

Ejercicio 166: La versión que rompió todo

Nivel 4: Semi-Senior I · Eficiencia · Temática: Seguridad informática · Dificultad 34/100

Situación: Una fintech detectó que su app de pagos tiene un bug de seguridad en la versión 1000, y sabe que la versión 1 estaba bien. Cada prueba de una versión tarda 20 minutos (hay que compilarla y correr la suite de seguridad), así que no pueden probar las 1000. Tienen que encontrar la primera versión rota con la menor cantidad de pruebas.

Datos de entrada

Versiones: de 1 a 1000. La 1 está bien, la 1000 está rota, y si una versión está rota todas las siguientes también.

Para que puedas verificar tu salida, en este caso la primera versión rota es la 617 (tu programa solo puede "probar" una versión y saber si está rota: en el código, `probar(v)` devuelve verdadero si $v \geq 617$).

Tu programa debe

1. Implementar una búsqueda binaria que solo use la función `probar(versión)`.
2. Registrar cada versión probada y su resultado (ok o x).
3. Informar la primera versión rota y la cantidad de pruebas.

Salida esperada

```
Versiones probadas: 500ok 750x 625x 563ok 594ok 610ok 618x 614ok 616ok 617x
Primera versión rota: 617
Pruebas realizadas: 10 (en lugar de hasta 1000)
```

Reglas

- Límites: hasta $2^{31} - 1$ versiones.
- Complejidad esperada: $O(\log n)$ llamadas a probar.
- Implementación exacta: `inicio = 1`, `fin = n`; mientras `inicio < fin`: `medio = (inicio + fin) / 2` con división entera; si `probar(medio)` está rota, `fin = medio`; si no, `inicio = medio + 1`.
- Con n cercano a 2^{31} , `inicio + fin` puede desbordar un entero de 32 bits: calculá `medio = inicio + (fin - inicio) / 2`.
- Formato: cada prueba como número seguido de "ok" o "x", separadas por espacios.

Pista (leela solo si te trabás): buscás el primer verdadero en una secuencia de falsos seguida de verdaderos. Cuando una versión está rota, puede ser la primera, así que no la descartes: `fin = medio` (no `medio - 1`).

Bonus (opcional)

- Si cada prueba falla al azar el 5 % de las veces (un falso positivo), ¿cómo harías la búsqueda confiable?
- Con dos máquinas de prueba en paralelo, ¿cuántas rondas hacen falta? (Pista: dividí en tres en lugar de en dos.)

Cierre: Es exactamente lo que hace el comando de bisección de los sistemas de control de versiones para encontrar el commit que introdujo un bug.

Ejercicio 167: La ventana de mayor pulso promedio

Nivel 4: Semi-Senior I · Eficiencia · Temática: Salud · Dificultad 34/100

Situación: Un gimnasio con clases de spinning registra el pulso de cada alumno minuto a minuto con bandas cardíacas. El entrenador quiere detectar los tramos de 5 minutos donde el promedio fue más alto, y marcar todos los tramos donde el promedio superó 140 latidos por minuto para ajustar la intensidad.

Datos de entrada

Pulso por minuto (minutos 1 a 25): 88, 92, 101, 115, 128, 136, 142, 150, 155, 149, 138, 131, 124, 118, 145, 152, 158, 160, 154, 139, 120, 105, 98, 92, 90

Ventana: 5 minutos. Umbral de alerta: promedio mayor a 140 lpm.

Tu programa debe

1. Recorrer todas las ventanas de 5 minutos consecutivos con una suma que se actualiza al deslizar.
2. Encontrar la ventana de mayor promedio (en empate, la primera).
3. Listar las ventanas cuyo promedio supera 140 lpm, como "inicio-fin".

Salida esperada

Ventana de mayor pulso promedio: minutos 15 a 19 (153.8 lpm)
Ventanas con promedio mayor a 140 lpm (8): 5-9, 6-10, 7-11, 8-12, 14-18, 15-19, 16-20, 17-21

Reglas

- Límites: hasta 10^6 minutos (sesiones de meses concatenadas); ventana entre 1 y n.
- Complejidad esperada: $O(n)$.
- Promedio con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- La comparación con el umbral es estricta y con el promedio sin redondear.

Pista (leela solo si te trabás): comparar promedios de ventanas del mismo largo es igual que comparar sus sumas: no hace falta dividir hasta el final.

Bonus (opcional)

- Fusioná las ventanas de alerta superpuestas en tramos continuos ("de 6 a 12").
- Calculá la ventana de mayor promedio con largo variable, de al menos 5 minutos.

Cierre: Es el mismo cálculo de promedio móvil que muestran los relojes deportivos y los monitores cardíacos.

Ejercicio 168: Atrás y adelante en el navegador

Nivel 4: Semi-Senior I · Eficiencia · Temática: Telecomunicaciones · Dificultad 34/100

Situación: Una operadora de telecomunicaciones desarrolla un navegador liviano para celulares de gama baja. Los botones de atrás y adelante tienen que comportarse igual que en cualquier navegador, incluido el detalle de que visitar una página nueva borra el historial "adelante".

Datos de entrada

Paso	Acción	Página
1	visitar	inicio
2	visitar	productos
3	visitar	zapatillas
4	atrás	
5	atrás	
6	adelante	
7	visitar	ofertas
8	adelante	
9	atrás	
10	atrás	
11	atrás	
12	atrás	

Tu programa debe

1. Mantener la página actual y dos pilas: atrás y adelante.
2. Al visitar una página, la actual pasa a la pila atrás y la pila adelante se vacía.
3. Atrás y adelante mueven la página actual entre las pilas; si la pila está vacía, se informa y no cambia nada.
4. Mostrar la página actual después de cada acción y el contenido final de las pilas (de abajo hacia arriba).

Salida esperada

```
visitar inicio: inicio
visitar productos: productos
visitar zapatillas: zapatillas
atrás: productos
atrás: inicio
adelante: productos
visitar ofertas: ofertas
adelante: ofertas (no hay página siguiente)
atrás: productos
atrás: inicio
atrás: inicio (no hay página anterior)
atrás: inicio (no hay página anterior)
Pila atrás: [] | Pila adelante: [ofertas, productos]
```

Reglas

- Límites: hasta 10^6 acciones.
- Complejidad esperada: $O(1)$ por acción.
- El primer visitar no tiene página actual previa, así que no apila nada.
- Formato: la acción, la página (si tiene) y la página actual; el aviso entre paréntesis en la misma línea.

Pista (leela solo si te trabás): atrás y adelante son simétricos: lo que sale de una pila entra en la otra, junto con la página actual.

Bonus (opcional)

- Limitá el historial a las últimas 50 páginas descartando las más viejas (la pila se vuelve una cola doble).
- Mostrá el historial completo en orden cronológico, incluyendo las páginas que se "perdieron" al visitar una nueva.

Cierre: Es el mismo modelo de dos pilas que usan todos los navegadores web y muchas apps con navegación entre pantallas.

Ejercicio 169: Turnos de la impresora de la facultad

Nivel 4: Semi-Senior I · Eficiencia · Temática: Educación · Dificultad 34/100

Situación: En la biblioteca de una facultad hay una sola impresora y dos filas: los trabajos de docentes tienen prioridad sobre los de alumnos. Los alumnos se quejan de que sus trabajos "nunca salen". La biblioteca quiere simular la cola de un día típico para medir las esperas.

Datos de entrada

Velocidad: 10 páginas por minuto (un trabajo de 25 páginas tarda 3 minutos, redondeando hacia arriba).

Trabajo	Tipo	Llega (min)	Páginas
T1	alumno	0	25
T2	alumno	1	12
T3	docente	2	40
T4	alumno	3	5
T5	docente	6	10
T6	alumno	7	30
T7	docente	20	8

Tu programa debe

1. Mantener dos colas: docentes y alumnos.
2. Cada vez que la impresora se libera, agregar a las colas los trabajos que ya llegaron y tomar el primero de la cola de docentes; si está vacía, el de alumnos.
3. Si no hay nada en ninguna cola, avanzar el tiempo hasta la próxima llegada.
4. Mostrar, en el orden en que se imprimen, la llegada, el inicio, el fin y la espera de cada trabajo.

Salida esperada

```
T1 (alumno, 25 págs): llega 0 | imprime 0 a 3 | espera 0
T3 (docente, 40 págs): llega 2 | imprime 3 a 7 | espera 1
T5 (docente, 10 págs): llega 6 | imprime 7 a 8 | espera 1
T2 (alumno, 12 págs): llega 1 | imprime 8 a 10 | espera 7
T4 (alumno, 5 págs): llega 3 | imprime 10 a 11 | espera 7
T6 (alumno, 30 págs): llega 7 | imprime 11 a 14 | espera 4
T7 (docente, 8 págs): llega 20 | imprime 20 a 21 | espera 0
```

Reglas

- Límites: hasta 10^5 trabajos.
- Complejidad esperada: $O(n)$ después de ordenar por llegada (cada trabajo entra y sale de una cola una sola vez).
- Un trabajo que llega justo cuando la impresora se libera ya puede ser elegido.
- Un trabajo no se interrumpe una vez que empezó.

Pista (leela solo si te trabás): el tiempo de la simulación salta de evento en evento: cuando la impresora termina un trabajo, o cuando llega un trabajo con la impresora ociosa.

Bonus (opcional)

- Evitá la inanición de los alumnos: si un alumno lleva esperando más de 5 minutos, pasa adelante de los docentes.
- Agregá una segunda impresora solo para trabajos de hasta 10 páginas.

Cierre: Es el mismo esquema de colas con prioridad que usan los sistemas operativos para decidir qué proceso usa la CPU.

Ejercicio 170: Tramos de gastos que suman exacto

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 34/100

Situación: Un auditor revisa los gastos diarios de caja chica de una empresa (con devoluciones que aparecen como montos negativos). Sospecha que alguien fracciona un gasto de \$10.000 en varios días consecutivos para esquivar el límite de aprobación. Necesita encontrar todos los tramos de días consecutivos que suman exactamente \$10.000.

Datos de entrada

Movimientos de caja chica por día (días 1 a 13): 4000, 6000, -2000, 3000, 5000, 4000, -6000, 8000, 2000, 1000, 7000, -3000, 3000

Monto buscado: \$10.000

Tu programa debe

1. Calcular la suma acumulada día por día.
2. Guardar en un mapa, para cada valor de suma acumulada, en qué días apareció (con la suma 0 "antes del día 1").
3. En cada día, los tramos que terminan ahí y suman el monto buscado empiezan justo después de los días donde la suma acumulada era (actual – monto).
4. Mostrar la cantidad de tramos y cada uno, ordenados por día de fin y, a igual fin, por día de inicio.

Salida esperada

```
Tramos que suman exactamente $10000: 6
días 1 a 2
días 3 a 6
días 2 a 7
días 8 a 9
días 9 a 11
días 9 a 13
```

Reglas

- Límites: hasta 10^6 días; montos entre -10^9 y 10^9 (pueden ser negativos, así que la ventana deslizante no sirve).
- Complejidad esperada: $O(n + \text{cantidad de tramos})$ con el mapa de sumas acumuladas. Revisar todos los tramos es $O(n^2)$.
- Usá enteros de 64 bits para las sumas.
- Casos borde: un solo día que ya suma el monto; monto 0 (tramos que se compensan).

Pista (leela solo si te trabás): la suma de los días a..b es $S[b] - S[a - 1]$. Pedir que valga k es pedir $S[a - 1] = S[b] - k$: buscás en el mapa hacia atrás en lugar de recorrer.

Bonus (opcional)

- Encontrá el tramo más largo que suma exactamente k.
- Buscá tramos que sumen entre \$9.500 y \$10.500 (tolerancia), manteniendo una complejidad razonable.

Cierre: Es la técnica de sumas prefijas con mapa, usada en auditoría y en detección de patrones de fraude por fraccionamiento.

Ejercicio 171: Canciones que completan justo una hora

Nivel 4: Semi-Senior I · Eficiencia · Temática: Música · Dificultad 35/100

Situación: Una radio online arma bloques de exactamente una hora combinando dos pistas largas (sesiones en vivo, entrevistas, sets de DJ). El catálogo tiene miles de pistas, y el programador quiere encontrar rápido todos los pares que suman exactamente 60 minutos.

Datos de entrada

Pista	Duración (segundos)
Faro	412
Humo	245
Marea	1980
Otoño	1620
Ruta 40	1355
Brasa	3188
Siesta	2245
Luz de Neón	1810
Tren de Noche	1790

Tu programa debe

1. Ordenar las pistas por duración (en empate, por nombre).
2. Recorrer con dos punteros buscando pares que sumen exactamente 3.600 segundos.
3. Registrar también el par con mayor duración total sin pasarse de la hora.
4. Mostrar los pares exactos (primero la pista más corta) y el mejor par sin pasarse en formato M:SS.

Salida esperada

```
Pares de exactamente 1 hora: 4
Faro + Brasa
Ruta 40 + Siesta
Otoño + Marea
Tren de Noche + Luz de Neón
Mejor par sin pasarse: Faro + Brasa = 60:00
```

Reglas

- Límites: hasta 200.000 pistas; duraciones hasta 10.800 segundos.
- Complejidad esperada: $O(n \log n)$.
- Una pista no se combina consigo misma.
- Casos borde: ningún par exacto, duraciones repetidas.

Pista (leela solo si te trabás): ordenar primero es lo que habilita los dos punteros: con la lista ordenada, sabés hacia qué lado mover cada índice.

Bonus (opcional)

- Buscá tríos que sumen exactamente una hora.
- Con un conjunto de duraciones vistas, resolvé los pares exactos sin ordenar, en $O(n)$.

Cierre: Es la misma búsqueda de complementos que usan los sistemas de programación de radio y TV para llenar tandas exactas.

Ejercicio 172: La papa caliente

Nivel 4: Semi-Senior I · Eficiencia · Temática: Videojuegos · Dificultad 35/100

Situación: Un juego de fiesta para celular recrea "la papa caliente": los jugadores están en ronda, se pasa la papa y el que la tiene cuando suena la alarma queda afuera. En la versión digital la alarma suena siempre en la tercera persona desde la que empieza, y el diseñador quiere saber quién gana según el orden inicial.

Datos de entrada

Ronda inicial (en orden): Ana, Beto, Cami, Dani, Eli, Fer, Gabi

Empieza con la papa Ana. Se cuenta desde quien tiene la papa (esa persona es la 1): la persona número 3 queda eliminada, y la papa pasa a la persona siguiente a la eliminada, que vuelve a contar como 1.

Tu programa debe

1. Representar la ronda con una cola.
2. Simular cada eliminación: pasar los primeros $k - 1$ jugadores del frente al final y sacar al que queda al frente.
3. Repetir hasta que quede uno solo.
4. Mostrar el orden de eliminación y el ganador.

Salida esperada

Orden de eliminación: Cami, Fer, Beto, Gabi, Eli, Ana
Ganador: Dani

Reglas

- Límites: hasta 10^5 jugadores; k hasta 10^5 .
- Complejidad esperada: $O(n \cdot k)$ con la cola es aceptable para este ejemplo; con n y k grandes vas a necesitar la fórmula del bonus.
- Con $k = 1$, se eliminan en el orden de la ronda.

Pista (leela solo si te trabás): rotar la cola $k - 1$ veces deja al eliminado al frente. Sacarlo deja al siguiente ya en posición de empezar a contar.

Bonus (opcional)

- Calculá solo el ganador en $O(n)$ con la recurrencia $J(1) = 0$, $J(n) = (J(n - 1) + k) \text{ resto } n$.
- Hacé que k cambie en cada ronda según una lista de valores.

Cierre: Es el problema de Josefo, un clásico de algoritmos que aparece en juegos, en planificación rotativa y en asignación circular de recursos.

Ejercicio 173: La calculadora de la base antártica

Nivel 4: Semi-Senior I · Eficiencia · Temática: Ciencia · Dificultad 35/100

Situación: En una base antártica, los instrumentos científicos más viejos usan una calculadora en notación posfija (primero los operandos, después el operador), que no necesita paréntesis. Los técnicos nuevos cargan las fórmulas en el software de análisis y necesitan un evaluador que además explique los errores.

Datos de entrada

Expresiones (tokens separados por espacios, enteros):

- 3 4 + 2 *
- 5 1 2 + 4 * + 3 -
- 7 2 /
- -7 2 /
- 2 3 4
- 4 0 /
- 1 +
- 10 2 8 * + 3 -

Ejemplo: 3 4 + 2 * significa $(3 + 4) \times 2$.

Tu programa debe

1. Recorrer los tokens: los números se apilan.
2. Ante un operador, desapilar dos valores (el primero que sale es el operando derecho), operar y apilar el resultado.
3. Detectar errores: faltan operandos, división por cero, o al final no queda exactamente un valor.
4. Mostrar el resultado o el error de cada expresión.

Salida esperada

```
"3 4 + 2 *" = 14
"5 1 2 + 4 * + 3 -" = 14
"7 2 /" = 3
"-7 2 /" = -3
"2 3 4" = ERROR: sobran 2 valores en la pila
"4 0 /" = ERROR: división por cero
"1 +" = ERROR: faltan operandos para '+'
"10 2 8 * + 3 -" = 23
```

Reglas

- Límites: hasta 10^6 tokens; resultados intermedios dentro de 64 bits.
- Complejidad esperada: $O(n)$.
- La división es entera y trunca hacia cero: $7 / 2 = 3$ y $-7 / 2 = -3$ (atención, porque algunas herramientas redondean hacia menos infinito y darían -4).
- Un número negativo se escribe pegado al signo (" -7 "); un "-" solo es la resta.

Pista (leela solo si te trabás): el orden de los operandos importa en la resta y la división: si la pila tiene $[10, 2]$ y llega "-", el resultado es $10 - 2$, no $2 - 10$.

Bonus (opcional)

- Convertí una expresión infija con paréntesis a posfija (algoritmo shunting-yard) y evaluala.
- Agregá variables: " $x 2 *$ " con x definida en un mapa.

Cierre: La evaluación con pila es la base de cómo los compiladores y las calculadoras científicas evalúan expresiones.

Ejercicio 174: Stock mínimo en cada ventana de días

Nivel 4: Semi-Senior I · Eficiencia · Temática: Logística y envíos · Dificultad 35/100

Situación: Un centro de distribución de alimentos revisa el stock diario de leche larga vida. La política dice que en ningún período de 4 días el stock mínimo puede bajar de 40 unidades (el stock de seguridad). Con años de datos de cientos de productos, recalculer el mínimo de cada ventana desde cero es demasiado lento.

Datos de entrada

Stock diario (días 1 a 14): 85, 72, 64, 58, 70, 44, 39, 52, 61, 35, 48, 66, 73, 80

Ventana: 4 días. Stock de seguridad: 40.

Tu programa debe

1. Recorrer los días manteniendo una cola doble de índices con stocks crecientes.
2. Al entrar un día, sacar del final los índices con stock mayor o igual; al salir de la ventana, sacar del frente.
3. El frente de la cola es siempre el mínimo de la ventana actual.
4. Mostrar el mínimo de cada ventana y las ventanas por debajo del stock de seguridad (como "inicio-fin").

Salida esperada

Mínimo de cada ventana de 4 días: 58 58 44 39 39 39 35 35 35 35 48
Ventanas por debajo del stock de seguridad (40): 4-7, 5-8, 6-9, 7-10, 8-11, 9-12, 10-13

Reglas

- Límites: hasta 10^6 días; ventana entre 1 y n.
- Complejidad esperada: $O(n)$ (cada índice entra y sale de la cola una vez). Recalculer cada ventana es $O(n \cdot k)$.
- "Por debajo" es estrictamente menor que 40.

Pista (leela solo si te trabás): si un día tiene menos stock que otro anterior dentro de la ventana, el anterior ya no puede ser el mínimo de ninguna ventana futura: se puede tirar.

Bonus (opcional)

- Calculá también el máximo de cada ventana con una segunda cola doble.
- Encontrá la ventana de largo mínimo que contiene al menos un día con stock por debajo de 40 y uno por encima de 80.

Cierre: Es la técnica de cola doble monótona, usada en monitoreo de métricas, análisis financiero y control de inventario.

Ejercicio 175: Parejas de tenis con nivel parecido

Nivel 4: Semi-Senior I · Eficiencia · Temática: Deportes · Dificultad 35/100

Situación: Un club de tenis organiza un torneo de dobles mixto y quiere parejas equilibradas: la diferencia de rating entre compañeros no puede superar 50 puntos. El capitán quiere saber cuántos pares posibles cumplen la regla y armar la mayor cantidad de parejas sin repetir jugadores.

Datos de entrada

Ratings de los jugadores: 1420, 1510, 1375, 1600, 1455, 1390, 1720, 1480, 1540, 1690

Diferencia máxima: 50.

Tu programa debe

1. Ordenar los ratings.
2. Contar los pares ($i < j$) con diferencia menor o igual a 50 usando dos punteros.
3. Armar parejas de forma voraz recorriendo la lista ordenada: si el jugador actual y el siguiente cumplen, forman pareja y se saltean ambos; si no, se saltea el actual.
4. Mostrar los ratings ordenados, la cantidad de pares posibles, las parejas y cuántos quedan sin pareja.

Salida esperada

```
Ratings ordenados: 1375 1390 1420 1455 1480 1510 1540 1600 1690 1720
Pares posibles con diferencia ≤ 50: 8
Parejas armadas (4): 1375-1390, 1420-1455, 1480-1510, 1690-1720
Sin pareja: 2
```

Reglas

- Límites: hasta 200.000 jugadores; ratings hasta 10^6 .
- Complejidad esperada: $O(n \log n)$ (ordenamiento + recorridos lineales). Comparar todos los pares es $O(n^2)$.
- El armado voraz de parejas consecutivas da la máxima cantidad posible de parejas (se puede demostrar con un argumento de intercambio).
- Casos borde: todos con el mismo rating; nadie puede formar pareja.

Pista (leela solo si te trabás): con la lista ordenada, para cada jugador i , los compatibles hacia atrás forman un bloque contiguo que empieza en un índice j que solo avanza. Eso cuenta los pares en $O(n)$.

Bonus (opcional)

- Minimizá la suma de las diferencias dentro de las parejas, sin importar cuántas quedan afuera.
- Agregá la condición de dobles mixto: cada pareja tiene que tener un jugador de cada categoría.

Cierre: Es el mismo emparejamiento por nivel que hacen los sistemas de matchmaking de juegos online y de ligas deportivas.

Lote 08: ejercicios 176 a 200

Ejercicio 176: Logs entre dos horarios

Nivel 4: Semi-Senior I · Eficiencia · Temática: Seguridad informática · Dificultad 36/100

Situación: Un equipo de seguridad informática investiga un incidente y necesita contar cuántos eventos del log ocurrieron entre dos horarios. El log del día tiene decenas de millones de líneas ya ordenadas por hora, y los analistas hacen cientos de consultas distintas mientras investigan.

Datos de entrada

Horas de los eventos (ordenadas; puede haber repetidas): 08:59:58, 09:00:00, 09:00:00, 09:14:31, 09:30:12, 09:59:59, 10:00:00, 10:00:01, 10:45:10, 11:02:33, 11:02:33, 11:59:00

Consultas [desde, hasta], ambas inclusive:

- [09:00:00, 09:59:59]
- [10:00:00, 10:00:00]
- [11:10:00, 11:50:00]
- [07:00:00, 23:59:59]
- [09:15:00, 11:02:33]

Tu programa debe

1. Convertir las horas a segundos desde la medianoche.
2. Implementar el límite inferior: la primera posición con hora $\geq$ desde.
3. Implementar el límite superior: la primera posición con hora $>$ hasta.
4. Para cada consulta, la cantidad es superior – inferior; mostrar también el primer y el último evento del rango con su número de línea (desde 1).

Salida esperada

```
[09:00:00, 09:59:59]: 5 eventos | primero 09:00:00 (#2), último 09:59:59 (#6)
[10:00:00, 10:00:00]: 1 evento | primero 10:00:00 (#7), último 10:00:00 (#7)
[11:10:00, 11:50:00]: 0 eventos
[07:00:00, 23:59:59]: 12 eventos | primero 08:59:58 (#1), último 11:59:00 (#12)
[09:15:00, 11:02:33]: 7 eventos | primero 09:30:12 (#5), último 11:02:33 (#11)
```

Reglas

- Límites: hasta $5 \cdot 10^7$ eventos y 10^5 consultas.
- Complejidad esperada: $O(\log n)$ por consulta, sin preprocesamiento extra.
- Implementá las dos búsquedas vos; los límites son semiabiertos: inicio = 0, fin = n; mientras inicio $<$ fin, medio = (inicio + fin) / 2.
- Si el rango no tiene eventos, no se muestran primero y último.
- Casos borde: horas repetidas justo en los límites, consultas que no tocan ningún evento, consultas que abarcan todo.

Pista (leela solo si te trabás): la diferencia entre las dos búsquedas es un solo símbolo: $<$ en el límite inferior y $\leq$ en el superior. Con eso cubrís bien los valores repetidos.

Bonus (opcional)

- Contá los eventos en ventanas de 5 minutos a lo largo de todo el día y encontrá el pico de actividad.
- Si los eventos llegan en tiempo real y hay que insertarlos manteniendo el orden, ¿qué estructura usarías?

Cierre: Es la misma búsqueda por rango que hacen las herramientas de análisis de logs y las bases de datos de series temporales.

Ejercicio 177: La racha más larga de días en verde

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 36/100

Situación: Un portal financiero publica curiosidades del mercado: "¿cuál fue la racha más larga de días en verde del índice?". Con décadas de datos diarios hay que resolverlo en una sola pasada, y además calcular cuánto subió (o bajó) el índice acumulado durante la racha.

Datos de entrada

Variación diaria del índice (%), días 1 a 16: 0.8, 1.2, -0.4, 0.5, 0.3, 0.9, 1.1, 0.2, -1.5, -0.7, -2.1, 0.4, -0.3, 0.6, 0.7, -0.2

Tu programa debe

1. Recorrer las variaciones una vez, llevando el inicio de la racha actual.
2. Encontrar la racha más larga de días con variación positiva (en empate, la primera).
3. Hacer lo mismo para los días con variación negativa.
4. Calcular la variación acumulada de cada racha componiendo los porcentajes: producto de $(1 + v / 100) - 1$.

Salida esperada

Racha más larga en verde: 5 días (del día 4 al 8) | variación acumulada +3.03%
Racha más larga en rojo: 3 días (del día 9 al 11) | variación acumulada -4.24%

Reglas

- Límites: hasta 10^6 días.
- Complejidad esperada: $O(n)$ en tiempo y $O(1)$ de memoria.
- Un día con variación 0 corta ambas rachas.
- Las variaciones se componen, no se suman: +1 % y +1 % dan +2,01 %.
- Variación acumulada con 2 decimales (0,5 hacia arriba), con signo + si es positiva. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): usá una función que reciba la condición (positiva o negativa) para no escribir dos veces el mismo recorrido.

Bonus (opcional)

- Encontrá el período (de cualquier largo) con mayor subida acumulada.
- Calculá la máxima caída desde un pico (drawdown máximo), un indicador de riesgo muy usado.

Cierre: Es el tipo de estadística que muestran los portales financieros y que usan los analistas para medir el momentum de un activo.

Ejercicio 178: El ascensor del edificio de oficinas

Nivel 4: Semi-Senior I · Eficiencia · Temática: Transporte · Dificultad 36/100

Situación: El consorcio de un edificio de oficinas quiere reprogramar su único ascensor: hoy atiende los llamados en el orden en que llegan y va y viene de punta a punta. La empresa de mantenimiento propone la estrategia de "barrido": subir atendiendo todos los llamados de arriba y después bajar atendiendo los de abajo. Te piden comparar las dos.

Datos de entrada

El ascensor está en el piso 4. Llamados pendientes, en orden de llegada: 5, 2, 8, 3, 9, 1, 7

Tu programa debe

1. Simular la estrategia por orden de llegada y sumar los pisos recorridos.
2. Simular el barrido: primero todos los pisos mayores o iguales al actual en orden ascendente, después los menores en orden descendente.
3. Mostrar el recorrido y los pisos de cada estrategia.
4. Calcular el ahorro del barrido en pisos y en porcentaje.

Salida esperada

```
Por orden de llegada: 4 -> 5 -> 2 -> 8 -> 3 -> 9 -> 1 -> 7 | 35 pisos recorridos
Barrido (sube y después baja): 4 -> 5 -> 7 -> 8 -> 9 -> 3 -> 2 -> 1 | 13 pisos recorridos
Ahorro del barrido: 22 pisos (62.9%)
```

Reglas

- Límites: hasta 10^5 llamados; pisos de 0 a 200.
- Complejidad esperada: $O(n \log n)$ por el ordenamiento del barrido.
- Ahorro en porcentaje con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Suponé que no llegan llamados nuevos durante la simulación.

Pista (leela solo si te trabás): la distancia entre dos pisos es el valor absoluto de la diferencia. El barrido es separar en dos grupos, ordenar cada uno y recorrer.

Bonus (opcional)

- Hacé que lleguen llamados nuevos mientras el ascensor se mueve (con la hora de cada llamado) y simulá el barrido de forma dinámica.
- Compará con la estrategia "el más cercano primero" y mostrá un caso donde un pasajero espera muchísimo (inanición).

Cierre: Es el algoritmo del ascensor (SCAN), que además se usa para planificar el movimiento del cabezal de los discos rígidos.

Ejercicio 179: Vistas únicas en la última hora

Nivel 4: Semi-Senior I · Eficiencia · Temática: Redes sociales · Dificultad 36/100

Situación: Un sitio de noticias muestra en vivo "X personas leyendo ahora", que en realidad son los usuarios únicos que vieron alguna nota en los últimos 60 minutos. Con miles de vistas por segundo, no se puede recorrer todo el historial en cada actualización.

Datos de entrada

Vistas (minuto, usuario), en orden: (0, ana), (5, beto), (12, ana), (30, caro), (44, dani), (58, beto), (61, eva), (70, ana), (95, caro), (104, fede), (118, fede), (125, gabi)

Minutos en los que se actualiza el contador: 59, 60, 90, 120, 130

Tu programa debe

1. Mantener una cola con las vistas dentro de la ventana y un mapa usuario → cantidad de vistas en la ventana.
2. En cada actualización, agregar las vistas nuevas ($\text{minuto} \leq \text{actual}$) y sacar las viejas ($\text{minuto} \leq \text{actual} - 60$).
3. Un usuario sale del mapa cuando su contador llega a 0.
4. Mostrar la cantidad de usuarios únicos y quiénes son, en orden alfabético.

Salida esperada

```
Minuto 59: 4 usuarios únicos en la última hora (ana, beto, caro, dani)
Minuto 60: 4 usuarios únicos en la última hora (ana, beto, caro, dani)
Minuto 90: 4 usuarios únicos en la última hora (ana, beto, dani, eva)
Minuto 120: 4 usuarios únicos en la última hora (ana, caro, eva, fede)
Minuto 130: 3 usuarios únicos en la última hora (caro, fede, gabi)
```

Reglas

- Límites: hasta 10^7 vistas y 10^5 actualizaciones.
- Complejidad esperada: $O(1)$ amortizado por vista (cada vista entra y sale de la cola una sola vez).
- La ventana del minuto t incluye las vistas de los minutos $t - 59$ a t .
- Casos borde: un usuario con varias vistas que van saliendo de a una; actualizaciones sin vistas nuevas.

Pista (leela solo si te trabás): la cola sabe qué vistas salen de la ventana; el mapa sabe si un usuario todavía tiene alguna vista adentro. Necesitás las dos cosas.

Bonus (opcional)

- Con cientos de millones de usuarios, el mapa no entra en memoria: vas a ver en el nivel 10 cómo estimarlo con HyperLogLog.
- Mostrá también el usuario más activo de la ventana.

Cierre: Es el mismo cálculo de "usuarios activos ahora" que muestran los paneles de analítica web en tiempo real.

Ejercicio 180: Hornos compartidos en la pastelería

Nivel 4: Semi-Senior I · Eficiencia · Temática: Gastronomía · Dificultad 36/100

Situación: Una pastelería artesanal tiene dos hornos y en diciembre se le acumulan los pedidos de pan dulce. Cada pedido está listo para hornear a cierta hora, tarda un tiempo fijo y tiene una hora límite de entrega. La dueña quiere saber cuántos pedidos llegan tarde y cuántos hornos necesitaría para cumplir con todos.

Datos de entrada

Tiempos en minutos desde las 6:00.

Pedido	Listo para hornear	Duración	Límite de salida
P1	0	40	50
P2	5	25	40
P3	10	60	90
P4	12	30	50
P5	20	45	80
P6	30	20	70
P7	35	35	90

Tu programa debe

1. Simular con 2 hornos: cada pedido, en orden, va al horno que le permite empezar antes (en empate, el de menor número).
2. Calcular inicio y fin de cada pedido y marcar los que terminan después de su límite.
3. Contar los pedidos tarde.
4. Repetir la simulación con 1, 2, 3... hornos hasta encontrar la menor cantidad sin pedidos tarde.

Salida esperada

```
P1: horno 1 | 0 a 40 | límite 50
P2: horno 2 | 5 a 30 | límite 40
P3: horno 2 | 30 a 90 | límite 90
P4: horno 1 | 40 a 70 | límite 50 | TARDE por 20 min
P5: horno 1 | 70 a 115 | límite 80 | TARDE por 35 min
P6: horno 2 | 90 a 110 | límite 70 | TARDE por 40 min
P7: horno 2 | 110 a 145 | límite 90 | TARDE por 55 min
Pedidos tarde con 2 hornos: 4
Hornos necesarios para que ninguno llegue tarde: 4
```

Reglas

- Límites: hasta 10^4 pedidos.
- Complejidad esperada: $O(n \cdot k)$ por simulación y $O(n \cdot k^2)$ para la búsqueda de k ; aceptable para estos límites (en el nivel 6 lo mejorarás con un heap).
- Terminar exactamente en el límite no es tarde.
- Formato: fin y límite alineados a la derecha en 3 y 2 caracteres.

Pista (leela solo si te trabás): escribí la simulación como una función que recibe la cantidad de hornos y devuelve los resultados. Así probar con distintas cantidades es un bucle.

Bonus (opcional)

- En lugar de probar $k = 1, 2, 3, \dots$, usá búsqueda binaria sobre k (¿es monótono? Pensá por qué sí).
- Reordená los pedidos por límite de entrega (el más urgente primero) y compará cuántos llegan tarde.

Cierre: Es el mismo problema de asignación de máquinas que resuelven las fábricas, las cocinas industriales y los centros de datos.

Ejercicio 181: La tasa que hace cerrar la cuota

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 37/100

Situación: Una financiera publica sus préstamos solo con el valor de la cuota ("llevate \$1.000.000 en 12 cuotas de \$100.000") y no informa la tasa. Una asociación de consumidores quiere calcular la tasa real de cada oferta. La fórmula de la cuota no se puede despejar para la tasa, así que hay que buscarla numéricamente.

Datos de entrada

Oferta	Monto	Cuotas	Cuota mensual
1	\$1.000.000	12	\$100.000
2	\$500.000	24	\$30.000
3	\$2.000.000	6	\$360.000

Cuota del sistema francés con tasa mensual i : $\text{cuota} = \text{monto} \times i / (1 - (1 + i)^{-n})$

Tu programa debe

1. Escribir una función que calcule la cuota para una tasa mensual dada.
2. Buscar la tasa mensual (TEM) con búsqueda binaria sobre números reales: si la cuota calculada es menor que la real, la tasa tiene que ser más alta.
3. Detener la búsqueda cuando el intervalo mida menos de 10^{-12} .
4. Mostrar la TEM, la TNA ($\text{TEM} \times 12$) y la TEA $((1 + \text{TEM})^{12} - 1)$, y la cantidad de iteraciones.

Salida esperada

```
$1000000 en 12 cuotas de $100000: TEM 2.9229% | TNA 35.07% | TEA 41.30% | 40 iteraciones
$500000 en 24 cuotas de $30000: TEM 3.1491% | TNA 37.79% | TEA 45.07% | 40 iteraciones
$2000000 en 6 cuotas de $360000: TEM 2.2442% | TNA 26.93% | TEA 30.52% | 40 iteraciones
```

Reglas

- Límites: montos hasta 10^9 ; hasta 360 cuotas; tasas mensuales entre 0,0001 % y 100 %.
- Complejidad esperada: $O(\log((\text{fin} - \text{inicio}) / \text{tolerancia}))$ evaluaciones de la fórmula.
- Implementación exacta: inicio = 0,000001, fin = 1; mientras fin - inicio > 10^{-12} , probar medio = (inicio + fin) / 2. El resultado es el punto medio final.
- TEM con 4 decimales; TNA y TEA con 2 (0,5 hacia arriba). Usá números decimales de alta precisión o dobles con cuidado. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la cuota crece cuando crece la tasa: esa monotonía es lo que permite descartar la mitad del intervalo en cada paso, igual que en la búsqueda binaria de enteros.

Bonus (opcional)

- Incluí gastos de otorgamiento (3 % del monto descontado al inicio) y calculá el costo financiero total.
- Resolvé lo mismo con el método de Newton-Raphson y compará las iteraciones (lo vas a ver a fondo en el nivel 9).

Cierre: Es el cálculo que hacen los simuladores de préstamos de las asociaciones de consumidores para exponer el costo financiero real.

Ejercicio 182: Ventas por región y rango de meses

Nivel 4: Semi-Senior I · Eficiencia · Temática: E-commerce · Dificultad 37/100

Situación: Un e-commerce nacional tiene un tablero donde los gerentes consultan ventas "de tales regiones en tales meses". Con cientos de regiones y años de datos, sumar la submatriz en cada consulta hace que el tablero tarde. Te piden precalcular una tabla que responda cualquier consulta rectangular en tiempo constante.

Datos de entrada

Ventas (en millones de pesos):

Región	Ene	Feb	Mar	Abr	May	Jun
Norte	120	135	150	142	160	171
Centro	310	298	325	340	356	362
Sur	88	91	97	102	99	110
Cuyo	140	152	149	161	170	168

Consultas (región desde, mes desde, región hasta, mes hasta), en el orden de la tabla:

- Centro-Sur, Feb-Abr
- Norte-Cuyo, Ene-Jun
- Sur-Cuyo, May-Jun
- Centro-Centro, Ene-Ene

Tu programa debe

1. Construir la matriz de sumas prefijas: $P[i][j]$ = suma de todas las celdas de filas $< i$ y columnas $< j$.
2. Calcularla con $P[i + 1][j + 1] = \text{valor} + P[i][j + 1] + P[i + 1][j] - P[i][j]$.
3. Responder cada consulta con inclusión-exclusión de cuatro valores de P .
4. Mostrar el resultado de cada consulta.

Salida esperada

```
Regiones Centro a Sur, meses Feb a Abr: 1253
Regiones Norte a Cuyo, meses Ene a Jun: 4396
Regiones Sur a Cuyo, meses May a Jun: 547
Regiones Centro a Centro, meses Ene a Ene: 310
```

Reglas

- Límites: hasta 2.000×2.000 celdas y 10^6 consultas.
- Complejidad esperada: $O(\text{filas} \times \text{columnas})$ de preprocesamiento y $O(1)$ por consulta.
- Usá enteros de 64 bits: la suma total puede superar 2^{31} .
- La matriz P tiene una fila y una columna extra de ceros para no tratar bordes como casos especiales.

Pista (leela solo si te trabás): la suma del rectángulo es el rectángulo grande hasta la esquina inferior derecha, menos la franja de arriba, menos la franja de la izquierda, más la esquina que restaste dos veces.

Bonus (opcional)

- Encontrá el rectángulo de 2 regiones $\times$ 3 meses consecutivos con mayores ventas.
- Soportá actualizaciones de celdas: vas a necesitar un árbol de Fenwick en dos dimensiones (nivel 7).

Cierre: Es la técnica de sumas prefijas en dos dimensiones, usada en tableros de BI, procesamiento de imágenes y juegos.

Ejercicio 183: Ráfagas de intentos de login

Nivel 4: Semi-Senior I · Eficiencia · Temática: Seguridad informática · Dificultad 37/100

Situación: Un home banking detecta ataques cuando llegan muchos intentos de login en pocos segundos, sin importar de qué IP vengan (los atacantes reparten el ataque). La regla es: si en cualquier ventana de 10 segundos hay más de 5 intentos, se activa una alerta que dura hasta que la ventana vuelva a tener 5 o menos.

Datos de entrada

Segundos en que llegó cada intento (ordenados): 0, 1, 1, 3, 4, 9, 10, 11, 12, 12, 13, 14, 15, 30, 31, 40, 41, 41, 42, 43, 43, 44, 49

Tu programa debe

1. Procesar los intentos en orden, manteniendo una cola con los de los últimos 10 segundos (los que cumplen $t > \text{actual} - 10$).
2. Al llegar cada intento, sacar del frente los que quedaron fuera de la ventana.
3. Informar cuando la ventana supera 5 intentos (inicio de alerta) y cuando vuelve a 5 o menos (fin de alerta), evaluado solo al llegar cada intento.
4. Informar el pico de intentos en una ventana y en qué segundo terminó.

Salida esperada

```
ALERTA en el segundo 9: 6 intentos en los últimos 10 s
Fin de la alerta en el segundo 11
ALERTA en el segundo 12: 6 intentos en los últimos 10 s
Fin de la alerta en el segundo 30
ALERTA en el segundo 43: 6 intentos en los últimos 10 s
Pico: 8 intentos en la ventana que termina en el segundo 15
```

Reglas

- Límites: hasta 10^7 intentos.
- Complejidad esperada: $O(n)$ (cada intento entra y sale una vez de la cola).
- La ventana del segundo t incluye los intentos de $t - 9$ a t .
- En empate de pico, se informa el primero.

Pista (leela solo si te trabás): la cola solo necesita los tiempos, no los datos completos del intento. El tamaño de la cola es la cantidad de intentos en la ventana.

Bonus (opcional)

- Hacé que el fin de alerta también se detecte cuando pasa el tiempo sin nuevos intentos (evaluando en cada segundo).
- Usá dos ventanas a la vez (10 s y 60 s) con umbrales distintos.

Cierre: Es el mismo mecanismo de detección por ventana deslizante que usan los sistemas de prevención de intrusiones y de rate limiting.

Ejercicio 184: El rectángulo más grande de la tribuna

Nivel 4: Semi-Senior I · Eficiencia · Temática: Deportes · Dificultad 37/100

Situación: Un club de fútbol quiere colgar la bandera más grande posible sobre la tribuna popular, que tiene sectores de distintas alturas. La bandera es rectangular, va apoyada en el piso y no puede sobresalir de ningún sector. El club necesita saber el área máxima y dónde ubicarla.

Datos de entrada

Altura de cada sector de 1 m de ancho, de izquierda a derecha (m): 3, 5, 5, 2, 6, 7, 7, 4, 1, 4, 4, 3

Tu programa debe

1. Recorrer los sectores manteniendo una pila de sectores con alturas crecientes (con la posición desde la que se puede extender cada uno).
2. Cuando llega un sector más bajo que el tope, sacar de la pila y calcular el área del rectángulo que termina justo antes.
3. Agregar un sector ficticio de altura 0 al final para vaciar la pila.
4. Informar el área máxima, las columnas que ocupa (desde 1) y su altura.

Salida esperada

Área máxima: 18 m² | columnas 5 a 7 | altura 6 m

Reglas

- Límites: hasta 10^6 sectores; alturas hasta 10^9 (el área puede superar 2^{31} : usá 64 bits).
- Complejidad esperada: $O(n)$. Probar todos los pares de extremos es $O(n^2)$.
- En empate de área, se informa el primero encontrado por el algoritmo con pila.
- Casos borde: todas las alturas iguales; alturas estrictamente crecientes o decrecientes.

Pista (leela solo si te trabás): cuando un sector sale de la pila, el rectángulo con su altura puede extenderse desde donde empezó (guardado en la pila) hasta el sector anterior al actual.

Bonus (opcional)

- Resolvé la versión en dos dimensiones: el rectángulo más grande de celdas libres en una grilla (aplicá este algoritmo fila por fila).
- Permití que la bandera tenga altura máxima de 5 m aunque haya sectores más altos.

Cierre: Es el problema del mayor rectángulo en un histograma, con aplicaciones en procesamiento de imágenes y planificación de espacios.

Ejercicio 185: Cola de pedidos armada con dos pilas

Nivel 4: Semi-Senior I · Eficiencia · Temática: Logística y envíos · Dificultad 37/100

Situación: Un sistema de despacho de pedidos corre sobre un componente viejo que solo ofrece pilas (último en entrar, primero en salir). Los pedidos tienen que salir en el orden en que entraron. Te piden armar una cola usando dos pilas y demostrar que, en promedio, cada operación es barata.

Datos de entrada

Paso	Operación	Pedido
1	encolar	P1
2	encolar	P2
3	encolar	P3
4	desencolar	
5	encolar	P4
6	desencolar	
7	desencolar	
8	encolar	P5
9	desencolar	
10	desencolar	
11	desencolar	

Tu programa debe

1. Usar una pila de entrada (donde se encolan los pedidos) y una de salida (de donde se desencolan).
2. Al desencolar, si la pila de salida está vacía, pasar todos los elementos de la de entrada a la de salida (eso invierte el orden).
3. Contar cada operación de apilar o desapilar como un movimiento.
4. Mostrar el pedido que sale en cada desencolar, el orden de salida y los movimientos por operación.

Salida esperada

```
desencolar -> P1
desencolar -> P2
desencolar -> P3
desencolar -> P4
desencolar -> P5
desencolar -> cola vacía
Orden de salida: P1, P2, P3, P4, P5
Movimientos entre pilas: 15 para 11 operaciones (1.36 por operación)
```

Reglas

- Límites: hasta 10^7 operaciones.
- Complejidad esperada: $O(1)$ amortizado por operación (cada elemento se mueve como máximo 3 veces en total: entra, pasa y sale).
- Desencolar de una cola vacía informa "cola vacía" y no rompe el programa.
- Movimientos por operación con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): no pases elementos a la pila de salida mientras todavía tenga algo: si lo hacés, rompés el orden.

Bonus (opcional)

- Implementá lo inverso: una pila usando dos colas.
- Agregá la operación "ver el mínimo" en $O(1)$ amortizado (pista: cada pila guarda también el mínimo hasta ese punto).

Cierre: Es una pregunta clásica de entrevistas y una lección central de análisis amortizado: algunas operaciones son caras, pero el promedio es constante.

Ejercicio 186: Pulsos del marcapasos

Nivel 4: Semi-Senior I · Eficiencia · Temática: Salud · Dificultad 37/100

Situación: Un equipo de ingeniería biomédica prueba el software de un marcapasos a demanda: si el corazón no late por su cuenta dentro de un tiempo máximo, el dispositivo manda un estímulo. Además, justo después de cada latido hay un período refractario en el que cualquier señal se ignora (suele ser ruido). Simulan una secuencia de señales registradas. (Parámetros ilustrativos.)

Datos de entrada

Señales detectadas (ms): 0, 820, 1650, 1790, 2900, 3150, 3400, 5200, 5480

- Intervalo máximo sin latido: 1.000 ms (si pasa ese tiempo desde el último latido, propio o estimulado, se estimula).
- Período refractario: una señal a menos de 250 ms del último latido se ignora.
- La simulación termina en el instante de la última señal.

Tu programa debe

- Mantener el instante del último latido válido (propio o estimulado).
- Para cada señal: si cae en el período refractario, ignorarla; si llega a tiempo (a 1.000 ms o menos del último latido), es un latido propio.
- Si la próxima señal llega más tarde que último latido + 1.000 ms (o no hay más señales antes del final), estimular en ese instante y seguir desde ahí.
- Mostrar la secuencia de eventos y los totales.

Salida esperada

```
0 ms: latido propio
820 ms: latido propio
1650 ms: latido propio
1790 ms: ignorado (período refractario)
2650 ms: ESTÍMULO del marcapasos
2900 ms: latido propio
3150 ms: latido propio
3400 ms: latido propio
4400 ms: ESTÍMULO del marcapasos
5200 ms: latido propio
5480 ms: latido propio
Estímulos: 2 | Latidos ignorados: 1
```

Reglas

- Límites: hasta 10^6 señales; tiempos hasta 10^9 ms.
- Complejidad esperada: $O(n + \text{cantidad de estímulos})$.
- Una señal exactamente a 1.000 ms del último latido llega a tiempo; una a exactamente 250 ms ya no está en el período refractario.
- Un estímulo no se dispara después del final de la simulación.

Pista (leela solo si te trabás): es una máquina de estados muy chica: el único estado es "cuándo fue el último latido". En cada paso comparás la próxima señal con ese instante + 250 y + 1.000.

Bonus (opcional)

- Agregá un modo de frecuencia adaptativa: el intervalo máximo baja a 750 ms si el paciente está haciendo ejercicio (según otra lista de tiempos).
- Detectá y reportá arritmias: tres latidos propios seguidos con intervalos que varían más del 30 %.

Cierre: Es una versión didáctica de la lógica de los marcapasos a demanda, donde la simulación y la verificación del software son críticas.

Ejercicio 187: Llamadas simultáneas en la central

Nivel 4: Semi-Senior I · Eficiencia · Temática: Telecomunicaciones · Dificultad 38/100

Situación: Un call center quiere contratar la cantidad justa de líneas telefónicas. Tiene el registro de inicio y fin de cada llamada de la hora pico y necesita saber cuál fue el máximo de llamadas en curso al mismo tiempo. Con millones de llamadas por mes, comparar cada par de llamadas no es opción.

Datos de entrada

Llamada	Inicio	Fin
1	09:00	09:12
2	09:05	09:07
3	09:06	09:20
4	09:07	09:15
5	09:10	09:11
6	09:12	09:30
7	09:14	09:16
8	09:25	09:40

Una llamada ocupa la línea desde su inicio hasta antes de su fin: si una termina a las 09:07 y otra empieza a las 09:07, pueden usar la misma línea.

Tu programa debe

1. Crear un evento +1 por cada inicio y un evento -1 por cada fin.
2. Ordenar los eventos por hora; a igual hora, los fines antes que los inicios.
3. Recorrerlos acumulando las llamadas en curso y registrar el máximo (y la primera hora en que se alcanza).
4. Mostrar el máximo y las líneas necesarias.

Salida esperada

```
Máximo de llamadas simultáneas: 4 (a las 09:10)
Líneas necesarias: 4
```

Reglas

- Límites: hasta 10^6 llamadas.
- Complejidad esperada: $O(n \log n)$ por el ordenamiento; el barrido es $O(n)$.
- El orden "fines antes que inicios" a la misma hora es lo que implementa la regla de reutilizar la línea.
- Casos borde: todas las llamadas simultáneas; ninguna superpuesta.

Pista (leela solo si te trabás): no importa qué llamada es cuál: solo importa cuántas empiezan y terminan en cada instante. Eso convierte un problema de intervalos en un recorrido de eventos.

Bonus (opcional)

- Calculá cuántos minutos en total hubo 3 o más llamadas simultáneas.
- Asigná un número de línea concreto a cada llamada, reutilizando las que se liberan (vas a necesitar un heap de líneas libres).

Cierre: Es la técnica de barrido de eventos, usada para dimensionar líneas telefónicas, servidores, salas de reuniones y estacionamientos.

Ejercicio 188: Dosis máxima en cualquier ventana de 24 horas

Nivel 4: Semi-Senior I · Eficiencia · Temática: Salud · Dificultad 38/100

Situación: Una app de salud registra cada toma de un analgésico que la persona se autoadministra. El prospecto indica un máximo diario, que en realidad aplica a cualquier período de 24 horas corridas, no al día calendario. La app tiene que avisar en el momento en que una toma hace superar ese máximo. (Valores ficticios.)

Datos de entrada

Máximo: 3.000 mg en cualquier período de 24 horas.

Hora (desde el inicio del registro)	Dosis (mg)
0	800
6	600
10	800
16	500
22	400
26	900
30	700
36	600
40	500
46	800
52	200

Tu programa debe

1. Recorrer las tomas en orden con dos punteros que delimitan las tomas de las últimas 24 horas.
2. Al agregar una toma, sacar las que tienen hora $\leq$ hora actual $- 24$.
3. Si la suma de la ventana supera el máximo, avisar en esa toma.
4. Informar la mayor suma en cualquier ventana y qué tomas la forman.

Salida esperada

```
Toma de la hora 22: 3100 mg en las últimas 24 h -> SUPERA el máximo
Toma de la hora 26: 3200 mg en las últimas 24 h -> SUPERA el máximo
Toma de la hora 30: 3300 mg en las últimas 24 h -> SUPERA el máximo
Toma de la hora 36: 3100 mg en las últimas 24 h -> SUPERA el máximo
Toma de la hora 40: 3100 mg en las últimas 24 h -> SUPERA el máximo
Toma de la hora 46: 3500 mg en las últimas 24 h -> SUPERA el máximo
Máximo en cualquier ventana de 24 h: 3500 mg (tomas de la hora 26 a la 46)
```

Reglas

- Límites: hasta 10^6 tomas.
- Complejidad esperada: $O(n)$.
- La ventana de una toma en la hora h incluye las tomas en horas mayores que $h - 24$ y hasta h .
- Si ninguna toma supera el máximo, se informa "Ninguna toma supera el máximo" al principio.

Pista (leela solo si te trabás): es la misma ventana deslizante de tamaño fijo que ya usaste, pero sobre tiempo: la cantidad de tomas dentro de la ventana varía.

Bonus (opcional)

- Calculá, después de cada toma, cuál es la dosis máxima que se podría tomar ahora sin superar el límite.

- Agregá un segundo límite: no más de 1.000 mg en 4 horas.

Cierre: Es el mismo control de ventana móvil que usan los sistemas de prescripción y de dispensación automática de medicamentos.

Ejercicio 189: El dólar más caro de cada semana

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 38/100

Situación: Un portal financiero muestra, para cada día, el valor más alto que tocó el dólar en los últimos 5 días hábiles. Con series de muchos años y varias monedas que se actualizan todo el tiempo, recalculer el máximo de cada ventana desde cero es un desperdicio.

Datos de entrada

Cotización de cierre por día hábil (días 1 a 15): 1210, 1225, 1219, 1240, 1236, 1231, 1228, 1255, 1249, 1262, 1258, 1244, 1239, 1270, 1266

Ventana: 5 días hábiles.

Tu programa debe

1. Recorrer los días manteniendo una cola doble de índices con cotizaciones decrecientes.
2. Al entrar un día, sacar del final los índices con cotización menor o igual; al salir de la ventana, sacar del frente.
3. El frente es el máximo de la ventana actual.
4. Mostrar, para cada ventana completa, el rango de días, el máximo y en qué día se dio (si se repite, el último día con ese valor).

Salida esperada

```
Días 1- 5: máximo $1240 (día 4)
Días 2- 6: máximo $1240 (día 4)
Días 3- 7: máximo $1240 (día 4)
Días 4- 8: máximo $1255 (día 8)
Días 5- 9: máximo $1255 (día 8)
Días 6-10: máximo $1262 (día 10)
Días 7-11: máximo $1262 (día 10)
Días 8-12: máximo $1262 (día 10)
Días 9-13: máximo $1262 (día 10)
Días 10-14: máximo $1270 (día 14)
Días 11-15: máximo $1270 (día 14)
```

Reglas

- Límites: hasta 10^6 días; ventana entre 1 y n.
- Complejidad esperada: $O(n)$.
- Al sacar del final los valores menores o iguales, ante empates queda el día más reciente.
- Formato: días del rango en 2 caracteres a la derecha.

Pista (leela solo si te trabás): es la cola doble monótona que usaste para el mínimo de stock, con la desigualdad dada vuelta.

Bonus (opcional)

- Calculá, para cada día, la diferencia entre el máximo y el mínimo de la ventana (volatilidad del rango).
- Marcá los días en que la cotización fue un "máximo de 5 días" (el día es el máximo de la ventana que termina en él).

Cierre: Es el cálculo de máximos móviles que usan los indicadores técnicos de trading y los paneles de monitoreo.

Ejercicio 190: El mazo de la escoba

Nivel 4: Semi-Senior I · Eficiencia · Temática: Videojuegos · Dificultad 38/100

Situación: Un estudio de videojuegos está haciendo una versión digital simplificada de la escoba de 15 para jugar contra la compu. Antes de programar la inteligencia de los jugadores, quieren un simulador con reglas fijas y un mazo en orden conocido, para poder testear el reparto y el conteo.

Datos de entrada

Mazo simplificado de 24 cartas con valores del 1 al 10, en el orden en que se reparten:

```
7 3 5 10 2 8 4 6 1 9 5 7 3 6 2 10 8 4 1 9 6 3 7 5
```

Reglas simplificadas:

1. Se ponen las 4 primeras cartas del mazo en la mesa.
2. Cuando los dos jugadores se quedan sin cartas, se reparten 3 a cada uno, alternando (una a J1, una a J2...).
3. Juegan por turno, empezando J1 y siempre con la primera carta de su mano.
4. Si la carta jugada más alguna carta de la mesa suman 15, el jugador levanta las dos (se elige la primera carta de la mesa que cumple, en orden). Si la mesa queda vacía, es escoba.
5. Si no, la carta queda en la mesa (al final).

Tu programa debe

1. Representar el mazo y las manos como colas, y la mesa como una lista.
2. Simular repartos y turnos según las reglas hasta que no queden cartas en el mazo ni en las manos.
3. Registrar cada jugada, las cartas levantadas y las escobas.
4. Mostrar la partida completa y el resumen final.

Salida esperada

```
Mesa inicial: 7 3 5 10
Reparto 1: J1 2 4 1 | J2 8 6 9
  J1 juega 2 a la mesa
  J2 juega 8 y levanta 7
  J1 juega 4 a la mesa
  J2 juega 6 a la mesa
  J1 juega 1 a la mesa
  J2 juega 9 y levanta 6
Reparto 2: J1 5 3 2 | J2 7 6 10
  J1 juega 5 y levanta 10
  J2 juega 7 a la mesa
  J1 juega 3 a la mesa
  J2 juega 6 a la mesa
  J1 juega 2 a la mesa
  J2 juega 10 y levanta 5
Reparto 3: J1 8 1 6 | J2 4 9 3
  J1 juega 8 y levanta 7
  J2 juega 4 a la mesa
  J1 juega 1 a la mesa
  J2 juega 9 y levanta 6
  J1 juega 6 a la mesa
  J2 juega 3 a la mesa
Reparto 4: J1 7 | J2 5
  J1 juega 7 a la mesa
  J2 juega 5 a la mesa
Cartas levantadas: J1 4, J2 8 | Escobas: J1 0, J2 0 | Quedan en la mesa: 12
```

Reglas

- Límites: mazo de hasta 10^5 cartas (para probar rendimiento del simulador).

- Complejidad esperada: $O(n \cdot m)$ con m el tamaño de la mesa (buscar el par en la mesa es lineal).
- Cada vez que un jugador levanta, suma 2 cartas a su pila.
- Formato: cada jugada con dos espacios de sangría.

Pista (leela solo si te trabás): separá la simulación en funciones chicas: repartir, jugar_carta, buscar_par. Con un mazo en orden fijo, la salida tiene que ser siempre la misma: eso es lo que permite testear.

Bonus (opcional)

- Implementá la regla completa: la carta jugada puede sumar 15 con cualquier combinación de cartas de la mesa (vas a necesitar backtracking, nivel 5).
- Hacé que el jugador elija la carta de su mano que levanta más cartas, en lugar de jugar siempre la primera.

Cierre: Es el mismo tipo de simulador determinista con colas que usan los estudios de juegos para testear reglas antes de agregar gráficos e inteligencia.

Ejercicio 191: Tres productos para el presupuesto exacto

Nivel 4: Semi-Senior I · Eficiencia · Temática: E-commerce · Dificultad 38/100

Situación: Una tienda de regalos online arma "cajas de \$10.000" con exactamente tres productos distintos. El catálogo ya viene ordenado por precio y tiene miles de productos, así que probar todos los tríos (del orden de n^3) es inviable. Te piden hacerlo en tiempo cuadrático.

Datos de entrada

Precios (ordenados, todos distintos): \$1.200, \$1.800, \$2.500, \$3.100, \$3.600, \$4.200, \$4.900, \$5.500, \$6.300, \$7.000

Presupuesto: \$10.000

Tu programa debe

1. Fijar el producto más barato del trío recorriendo la lista.
2. Para el resto, buscar pares que completen el presupuesto con dos punteros (como en el ejercicio de los dos productos).
3. Registrar todos los tríos exactos.
4. Mostrar la cantidad y los tríos ordenados de menor a mayor por su primer precio, luego por el segundo.

Salida esperada

```
Tríos que suman exactamente $10000: 2
$1200 + $1800 + $7000
$1200 + $2500 + $6300
```

Reglas

- Límites: hasta 5.000 productos; precios hasta 10^7 .
- Complejidad esperada: $O(n^2)$. La fuerza bruta $O(n^3)$ no pasa con 5.000 productos (unos $2 \cdot 10^{10}$ tríos).
- Un producto no se repite dentro del trío.
- Casos borde: ningún trío; menos de 3 productos.

Pista (leela solo si te trabás): reducir un problema de tres a uno de dos fijando un elemento es un patrón muy común: el "3-sum" es un "2-sum" dentro de un bucle.

Bonus (opcional)

- Encontrá el trío con suma más cercana al presupuesto sin pasarse.
- Generalizá a k productos: ¿qué complejidad obtenés?

Cierre: Es el problema 3-sum, un clásico de entrevistas y de teoría de la complejidad.

Ejercicio 192: Radar de velocidad en la ruta

Nivel 4: Semi-Senior I · Eficiencia · Temática: Transporte · Dificultad 39/100

Situación: Una autopista controla la velocidad por tramos: varias cámaras registran la hora en que pasa cada patente, y la velocidad promedio entre dos cámaras decide la multa. Los inspectores también consultan la velocidad promedio entre cámaras no consecutivas.

Datos de entrada

Cámara	Kilómetro	Hora de paso de la patente AB 123 CD
1	0	10:00:00
2	12	10:06:00
3	27	10:13:30
4	41	10:20:00
5	58	10:28:20
6	70	10:34:30

Velocidad máxima: 110 km/h. Consultas: cámaras 1 a 6, 2 a 4 y 4 a 5.

Tu programa debe

- Convertir las horas a segundos.
- Calcular la velocidad promedio de cada tramo entre cámaras consecutivas y marcar MULTA si supera 110 km/h.
- Responder cada consulta entre dos cámaras cualesquiera usando la distancia y el tiempo acumulados (sin recorrer los tramos intermedios).
- Mostrar los tramos y las consultas.

Salida esperada

```
Tramo cámara 1-2: 12 km en 360 s -> 120.0 km/h MULTA
Tramo cámara 2-3: 15 km en 450 s -> 120.0 km/h MULTA
Tramo cámara 3-4: 14 km en 390 s -> 129.2 km/h MULTA
Tramo cámara 4-5: 17 km en 500 s -> 122.4 km/h MULTA
Tramo cámara 5-6: 12 km en 370 s -> 116.8 km/h MULTA
Promedio entre cámaras 1 y 6: 121.7 km/h
Promedio entre cámaras 2 y 4: 124.3 km/h
Promedio entre cámaras 4 y 5: 122.4 km/h
```

Reglas

- Límites: hasta 10^5 cámaras por ruta y 10^6 consultas.
- Complejidad esperada: $O(1)$ por consulta (los kilómetros y las horas ya son acumulados: son sumas prefijas naturales).
- Velocidades con 1 decimal (0,5 hacia arriba). La multa se decide con el valor sin redondear. Los decimales se muestran con punto y sin separador de miles en la salida.
- Suponé que la patente pasa por todas las cámaras en orden y el mismo día.

Pista (leela solo si te trabás): la velocidad promedio entre dos cámaras es distancia total / tiempo total, no el promedio de las velocidades de cada tramo (dan distinto si los tramos tienen distinto largo).

Bonus (opcional)

- Encontrá el par de cámaras (no necesariamente consecutivas) con mayor velocidad promedio.
- Procesá miles de patentes con sus horas de paso y listá todas las que merecen multa.

Cierre: Es el sistema de control de velocidad por tramo que usan muchas autopistas, más justo que un radar puntual.

Ejercicio 193: ¿Está la melodía escondida en la canción?

Nivel 4: Semi-Senior I · Eficiencia · Temática: Música · Dificultad 39/100

Situación: Un sistema de detección de plagio musical busca si un motivo (una secuencia corta de notas) aparece "escondido" en una canción: las notas en el mismo orden, aunque haya otras notas en el medio. Con catálogos de millones de canciones, cada verificación tiene que ser lineal.

Datos de entrada

Canción (notas en orden): Do Mi Sol Mi Re Fa La Sol Mi Do Re Mi Fa Sol La Si Do

- Motivo 1: Do Mi Re Sol
- Motivo 2: Mi Fa Sol La Si
- Motivo 3: Sol Fa Mi Re Do
- Motivo 4: Do Do Do

Tu programa debe

1. Recorrer la canción con un índice y el motivo con otro.
2. Cada vez que la nota de la canción coincide con la nota esperada del motivo, avanzar en el motivo y registrar la posición.
3. El motivo está escondido si se recorrió completo.
4. Mostrar, para cada motivo, las posiciones (desde 1) o cuántas notas se encontraron.

Salida esperada

```
Motivo 1 (Do Mi Re Sol): está, en las posiciones 1, 2, 5, 8
Motivo 2 (Mi Fa Sol La Si): está, en las posiciones 2, 6, 8, 15, 16
Motivo 3 (Sol Fa Mi Re Do): está, en las posiciones 3, 6, 9, 11, 17
Motivo 4 (Do Do Do): está, en las posiciones 1, 10, 17
```

Reglas

- Límites: canciones de hasta 10^6 notas; motivos de hasta 10^3 .
- Complejidad esperada: $O(\text{largo de la canción})$ por motivo.
- Se toma siempre la primera coincidencia posible (estrategia voraz): se puede demostrar que si el motivo está, esta estrategia lo encuentra.
- Casos borde: motivo vacío (está siempre), motivo más largo que la canción.

Pista (leela solo si te trabás): no hace falta volver atrás nunca: si una nota coincide, tomarla nunca es peor que esperar a una coincidencia posterior.

Bonus (opcional)

- Contá de cuántas formas distintas aparece el motivo como subsecuencia (vas a necesitar programación dinámica, nivel 6).
- Hacé la búsqueda independiente de la tonalidad: compará intervalos entre notas en lugar de notas.

Cierre: Es el chequeo de subsecuencia, usado en detección de plagio, comparación de secuencias genéticas y búsqueda difusa.

Ejercicio 194: Baterías de la red domiciliaria

Nivel 4: Semi-Senior I · Eficiencia · Temática: Clima y energía · Dificultad 39/100

Situación: Una casa con paneles solares y una batería de 10 kWh quiere minimizar lo que le compra a la red. Durante el día, la energía sobrante carga la batería y, cuando está llena, se vende a la red; de noche, la batería cubre el consumo y lo que falta se compra. Tu programa simula un día hora por hora.

Datos de entrada

Batería: capacidad 10 kWh, carga inicial 4 kWh (a las 06:00).

Hora	Generación solar (kWh)	Consumo (kWh)
06:00	0	0,6
07:00	0	0,5
08:00	0,5	0,9
09:00	1,8	1,1
10:00	3,2	1,0
11:00	4,1	0,8
12:00	4,4	1,2
13:00	3,9	1,5
14:00	2,7	1,4
15:00	1,2	2,3
16:00	0,2	2,8
17:00	0	1,9

Tu programa debe

1. Calcular el neto de cada hora (generación – consumo).
2. Si el neto es positivo, cargar la batería hasta su capacidad y vender el excedente.
3. Si es negativo, descargar la batería hasta vaciarla y comprar lo que falte.
4. Mostrar el estado de cada hora y los totales comprados, vendidos y la batería final.

Salida esperada

```
06:00 sol 0.0 cons 0.6 -> descarga 0.6 | batería 3.4
07:00 sol 0.0 cons 0.5 -> descarga 0.5 | batería 2.9
08:00 sol 0.5 cons 0.9 -> descarga 0.4 | batería 2.5
09:00 sol 1.8 cons 1.1 -> carga 0.7 | batería 3.2
10:00 sol 3.2 cons 1.0 -> carga 2.2 | batería 5.4
11:00 sol 4.1 cons 0.8 -> carga 3.3 | batería 8.7
12:00 sol 4.4 cons 1.2 -> carga 1.3, vende 1.9 a la red | batería 10.0
13:00 sol 3.9 cons 1.5 -> carga 0.0, vende 2.4 a la red | batería 10.0
14:00 sol 2.7 cons 1.4 -> carga 0.0, vende 1.3 a la red | batería 10.0
15:00 sol 1.2 cons 2.3 -> descarga 1.1 | batería 8.9
16:00 sol 0.2 cons 2.8 -> descarga 2.6 | batería 6.3
17:00 sol 0.0 cons 1.9 -> descarga 1.9 | batería 4.4
Comprado a la red: 0.0 kWh | Vendido a la red: 5.6 kWh | Batería final: 4.4 kWh
```

Reglas

- Límites: simulaciones de hasta 10^6 horas (un siglo de datos horarios).
- Complejidad esperada: $O(n)$.
- Todos los valores en kWh con 1 decimal (0,5 hacia arriba). Usá decimales exactos para que la batería no acumule errores. Los decimales se muestran con punto y sin separador de miles en la salida.
- Se ignoran las pérdidas de carga y descarga (ver Bonus).

Pista (leela solo si te trabás): el estado de la simulación es un solo número (la carga de la batería); cada hora lo actualizás con dos mínimos: cuánto entra y cuánto sale.

Bonus (opcional)

- Agregá una eficiencia del 90 %: solo el 90 % de lo que se carga queda disponible para descargar.
- Con tarifa horaria (cara de 18 a 23), decidí si conviene reservar batería para la hora pico.

Cierre: Es la lógica básica de los sistemas de gestión de energía de hogares con paneles solares y baterías.

Ejercicio 195: Buscar un precio en la grilla ordenada

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 39/100

Situación: Una aseguradora tiene una tabla de primas mensuales: cada fila es una franja de edad y cada columna un nivel de cobertura, y la tabla crece hacia la derecha y hacia abajo. Los vendedores buscan si existe un plan con una prima exacta que el cliente puede pagar. Con tablas de miles de filas y columnas, hay que buscar sin recorrer todo.

Datos de entrada

Primas (cada fila y cada columna están ordenadas de menor a mayor):

Franja	Cob. 1	Cob. 2	Cob. 3	Cob. 4	Cob. 5
1	1200	1350	1500	1720	1980
2	1300	1480	1650	1800	2100
3	1420	1600	1790	1950	2250
4	1510	1700	1880	2090	2400

Primas a buscar: \$1.790, \$1.500, \$2.000, \$1.100, \$2.400

Tu programa debe

1. Empezar en la esquina superior derecha.
2. Si el valor es mayor que el buscado, moverse una columna a la izquierda; si es menor, una fila hacia abajo; si es igual, encontrado.
3. Detenerse al salir de la matriz.
4. Mostrar la posición (desde 1) o "no está", con la cantidad de pasos.

Salida esperada

```
$1790: fila 3, columna 3 (5 pasos)
$1500: fila 1, columna 3 (3 pasos)
$2000: no está (6 pasos)
$1100: no está (5 pasos)
$2400: fila 4, columna 5 (4 pasos)
```

Reglas

- Límites: matrices de hasta 5.000 × 5.000.
- Complejidad esperada: O(filas + columnas) por búsqueda. Recorrer toda la matriz es O(filas × columnas).
- Un paso es cada celda que se examina.
- Casos borde: valor menor que todos, mayor que todos, en una esquina.

Pista (leela solo si te trabás): desde la esquina superior derecha, cada comparación descarta una fila entera (todo lo de la izquierda es menor) o una columna entera (todo lo de abajo es mayor).

Bonus (opcional)

- Contá cuántas primas de la tabla son menores o iguales a un presupuesto, también en O(filas + columnas).
- Probá empezar desde la esquina inferior izquierda: ¿funciona igual? ¿Y desde la superior izquierda?

Cierre: Es la búsqueda en escalera sobre matrices ordenadas, un clásico de entrevistas y de indexación de tablas.

Ejercicio 196: El catálogo ordenado que alguien rotó

Nivel 4: Semi-Senior I · Eficiencia · Temática: E-commerce · Dificultad 39/100

Situación: Un e-commerce guarda su catálogo ordenado por precio en un arreglo, pero un proceso nocturno con un bug "rotó" la lista: la movió de manera que el más barato ya no está al principio. Mientras arreglan el bug, necesitan seguir buscando productos por precio en tiempo logarítmico.

Datos de entrada

Catálogo rotado (era ascendente y se rotó una cantidad desconocida de posiciones; todos los precios son distintos): 4300, 4850, 5200, 6100, 7400, 890, 1250, 1900, 2600, 3350, 3900

Precios a buscar: \$1.900, \$6.100, \$4.300, \$5.000, \$890

Tu programa debe

1. Encontrar con búsqueda binaria la posición del mínimo (el punto de rotación), comparando el medio con el último elemento del rango.
2. Para cada precio, decidir en cuál de las dos mitades ordenadas puede estar.
3. Hacer una búsqueda binaria común en esa mitad, contando las comparaciones.
4. Mostrar el punto de rotación y el resultado de cada búsqueda.

Salida esperada

```
Punto de rotación: posición 6 (el más barato, $890)
$1900: posición 8 (1 comparación)
$6100: posición 4 (2 comparaciones)
$4300: posición 1 (2 comparaciones)
$5000: no está (3 comparaciones)
$890: posición 6 (2 comparaciones)
```

Reglas

- Límites: hasta 10^7 productos; 10^6 búsquedas.
- Complejidad esperada: $O(\log n)$ para encontrar la rotación y $O(\log n)$ por búsqueda.
- Búsqueda del mínimo: inicio = 0, fin = $n - 1$; mientras inicio < fin, medio = (inicio + fin) / 2; si $a[\text{medio}] > a[\text{fin}]$, inicio = medio + 1; si no, fin = medio.
- Búsqueda binaria común: inicio $\leq$ fin, medio = (inicio + fin) / 2, una comparación por iteración.
- Casos borde: lista sin rotar; rotación de una posición.

Pista (leela solo si te trabás): si el elemento del medio es mayor que el último, el "salto" (donde está el mínimo) quedó a la derecha del medio; si no, está en el medio o a su izquierda.

Bonus (opcional)

- Resolvé la búsqueda en una sola pasada binaria, sin encontrar primero el punto de rotación.
- ¿Qué pasa si puede haber precios repetidos? Mostrá un caso donde la búsqueda binaria deja de funcionar en el peor caso.

Cierre: Es un clásico de entrevistas que pone a prueba si entendés por qué funciona la búsqueda binaria, no solo cómo se escribe.

Ejercicio 197: Calendario de guardias rotativas

Nivel 4: Semi-Senior I · Eficiencia · Temática: Recursos humanos · Dificultad 39/100

Situación: La guardia nocturna de un hospital se reparte en forma rotativa entre cinco médicos. Cuando alguien no puede un día (licencia, congreso), toma la guardia el siguiente en la rotación, pero quien no pudo conserva su lugar para no quedar "en deuda". Recursos Humanos quiere el cronograma de los próximos 10 días.

Datos de entrada

Orden inicial de la rotación: Paz, Ruiz, Soto, Vega, Lima

Día	No pueden
2	Paz
3	Ruiz, Soto
5	Vega
6	Lima, Paz
8	Soto

Tu programa debe

1. Representar la rotación con una cola.
2. Cada día, elegir al primero de la cola que pueda ese día.
3. Mover al elegido al final de la cola; los que no pudieron mantienen su posición.
4. Mostrar el cronograma y cuántas guardias hizo cada médico.

Salida esperada

```
Día 1: Paz
Día 2: Ruiz (no pueden: Paz)
Día 3: Vega (no pueden: Ruiz, Soto)
Día 4: Soto
Día 5: Lima (no pueden: Vega)
Día 6: Ruiz (no pueden: Lima, Paz)
Día 7: Paz
Día 8: Vega (no pueden: Soto)
Día 9: Soto
Día 10: Lima
Guardias por médico: Paz 2, Ruiz 2, Soto 2, Vega 2, Lima 2
```

Reglas

- Límites: hasta 1.000 médicos y 10^5 días; hasta 10^6 restricciones.
- Complejidad esperada: $O(\text{días} \times \text{médicos})$ en el peor caso; guardá las restricciones en un mapa día $\rightarrow$ conjunto para consultarlas en $O(1)$.
- Si nadie puede, el día queda "SIN GUARDIA" y la cola no cambia.
- Las restricciones se muestran en orden alfabético.

Pista (leela solo si te trabás): sacar a alguien del medio de la cola y ponerlo al final es lo que mantiene el "lugar" de quienes no pudieron: quedan adelante para el día siguiente.

Bonus (opcional)

- Agregá la regla de descanso: nadie hace dos guardias en días consecutivos.
- Balanceá la carga a fin de mes: si alguien quedó con menos guardias, que tenga prioridad en la rotación siguiente.

Cierre: Es el mismo esquema de rotación con excepciones que usan los sistemas de turnos de hospitales, soporte técnico y guardias de sistemas.

Ejercicio 198: Desafío de nivel: La caja de ahorro con débitos automáticos

Nivel 4: Semi-Senior I · Eficiencia · Temática: Finanzas y FinTech · Dificultad 40/100

Situación: Una fintech ofrece una caja de ahorro con débitos automáticos y quiere mostrarle al cliente, antes de que pase, qué va a ocurrir en el mes: qué débitos se rechazan por falta de saldo, cuántos cargos por rechazo va a pagar y cuándo se terminan cobrando los reintentos. Es el desafío que integra el nivel 4.

Datos de entrada

Saldo al comenzar el día 1: \$35.000. Mes de 30 días.

Ingresos: día 5 \$480.000 (sueldo), día 12 \$250.000 (cobro de un trabajo extra), día 20 \$60.000 (reintegro).

Día de vencimiento	Débito	Monto
2	Seguro del auto	\$42.000
3	Gimnasio	\$18.000
4	Tarjeta de crédito	\$390.000
6	Internet	\$22.000
10	Préstamo	\$95.000
12	Streaming	\$9.000
15	Colegio	\$120.000
25	Obra social	\$85.000

Reglas:

1. Cada día se acreditan primero los ingresos.
2. Después se procesa una cola de débitos: los que vencen ese día entran al final de la cola; se intenta cobrar cada débito de la cola una vez, en orden.
3. Si el saldo alcanza, se cobra. Si no, se cobra un cargo de \$1.500 por rechazo (aunque el saldo quede negativo) y el débito vuelve al final de la cola para reintentarse al día siguiente. Al tercer rechazo de un mismo débito, se da de baja y no se reintenta más.
4. Al final del mes, se muestra el movimiento neto de algunos rangos de días: 1 a 10, 11 a 20 y 1 a 30.

Tu programa debe

1. Simular el mes día por día con la cola de débitos pendientes.
2. Registrar cada ingreso, débito cobrado (indicando si es un reintento), rechazo y cargo.
3. Guardar el movimiento neto de cada día y construir las sumas prefijas.
4. Mostrar el log, los pendientes a fin de mes, los cargos por rechazo y los movimientos netos por rango.

Salida esperada

```
Día 2: RECHAZADO Seguro del auto (42000), cargo por rechazo -1500 -> saldo 33500
Día 3: RECHAZADO Seguro del auto (42000), cargo por rechazo -1500 -> saldo 32000
Día 3: débito Gimnasio -18000 -> saldo 14000
Día 4: RECHAZADO Seguro del auto (42000), cargo por rechazo -1500 -> saldo 12500 | DADO DE BAJA
Día 4: RECHAZADO Tarjeta de crédito (390000), cargo por rechazo -1500 -> saldo 11000
Día 5: ingreso +480000 -> saldo 491000
Día 5: débito Tarjeta de crédito -390000 (reintento, venció el día 4) -> saldo 101000
Día 6: débito Internet -22000 -> saldo 79000
Día 10: RECHAZADO Préstamo (95000), cargo por rechazo -1500 -> saldo 77500
Día 11: RECHAZADO Préstamo (95000), cargo por rechazo -1500 -> saldo 76000
Día 12: ingreso +250000 -> saldo 326000
Día 12: débito Préstamo -95000 (reintento, venció el día 10) -> saldo 231000
```

```
Día 12: débito Streaming -9000 -> saldo 222000
Día 15: débito Colegio -120000 -> saldo 102000
Día 20: ingreso +60000 -> saldo 162000
Día 25: débito Obra social -85000 -> saldo 77000
Pendientes a fin de mes: ninguno | Datos de baja: Seguro del auto
Cargos por rechazo: 6 ($9000)
Movimiento neto días 1 a 10: +42500
Movimiento neto días 11 a 20: +84500
Movimiento neto días 1 a 30: +42000
```

Reglas

- Límites: hasta 10^5 débitos y simulaciones de hasta 3.650 días; consultas por rango en $O(1)$.
- Complejidad esperada: $O(\text{días} + \text{intentos de débito})$ para la simulación y $O(1)$ por consulta de rango.
- En cada día, cada débito de la cola se intenta como máximo una vez (los que se reencolan esperan al día siguiente).
- Los montos son enteros. Formato: día en 2 caracteres a la derecha.

Pista (leela solo si te trabás): antes de procesar la cola de un día, guardá cuántos elementos tiene: esa es la cantidad de intentos del día. Los que se reencolan quedan detrás y no se vuelven a intentar ese mismo día.

Bonus (opcional)

- Agregá prioridades: la tarjeta y el préstamo se cobran antes que el resto, aunque hayan vencido después.
- Sugerí al cliente el menor monto a depositar el día 1 para que no haya ningún rechazo en el mes (búsqueda binaria sobre la respuesta).

Cierre: Es la simulación que corren los bancos y las fintech para proyectar saldos y avisar a los clientes antes de un rechazo.

Ejercicio 199: Desafío de nivel: Monitor de tráfico de la autopista

Nivel 4: Semi-Senior I · Eficiencia · Temática: Transporte · Dificultad 40/100

Situación: El centro de monitoreo de una autopista urbana cuenta los autos que pasan por minuto en la hora pico y registra los incidentes. El panel necesita tres cosas en tiempo real: el tramo de 5 minutos más cargado, cuándo fue la última vez que hubo tanto tráfico como en cada pico, y cuántos incidentes hubo entre dos horarios. Es el desafío de técnicas del nivel 4.

Datos de entrada

Autos por minuto, de 07:00 a 07:19: 42, 45, 51, 60, 58, 72, 80, 77, 69, 90, 95, 88, 70, 65, 71, 84, 99, 102, 97, 81

Incidentes (ordenados): 07:03, 07:07, 07:08, 07:12, 07:15, 07:18

Consultas de incidentes: [07:05, 07:12], [07:00, 07:02], [07:15, 07:19]

Un pico es un minuto con 90 autos o más.

Tu programa debe

1. Encontrar la ventana de 5 minutos con más autos (ventana deslizante; en empate, la primera).
2. Para cada pico, encontrar el minuto anterior más cercano con estrictamente más autos, usando una pila monótona.
3. Responder las consultas de incidentes con búsqueda binaria de límites inferior y superior.
4. Mostrar los tres resultados.

Salida esperada

```
Ventana de 5 minutos más cargada: 07:15 a 07:19 (463 autos)
07:09 (90 autos): récord de la mañana
07:10 (95 autos): récord de la mañana
07:16 (99 autos): récord de la mañana
07:17 (102 autos): récord de la mañana
07:18 (97 autos): no había tanto tráfico desde las 07:17 (102 autos)
Incidentes entre 07:05 y 07:12: 3
Incidentes entre 07:00 y 07:02: 0
Incidentes entre 07:15 y 07:19: 2
```

Reglas

- Límites: hasta 10^6 minutos de datos, 10^6 incidentes y 10^6 consultas.
- Complejidad esperada: $O(n)$ para la ventana y la pila; $O(\log m)$ por consulta de incidentes.
- Si no hay un minuto anterior con más autos, es "récord de la mañana".
- Las consultas incluyen ambos extremos.

Pista (leela solo si te trabás): son tres técnicas del nivel en un mismo problema: resolvé cada parte como una función independiente y probala por separado antes de juntarlas.

Bonus (opcional)

- Detectá "embotellamientos": 3 minutos seguidos con más de 90 autos.
- Soportá datos que llegan en vivo: la ventana y la pila se actualizan minuto a minuto sin recalcular.

Cierre: Es el tipo de panel que usan los centros de control de tránsito, combinando varias técnicas eficientes en un mismo flujo de datos.

Ejercicio 200: Desafío de nivel: La cocina del restaurante en hora pico

Nivel 4: Semi-Senior I · Eficiencia · Temática: Gastronomía · Dificultad 40/100

Situación: La cocina de un restaurante tiene dos cocineros y en hora pico los pedidos se amontonan. Cada pedido tiene varios platos, que se preparan en orden de llegada con el primer cocinero disponible, y un pedido sale cuando está listo su último plato. El dueño quiere saber qué mesas esperan más de 20 minutos. Es el desafío que cierra el nivel 4.

Datos de entrada

Tiempos en minutos desde las 21:00. Cocineros: 2.

Plato	Preparación (min)
Milanesa	12
Ensalada	5
Pasta	9
Bife	15
Postre	4

Mesa	Pide en el minuto	Platos (en orden)
Mesa 1	0	Milanesa, Ensalada
Mesa 2	2	Pasta
Mesa 3	3	Bife, Bife, Postre
Mesa 4	8	Ensalada, Pasta
Mesa 5	10	Postre
Mesa 6	25	Milanesa

Tu programa debe

1. Armar una cola con todos los platos en orden de pedido (y, dentro de cada pedido, en el orden en que se pidieron).
2. Asignar cada plato al cocinero que puede empezarlo antes (en empate, el de menor número), sin empezar antes de que llegue el pedido.
3. Calcular cuándo sale cada pedido (cuando termina su último plato) y la espera de cada mesa.
4. Mostrar la preparación de cada plato, los pedidos con su espera (marcando los de más de 20 minutos) y el resumen.

Salida esperada

```

Milanesa (Mesa 1): cocinero 1 de 0 a 12
Ensalada (Mesa 1): cocinero 2 de 0 a 5
Pasta (Mesa 2): cocinero 2 de 5 a 14
Bife (Mesa 3): cocinero 1 de 12 a 27
Bife (Mesa 3): cocinero 2 de 14 a 29
Postre (Mesa 3): cocinero 1 de 27 a 31
Ensalada (Mesa 4): cocinero 2 de 29 a 34
Pasta (Mesa 4): cocinero 1 de 31 a 40
Postre (Mesa 5): cocinero 2 de 34 a 38
Milanesa (Mesa 6): cocinero 2 de 38 a 50
Pedidos completos:
Mesa 1: pidió en 0, sale en 12 -> espera 12 min
Mesa 2: pidió en 2, sale en 14 -> espera 12 min
Mesa 3: pidió en 3, sale en 31 -> espera 28 min (DEMORADO)
Mesa 4: pidió en 8, sale en 40 -> espera 32 min (DEMORADO)
Mesa 5: pidió en 10, sale en 38 -> espera 28 min (DEMORADO)
Mesa 6: pidió en 25, sale en 50 -> espera 25 min (DEMORADO)
Espera promedio: 22.8 min | Pedidos demorados: 4

```

Reglas

- Límites: hasta 10^5 platos y hasta 50 cocineros.
- Complejidad esperada: $O(\text{platos} \times \text{cocineros})$ (en el nivel 6 lo bajás a $O(\text{platos} \times \log \text{cocineros})$ con un heap).
- Formato: nombre del plato en 9 caracteres a la izquierda; minutos en 2 caracteres a la derecha.
- Espera promedio con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la asignación de cocineros es la misma idea que las cajas del banco y los hornos de la pastelería: lo nuevo es agrupar los platos por pedido para saber cuándo sale cada mesa.

Bonus (opcional)

- Priorizá los platos de pedidos que ya llevan más de 15 minutos esperando.
- Agregá la restricción de que los bifés solo los puede hacer el cocinero 1 (el parrillero).

Cierre: Es el mismo tipo de simulación que usan los sistemas de gestión de cocina (pantallas de comandas) para estimar tiempos de salida.

Lote 09: ejercicios 201 a 225

Nivel 5

Semi-Senior II · Estrategia

Ejercicios 201 a 250

Qué vas a practicar

Estrategias: algoritmos voraces, backtracking, recursión, programación dinámica básica y máquinas de estados.

Qué se espera al terminar

Elegir la estrategia correcta y justificar por qué funciona.

Conceptos del nivel

greedy con justificación, backtracking, máquina de estados, evaluación de expresiones, precedencia de operadores, greedy por hora de finalización, hashing, modelado de reglas de negocio, conteo, greedy fraccional, reglas de negocio, validación por prefijos, backtracking con restricciones, backtracking de generación, resolución de colisiones, coloreo, hashing de características, backtracking con poda, parseo, evaluación de expresiones booleanas, partición, tramos, normalización de texto, backtracking con conjuntos, scoring, evaluación con pilas, greedy con plazos, greedy por intervalos, fechas, maximización, backtracking sobre texto, greedy, evaluación de expresiones con variables, claves compuestas, reglas, expresiones, referencias, detección de ciclos simples, restricciones.

Desafíos de nivel

- 248. Motor de promociones del supermercado
- 249. El intérprete de la hoja de cálculo
- 250. Agenda del congreso médico

Ejercicio 201: Vuelto con la menor cantidad de piezas

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 41/100

Situación: Una fintech que fabrica máquinas de autoservicio vende a varios países y a parques de diversiones con fichas propias. El algoritmo que arma el vuelto "siempre empieza por la moneda más grande", y un cliente reclama que su máquina da más piezas de las necesarias. Te piden verificar en qué sistemas de monedas ese algoritmo voraz es realmente óptimo.

Datos de entrada

Sistema	Denominaciones	Monto de ejemplo
Pesos	1, 2, 5, 10, 20, 50, 100, 200, 500, 1000, 2000, 10000	37
Fichas del parque	1, 3, 4	6
Monedas de Isla Ventura	1, 7, 10, 25	14

Verificar todos los montos de 1 a 60.

Tu programa debe

1. Implementar el algoritmo voraz: tantas piezas como se pueda de la denominación más grande, después la siguiente.
2. Implementar una búsqueda exhaustiva con backtracking que encuentre la menor cantidad de piezas (probando, para cada denominación de mayor a menor, cuántas usar, y podando cuando ya se superó el mejor encontrado).
3. Comparar ambos para el monto de ejemplo y para todos los montos de 1 a 60.
4. Informar, para cada sistema, si el voraz fue óptimo en todo el rango o el primer contraejemplo.

Salida esperada

```
Pesos [1, 2, 5, 10, 20, 50, 100, 200, 500, 1000, 2000, 10000]: $ 37 -> voraz 4 piezas, óptimo 4
  el voraz es óptimo para todos los montos hasta 60
Fichas del parque [1, 3, 4]: $ 6 -> voraz 3 piezas, óptimo 2
  primer contraejemplo: 6 (voraz 3, óptimo 2)
Monedas de Isla Ventura [1, 7, 10, 25]: $ 14 -> voraz 5 piezas, óptimo 2
  primer contraejemplo: 14 (voraz 5, óptimo 2)
```

Reglas

- Límites: hasta 15 denominaciones; montos de hasta 10.000 para la verificación.
- Complejidad: el voraz es $O(d)$ por monto; la búsqueda exhaustiva es exponencial en el peor caso, pero la poda la hace viable para estos tamaños.
- Se supone cantidad ilimitada de cada denominación y que siempre existe la de 1.
- Formato: "\$ monto" con un espacio entre el signo y el número en la línea de ejemplo.

Pista (leela solo si te trabás): el voraz es una apuesta: "lo más grande ahora nunca me perjudica". Para demostrar que una apuesta es mala alcanza un contraejemplo; para confiar en ella necesitás una justificación o una verificación exhaustiva.

Bonus (opcional)

- Explicá con un argumento de intercambio por qué el voraz es óptimo para el sistema de pesos.
- Resolvé el óptimo con programación dinámica (lo vas a ver en el nivel 6) y compará tiempos con el backtracking para montos de 10.000.

Cierre: Es el problema del cambio de monedas: los sistemas monetarios reales se diseñan para que el voraz funcione, pero no todos lo cumplen.

Ejercicio 202: El sudoku del diario del domingo

Nivel 5: Semi-Senior II · Estrategia · Temática: Videojuegos · Dificultad 41/100

Situación: Un diario del domingo publica un sudoku y quiere incluir en la web un botón "ver solución" que se calcule en el momento. Además, para clasificar la dificultad, quieren medir cuánto trabajo le cuesta al algoritmo resolverlo.

Datos de entrada

Sudoku (. = casilla vacía):

```
.1.9..45.  
8.5..7...  
.2.....  
...4.97..  
4..1.....  
.78..35.4  
7...1..5  
63..9.84.  
5927..3..
```

Reglas: cada fila, cada columna y cada bloque de 3×3 tienen los dígitos del 1 al 9 sin repetir.

Tu programa debe

1. Buscar la primera casilla vacía recorriendo fila por fila, de izquierda a derecha.
2. Probar los dígitos del 1 al 9 en orden; colocar el primero que no viole ninguna regla y seguir recursivamente.
3. Si ningún dígito sirve, deshacer la última decisión (backtracking) y probar el siguiente dígito ahí.
4. Mostrar la grilla resuelta (bloques separados por espacios y líneas en blanco) y cuántos dígitos se colocaron en total.

Salida esperada

```
317 926 458  
845 317 269  
926 845 173  
  
263 459 781  
459 178 632  
178 263 594  
  
784 631 925  
631 592 847  
592 784 316  
Dígitos colocados (incluyendo los que después se deshicieron): 30751
```

Reglas

- Límites: grillas de 9×9 ; el sudoku tiene solución única.
- Complejidad: exponencial en el peor caso (hasta $9^{\text{casillas vacías}}$), pero la poda por reglas lo hace resolverse en milisegundos.
- Para que el contador coincida, respetá exactamente el orden: casillas en orden de lectura y dígitos de 1 a 9; se cuenta cada vez que se coloca un dígito válido.
- Casos borde: sudoku ya resuelto (0 colocaciones); sudoku sin solución (el programa tiene que terminar e informarlo).

Pista (leela solo si te trabás): una función recursiva `resolver()` que devuelve verdadero si pudo completar la grilla: busca la primera casilla vacía, prueba dígitos, se llama a sí misma y, si falla, borra el dígito antes de probar el siguiente.

Bonus (opcional)

- Elegí siempre la casilla vacía con menos opciones posibles (heurística MRV) y compará el contador.
- Generá sudokus nuevos con solución única, quitando números de una grilla completa y verificando unicidad.

Cierre: Es el backtracking clásico, la misma estrategia que usan los resolvers de restricciones para armar horarios, rutas y configuraciones.

Ejercicio 203: Estados de un pedido online

Nivel 5: Semi-Senior II · Estrategia · Temática: E-commerce · Dificultad 41/100

Situación: Una tienda online tuvo problemas graves: pedidos cancelados que igual se despacharon y devoluciones aceptadas meses después de la entrega. El equipo decide modelar el ciclo de vida del pedido como una máquina de estados estricta, donde cualquier evento no permitido se rechaza y queda registrado.

Datos de entrada

Estado actual	Evento	Nuevo estado
CREADO	pagar	PAGADO
CREADO	cancelar	CANCELADO
PAGADO	preparar	PREPARANDO
PAGADO	cancelar	CANCELADO
PREPARANDO	despachar	ENVIADO
ENVIADO	entregar	ENTREGADO
ENTREGADO	devolver	DEVUELTO

- Todo pedido empieza en CREADO. Cualquier par (estado, evento) que no está en la tabla se rechaza.
- "devolver" solo se acepta hasta 10 días después de la entrega (inclusive).
- Hay reembolso si se cancela un pedido PAGADO o si se devuelve.

Eventos por pedido (día, evento):

- #1001: (1, pagar), (1, preparar), (2, despachar), (4, entregar)
- #1002: (1, pagar), (2, cancelar)
- #1003: (1, preparar), (1, pagar), (3, despachar), (3, preparar), (3, despachar)
- #1004: (2, pagar), (2, preparar), (3, despachar), (5, entregar), (18, devolver)
- #1005: (3, pagar), (3, preparar), (4, cancelar), (4, despachar), (6, entregar), (9, devolver)

Tu programa debe

1. Guardar las transiciones válidas en una estructura clave-valor (estado, evento) → nuevo estado.
2. Procesar los eventos de cada pedido en orden, aplicando la transición o rechazándola con su motivo.
3. Validar la regla de plazo de la devolución.
4. Mostrar el estado final de cada pedido, si tuvo reembolso y los eventos rechazados.

Salida esperada

```
#1001: ENTREGADO
#1002: CANCELADO (con reembolso)
#1003: ENVIADO
rechazado -> día 1 preparar: no permitido en CREADO
rechazado -> día 3 despachar: no permitido en PAGADO
#1004: ENTREGADO
rechazado -> día 18 devolver: fuera de plazo (13 días desde la entrega)
#1005: DEVUELTO (con reembolso)
rechazado -> día 4 cancelar: no permitido en PREPARANDO
```

Reglas

- Límites: hasta 10^6 pedidos y 10^7 eventos.
- Complejidad esperada: $O(1)$ por evento.
- Un evento rechazado no cambia el estado.
- Los rechazos se listan en orden, con dos espacios de sangría.

Pista (leela solo si te trabás): con la tabla de transiciones, el programa no tiene ni un condicional por estado: busca (estado, evento) y, si no está, rechaza. Agregar un estado nuevo es agregar filas a la tabla.

Bonus (opcional)

- Guardá el historial completo de cada pedido y reconstruí en qué estado estaba en un día cualquiera.
- Agregá el estado EN_DISPUTA, al que se llega si el cliente reclama un pedido ENVIADO que no llegó en 15 días.

Cierre: Es el mismo modelo de estados que usan las plataformas de comercio electrónico y los sistemas de pagos para evitar transiciones imposibles.

Ejercicio 204: La calculadora del presupuesto de obra

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 42/100

Situación: Una app para constructores permite escribir fórmulas en los ítems del presupuesto de obra, como "3 * (1250.5 + 480)" para calcular materiales. Hasta ahora se evaluaban de izquierda a derecha sin respetar la precedencia y hubo presupuestos mal cotizados. Te piden un evaluador correcto.

Datos de entrada

Expresiones:

- 3 * (1250.5 + 480)
- 2 + 3 * 4 - 10 / 4
- -(15 - 20) * 2
- (1200 - 300) * 1.21
- 10 / (5 - 5)
- (2 + 3
- 4 * -2 + 7
- 12 \$ 3

Gramática:

```
expresión := término (('+' | '-') término)*
término   := factor (('*' | '/') factor)*
factor     := '-' factor | '(' expresión ')' | número
```

Tu programa debe

1. Separar la expresión en tokens: números (con punto decimal), operadores y paréntesis; cualquier otro carácter es un error.
2. Evaluar con descenso recursivo según la gramática (una función por regla).
3. Detectar errores: carácter inválido, división por cero, paréntesis sin cerrar, expresión incompleta o símbolos sobrantes.
4. Mostrar el resultado con 2 decimales o el error.

Salida esperada

```
3 * (1250.5 + 480) = 5191.50
2 + 3 * 4 - 10 / 4 = 11.50
-(15 - 20) * 2 = 10.00
(1200 - 300) * 1.21 = 1089.00
10 / (5 - 5) -> ERROR: división por cero
(2 + 3 -> ERROR: falta cerrar un paréntesis
4 * -2 + 7 = -1.00
12 $ 3 -> ERROR: carácter inválido '$'
```

Reglas

- Límites: expresiones de hasta 10^5 caracteres y 10^4 niveles de paréntesis (cuidado con la profundidad de recursión).
- Complejidad esperada: $O(n)$.
- Resultados con 2 decimales (0,5 hacia arriba), usando decimales exactos durante el cálculo. Los decimales se muestran con punto y sin separador de miles en la salida.
- El menos unario tiene más precedencia que la multiplicación: "4 * -2" es $4 \times (-2)$.

Pista (leela solo si te trabás): la precedencia sale sola de la estructura de la gramática: la función de expresión llama a la de término, que llama a la de factor. Lo que está más abajo se evalúa primero.

Bonus (opcional)

- Agregá la potencia ($\wedge$), asociativa a derecha y con más precedencia que la multiplicación.
- Agregá porcentajes: "1200 + 21%" significa $1200 \times 1,21$.

Cierre: Es el mismo tipo de evaluador que tienen las planillas de cálculo, las calculadoras y los intérpretes de lenguajes.

Ejercicio 205: Máximo de charlas en el auditorio

Nivel 5: Semi-Senior II · Estrategia · Temática: Educación · Dificultad 42/100

Situación: Un congreso de tecnología educativa tiene un auditorio principal y más propuestas de charlas que horarios. La organización quiere meter la mayor cantidad posible de charlas en el auditorio sin superposiciones (las que queden afuera van a salas chicas). Además, quieren entender por qué la estrategia elegida es la correcta.

Datos de entrada

Charla	Inicio	Fin
Keynote de apertura	09:00	10:30
Taller de datos	09:30	10:00
Panel de fintech	10:00	11:00
Charla de seguridad	10:15	10:45
IA en educación	10:45	12:00
Café con startups	11:00	11:30
Cierre del bloque	11:30	12:30
Demo de robótica	12:00	12:15

Una charla puede empezar exactamente cuando termina otra.

Tu programa debe

1. Ordenar las charlas por hora de fin (en empate, por hora de inicio).
2. Recorrerlas eligiendo cada charla que empieza a la hora de fin de la última elegida o después.
3. Mostrar las charlas elegidas.
4. Comparar con otras dos estrategias voraces (elegir por hora de inicio y por duración más corta, siempre que no se superpongan con las ya elegidas) y mostrar cuántas charlas logra cada una.

Salida esperada

```
Voraz por hora de fin: 4 charlas
09:30-10:00 Taller de datos
10:15-10:45 Charla de seguridad
11:00-11:30 Café con startups
12:00-12:15 Demo de robótica
Otras estrategias: por hora de inicio 3 charlas | por duración más corta 4 charlas
```

Reglas

- Límites: hasta 10^6 charlas.
- Complejidad esperada: $O(n \log n)$.
- Justificación: elegir la que termina antes deja el mayor tiempo libre posible para el resto; cualquier solución óptima se puede transformar, charla por charla, en la voraz sin perder cantidad (argumento de intercambio).
- Las estrategias alternativas se muestran para ver que un voraz "razonable" puede fallar.

Pista (leela solo si te trabás): la charla que termina más temprano siempre puede formar parte de una solución óptima: si una solución óptima empieza con otra, la podés cambiar por esta sin romper nada.

Bonus (opcional)

- Si cada charla tiene un puntaje de interés, maximizá la suma de puntajes (ya no alcanza el voraz: vas a necesitar programación dinámica).
- Calculá cuántas salas hacen falta para dar todas las charlas (ver el ejercicio de aulas mínimas).

Cierre: Es el problema de selección de actividades, el ejemplo clásico de algoritmo voraz con demostración de optimalidad.

Ejercicio 206: Archivos duplicados en el servidor

Nivel 5: Semi-Senior II · Estrategia · Temática: Seguridad informática · Dificultad 42/100

Situación: El servidor de archivos de una pyme se quedó sin espacio y el administrador sospecha que hay muchas copias del mismo archivo con distintos nombres. Comparar cada archivo contra todos los demás, byte a byte, tardaría días. Te pide usar hashing para encontrar los duplicados rápido y sin falsos positivos.

Datos de entrada

Archivos (ruta y contenido; en este ejemplo el contenido es un texto corto y su tamaño en bytes es su cantidad de caracteres):

Ruta	Contenido
/home/ana/foto_playa.jpg	SOLARENA2026
/backup/fotos/playa(1).jpg	SOLARENA2026
/home/beto/informe.pdf	INFORME-Q3-FINAL
/home/ana/informe_v2.pdf	INFORME-Q3-FINAL
/tmp/cancion.mp3	MAREA-VIVO-MIX
/home/beto/notas.txt	COMPRAR PAN
/backup/cancion_copia.mp3	MAREA-VIVO-MIX
/backup/playa_final.jpg	SOLARENA2027
/home/caro/lista.txt	COMPRAR PAN

Función de hash: $h = 0$; para cada carácter c : $h = (h \times 31 + \text{código}(c)) \text{ resto } 2^{32}$, donde $\text{código}(c)$ es el código numérico del carácter (ASCII).

Tu programa debe

1. Agrupar los archivos por tamaño: si el tamaño es distinto, no pueden ser iguales (y no hace falta calcular su hash).
2. Dentro de cada grupo de tamaño con 2 o más archivos, calcular el hash del contenido y agrupar por hash.
3. Confirmar byte a byte dentro de cada grupo de hash (dos contenidos distintos podrían tener el mismo hash).
4. Mostrar los grupos de duplicados con su hash (ordenados por tamaño), el espacio recuperable y cuántos hashes se calcularon.

Salida esperada

```
Hash 3467992305: /home/beto/notas.txt, /home/caro/lista.txt
Hash 2907525369: /home/ana/foto_playa.jpg, /backup/fotos/playa(1).jpg
Hash 3232648110: /tmp/cancion.mp3, /backup/cancion_copia.mp3
Hash 4042154620: /home/beto/informe.pdf, /home/ana/informe_v2.pdf
Grupos de duplicados: 4 | Espacio recuperable: 53 bytes
Hashes calculados: 9 de 9 archivos
```

Reglas

- Límites: hasta 10^7 archivos; tamaños de hasta 10^{10} bytes (en la vida real, el hash se calcula leyendo el archivo por partes).
- Complejidad esperada: $O(\text{total de bytes leídos})$ en promedio, en lugar de $O(n^2)$ comparaciones.
- Espacio recuperable: por cada grupo, tamaño $\times$ (cantidad $- 1$).
- Dentro de cada grupo, los archivos se listan en el orden de la tabla.

Pista (leela solo si te trabás): el tamaño es un "hash gratis": descarta la mayoría de los pares sin leer nada. El hash descarta casi todo el resto, y la comparación final garantiza que no haya falsos positivos.

Bonus (opcional)

- Calculá primero un hash de los primeros 4 KB y solo si coincide, el hash completo (así se leen menos bytes).
- Reemplazá los duplicados por enlaces al original y generá un informe de lo que se hizo.

Cierre: Es la técnica que usan los programas de deduplicación, los sistemas de backup y los servicios de almacenamiento en la nube.

Ejercicio 207: Ocho damas en el tablero del club

Nivel 5: Semi-Senior II · Estrategia · Temática: Deportes · Dificultad 42/100

Situación: El club de ajedrez del barrio arma un desafío para los chicos: ubicar ocho damas en el tablero sin que ninguna ataque a otra. El profe quiere un programa que encuentre todas las soluciones, muestre la primera y cuente cuántas hay para tableros de otros tamaños.

Datos de entrada

Tablero de 8×8 . Dos damas se atacan si están en la misma fila, la misma columna o la misma diagonal.

Tamaños a contar: de 4 a 10.

Tu programa debe

1. Ubicar una dama por fila, probando columnas de izquierda a derecha.
2. Descartar una columna si ya hay una dama en esa columna o en alguna de sus dos diagonales (usá conjuntos para saberlo en $O(1)$).
3. Al completar las 8 filas, registrar la solución y seguir buscando (backtracking).
4. Mostrar la cantidad de soluciones, la primera encontrada (con un dibujo) y la cantidad para tamaños de 4 a 10.

Salida esperada

```
Soluciones para 8 damas: 92
Primera solución (columna de la dama en cada fila, desde 1): 1 5 8 6 3 7 2 4
D . . . . .
. . . . D . .
. . . . . D
. . . . D .
. . D . . .
. . . . . D .
. D . . . .
. . . D . .
Soluciones por tamaño: 4=2, 5=10, 6=4, 7=40, 8=92, 9=352, 10=724
```

Reglas

- Límites: tableros de hasta 14×14 .
- Complejidad: exponencial; la poda por columnas y diagonales la mantiene en milisegundos para $n \leq 10$.
- Las diagonales se identifican con fila – columna (una dirección) y fila + columna (la otra).
- La "primera solución" es la primera que encuentra el recorrido descrito (filas de arriba a abajo, columnas de izquierda a derecha).

Pista (leela solo si te trabás): no hace falta revisar las filas: al poner una dama por fila, nunca hay dos en la misma. Solo quedan columnas y diagonales, y tres conjuntos alcanzan para saber si una casilla está atacada.

Bonus (opcional)

- Contá solo las soluciones distintas si se consideran iguales las que son rotaciones o reflejos del tablero (para $n = 8$ son 12).
- Resolvé el problema con máscaras de bits en lugar de conjuntos y medí la mejora (nivel 7).

Cierre: El problema de las ocho damas es el ejemplo clásico de backtracking y de satisfacción de restricciones.

Ejercicio 208: El semáforo inteligente

Nivel 5: Semi-Senior II · Estrategia · Temática: Transporte · Dificultad 42/100

Situación: En el cruce de una avenida con una calle de poco tránsito, el semáforo le daba verde a la calle secundaria aunque no hubiera nadie esperando. La municipalidad instaló un sensor de autos en la secundaria y quiere un controlador que solo le dé paso cuando hace falta, sin dejar a la avenida con menos de 30 segundos de verde.

Datos de entrada

Estados y duraciones:

- VERDE_PRINCIPAL: dura al menos 30 s; después, pasa a amarillo solo si hay un auto esperando en la secundaria.
- AMARILLO_PRINCIPAL: 4 s.
- VERDE_SECUNDARIA: 20 s fijos. Al entrar a este estado se considera atendida la espera (se borra la marca de auto esperando).
- AMARILLO_SECUNDARIA: 4 s, y vuelve a VERDE_PRINCIPAL.

El sensor detecta autos en la secundaria en los segundos: 10, 75, 76, 140. La simulación va de 0 a 200 s.

Tu programa debe

1. Modelar el semáforo como una máquina de estados con el estado actual, el instante en que se entró y una marca de "auto esperando".
2. En cada segundo, registrar las detecciones del sensor y evaluar si corresponde una transición (pueden ocurrir en el mismo segundo en que se cumple la condición).
3. Registrar cada cambio de estado con su instante.
4. Mostrar la secuencia de cambios.

Salida esperada

```
0 s: VERDE_PRINCIPAL
30 s: AMARILLO_PRINCIPAL
34 s: VERDE_SECUNDARIA
54 s: AMARILLO_SECUNDARIA
58 s: VERDE_PRINCIPAL
88 s: AMARILLO_PRINCIPAL
92 s: VERDE_SECUNDARIA
112 s: AMARILLO_SECUNDARIA
116 s: VERDE_PRINCIPAL
146 s: AMARILLO_PRINCIPAL
150 s: VERDE_SECUNDARIA
170 s: AMARILLO_SECUNDARIA
174 s: VERDE_PRINCIPAL
```

Reglas

- Límites: simulaciones de hasta 10^7 segundos y 10^6 detecciones.
- Complejidad esperada: $O(\text{duración} + \text{detecciones})$. (En el bonus lo hacés por eventos, sin recorrer cada segundo.)
- Un auto detectado mientras la secundaria está en verde o en amarillo no genera un nuevo ciclo si llegó antes de que se borrara la marca; si llega después, sí.
- En un mismo segundo se procesan primero las detecciones y después las transiciones.

Pista (leela solo si te trabás): escribí la lógica como "según el estado actual, ¿se cumple la condición de salida?". Cada estado tiene una sola condición; eso hace el código muy fácil de revisar.

Bonus (opcional)

- Simulá por eventos: calculá directamente el próximo instante de cambio en lugar de avanzar de a un segundo.
- Agregá un botón peatonal que funcione como la detección de autos pero con verde de 15 s.

Cierre: Es el mismo diseño de controladores que se usa en semáforos inteligentes, ascensores y cualquier sistema embebido con estados.

Ejercicio 209: Descuentos que se acumulan (o no)

Nivel 5: Semi-Senior II · Estrategia · Temática: E-commerce · Dificultad 42/100

Situación: Un supermercado tiene tres promociones y los cajeros nunca saben si se acumulan. El área comercial escribió las reglas con precisión y quiere que el sistema las aplique siempre igual, mostrando al cliente cada descuento por separado.

Datos de entrada

Hoy es martes y el cliente paga con la tarjeta del súper.

Producto	Categoría	Precio	Cantidad
Gaseosa 2,25 L	gaseosas	\$2.800	4
Leche entera	lácteos	\$1.450	6
Yogur	lácteos	\$1.200	3
Yerba 1 kg	almacén	\$5.400	2
Agua saborizada	gaseosas	\$1.900	3

Reglas, en este orden:

1. 2x1 en gaseosas: por cada 2 unidades del mismo producto de la categoría gaseosas, una es gratis.
2. Martes: 20 % de descuento en lácteos (sobre el precio de lista de cada línea).
3. Tarjeta del súper: 10 % sobre el total que queda después de las reglas 1 y 2, con tope de \$3.000.

Tu programa debe

1. Calcular el subtotal.
2. Aplicar el 2x1 por producto y mostrarlo por línea.
3. Aplicar el 20 % de los martes a cada línea de lácteos.
4. Calcular el reintegro de la tarjeta con su tope y el total a pagar.

Salida esperada

```
Subtotal: $40000
2x1 en Gaseosa 2,25 L: -$5600
2x1 en Agua saborizada: -$1900
20% martes en Leche entera: -$1740.00
20% martes en Yogur: -$720.00
Total con descuentos: $30040.00
Reintegro con tarjeta del súper (10%, tope $3000): -$3000.00
A pagar: $27040.00
```

Reglas

- Límites: carritos de hasta 10^4 líneas; hasta 100 promociones (en el bonus las parametrizás).
- Complejidad esperada: $O(\text{líneas} \times \text{promociones})$.
- Cada descuento se redondea a 2 decimales (0,5 hacia arriba) al calcularlo. Los decimales se muestran con punto y sin separador de miles en la salida.
- El 2x1 no se combina entre productos distintos (2 gaseosas y 1 agua saborizada no suman un par).

Pista (leela solo si te trabás): modelá cada regla como una función que recibe el carrito (o el total parcial) y devuelve los descuentos aplicados. El orden de aplicación es parte de la regla de negocio.

Bonus (opcional)

- Parametrizá las promociones en una tabla (tipo, categoría, porcentaje, día, tope) para que marketing pueda cargar nuevas sin tocar el código.
- Agregá una promoción no acumulable: "30 % en almacén" que, si aplica, anula las demás; el sistema elige lo que más le conviene al cliente.

Cierre: Es el corazón de los motores de promociones de supermercados y tiendas online.

Ejercicio 210: Todas las formas de pagar justo

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 43/100

Situación: Un kiosco no tiene cambio y le pide a Lucas que pague justo. Lucas tiene algunos billetes en la billetera y quiere saber todas las formas de juntar exactamente el monto, y cuál usa menos billetes. Una fintech que hace una app de "billetera inteligente" quiere resolver esto en general.

Datos de entrada

Billete	Cantidad en la billetera
\$1.000	2
\$500	3
\$200	2
\$100	4

Monto a pagar: \$1.700

Tu programa debe

1. Recorrer las denominaciones de mayor a menor, decidiendo cuántos billetes usar de cada una (de la mayor cantidad posible a 0).
2. Podar: no usar más billetes de los disponibles ni pasarse del monto.
3. Registrar cada combinación que llega exactamente al monto.
4. Mostrar la cantidad de combinaciones, cada una con sus piezas y la que usa menos piezas (en empate, la primera).

Salida esperada

```
Formas de pagar $1700 exactos: 6
1. 1x$1000 + 1x$500 + 1x$200 (3 piezas)
2. 1x$1000 + 1x$500 + 2x$100 (4 piezas)
3. 1x$1000 + 2x$200 + 3x$100 (6 piezas)
4. 3x$500 + 1x$200 (4 piezas)
5. 3x$500 + 2x$100 (5 piezas)
6. 2x$500 + 2x$200 + 3x$100 (7 piezas)
Menos piezas: opción 1
```

Reglas

- Límites: hasta 10 denominaciones con hasta 20 unidades cada una.
- Complejidad: exponencial en el peor caso (producto de las cantidades + 1); la poda por monto restante la reduce mucho.
- Las combinaciones se listan en el orden en que las genera el recorrido descripto.
- Casos borde: monto imposible (0 combinaciones); monto 0 (una combinación vacía).

Pista (leela solo si te trabás): cada nivel de la recursión decide una denominación. Probar de la mayor cantidad a la menor hace que las primeras soluciones encontradas usen billetes grandes.

Bonus (opcional)

- Contá las formas sin listarlas cuando las cantidades son grandes, con programación dinámica (nivel 6).
- Si el kiosco sí tiene algo de cambio, encontrará la forma de pagar que minimiza la cantidad total de billetes que cambian de mano (los que das más los que recibís).

Cierre: Es el problema de conteo de combinaciones con cantidades limitadas, que aparece en cajeros, máquinas expendedoras y armado de paquetes.

Ejercicio 211: El barco con carga fraccionable

Nivel 5: Semi-Senior II · Estrategia · Temática: Logística y envíos · Dificultad 43/100

Situación: Un barco de carga sale del puerto de Rosario con 1.000 toneladas de capacidad y hay más granos esperando que lugar en la bodega. Los granos se pueden cargar en cualquier cantidad (no hace falta llevar un lote completo). El exportador quiere maximizar el valor de lo que sale en este viaje.

Datos de entrada

Capacidad de la bodega: 1.000 t

Grano	Toneladas disponibles	Valor total del lote
Soja	600	\$312.000
Maíz	500	\$190.000
Trigo	400	\$176.000
Girasol	250	\$157.500
Sorgo	300	\$96.000

Tu programa debe

1. Calcular el valor por tonelada de cada grano.
2. Ordenar de mayor a menor valor por tonelada (en empate, por nombre).
3. Cargar cada grano completo mientras entre; del primero que no entra, cargar solo la parte que completa la bodega.
4. Mostrar lo cargado de cada grano, el valor total y la capacidad libre.

Salida esperada

```
Girasol: 250.0 t de 250 ($630.00 por t) -> $157500.00
Soja: 600.0 t de 600 ($520.00 por t) -> $312000.00
Trigo: 150.0 t de 400 ($440.00 por t) -> $66000.00
Valor total cargado: $535500.00 | Capacidad libre: 0.0 t
```

Reglas

- Límites: hasta 10^6 tipos de carga.
- Complejidad esperada: $O(n \log n)$.
- Justificación: como la carga se puede fraccionar, cualquier tonelada de un grano menos valioso se puede cambiar por una de uno más valioso sin empeorar; por eso el voraz por valor unitario es óptimo.
- Toneladas con 1 decimal y montos con 2 (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la clave es que se puede fraccionar. Si no se pudiera (lotes enteros), el voraz dejaría de ser óptimo: probá con capacidad 1.000 y lotes indivisibles para ver un contraejemplo.

Bonus (opcional)

- Resolví la versión con lotes indivisibles (mochila 0/1, nivel 6) y comparé el resultado con estos datos.
- Agregá un límite de volumen además del de peso: ¿sigue funcionando el voraz?

Cierre: Es la mochila fraccionaria, el caso donde el voraz es óptimo, usado en carga de buques, asignación de presupuesto divisible y mezclas.

Ejercicio 212: La marca de la tarjeta según su número

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 43/100

Situación: Un checkout de pagos quiere mostrar el logo de la tarjeta apenas el usuario tipea los primeros dígitos y validar el número antes de mandarlo al procesador. Cada marca tiene sus prefijos y sus largos válidos, y algunos prefijos se superponen: gana el más largo que coincide.

Datos de entrada

Marca	Prefijos	Largos
Amex	34, 37	15
Diners	36, 300 a 305	14
Discover	6011, 65, 644 a 649	16
Mastercard	51 a 55, 2221 a 2720	16
Visa	4	13, 16, 19

Números a verificar (ficticios, generados para el ejercicio):

- 4111000000000008
- 5500000000000004
- 3782000000000000
- 6011000000000004
- 3056000000000007
- 2223000000000007
- 4111000000000001
- 4111111111111112
- 9900000000000002
- 3712000000000003

Dígito verificador: algoritmo de Luhn (ver el ejercicio "El dígito que delata una tarjeta mal tipeada").

Tu programa debe

1. Identificar la marca buscando el prefijo más largo que coincide con el principio del número.
2. Validar que el largo sea uno de los permitidos para esa marca.
3. Validar el dígito verificador con Luhn.
4. Mostrar, para cada número, la marca y el resultado (se informa el primer problema en ese orden).

Salida esperada

```
4111000000000008: Visa válida
5500000000000004: Mastercard válida
3782000000000000: Amex válida
6011000000000004: Discover válida
3056000000000007: Diners válida
2223000000000007: Mastercard válida
4111000000000001: Visa, pero el largo 15 no es válido para esa marca
4111111111111112: Visa, dígito verificador incorrecto
9900000000000002: marca desconocida
3712000000000003: Amex, pero el largo 16 no es válido para esa marca
```

Reglas

- Límites: hasta 10^6 validaciones por segundo en el checkout.
- Complejidad esperada: $O(\text{largo del número})$ por validación, con la tabla de prefijos precargada (por ejemplo, en un mapa de prefijo a marca).
- Los rangos (2221 a 2720) se expanden a prefijos o se validan comparando numéricamente los primeros 4 dígitos.

- Si ningún prefijo coincide, "marca desconocida".

Pista (leela solo si te trabás): con los prefijos en un mapa, probá los primeros 4, 3, 2 y 1 dígitos del número (en ese orden): el primero que esté en el mapa es el más largo que coincide.

Bonus (opcional)

- Mostrá la marca "en vivo" mientras se tipea: con cada dígito nuevo, decí si ya se puede determinar la marca o todavía es ambigua.
- Formateá el número como lo imprime cada marca (Amex en grupos de 4-6-5).

Cierre: Es la misma lógica que usan los formularios de pago para mostrar el logo de la tarjeta y rechazar números inválidos antes de enviarlos.

Ejercicio 213: El fixture con sedes y descansos

Nivel 5: Semi-Senior II · Estrategia · Temática: Deportes · Dificultad 43/100

Situación: La liga regional tiene 6 equipos y ya definió quién juega con quién en cada una de las 5 fechas. Falta decidir quién es local en cada partido, y hay restricciones: Andes y Bosque comparten estadio, así que no pueden ser locales en la misma fecha; nadie puede jugar tres fechas seguidas con la misma condición; y todos tienen que ser locales 2 o 3 veces.

Datos de entrada

Cruces por fecha (el orden dentro de cada par no indica localía):

- Fecha 1: Andes-Faro, Bosque-Esteba, Cumbre-Delta
- Fecha 2: Andes-Esteba, Faro-Delta, Bosque-Cumbre
- Fecha 3: Andes-Delta, Esteba-Cumbre, Faro-Bosque
- Fecha 4: Andes-Cumbre, Delta-Bosque, Esteba-Faro
- Fecha 5: Andes-Bosque, Cumbre-Faro, Delta-Esteba

Tu programa debe

1. Recorrer los partidos en orden (fecha por fecha, y dentro de cada fecha en el orden dado).
2. Para cada partido, probar primero como local al primer equipo del par y después al segundo.
3. Podar cuando se viola una restricción: estadio compartido en la misma fecha, tres condiciones iguales seguidas, más de 3 localías o imposibilidad de llegar a 2.
4. Contar todas las asignaciones válidas y mostrar la primera encontrada.

Salida esperada

Asignaciones válidas de localía: 70

Primera encontrada:

```
Fecha 1: Andes (L) vs Faro | Esteba (L) vs Bosque | Cumbre (L) vs Delta
Fecha 2: Andes (L) vs Esteba | Faro (L) vs Delta | Cumbre (L) vs Bosque
Fecha 3: Delta (L) vs Andes | Esteba (L) vs Cumbre | Bosque (L) vs Faro
Fecha 4: Andes (L) vs Cumbre | Delta (L) vs Bosque | Faro (L) vs Esteba
Fecha 5: Bosque (L) vs Andes | Cumbre (L) vs Faro | Esteba (L) vs Delta
```

Reglas

- Límites: ligas de hasta 20 equipos (en la vida real se combinan poda fuerte y heurísticas).
- Complejidad: exponencial (2^{partidos}) sin poda; la poda temprana hace que sea instantáneo para 15 partidos.
- La restricción de 2 a 3 localías se chequea en cada paso: si con las fechas que le quedan a un equipo ya no puede llegar a 2, se poda.
- Formato: el local primero, marcado con (L).

Pista (leela solo si te trabás): verificá las restricciones apenas asignás un partido, no al final: descartar una rama temprano evita explorar miles de combinaciones que ya estaban condenadas.

Bonus (opcional)

- Generá también los cruces (quién juega con quién) con el método del círculo y las mismas restricciones.
- Agregá la preferencia "Cumbre quiere ser local en la fecha 5 (aniversario del club)" como restricción blanda y elegí, entre las válidas, la que la cumple.

Cierre: Es el mismo tipo de problema que resuelven las ligas profesionales para armar el fixture, combinando backtracking con restricciones.

Ejercicio 214: La máquina expendedora

Nivel 5: Semi-Senior II · Estrategia · Temática: E-commerce · Dificultad 43/100

Situación: Una empresa que instala máquinas expendedoras en estaciones de tren recibe quejas: máquinas que se "comen" monedas, que entregan productos agotados o que no devuelven el vuelto al cancelar. Te piden reescribir el controlador como una máquina de estados y probarlo con una secuencia de acciones.

Datos de entrada

Código	Producto	Precio	Stock
A1	Agua	\$700	5
A2	Alfajor	\$850	0
B1	Gaseosa	\$1.200	2

Monedas aceptadas: \$50, \$100, \$200 y \$500 (cualquier otra se devuelve).

Secuencia de acciones: moneda \$500, moneda \$100, elegir A1, moneda \$500, moneda \$20, moneda \$500, elegir B1, elegir A2, moneda \$500, moneda \$500, elegir A2, cancelar, elegir A1

Tu programa debe

1. Modelar dos estados: ESPERANDO (sin crédito) y CON_CREDITO.
2. Procesar cada acción según el estado: insertar moneda, elegir producto o cancelar.
3. Al entregar un producto, calcular el vuelto y desglosarlo en monedas de mayor a menor; al cancelar, devolver todo el crédito.
4. Mostrar la respuesta a cada acción y el stock final.

Salida esperada

```
moneda $500: crédito $500
moneda $100: crédito $600
elegir A1: faltan $100 para Agua
moneda $500: crédito $1100
moneda $20: rechazada (no se acepta), se devuelve
moneda $500: crédito $1600
elegir B1: entrega Gaseosa, vuelto $400 (2x$200)
elegir A2: primero ingresá dinero
moneda $500: crédito $500
moneda $500: crédito $1000
elegir A2: Alfajor agotado
cancelar: devuelve $1000 (2x$500)
elegir A1: primero ingresá dinero
Stock final: A1 5, A2 0, B1 1
```

Reglas

- Límites: hasta 10^6 acciones por máquina por mes.
- Complejidad esperada: $O(1)$ por acción.
- Elegir un producto agotado o sin crédito suficiente no consume el crédito.
- La máquina tiene monedas ilimitadas para dar vuelto (ver el bonus).

Pista (leela solo si te trabás): para cada estado, listá qué hace cada acción. Los errores (agotado, falta plata) no cambian de estado: el cliente puede seguir agregando monedas o elegir otra cosa.

Bonus (opcional)

- Limitá las monedas disponibles para el vuelto y, si no se puede dar vuelto exacto, rechazá la venta antes de entregar.

- Agregá un estado MANTENIMIENTO al que se entra con un código secreto y que permite reponer stock.

Cierre: Es el ejemplo de manual de máquina de estados, y el mismo diseño se usa en cajeros, terminales de pago y kioscos de autoservicio.

Ejercicio 215: Contraseñas débiles por diccionario

Nivel 5: Semi-Senior II · Estrategia · Temática: Seguridad informática · Dificultad 44/100

Situación: El formulario de registro de un club deportivo deja que los socios elijan su contraseña, y muchos ponen el nombre del club con alguna letra cambiada por un número, algo fácil de adivinar. El equipo de seguridad quiere que, al momento de registrarse, el sistema arme una lista de bloqueo con todas las variantes "disfrazadas" de unas pocas palabras prohibidas y rechace la contraseña si coincide con alguna.

Datos de entrada

Palabras prohibidas: river, boca, club

Sustituciones que se consideran "la misma letra": a → a, 4, @; e → e, 3; o → o, 0; i → i, 1 (las demás letras quedan igual).

Variaciones: la primera letra puede ir en minúscula o mayúscula; sufijos posibles: ninguno, 123, !.

Contraseñas propuestas al registrarse: R0jo2026, r1ver!, mi perro fiel, CLUB123, b0ca, Sol y Luna 88, Riv3r, arquitecto

Tu programa debe

1. Generar con backtracking, para cada palabra prohibida, todas las combinaciones de sustituciones de sus letras.
2. Aplicar las variaciones de mayúscula inicial y sufijo, y quitar duplicados conservando el orden.
3. Juntar todas las variantes en una lista de bloqueo (un conjunto).
4. Para cada contraseña propuesta, aceptarla o rechazarla según si está en la lista de bloqueo.

Salida esperada

```
Variantes generadas por palabra:
river: 24 variantes
boca: 36 variantes
club: 6 variantes
Total en la lista de bloqueo: 66
Registros:
"R0jo2026": aceptada
"r1ver!": RECHAZADA (coincide con una palabra prohibida disfrazada)
"mi perro fiel": aceptada
"CLUB123": aceptada
"b0ca": RECHAZADA (coincide con una palabra prohibida disfrazada)
"Sol y Luna 88": aceptada
"Riv3r": RECHAZADA (coincide con una palabra prohibida disfrazada)
"arquitecto": aceptada
```

Reglas

- Límites: hasta 20 palabras prohibidas de hasta 12 caracteres; hasta 6 sustituciones por letra.
- Complejidad: exponencial en el largo de cada palabra (producto de las opciones por letra), pero acotada para palabras cortas.
- La comparación es exacta contra el conjunto de variantes: no se buscan coincidencias parciales.
- Casos borde: palabra sin letras sustituibles (pocas variantes); contraseña que no se parece a ninguna prohibida (aceptada).

Pista (leela solo si te trabás): el backtracking recorre letra por letra: en cada posición prueba todas las formas de escribir esa letra. Al llegar al final de la palabra, aplicás mayúscula y sufijos.

Bonus (opcional)

- Sumá el símbolo \$ como variante de la s y medí cuánto crece la lista de bloqueo.

- En lugar de generar todas las variantes, verificá una contraseña "al revés": normalizala (bajar a minúscula, deshacer las sustituciones, sacar sufijos) y comparala con las palabras prohibidas.
¿Qué enfoque escala mejor?

Cierre: Es la lógica que usan los formularios de registro para bloquear contraseñas débiles conocidas antes de que un socio las elija.

Ejercicio 216: Turnos de las bombas de riego

Nivel 5: Semi-Senior II · Estrategia · Temática: Agricultura · Dificultad 44/100

Situación: Un productor tiene un tanque con agua limitada por el turno de riego y varias bombas, cada una con un consumo fijo y las hectáreas que alcanza a regar. No puede fraccionar: una bomba se enciende entera o no se enciende. Quiere regar la mayor superficie posible, y hay que decidir con qué criterio elegir.

Datos de entrada

Agua disponible: 100 unidades.

Bomba	Agua que consume	Hectáreas que riega
B1	40	9
B2	25	5
B3	60	12
B4	30	8
B5	20	3

Una bomba se usa entera o no se usa.

Tu programa debe

1. Calcular el rendimiento de cada bomba (hectáreas por unidad de agua).
2. Ordenar de mayor a menor rendimiento (en empate, por nombre) y encender cada bomba que todavía tenga agua suficiente.
3. Mostrar qué hace cada bomba, las hectáreas regadas y el agua sin usar.

Salida esperada

```
B4: usa 30 (rinde 0.267 ha/unidad) -> riega 8 ha
B1: usa 40 (rinde 0.225 ha/unidad) -> riega 9 ha
B2: usa 25 (rinde 0.200 ha/unidad) -> riega 5 ha
B3: no alcanza el agua (60 > 5 disponibles)
B5: no alcanza el agua (20 > 5 disponibles)
Hectáreas regadas: 22 | Agua sin usar: 5
```

Reglas

- Límites: hasta 10^6 bombas.
- Complejidad esperada: $O(n \log n)$.
- Rendimiento con 3 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Ojo: como no se puede fraccionar, este voraz por rendimiento NO siempre es óptimo (a diferencia del barco con carga fraccionable). Es una heurística razonable; ver el Bonus.

Pista (leela solo si te trabás): acá el voraz es una buena aproximación, no una garantía. El problema exacto (elegir un subconjunto que maximiza hectáreas sin pasar del agua) es la mochila 0/1 del nivel 6.

Bonus (opcional)

- Construí un caso donde el voraz por rendimiento no da el óptimo.
- Resolvé el óptimo exacto con programación dinámica (nivel 6) y compará con el voraz en estos datos.

Cierre: Es la diferencia clave entre la mochila fraccionaria (voraz óptimo) y la 0/1 (voraz solo aproxima), que aparece en riego, presupuesto y asignación de recursos indivisibles.

Ejercicio 217: Casilleros del gimnasio

Nivel 5: Semi-Senior II · Estrategia · Temática: Deportes · Dificultad 44/100

Situación: Un gimnasio asigna a cada socio un casillero calculando el resto del DNI dividido por la cantidad de casilleros. Cuando dos socios caen en el mismo casillero (colisión), el sistema busca el siguiente casillero libre. Quieren simular las asignaciones para ver cuántas colisiones se producen.

Datos de entrada

Cantidad de casilleros: 13. Casillero base = DNI resto 13.

Socio	DNI
Ríos	30111
Paz	30124
Gil	30202
Sosa	30124
Vera	30150
Luna	30176
Díaz	30137
Mota	30215

Tu programa debe

1. Para cada socio, calcular su casillero base como DNI resto 13.
2. Si está ocupado, avanzar al siguiente (de forma circular) hasta encontrar uno libre, contando los saltos.
3. Si se recorren todos sin encontrar lugar, informar que no hay casilleros.
4. Mostrar la asignación de cada socio, el total de colisiones y la ocupación final.

Salida esperada

```
Ríos (DNI 30111): casillero 3
Paz (DNI 30124): casillero 3 ocupado, va al 4 (1 saltos)
Gil (DNI 30202): casillero 3 ocupado, va al 5 (2 saltos)
Sosa (DNI 30124): casillero 3 ocupado, va al 6 (3 saltos)
Vera (DNI 30150): casillero 3 ocupado, va al 7 (4 saltos)
Luna (DNI 30176): casillero 3 ocupado, va al 8 (5 saltos)
Díaz (DNI 30137): casillero 3 ocupado, va al 9 (6 saltos)
Mota (DNI 30215): casillero 3 ocupado, va al 10 (7 saltos)
Colisiones totales: 28
Ocupación: 0:- 1:- 2:- 3:Ríos 4:Paz 5:Gil 6:Sosa 7:Vera 8:Luna 9:Díaz 10:Mota 11:- 12:-
```

Reglas

- Límites: tablas de hasta 10^6 casilleros.
- Complejidad: $O(1)$ promedio por inserción si la tabla no está muy llena; se degrada cuando se llena.
- Una colisión es cada salto que hay que dar porque un casillero ya estaba ocupado.
- La búsqueda es circular: después del último casillero se vuelve al 0.

Pista (leela solo si te trabás): esto es una tabla de hash con "sondeo lineal": la función de hash es el resto, y ante colisión se prueba el casillero siguiente. Cuanto más llena la tabla, más largos los saltos.

Bonus (opcional)

- Compará con sondeo cuadrático (saltos de 1, 4, 9...) y con encadenamiento (una lista por casillero).

- Mostrá cómo cambia el promedio de colisiones al variar el factor de carga (socios / casilleros).

Cierre: Es exactamente cómo funcionan por dentro las tablas de hash, la estructura detrás de los diccionarios y conjuntos que venís usando.

Ejercicio 218: Mesas de examen sin choques

Nivel 5: Semi-Senior II · Estrategia · Temática: Educación · Dificultad 44/100

Situación: La facultad tiene que armar las mesas de examen de la semana. Dos materias no pueden examinar en el mismo turno si hay alumnos anotados en las dos (chocarían). Quieren usar la menor cantidad posible de turnos.

Datos de entrada

Materias: Álgebra, Física, Química, Programación, Análisis

Pares que NO pueden ir en el mismo turno (comparten alumnos):

- Álgebra y Física
- Álgebra y Análisis
- Física y Química
- Química y Programación
- Programación y Análisis
- Física y Análisis

Tu programa debe

1. Armar el grafo de conflictos: cada materia es un nodo y cada par en conflicto, una arista.
2. Recorrer las materias en el orden dado y asignar a cada una el turno más chico que no usen sus materias en conflicto.
3. Mostrar la cantidad de turnos usados y qué materias hay en cada uno.

Salida esperada

```
Turnos necesarios: 3
Turno 1: Álgebra, Química
Turno 2: Física, Programación
Turno 3: Análisis
```

Reglas

- Límites: hasta 10^4 materias y 10^5 conflictos.
- Complejidad esperada: $O(\text{materias} + \text{conflictos})$ para esta asignación voraz.
- El voraz da una cantidad de turnos válida pero no siempre la mínima (el coloreo mínimo es un problema difícil): acá se pide la asignación voraz en el orden dado.
- El turno de una materia es el menor número no usado por sus vecinas ya asignadas.

Pista (leela solo si te trabás): es coloreo de grafos: cada turno es un color, y dos materias en conflicto no pueden tener el mismo color. El voraz asigna a cada nodo el color más chico disponible.

Bonus (opcional)

- Probá distintos órdenes de las materias y mostrá cómo cambia la cantidad de turnos.
- Ordená las materias por cantidad de conflictos (de mayor a menor) antes de colorear y compará.

Cierre: Es el coloreo de grafos, usado para armar horarios de exámenes, asignar frecuencias a antenas y registros a variables en un compilador.

Ejercicio 219: Estados de un turno médico

Nivel 5: Semi-Senior II · Estrategia · Temática: Salud · Dificultad 44/100

Situación: Un sistema de turnos de un centro de salud registró inconsistencias: pacientes "atendidos" que nunca se presentaron, turnos cancelados que después figuraban en sala. Modelan el ciclo de vida del turno como una máquina de estados estricta.

Datos de entrada

Estado	Evento	Nuevo estado
SOLICITADO	confirmar	CONFIRMADO
SOLICITADO	rechazar	RECHAZADO
CONFIRMADO	presentarse	EN_SALA
CONFIRMADO	cancelar	CANCELADO
CONFIRMADO	ausentarse	AUSENTE
EN_SALA	atender	ATENDIDO

Todo turno empieza en SOLICITADO. Un evento no contemplado se rechaza y no cambia el estado.

Eventos por turno:

- Turno 1: confirmar, presentarse, atender
- Turno 2: confirmar, cancelar
- Turno 3: presentarse
- Turno 4: confirmar, ausentarse, presentarse
- Turno 5: rechazar, confirmar

Tu programa debe

1. Guardar las transiciones en una estructura clave-valor (estado, evento) → nuevo estado.
2. Procesar los eventos de cada turno en orden.
3. Rechazar los eventos no permitidos sin cambiar el estado, registrando el motivo.
4. Mostrar el estado final de cada turno y los eventos rechazados.

Salida esperada

```
Turno 1: ATENDIDO
Turno 2: CANCELADO
Turno 3: SOLICITADO | presentarse no permitido en SOLICITADO
Turno 4: AUSENTE | presentarse no permitido en AUSENTE
Turno 5: RECHAZADO | confirmar no permitido en RECHAZADO
```

Reglas

- Límites: hasta 10^7 eventos.
- Complejidad esperada: $O(1)$ por evento.
- Los estados RECHAZADO, CANCELADO, AUSENTE, ATENDIDO son finales (no salen de ellos).
- Los errores de un turno se separan con "; " después de una barra.

Pista (leela solo si te trabás): es el mismo patrón que la máquina de estados del pedido online: una tabla de transiciones y ningún condicional por estado.

Bonus (opcional)

- Agregá tiempos: un turno CONFIRMADO al que no se le hace "presentarse" dentro de los 15 minutos pasa solo a AUSENTE.
- Contá cuántos turnos terminaron en cada estado final para un tablero de gestión.

Cierre: Es el modelo de estados que usan los sistemas de turnos médicos, de reservas y de tickets de soporte.

Ejercicio 220: Tarifa de taxi con recargo nocturno

Nivel 5: Semi-Senior II · Estrategia · Temática: Transporte · Dificultad 44/100

Situación: Los taxis de la ciudad cobran una bajada de bandera fija más "fichas" que suman por distancia y por tiempo de espera, con un recargo nocturno. Una app quiere mostrar la tarifa estimada antes del viaje, aplicando el reglamento tal cual.

Datos de entrada

Concepto	Valor
Bajada de bandera	\$1.200
Ficha	\$180
Metros por ficha	100 (o fracción)
Segundos de espera por ficha	30 (o fracción)
Recargo nocturno	20 %

Viaje	Metros	Espera (s)	Nocturno
Viaje 1	5400	480	No
Viaje 2	12000	900	Sí
Viaje 3	800	120	No

Tu programa debe

1. Calcular las fichas por distancia (metros / 100, redondeando hacia arriba) y por espera (segundos / 30, redondeando hacia arriba).
2. Calcular la tarifa: bajada + fichas × valor de la ficha.
3. Aplicar el recargo nocturno del 20 % si corresponde.
4. Mostrar el detalle y la tarifa de cada viaje.

Salida esperada

Viaje 1: 5400 m, 480 s -> 70 fichas -> \$13800
 Viaje 2: 12000 m, 900 s (nocturno) -> 150 fichas -> \$33840
 Viaje 3: 800 m, 120 s -> 12 fichas -> \$3360

Reglas

- Límites: hasta 10^6 viajes.
- Complejidad esperada: $O(1)$ por viaje.
- Las fichas se cuentan por unidad o fracción (150 m son 2 fichas de distancia).
- El recargo nocturno se aplica sobre el total y se redondea al peso (0,5 hacia arriba).

Pista (leela solo si te trabás): "o fracción" es una división redondeada hacia arriba: podés hacerla como $(\text{valor} + \text{tamaño} - 1) / \text{tamaño}$.

Bonus (opcional)

- Agregá una tarifa mínima: ningún viaje cuesta menos de \$2.500.
- Dividí la tarifa entre 2 o 3 pasajeros que comparten el taxi en distintos tramos.

Cierre: Es la misma lógica de tarificación que usan los relojes de taxi y las apps de viajes.

Ejercicio 221: Fotos casi iguales

Nivel 5: Semi-Senior II · Estrategia · Temática: Redes sociales · Dificultad 45/100

Situación: Una red social quiere detectar fotos casi idénticas (la misma imagen recomprimida, recortada apenas o con un filtro leve) para no mostrarlas repetidas en el feed. Cada foto se resume en una "huella" de bits, y dos fotos son casi iguales si sus huellas difieren en pocos bits.

Datos de entrada

Huella de cada foto (en este ejemplo, 12 bits):

Foto	Huella
Foto 1	111100001111
Foto 2	111100011111
Foto 3	000011110000
Foto 4	111100000111
Foto 5	101010101010
Foto 6	111100001110

Dos fotos son "casi iguales" si sus huellas difieren en 2 bits o menos (distancia de Hamming ≤ 2).

Tu programa debe

1. Escribir una función que calcule la distancia de Hamming (cantidad de bits distintos) entre dos huellas.
2. Calcular la distancia entre todos los pares de fotos.
3. Agrupar: recorrer las fotos en orden y, por cada una no agrupada, juntar todas las que estén a distancia ≤ 2 de ella.
4. Mostrar las distancias y los grupos (marcando cuáles tienen duplicadas).

Salida esperada

```
Distancias entre fotos:
Foto 1-Foto 2: 1
Foto 1-Foto 3: 12
Foto 1-Foto 4: 1
Foto 1-Foto 5: 6
Foto 1-Foto 6: 1
Foto 2-Foto 3: 11
Foto 2-Foto 4: 2
Foto 2-Foto 5: 7
Foto 2-Foto 6: 2
Foto 3-Foto 4: 11
Foto 3-Foto 5: 6
Foto 3-Foto 6: 11
Foto 4-Foto 5: 7
Foto 4-Foto 6: 2
Foto 5-Foto 6: 5
Grupos de fotos casi iguales (distancia  $\leq 2$ ):
{Foto 1, Foto 2, Foto 4, Foto 6} (duplicadas)
{Foto 3} (única)
{Foto 5} (única)
```

Reglas

- Límites: en este ejercicio, hasta 10^4 fotos; huellas de 64 bits.
- Complejidad: $O(n^2)$ para comparar todos los pares. (Con millones de fotos se usan técnicas como LSH, que vas a ver en el nivel 10.)
- La agrupación es simple: cada foto se agrupa con la primera que la "reclama".
- Casos borde: todas iguales; ninguna parecida.

Pista (leela solo si te trabás): la distancia de Hamming se calcula recorriendo los bits en paralelo y contando en cuántas posiciones difieren.

Bonus (opcional)

- Calculá la huella con XOR y contá los bits en 1 del resultado (nivel 7).
- ¿Por qué la agrupación por umbral no es transitiva? Mostrá un caso donde la foto 1 se parece a la 2 y la 2 a la 3, pero la 1 y la 3 no.

Cierre: Es la idea detrás de la detección de contenido duplicado y de los sistemas que reconocen la misma canción o imagen recomprimida.

Ejercicio 222: Cartones de bingo que suman justo

Nivel 5: Semi-Senior II · Estrategia · Temática: Videojuegos · Dificultad 45/100

Situación: Un juego de bingo temático premia a quien tape en su cartón un subconjunto de números que sume exactamente el número mágico de la ronda. Un jugador quiere saber de cuántas formas puede lograrlo con su cartón, y el desarrollador quiere que el cálculo sea rápido incluso con cartones grandes.

Datos de entrada

Número mágico: 45.

Números del cartón (todos distintos): 3, 8, 12, 15, 22, 27

Tu programa debe

1. Recorrer los números decidiendo, para cada uno, si se incluye o no en la suma.
2. Podar cuando la suma parcial ya se pasó (resto negativo) o cuando ni sumando todos los que quedan se llega al objetivo.
3. Registrar cada subconjunto que suma exactamente el objetivo.
4. Mostrar las formas y cuántos nodos exploró el backtracking con la poda.

Salida esperada

```
Formas de sumar 45: 3
  3 + 8 + 12 + 22
  3 + 15 + 27
  8 + 15 + 22
Nodos explorados con poda: 99
```

Reglas

- Límites: hasta 40 números por cartón.
- Complejidad: hasta 2^n subconjuntos sin poda; la poda por suma restante lo reduce drásticamente.
- Para que la poda "ni sumando todos alcanzo" funcione, conviene recorrer los números en el orden dado y tener a mano la suma de los que faltan.
- Casos borde: objetivo 0 (una forma: el subconjunto vacío); objetivo mayor que la suma total (0 formas).

Pista (leela solo si te trabás): dos podas hacen la diferencia: cortar si te pasaste y cortar si lo que queda por sumar no alcanza para llegar al objetivo.

Bonus (opcional)

- Solo contar las formas (sin listarlas) con programación dinámica de suma de subconjuntos (nivel 6).
- Encontrá la forma con menos números y la con más números.

Cierre: Es el problema de suma de subconjuntos, base de muchos problemas de selección y del análisis de seguridad de algunos sistemas de cifrado.

Ejercicio 223: Liquidación de sueldos según convenio

Nivel 5: Semi-Senior II · Estrategia · Temática: Recursos humanos · Dificultad 45/100

Situación: El estudio contable que liquida los sueldos de una empresa metalúrgica aplica un convenio con varias reglas por categoría: sueldo básico, adicional por antigüedad, horas extra al 50 % y presentismo. Quieren un motor de reglas que arme el bruto de cada empleado sin errores.

Datos de entrada

Categoría	Básico	Adicional por antigüedad
N1	\$900.000	20 %
N2	\$1.100.000	15 %
N3	\$1.400.000	10 %

- Jornada base: 160 horas. Valor hora = básico / 160.
- Horas extra: valor hora × 1,5 por hora extra.
- Presentismo: 8,33 % del básico, solo si no tuvo faltas.
- Bruto = sueldo por horas normales + extra + antigüedad + presentismo.

Empleado	Cat.	Horas	Extra	Faltas
Pérez	N1	176	10	0
Gómez	N2	160	0	3
Sosa	N3	184	6	0
Díaz	N1	152	0	0

Tu programa debe

1. Buscar el básico y el adicional por antigüedad según la categoría.
2. Calcular el valor hora, el sueldo por horas normales (hasta 160) y las horas extra.
3. Calcular el adicional por antigüedad y el presentismo (si no tuvo faltas).
4. Mostrar el detalle y el bruto de cada empleado.

Salida esperada

```
Pérez (cat N1):
  Sueldo por horas: $900000.00 | Extra: $84375.00
  Antigüedad: $180000.00 | Presentismo: $74970.00
  Bruto: $1239345.00
Gómez (cat N2):
  Sueldo por horas: $1100000.00 | Extra: $0.00
  Antigüedad: $165000.00 | Presentismo: $0
  Bruto: $1265000.00
Sosa (cat N3):
  Sueldo por horas: $1400000.00 | Extra: $78750.00
  Antigüedad: $140000.00 | Presentismo: $116620.00
  Bruto: $1735370.00
Díaz (cat N1):
  Sueldo por horas: $855000.00 | Extra: $0.00
  Antigüedad: $180000.00 | Presentismo: $74970.00
  Bruto: $1109970.00
```

Reglas

- Límites: hasta 10^5 empleados; hasta 100 categorías.
- Complejidad esperada: $O(\text{empleados})$.
- Cada importe se redondea a 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Si trabajó menos de 160 horas, se paga solo lo trabajado (no hay descuento adicional).

Pista (leela solo si te trabás): las categorías son un mapa: cambiar un básico o agregar una categoría no toca la lógica de cálculo. Esa separación entre datos y reglas es lo que hace mantenible una liquidación.

Bonus (opcional)

- Agregá el adicional por antigüedad escalonado según años de trabajo.
- Sumá los descuentos (jubilación, obra social) para llegar del bruto al neto, como en el ejercicio del recibo de sueldo.

Cierre: Es la estructura de un motor de liquidación de sueldos por convenio, donde las reglas cambian seguido y deben estar separadas del código.

Ejercicio 224: Comandos del chatbot de atención

Nivel 5: Semi-Senior II · Estrategia · Temática: Telecomunicaciones · Dificultad 45/100

Situación: Una empresa de telecomunicaciones tiene un chatbot de atención que interpreta mal las frases: contesta el saldo cuando le piden el último movimiento. Lo rehacen como una máquina de estados que, según en qué punto de la conversación está, busca palabras clave para decidir a qué estado ir.

Datos de entrada

Estados y palabras clave que llevan a otro estado:

- Desde SALUDO: "saldo" → SALDO, "ultimo" → MOVIMIENTO, "humano" → HUMANO, "gracias" → DESPEDIDA
- Desde SALDO: "ultimo" → MOVIMIENTO, "humano" → HUMANO, "gracias" → DESPEDIDA
- Desde MOVIMIENTO: "saldo" → SALDO, "humano" → HUMANO, "gracias" → DESPEDIDA

La conversación empieza en SALUDO. HUMANO y DESPEDIDA terminan la conversación automática.

Frases del usuario, en orden: "cual es mi saldo", "y el ultimo movimiento", "cuanto gaste", "quiero un humano", "gracias"

Tu programa debe

1. Pasar cada frase a minúsculas y buscar, entre las palabras clave válidas para el estado actual, la primera que aparezca.
2. Si ninguna coincide, responder que no entendió y ofrecer las opciones del estado actual (sin cambiar de estado).
3. Si coincide, cambiar de estado y dar la respuesta correspondiente.
4. Terminar al llegar a HUMANO o DESPEDIDA.

Salida esperada

```
"cual es mi saldo" [-> SALDO]: Tu saldo es $152.300.  
"y el ultimo movimiento" [-> MOVIMIENTO]: Tu último movimiento fue una compra de $8.400.  
"cuanto gaste" [MOVIMIENTO]: No entendí. Opciones desde MOVIMIENTO: saldo, humano, gracias  
"quiero un humano" [-> HUMANO]: Te derivo con un representante.  
(fin de la conversación automática)
```

Reglas

- Límites: hasta 10^6 mensajes por conversación; hasta 100 estados.
- Complejidad esperada: $O(\text{largo de la frase} \times \text{palabras clave del estado})$ por mensaje.
- Las palabras clave dependen del estado: la misma palabra puede no estar disponible en otro punto de la conversación.
- Se toma la primera palabra clave que aparece, en el orden definido para ese estado.

Pista (leela solo si te trabás): combinás dos cosas del nivel: una máquina de estados (dónde estoy en la conversación) y un parseo simple (qué palabra clave aparece en el mensaje).

Bonus (opcional)

- Normalizá las tildes para que "último" también active la palabra clave "ultimo".
- Agregá un contador de "no entendí" seguidos: al tercero, derivar automáticamente a HUMANO.

Cierre: Es la base de los chatbots basados en reglas y estados, previos a los que usan modelos de lenguaje.

Ejercicio 225: Cargar primero lo que más rinde

Nivel 5: Semi-Senior II · Estrategia · Temática: Clima y energía · Dificultad 45/100

Situación: Un campo aislado carga sus baterías con paneles solares, pero el inversor soporta como máximo 8.000 W conectados a la vez. Cada grupo de paneles aporta cierta potencia y carga cierta cantidad de amperios-hora por día. El encargado quiere conectar los grupos que más carguen las baterías sin pasar el límite del inversor.

Datos de entrada

Límite del inversor: 8.000 W.

Grupo de paneles	Potencia (W)	Carga diaria (Ah)
Techo norte	3200	45
Galpón	2800	40
Pérgola	1500	30
Tinglado	4000	52
Bomba solar	900	20

Un grupo se conecta entero o no se conecta.

Tu programa debe

1. Calcular el rendimiento de cada grupo (carga por vatio).
2. Ordenar de mayor a menor rendimiento (en empate, por nombre) y conectar cada grupo que entre en la potencia disponible.
3. Mostrar qué se conecta, la carga total y la potencia sin asignar.

Salida esperada

```
Bomba solar: 900 W (rinde 22.22 Ah/kW) -> +20 Ah
Pérgola: 1500 W (rinde 20.00 Ah/kW) -> +30 Ah
Galpón: 2800 W (rinde 14.29 Ah/kW) -> +40 Ah
Techo norte: 3200 W no entra (quedan 2800 W del inversor)
Tinglado: 4000 W no entra (quedan 2800 W del inversor)
Carga total: 90 Ah | Potencia sin asignar: 2800 W
```

Reglas

- Límites: hasta 10^6 grupos.
- Complejidad esperada: $O(n \log n)$.
- Rendimiento con 2 decimales (0,5 hacia arriba, en Ah por kW). Los decimales se muestran con punto y sin separador de miles en la salida.
- Como los grupos son indivisibles, el voraz por rendimiento es una heurística, no garantiza el óptimo (igual que el riego con bombas).

Pista (leela solo si te trabás): es el mismo patrón que las bombas de riego: cargar primero lo que más rinde por unidad de recurso limitado. Y la misma advertencia: con ítems indivisibles, el voraz puede no ser óptimo.

Bonus (opcional)

- Encontrá un caso donde conviene dejar afuera el grupo de mejor rendimiento.
- Resolvé el óptimo exacto con la mochila 0/1 del nivel 6.

Cierre: Es una heurística voraz para asignar un recurso indivisible limitado, común en energía, computación y logística.

Lote 10: ejercicios 226 a 250

Ejercicio 226: Reglas del firewall

Nivel 5: Semi-Senior II · Estrategia · Temática: Seguridad informática · Dificultad 46/100

Situación: El firewall de una empresa bloquea paquetes según reglas booleanas escritas por el equipo de seguridad. Cuando una regla está mal escrita, el firewall deja pasar tráfico peligroso o corta el legítimo. Te piden un evaluador que aplique las reglas exactamente y diga cuál fue la primera que bloqueó cada paquete.

Datos de entrada

Reglas (se aplican en orden; el paquete se bloquea con la primera que da verdadero):

1. origen == interna AND destino == externa
2. puerto == 22 AND NOT (origen == confiable)
3. (protocolo == tcp OR protocolo == udp) AND puerto == 53
4. NOT (origen == confiable OR destino == interna)

Operadores: AND, OR, NOT, paréntesis y comparaciones campo == valor. Precedencia: NOT, después AND, después OR.

Paquete	origen	destino	puerto	protocolo
1	interna	externa	443	tcp
2	externa	interna	22	tcp
3	confiable	interna	22	tcp
4	externa	externa	53	udp

Tu programa debe

1. Separar cada regla en tokens: paréntesis, operadores, y comparaciones campo == valor.
2. Resolver cada comparación a verdadero o falso según los datos del paquete.
3. Evaluar la expresión booleana con descenso recursivo respetando la precedencia NOT > AND > OR.
4. Para cada paquete, informar la primera regla que da verdadero (bloqueado) o que pasa.

Salida esperada

```
Paquete 1 (origen=interna destino=externa puerto=443 protocolo=tcp): BLOQUEADO por la regla 1
Paquete 2 (origen=externa destino=interna puerto=22 protocolo=tcp): BLOQUEADO por la regla 2
Paquete 3 (origen=confiable destino=interna puerto=22 protocolo=tcp): permitido
Paquete 4 (origen=externa destino=externa puerto=53 protocolo=udp): BLOQUEADO por la regla 3
```

Reglas

- Límites: hasta 10^5 reglas y 10^7 paquetes por segundo; expresiones de hasta 10^3 tokens.
- Complejidad esperada: $O(\text{tokens})$ por regla y paquete.
- NOT es unario y prefijo; AND y OR son binarios e infijos.
- Casos borde: regla sin operadores (una sola comparación); paréntesis anidados.

Pista (leela solo si te trabás): es el mismo evaluador de expresiones que el del presupuesto de obra, pero con valores booleanos y tres niveles de precedencia (OR llama a AND, que llama a NOT).

Bonus (opcional)

- Agregá los operadores de comparación !=, > y < para los puertos.
- Detectá reglas que nunca pueden dar verdadero (contradicciones) y reglas cubiertas por otras anteriores (nunca se llegan a evaluar).

Cierre: Es el mismo motor de evaluación de reglas que usan los firewalls, los sistemas de permisos y los motores de decisión.

Ejercicio 227: Dos equipos parejos

Nivel 5: Semi-Senior II · Estrategia · Temática: Deportes · Dificultad 46/100

Situación: Para un picadito de fin de año, ocho jugadores con niveles conocidos quieren armar dos equipos de cuatro lo más parejos posible, para que el partido sea entretenido. El organizador quiere minimizar la diferencia de nivel total entre los dos equipos.

Datos de entrada

Nivel de cada jugador (jugadores 1 a 8): 85, 78, 92, 70, 88, 75, 95, 82

Tu programa debe

1. Probar todas las formas de elegir la mitad de los jugadores para el equipo 1 (el resto va al equipo 2).
2. Podar las ramas que ya no pueden completar medio equipo.
3. Quedarse con la partición que minimiza la diferencia de niveles totales.
4. Mostrar los dos equipos, sus niveles y la diferencia.

Salida esperada

```
Equipo 1 (jugadores 1, 2, 5, 8): niveles [85, 78, 88, 82] = 333
Equipo 2 (jugadores 3, 4, 6, 7): niveles [92, 70, 75, 95] = 332
Diferencia de nivel: 1
```

Reglas

- Límites: hasta 30 jugadores (la mitad en cada equipo).
- Complejidad: del orden de las combinaciones de N sobre $N/2$; para 8 son 70, para 30 ya conviene un enfoque tipo meet-in-the-middle (nivel 7).
- Los dos equipos tienen exactamente la misma cantidad de jugadores.
- En empate de diferencia, se queda la primera partición encontrada.

Pista (leela solo si te trabás): fijá que el jugador va o no va al equipo 1, con la restricción de que el equipo 1 termine con la mitad. La poda evita explorar ramas que ya no pueden llegar a la mitad.

Bonus (opcional)

- Si la cantidad de jugadores es impar, permití equipos de tamaños que difieran en uno.
- Resolvé para 40 jugadores dividiendo el conjunto en dos mitades y combinando sus sumas parciales (meet in the middle).

Cierre: Es el problema de partición en dos conjuntos balanceados, que aparece en asignación de cargas, balanceo de servidores y armado de equipos.

Ejercicio 228: El crucigrama de la revista escolar

Nivel 5: Semi-Senior II · Estrategia · Temática: Educación · Dificultad 46/100

Situación: La revista de la escuela publica un crucigrama con una grilla fija (con casillas negras) y una lista de palabras para ubicar. El diseñador quiere que un programa las acomode, encajando cada palabra en un hueco del largo justo, sin dejar palabras afuera.

Datos de entrada

Grilla (# = casilla negra, . = casilla a completar):

```
...  
...  
...
```

Palabras a ubicar: SOL, OLA, ROL, ALA, OSA, ORO

Tu programa debe

1. Encontrar los huecos: tramos horizontales y verticales de 2 o más casillas blancas entre bordes o casillas negras.
2. Probar, para cada hueco, cada palabra del largo correcto que sea compatible con las letras ya puestas (por los cruces).
3. Usar backtracking: colocar una palabra, seguir con el siguiente hueco y, si no se puede completar, deshacer.
4. Mostrar la grilla resuelta o "Sin solución".

Salida esperada

```
Solución:  
OSA  
ROL  
OLA
```

Reglas

- Límites: grillas de hasta 15 × 15 y hasta 200 palabras.
- Complejidad: exponencial; la poda por largo y por letras cruzadas la hace viable para crucigramas de revista.
- Cada palabra se usa como máximo una vez.
- Las letras de los cruces tienen que coincidir en las dos palabras que se cruzan.

Pista (leela solo si te trabás): lo que hace manejable el backtracking es la restricción de los cruces: apenas colocás una palabra, muchas de las siguientes quedan descartadas por una letra que no coincide.

Bonus (opcional)

- Elegí siempre el hueco con menos palabras candidatas (heurística MRV) para acelerar la búsqueda.
- Contá cuántas soluciones distintas tiene el crucigrama.

Cierre: Es un problema de satisfacción de restricciones, la misma familia que el sudoku y la asignación de horarios.

Ejercicio 229: Impuesto progresivo por tramos

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 46/100

Situación: Un impuesto a las ganancias se calcula por tramos: la parte del ingreso que cae en cada tramo paga la tasa de ese tramo (es progresivo, no es que todo el ingreso pague la tasa del tramo más alto). Mucha gente cree que "pasar de tramo" hace que gane menos en mano, y una calculadora quiere mostrar que no es así.

Datos de entrada

Tramo del ingreso	Tasa
hasta 1.500.000	5 %
de 1.500.001 a 3.000.000	12 %
de 3.000.001 a 6.000.000	20 %
más de 6.000.000	30 %

Ingresos a calcular: 850000, 2200000, 5000000, 9000000

Tu programa debe

1. Repartir el ingreso en los tramos que correspondan.
2. Calcular el impuesto de la parte que cae en cada tramo (solo esa parte paga esa tasa).
3. Sumar el impuesto total y calcular la tasa efectiva (impuesto / ingreso).
4. Mostrar, para cada ingreso, el impuesto, la tasa efectiva y el detalle por tramo.

Salida esperada

```
Ingreso 850000: impuesto 42500.00 (tasa efectiva 5.00%)
  850000 al 5% = 42500.00
Ingreso 2200000: impuesto 159000.00 (tasa efectiva 7.23%)
  1500000 al 5% = 75000.00
  700000 al 12% = 84000.00
Ingreso 5000000: impuesto 655000.00 (tasa efectiva 13.10%)
  1500000 al 5% = 75000.00
  1500000 al 12% = 180000.00
  2000000 al 20% = 400000.00
Ingreso 9000000: impuesto 1755000.00 (tasa efectiva 19.50%)
  1500000 al 5% = 75000.00
  1500000 al 12% = 180000.00
  3000000 al 20% = 600000.00
  3000000 al 30% = 900000.00
```

Reglas

- Límites: ingresos de hasta 10^{12} .
- Complejidad esperada: $O(\text{cantidad de tramos})$ por ingreso.
- Montos y tasas efectivas con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Solo se muestran los tramos con parte del ingreso.

Pista (leela solo si te trabás): es la misma lógica que la factura de luz escalonada del nivel 1: cada tramo cobra solo lo que cae dentro de él. La tasa efectiva siempre es menor que la del tramo más alto.

Bonus (opcional)

- Mostrá que ganar \$1 más nunca reduce lo que queda en mano (calculá el ingreso neto en el borde de cada tramo).
- Agregá un mínimo no imponible: los primeros \$500.000 no pagan nada.

Cierre: Es exactamente cómo funciona el impuesto a las ganancias y otros impuestos progresivos, y desarma el mito de que conviene ganar menos para no cambiar de tramo.

Ejercicio 230: El mismo tema cargado dos veces

Nivel 5: Semi-Senior II · Estrategia · Temática: Música · Dificultad 46/100

Situación: Una plataforma de música tiene el catálogo lleno de temas cargados dos veces por distintos sellos, con títulos que difieren en mayúsculas, tildes, espacios o etiquetas como "(en vivo)". Quieren detectar los duplicados para unificarlos, comparando una versión normalizada de título y artista.

Datos de entrada

Título	Artista	Duración (s)
Marea (En vivo)	The Waves	243
marea en vivo	the waves	244
Otoño	Dúo Sur	198
MAREA EN VIVO!	THE WAVES	243
Otoño (remaster)	Dúo Sur	350
otono	duo sur	198

Normalización: pasar a minúsculas, quitar tildes (ñ → n), eliminar las etiquetas "(en vivo)", "en vivo", "(remaster)", "remaster" y los signos ! (), y colapsar espacios múltiples en uno.

Tu programa debe

1. Escribir la función de normalización de texto.
2. Agrupar los temas por la clave (título normalizado, artista normalizado), usando un mapa.
3. Para cada grupo con más de un tema, informarlo como duplicado; si las duraciones difieren, marcarlo para revisión.
4. Mostrar los duplicados y el resumen de cargados, únicos y duplicados.

Salida esperada

```
Duplicado: "Marea (En vivo)" - The Waves | "marea en vivo" - the waves | "MAREA EN VIVO!" - THE WAVES (duración distinta, revisar)
Duplicado: "Otoño" - Dúo Sur | "Otoño (remaster)" - Dúo Sur | "otono" - duo sur (duración distinta, revisar)
Temas cargados: 6 | Temas únicos: 2 | Duplicados: 4
```

Reglas

- Límites: hasta 10^7 temas.
- Complejidad esperada: $O(\text{total de caracteres})$.
- La clave es el par normalizado; los que caen en la misma clave son el mismo tema.
- Dos versiones con la misma clave pero duración muy distinta pueden no ser el mismo tema: se marcan para revisión.

Pista (leela solo si te trabás): normalizar convierte el problema difuso (¿son el mismo tema?) en uno exacto (¿tienen la misma clave?). Toda la inteligencia está en qué se ignora al normalizar.

Bonus (opcional)

- Detectá duplicados aunque el título tenga una palabra de más o de menos.
- Elegí, entre los duplicados, cuál conservar: el de mayor duración o el del sello con mejor calidad.

Cierre: Es la misma deduplicación por clave normalizada que hacen las plataformas de streaming, las tiendas y los catálogos de productos.

Ejercicio 231: El cajero automático como máquina de estados

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 47/100

Situación: Un banco moderniza el software de sus cajeros automáticos y quiere modelarlo como una máquina de estados clara, con una regla crítica: al tercer PIN incorrecto, la tarjeta se retiene. Te piden implementar el controlador y probarlo con una secuencia de eventos.

Datos de entrada

Estado	Evento	Nuevo estado
INACTIVO	insertar	TARJETA
TARJETA	pin_ok	MENU
TARJETA	pin_mal	TARJETA
TARJETA	cancelar	INACTIVO
MENU	extraer	EXTRAYENDO
MENU	cancelar	INACTIVO
EXTRAYENDO	listo	MENU
EXTRAYENDO	sin_fondos	MENU

- Estado inicial: INACTIVO. Saldo de la cuenta: \$30.000.
- Al tercer pin_mal seguido, la tarjeta se retiene (estado BLOQUEADO) y se ignoran los eventos siguientes.
- pin_ok reinicia el contador de intentos.
- extraer con monto mayor al saldo informa fondos insuficientes y vuelve a MENU.
- El evento listo entrega el efectivo del último monto extraído válido (\$20.000 en esta secuencia) y descuenta del saldo.

Eventos (evento, monto): insertar, pin_mal, pin_mal, pin_ok, extraer(50000), sin_fondos, extraer(20000), listo, cancelar, extraer(10000)

Tu programa debe

1. Modelar los estados con la tabla de transiciones y llevar el contador de PINs fallidos y el saldo.
2. Procesar cada evento aplicando la transición o rechazándolo.
3. Manejar las reglas especiales: bloqueo por 3 PINs, reinicio con pin_ok, fondos insuficientes y descuento al entregar.
4. Mostrar la respuesta a cada evento y el estado y saldo finales.

Salida esperada

```
insertar: -> TARJETA
pin_mal: PIN incorrecto (1/3)
pin_mal: PIN incorrecto (2/3)
pin_ok: -> MENU
extraer 50000: fondos insuficientes (saldo 30000), vuelve a MENU
sin_fondos: no permitido en MENU
extraer 20000: procesando...
listo: entrega efectivo, saldo 10000
cancelar: -> INACTIVO
extraer: no permitido en INACTIVO
Estado final: INACTIVO | Saldo: 10000
```

Reglas

- Límites: hasta 10^6 eventos.
- Complejidad esperada: $O(1)$ por evento.
- Un cajero BLOQUEADO ignora todo lo que sigue.

- pin_mal no cambia de estado (sigue esperando el PIN) salvo que llegue al tercero.

Pista (leela solo si te trabás): la mayoría de las transiciones salen de la tabla; solo las reglas con estado extra (intentos, saldo) necesitan código aparte. Mantené esa separación.

Bonus (opcional)

- Agregá un tiempo de espera: si pasan 30 s sin eventos en MENU, vuelve a INACTIVO por seguridad.
- Registrá el historial para poder auditar qué pasó con una tarjeta retenida.

Cierre: Es el modelo de estados que hay detrás de cualquier cajero automático o terminal de autoservicio.

Ejercicio 232: Menú de pasos sin ingredientes repetidos

Nivel 5: Semi-Senior II · Estrategia · Temática: Gastronomía · Dificultad 47/100

Situación: Un restaurante de bowls arma menús eligiendo una opción de cada paso (base, salsa, topping). No puede repetir ningún ingrediente entre pasos y hay combinaciones prohibidas por sabor (el ajo con la palta, dice el chef). Quieren listar todos los menús válidos.

Datos de entrada

Paso	Ingredientes de la opción
Base	papa
Base	arroz
Salsa	ajo, tomate
Salsa	ajo, crema
Topping	queso
Topping	palta
Topping	tomate

Combinaciones prohibidas: ajo + palta.

Tu programa debe

1. Agrupar las opciones por paso (base, salsa, topping), en el orden de aparición.
2. Elegir una opción por paso con backtracking.
3. Descartar una opción si repite un ingrediente ya usado o si crea una combinación prohibida.
4. Listar todos los menús válidos.

Salida esperada

```
Menús válidos: 6
Base[papa] + Salsa[ajo, tomate] + Topping[queso]
Base[papa] + Salsa[ajo, crema] + Topping[queso]
Base[papa] + Salsa[ajo, crema] + Topping[tomate]
Base[arroz] + Salsa[ajo, tomate] + Topping[queso]
Base[arroz] + Salsa[ajo, crema] + Topping[queso]
Base[arroz] + Salsa[ajo, crema] + Topping[tomate]
```

Reglas

- Límites: hasta 20 pasos con hasta 20 opciones cada uno.
- Complejidad: producto de la cantidad de opciones por paso; la poda por ingredientes repetidos y prohibiciones la reduce.
- Un ingrediente no puede aparecer en dos pasos del mismo menú.
- Los ingredientes de cada opción se listan en orden alfabético.

Pista (leela solo si te trabás): Llevá un conjunto de ingredientes ya usados en el menú que estás armando: preguntar "¿esta opción choca?" es una intersección de conjuntos.

Bonus (opcional)

- Agregá restricciones de dieta (sin TACC, vegano) como filtros combinables.
- Contá los menús sin listarlos cuando hay muchos pasos.

Cierre: Es la generación de combinaciones con restricciones, usada en configuradores de productos (autos, computadoras, menús) que arman opciones válidas.

Ejercicio 233: Paradas de carga del auto eléctrico

Nivel 5: Semi-Senior II · Estrategia · Temática: Transporte · Dificultad 47/100

Situación: Una familia viaja 520 km en su auto eléctrico, que carga a full para 350 km de autonomía. En la ruta hay estaciones de carga a distintas distancias. Quieren parar la menor cantidad de veces posible, cargando siempre a full en cada parada.

Datos de entrada

Distancia total: 520 km. Autonomía con carga full: 350 km. Salen con el auto cargado desde el km 0.

Estaciones de carga en los kilómetros: 80, 150, 220, 300, 410, 480

Tu programa debe

1. Avanzar mientras la autonomía alcance; cuando no llegue al destino, elegir la estación más lejana que todavía esté al alcance.
2. Cargar a full ahí (la autonomía pasa a estación + 350) y contar la parada.
3. Repetir hasta llegar o hasta que no haya ninguna estación al alcance.
4. Mostrar la cantidad de paradas y dónde, o que no se puede llegar.

Salida esperada

Paradas de carga necesarias: 1
Carga en km 300 (autonomía hasta km 650)

Reglas

- Límites: hasta 10^6 estaciones; distancias hasta 10^9 .
- Complejidad esperada: $O(\text{estaciones})$.
- Justificación: parar lo más tarde posible (en la estación más lejana al alcance) minimiza las paradas; adelantar una parada nunca ayuda.
- Casos borde: se llega sin parar; hay un tramo entre estaciones mayor que la autonomía (no se puede).

Pista (leela solo si te trabás): el voraz acá es "estirá cada carga lo más posible": entre todas las estaciones a las que llegás, elegí la más lejana. Cargar antes solo agrega paradas.

Bonus (opcional)

- Si no hace falta cargar a full (podés cargar lo justo), minimizá la energía total cargada en lugar de la cantidad de paradas.
- Agregá el tiempo de cada carga (proporcional a lo que cargás) y minimizá el tiempo total de viaje.

Cierre: Es el problema de las estaciones de servicio, un clásico voraz que usan los planificadores de rutas de vehículos eléctricos.

Ejercicio 234: Aprobación de créditos por puntaje

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 47/100

Situación: Una fintech automatiza la primera etapa de aprobación de créditos con un sistema de puntaje (scoring): cada regla cumplida suma puntos, y el total decide si se aprueba, se manda a revisión manual o se rechaza. Quieren que el sistema explique qué reglas sumaron, para poder auditar cada decisión.

Datos de entrada

Regla	Puntos
Ingreso mensual mayor a 3 veces la cuota	30
Antigüedad laboral de 12 meses o más	20
Sin deudas en el sistema financiero	25
Cliente hace más de 2 años	15
Edad entre 25 y 60	10

Decisión: 70 o más → APROBADO; de 50 a 69 → REVISIÓN MANUAL; menos de 50 → RECHAZADO.

Solicitante	Ingreso	Cuota	Antig. (meses)	Deudas	Años cliente	Edad
Ana	900.000	250.000	18	No	3	34
Beto	600.000	250.000	8	Sí	1	22
Caro	1.200.000	300.000	24	No	5	61

Tu programa debe

1. Definir cada regla con su condición y su puntaje.
2. Para cada solicitante, evaluar todas las reglas y sumar los puntos de las que cumple.
3. Aplicar la decisión según el total.
4. Mostrar el puntaje, la decisión y el detalle de las reglas que sumaron.

Salida esperada

```
Ana: 100 puntos -> APROBADO
    +30 Ingreso mensual mayor a 3 veces la cuota
    +20 Antigüedad laboral de 12 meses o más
    +25 Sin deudas en el sistema financiero
    +15 Cliente hace más de 2 años
    +10 Edad entre 25 y 60
Beto: 0 puntos -> RECHAZADO
Caro: 90 puntos -> APROBADO
    +30 Ingreso mensual mayor a 3 veces la cuota
    +20 Antigüedad laboral de 12 meses o más
    +25 Sin deudas en el sistema financiero
    +15 Cliente hace más de 2 años
```

Reglas

- Límites: hasta 10^6 solicitudes; hasta 100 reglas.
- Complejidad esperada: $O(\text{solicitudes} \times \text{reglas})$.
- Las reglas están separadas de la lógica de decisión: cambiar un puntaje o agregar una regla no toca el resto.
- El detalle lista las reglas que sumaron, en el orden en que están definidas.

Pista (leela solo si te trabás): modelá cada regla como una condición + un puntaje. El motor recorre las reglas sin saber qué evalúa cada una; eso permite cambiar las reglas sin tocar el motor.

Bonus (opcional)

- Agregá reglas excluyentes: una deuda en default rechaza directamente, sin importar el puntaje.
- Calibrá los umbrales para que cierto porcentaje de solicitudes caiga en cada categoría.

Cierre: Es la estructura de los motores de scoring crediticio que usan bancos y fintechs para la primera capa de decisión.

Ejercicio 235: Validar fórmulas químicas

Nivel 5: Semi-Senior II · Estrategia · Temática: Ciencia · Dificultad 47/100

Situación: Un laboratorio de química de la universidad tiene un software que calcula la masa molar de un compuesto a partir de su fórmula, pero se cuelga con los paréntesis anidados y no avisa cuando la fórmula está mal escrita. Te piden un evaluador robusto que use una pila.

Datos de entrada

Masas atómicas: H=1, O=16, Na=23, Cl=35, C=12, Ca=40, Al=27, S=32, Mg=24, N=14, K=39, Fe=56

Fórmulas a evaluar: H2O, NaCl, C6H12O6, Ca(OH)2, Al2(SO4)3, Mg(NO3)2, K4[Fe(CN)6], (NH4)2SO4, H2O), Fe2(O3

Una fórmula es una secuencia de elementos (símbolo con mayúscula y posibles minúsculas) con un número opcional, y grupos entre (), [] o { } con un número que multiplica todo el grupo.

Tu programa debe

1. Recorrer la fórmula con una pila de sumas parciales: al abrir un grupo, apilar una suma nueva.
2. Al leer un elemento, sumar masa atómica × cantidad a la suma del tope.
3. Al cerrar un grupo, multiplicar su suma por el número que sigue y sumarla al grupo de abajo.
4. Detectar errores (cierre sin apertura, grupo sin cerrar, elemento o carácter inválido) y mostrar la masa molar o el error.

Salida esperada

```
H2O: masa molar 18 g/mol
NaCl: masa molar 58 g/mol
C6H12O6: masa molar 180 g/mol
Ca(OH)2: masa molar 74 g/mol
Al2(SO4)3: masa molar 342 g/mol
Mg(NO3)2: masa molar 148 g/mol
K4[Fe(CN)6]: masa molar 368 g/mol
(NH4)2SO4: masa molar 132 g/mol
H2O): ERROR (cierre sin apertura)
Fe2(O3: ERROR (falta cerrar un grupo)
```

Reglas

- Límites: fórmulas de hasta 10^5 caracteres.
- Complejidad esperada: $O(n)$.
- Un número después de un elemento o de un grupo indica cuántas veces se repite (1 si no hay número).
- Los tres tipos de paréntesis se tratan igual para la anidación (no hace falta que coincida el tipo, pero sí que estén balanceados).

Pista (leela solo si te trabas): la pila guarda la suma parcial de cada nivel de anidación. Cerrar un grupo es multiplicar el tope por su número y volcarlo en el nivel de abajo: la misma idea que evaluar expresiones con paréntesis.

Bonus (opcional)

- Exigí que el tipo de paréntesis de cierre coincida con el de apertura.
- Balancea una ecuación química sencilla (los coeficientes) planteando un sistema de ecuaciones (nivel 9).

Cierre: Es la evaluación con pila de estructuras anidadas, la misma técnica detrás de los parsers de fórmulas, JSON y lenguajes de marcado.

Ejercicio 236: Tareas del sprint con fecha límite

Nivel 5: Semi-Senior II · Estrategia · Temática: Recursos humanos · Dificultad 47/100

Situación: Un equipo de desarrollo tiene varias tareas para el sprint, cada una con una duración estimada (en días) y una fecha límite. Trabajan en una sola tarea por vez. Quieren ordenarlas para cumplir la mayor cantidad de fechas límite posible.

Datos de entrada

Tarea	Duración (días)	Fecha límite (día)
Login	3	5
Reportes	2	3
Búsqueda	5	8
Notificaciones	1	2
Exportar	4	6
Perfil	2	4
Dashboard	3	10

Tu programa debe

1. Ordenar las tareas por fecha límite ascendente (en empate, la más larga primero).
2. Ejecutarlas en ese orden, acumulando el tiempo transcurrido.
3. Marcar cada tarea como a tiempo o tarde según si termina antes de su límite.
4. Mostrar el orden de finalización y cuántas se cumplieron.

Salida esperada

```
Orden de ejecución (por fecha límite):
Notificaciones: termina en día 1 (límite 2)
Reportes: termina en día 3 (límite 3)
Perfil: termina en día 5 (límite 4) TARDE
Login: termina en día 8 (límite 5) TARDE
Exportar: termina en día 12 (límite 6) TARDE
Búsqueda: termina en día 17 (límite 8) TARDE
Dashboard: termina en día 20 (límite 10) TARDE
Tareas a tiempo: 2 de 7
```

Reglas

- Límites: hasta 10^6 tareas.
- Complejidad esperada: $O(n \log n)$.
- Ordenar por fecha límite es una heurística razonable para este planteo simple; el óptimo exacto para maximizar tareas a tiempo requiere un enfoque con heap (ver el bonus).
- Una tarea que termina justo en su día límite está a tiempo.

Pista (leela solo si te trabás): atacar primero lo que vence antes es una regla voraz clásica. Para el óptimo real, cuando una tarea no entra conviene descartar la más larga de las ya elegidas.

Bonus (opcional)

- Maximizá las tareas a tiempo con el algoritmo óptimo: recorré por fecha límite y, si una no entra, sacá del conjunto elegido la de mayor duración (usá un heap, nivel 6).
- Si cada tarea tiene una penalización por atraso, minimizá la penalización total.

Cierre: Es la planificación de tareas con plazos, base de los planificadores de proyectos y de procesos.

Ejercicio 237: El saludo de tres pasos entre módems

Nivel 5: Semi-Senior II · Estrategia · Temática: Telecomunicaciones · Dificultad 48/100

Situación: Dos módems establecen una conexión con un saludo de tres pasos (como el handshake de TCP): el que inicia manda SYN, el otro responde SYN-ACK y el primero confirma con ACK. Un analizador de red quiere verificar, desde el lado del servidor, qué conexiones completaron bien el saludo y cuáles quedaron a medias o recibieron mensajes fuera de orden.

Datos de entrada

Estados del lado servidor:

Estado	Mensaje	Nuevo estado
CERRADO	SYN	SYN_RECIBIDO
SYN_RECIBIDO	ACK	ESTABLECIDO

Todo empieza en CERRADO. Un mensaje inesperado se registra y no cambia el estado. El saludo está completo si se llega a ESTABLECIDO.

Mensajes recibidos por conexión:

- Conexión 1: SYN, ACK
- Conexión 2: ACK
- Conexión 3: SYN, SYN, ACK
- Conexión 4: SYN

Tu programa debe

1. Modelar los estados del servidor con la tabla de transiciones.
2. Procesar los mensajes de cada conexión en orden.
3. Registrar los mensajes inesperados sin cambiar el estado.
4. Informar si el handshake se completó (ESTABLECIDO) o quedó incompleto, y los errores.

Salida esperada

```
Conexión 1: ESTABLECIDO (handshake completo)
Conexión 2: CERRADO (incompleto) | ACK inesperado en CERRADO
Conexión 3: ESTABLECIDO (handshake completo) | SYN inesperado en SYN_RECIBIDO
Conexión 4: SYN_RECIBIDO (incompleto)
```

Reglas

- Límites: hasta 10^7 conexiones.
- Complejidad esperada: $O(\text{mensajes})$ por conexión.
- Un ACK en CERRADO es un mensaje inesperado (llegó sin el SYN previo).
- Un segundo SYN estando en SYN_RECIBIDO es inesperado (retransmisión que el modelo simple no espera).

Pista (leela solo si te trabás): es una máquina de estados de tres estados; lo importante es que un mensaje fuera de orden no rompe la conexión, solo se registra.

Bonus (opcional)

- Agregá el cierre de la conexión (FIN, FIN-ACK, ACK) como una segunda máquina de estados.
- Manejá las retransmisiones: un SYN repetido no es un error, se reconoce igual.

Cierre: Es una versión didáctica del handshake de TCP, el saludo que inicia casi todas las conexiones de internet.

Ejercicio 238: Aulas mínimas para la cursada

Nivel 5: Semi-Senior II · Estrategia · Temática: Educación · Dificultad 48/100

Situación: Una facultad tiene que garantizar un aula para cada materia de la franja de la mañana. Muchas se superponen. Quieren saber la cantidad mínima de aulas para que ninguna materia se quede sin lugar.

Datos de entrada

Materia	Inicio	Fin
Análisis	08:00	10:00
Álgebra	08:00	09:30
Física	09:00	11:00
Química	10:00	12:00
Programación	10:30	12:30
Historia	11:30	13:00
Inglés	12:00	13:30

Una materia que termina a la misma hora en que empieza otra libera el aula a tiempo.

Tu programa debe

1. Crear un evento +1 por cada inicio y −1 por cada fin.
2. Ordenar los eventos por hora; a igual hora, los fines antes que los inicios.
3. Recorrerlos acumulando las materias en curso y registrar el máximo.
4. El máximo es la cantidad mínima de aulas.

Salida esperada

Aulas mínimas necesarias: 3
(Cantidad máxima de materias en curso al mismo tiempo)

Reglas

- Límites: hasta 10^6 materias.
- Complejidad esperada: $O(n \log n)$.
- Justificación: nunca hacen falta más aulas que la mayor cantidad de materias simultáneas, y esa cantidad siempre se necesita.
- Los fines antes que los inicios a la misma hora implementan la regla de liberar el aula a tiempo.

Pista (leela solo si te trabás): es el mismo barrido de eventos que el de las llamadas simultáneas del nivel 4: la respuesta es el pico de solapamiento.

Bonus (opcional)

- Asigná un número de aula concreto a cada materia, reutilizando las que se liberan (heap de aulas libres, nivel 6).
- Si algunas materias se pueden mover de horario, minimizá las aulas moviendo la menor cantidad de materias.

Cierre: Es el problema de la cantidad mínima de salas, usado para asignar aulas, quirófanos, andenes y servidores.

Ejercicio 239: Política de cancelación de reservas

Nivel 5: Semi-Senior II · Estrategia · Temática: Turismo · Dificultad 48/100

Situación: Un hotel tiene una política de cancelación escalonada según cuántos días antes del check-in se cancela. El sistema de reservas tiene que calcular el cargo y el reembolso de cada cancelación, y hay reclamos porque a veces cobra de más. El check-in de todas estas reservas es el 20/12/2026.

Datos de entrada

Política (días de anticipación de la cancelación respecto del check-in):

Anticipación	Cargo
30 días o más	sin cargo
14 a 29 días	25 %
7 a 13 días	50 %
menos de 7 días	100 % (sin reembolso)

Check-in: 20/12/2026. Todas las reservas son de \$180.000.

Reserva	Fecha de cancelación
R1	05/12/2026
R2	12/12/2026
R3	17/12/2026
R4	19/12/2026

Tu programa debe

1. Calcular los días de anticipación de cada cancelación respecto del check-in.
2. Aplicar el tramo de la política que corresponde.
3. Calcular el cargo y el reembolso.
4. Mostrar el detalle de cada reserva.

Salida esperada

```
R1: cancela 15 días antes -> cargo 25% = $45000, reembolso $135000
R2: cancela 8 días antes -> cargo 50% = $90000, reembolso $90000
R3: cancela 3 días antes -> cargo 100% (sin reembolso) = $180000, reembolso $0
R4: cancela 1 días antes -> cargo 100% (sin reembolso) = $180000, reembolso $0
```

Reglas

- Límites: hasta 10^6 reservas.
- Complejidad esperada: $O(\text{reservas})$, con el cálculo de días entre fechas.
- Los límites de los tramos son inclusivos hacia el tramo de menor cargo (exactamente 30 días es sin cargo; exactamente 14, es 25 %).
- El monto es entero; el cargo se calcula con división entera del porcentaje.

Pista (leela solo si te trabás): reutilizá el cálculo de días entre fechas de los ejercicios del nivel 3. El resto es aplicar el tramo correcto, como en cualquier tabla escalonada.

Bonus (opcional)

- Agregá tarifas no reembolsables (siempre 100 % de cargo) y tarifas flexibles (siempre sin cargo).
- Calculá cuánto pierde el hotel en promedio si el 20 % de las reservas se cancela a menos de 7 días.

Cierre: Es la lógica de las políticas de cancelación de hoteles, aerolíneas y plataformas de reservas.

Ejercicio 240: El ahorcado con estados

Nivel 5: Semi-Senior II · Estrategia · Temática: Videojuegos · Dificultad 48/100

Situación: Un estudio hace una versión digital del ahorcado. El motor del juego tiene que ir revelando las letras acertadas, descontar una vida por cada letra que no está, y terminar el juego al ganar (todas reveladas) o al quedarse sin vidas. Te piden implementarlo y probarlo con una secuencia de intentos.

Datos de entrada

Palabra oculta: programacion. Vidas: 6.

Letras intentadas, en orden: 'p', 'o', 'r', 'a', 'e', 'i', 'u', 'x', 'z', 'm', 'g'

Tu programa debe

1. Llevar el conjunto de letras reveladas y la cantidad de vidas.
2. Para cada letra intentada: si está en la palabra, revelarla; si no, descontar una vida.
3. Después de cada intento, mostrar el estado de la palabra (letras reveladas y guiones bajos).
4. Terminar al revelar todas las letras (ganó) o al llegar a 0 vidas (game over).

Salida esperada

```
'p': bien -> p_____
'o': bien -> p_o_____o_
'r': bien -> pro_r_____o_
'a': bien -> pro_ra_a_o_
'e': mal -> pro_ra_a_o_ (vidas: 5)
'i': bien -> pro_ra_a_io_
'u': mal -> pro_ra_a_io_ (vidas: 4)
'x': mal -> pro_ra_a_io_ (vidas: 3)
'z': mal -> pro_ra_a_io_ (vidas: 2)
'm': bien -> pro_rama_io_
'g': bien -> programa_io_
```

Reglas

- Límites: palabras de hasta 10^5 letras; hasta 10^6 intentos.
- Complejidad esperada: $O(1)$ por intento (con la palabra en un conjunto de letras).
- Una letra que ya se acertó y se repite no cambia nada.
- El juego termina exactamente al ganar o perder; los intentos posteriores se ignoran.

Pista (leela solo si te trabás): el estado del juego son dos cosas: qué letras se revelaron y cuántas vidas quedan. Todo lo demás se deriva de ahí.

Bonus (opcional)

- Penalizá repetir una letra ya intentada (acertada o no) con la pérdida de una vida.
- Agregá pistas: revelar una letra al azar a cambio de una vida.

Cierre: Es una máquina de estados con un contador, el mismo patrón de cualquier juego con vidas y condición de victoria.

Ejercicio 241: Cupones incompatibles

Nivel 5: Semi-Senior II · Estrategia · Temática: E-commerce · Dificultad 48/100

Situación: Una tienda online lanzó varios cupones y algunos no se pueden combinar entre sí. El cliente quiere aplicar el conjunto de cupones que le dé el mayor ahorro total, respetando las incompatibilidades. La tienda quiere que el carrito calcule esa combinación automáticamente.

Datos de entrada

Precio del producto: \$50.000.

Cupón	Descuento	Incompatible con
BIENVENIDA	\$8.000	—
PRIMERACOMPRA	\$12.000	OTROS
OTROS	\$5.000	PRIMERACOMPRA
ENVIOGRATIS	\$6.000	—
BLACKFRIDAY	\$15.000	BIENVENIDA, ENVIOGRATIS

Tu programa debe

1. Recorrer los cupones decidiendo, para cada uno, incluirlo o no.
2. Descartar la inclusión de un cupón que sea incompatible con alguno de los ya elegidos.
3. Quedarse con la combinación de mayor ahorro total.
4. Mostrar los cupones elegidos, el ahorro y el precio final.

Salida esperada

Mejor combinación de cupones: PRIMERACOMPRA, BLACKFRIDAY
Ahorro total: \$27000
Precio final: \$23000

Reglas

- Límites: hasta 25 cupones (el espacio de combinaciones es 2^{25}).
- Complejidad: exponencial (2^n); con más cupones se necesita programación dinámica sobre grafos de compatibilidad.
- La incompatibilidad es simétrica: si A es incompatible con B, B lo es con A.
- En empate de ahorro, se queda la primera combinación encontrada.

Pista (leela solo si te trabás): es como el problema de armar un subconjunto sin conflictos maximizando un valor. El backtracking prueba incluir o no cada cupón y poda cuando hay incompatibilidad.

Bonus (opcional)

- Modelalo como un grafo donde los cupones incompatibles están unidos por una arista: buscás el conjunto independiente de mayor peso.
- Agregá un tope: el descuento total no puede superar el 40 % del precio.

Cierre: Es un problema de selección con restricciones de incompatibilidad, presente en promociones, asignación de recursos y planificación.

Ejercicio 242: Filtros con comodines

Nivel 5: Semi-Senior II · Estrategia · Temática: Seguridad informática · Dificultad 49/100

Situación: Un sistema de seguridad aplica reglas a los nombres de archivo con comodines: * (cualquier cantidad de caracteres, incluso ninguno) y ? (exactamente un carácter). El equipo necesita un comparador correcto, porque una regla mal interpretada puede dejar pasar un archivo peligroso.

Datos de entrada

Comodines: * = cualquier secuencia (incluso vacía); ? = un carácter exacto.

Comparaciones a resolver:

- archivo*.log vs archivo2026.log
- archivo*.log vs archivo.txt
- data_?.csv vs data_01.csv
- data_?.csv vs data_1.csv
- *.jpg vs foto.jpg
- back*up*.zip vs backup.zip
- back*up*.zip vs back_full_up_v2.zip
- a*b*c vs aXbYc
- a*b*c vs abX

Tu programa debe

1. Comparar el patrón con el texto recorriendo ambos.
2. Cuando el patrón tiene *, probar que consume 0 o más caracteres del texto (acá aparece la ramificación, resoluble con backtracking o con una tabla de coincidencias).
3. ? coincide con cualquier carácter; un carácter común tiene que coincidir exacto.
4. Informar, para cada par, si coincide o no.

Salida esperada

```
'archivo*.log' vs 'archivo2026.log': coincide
'archivo*.log' vs 'archivo.txt': no coincide
'data_?.csv' vs 'data_01.csv': coincide
'data_?.csv' vs 'data_1.csv': no coincide
'*.jpg' vs 'foto.jpg': coincide
'back*up*.zip' vs 'backup.zip': coincide
'back*up*.zip' vs 'back_full_up_v2.zip': coincide
'a*b*c' vs 'aXbYc': coincide
'a*b*c' vs 'abX': no coincide
```

Reglas

- Límites: patrones y textos de hasta 10^4 caracteres.
- Complejidad esperada: $O(\text{largo del patrón} \times \text{largo del texto})$ con la tabla; el backtracking ingenuo puede ser exponencial con muchos *.
- * coincide con la cadena vacía: 'a*' coincide con 'a'.
- Casos borde: patrón solo de *; texto vacío; patrón sin comodines.

Pista (leela solo si te trabás): el * es el que ramifica: puede consumir un carácter más o cerrar acá. Si lo resolvés con una tabla ($\text{coincide}[i][j] = \text{¿los primeros } i \text{ del patrón matchean los primeros } j \text{ del texto?}$), evitás la explosión del backtracking.

Bonus (opcional)

- Agregá clases de caracteres como [a-z] o [0-9].
- Contá cuántos nombres de una lista coinciden con un patrón dado.

Cierre: Es el motor de coincidencia con comodines (glob) que usan las terminales, los buscadores de archivos y las reglas de firewall.

Ejercicio 243: Rotación de cultivos con restricciones

Nivel 5: Semi-Senior II · Estrategia · Temática: Agricultura · Dificultad 49/100

Situación: Un ingeniero agrónomo planifica qué sembrar en cuatro parcelas contiguas. Para cuidar el suelo, no puede repetir el mismo cultivo en parcelas vecinas si es soja o maíz (esquilman la tierra), y al menos una de las cuatro parcelas tiene que quedar en descanso. Quiere ver todas las rotaciones válidas.

Datos de entrada

Cultivos posibles: Soja, Maíz, Trigo, Descanso (4 parcelas en fila).

Restricciones: no soja seguida de soja ni maíz seguido de maíz en parcelas vecinas; al menos una parcela en descanso.

Tu programa debe

1. Asignar un cultivo a cada parcela, en orden, con backtracking.
2. Podar si un cultivo viola la restricción con la parcela vecina anterior.
3. Al completar las 4 parcelas, aceptar la rotación solo si incluye al menos un descanso.
4. Mostrar la cantidad de rotaciones válidas y las primeras cinco.

Salida esperada

Rotaciones válidas para 4 parcelas consecutivas: 137

Primeras 5:

```
Soja -> Maíz -> Soja -> Descanso
Soja -> Maíz -> Trigo -> Descanso
Soja -> Maíz -> Descanso -> Soja
Soja -> Maíz -> Descanso -> Maíz
Soja -> Maíz -> Descanso -> Trigo
```

Reglas

- Límites: hasta 12 parcelas y 10 cultivos.
- Complejidad: hasta $\text{cultivos}^{\text{parcelas}}$; la poda por vecindad la reduce.
- La restricción de descanso se verifica al final (es una condición global, no local).
- Las rotaciones se listan en el orden en que las genera el recorrido (cultivos en el orden dado).

Pista (leela solo si te trabás): las restricciones entre vecinos se podan en cada paso; la de "al menos un descanso" es global, así que se chequea recién cuando la secuencia está completa.

Bonus (opcional)

- Agregá una restricción de historia: ninguna parcela puede tener el mismo cultivo que tuvo el año pasado (dato por parcela).
- Maximizá la ganancia esperada asignando un rinde a cada cultivo.

Cierre: Es la planificación de rotación de cultivos, un problema de asignación con restricciones que también aparece en horarios y coloreo.

Ejercicio 244: Saltos entre antenas repetidoras

Nivel 5: Semi-Senior II · Estrategia · Temática: Telecomunicaciones · Dificultad 49/100

Situación: Una cooperativa de internet quiere dar cobertura a una ruta de 100 km encendiendo la menor cantidad de antenas repetidoras. Cada antena cubre 25 km hacia adelante desde su posición. La cobertura tiene que ser continua desde el km 0.

Datos de entrada

Ruta: de km 0 a km 100. Alcance de cada antena: 25 km hacia adelante.

Antenas disponibles en los kilómetros: 10, 20, 30, 45, 60, 62, 80, 95

Tu programa debe

1. Llevar hasta qué kilómetro está cubierta la ruta (empieza en 0).
2. Entre las antenas que cubren el próximo tramo (posición $\leq$ cobertura actual), elegir la más lejana.
3. Extender la cobertura hasta esa antena + 25 y contarla.
4. Repetir hasta cubrir los 100 km o hasta que no haya antena al alcance.

Salida esperada

Antenas encendidas: 3 (en km 20, 62, 95)
Cobertura desde km 0 hasta km 120 (objetivo: 100)

Reglas

- Límites: hasta 10^6 antenas; ruta de hasta 10^9 km.
- Complejidad esperada: $O(\text{antenas})$ si están ordenadas.
- Justificación: elegir la antena más lejana que aún cubre el punto pendiente maximiza el avance de cada antena encendida.
- Casos borde: un tramo sin ninguna antena al alcance (no se puede cubrir).

Pista (leela solo si te trabás): es el mismo voraz que las paradas de carga del auto eléctrico: estirá cada elección lo máximo posible. Acá 'estirar' es elegir la antena más adelantada que todavía tapa el hueco.

Bonus (opcional)

- Si cada antena tiene un costo distinto, minimizá el costo total en lugar de la cantidad (ya no es voraz simple).
- Permití antenas que cubren hacia ambos lados (un radio en vez de un alcance hacia adelante).

Cierre: Es el problema de cobertura de intervalos, usado para ubicar antenas, sensores, cámaras y estaciones de servicio.

Ejercicio 245: Fórmulas de precios con variables

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 49/100

Situación: Una fintech tiene una planilla de cálculo de precios donde cada celda es una fórmula que puede usar otras celdas ya definidas, como "total = subtotal + envio". Necesitan un motor que evalúe las fórmulas en orden, resolviendo las referencias a otras variables.

Datos de entrada

Definiciones (en orden; cada una puede usar variables ya definidas):

```
base = 1000
iva = base * 0.21
envio = 500
subtotal = base + iva
total = subtotal + envio
descuento = total * 0.1
final = total - descuento
```

Tu programa debe

1. Para cada definición, separar el nombre de la variable y su expresión.
2. Evaluar la expresión reemplazando las variables por sus valores ya calculados y respetando la precedencia ($\times$ y $/$ antes que $+$ y $-$).
3. Guardar el valor de la variable para las definiciones siguientes.
4. Mostrar cada variable con su valor con 2 decimales.

Salida esperada

```
base = 1000.00
iva = 210.00
envio = 500.00
subtotal = 1210.00
total = 1710.00
descuento = 171.00
final = 1539.00
```

Reglas

- Límites: hasta 10^5 definiciones.
- Complejidad esperada: $O(\text{total de tokens})$.
- Las variables se usan solo después de definirse (en este ejercicio no hay referencias hacia adelante).
- Valores con 2 decimales (0,5 hacia arriba), calculando con decimales exactos. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): es el evaluador de expresiones del ejercicio del presupuesto de obra, más un mapa de variables: cuando un token es un nombre, lo reemplazás por su valor guardado.

Bonus (opcional)

- Detectá el uso de una variable no definida y reportá el error con el nombre.
- Permití referencias en cualquier orden y resolvé las dependencias (esto lleva al intérprete de la hoja de cálculo del desafío 249).

Cierre: Es el corazón de una hoja de cálculo: fórmulas que referencian otras celdas y se evalúan en cadena.

Ejercicio 246: El city tour con presupuesto

Nivel 5: Semi-Senior II · Estrategia · Temática: Turismo · Dificultad 49/100

Situación: Una app de turismo arma un city tour para un visitante que tiene un presupuesto y un tiempo limitados. Cada lugar cuesta una entrada, lleva unas horas y tiene un puntaje de interés. La app quiere maximizar el interés total del recorrido sin pasarse ni de plata ni de tiempo.

Datos de entrada

Presupuesto: \$100. Tiempo disponible: 8 h.

Lugar	Costo	Horas	Interés
Museo	\$30	2	50
Parque	\$0	3	40
Feria	\$25	2	45
Mirador	\$15	1	35
Teatro	\$60	3	80

Tu programa debe

1. Recorrer los lugares decidiendo, para cada uno, si se incluye o no.
2. Podar las ramas que se pasan del presupuesto o del tiempo.
3. Quedarse con la combinación de mayor interés total.
4. Mostrar el recorrido, el interés total, el costo y el tiempo usados.

Salida esperada

Mejor recorrido: Museo, Parque, Feria, Mirador
Valor (interés) total: 170
Costo: \$70 de \$100 | Tiempo: 8 h de 8 h

Reglas

- Límites: hasta 30 lugares (2^{30} combinaciones en el peor caso, viable con poda).
- Complejidad: exponencial; la poda por presupuesto y tiempo la reduce. Con más lugares se usa programación dinámica (mochila con dos restricciones, nivel 6).
- Es una mochila 0/1 con dos límites (dinero y tiempo).
- En empate de interés, se queda la primera combinación encontrada.

Pista (leela solo si te trabás): es una mochila con dos capacidades. La poda es doble: cortás una rama si te pasaste del dinero o del tiempo.

Bonus (opcional)

- Resuélvelo con programación dinámica sobre las dos dimensiones (nivel 6) y compará con el backtracking.
- Agregá horarios de apertura: un lugar solo se puede visitar en cierta franja.

Cierre: Es la mochila con múltiples restricciones, usada en armado de itinerarios, selección de inversiones y planificación de recursos.

Ejercicio 247: Transacciones duplicadas por reintento

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 49/100

Situación: Una pasarela de pagos recibe a veces la misma transacción dos veces porque la app del comercio, al no recibir respuesta a tiempo, reintenta. Hay que cobrar una sola vez: si llega una transacción con el mismo id y monto dentro de los 2 minutos de una ya vista, es un duplicado; si pasa más tiempo, se considera un reintento legítimo (el cliente realmente volvió a pagar).

Datos de entrada

ID	Momento	Monto
tx-100	2026-05-01 10:00:05	\$15.000
tx-100	2026-05-01 10:00:05	\$15.000
tx-101	2026-05-01 10:00:06	\$8.000
tx-100	2026-05-01 10:03:00	\$15.000
tx-102	2026-05-01 10:03:01	\$22.000
tx-101	2026-05-01 10:00:06	\$8.000
tx-103	2026-05-01 10:05:00	\$5.000
tx-102	2026-05-01 10:20:00	\$22.000

Duplicado: mismo id y mismo monto dentro de los 120 segundos de una transacción ya procesada.

Tu programa debe

1. Guardar, por clave (id, monto), el momento de la última transacción procesada.
2. Al llegar una transacción, si la clave ya se vio y pasaron 120 segundos o menos, es un duplicado y se ignora.
3. Si pasaron más de 120 segundos, es un reintento legítimo: se procesa y se actualiza el momento.
4. Mostrar el resultado de cada transacción y el resumen.

Salida esperada

```
tx-100 $15000 @ 10:00:05: nueva, se procesa
tx-100 $15000 @ 10:00:05: DUPLICADO (a 0 s del original), se ignora
tx-101 $8000 @ 10:00:06: nueva, se procesa
tx-100 $15000 @ 10:03:00: reintento legítimo (a 175 s), se procesa
tx-102 $22000 @ 10:03:01: nueva, se procesa
tx-101 $8000 @ 10:00:06: DUPLICADO (a 0 s del original), se ignora
tx-103 $5000 @ 10:05:00: nueva, se procesa
tx-102 $22000 @ 10:20:00: reintento legítimo (a 1019 s), se procesa
Procesadas: 6 | Duplicados ignorados: 2
```

Reglas

- Límites: hasta 10^7 transacciones.
- Complejidad esperada: $O(1)$ por transacción con la clave en un mapa.
- La clave es compuesta: (id, monto). Dos transacciones con el mismo id pero distinto monto no son el mismo pago.
- Los momentos vienen en formato ISO; se comparan en segundos.

Pista (leela solo si te trabás): una clave compuesta (id, monto) en un mapa hacia el último momento visto resuelve el problema en tiempo constante. La ventana de 120 s distingue duplicado de reintento.

Bonus (opcional)

- Usá también el comercio y la tarjeta como parte de la clave.
- Con memoria limitada, olvidá las claves más viejas que 2 minutos (una cola además del mapa).

Cierre: Es la deduplicación con idempotencia que aplican las pasarelas de pago para no cobrar dos veces por un reintento.

Ejercicio 248: Desafío de nivel: Motor de promociones del supermercado

Nivel 5: Semi-Senior II · Estrategia · Temática: E-commerce · Dificultad 50/100

Situación: El desafío que integra el nivel: un supermercado quiere un motor de promociones completo. Aplica reglas por categoría (2x1, descuento porcentual), un descuento de club de socios sobre lo que queda, y todo tiene que quedar auditable, mostrando cada descuento por separado.

Datos de entrada

Producto	Categoría	Precio	Cantidad
Fideos	almacén	\$1.200	6
Aceite	almacén	\$3.800	2
Shampoo	perfumería	\$5.200	1
Jabón	perfumería	\$900	4
Gaseosa	bebidas	\$2.500	6
Vino	bebidas	\$4.500	3

El cliente es socio del club.

Promos, en orden:

1. 2x1 en bebidas (por cada 2 iguales, una gratis).
2. 30 % de descuento en perfumería (sobre cada línea).
3. Descuento de club: 10 % sobre el total que queda después de las promos.

Tu programa debe

1. Calcular el subtotal del carrito.
2. Aplicar el 2x1 en bebidas y el 30 % en perfumería, mostrando cada descuento por línea.
3. Aplicar el descuento del club sobre el parcial.
4. Mostrar el subtotal, el parcial con promos, el descuento del club y el total.

Salida esperada

```
Subtotal: $52100
2x1 Gaseosa: -$7500
2x1 Vino: -$4500
30% perfumería Shampoo: -$1560
30% perfumería Jabón: -$1080
Subtotal con promos: $37460
Descuento club (10%): -$3746
TOTAL: $33714
```

Reglas

- Alcanza con reglas de negocio y acumuladores: es el desafío integrador del nivel 5 (junto con los desafíos 249 y 250).
- Los montos son enteros; los porcentajes se calculan con división entera.
- El 2x1 es por producto, no entre productos distintos.
- El descuento del club se aplica sobre lo que queda después de las promos de categoría.

Pista (leela solo si te trabás): modelá cada promo como una función que recibe el carrito y devuelve descuentos. El orden importa: el club se calcula último, sobre el parcial ya rebajado.

Bonus (opcional)

- Parametrizá las promos en una tabla para que marketing cargue nuevas sin tocar el código.

- Agregá promos no acumulables y elegí automáticamente el mejor conjunto de promos para el cliente (como el ejercicio de cupones incompatibles).

Cierre: Es el motor de promociones completo de un supermercado, que combina reglas por categoría, descuentos globales y auditabilidad.

Ejercicio 249: Desafío de nivel: El intérprete de la hoja de cálculo

Nivel 5: Semi-Senior II · Estrategia · Temática: Finanzas y FinTech · Dificultad 50/100

Situación: El desafío del intérprete de hoja de cálculo: las celdas tienen fórmulas que referencian otras celdas, en cualquier orden, y hay que evaluarlas resolviendo las dependencias. Además, si dos celdas se referencian en círculo (A1 usa A2 y A2 usa A1), hay que detectar el ciclo en lugar de colgarse.

Datos de entrada

Celda	Fórmula
A1	10
A2	20
A3	A1 + A2
B1	A3 * 2
B2	B1 + A1
C1	C2 + 1
C2	C1 + 1

Operadores: +, −, × (con precedencia de × sobre + y −). Las referencias son nombres como A1, B2.

Tu programa debe

1. Guardar las fórmulas por celda.
2. Evaluar cada celda recursivamente: primero evaluar las celdas que referencia, después calcular su expresión.
3. Detectar ciclos: si una celda se está evaluando y se la vuelve a pedir, es una referencia circular.
4. Mostrar cada celda con su valor o #CICLO.

Salida esperada

```
A1 = 10
A2 = 20
A3 = 30
B1 = 60
B2 = 70
C1 = #CICLO (referencia circular)
C2 = #CICLO (referencia circular)
```

Reglas

- Alcanza con evaluación de expresiones, referencias y detección de ciclos: es un desafío integrador del nivel 5.
- Complejidad esperada: $O(\text{celdas} + \text{referencias})$ con memorización (cada celda se evalúa una sola vez).
- Un ciclo se detecta marcando las celdas "en curso": si se vuelve a una en curso, hay ciclo.
- Las celdas se muestran en el orden en que aparecen definidas.

Pista (leela solo si te trabás): es una evaluación recursiva con memorización sobre un grafo de dependencias. Marcar cada celda como no visitada / en curso / lista es exactamente la detección de ciclos con DFS (nivel 6).

Bonus (opcional)

- Al detectar un ciclo, mostrará qué celdas lo forman.

- Ordená las celdas topológicamente y evaluá sin recursión (orden de evaluación de una planilla real).

Cierre: Es el motor de recálculo de cualquier hoja de cálculo: dependencias entre celdas, evaluación en orden y detección de referencias circulares.

Ejercicio 250: Desafío de nivel: Agenda del congreso médico

Nivel 5: Semi-Senior II · Estrategia · Temática: Salud · Dificultad 50/100

Situación: El desafío que cierra el nivel: un congreso médico tiene 2 salas y más charlas que espacio. Cada charla tiene horario y un puntaje de interés, dos charlas no pueden estar en la misma sala si se superponen, y hay pares que no pueden darse en paralelo (mismos disertantes). Hay que programar el subconjunto de charlas de mayor interés total.

Datos de entrada

Salas: 2.

Charla	Inicio	Fin	Interés
Apertura	09:00	10:00	1
Cardio A	10:00	11:00	3
Cardio B	10:00	11:00	2
Pediatría	11:00	12:00	2
Taller X	11:30	12:30	3
Cierre	12:30	13:30	1

Restricción extra: "Cardio A" y "Cardio B" no pueden darse en paralelo (mismos disertantes).

Tu programa debe

1. Recorrer las charlas decidiendo, para cada una, no darla o darla en alguna de las 2 salas.
2. Podar si una charla se superpone con otra ya asignada a la misma sala, o si viola una restricción de no-paralelo.
3. Quedarse con la programación de mayor interés total.
4. Mostrar el interés total y la programación por sala.

Salida esperada

```

Valor total (interés) programado: 10
Programación:
  Sala 1: 09:00-10:00 Apertura (interés 1)
  Sala 1: 10:00-11:00 Cardio A (interés 3)
  Sala 1: 11:00-12:00 Pediatría (interés 2)
  Sala 1: 12:30-13:30 Cierre (interés 1)
  Sala 2: 11:30-12:30 Taller X (interés 3)

```

Reglas

- Alcanza con backtracking, restricciones y maximización: es el desafío integrador del nivel 5.
- Complejidad: exponencial (cada charla tiene salas + 1 opciones); la poda por conflictos la reduce.
- Dos charlas en salas distintas pueden superponerse, salvo que estén en la lista de no-paralelo.
- En empate de interés, se queda la primera programación encontrada.

Pista (leela solo si te trabás): combina la selección de actividades (voraz por sala no alcanza acá por el puntaje y las restricciones cruzadas) con el backtracking del fixture. Probá, para cada charla, todas sus ubicaciones posibles.

Bonus (opcional)

- Si el puntaje fuera irrelevante (maximizar cantidad de charlas), ¿se puede resolver de forma voraz por sala?
- Agregá salas con capacidad y charlas con asistentes esperados como segunda restricción.

Cierre: Es un problema de programación de eventos con recursos y restricciones, el mismo que resuelven los sistemas de agenda de congresos, aulas y estudios de TV.

Lote 11: ejercicios 251 a 275

Nivel 6

Semi-Senior III · Estructuras clásicas

Ejercicios 251 a 300

Qué vas a practicar

Estructuras clásicas: pilas, colas, montículos, grafos, caminos mínimos, union-find y programación dinámica.

Qué se espera al terminar

Modelar problemas como grafos y usar la estructura de datos precisa.

Conceptos del nivel

árboles, recorrido por niveles, heap, top-k, BFS en grilla, DFS, componentes conexas, orden topológico, BFS por niveles, mochila 0/1, distancia de edición, sumas en subárboles, cola de prioridad, union-find, Dijkstra con restricciones, subsecuencia común más larga, árbol binario de búsqueda, consultas por rango, programación dinámica de conteo, detección de ciclos, ancestro común más bajo, dos heaps, árbol generador mínimo, subsecuencia creciente más larga en $O(n \log n)$, Dijkstra, BFS multi-fuente, programación dinámica de suma de subconjuntos, fusión de k listas, programación dinámica de corte, recorrido, validación, detección de ciclos dirigidos, DFS con backtracking en grilla, BFS en grafo implícito, simulación, union-find con mapas, programación dinámica en grilla, profundidad máxima, grafo bipartito con BFS, serialización....

Desafíos de nivel

- 298. Centro de distribución con prioridades y rutas
- 299. El planificador de la cursada
- 300. Presupuesto de inversión con dependencias

Ejercicio 251: Niveles del organigrama

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Recursos humanos · Dificultad 51/100

Situación: Recursos Humanos de una empresa de 3.000 empleados quiere imprimir el organigrama por niveles jerárquicos: el director en el nivel 0, sus reportes directos en el nivel 1, y así. También quieren saber cuál es el nivel con más personas, para dimensionar las capacitaciones de mandos medios.

Datos de entrada

Formato: primera línea n ; después n líneas "empleado jefe", donde el jefe es "-" para la única persona sin jefe. Los reportes de cada jefe se listan en el orden en que aparecen.

Ejemplo

```
9
Vera -
Ruiz Vera
Soto Vera
Paz Ruiz
Gil Ruiz
Luna Ruiz
Díaz Soto
Mota Paz
Ríos Paz
```

Restricciones: $1 \leq n \leq 10^6$; nombres sin espacios; el organigrama es un árbol (sin ciclos).

Tu programa debe

1. Construir el árbol como un mapa jefe → lista de reportes.
2. Recorrerlo por niveles (BFS) desde la raíz.
3. Mostrar los empleados de cada nivel en el orden del recorrido.
4. Informar el nivel más ancho (en empate, el de menor número).

Salida esperada

```
Nivel 0: Vera
Nivel 1: Ruiz Soto
Nivel 2: Paz Gil Luna Díaz
Nivel 3: Mota Ríos
Nivel más ancho: 2 (4 personas)
```

Caso 2 (borde)

Una empresa unipersonal:

```
1
Única -
```

Salida esperada del caso 2:

```
Nivel 0: Única
Nivel más ancho: 0 (1 persona)
```

Reglas

- Complejidad esperada: $O(n)$.
- No uses recursión para recorrer: con cadenas de 10^6 niveles se desborda la pila. La cola del BFS no tiene ese problema.
- Casos borde: una sola persona; una cadena lineal (cada uno tiene un único reporte).

Pista (leela solo si te trabás): el BFS con una cola visita el árbol exactamente nivel por nivel si guardás junto a cada empleado su profundidad.

Bonus (opcional)

- Calculá para cada jefe la cantidad total de personas a su cargo (directas e indirectas).
- Imprimí el organigrama con sangría (recorrido en profundidad).

Cierre: Es el recorrido por niveles que usan los sistemas de gestión de personas para mostrar organigramas y cadenas de aprobación.

Ejercicio 252: Las 5 transacciones más grandes del día

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 51/100

Situación: El área de prevención de fraude de una billetera virtual revisa todos los días las transacciones más grandes. Hay decenas de millones por día y no tiene sentido ordenarlas todas para quedarse con cinco. Te piden obtener el top k en una sola pasada con memoria acotada.

Datos de entrada

Formato: primera línea k; después una línea por transacción "id monto" (monto entero).

Ejemplo

```
5
T01 15000
T02 320000
T03 8900
T04 1250000
T05 320000
T06 47000
T07 990000
T08 12
T09 500000
T10 1250000
```

Restricciones: $1 \leq k \leq 1.000$; hasta $5 \cdot 10^7$ transacciones; montos hasta 10^{12} .

Tu programa debe

1. Recorrer las transacciones manteniendo un heap de mínimos con como máximo k elementos.
2. Si el heap supera k elementos, sacar el mínimo.
3. Al final, ordenar las k que quedaron de mayor a menor monto (en empate, por id ascendente).
4. Mostrar el ranking.

Salida esperada

```
1. T04 $1250000
2. T10 $1250000
3. T07 $990000
4. T09 $500000
5. T02 $320000
```

Caso 2 (borde)

Hay menos transacciones que k:

```
5
T01 100
T02 300
```

Salida esperada del caso 2:

```
1. T02 $300
2. T01 $100
```

Reglas

- Complejidad esperada: $O(n \log k)$ en tiempo y $O(k)$ de memoria. Ordenar todo sería $O(n \log n)$ y $O(n)$ de memoria.
- Para los empates en el límite del top, el heap tiene que descartar primero la de id mayor: armá la prioridad como (monto, id invertido).
- Si hay menos de k transacciones, se muestran todas.

Pista (leela solo si te trabás): el heap de mínimos guarda a "las k mejores hasta ahora" y su raíz es la peor de ellas: la única candidata a salir cuando llega una mejor.

Bonus (opcional)

- Calculá el top k por comercio con un heap por comercio.
- Si las transacciones llegan de 10 servidores, combiná los top k de cada uno en un top k global.

Cierre: Es la técnica de top-k con heap que usan los sistemas de monitoreo, los rankings en vivo y los motores de búsqueda.

Ejercicio 253: El camino más corto en el depósito

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Logística y envíos · Dificultad 51/100

Situación: Un depósito de autopartes usa carros autónomos que tienen que ir de la estación de carga (S) al puesto de embalaje (E) esquivando estanterías. Cada movimiento a una celda vecina cuesta lo mismo, así que el camino más corto es el de menos pasos.

Datos de entrada

Formato: primera línea: filas y columnas; después la grilla con S (inicio), E (fin), # (estantería) y . (pasillo). Movimientos: arriba, abajo, izquierda y derecha.

Ejemplo

```
6 8
S..#....
.#.###.
.#...#.
.####.#.
.....#E
.##.....
```

Restricciones: $1 \leq \text{filas, columnas} \leq 2.000$; hay exactamente una S y una E.

Tu programa debe

1. Recorrer la grilla con BFS desde S, guardando de qué celda se llegó a cada una.
2. Explorar los vecinos en el orden arriba, abajo, izquierda, derecha.
3. Reconstruir el camino desde E hacia atrás y marcar con * las celdas intermedias.
4. Mostrar la grilla con el camino y su largo, o "No hay camino".

Salida esperada

```
S..#....
*#.###.
*#...#.
*####.#.
****..#E
.#####
Largo del camino: 13 pasos
```

Caso 2 (borde)

La salida está encerrada:

```
3 3
S#.
##.
..E
```

Salida esperada del caso 2:

```
No hay camino
```

Reglas

- Complejidad esperada: $O(\text{filas} \times \text{columnas})$.
- El BFS encuentra el camino con menos pasos porque explora por distancia creciente.
- El orden de exploración de los vecinos define cuál de los caminos más cortos se dibuja: respetalo para que coincida la salida.
- El largo es la cantidad de movimientos de S a E.

Pista (leela solo si te trabás): guardar el predecesor de cada celda (desde dónde la descubriste) alcanza para reconstruir el camino al final, caminando hacia atrás desde E.

Bonus (opcional)

- Agregá celdas con costo distinto (rampas que cuestan 3) y usá Dijkstra.
- Con varios carros, calculá para cada puesto cuál carro llega primero (BFS multi-fuente).

Cierre: Es el mismo BFS en grilla que usan los robots de depósito, los videojuegos y los ruteadores de circuitos.

Ejercicio 254: Islas de cultivo en la foto satelital

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Agricultura · Dificultad 52/100

Situación: Una cooperativa agrícola procesa fotos satelitales divididas en parcelas: cada parcela está marcada como cultivada o no. Quieren contar cuántos lotes cultivados separados hay (parcelas cultivadas que se tocan por un lado forman un mismo lote) y cuánto mide cada uno, para planificar la logística de cosecha.

Datos de entrada

Formato: primera línea: filas y columnas; después la grilla con x (cultivado) y . (no cultivado). Dos parcelas son del mismo lote si comparten un lado.

Ejemplo

```
7 10
xx..x.....
x...xx..x.
....x..xx.
.xx.....x.
.x...x....
....xxx..x
x...x...x
```

Restricciones: $1 \leq \text{filas}, \text{columnas} \leq 3.000$.

Tu programa debe

1. Recorrer la grilla y, al encontrar una parcela cultivada no visitada, explorar todo su lote con DFS.
2. Contar las parcelas de cada lote.
3. Mostrar la cantidad de lotes, sus tamaños de mayor a menor y el más grande.

Salida esperada

```
Lotes: 7
Tamaños (mayor a menor): 5 4 4 3 3 2 1
Lote más grande: 5 parcelas
```

Caso 2 (borde)

Foto sin cultivos:

```
2 3
...
...
```

Salida esperada del caso 2:

```
Lotes: 0
```

Reglas

- Complejidad esperada: $O(\text{filas} \times \text{columnas})$.
- Implementará el DFS con una pila explícita: un lote de 9 millones de parcelas desbordaría la recursión.
- Las diagonales no conectan.
- Si no hay cultivos, se muestra solo "Lotes: 0".

Pista (leela solo si te trabás): cada lote se "pinta" una sola vez: al marcar las parcelas como visitadas durante la exploración, el recorrido principal no vuelve a contarlas.

Bonus (opcional)

- Calculá el perímetro de cada lote (lados que dan a parcelas no cultivadas o al borde).
- Resuelvelo con union-find y compará con el DFS.

Cierre: Es el conteo de componentes conexas, usado en análisis de imágenes satelitales, detección de manchas y segmentación.

Ejercicio 255: Materias correlativas

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Educación · Dificultad 52/100

Situación: Una facultad cargó su plan de estudios con las correlatividades (para cursar una materia hay que tener aprobadas otras). Los alumnos quieren saber en qué orden cursar y en cuántos cuatrimestres, como mínimo, pueden terminar si cursan todo lo habilitado en cada uno. Y si alguien cargó mal y hay un círculo de correlativas, el sistema tiene que avisar.

Datos de entrada

Formato: primera línea: cantidad de materias n y de correlatividades m ; después m líneas "A B" que significan "A es correlativa de B" (hay que aprobar A antes de cursar B). Solo aparecen las materias mencionadas.

Ejemplo

```
8 8
Análisis_I Análisis_II
Álgebra Análisis_II
Álgebra Física_I
Análisis_I Física_I
Física_I Física_II
Análisis_II Física_II
Programación Estructuras
Estructuras Algoritmos
```

Restricciones: $1 \leq n \leq 10^5$; $0 \leq m \leq 5 \cdot 10^5$.

Tu programa debe

1. Armar el grafo dirigido y contar cuántas correlativas tiene cada materia (grado de entrada).
2. Aplicar el algoritmo de Kahn: tomar siempre, entre las habilitadas, la primera en orden alfabético (usá un heap).
3. Calcular el cuatrimestre mínimo de cada materia como $1 +$ el mayor cuatrimestre de sus correlativas.
4. Mostrar el orden, las materias por cuatrimestre (alfabéticamente) o el error de ciclo.

Salida esperada

```
Orden sugerido: Análisis_I, Programación, Estructuras, Algoritmos, Álgebra, Análisis_II, Física_I,
Física_II
Cuatrimestre 1: Análisis_I, Programación, Álgebra
Cuatrimestre 2: Análisis_II, Estructuras, Física_I
Cuatrimestre 3: Algoritmos, Física_II
```

Caso 2 (borde)

Un error de carga crea un ciclo:

```
2 2
A B
B A
```

Salida esperada del caso 2:

```
ERROR: hay correlatividades circulares
```

Reglas

- Complejidad esperada: $O((n + m) \log n)$ por el heap.
- Si no se pueden ordenar todas, hay un ciclo: se informa el error.
- El orden alfabético es el de los códigos de los caracteres: las letras con tilde quedan después de todas las letras sin tilde (por eso Álgebra aparece después de Programación).

Pista (leela solo si te trabás): una materia se habilita cuando su contador de correlativas pendientes llega a 0. El cuatrimestre sale del camino más largo de correlativas que llega a ella.

Bonus (opcional)

- Limitá a 4 materias por cuatrimestre y buscá una planificación que minimice los cuatrimestres (se vuelve un problema difícil).
- Si hay un ciclo, mostrá las materias que lo forman.

Cierre: Es el orden topológico, que usan los planificadores académicos, los gestores de dependencias y los sistemas de compilación.

Ejercicio 256: Amigos de amigos

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Redes sociales · Dificultad 52/100

Situación: Una red social quiere mostrar la sección "Personas que quizás conozcas". La idea es sugerir a los amigos de tus amigos que todavía no son tus amigos, ordenados por cuántos amigos tienen en común con vos. También quieren saber a cuántos saltos está cada persona de la red.

Datos de entrada

Formato: primera línea: el usuario; después, una línea por amistad "a b" (las amistades son mutuas).

Ejemplo

```
ana
ana beto
ana caro
beto dani
beto eli
caro eli
caro fede
dani gabi
eli gabi
fede hugo
```

Restricciones: hasta 10^6 usuarios y 10^7 amistades.

Tu programa debe

1. Armar el grafo como un mapa usuario → conjunto de amigos.
2. Hacer un BFS desde el usuario para calcular la distancia de cada persona.
3. Listar las personas por distancia (alfabéticamente).
4. Sugerir las personas a distancia 2, ordenadas por amigos en común (descendente) y después alfabéticamente.

Salida esperada

```
Distancia 1: beto, caro
Distancia 2: dani, eli, fede
Distancia 3: gabi, hugo
Sugerencias: eli (2 en común), dani (1 en común), fede (1 en común)
```

Caso 2 (borde)

Una red mínima:

```
ana
ana beto
```

Salida esperada del caso 2:

```
Distancia 1: beto
Sugerencias: ninguna
```

Reglas

- Complejidad esperada: $O(V + E)$ para el BFS.
- Los amigos en común son la intersección entre los amigos de la persona sugerida y los del usuario.
- Casos borde: usuario sin amigos de amigos (la lista de sugerencias queda vacía).

Pista (leela solo si te trabás): el BFS por niveles te da exactamente los grupos "amigos" (distancia 1) y "amigos de amigos" (distancia 2).

Bonus (opcional)

- Pesá las sugerencias: un amigo en común con pocos amigos cuenta más que uno con miles.
- Calculá los "grados de separación" promedio de toda la red.

Cierre: Es la base de las sugerencias de contactos de las redes sociales y de la idea de los "seis grados de separación".

Ejercicio 257: La mochila del mochilero

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Turismo · Dificultad 52/100

Situación: Una mochilera arma el equipo para una travesía de una semana en la Patagonia. La mochila soporta 4 kg de equipo extra y cada cosa tiene un peso y una "utilidad" que ella misma estimó. No puede llevar media carpa: cada objeto va entero o no va. Quiere maximizar la utilidad total.

Datos de entrada

Formato: primera línea: capacidad (en décimas de kg) y cantidad de objetos; después, una línea por objeto "nombre peso utilidad" (peso en décimas de kg).

Ejemplo

```
40 7
Carpa 18 90
Bolsa_de_dormir 12 80
Anafe 6 45
Campera_térmica 8 70
Linterna 1 20
Libro 3 10
Kit_primeros_auxilios 4 60
```

Restricciones: $1 \leq n \leq 1.000$ objetos; capacidad $\leq 10^5$.

Tu programa debe

1. Armar la tabla $dp[i][c]$ = mejor utilidad usando los primeros i objetos con capacidad c .
2. Para cada objeto, elegir entre no llevarlo o llevarlo (si entra).
3. Reconstruir qué objetos se eligieron recorriendo la tabla desde el final.
4. Mostrar los objetos (en el orden de la entrada), el peso en kg y la utilidad.

Salida esperada

```
Llevar: Carpa, Anafe, Campera_térmica, Linterna, Libro, Kit_primeros_auxilios
Peso: 4.0 kg de 4.0 kg | Utilidad: 295
```

Caso 2 (borde)

Mochila demasiado chica:

```
5 2
Carpa 18 90
Anafe 6 45
```

Salida esperada del caso 2:

```
No entra nada en la mochila
```

Reglas

- Complejidad esperada: $O(n \times \text{capacidad})$ en tiempo; $O(n \times \text{capacidad})$ de memoria (se puede bajar a $O(\text{capacidad})$ si no hace falta reconstruir).
- Al reconstruir desde el último objeto, un objeto se considera elegido si $dp[i][c] \neq dp[i - 1][c]$.
- Pesos en kg con 1 decimal. Los decimales se muestran con punto y sin separador de miles en la salida.
- El voraz por utilidad/peso no garantiza el óptimo acá (los objetos no se fraccionan).

Pista (leela solo si te trabás): la pregunta en cada celda es: "¿me conviene más lo que tenía sin este objeto, o este objeto más lo mejor que podía hacer con el espacio que me queda?".

Bonus (opcional)

- Agregá un segundo límite de volumen (mochila con dos restricciones).
- Mostrá todas las selecciones que alcanzan la utilidad máxima.

Cierre: Es la mochila 0/1, uno de los problemas de programación dinámica más usados: presupuestos, carga, selección de proyectos.

Ejercicio 258: El mismo cliente escrito distinto

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 52/100

Situación: El área de cumplimiento (KYC) de una fintech tiene que detectar si un cliente nuevo ya estaba registrado con el nombre escrito de otra forma: sin tildes, con una letra cambiada, con espacios de más. Una coincidencia exacta no alcanza, así que usan la distancia de edición entre nombres normalizados.

Datos de entrada

Formato: primera línea: cantidad de pares; después, un par por línea separado por "|".

Ejemplo

```
4
María José Pérez | Maria Jose Perez
Juan Carlos Gómez | Juan Carlo Gomes
Laura Fernández | Lautaro Fernández
Sebastián Ortiz | Sebastian Ortíz
```

Restricciones: hasta 10^4 pares por consulta; nombres de hasta 100 caracteres.

Tu programa debe

1. Normalizar cada nombre: minúsculas, sin tildes y con un solo espacio entre palabras.
2. Calcular la distancia de edición (mínima cantidad de inserciones, borrados o reemplazos de un carácter) con programación dinámica.
3. Calcular la similitud: $1 - \text{distancia} / \text{largo del nombre más largo}$.
4. Marcar como posible mismo cliente si la similitud es de 85 % o más.

Salida esperada

```
"María José Pérez" vs "Maria Jose Perez": distancia 0 | similitud 100.0% -> POSIBLE MISMO CLIENTE
"Juan Carlos Gómez" vs "Juan Carlo Gomes": distancia 2 | similitud 88.2% -> POSIBLE MISMO CLIENTE
"Laura Fernández" vs "Lautaro Fernández": distancia 3 | similitud 82.4% -> distintos
"Sebastián Ortiz" vs "Sebastian Ortíz": distancia 0 | similitud 100.0% -> POSIBLE MISMO CLIENTE
```

Caso 2 (borde)

Nombres idénticos:

```
1
Ana | Ana
```

Salida esperada del caso 2:

```
"Ana" vs "Ana": distancia 0 | similitud 100.0% -> POSIBLE MISMO CLIENTE
```

Reglas

- Complejidad esperada: $O(\text{largo1} \times \text{largo2})$ por par; $O(\text{largo2})$ de memoria guardando solo dos filas.
- Similitud en porcentaje con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- La decisión se toma con el valor sin redondear.
- Casos borde: nombres idénticos (distancia 0); un nombre vacío.

Pista (leela solo si te trabás): $dp[i][j]$ es la distancia entre los primeros i caracteres de un nombre y los primeros j del otro; cada celda sale del mínimo de tres vecinas.

Bonus (opcional)

- Comparará también con los nombres en otro orden ("Pérez María José") probando permutaciones de palabras.
- Con millones de clientes, no se puede comparar contra todos: agrupá por la primera letra del apellido o por una clave fonética antes de comparar.

Cierre: Es la distancia de Levenshtein, usada en controles antilavado, correctores ortográficos y deduplicación de bases de clientes.

Ejercicio 259: Árbol de categorías de la tienda

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: E-commerce · Dificultad 52/100

Situación: Un e-commerce organiza su catálogo en un árbol de categorías (Tienda → Electro → Audio → Auriculares...). Las ventas se registran en las categorías hoja, pero el tablero de gerencia necesita ver el total de cada categoría incluyendo todo lo que tiene adentro.

Datos de entrada

Formato: una línea por categoría "nombre padre [ventas]"; el padre de la raíz es "-" y las ventas (en millones) solo aparecen en algunas categorías. Cada categoría aparece después de su padre.

Ejemplo

```
Tienda -  
Electro Tienda  
Hogar Tienda  
Celulares Electro 420  
Notebooks Electro 610  
Audio Electro  
Auriculares Audio 95  
Parlantes Audio 130  
Muebles Hogar 380  
Bazar Hogar 145
```

Restricciones: hasta 10^6 categorías; profundidad hasta 10^6 (atención a la recursión).

Tu programa debe

1. Armar el árbol como un mapa padre → hijos, en orden de aparición.
2. Calcular el total de cada categoría: sus ventas propias más el total de cada hijo (recorrido en postorden).
3. Imprimir el árbol con dos espacios de sangría por nivel, en preorden, con el total de cada categoría.

Salida esperada

```
Tienda: 1780  
  Electro: 1255  
    Celulares: 420  
    Notebooks: 610  
    Audio: 225  
      Auriculares: 95  
      Parlantes: 130  
  Hogar: 525  
    Muebles: 380  
    Bazar: 145
```

Caso 2 (borde)

Una tienda sin categorías:

```
Tienda - 0
```

Salida esperada del caso 2:

```
Tienda: 0
```

Reglas

- Complejidad esperada: $O(n)$.
- Para árboles muy profundos, convertí la recursión en un recorrido con pila explícita.
- Casos borde: una sola categoría; categorías sin ventas en todo su subárbol (total 0).

Pista (leela solo si te trabás): el total de un nodo depende de los de sus hijos: por eso se calcula "de abajo hacia arriba" (postorden), aunque se imprima "de arriba hacia abajo" (preorden).

Bonus (opcional)

- Calculá qué porcentaje de las ventas de su padre representa cada categoría.
- Soportá mover una categoría a otro padre y actualizá solo los totales afectados.

Cierre: Es la agregación en árboles que usan los tableros de ventas, los reportes contables por cuentas y los analizadores de espacio en disco.

Ejercicio 260: Triage en la guardia

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Salud · Dificultad 53/100

Situación: En la guardia de un hospital hay un solo médico de turno a la madrugada. Los pacientes se atienden por prioridad de triage (1 es lo más urgente) y, a igual prioridad, por orden de llegada. La dirección quiere simular una noche para ver cuánto esperan los pacientes de cada prioridad.

Datos de entrada

Formato: una línea por paciente "nombre llegada prioridad duración" (minutos desde la medianoche; prioridad de 1 a 5).

Ejemplo

```
Paz 0 3 20
Luna 2 1 30
Gil 5 4 10
Sosa 8 2 15
Vera 12 1 25
Ríos 15 5 10
Mota 18 2 20
```

Restricciones: hasta 10^6 pacientes.

Tu programa debe

1. Procesar los pacientes en orden de llegada, poniendo en una cola de prioridad a los que ya llegaron.
2. Cada vez que el médico se libera, atender al de mayor prioridad (menor número) y, en empate, al que llegó antes.
3. Si no hay nadie esperando, avanzar el tiempo hasta la próxima llegada.
4. Mostrar cada atención y la espera promedio por prioridad.

Salida esperada

```
0- 20 Paz (prioridad 3, llegó 0, esperó 0)
20- 50 Luna (prioridad 1, llegó 2, esperó 18)
50- 75 Vera (prioridad 1, llegó 12, esperó 38)
75- 90 Sosa (prioridad 2, llegó 8, esperó 67)
90-110 Mota (prioridad 2, llegó 18, esperó 72)
110-120 Gil (prioridad 4, llegó 5, esperó 105)
120-130 Ríos (prioridad 5, llegó 15, esperó 105)
Espera promedio por prioridad: P1=28.0, P2=69.5, P3=0.0, P4=105.0, P5=105.0
```

Reglas

- Complejidad esperada: $O(n \log n)$.
- Una atención no se interrumpe aunque llegue alguien más urgente.
- Espera promedio con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Formato: inicio y fin en 3 caracteres alineados a la derecha.

Pista (leela solo si te trabás): la prioridad del heap es la tupla (prioridad de triage, hora de llegada): así el heap resuelve solo los empates.

Bonus (opcional)

- Hací que la prioridad de un paciente mejore un nivel por cada 30 minutos de espera (envejecimiento, para evitar que alguien espere para siempre).
- Simulé con dos médicos y comparé las esperas.

Cierre: Es la cola de prioridad, la estructura detrás del triage hospitalario, los planificadores de procesos y las redes con calidad de servicio.

Ejercicio 261: Barrios conectados a la red eléctrica

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Clima y energía · Dificultad 53/100

Situación: Una cooperativa eléctrica va conectando barrios a medida que tiende nuevas líneas. El centro de operaciones necesita responder al instante si dos barrios ya están en la misma red y cuántas redes separadas quedan, con millones de operaciones en el registro histórico.

Datos de entrada

Formato: primera línea: cantidad de operaciones; después, "unir A B" (se tiende una línea) o "consulta A B". Un barrio aparece la primera vez que se lo nombra.

Ejemplo

```
8
unir Centro Norte
unir Sur Oeste
consulta Norte Oeste
unir Norte Sur
consulta Centro Oeste
unir Este Puerto
consulta Puerto Centro
unir Puerto Oeste
```

Restricciones: hasta 10^6 barrios y 10^7 operaciones.

Tu programa debe

1. Implementar union-find con compresión de caminos y unión por tamaño.
2. En cada unir, fusionar los conjuntos si son distintos y actualizar la cantidad de redes.
3. En cada consulta, comparar los representantes.
4. Mostrar el resultado de cada operación.

Salida esperada

```
unir Centro Norte: redes separadas = 1
unir Sur Oeste: redes separadas = 2
consulta Norte Oeste: NO conectados
unir Norte Sur: redes separadas = 1
consulta Centro Oeste: conectados
unir Este Puerto: redes separadas = 2
consulta Puerto Centro: NO conectados
unir Puerto Oeste: redes separadas = 1
```

Reglas

- Complejidad esperada: casi $O(1)$ amortizado por operación ($O(\alpha(n))$, la inversa de Ackermann).
- Un barrio nuevo cuenta como una red nueva de un solo barrio en el momento en que se lo nombra.
- Unir dos barrios ya conectados no cambia la cantidad de redes.

Pista (leela solo si te trabás): cada red tiene un representante; dos barrios están conectados si tienen el mismo. La compresión de caminos hace que las búsquedas futuras sean casi directas.

Bonus (opcional)

- Guardá el consumo de cada barrio y respondé el consumo total de la red de un barrio en $O(\alpha(n))$.
- Soportá cortes de línea (desconexiones): union-find no los maneja; pensá una alternativa procesando las operaciones al revés.

Cierre: Es union-find, la estructura que usan los sistemas de redes, el análisis de conectividad y el algoritmo de Kruskal.

Ejercicio 262: Reparto con peajes y horarios de corte

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Logística y envíos · Dificultad 53/100

Situación: Una distribuidora sale del depósito a las 08:00 y tiene que llegar a un cliente antes de su horario de corte de recepción. Entre las rutas posibles hay autopistas con peaje (rápidas) y caminos sin peaje (lentos). Quieren la ruta más barata en peajes que igual llegue a tiempo.

Datos de entrada

Formato: primera línea: origen, destino y horario de corte en minutos desde la medianoche; después, un tramo por línea "A B minutos peaje" (los tramos son de doble mano). Se sale del origen a las 08:00 (minuto 480).

Ejemplo

```
Depósito Cliente 600
Depósito Norte 40 0
Depósito Sur 25 1200
Norte Cliente 90 0
Norte Autopista 20 900
Sur Autopista 15 0
Autopista Cliente 30 1500
Sur Cliente 70 500
```

Restricciones: hasta 10^4 lugares y $5 \cdot 10^4$ tramos; tiempos y cortes hasta 1.440 minutos.

Tu programa debe

1. Aplicar Dijkstra donde el estado es (lugar, minuto de llegada) y la prioridad es el costo en peajes (en empate, la llegada más temprana).
2. Descartar los tramos que harían llegar después del corte y no volver a pasar por un lugar ya recorrido en la misma ruta.
3. La primera vez que se saca del heap un estado del destino, esa es la ruta más barata que cumple el corte.
4. Mostrar la ruta, el total de peajes y la hora de llegada, o que es imposible.

Salida esperada

```
Ruta: Depósito -> Sur -> Cliente
Peajes: $1700
Llegada: 09:35 (corte 10:00)
```

Caso 2 (borde)

El cliente cierra a las 08:20 y el único camino tarda una hora:

```
Depósito Cliente 500
Depósito Cliente 60 0
```

Salida esperada del caso 2:

```
Imposible llegar antes del corte
```

Reglas

- Complejidad esperada: $O(E \times T \times \log(V \times T))$ con T la cantidad de minutos posibles; para este ejemplo, instantáneo.
- Por qué no alcanza un Dijkstra común por peaje: el camino más barato puede llegar tarde, y el que llega a tiempo puede no ser el más barato; el tiempo tiene que ser parte del estado.
- Casos borde: ningún camino llega a tiempo; origen igual al destino.

Pista (leela solo si te trabás): si solo guardás el mejor costo por lugar, podés descartar una ruta un poco más cara pero más rápida que después es la única que llega a tiempo. Por eso el estado es (lugar, tiempo).

Bonus (opcional)

- Agregá ventanas horarias: la autopista solo se puede usar hasta las 09:00.
- Con varios clientes, planificá el orden de visita (se vuelve el problema del viajante con ventanas de tiempo).

Cierre: Es el camino más corto con restricciones de recursos, el núcleo de los ruteadores de reparto con horarios de entrega.

Ejercicio 263: Qué cambió en el reglamento

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Educación · Dificultad 53/100

Situación: Una escuela modificó su reglamento de convivencia y las familias piden ver qué cambió. La secretaría quiere un comparador que muestre las líneas que se mantienen, las que se eliminaron y las que se agregaron, igual que las herramientas de comparación de documentos.

Datos de entrada

Formato: las líneas de la versión vieja, una línea con "---" y las líneas de la versión nueva. Se comparan líneas completas.

Ejemplo

```
Art 1: El horario es de 8 a 18
Art 2: Se exige uniforme
Art 3: La biblioteca cierra a las 20
Art 4: Prohibido comer en el aula
---
Art 1: El horario es de 8 a 18
Art 3: La biblioteca cierra a las 21
Art 4: Prohibido comer en el aula
Art 5: Se permite el uso de celulares en recreos
```

Restricciones: hasta 5.000 líneas por versión.

Tu programa debe

1. Calcular la subsecuencia común más larga (LCS) de líneas con programación dinámica: $dp[i][j]$ = LCS entre las líneas desde i de la versión vieja y desde j de la nueva.
2. Recorrer desde el principio: si las líneas coinciden, es una línea sin cambios; si no, se marca como eliminada la de la versión vieja cuando $dp[i + 1][j] \geq dp[i][j + 1]$, y si no, como agregada la de la nueva.
3. Mostrar el diff con prefijos " ", "- " y "+ ".
4. Mostrar el resumen de líneas sin cambios, eliminadas y agregadas.

Salida esperada

```
Art 1: El horario es de 8 a 18
- Art 2: Se exige uniforme
- Art 3: La biblioteca cierra a las 20
+ Art 3: La biblioteca cierra a las 21
Art 4: Prohibido comer en el aula
+ Art 5: Se permite el uso de celulares en recreos
Líneas sin cambios: 2 | eliminadas: 2 | agregadas: 2
```

Reglas

- Complejidad esperada: $O(n \times m)$ en tiempo y memoria.
- La regla de desempate (preferir eliminar antes que agregar) define el orden del diff: respetala para que coincida la salida.
- Una línea modificada aparece como eliminada (versión vieja) y agregada (versión nueva).

Pista (leela solo si te trabás): las líneas que forman la LCS son las que "sobrevivieron"; todo lo demás es borrado o agregado. La tabla dp te dice en cada paso qué decisión conserva la LCS más larga.

Bonus (opcional)

- Para las líneas modificadas, mostrá qué palabras cambiaron (LCS de palabras dentro de la línea).
- Con archivos de un millón de líneas, $O(n \times m)$ no entra en memoria: investigá el algoritmo de Myers.

Cierre: Es la subsecuencia común más larga, el corazón de las herramientas de diff, del control de versiones y de la comparación de documentos legales.

Ejercicio 264: Precios entre dos valores al instante

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: E-commerce · Dificultad 53/100

Situación: Un comparador de precios guarda los precios de un producto en un árbol binario de búsqueda para responder rápido "¿qué ofertas hay entre \$3.000 y \$6.000?". Quieren ver cuántos nodos visita cada consulta: con el árbol bien usado, no hace falta mirar todos los precios.

Datos de entrada

Formato: primera línea n ; segunda línea, los n precios en el orden en que se insertan; después, la cantidad de consultas y una consulta "desde hasta" por línea.

Ejemplo

```
12
5200 3100 8900 1500 4200 7600 9900 2800 6100 4700 12500 800
3
3000 6000
100 900
9000 20000
```

Restricciones: hasta 10^6 precios distintos y 10^5 consultas.

Tu programa debe

1. Insertar los precios en un árbol binario de búsqueda en el orden dado (menores a la izquierda, mayores o iguales a la derecha).
2. Calcular la altura del árbol (cantidad de nodos del camino más largo desde la raíz).
3. Para cada consulta, recorrer en orden podando: bajar a la izquierda solo si el nodo es mayor que "desde" y a la derecha solo si es menor que "hasta".
4. Mostrar los precios de cada rango en orden, la cantidad y los nodos visitados.

Salida esperada

```
Altura del árbol: 4
[3000, 6000]: 4 precios (3100 4200 4700 5200) | nodos visitados: 9
[100, 900]: 1 precios (800) | nodos visitados: 4
[9000, 20000]: 2 precios (9900 12500) | nodos visitados: 4
```

Reglas

- Complejidad esperada: $O(h + k)$ por consulta, con h la altura y k la cantidad de resultados.
- Sin balanceo, insertar precios ya ordenados deja un árbol de altura n (una lista): ver el bonus.
- Un nodo visitado es cada llamada del recorrido sobre un nodo existente.

Pista (leela solo si te trabás): el recorrido en orden de un árbol binario de búsqueda devuelve los valores ordenados; la poda evita entrar en subárboles que están enteros fuera del rango.

Bonus (opcional)

- Insertá los mismos precios ordenados de menor a mayor y compará la altura y los nodos visitados.
- Implementá un árbol balanceado (AVL o rojo-negro) o usá uno de tu entorno y medí la diferencia.

Cierre: Los árboles de búsqueda balanceados son la base de los índices de bases de datos y de las estructuras ordenadas de casi todos los entornos.

Ejercicio 265: De cuántas formas se junta un monto

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 54/100

Situación: Un simulador educativo de finanzas para chicos pregunta "¿de cuántas formas distintas podés juntar \$1.000 con billetes de 100, 200, 500 y 1.000?". El orden de los billetes no importa: 5 de 200 es una sola forma. Con montos grandes la cantidad se dispara y hay que calcularla sin listarlas.

Datos de entrada

Formato: primera línea: cantidad de denominaciones; segunda, las denominaciones; después, la cantidad de montos y un monto por línea.

Ejemplo

```
4
100 200 500 1000
3
1000
2500
100000
```

Restricciones: hasta 50 denominaciones; montos hasta 10^6 .

Tu programa debe

1. Armar $dp[v]$ = cantidad de formas de juntar v , con $dp[0] = 1$.
2. Procesar las denominaciones de a una: para cada monto v desde la denominación hasta el objetivo, sumar $dp[v - d]$ a $dp[v]$.
3. Mostrar la cantidad exacta y su resto módulo 1.000.000.007.

Salida esperada

```
$1000: 11 formas | módulo 1000000007: 11
$2500: 64 formas | módulo 1000000007: 64
$100000: 1712051 formas | módulo 1000000007: 1712051
```

Reglas

- Complejidad esperada: $O(\text{denominaciones} \times \text{monto})$.
- El bucle externo sobre denominaciones es lo que hace que se cuenten combinaciones (sin importar el orden) y no secuencias.
- La cantidad exacta puede superar los 64 bits con montos grandes: usá enteros de precisión arbitraria o reportá solo el módulo.

Pista (leela solo si te trabás): si invertís los bucles (monto afuera, denominación adentro), contás de más: "100 + 200" y "200 + 100" se volverían formas distintas.

Bonus (opcional)

- Contá las secuencias (el orden sí importa) y compará los números.
- Con cantidades limitadas de cada billete, adaptá la recurrencia.

Cierre: Es el conteo de combinaciones con programación dinámica, el mismo esquema de problemas de conteo en combinatoria y probabilidad.

Ejercicio 266: Orden de carga de los niveles del juego

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Videojuegos · Dificultad 54/100

Situación: Un estudio de videojuegos carga los recursos de cada nivel según sus dependencias: el menú necesita el motor de física, el motor necesita las texturas, etc. Si alguien agrega una dependencia circular, el juego se cuelga al iniciar. Quieren calcular un orden de carga válido o mostrar exactamente dónde está el ciclo.

Datos de entrada

Formato: primera línea: cantidad de dependencias; después "A B" significa "A necesita que B esté cargado antes".

Ejemplo

```
7
motor_fisica texturas
menu motor_fisica
nivel_1 menu
nivel_1 sonidos
nivel_2 nivel_1
jefe_final nivel_2
jefe_final sonidos
```

Restricciones: hasta 10^5 recursos y 10^6 dependencias.

Tu programa debe

1. Hacer un DFS marcando cada recurso como no visitado, en curso o terminado.
2. Recorrer los recursos en el orden en que aparecen y, para cada uno, sus dependencias en el orden dado.
3. Si se llega a un recurso "en curso", hay un ciclo: reconstruirlo desde la pila de recursos en curso.
4. Si no hay ciclo, el orden de carga es el orden en que los recursos terminan (postorden).

Salida esperada

```
Orden de carga: texturas, motor_fisica, menu, sonidos, nivel_1, nivel_2, jefe_final
```

Caso 2 (borde)

Un recurso depende indirectamente de sí mismo:

```
3
a b
b c
c a
```

Salida esperada del caso 2:

```
ERROR: dependencia circular: a -> b -> c -> a
```

Reglas

- Complejidad esperada: $O(V + E)$.
- El postorden funciona porque un recurso termina solo después de que terminaron todas sus dependencias.
- Para grafos muy profundos, usá una pila explícita en lugar de recursión.

Pista (leela solo si te trabás): los tres colores (blanco, gris, negro) son la clave: una arista hacia un nodo gris (en curso) es una arista hacia atrás, y eso es exactamente un ciclo.

Bonus (opcional)

- Cargá en paralelo: agrupá los recursos en tandas que se pueden cargar a la vez.
- Si hay varios ciclos, listalos todos (componentes fuertemente conexas).

Cierre: Es la detección de ciclos con DFS, usada por los gestores de paquetes, los sistemas de compilación y los motores de juegos.

Ejercicio 267: El antepasado común de dos especies

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Ciencia · Dificultad 54/100

Situación: Un museo de ciencias naturales arma una muestra interactiva sobre el árbol de la vida. El visitante elige dos especies o grupos y la pantalla muestra su antepasado común más cercano en la clasificación y cuántos saltos los separan.

Datos de entrada

Formato: una línea por grupo "nombre padre" (el padre de la raíz es "-"); una línea con "?" y después las consultas "A B".

Ejemplo

```
Vida -
Animales Vida
Plantas Vida
Mamíferos Animales
Aves Animales
Felinos Mamíferos
Cánidos Mamíferos
Gato Felinos
Puma Felinos
Lobo Cánidos
Hornero Aves
Ceibo Plantas
?
Gato Puma
Gato Lobo
Hornero Puma
Ceibo Lobo
Mamíferos Gato
```

Restricciones: hasta 10^6 grupos y 10^6 consultas (para esos tamaños, ver el bonus).

Tu programa debe

1. Guardar el padre de cada grupo y calcular su profundidad (la raíz tiene 0).
2. Para cada consulta, subir desde el nodo más profundo hasta igualar las profundidades.
3. Subir ambos a la vez hasta que coincidan: ese es el ancestro común más bajo.
4. Mostrar el ancestro y la distancia: $\text{prof}(A) + \text{prof}(B) - 2 \times \text{prof}(\text{ancestro})$.

Salida esperada

```
Gato y Puma: ancestro común Felinos | distancia 2 saltos
Gato y Lobo: ancestro común Mamíferos | distancia 4 saltos
Hornero y Puma: ancestro común Animales | distancia 5 saltos
Ceibo y Lobo: ancestro común Vida | distancia 6 saltos
Mamíferos y Gato: ancestro común Mamíferos | distancia 2 saltos
```

Reglas

- Complejidad esperada: $O(\text{profundidad})$ por consulta con este método simple.
- Si uno es ancestro del otro, el ancestro común es él mismo.
- Calculá las profundidades una vez y guardalas.

Pista (leela solo si te trabás): igualar profundidades primero garantiza que, al subir de a un paso los dos juntos, se encuentren exactamente en el ancestro común.

Bonus (opcional)

- Con árboles de profundidad 10^6 y 10^6 consultas, usá "saltos binarios" (guardar el ancestro a distancia 1, 2, 4, 8...) para responder en $O(\log n)$.

- Dada una lista de especies, encontrá el ancestro común de todas.

Cierre: Es el ancestro común más bajo (LCA), usado en filogenia, en sistemas de archivos, en control de versiones y en redes.

Ejercicio 268: Mediana de tiempos de respuesta en vivo

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Telecomunicaciones · Dificultad 54/100

Situación: El panel de monitoreo de una operadora muestra el tiempo de respuesta de sus servidores. El promedio engaña: un solo pedido de 2 segundos lo dispara. Prefieren la mediana, pero calcularla ordenando todos los tiempos en cada actualización es imposible con millones de mediciones por minuto.

Datos de entrada

Formato: una línea con los tiempos de respuesta en milisegundos, en el orden en que llegan.

Ejemplo

```
120 95 300 110 88 105 2000 130 99 102
```

Restricciones: hasta 10^7 mediciones.

Tu programa debe

1. Mantener dos heaps: uno de máximos con la mitad baja y uno de mínimos con la mitad alta.
2. Insertar cada medición en el heap que corresponde y rebalancear para que el de la mitad baja tenga igual cantidad o uno más.
3. La mediana es el tope de la mitad baja (cantidad impar) o el promedio de los dos topes (cantidad par).
4. Mostrar, después de cada medición, la mediana y el promedio para comparar.

Salida esperada

```
1. llega 120 ms -> mediana 120.0 | promedio 120.0
2. llega 95 ms -> mediana 107.5 | promedio 107.5
3. llega 300 ms -> mediana 120.0 | promedio 171.7
4. llega 110 ms -> mediana 115.0 | promedio 156.3
5. llega 88 ms -> mediana 110.0 | promedio 142.6
6. llega 105 ms -> mediana 107.5 | promedio 136.3
7. llega 2000 ms -> mediana 110.0 | promedio 402.6
8. llega 130 ms -> mediana 115.0 | promedio 368.5
9. llega 99 ms -> mediana 110.0 | promedio 338.6
10. llega 102 ms -> mediana 107.5 | promedio 314.9
```

Reglas

- Complejidad esperada: $O(\log n)$ por medición.
- Mediana y promedio con 1 decimal (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Una medición igual al tope de la mitad baja va a la mitad baja.
- Formato: índice en 2 caracteres, tiempo en 4 y valores en 6.

Pista (leela solo si te trabás): los dos heaps partan los datos en "la mitad menor" y "la mitad mayor": la mediana siempre está en la frontera, que es justo lo que los heaps te dan en $O(1)$.

Bonus (opcional)

- Calculá la mediana de los últimos 1.000 datos (ventana deslizante): vas a necesitar borrar elementos de los heaps (borrado perezoso).
- Calculá el percentil 99 en lugar de la mediana.

Cierre: Es la mediana en streaming con dos heaps, usada en monitoreo de latencias, finanzas en tiempo real y sistemas de alertas.

Ejercicio 269: Fibra óptica mínima entre edificios

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Telecomunicaciones · Dificultad 54/100

Situación: Una universidad quiere conectar todos sus edificios con fibra óptica usando la menor cantidad de metros de cable. No hace falta que cada par de edificios esté unido directamente: alcanza con que todos queden conectados a través de la red.

Datos de entrada

Formato: primera línea: edificios n y tendidos posibles m ; después, "A B metros" por cada tendido.

Ejemplo

```
6 9
Rectorado Biblioteca 120
Rectorado Ingeniería 300
Biblioteca Ingeniería 150
Biblioteca Medicina 400
Ingeniería Medicina 180
Ingeniería Comedor 90
Medicina Comedor 210
Comedor Deportes 250
Medicina Deportes 160
```

Restricciones: hasta 10^5 edificios y 10^6 tendidos posibles.

Tu programa debe

1. Ordenar los tendidos por metros (en empate, por orden de aparición).
2. Recorrerlos y agregar cada uno que una dos edificios que todavía no están conectados (verificar con union-find).
3. Detenerse con $n - 1$ tendidos, o informar que no se puede conectar todo.
4. Mostrar los tendidos elegidos en el orden en que se agregaron y el total.

Salida esperada

```
Ingeniería - Comedor: 90 m
Rectorado - Biblioteca: 120 m
Biblioteca - Ingeniería: 150 m
Medicina - Deportes: 160 m
Ingeniería - Medicina: 180 m
Fibra total: 700 m
```

Caso 2 (borde)

Hay un edificio sin ningún tendido posible:

```
3 1
A B 10
```

Salida esperada del caso 2:

```
No se pueden conectar todos los edificios
```

Reglas

- Complejidad esperada: $O(m \log m)$.
- Justificación: el tendido más barato que une dos partes separadas siempre forma parte de alguna red óptima (propiedad del corte).
- Casos borde: grafo desconectado; un solo edificio.

Pista (leela solo si te trabás): Kruskal = voraz + union-find: probás los cables de más barato a más caro y descartás los que cerrarían un ciclo (unen edificios ya conectados).

Bonus (opcional)

- Resuélvelo con el algoritmo de Prim y compará.
- Exigí redundancia: la red tiene que seguir conectada si se corta cualquier cable (se vuelve un problema mucho más difícil).

Cierre: Es el árbol generador mínimo, usado para diseñar redes de telecomunicaciones, eléctricas, de agua y de caminos.

Ejercicio 270: La racha creciente de precios más larga

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 54/100

Situación: Un analista quiere medir la "tendencia" de una acción: la cantidad máxima de cierres que se pueden elegir, en orden cronológico y no necesariamente consecutivos, con precios estrictamente crecientes. Con décadas de datos diarios, el algoritmo cuadrático es demasiado lento.

Datos de entrada

Formato: una línea con los precios de cierre en orden cronológico.

Ejemplo

```
1520 1480 1395 1440 1610 1575 1702 1650 1590 1720 1800 1760 1810 1795
```

Restricciones: hasta 10^6 precios.

Tu programa debe

1. Mantener un arreglo de "colas": en la posición k , el menor precio final posible de una subsecuencia creciente de largo $k + 1$.
2. Para cada precio, buscar con búsqueda binaria la primera cola mayor o igual y reemplazarla (o agregar al final si no hay).
3. Guardar para cada precio el índice del elemento anterior (el que estaba en la cola $k - 1$ en ese momento).
4. Reconstruir la subsecuencia desde la última cola y mostrar largo y secuencia.

Salida esperada

```
Largo de la racha creciente más larga: 7
Una racha que la alcanza: 1395 1440 1575 1590 1720 1760 1795
```

Reglas

- Complejidad esperada: $O(n \log n)$.
- Estrictamente creciente: un precio igual no extiende la racha (por eso se busca el primero mayor o igual).
- Reconstrucción exacta: se parte del índice guardado en la última cola y se sigue la cadena de anteriores.

Pista (leela solo si te trabás): el arreglo de colas no es una subsecuencia real, pero su largo sí es la respuesta. Para recuperar una subsecuencia válida necesitás los punteros al anterior.

Bonus (opcional)

- Contá cuántas subsecuencias crecientes de largo máximo existen.
- Versión con tolerancia: permití bajas de hasta 1 % sin cortar la racha.

Cierre: Es la subsecuencia creciente más larga, usada en análisis de tendencias, en el ordenamiento por paciencia y en bioinformática.

Ejercicio 271: Cuánto tarda en viajar un rumor

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Redes sociales · Dificultad 55/100

Situación: Un equipo de investigación estudia cómo se difunde una noticia falsa en una red social. Cada usuario tarda cierta cantidad de minutos en reenviarla a cada contacto (el reenvío es en un solo sentido). Quieren saber en qué minuto se entera cada usuario si la noticia arranca en una persona.

Datos de entrada

Formato: primera línea: quién la publica; después, "A B minutos" significa que A se la reenvía a B esa cantidad de minutos después de enterarse.

Ejemplo

```
ana
ana beto 5
ana caro 12
beto caro 4
beto dani 20
caro dani 7
caro eli 15
dani eli 3
eli fede 10
gabi hugo 1
```

Restricciones: hasta 10^6 usuarios y 10^7 conexiones; minutos positivos.

Tu programa debe

1. Aplicar Dijkstra desde el usuario que publica, con los minutos como pesos.
2. Usar un heap y descartar entradas desactualizadas.
3. Mostrar, en el orden de aparición de los usuarios, cuándo se entera cada uno o si nunca se entera.
4. Informar el último en enterarse (en empate de minuto, el de nombre mayor alfabéticamente).

Salida esperada

```
ana: se entera en el minuto 0
beto: se entera en el minuto 5
caro: se entera en el minuto 9
dani: se entera en el minuto 16
eli: se entera en el minuto 19
fede: se entera en el minuto 29
gabi: nunca se entera
hugo: nunca se entera
Último en enterarse: fede (minuto 29)
```

Reglas

- Complejidad esperada: $O((V + E) \log V)$.
- Dijkstra requiere pesos no negativos (se cumple: los minutos son positivos).
- Casos borde: usuarios sin conexión con la fuente.

Pista (leela solo si te trabás): cada usuario se entera por el camino más rápido, no por el primero que lo intenta. Dijkstra procesa a los usuarios en orden de "cuándo se enteran".

Bonus (opcional)

- Si cada usuario reenvía solo con cierta probabilidad, simulá la difusión muchas veces y estimá la probabilidad de que cada uno se entere.
- Encontrá el usuario desde el que la noticia llega a todos más rápido.

Cierre: Es Dijkstra aplicado a la difusión de información, el mismo algoritmo de los GPS y del ruteo en redes.

Ejercicio 272: Cadena de contagio en el hospital

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Salud · Dificultad 55/100

Situación: El comité de infecciones de un hospital modela un brote en un piso: cada hora, un paciente infectado contagia a los pacientes de las camas vecinas (arriba, abajo, izquierda, derecha). Los pasillos vacíos y las paredes cortan el contagio. Quieren saber cuándo se contagia cada cama y quiénes quedan a salvo.

Datos de entrada

Formato: primera línea: filas y columnas; después la grilla con I (infectado al inicio), P (paciente sano), . (cama vacía) y # (pared).

Ejemplo

```
5 7
P.PP#PP
PI.#PPP
PP.#..P
#PPPIP
PPP#PPP
```

Restricciones: hasta 2.000 × 2.000 celdas.

Tu programa debe

1. Poner en la cola todos los infectados iniciales con hora 0 (BFS con varias fuentes).
2. Propagar por niveles: cada paciente sano vecino se contagia una hora después.
3. Dibujar la grilla con la hora de contagio de cada paciente (S para los que nunca se contagian).
4. Mostrar la hora del último contagio y cuántos no se contagian.

Salida esperada

```
2.SS#54
10.#543
21.#..2
#232101
434#212
Horas hasta el último contagio: 5 | Pacientes que no se contagian: 2
```

Reglas

- Complejidad esperada: O(filas × columnas).
- Arrancar el BFS con todas las fuentes a la vez da la hora correcta; hacer un BFS por fuente y quedarse con el mínimo es mucho más lento.
- Las camas vacías (.) no contagian ni se contagian. En el dibujo, las horas se muestran con un dígito (en el ejemplo no superan 9).

Pista (leela solo si te trabás): el BFS multi-fuente es exactamente un BFS normal con varios nodos en la cola desde el principio: la cola sigue ordenada por hora.

Bonus (opcional)

- Agregá un tiempo de recuperación: un paciente deja de contagiar 3 horas después de contagiarse.
- Decidí en qué 2 camas conviene aislar pacientes para minimizar los contagios.

Cierre: Es el BFS multi-fuente, usado en epidemiología, en propagación de incendios y en el cálculo de distancias a la estación más cercana.

Ejercicio 273: Dos cosechas del mismo peso

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Agricultura · Dificultad 55/100

Situación: Un productor tiene varios bolsones de cosecha de distintos pesos y dos camiones iguales para llevarlos al acopio. Quiere repartirlos lo más parejo posible (idealmente, el mismo peso en cada camión), sin abrir bolsones.

Datos de entrada

Formato: una línea con los pesos de los bolsones (toneladas enteras).

Ejemplo

```
12 7 5 9 14 3 8 6
```

Restricciones: hasta 200 bolsones; suma total hasta 10^5 .

Tu programa debe

1. Calcular todas las sumas alcanzables con subconjuntos, guardando de qué suma anterior y con qué bolsón se llegó a cada una (procesando los bolsones en orden).
2. Elegir la mayor suma alcanzable que no supere la mitad del total.
3. Reconstruir los bolsones del camión 1; el resto va al camión 2.
4. Mostrar el reparto y la diferencia.

Salida esperada

```
Peso total: 64 t
Camión 1: 12 + 7 + 5 + 8 = 32 t
Camión 2: 9 + 14 + 3 + 6 = 32 t
Diferencia: 0 t (reparto perfecto)
```

Caso 2 (borde)

No existe un reparto perfecto:

```
10 1
```

Salida esperada del caso 2:

```
Peso total: 11 t
Camión 1: 1 = 1 t
Camión 2: 10 = 10 t
Diferencia: 9 t
```

Reglas

- Complejidad esperada: $O(n \times \text{suma total})$.
- Una suma se registra la primera vez que se alcanza (ese registro define la reconstrucción).
- Los bolsones del camión 1 se muestran en el orden en que se reconstruyen invertido (orden de la entrada).

Pista (leela solo si te trabás): es suma de subconjuntos: la pregunta "¿se puede armar exactamente S?" se responde para todos los S a la vez, agregando un bolsón por vez.

Bonus (opcional)

- Repartí entre tres camiones (el problema se vuelve bastante más difícil).
- Con sumas de 10^9 la tabla no entra en memoria: probá meet in the middle para 40 bolsones.

Cierre: Es el problema de partición, usado para balancear cargas en camiones, servidores y máquinas.

Ejercicio 274: Combinar vuelos de varias aerolíneas

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Turismo · Dificultad 55/100

Situación: Un buscador de vuelos recibe de cada aerolínea su lista de salidas del día, ya ordenada por horario. Tiene que mostrar un único listado cronológico. Con cientos de aerolíneas, intercalarlas de a pares es lento; la idea es usar un heap con el próximo vuelo de cada lista.

Datos de entrada

Formato: primera línea: cantidad de aerolíneas k ; después, una línea por aerolínea "nombre: HH:MM código, HH:MM código, ..." (ordenada).

Ejemplo

```
3
Aerolíneas_Sur: 06:10 AS100, 08:45 AS210, 13:20 AS330, 19:05 AS450
Vuela_Ya: 07:00 VY11, 08:45 VY19, 11:30 VY27
Andes_Air: 05:55 AA7, 09:15 AA8, 13:20 AA9, 22:40 AA10
```

Restricciones: hasta 10^4 listas y 10^7 vuelos en total.

Tu programa debe

1. Poner en un heap el primer vuelo de cada lista, junto con el número de lista y la posición.
2. Sacar el menor, mostrarlo y agregar el siguiente vuelo de esa misma lista.
3. Repetir hasta vaciar el heap.
4. En empate de horario, sale primero el de la aerolínea que aparece antes en la entrada.

Salida esperada

```
05:55 AA7
06:10 AS100
07:00 VY11
08:45 AS210
08:45 VY19
09:15 AA8
11:30 VY27
13:20 AS330
13:20 AA9
19:05 AS450
22:40 AA10
```

Reglas

- Complejidad esperada: $O(N \log k)$ con N vuelos en total y k listas.
- El heap nunca tiene más de k elementos.
- Los horarios HH:MM con dos dígitos se pueden comparar como texto.

Pista (leela solo si te trabás): es el paso de mezcla del merge sort generalizado a k listas: el heap te dice en $O(\log k)$ cuál de los k "frentes" es el menor.

Bonus (opcional)

- Mostrá solo los primeros 10 vuelos después de una hora dada sin recorrer las listas completas (búsqueda binaria en cada lista para arrancar).
- Combiná además con listas de trenes y micros para armar conexiones multimodales.

Cierre: Es la fusión de k listas ordenadas, usada en buscadores de viajes, en bases de datos y en el ordenamiento externo de archivos gigantes.

Ejercicio 275: Cortes de tela en la fábrica

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: E-commerce · Dificultad 55/100

Situación: Una fábrica textil vende retazos de tela de largos enteros, y el precio no es proporcional al largo (algunos largos se venden mucho mejor). Tiene un rollo de 10 metros y quiere decidir cómo cortarlo para maximizar lo que obtiene por la venta.

Datos de entrada

Formato: primera línea: largo del rollo n ; después, "largo precio" para cada largo de retazo que se puede vender.

Ejemplo

```
10
1 1200
2 2800
3 4500
4 5400
5 7000
6 8200
7 9900
8 10600
9 12000
10 13500
```

Restricciones: n hasta 10^4 .

Tu programa debe

1. Armar $dp[m]$ = mayor ingreso posible con un rollo de m metros.
2. Para cada m , probar cada primer corte c (de 1 a m): $precio[c] + dp[m - c]$. Guardar el primer c que da el máximo.
3. Reconstruir los cortes siguiendo el corte guardado.
4. Mostrar el ingreso máximo, los cortes (ordenados) y cuánto valdría sin cortar.

Salida esperada

```
Ingreso máximo: $14700
Cortes: 1 + 3 + 3 + 3 metros
Sin cortar valdría: $13500
```

Reglas

- Complejidad esperada: $O(n^2)$.
- Si varios primeros cortes empatan, se guarda el más chico (el primero que se encuentra al probar de 1 a m).
- Casos borde: conviene no cortar; rollo de 1 metro.

Pista (leela solo si te trabás): cualquier forma de cortar tiene un primer corte; después de hacerlo, lo que queda es el mismo problema más chico. Eso es la subestructura óptima.

Bonus (opcional)

- Agregá un costo fijo de \$300 por cada corte y recalculá.
- Con telas de dos dimensiones (cortes de rectángulos), la recurrencia pasa a tener dos parámetros.

Cierre: Es el problema del corte de varillas, usado en la industria textil, maderera y del acero para minimizar desperdicio y maximizar ventas.

Lote 12: ejercicios 276 a 300

Ejercicio 276: El árbol de decisión del crédito

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 56/100

Situación: Una fintech define su política de créditos como un árbol de decisión que editan los analistas de riesgo desde un panel. Un error de carga (una rama que apunta a un nodo borrado, o un ciclo) puede aprobar créditos que no debería. Antes de usar el árbol, el sistema tiene que validarlo; después, evaluar a los solicitantes mostrando el camino recorrido.

Datos de entrada

Formato: una línea por nodo. Pregunta: "id campo operador valor id_si id_no" (operadores $\geq$, $=$, $<$). Hoja: "id resultado" (los id de hojas empiezan con H_). El primer nodo es la raíz. Después, una línea "?" y un solicitante por línea "nombre campo=valor ...".

Ejemplo

```
N1 ingreso >= 500000 N2 N3
N2 deudas == 0 N4 H_REVISAR
N3 antigüedad >= 24 H_REVISAR H_RECHAZO
N4 edad >= 21 H_APROBADO H_REVISAR
H_APROBADO APROBADO
H_REVISAR REVISIÓN MANUAL
H_RECHAZO RECHAZADO
?
Ana ingreso=820000 deudas=0 antigüedad=30 edad=35
Beto ingreso=420000 deudas=1 antigüedad=10 edad=40
Caro ingreso=650000 deudas=2 antigüedad=5 edad=29
Dani ingreso=300000 deudas=0 antigüedad=36 edad=19
```

Restricciones: hasta 10^5 nodos y 10^6 solicitantes.

Tu programa debe

1. Validar el árbol: que todos los nodos referenciados existan, que no haya ciclos y que todos los nodos sean alcanzables desde la raíz.
2. Si hay errores, listarlos y no evaluar.
3. Si es válido, recorrerlo para cada solicitante desde la raíz según sus datos hasta llegar a una hoja.
4. Mostrar la decisión y el camino (nodo:respuesta).

Salida esperada

```
Árbol válido
Ana: APROBADO (N1:sí > N2:sí > N4:sí)
Beto: RECHAZADO (N1:no > N3:no)
Caro: REVISIÓN MANUAL (N1:sí > N2:no)
Dani: REVISIÓN MANUAL (N1:no > N3:sí)
```

Caso 2 (borde)

Un árbol mal cargado:

```
N1 ingreso >= 500000 N2 N9
N2 deudas == 0 H_OK N1
H_OK APROBADO
H_HUERFANA RECHAZADO
?
```

Salida esperada del caso 2:

```
Árbol inválido:
N1 apunta a N9, que no existe
ciclo que pasa por N1
H_HUERFANA es inalcanzable
```

Reglas

- Complejidad esperada: $O(\text{nodos})$ para validar y $O(\text{profundidad})$ por solicitante.
- Los errores se informan en este orden: referencias inexistentes (en el orden de los nodos), ciclos (en el orden del DFS desde la raíz) y nodos inalcanzables.
- El camino se muestra con " > " entre pasos.

Pista (leela solo si te trabás): validar es un DFS con tres estados, igual que la detección de ciclos: si en un árbol encontrás un nodo "en curso", no es un árbol.

Bonus (opcional)

- Detectá ramas redundantes: una pregunta cuyas dos respuestas llevan al mismo resultado.
- Calculá qué porcentaje de solicitantes termina en cada hoja para detectar reglas que nunca se usan.

Cierre: Es la validación y evaluación de árboles de decisión, usados en scoring crediticio, triage médico y motores de reglas.

Ejercicio 277: Pagos circulares entre cuentas

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 56/100

Situación: El área antilavado de un banco busca patrones de "dinero que da la vuelta": una cuenta transfiere a otra, que transfiere a otra, y así hasta que la plata vuelve a una cuenta ya recorrida. Es una señal clásica para ocultar el origen de los fondos. Hay millones de transferencias por día.

Datos de entrada

Formato: una transferencia por línea "origen destino monto".

Ejemplo

```
C01 C02 500000
C02 C03 480000
C03 C04 470000
C04 C02 460000
C05 C06 20000
C06 C07 15000
C03 C08 5000
```

Restricciones: hasta 10^6 cuentas y 10^7 transferencias.

Tu programa debe

1. Armar el grafo dirigido de transferencias.
2. Hacer un DFS con tres estados, recorriendo las cuentas y sus transferencias en el orden de la entrada.
3. Al encontrar una transferencia hacia una cuenta en curso, reconstruir el ciclo desde la pila.
4. Mostrar el primer ciclo encontrado, los montos de cada tramo y el mínimo.

Salida esperada

```
ALERTA: pagos circulares C02 -> C03 -> C04 -> C02
Montos: 480000 -> 470000 -> 460000
Monto mínimo que dio la vuelta completa: 460000
```

Caso 2 (borde)

Transferencias sin ciclos:

```
A B 100
B C 50
```

Salida esperada del caso 2:

```
Sin pagos circulares
```

Reglas

- Complejidad esperada: $O(V + E)$.
- Se informa solo el primer ciclo que encuentra el recorrido descripto.
- Con grafos de millones de cuentas, usá una pila explícita.
- El monto mínimo del ciclo es una cota de cuánto dinero pudo dar la vuelta completa.

Pista (leela solo si te trabás): el ciclo está en la pila del DFS: desde la cuenta a la que volviste hasta el tope.

Bonus (opcional)

- Listá todos los ciclos simples de hasta 5 cuentas.

- Agregá la dimensión temporal: el ciclo solo es sospechoso si todas las transferencias ocurrieron en menos de 48 horas y en orden.

Cierre: Es la detección de ciclos en grafos dirigidos, una herramienta estándar de los sistemas antilavado y antifraude.

Ejercicio 278: Palabras escondidas en el tablero

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Videojuegos · Dificultad 56/100

Situación: Un juego de palabras para celular muestra un tablero de letras: hay que formar palabras uniendo letras vecinas (también en diagonal), sin usar dos veces la misma casilla. El servidor tiene que validar las palabras que manda cada jugador y devolver el recorrido para animarlo en pantalla.

Datos de entrada

Formato: las filas del tablero; una línea "?" y las palabras a verificar separadas por espacios.

Ejemplo

```
MATE
OSAL
RIEP
TUNA
?
MATE SALIR MOTOR RIO PANEL LUNA TIERRA
```

Restricciones: tableros de hasta 6×6 ; palabras de hasta 16 letras; hasta 10^4 palabras por partida.

Tu programa debe

1. Para cada palabra, probar empezar en cada casilla (fila por fila, de izquierda a derecha).
2. Avanzar con backtracking a las 8 vecinas (en el orden: arriba-izquierda, arriba, arriba-derecha, izquierda, derecha, abajo-izquierda, abajo, abajo-derecha) sin reutilizar casillas.
3. Deshacer la elección al volver atrás.
4. Mostrar el primer recorrido encontrado (posiciones desde 1) o "no está", y el total de encontradas.

Salida esperada

```
MATE: encontrada (1,1) (1,2) (1,3) (1,4)
SALIR: no está
MOTOR: no está
RIO: encontrada (3,1) (3,2) (2,1)
PANEL: encontrada (3,4) (4,4) (4,3) (3,3) (2,4)
LUNA: no está
TIERRA: no está
Encontradas: 3
```

Reglas

- Complejidad: exponencial en el largo de la palabra en el peor caso (hasta 8^{largo}), pero la poda por letra la vuelve instantánea en la práctica.
- Cada casilla se usa como máximo una vez por palabra.
- El orden de exploración define el recorrido mostrado: respetalo.

Pista (leela solo si te trabás): la poda es inmediata: si la letra de la casilla no coincide con la que buscás, se corta sin explorar sus vecinas.

Bonus (opcional)

- Encontrá todas las palabras de un diccionario de 100.000 palabras que están en el tablero (usá un trie para podar prefijos inexistentes, nivel 7).
- Asigná puntaje por largo y calculá el máximo puntaje posible del tablero.

Cierre: Es el backtracking en grilla que usan los juegos de palabras y, en general, la búsqueda de patrones en matrices.

Ejercicio 279: Escalera de palabras

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Educación · Dificultad 56/100

Situación: Una plataforma educativa de lengua propone el juego de la escalera: transformar una palabra en otra cambiando una letra por vez, y cada paso intermedio tiene que ser una palabra válida. El profe quiere un programa que encuentre la escalera más corta dentro del vocabulario de la unidad.

Datos de entrada

Formato: primera línea: palabra inicial y final; segunda línea: el diccionario (palabras del mismo largo, en minúsculas).

Ejemplo

```
casa mesa
casa cosa masa mesa mema meta pesa rosa cesa caso case
```

Restricciones: hasta 10^5 palabras de hasta 10 letras.

Tu programa debe

1. Tratar cada palabra como un nodo; dos palabras están conectadas si difieren en exactamente una letra.
2. No armar el grafo completo: generar los vecinos de cada palabra probando reemplazar cada posición por cada letra (a-z con ñ, en ese orden) y ver si está en el diccionario.
3. Hacer un BFS desde la palabra inicial y reconstruir el camino.
4. Mostrar la escalera más corta o que no existe.

Salida esperada

```
Escalera más corta (2 pasos): casa -> masa -> mesa
```

Caso 2 (borde)

No hay camino dentro del diccionario:

```
sol mar
sol sal mal
```

Salida esperada del caso 2:

```
No hay escalera de sol a mar
```

Reglas

- Complejidad esperada: $O(\text{palabras} \times \text{largo} \times 27)$ consultas al conjunto.
- Comparar cada par de palabras sería $O(n^2 \times \text{largo})$: con 10^5 palabras no pasa.
- El orden de generación de vecinos (posiciones de izquierda a derecha, letras en orden alfabético) define qué escalera se muestra cuando hay varias de igual largo.

Pista (leela solo si te trabás): el grafo está "implícito": no lo guardás, lo vas generando. Cada palabra tiene como mucho $\text{largo} \times 27$ vecinos candidatos.

Bonus (opcional)

- Usá BFS bidireccional (desde las dos puntas) y medí cuántos nodos menos explora.
- Mostrá todas las escaleras de largo mínimo.

Cierre: Es el BFS sobre grafos implícitos, la técnica que resuelve rompecabezas, juegos de estados y búsquedas en espacios enormes.

Ejercicio 280: El planificador del router

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Telecomunicaciones · Dificultad 56/100

Situación: El router de una empresa transmite paquetes por un enlace de a uno por vez, y prioriza el tráfico de voz (clase 1) sobre el de video (clase 2) y el de datos (clase 3). Tiene un buffer chico: si está lleno cuando llega un paquete, el paquete se descarta. El equipo de redes quiere simular una ráfaga para ver qué se pierde.

Datos de entrada

Formato: primera línea: capacidad del buffer (paquetes en espera, sin contar el que se está transmitiendo); después, "id llegada clase tamaño" (tiempos en milisegundos; el tamaño es lo que tarda en transmitirse).

Ejemplo

```
4
P1 0 2 3
P2 1 1 4
P3 1 3 2
P4 2 1 1
P5 2 2 2
P6 3 3 1
P7 3 1 2
P8 4 2 1
```

Restricciones: hasta 10^7 paquetes; clases de 1 a 8.

Tu programa debe

1. Procesar las llegadas en orden de tiempo; un paquete que llega cuando el buffer está lleno se descarta.
2. Cuando el enlace se libera, transmitir el paquete del buffer con menor clase (en empate, el que llegó antes).
3. Las llegadas que ocurren justo cuando el enlace se libera entran al buffer antes de elegir.
4. Mostrar cada transmisión, cada descarte y la lista de descartados.

Salida esperada

```
t=0: transmite P1 (clase 2, llegó 0, tamaño 3) hasta t=3
t=3: P6 DESCARTADO (buffer lleno)
t=3: P7 DESCARTADO (buffer lleno)
t=3: transmite P2 (clase 1, llegó 1, tamaño 4) hasta t=7
t=7: transmite P4 (clase 1, llegó 2, tamaño 1) hasta t=8
t=8: transmite P5 (clase 2, llegó 2, tamaño 2) hasta t=10
t=10: transmite P8 (clase 2, llegó 4, tamaño 1) hasta t=11
t=11: transmite P3 (clase 3, llegó 1, tamaño 2) hasta t=13
Descartados: P6, P7
```

Reglas

- Complejidad esperada: $O(n \log b)$ con b la capacidad del buffer.
- Mientras el enlace está ocupado, los paquetes que llegan se acumulan (y se descartan si no hay lugar).
- El paquete que se está transmitiendo no ocupa lugar en el buffer.

Pista (leela solo si te trabás): es la misma simulación por eventos que la guardia con triage, con un detalle nuevo: el tamaño limitado de la cola define las pérdidas.

Bonus (opcional)

- En lugar de descartar el que llega, descartá el de peor clase del buffer si el que llega es mejor (se necesita sacar el máximo y el mínimo: dos heaps o un árbol balanceado).
- Evitá que la clase 3 espere para siempre: reservale un paquete cada 5 transmisiones.

Cierre: Es la planificación con colas de prioridad y descarte que usan los routers para garantizar calidad de servicio en llamadas y videos.

Ejercicio 281: Cuentas del mismo titular

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 57/100

Situación: Un banco digital sospecha que algunas personas abren varias cuentas para aprovechar promociones de bienvenida. Dos cuentas pertenecen al mismo titular si comparten algún dato (email, teléfono o documento), directamente o a través de una cadena de cuentas. Te piden agrupar las cuentas por titular.

Datos de entrada

Formato: una cuenta por línea: "id dato1 dato2 ..." (emails, teléfonos o documentos, sin distinguir el tipo).

Ejemplo

```
A1 ana@mail.com 1155550001
A2 ana.paz@work.com 1155550001
A3 beto@mail.com 1155550002
A4 ana.paz@work.com 30111222
A5 caro@mail.com 1155550003
A6 beto.r@mail.com 1155550002
A7 dani@mail.com 30999888
```

Restricciones: hasta 10^7 cuentas y $3 \cdot 10^7$ datos.

Tu programa debe

1. Usar union-find sobre las cuentas y un mapa dato → primera cuenta que lo tuvo.
2. Por cada dato de una cuenta, si ya estaba en el mapa, unir la cuenta con la que lo tenía.
3. Agrupar las cuentas por representante.
4. Mostrar los grupos (en el orden de su primera cuenta, y dentro de cada grupo en orden de aparición) y los totales.

Salida esperada

```
Titular 1: A1, A2, A4
Titular 2: A3, A6
Titular 3: A5
Titular 4: A7
Cuentas: 7 | Titulares distintos: 4
```

Reglas

- Complejidad esperada: casi $O(\text{total de datos})$ con union-find.
- La relación es transitiva: A1 comparte teléfono con A2 y A2 comparte email con A4, así que A1 y A4 son del mismo titular aunque no compartan nada directamente.
- Casos borde: cuentas sin datos en común con nadie (grupos de una sola cuenta).

Pista (leela solo si te trabás): el mapa de datos convierte "comparar todas las cuentas entre sí" (cuadrático) en "¿alguien ya usó este dato?" (constante).

Bonus (opcional)

- Normalizá los datos antes (emails en minúscula, teléfonos sin prefijos) para no perder coincidencias.
- Marcá como sospechosos los titulares con más de 3 cuentas abiertas en la misma semana.

Cierre: Es la resolución de entidades con union-find, usada en prevención de fraude, en CRM y en unificación de bases de datos.

Ejercicio 282: Tendido eléctrico sobre terreno irregular

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Clima y energía · Dificultad 57/100

Situación: Una cooperativa eléctrica tiene que llevar una línea desde la esquina noroeste de un campo hasta la esquina sureste. El terreno está dividido en parcelas con distintos costos de tendido (según pendiente, rocas, bosque). Por cuestiones técnicas, la línea solo puede avanzar hacia el este o hacia el sur. Buscan el tendido más barato.

Datos de entrada

Formato: primera línea: filas y columnas; después la grilla de costos por parcela (millones).

Ejemplo

```
5 6
3 1 4 8 2 7
2 9 1 3 5 1
6 2 8 1 1 4
4 7 2 6 3 2
1 5 3 2 8 1
```

Restricciones: hasta 5.000×5.000 parcelas; costos de 1 a 10^4 .

Tu programa debe

1. Armar $dp[r][c]$ = costo mínimo para llegar a la parcela (r, c) desde (0, 0), sumando el costo de cada parcela atravesada (incluidas la inicial y la final).
2. Cada celda se alcanza desde arriba o desde la izquierda: dp = mínimo de esas dos + costo propio.
3. Reconstruir el camino desde el final: ir hacia arriba si su dp es menor o igual que el de la izquierda; si no, hacia la izquierda.
4. Dibujar la grilla marcando el camino con corchetes y mostrar el costo.

Salida esperada

```
[3][1][4] 8 2 7
2 9 [1][3] 5 1
6 2 8 [1][1] 4
4 7 2 6 [3][2]
1 5 3 2 8 [1]
Costo mínimo del tendido: 20 (millones)
```

Caso 2 (borde)

Un terreno de una sola parcela:

```
1 1
5
```

Salida esperada del caso 2:

```
[5]
Costo mínimo del tendido: 5 (millones)
```

Reglas

- Complejidad esperada: $O(\text{filas} \times \text{columnas})$.
- Con 5.000×5.000 , guardar toda la tabla usa mucha memoria: se puede calcular el costo con una sola fila (pero para reconstruir el camino hace falta más información).
- La regla de desempate (preferir venir de arriba) define el camino dibujado.

Pista (leela solo si te trabás): como solo se avanza al este o al sur, cuando llegás a una celda ya conocés el mejor costo de las dos únicas celdas desde las que se puede llegar.

Bonus (opcional)

- Permití también avanzar en diagonal (sureste) con costo multiplicado por 1,4.
- Si se puede mover en las 4 direcciones, la programación dinámica ya no alcanza: usá Dijkstra.

Cierre: Es la programación dinámica en grilla, usada en planificación de tendidos, en alineamiento de secuencias y en procesamiento de imágenes.

Ejercicio 283: La sala más profunda del museo

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Turismo · Dificultad 57/100

Situación: Un museo arqueológico tiene salas conectadas por pasillos formando un árbol (no hay circuitos). La dirección quiere saber cuál es la sala más alejada de la entrada (para ubicar la señalética y las salidas de emergencia) y el recorrido más largo posible entre dos salas cualesquiera.

Datos de entrada

Formato: un pasillo por línea "sala_a sala_b metros"; las conexiones forman un árbol.

Ejemplo

```
Entrada Hall 10
Hall Egipto 25
Hall Roma 15
Roma Pompeya 30
Roma Catacumbas 45
Egipto Tumba 20
Tumba Sarcófago 35
Hall Cafetería 5
```

Restricciones: hasta 10^6 salas.

Tu programa debe

1. Hacer un recorrido en profundidad desde la entrada acumulando metros, y encontrar la sala más alejada.
2. Para el recorrido más largo entre dos salas (diámetro del árbol): desde la sala más alejada de la entrada, hacer otro recorrido y encontrar la más alejada de ella.
3. Reconstruir ese recorrido.
4. Mostrar la sala más alejada de la entrada, el diámetro y su camino.

Salida esperada

```
Sala más alejada de la entrada: Sarcófago (90 m)
Recorrido más largo entre dos salas: 140 m
Catacumbas - Roma - Hall - Egipto - Tumba - Sarcófago
```

Reglas

- Complejidad esperada: $O(n)$ (dos recorridos).
- Usá una pila explícita: con un millón de salas en fila, la recursión se desborda.
- En empate de distancia, se elige la sala que aparece primero en la entrada.
- Por qué funciona: en un árbol, la sala más alejada de cualquier punto es siempre un extremo de algún camino más largo.

Pista (leela solo si te trabás): dos recorridos alcanzan: el primero encuentra un extremo del diámetro; el segundo, desde ese extremo, encuentra el otro.

Bonus (opcional)

- Encontrá la sala "central": la que minimiza la distancia máxima a cualquier otra (ideal para el puesto de información).
- Si los pasillos formaran circuitos, ¿seguiría funcionando este método? Mostrá un contraejemplo.

Cierre: Es el diámetro de un árbol, usado para diseñar redes, ubicar servicios y analizar recorridos en edificios.

Ejercicio 284: ¿Se pueden armar dos bandos rivales?

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Deportes · Dificultad 57/100

Situación: En un torneo de truco por parejas enfrentadas, los organizadores quieren dividir a los jugadores en dos bandos de modo que cada par de jugadores que se declararon rivales quede en bandos distintos. No siempre es posible (si hay tres rivales mutuos, alguno queda del mismo lado), y el sistema tiene que avisar.

Datos de entrada

Formato: un par de rivales por línea "jugador1 jugador2".

Ejemplo

```
Aguirre Benítez
Benítez Castro
Castro Díaz
Díaz Aguirre
Espinoza Fonseca
Fonseca Gutiérrez
```

Restricciones: hasta 10^6 jugadores y 10^7 rivalidades.

Tu programa debe

1. Armar el grafo de rivalidades.
2. Recorrer cada componente con BFS, empezando por el primer jugador no asignado (en orden de aparición) en el bando Celeste y alternando bandos entre vecinos.
3. Si dos rivales quedan en el mismo bando, informar que es imposible (con los dos jugadores del conflicto).
4. Si no, mostrar los dos bandos en orden de aparición.

Salida esperada

```
Bando Celeste: Aguirre, Castro, Espinoza, Gutiérrez
Bando Blanco: Benítez, Díaz, Fonseca
```

Caso 2 (borde)

Tres jugadores rivales entre sí:

```
A B
B X
X A
```

Salida esperada del caso 2:

```
IMPOSIBLE: B y X son rivales y quedarían en el mismo bando
```

Reglas

- Complejidad esperada: $O(V + E)$.
- Es posible si y solo si el grafo no tiene ciclos de largo impar (grafo bipartito).
- Cada componente se colorea por separado: jugadores sin relación entre grupos pueden ir a cualquier bando.

Pista (leela solo si te trabás): al pintar un jugador de un color, sus rivales quedan obligados al otro color. Si alguna vez un rival ya tiene tu mismo color, no hay forma de arreglarlo.

Bonus (opcional)

- Si es imposible, encontrá el ciclo impar que lo impide.

- Balanceá los bandos: como cada componente puede invertirse, elegí las orientaciones que dejan los bandos lo más parejos posible.

Cierre: Es la verificación de grafos bipartitos, usada en planificación con conflictos, asignación de turnos y detección de inconsistencias.

Ejercicio 285: Guardar y recuperar el menú anidado

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Gastronomía · Dificultad 57/100

Situación: Una app de restaurantes guarda la carta como un árbol de categorías y la tiene que mandar a los celulares como una sola línea de texto compacta. Del otro lado, el celular reconstruye el árbol. Te piden las dos funciones (serializar y deserializar) y verificar que el viaje de ida y vuelta no pierde nada.

Datos de entrada

Formato: la carta con una categoría por línea y 2 espacios de sangría por nivel.

Ejemplo

```
Carta
  Entradas
    Empanadas
    Provoleta
  Principales
    Carnes
      Bife
      Vacío
    Pastas
      Ñoquis
  Postres
    Flan
```

Restricciones: hasta 10^6 nodos; los nombres no contienen paréntesis ni comas.

Tu programa debe

1. Construir el árbol a partir de la sangría (usando una pila de ancestros).
2. Serializar en preorden: nombre y, si tiene hijos, sus serializaciones separadas por coma entre paréntesis.
3. Deserializar la línea de texto reconstruyendo el árbol.
4. Mostrar la serialización, la cantidad de nodos y si el árbol reconstruido es idéntico al original.

Salida esperada

```
Serializado:
  Carta(Entradas(Empanadas,Provoleta),Principales(Carnes(Bife,Vacío),Pastas(Ñoquis)),Postres(Fla
n))
Nodos: 12
Ida y vuelta idéntica: sí
```

Reglas

- Complejidad esperada: $O(\text{tamaño del texto})$ para ambas operaciones.
- La pila de ancestros resuelve la sangría: al leer un nivel menor, se desapilan los niveles más profundos.
- Para árboles muy profundos, reescribí la deserialización sin recursión.

Pista (leela solo si te trabás): la serialización en preorden con paréntesis es suficiente para reconstruir el árbol sin ambigüedad: los paréntesis marcan exactamente dónde empiezan y terminan los hijos de cada nodo.

Bonus (opcional)

- Escapá los caracteres especiales para permitir nombres con paréntesis y comas.
- Compará el tamaño con una serialización en formato clave-valor estándar y con una binaria.

Cierre: Es la serialización de árboles, la base de formatos como JSON y XML y de cómo viajan los datos entre servidores y apps.

Ejercicio 286: Monedas con denominaciones raras

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 57/100

Situación: Un parque temático usa fichas de valores raros (1, 6, 9 y 20 puntos) y las máquinas tienen que dar el vuelto con la menor cantidad de fichas. Con estos valores el algoritmo voraz falla (ya lo viste en el nivel 5); ahora hay que encontrar el óptimo de forma eficiente para cualquier monto.

Datos de entrada

Formato: primera línea: los valores de las fichas; segunda: los montos a pagar.

Ejemplo

```
1 6 9 20
12 18 24 29 35
```

Restricciones: hasta 100 valores de ficha; montos hasta 10^6 .

Tu programa debe

1. Armar $dp[v]$ = menor cantidad de fichas para pagar v ($dp[0] = 0$, infinito si es imposible).
2. Para cada v , probar cada ficha $d \leq v$: $dp[v - d] + 1$, guardando la ficha usada (se elige la primera que mejora, en el orden dado).
3. Reconstruir la composición y compararla con el voraz.
4. Mostrar, para cada monto, el óptimo, su composición (de mayor a menor) y el resultado del voraz.

Salida esperada

```
12: óptimo 2 (6 + 6) | voraz 4
18: óptimo 2 (9 + 9) | voraz 2
24: óptimo 3 (9 + 9 + 6) | voraz 5
29: óptimo 2 (20 + 9) | voraz 2
35: óptimo 3 (20 + 9 + 6) | voraz 3
```

Caso 2 (borde)

Un sistema sin moneda de 1:

```
4 7
5 11 12
```

Salida esperada del caso 2:

```
5: imposible
11: óptimo 2 (7 + 4) | voraz 2
12: óptimo 3 (4 + 4 + 4) | voraz imposible
```

Reglas

- Complejidad esperada: $O(\text{monto máximo} \times \text{fichas})$.
- Los montos imposibles se informan como tales.
- Una tabla calculada hasta el monto máximo sirve para todos los montos.

Pista (leela solo si te trabás): el óptimo para v es 1 + el mejor óptimo entre $v - 1$, $v - 6$, $v - 9$ y $v - 20$: una ficha menos y el problema es más chico.

Bonus (opcional)

- Con cantidades limitadas de cada ficha, adaptá la tabla.
- Encontrá todos los montos hasta 100 donde el voraz falla con estas fichas.

Cierre: Es el problema del cambio con programación dinámica, el ejemplo clásico de cuándo un voraz no alcanza y hace falta explorar sistemáticamente.

Ejercicio 287: Trenes con horarios fijos

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Transporte · Dificultad 58/100

Situación: Una app de transporte ferroviario tiene la grilla de horarios de todos los trenes y los usuarios preguntan "si estoy en Retiro a las 7:00, ¿a qué hora es lo más temprano que puedo llegar a Pilar?". Los trenes salen en horarios fijos, así que no alcanza con sumar duraciones: hay que esperar el próximo tren.

Datos de entrada

Formato: primera línea: estación de origen, hora en que el usuario está ahí y destino; después, un tren por línea "desde hasta salida llegada".

Ejemplo

```
Retiro 07:00 Pilar
Retiro Belgrano 07:05 07:15
Retiro Belgrano 07:20 07:30
Belgrano Victoria 07:18 07:40
Belgrano Victoria 07:35 07:55
Retiro Victoria 07:10 08:05
Victoria Pilar 07:45 08:30
Victoria Pilar 08:00 08:40
Belgrano Pilar 07:32 08:50
```

Restricciones: hasta 10^7 conexiones por día y 10^4 estaciones.

Tu programa debe

1. Ordenar las conexiones por horario de salida (en empate, por llegada).
2. Recorrerlas una vez: una conexión se puede tomar si ya se llegó a su estación de salida a esa hora o antes; si mejora la llegada a su destino, se actualiza (guardando por qué conexión se llegó).
3. Reconstruir el viaje desde el destino.
4. Mostrar los tramos y la llegada más temprana.

Salida esperada

```
Retiro 07:05 -> Belgrano 07:15
Belgrano 07:18 -> Victoria 07:40
Victoria 07:45 -> Pilar 08:30
Llegada más temprana a Pilar: 08:30
```

Reglas

- Complejidad esperada: $O(\text{conexiones} \log \text{conexiones})$ por el ordenamiento y $O(\text{conexiones})$ por consulta.
- Transbordo instantáneo: se puede tomar un tren que sale a la misma hora en que se llegó.
- Es el algoritmo de escaneo de conexiones: una variante de Dijkstra para horarios fijos.

Pista (leela solo si te trabás): recorrer las conexiones en orden de salida garantiza que, cuando mirás un tren, ya sabés la mejor hora de llegada a su estación de origen entre todos los trenes que salen antes.

Bonus (opcional)

- Agregá tiempos mínimos de transbordo por estación (5 minutos para cambiar de andén).
- Mostrá también la opción con menos transbordos aunque llegue un poco más tarde (optimización con dos criterios).

Cierre: Es el problema de rutas en horarios fijos que resuelven los planificadores de transporte público y las apps de viajes.

Ejercicio 288: Reposición con prioridades que cambian

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Logística y envíos · Dificultad 58/100

Situación: Un centro de distribución mantiene una cola de reposición: el producto más urgente se repone primero. La urgencia de cada producto cambia todo el tiempo según las ventas, y sacar un elemento del medio de un heap es caro. El equipo decide usar "borrado perezoso": en cada cambio se agrega una entrada nueva y las viejas se descartan al salir.

Datos de entrada

Formato: una operación por línea: "alta producto urgencia", "actualizar producto urgencia" o "atender" (reponer el más urgente).

Ejemplo

```
alta Leche 50
alta Pan 80
alta Yerba 30
alta Aceite 60
atender
actualizar Yerba 95
actualizar Leche 10
atender
atender
alta Arroz 70
atender
atender
atender
```

Restricciones: hasta 10^7 operaciones.

Tu programa debe

1. Guardar en un mapa la urgencia vigente de cada producto pendiente.
2. En cada alta o actualización, agregar al heap una entrada (urgencia, orden de inserción, producto).
3. Al atender, sacar entradas del heap hasta encontrar una que coincida con la urgencia vigente del producto; las demás son viejas y se descartan.
4. Mostrar cada atención (con cuántas entradas viejas se descartaron) o que no hay nada pendiente.

Salida esperada

```
atender: Pan (urgencia 80)
atender: Yerba (urgencia 95)
atender: Aceite (urgencia 60)
atender: Arroz (urgencia 70)
atender: Leche (urgencia 10) [descartadas 2 entrada(s) vieja(s)]
atender: no hay nada pendiente
```

Reglas

- Complejidad esperada: $O(\log n)$ amortizado por operación.
- Mayor urgencia primero; en empate, la entrada insertada antes.
- Un producto atendido sale del mapa: sus entradas que queden en el heap también son viejas.

Pista (leela solo si te trabás): en lugar de buscar y modificar la entrada vieja (caro), dejala en el heap y marcá como válida solo la última. Al salir, verificás contra el mapa.

Bonus (opcional)

- Si hay demasiadas entradas viejas (más del doble de las válidas), reconstruí el heap desde cero.
- Implementá un heap indexado que permita actualizar la prioridad en $O(\log n)$ sin entradas viejas.

Cierre: Es el borrado perezoso en colas de prioridad, la misma técnica que usa Dijkstra y muchos planificadores con prioridades cambiantes.

Ejercicio 289: Mutaciones con costos distintos

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Ciencia · Dificultad 58/100

Situación: Un laboratorio de genética compara secuencias de ADN de dos cepas de una bacteria. No todas las mutaciones son igual de probables: cambiar una purina por otra purina ($A \leftrightarrow G$) o una pirimidina por otra ($C \leftrightarrow T$) es más común (transición) que cruzar de grupo (transversión), y las inserciones o borrados son todavía más raros. El costo de alinear las secuencias tiene que reflejar eso.

Datos de entrada

Formato: dos líneas con las secuencias (letras A, C, G, T).

Costos: igual = 0; transición ($A \leftrightarrow G$ o $C \leftrightarrow T$) = 1; transversión (cualquier otro cambio) = 2; inserción o borrado = 3.

Ejemplo

```
GATTACA
GACTATA
```

Restricciones: secuencias de hasta 10^4 bases.

Tu programa debe

1. Armar $dp[i][j]$ = costo mínimo para transformar los primeros i caracteres de la primera secuencia en los primeros j de la segunda.
2. Cada celda sale del mínimo entre sustituir (según el tipo de cambio), borrar o insertar.
3. Reconstruir el alineamiento desde el final, prefiriendo en ese orden: sustitución, borrado, inserción.
4. Mostrar el alineamiento (| igual, t transición, v transversión, - hueco) y el costo.

Salida esperada

```
GATTACA
||t||t|
GACTATA
Costo mínimo de mutación: 2
```

Reglas

- Complejidad esperada: $O(n \times m)$.
- Es la distancia de edición con costos distintos por operación: la estructura de la tabla es la misma que la de Levenshtein.
- La preferencia al reconstruir define cuál de los alineamientos óptimos se muestra.

Pista (leela solo si te trabás): solo cambia el costo de cada movimiento, no la forma de la tabla: en Levenshtein todo costaba 1; acá la sustitución depende de las dos letras.

Bonus (opcional)

- Implementá el alineamiento local (Smith-Waterman): encontrar el tramo más parecido entre las dos secuencias.
- Agregá costo de hueco afín: abrir un hueco cuesta 5 y extenderlo, 1 por base.

Cierre: Es el alineamiento de secuencias con costos ponderados, el núcleo de las herramientas de bioinformática.

Ejercicio 290: Estaciones críticas de la red de subte

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Transporte · Dificultad 58/100

Situación: La empresa de subtes quiere saber qué estaciones son críticas: si una de ellas se cierra (por obras o un incidente), la red queda partida y hay pasajeros que no pueden llegar a su destino. También quiere los tramos críticos (un tramo cuya interrupción parte la red). Con eso prioriza el mantenimiento preventivo.

Datos de entrada

Formato: un tramo por línea "estación_a estación_b" (los tramos son de doble mano).

Ejemplo

```
Plaza_de_Mayo Perú
Perú Lima
Lima Congreso
Congreso Pasco
Lima Avenida_de_Mayo
Avenida_de_Mayo Plaza_de_Mayo
Pasco Once
Once Miserere
Miserere Pasco
Once Plaza_Miserere_Norte
```

Restricciones: hasta 10^5 estaciones y 10^6 tramos.

Tu programa debe

1. Hacer un DFS registrando el orden de visita de cada estación (tin) y el menor orden alcanzable desde su subárbol usando como máximo un tramo hacia atrás (low).
2. Un tramo $u-v$ (con v hijo de u en el DFS) es crítico si $low[v] > tin[u]$.
3. Una estación u que no es raíz es crítica si tiene un hijo v con $low[v] \geq tin[u]$; la raíz es crítica si tiene más de un hijo en el DFS.
4. Mostrar las estaciones críticas y los tramos críticos, en orden alfabético.

Salida esperada

```
Estaciones críticas: Congreso, Lima, Once, Pasco
Tramos críticos:
  Congreso - Lima
  Congreso - Pasco
  Once - Plaza_Miserere_Norte
```

Reglas

- Complejidad esperada: $O(V + E)$ con un solo DFS (algoritmo de Tarjan).
- Probar sacar cada estación y ver si la red se parte sería $O(V \times (V + E))$: no pasa con 10^5 estaciones.
- Cada tramo se muestra con sus estaciones en orden alfabético.

Pista (leela solo si te trabás): $low[v]$ responde "¿puede el subárbol de v volver por encima de u sin pasar por u ?". Si no puede, sacar u (o el tramo $u-v$) lo deja aislado.

Bonus (opcional)

- Calculá, para cada estación crítica, cuántas estaciones quedarían desconectadas del resto si se cierra.
- Proponé el tramo nuevo que, agregado, elimina más puntos críticos.

Cierre: Es el algoritmo de Tarjan para puentes y puntos de articulación, usado en el diseño de redes de transporte, eléctricas y de datos.

Ejercicio 291: Campañas publicitarias con cupos

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Redes sociales · Dificultad 58/100

Situación: Una marca de indumentaria tiene \$1.000.000 para una campaña en redes. Hay varios formatos de publicidad, cada uno con su costo, su alcance estimado y un cupo máximo de piezas que la plataforma le vende. El community manager quiere la combinación que maximiza el alcance total sin pasarse del presupuesto.

Datos de entrada

Formato: primera línea: presupuesto y cantidad de formatos; después "formato costo alcance cupo" (los costos son múltiplos de \$1.000).

Ejemplo

```
1000000 4
Posteo_patrocinado 120000 15000 5
Historia 50000 5000 8
Influencer 400000 60000 2
Anuncio_en_video 250000 32000 3
```

Restricciones: hasta 100 formatos; cupos hasta 1.000; presupuesto hasta 10^8 .

Tu programa debe

1. Trabajar en miles de pesos para achicar la tabla.
2. Armar $dp[i][b]$ = mayor alcance con los primeros i formatos y presupuesto b , probando comprar de 0 a cupo piezas del formato i .
3. Guardar cuántas piezas se eligieron en cada celda (la primera cantidad que logra el máximo) y reconstruir.
4. Mostrar las piezas de cada formato, el alcance total y el gasto.

Salida esperada

```
Posteo_patrocinado: 1
Historia: 1
Influencer: 2
Anuncio_en_video: 0
Alcance total: 140000 personas | Gasto: $970000 de $1000000
```

Reglas

- Complejidad esperada: $O(\text{formatos} \times \text{presupuesto} \times \text{cupo})$. Con cupos grandes se puede bajar a $O(\text{formatos} \times \text{presupuesto} \times \log \text{cupo})$ partiendo cada cupo en potencias de 2.
- Es la mochila acotada: cada ítem se puede tomar varias veces, pero con un tope.
- El alcance se suma sin considerar superposición de audiencias (ver el bonus).

Pista (leela solo si te trabás): es la mochila 0/1 del mochilero con un bucle más adentro: en lugar de "llevar o no llevar", la decisión es "cuántas piezas llevar".

Bonus (opcional)

- Modelá rendimientos decrecientes: cada pieza adicional del mismo formato aporta 20 % menos alcance que la anterior.
- Exigí al menos una pieza de video y resolvé con la restricción extra.

Cierre: Es la mochila acotada, usada para asignar presupuestos de marketing, compras con cupos y carga de vehículos con límites por producto.

Ejercicio 292: Árbol de géneros musicales

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Música · Dificultad 59/100

Situación: Una plataforma de streaming organiza la música en un árbol de géneros y subgéneros. El equipo de datos necesita distintos recorridos del árbol: para mostrar el menú (preorden), para calcular estadísticas de abajo hacia arriba (postorden), para listar los géneros "finales" (hojas) y para mostrar la ruta de navegación hasta un género.

Datos de entrada

Formato: una línea por género "nombre padre" (el padre de la raíz es "-"); los hijos se listan en orden de aparición. Después, "?" y el género del que se quiere la ruta.

Ejemplo

```
Música -
Rock Música
Folclore Música
Electrónica Música
Rock_nacional Rock
Punk Rock
Chacarera Folclore
Zamba Folclore
Chamamé Folclore
House Electrónica
Techno Electrónica
Minimal Techno
?
Minimal
```

Restricciones: hasta 10^5 géneros.

Tu programa debe

1. Construir el árbol (hijos por padre, y el padre de cada género).
2. Hacer los recorridos preorden y postorden.
3. Listar las hojas (en preorden) y calcular la altura del árbol (cantidad de niveles).
4. Mostrar la ruta desde la raíz hasta el género consultado.

Salida esperada

```
Preorden: Música Rock Rock_nacional Punk Folclore Chacarera Zamba Chamamé Electrónica House Techno
Minimal
Postorden: Rock_nacional Punk Rock Chacarera Zamba Chamamé Folclore House Minimal Techno
Electrónica Música
Hojas (7): Rock_nacional Punk Chacarera Zamba Chamamé House Minimal
Altura: 4 niveles
Camino a Minimal: Música > Electrónica > Techno > Minimal
```

Reglas

- Complejidad esperada: $O(n)$ para cada recorrido.
- Preorden: el nodo antes que sus hijos. Postorden: los hijos antes que el nodo.
- La ruta se arma subiendo por los padres y después se invierte.

Pista (leela solo si te trabás): los dos recorridos son la misma función recursiva: lo único que cambia es si anotás el nodo antes o después de visitar a sus hijos.

Bonus (opcional)

- Implementá los recorridos sin recursión (con una pila) y verificá que dan lo mismo.
- Contá cuántas canciones hay en cada género sumando las de sus subgéneros (postorden).

Cierre: Son los recorridos de árboles, la base de menús de navegación, sistemas de archivos y taxonomías de productos.

Ejercicio 293: El proveedor más confiable

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Logística y envíos · Dificultad 59/100

Situación: Una fábrica de electrodomésticos elige por qué proveedores y depósitos enviar un pedido urgente. Cada tramo tiene una probabilidad de cumplir a tiempo, y la de toda la cadena es el producto de las probabilidades de sus tramos. Quieren la cadena con mayor probabilidad de que el pedido llegue a tiempo.

Datos de entrada

Formato: primera línea: origen y destino; después, "desde hasta probabilidad" para cada tramo (dirigido).

Ejemplo

```
Fábrica Cliente
Fábrica ProvA 0.95
Fábrica ProvB 0.90
ProvA Depósito1 0.85
ProvB Depósito1 0.99
ProvA Cliente 0.70
Depósito1 Cliente 0.97
ProvB Depósito2 0.92
Depósito2 Cliente 0.96
```

Restricciones: hasta 10^5 nodos y 10^6 tramos; probabilidades entre 0 y 1.

Tu programa debe

1. Adaptar Dijkstra para maximizar un producto: el heap saca siempre el nodo con mayor probabilidad acumulada.
2. Al relajar un tramo, la probabilidad del vecino es la del nodo por la del tramo.
3. Reconstruir la cadena desde el destino.
4. Mostrar la cadena y la probabilidad en porcentaje.

Salida esperada

```
Cadena más confiable: Fábrica -> ProvB -> Depósito1 -> Cliente
Probabilidad de entrega a tiempo: 86.43%
```

Reglas

- Complejidad esperada: $O((V + E) \log V)$.
- Funciona porque multiplicar por una probabilidad (≤ 1) nunca aumenta el valor: es el equivalente de los pesos no negativos en Dijkstra.
- Otra forma: usar pesos $-\log(p)$ y un Dijkstra común de suma mínima (y volver con la exponencial).
- Porcentaje con 2 decimales (0,5 hacia arriba), calculando con decimales exactos. Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): maximizar un producto de números entre 0 y 1 es lo mismo que minimizar la suma de sus $-\log$: así un problema que parece distinto se convierte en un camino más corto clásico.

Bonus (opcional)

- Buscá la mejor cadena con como máximo 2 intermediarios.
- Considerá también el costo: encontrá la cadena más barata con probabilidad de al menos 85 %.

Cierre: Es el camino más confiable, usado en redes de suministro, telecomunicaciones y sistemas con componentes que pueden fallar.

Ejercicio 294: El camino crítico del proyecto

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Recursos humanos · Dificultad 59/100

Situación: Una consultora planifica el desarrollo de un sistema con tareas que dependen unas de otras. El gerente quiere saber la duración mínima del proyecto, cuánta holgura tiene cada tarea (cuánto se puede atrasar sin atrasar todo) y cuáles son las tareas críticas, las que no pueden atrasarse ni un día.

Datos de entrada

Formato: una tarea por línea "nombre duración dependencias" (semanas; dependencias separadas por comas, o "-" si no tiene). Cada tarea aparece después de sus dependencias.

Ejemplo

```
Relevamiento 3 -
Diseño 5 Relevamiento
Base_de_datos 4 Diseño
Backend 8 Base_de_datos
Frontend 6 Diseño
Integración 3 Backend,Frontend
Pruebas 4 Integración
Capacitación 2 Frontend
Lanzamiento 1 Pruebas,Capacitación
```

Restricciones: hasta 10^5 tareas y 10^6 dependencias.

Tu programa debe

1. Calcular el inicio más temprano de cada tarea: el máximo de los fines más tempranos de sus dependencias (0 si no tiene).
2. La duración del proyecto es el mayor fin más temprano.
3. Recorriendo en orden inverso, calcular el fin más tardío (el mínimo de los inicios más tardíos de las tareas que dependen de ella, o la duración del proyecto) y la holgura = inicio tardío – inicio temprano.
4. Mostrar la tabla, la duración y el camino crítico (tareas con holgura 0, en el orden de la entrada).

Salida esperada

Tarea	Inicio	Fin	Holgura
Relevamiento	0	3	0
Diseño	3	8	0
Base_de_datos	8	12	0
Backend	12	20	0
Frontend	8	14	6
Integración	20	23	0
Pruebas	23	27	0
Capacitación	14	16	11
Lanzamiento	27	28	0

Duración del proyecto: 28 semanas
Camino crítico: Relevamiento -> Diseño -> Base_de_datos -> Backend -> Integración -> Pruebas -> Lanzamiento

Reglas

- Complejidad esperada: $O(V + E)$ con el orden topológico dado por la entrada.
- Formato: nombre en 15 caracteres a la izquierda; inicio en 6, fin en 4 y holgura en 7, alineados a la derecha.
- Es el método del camino crítico (CPM): programación dinámica sobre un grafo acíclico.

Pista (leela solo si te trabas): el inicio temprano sale de ir hacia adelante (el camino más largo hasta la tarea); el tardío, de ir hacia atrás desde el final. Donde coinciden, no hay margen.

Bonus (opcional)

- Si la tarea Backend se atrasa 3 semanas, ¿cuánto se atrasa el proyecto?
- Con recursos limitados (dos desarrolladores), las tareas ya no pueden hacerse todas en paralelo: ajustá la planificación.

Cierre: Es el método del camino crítico, la base de los diagramas de Gantt y de la gestión de proyectos en ingeniería y construcción.

Ejercicio 295: La frase palíndroma más larga de la melodía

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Música · Dificultad 59/100

Situación: Un compositor analiza melodías buscando "palíndromos musicales": secuencias de notas que se leen igual en los dos sentidos, formadas eligiendo notas de la melodía en orden (no necesariamente consecutivas). Quiere saber cuál es la más larga escondida en cada melodía.

Datos de entrada

Formato: una línea con las notas de la melodía separadas por espacios.

Ejemplo

```
Do Mi Sol Re Mi Fa Mi Re Sol La Mi Do
```

Restricciones: melodías de hasta 5.000 notas.

Tu programa debe

1. Armar $dp[i][j]$ = largo de la subsecuencia palíndroma más larga entre las notas i y j .
2. Si las notas de los extremos son iguales, suman 2 al resultado del interior; si no, es el máximo entre descartar un extremo u otro.
3. Reconstruir una subsecuencia desde los extremos hacia el centro (descartando el extremo izquierdo cuando $dp[i + 1][j] \geq dp[i][j - 1]$).
4. Mostrar el largo y una frase que lo alcanza.

Salida esperada

```
Largo de la frase palíndroma más larga: 11 notas  
Una frase que lo logra: Do Mi Sol Re Mi Fa Mi Re Sol Mi Do
```

Reglas

- Complejidad esperada: $O(n^2)$ en tiempo y memoria.
- Es programación dinámica sobre intervalos: la tabla se llena de intervalos cortos a largos (i de mayor a menor).
- Una sola nota es un palíndromo de largo 1.

Pista (leela solo si te trabás): la subsecuencia palíndroma más larga de una melodía es la subsecuencia común más larga entre la melodía y su reverso. Podés resolverla con el mismo algoritmo del reglamento.

Bonus (opcional)

- Encontrá el palíndromo más largo formado por notas consecutivas (subcadena, no subsecuencia).
- Contá cuántas subsecuencias palíndromas distintas tiene la melodía.

Cierre: Es la subsecuencia palíndroma más larga, un clásico de programación dinámica sobre intervalos que aparece en bioinformática y análisis de patrones.

Ejercicio 296: Camas disponibles en la clínica

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Salud · Dificultad 59/100

Situación: Una clínica de pocas camas tiene internaciones programadas y quiere simular cómo se ocupan. Cada paciente llega a una hora y se queda cierta cantidad de horas. Se le asigna la cama libre de menor número; si no hay, espera en orden de llegada. Las altas se procesan antes que las llegadas de la misma hora.

Datos de entrada

Formato: primera línea: cantidad de camas; después "paciente llegada horas_de_internación" (horas desde el inicio).

Ejemplo

```
3
Paz 0 48
Luna 5 24
Gil 10 72
Sosa 20 12
Vera 30 36
Ríos 40 24
Mota 50 10
```

Restricciones: hasta 10^4 camas y 10^6 pacientes.

Tu programa debe

1. Mantener un heap de camas libres (menor número primero) y un heap de altas (menor hora primero).
2. Procesar los eventos en orden de tiempo: en cada momento, primero las altas (que liberan camas y, si hay alguien esperando, se la dan) y después las llegadas.
3. Al llegar un paciente, asignarle la cama libre de menor número o ponerlo en la fila de espera.
4. Mostrar cada asignación (con espera y hora de alta) y cada espera.

Salida esperada

```
t= 0: Paz -> cama 1 (llegó 0, esperó 0 h, alta 48)
t= 5: Luna -> cama 2 (llegó 5, esperó 0 h, alta 29)
t= 10: Gil -> cama 3 (llegó 10, esperó 0 h, alta 82)
t= 20: Sosa espera (no hay camas)
t= 29: Sosa -> cama 2 (llegó 20, esperó 9 h, alta 41)
t= 30: Vera espera (no hay camas)
t= 40: Ríos espera (no hay camas)
t= 41: Vera -> cama 2 (llegó 30, esperó 11 h, alta 77)
t= 48: Ríos -> cama 1 (llegó 40, esperó 8 h, alta 72)
t= 50: Mota espera (no hay camas)
t= 72: Mota -> cama 1 (llegó 50, esperó 22 h, alta 82)
```

Reglas

- Complejidad esperada: $O(n \log \text{camas})$.
- Si dos altas ocurren a la misma hora, sale primero la de menor número de cama.
- Formato: tiempo en 3 caracteres alineados a la derecha.

Pista (leela solo si te trabás): dos heaps con dos trabajos distintos: uno responde "¿qué cama doy?" y el otro "¿cuándo se libera la próxima?". La simulación salta de evento en evento.

Bonus (opcional)

- Agregá prioridad: los pacientes de urgencia pasan adelante de la fila de espera.
- Calculá la ocupación promedio y el pico de pacientes esperando.

Cierre: Es la simulación por eventos con heaps, la misma que usan los sistemas de gestión de camas, estacionamientos y máquinas virtuales.

Ejercicio 297: Conversiones encadenadas de monedas

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 59/100

Situación: Una fintech de pagos internacionales no tiene cotización directa entre todos los pares de monedas: tiene algunas tasas y el resto se calcula encadenando conversiones (de euro a dólar y de dólar a peso, por ejemplo). Te piden convertir cualquier monto usando la cadena con menos pasos disponible.

Datos de entrada

Formato: líneas "A B tasa" (1 unidad de A = tasa unidades de B; la inversa se deduce); una línea "?" y consultas "desde hasta monto".

Ejemplo

```
USD ARS 1200
EUR USD 1.08
BRL USD 0.19
CLP ARS 1.25
JPY USD 0.0067
?
EUR ARS 100
BRL CLP 500
JPY EUR 10000
ARS GBP 1000
```

Restricciones: hasta 500 monedas y 10^5 tasas; hasta 10^6 consultas.

Tu programa debe

1. Armar el grafo de monedas con la tasa en cada sentido (la inversa es $1 / \text{tasa}$).
2. Para cada consulta, hacer un BFS desde la moneda de origen multiplicando las tasas del camino.
3. Reconstruir el camino usado.
4. Mostrar el monto convertido con 2 decimales y la cadena, o que no hay conversión.

Salida esperada

```
100 EUR = 129600.00 ARS (vía EUR -> USD -> ARS)
500 BRL = 91200.00 CLP (vía BRL -> USD -> ARS -> CLP)
10000 JPY = 62.04 EUR (vía JPY -> USD -> EUR)
1000 ARS -> GBP: moneda sin cotización
```

Reglas

- Complejidad esperada: $O(V + E)$ por consulta (en el bonus se precalcula).
- El BFS usa la cadena con menos conversiones, lo que en la práctica minimiza las comisiones y el error de redondeo.
- Calculá con decimales exactos y redondeá solo al mostrar (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.
- Los vecinos se exploran en el orden en que aparecen las tasas.

Pista (leela solo si te trabás): el valor de una moneda respecto de la de origen es el producto de las tasas del camino: se puede ir calculando mientras el BFS avanza.

Bonus (opcional)

- Precalculá el valor de todas las monedas respecto del dólar para responder cada consulta en $O(1)$.
- Si hay varias tasas y algunas se contradicen, detectá si hay un ciclo que permite ganar plata (arbitraje, lo vas a ver en el nivel 9).

Cierre: Es la conversión por monedas puente, usada por las plataformas de pagos internacionales cuando no hay cotización directa.

Ejercicio 298: Desafío de nivel: Centro de distribución con prioridades y rutas

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Logística y envíos · Dificultad 60/100

Situación: El desafío que integra el nivel: un centro de distribución del conurbano recibe pedidos de clientes en distintas localidades. Primero hay que saber qué localidades están conectadas por rutas con el centro (las demás se derivan a otro centro); después, la distancia más corta a cada una; y por último, despachar por urgencia (1 es la más urgente) y, a igual urgencia, por cercanía.

Datos de entrada

Formato: una línea "rutas" y los tramos "A B km" (doble mano); una línea "clientes" y cada pedido "cliente localidad urgencia". El centro está en "Centro".

Ejemplo

```
rutas
Centro Norte 12
Centro Sur 8
Norte Pilar 20
Sur Quilmes 15
Quilmes Berazategui 9
Oeste Morón 7
clientes
C1 Pilar 2
C2 Berazategui 1
C3 Morón 1
C4 Sur 3
C5 Norte 1
```

Restricciones: hasta 10^5 localidades, 10^6 rutas y 10^6 pedidos.

Tu programa debe

1. Con union-find, determinar qué localidades están en la misma red de rutas que el centro.
2. Con Dijkstra desde el centro, calcular la distancia más corta a cada localidad.
3. Derivar los pedidos de localidades fuera de la red (en el orden de la entrada).
4. Ordenar el resto con un heap por (urgencia, distancia, cliente) y mostrar el orden de despacho.

Salida esperada

```
C3 (Morón): FUERA DE LA RED, derivar a otro centro
Orden de despacho:
1. C5 -> Norte (urgencia 1, 12 km)
2. C2 -> Berazategui (urgencia 1, 32 km)
3. C1 -> Pilar (urgencia 2, 32 km)
4. C4 -> Sur (urgencia 3, 8 km)
```

Reglas

- Es un desafío integrador del nivel 6: union-find, Dijkstra y heap.
- Complejidad esperada: $O((V + E) \log V + P \log P)$ con P pedidos.
- Dijkstra ya dice qué localidades son alcanzables; el union-find se pide para practicar y porque, con rutas que cambian durante el día, se actualiza mucho más barato que recalcularlo.

Pista (leela solo si te trabás): resolvé cada parte con su estructura y conectalas por datos: el conjunto de alcanzables filtra, las distancias ordenan, el heap despacha.

Bonus (opcional)

- Agregá la capacidad de los camiones y agrupá pedidos de localidades cercanas en un mismo viaje.

- Si una ruta se corta, actualizá qué clientes quedan fuera de la red.

Cierre: Es la combinación de estructuras que usa cualquier sistema real de gestión de despacho: conectividad, caminos mínimos y colas de prioridad.

Ejercicio 299: Desafío de nivel: El planificador de la cursada

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Educación · Dificultad 60/100

Situación: Es el desafío del planificador de cursada. Un alumno de una tecnicatura quiere saber en cuántos cuatrimestres puede recibirse como mínimo cursando todo lo que tenga habilitado, cuántas horas semanales le implica cada cuatrimestre y qué cadena de correlativas es la que "alarga" la carrera.

Datos de entrada

Formato: una materia por línea "nombre horas_semanales correlativas" (separadas por comas, o "-").

Ejemplo

```
Matemática_I 6 -
Programación_I 6 -
Inglés 4 -
Matemática_II 6 Matemática_I
Programación_II 6 Programación_I
Algoritmos 8 Programación_II,Matemática_I
Bases_de_datos 6 Programación_II
Sistemas_operativos 6 Algoritmos
Redes 4 Sistemas_operativos
Proyecto_final 10 Bases_de_datos,Redes,Inglés
```

Restricciones: hasta 10^4 materias.

Tu programa debe

1. Calcular el cuatrimestre mínimo de cada materia con programación dinámica sobre el grafo: 1 + el máximo de sus correlativas (con memorización y detección de ciclos).
2. Agrupar las materias por cuatrimestre (en el orden de la entrada) y sumar sus horas.
3. Informar la cantidad mínima de cuatrimestres y la carga máxima.
4. Reconstruir la cadena de correlativas más larga (en empates, la materia que aparece primero).

Salida esperada

```
Cuatrimestre 1: Matemática_I, Programación_I, Inglés (16 h semanales)
Cuatrimestre 2: Matemática_II, Programación_II (12 h semanales)
Cuatrimestre 3: Algoritmos, Bases_de_datos (14 h semanales)
Cuatrimestre 4: Sistemas_operativos (6 h semanales)
Cuatrimestre 5: Redes (4 h semanales)
Cuatrimestre 6: Proyecto_final (10 h semanales)
Cuatrimestres mínimos: 6 | Carga máxima: 16 h semanales
Cadena que define la duración: Programación_I -> Programación_II -> Algoritmos ->
    Sistemas_operativos -> Redes -> Proyecto_final
```

Reglas

- Es un desafío integrador del nivel 6: orden topológico implícito, programación dinámica en un DAG y reconstrucción.
- Complejidad esperada: $O(V + E)$.
- Si hay un ciclo de correlativas, se informa el error.

Pista (leela solo si te trabás): el cuatrimestre de una materia es el largo del camino más largo que llega a ella en el grafo de correlativas: ese camino más largo es el que define la duración de la carrera.

Bonus (opcional)

- Limita la carga a 20 horas semanales por cuatrimestre y busca una planificación válida con la menor cantidad de cuatrimestres.
- Si el alumno ya aprobó algunas materias, recalcula desde su situación actual.

Cierre: Es el mismo análisis que hacen los sistemas de gestión académica para proyectar egresos y detectar materias "cuello de botella".

Ejercicio 300: Desafío de nivel: Presupuesto de inversión con dependencias

Nivel 6: Semi-Senior III · Estructuras clásicas · Temática: Finanzas y FinTech · Dificultad 60/100

Situación: El desafío que cierra el nivel: el comité de inversiones de una fintech tiene 100 millones para el año y una lista de proyectos con costo y retorno esperado. Algunos proyectos dependen de otro (no se puede lanzar Pagos QR sin la App móvil). Hay que elegir el conjunto de proyectos de mayor retorno total que respete las dependencias y el presupuesto.

Datos de entrada

Formato: primera línea: presupuesto; después "proyecto costo retorno depende_de" ("- " si no depende de ninguno). Cada proyecto depende como máximo de uno, así que las dependencias forman un bosque.

Ejemplo

```
100
Plataforma_base 40 60 -
App_móvil 25 45 Plataforma_base
Pagos_QR 20 50 App_móvil
Tarjeta_virtual 30 40 Plataforma_base
Inversiones 35 70 Tarjeta_virtual
Préstamos 15 30 -
Seguros 20 25 Préstamos
```

Restricciones: hasta 1.000 proyectos; presupuesto hasta 10^4 .

Tu programa debe

1. Armar el bosque de dependencias (cada proyecto con los que dependen de él).
2. Para cada proyecto, calcular una tabla: para cada inversión b , el mayor retorno de un conjunto válido de su subárbol que lo incluye a él con costo exactamente b (o no elegir nada).
3. Combinar las tablas de los hijos como una mochila (probando cómo repartir el presupuesto entre el padre y cada hijo), y después combinar las raíces.
4. Mostrar los proyectos elegidos (en el orden de la entrada), la inversión y el retorno (en empate de retorno, la menor inversión).

Salida esperada

```
Proyectos elegidos: Plataforma_base, App_móvil, Pagos_QR, Préstamos
Inversión: 100 millones de 100
Retorno total: 185 millones
```

Reglas

- Es un desafío integrador del nivel 6: mochila combinada con la estructura de dependencias (programación dinámica en árboles).
- Complejidad esperada: $O(n \times \text{presupuesto}^2)$ en esta versión simple.
- Un proyecto solo puede elegirse si también se elige el proyecto del que depende.

Pista (leela solo si te trabás): el proyecto padre funciona como "llave": si no se elige, ningún proyecto de su subárbol puede elegirse. Por eso la tabla de cada nodo solo combina hijos cuando el padre está incluido.

Bonus (opcional)

- Si un proyecto puede depender de varios (grafo acíclico en lugar de bosque), el problema se vuelve mucho más difícil: investigá el "problema de clausura de peso máximo" (nivel 9).
- Agregá una segunda restricción de personal disponible.

Cierre: Es la selección de proyectos con dependencias, un problema real de planificación de portafolios de inversión y roadmaps de producto.

Lote 13: ejercicios 301 a 325

Nivel 7

Senior I · Algoritmia avanzada

Ejercicios 301 a 350

Qué vas a practicar

Algoritmia avanzada: tries, árboles de segmentos y de Fenwick, programación dinámica avanzada, bits y teoría de números.

Qué se espera al terminar

Resolver consultas masivas en tiempo logarítmico y contar a escala.

Conceptos del nivel

trie, manipulación de bits, criba de Eratóstenes, MCD y MCM, combinatoria, aritmética modular, exponenciación rápida modular, programación dinámica sobre intervalos, programación dinámica con bitmask, programación dinámica en árboles, árbol de segmentos, búsqueda de múltiples patrones, árbol de Fenwick, actualizaciones y consultas, bits, prefijos XOR, inverso modular, conteo de inversiones, factorización, cantidad de divisores, programación dinámica modular, árbol de segmentos con propagación perezosa, XOR, números de Catalan, redes, criba segmentada, programación dinámica en árboles con re-enraizado, trie con contadores, enumeración por bits, meet in the middle, programación dinámica con bitmask por perfil, árbol de Fenwick con consultas offline, árbol de segmentos con actualización puntual, árbol de Fenwick con búsqueda binaria, programación dinámica sobre dígitos, teorema chino del resto, recorrido de Euler, programación dinámica en DAG, trie binario, estrellas y barras....

Desafíos de nivel

- 348. Libro mayor con correcciones retroactivas
- 349. El teclado predictivo
- 350. Plan de inversión en la red de riego

Ejercicio 301: Autocompletado del buscador de la tienda

Nivel 7: Senior I · Algoritmia avanzada · Temática: E-commerce · Dificultad 61/100

Situación: El buscador de una tienda online de regionales tiene que sugerir productos mientras el usuario tipea, en menos de 10 milisegundos. Con un catálogo de millones de nombres, recorrer la lista entera en cada tecla no es posible. La idea es guardar los nombres en un árbol de prefijos (trie) y sugerir los 3 más buscados.

Datos de entrada

Formato: primera línea: cantidad de productos; después "nombre popularidad"; luego la cantidad de consultas y un prefijo por línea.

Ejemplo

```
8
mate 950
mate_de_calabaza 420
matera 610
mantel 300
manta 780
mochila 510
molde 120
materas_de_cuero 610
3
ma
mat
x
```

Restricciones: hasta 10^6 productos de hasta 50 caracteres; hasta 10^6 consultas.

Tu programa debe

1. Insertar cada nombre en un trie (un nodo por carácter), marcando en el nodo final la popularidad.
2. Para cada prefijo, bajar por el trie carácter por carácter; si el camino no existe, no hay sugerencias.
3. Recolectar las palabras del subárbol del último nodo.
4. Mostrar las 3 más populares (en empate, alfabéticamente).

Salida esperada

```
"ma": mate (950), manta (780), matera (610)
"mat": mate (950), matera (610), materas_de_cuero (610)
"x": sin sugerencias
```

Reglas

- Complejidad esperada: $O(\text{largo del prefijo} + \text{tamaño del subárbol})$ por consulta. En el bonus se baja a $O(\text{largo del prefijo})$.
- La búsqueda del prefijo no depende de la cantidad total de productos, solo de su largo.
- Casos borde: prefijo que es una palabra completa; prefijo inexistente.

Pista (leela solo si te trabás): en un trie, todas las palabras que empiezan con un prefijo viven en el mismo subárbol: encontrar el prefijo es bajar tantos niveles como letras tiene.

Bonus (opcional)

- Guardá en cada nodo el top 3 de su subárbol ya calculado, para responder en $O(\text{largo del prefijo})$.
- Tolerá un error de tipeo: sugerí también palabras cuyo prefijo difiere en una letra.

Cierre: Es la estructura detrás del autocompletado de buscadores, teclados de celular y barras de direcciones.

Ejercicio 302: Permisos que caben en un número

Nivel 7: Senior I · Algoritmia avanzada · Temática: Seguridad informática · Dificultad 61/100

Situación: Un sistema de gestión de una fintech guarda los permisos de cada usuario como un solo número, donde cada bit es un permiso. Así, verificar un permiso, combinar roles o revocar un permiso son operaciones de bits instantáneas, y la base de datos guarda un entero por usuario en lugar de una tabla de permisos.

Datos de entrada

Formato: una operación por línea: "rol nombre permisos...", "usuario nombre roles..." (la unión de los roles), "otorgar u permiso", "revocar u permiso", "consultar u permiso", "comunes u1 u2" y "mostrar u".

Bits: LEER = bit 0, ESCRIBIR = bit 1, BORRAR = bit 2, EXPORTAR = bit 3, APROBAR_PAGOS = bit 4, ADMIN = bit 5.

Ejemplo

```
rol cajero LEER ESCRIBIR
rol supervisor LEER ESCRIBIR BORRAR APROBAR_PAGOS
rol auditor LEER EXPORTAR
usuario ana cajero
usuario beto supervisor auditor
otorgar ana EXPORTAR
revocar beto BORRAR
consultar ana ESCRIBIR
consultar beto BORRAR
comunes ana beto
mostrar ana
mostrar beto
```

Restricciones: hasta 64 permisos (un entero de 64 bits) y 10^7 operaciones.

Tu programa debe

1. Representar cada permiso como una máscara con un solo bit en 1 (1 << posición).
2. Combinar roles con OR, otorgar con OR, revocar con AND de la negación y consultar con AND.
3. Calcular los permisos comunes con AND.
4. Mostrar la máscara en binario (6 bits), su valor decimal y los nombres.

Salida esperada

```
¿ana puede ESCRIBIR? sí
¿beto puede BORRAR? no
Permisos comunes de ana y beto: LEER, ESCRIBIR, EXPORTAR
ana: 001011 = 11 -> LEER, ESCRIBIR, EXPORTAR
beto: 011011 = 27 -> LEER, ESCRIBIR, EXPORTAR, APROBAR_PAGOS
```

Reglas

- Complejidad esperada: $O(1)$ por operación.
- El binario se lee de derecha a izquierda: el bit menos significativo es LEER.
- Revocar un permiso que no se tenía no cambia nada.

Pista (leela solo si te trabás): con máscaras, "tiene el permiso" es (máscara AND bit) $\neq 0$ y "quitar el permiso" es máscara AND (NOT bit).

Bonus (opcional)

- Contá cuántos permisos tiene cada usuario contando los bits en 1.
- Detectá combinaciones peligrosas: nadie puede tener a la vez ESCRIBIR y APROBAR_PAGOS (segregación de funciones).

Cierre: Es la forma en que los sistemas operativos, las bases de datos y muchos backends guardan permisos y banderas de configuración.

Ejercicio 303: Primos para rifas gigantes

Nivel 7: Senior I · Algoritmia avanzada · Temática: Ciencia · Dificultad 61/100

Situación: La rifa de la cooperadora creció: ahora vende números de hasta 7 cifras, y los primos siguen cobrando el doble. Probar la primalidad de cada número por separado es lento cuando hay que verificar millones de boletos. La solución es precalcular de una sola vez qué números son primos con la criba de Eratóstenes.

Datos de entrada

Formato: primera línea: un límite chico para listar los primos; segunda: dos límites para contar primos; tercera: números de la rifa a verificar.

Ejemplo

```
100
1000000 10000000
7919 1001 999983 1000003 2
```

Restricciones: límites hasta 10^7 (la criba ocupa un byte por número).

Tu programa debe

1. Crear un arreglo de marcas del 0 al límite, todos "posiblemente primos" salvo 0 y 1.
2. Para cada i desde 2 hasta la raíz del límite, si sigue marcado, tachar sus múltiplos desde $i \times i$.
3. Listar los primos hasta el límite chico y contar los primos hasta los otros límites.
4. Informar si cada número de la rifa es primo o compuesto.

Salida esperada

```
Primos hasta 100: 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97
Cantidad de primos hasta 1000000: 78498
Cantidad de primos hasta 10000000: 664579
Números de la rifa: 7919 primo, 1001 compuesto, 999983 primo, 1000003 primo, 2 primo
```

Reglas

- Complejidad esperada: $O(N \log \log N)$ en tiempo y $O(N)$ de memoria.
- Se tacha desde $i \times i$ porque los múltiplos menores ya fueron tachados por primos más chicos.
- Con $N = 10^7$ la criba entra cómodamente en memoria si usás un byte (o un bit) por número.

Pista (leela solo si te trabás): la criba invierte la pregunta: en vez de preguntar "¿quién divide a este número?", cada primo tacha a todos los números que divide.

Bonus (opcional)

- Guardá una marca por bit en lugar de por byte y medí la memoria.
- Precalculá también el menor factor primo de cada número para factorizar cualquier número hasta N en $O(\log N)$.

Cierre: La criba de Eratóstenes tiene más de 2.000 años y sigue siendo la forma más eficiente de generar todos los primos hasta un límite.

Ejercicio 304: Colectivos que coinciden en la parada

Nivel 7: Senior I · Algoritmia avanzada · Temática: Transporte · Dificultad 62/100

Situación: En una parada muy concurrida de Buenos Aires pasan cuatro líneas de colectivo con frecuencias fijas, y todas salen juntas a las 6:00. La empresa quiere saber cada cuánto vuelven a coincidir las cuatro (se llena la parada) y cada cuánto coincide cada par, para coordinar a los inspectores.

Datos de entrada

Formato: primera línea: hora de salida común y hora de fin del servicio; después "línea frecuencia_en_minutos".

Ejemplo

```
06:00 24:00
Línea_60 12
Línea_152 18
Línea_39 20
Línea_29 15
```

Restricciones: hasta 10^5 líneas; frecuencias hasta 10^6 minutos (el MCM puede ser enorme: usá enteros grandes).

Tu programa debe

1. Calcular el MCD con el algoritmo de Euclides.
2. Calcular el MCM de todas las frecuencias acumulando: $mcm(a, b) = a / mcd(a, b) \times b$.
3. Listar las horas del día en que coinciden todas (después de la salida y hasta el fin del servicio).
4. Mostrar, para cada par de líneas, cada cuánto coinciden y su MCD.

Salida esperada

```
Las 4 líneas coinciden cada 180 minutos (3 h 00 min)
Coincidencias de todas en el día: 09:00, 12:00, 15:00, 18:00, 21:00, 24:00
Línea_60 y Línea_152: cada 36 min (MCD 6)
Línea_60 y Línea_39: cada 60 min (MCD 4)
Línea_60 y Línea_29: cada 60 min (MCD 3)
Línea_152 y Línea_39: cada 180 min (MCD 2)
Línea_152 y Línea_29: cada 90 min (MCD 3)
Línea_39 y Línea_29: cada 60 min (MCD 5)
```

Reglas

- Complejidad esperada: $O(n \log(\max \text{ frecuencia}))$ para el MCM total.
- Calculá $a / mcd \times b$ (dividir primero) para no desbordar en los pasos intermedios.
- Los pares se listan en el orden de la entrada.

Pista (leela solo si te trabás): dos colectivos que salen juntos vuelven a coincidir en el primer múltiplo común de sus frecuencias: el MCM. Y el MCM sale del MCD.

Bonus (opcional)

- Si cada línea sale a una hora distinta, la coincidencia se resuelve con el teorema chino del resto (ejercicio de los relojes).
- Encontrá el horario de salida de una quinta línea de frecuencia 25 que minimice las coincidencias con las demás.

Cierre: Es la aritmética del MCD y el MCM, que aparece en sincronización de semáforos, engranajes, calendarios y planificación periódica.

Ejercicio 305: Formas de armar el podio

Nivel 7: Senior I · Algoritmia avanzada · Temática: Deportes · Dificultad 62/100

Situación: Una app de apuestas deportivas amistosas (sin dinero) calcula cuántos podios distintos (oro, plata, bronce) son posibles en una competencia, y cuántos grupos de tres finalistas sin importar el orden. Con maratones de un millón de corredores los números son astronómicos, así que se informan módulo 1.000.000.007, como hacen los sistemas que comparan resultados exactos sin guardar números gigantes.

Datos de entrada

Formato: primera línea: cantidad de consultas; después "n k" (competidores y puestos del podio).

Ejemplo

```
4
8 3
20 3
1000 3
1000000 3
```

Restricciones: n hasta 10^6 ; $k \leq n$; hasta 10^5 consultas.

Tu programa debe

1. Precalcular los factoriales módulo 1.000.000.007 hasta el mayor n.
2. Podios (el orden importa): $n! / (n - k)!$, usando el inverso modular del denominador.
3. Grupos sin orden: además dividir por $k!$.
4. Mostrar ambos módulo 1.000.000.007 y, cuando el valor exacto es menor que el módulo, también el exacto.

Salida esperada

```
n=8: podios 336 (exacto 336) | grupos de 3 sin orden 56
n=20: podios 6840 (exacto 6840) | grupos de 3 sin orden 1140
n=1000: podios 997002000 (exacto 997002000) | grupos de 3 sin orden 166167000
n=1000000: podios 2021049 | grupos de 3 sin orden 500336845
```

Reglas

- Complejidad esperada: $O(\text{máximo } n)$ de precálculo y $O(\log \text{MOD})$ por consulta (por el inverso).
- Dividir en aritmética modular es multiplicar por el inverso: con módulo primo p, el inverso de a es $a^{(p-2) \bmod p}$ (pequeño teorema de Fermat).
- El módulo 1.000.000.007 es primo, por eso todos los factoriales menores que él tienen inverso.

Pista (leela solo si te trabás): no se puede "dividir" un resto por otro resto directamente; pero multiplicar por el inverso modular da exactamente el mismo resto que dividir los números completos.

Bonus (opcional)

- Precalcular también los inversos de los factoriales para responder cada consulta en $O(1)$.
- Contar los podios donde dos corredores del mismo club no pueden estar juntos.

Cierre: Es la combinatoria modular que usan los jueces de programación, las loterías y los sistemas criptográficos para manejar números gigantes.

Ejercicio 306: Interés compuesto a plazos astronómicos

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 62/100

Situación: Un simulador financiero calcula el crecimiento de un capital con interés diario compuesto durante plazos enormes (hasta un millón de días, para escenarios teóricos), y también cuántas carteras distintas se pueden armar con n activos (cada activo se incluye o no: 2^n). Multiplicar n veces es inviable; con exponenciación rápida alcanzan unas decenas de multiplicaciones.

Datos de entrada

Formato: primera línea: capital y tasa diaria; después, plazos en días; una línea "?" y cantidades de activos para las carteras.

Ejemplo

```
1000000 0.0001
365
3650
1000000
?
10
64
1000000000000000000
```

Restricciones: plazos hasta 10^6 días; activos hasta 10^{18} (el resultado se da módulo 1.000.000.007).

Tu programa debe

1. Implementar la exponenciación rápida: recorrer los bits del exponente, elevando la base al cuadrado en cada paso y multiplicando el resultado cuando el bit es 1.
2. Calcular $\text{capital} \times (1 + \text{tasa})^{\text{días}}$ con decimales de alta precisión.
3. Calcular 2^n módulo 1.000.000.007 con la misma función, reduciendo módulo en cada multiplicación.
4. Mostrar los resultados y la cantidad de multiplicaciones hechas.

Salida esperada

```
365 días: capital final 1.037172e+6 | multiplicaciones: 13 (en lugar de 364)
3650 días: capital final 1.440488e+6 | multiplicaciones: 15 (en lugar de 3649)
1000000 días: capital final 2.674711e+49 | multiplicaciones: 25 (en lugar de 999999)
Carteras posibles con 10 activos:  $2^{10}$  resto 1000000007 = 1024 (4 multiplicaciones)
Carteras posibles con 64 activos:  $2^{64}$  resto 1000000007 = 582344008 (6 multiplicaciones)
Carteras posibles con 1000000000000000000 activos:  $2^{1000000000000000000}$  resto 1000000007 =
719476260 (82 multiplicaciones)
```

Reglas

- Complejidad esperada: $O(\log n)$ multiplicaciones.
- El capital se muestra en notación científica con 6 decimales en la mantisa (por ejemplo, 1.037172e+6: mantisa, la letra e y el exponente sin ceros a la izquierda).
- Se cuenta como multiplicación cada producto entre dos números (los cuadrados y las multiplicaciones del resultado; la primera asignación del resultado no cuenta).
- Con exponentes de 10^{18} , sin reducir módulo en cada paso los números tendrían trillones de dígitos.

Pista (leela solo si te trabás): $x^{13} = x^8 \times x^4 \times x^1$, porque 13 en binario es 1101. Elevando al cuadrado sucesivamente obtenés x , x^2 , x^4 , x^8 ... y multiplicás los que corresponden a bits en 1.

Bonus (opcional)

- Compará la precisión: calculá $(1 + \text{tasa})^n$ con decimales comunes de 64 bits y medí el error relativo.
- Usá la misma idea con matrices para calcular el término 10^{18} de una recurrencia (nivel 9).

Cierre: La exponenciación rápida está en todas partes: criptografía, simuladores financieros y cálculos con números gigantes.

Ejercicio 307: Orden óptimo para fusionar archivos de logs

Nivel 7: Senior I · Algoritmia avanzada · Temática: Seguridad informática · Dificultad 62/100

Situación: Un equipo de seguridad tiene los logs del día repartidos en archivos consecutivos (en orden cronológico) y necesita juntarlos en uno solo para una investigación. Solo se pueden fusionar archivos vecinos (para no romper el orden temporal), y fusionar dos archivos cuesta la suma de sus tamaños. El orden en que se fusionan cambia muchísimo el costo total.

Datos de entrada

Formato: una línea con los tamaños de los archivos en MB, en orden cronológico.

Ejemplo

```
40 10 30 20 50
```

Restricciones: hasta 500 archivos.

Tu programa debe

1. Calcular sumas prefijas para obtener en $O(1)$ el tamaño total de cualquier tramo de archivos.
2. Armar $dp[i][j]$ = costo mínimo de fusionar los archivos i a j en uno: el mínimo, sobre cada punto de corte k , de $dp[i][k] + dp[k + 1][j] +$ tamaño del tramo $i..j$.
3. Llenar la tabla por largo de tramo creciente y guardar el mejor corte (el primero que da el mínimo).
4. Mostrar el costo mínimo, el orden de fusión con paréntesis y, para comparar, el costo de fusionar siempre los dos primeros.

Salida esperada

```
Costo mínimo: 340 MB leídos y escritos  
Orden de fusión: ((L1 + (L2 + L3)) + (L4 + L5))  
Fusionando de izquierda a derecha costaría: 380 MB
```

Reglas

- Complejidad esperada: $O(n^3)$ (con la optimización de Knuth se baja a $O(n^2)$).
- Solo se fusionan archivos adyacentes: si se pudieran fusionar cualquier par, el voraz de Huffman sería óptimo (nivel 10).
- Los archivos se nombran $L1, L2, \dots$ en el orden de la entrada.

Pista (leela solo si te trabás): la última fusión junta un bloque izquierdo y uno derecho; su costo es fijo (el total del tramo). Lo único que se decide es dónde cortar, y cada lado es el mismo problema más chico.

Bonus (opcional)

- Aplicá la optimización de Knuth (el mejor corte es monótono) para llegar a $O(n^2)$.
- Resolvé la versión circular: los archivos están en una lista circular (el último es vecino del primero).

Cierre: Es la programación dinámica sobre intervalos, la misma familia que la multiplicación óptima de matrices y el análisis sintáctico de lenguajes.

Ejercicio 308: Visitar 15 sucursales en un solo día

Nivel 7: Senior I · Algoritmia avanzada · Temática: Logística y envíos · Dificultad 62/100

Situación: Un camión de caudales sale del depósito, tiene que pasar por todas las sucursales de un banco una sola vez y volver. Con pocas sucursales se pueden probar todos los órdenes, pero con 15 son más de un billón. La programación dinámica sobre subconjuntos baja eso a unos pocos millones de estados.

Datos de entrada

Formato: primera línea: cantidad de sucursales k (sin contar el depósito); segunda: los nombres, empezando por el depósito; después, la matriz de distancias en km ($k + 1$ filas).

Ejemplo

```
5
Depósito Palermo Belgrano Caballito Flores Boedo
0 7 12 9 14 8
7 0 5 8 13 10
12 5 0 11 16 14
9 8 11 0 5 4
14 13 16 5 0 6
8 10 14 4 6 0
```

Restricciones: $k \leq 15$ (el estado ocupa $2^{16} \times 16$ celdas).

Tu programa debe

1. Definir $dp[S][u]$ = distancia mínima saliendo del depósito, visitando exactamente el conjunto S (representado con bits) y terminando en u .
2. Transición: desde $dp[S][u]$, ir a una sucursal v fuera de S : $dp[S \cup \{v\}][v]$ = mínimo de $dp[S][u]$ + distancia(u, v).
3. Cerrar el ciclo volviendo al depósito desde la mejor última sucursal y reconstruir el recorrido con los predecesores.
4. Mostrar el recorrido, la distancia y la cantidad de estados.

Salida esperada

```
Recorrido óptimo: Depósito -> Boedo -> Flores -> Caballito -> Belgrano -> Palermo -> Depósito
Distancia total: 42 km
Estados evaluados: 384
```

Reglas

- Complejidad esperada: $O(2^n \times n^2)$. Con $n = 16$ son unos 16 millones de operaciones; probar todas las permutaciones sería $15! \approx 1,3 \cdot 10^{12}$.
- El depósito es el bit 0 y está siempre incluido.
- En empate de distancia final, se elige la última sucursal de menor índice; en empate dentro de la tabla, se conserva el primer predecesor encontrado.

Pista (leela solo si te trabás): al elegir la próxima sucursal, lo único que importa del pasado es qué sucursales ya visitaste y dónde estás parado, no en qué orden las visitaste: por eso el estado es (conjunto, posición).

Bonus (opcional)

- Agregá ventanas horarias: cada sucursal solo recibe de 10 a 15.
- Para 100 sucursales, la DP ya no alcanza: probá una heurística (el vecino más cercano y mejoras 2-opt) y compará con el óptimo en casos chicos.

Cierre: Es el problema del viajante resuelto con el algoritmo de Held-Karp, usado en ruteo de camiones de caudales, técnicos y repartidores.

Ejercicio 309: Guardias en el árbol de la empresa

Nivel 7: Senior I · Algoritmia avanzada · Temática: Recursos humanos · Dificultad 62/100

Situación: Una empresa de software arma la guardia de fin de semana eligiendo empleados que maximicen la experiencia total, pero con una regla: un jefe y su subordinado directo no pueden estar de guardia el mismo fin de semana (si no, nadie cubre la semana siguiente). La estructura es el organigrama, un árbol.

Datos de entrada

Formato: una línea por empleado "nombre jefe experiencia" (el jefe de la raíz es "-"); cada empleado aparece después de su jefe.

Ejemplo

```
Directora - 9
Pérez Directora 6
Gómez Directora 7
Ruiz Pérez 4
Sosa Pérez 5
Vera Gómez 3
Luna Gómez 8
Paz Luna 2
Díaz Luna 6
```

Restricciones: hasta 10^6 empleados.

Tu programa debe

1. Para cada empleado, calcular dos valores: $\text{con}(e)$ = mejor total de su subárbol si él está de guardia, y $\text{sin}(e)$ = mejor total si no está.
2. $\text{con}(e) = \text{experiencia}(e) + \text{suma de sin(hijo)}$; $\text{sin}(e) = \text{suma de máximo}(\text{con(hijo)}, \text{sin(hijo)})$.
3. Procesar de abajo hacia arriba (en el orden inverso de la entrada alcanza, porque cada uno aparece después de su jefe).
4. Reconstruir desde la raíz quiénes quedan de guardia (en empate, se prefiere incluir) y mostrar el total.

Salida esperada

```
De guardia: Directora, Ruiz, Sosa, Vera, Luna
Experiencia total: 29
```

Reglas

- Complejidad esperada: $O(n)$.
- Es el conjunto independiente de peso máximo en un árbol: en un grafo general es un problema muy difícil, pero en un árbol sale con programación dinámica.
- Los elegidos se muestran en el orden de la entrada.

Pista (leela solo si te trabás): la decisión sobre un empleado solo restringe a sus hijos directos. Por eso alcanza con dos números por nodo: el mejor resultado de su subárbol con él y sin él.

Bonus (opcional)

- Exigí que haya al menos una persona de guardia en cada equipo (cada hijo de la directora).
- Calculá cuántas selecciones distintas alcanzan el máximo.

Cierre: Es la programación dinámica en árboles, usada para planificación con jerarquías, redes de sensores y optimización en estructuras ramificadas.

Ejercicio 310: Mínimo de temperatura por rango

Nivel 7: Senior I · Algoritmia avanzada · Temática: Clima y energía · Dificultad 63/100

Situación: El servicio meteorológico publica consultas del tipo "¿cuál fue la mínima entre el 3 y el 20 de julio?" sobre series de décadas, y además corrige lecturas viejas cuando detecta errores de un sensor. Con millones de consultas y correcciones, recorrer el rango en cada consulta es demasiado lento.

Datos de entrada

Formato: primera línea n ; segunda, las n temperaturas; después la cantidad de operaciones y cada una: "min a b" (mínima de los días a a b) o "act i v" (el día i pasa a valer v). Días desde 1.

Ejemplo

```
8
12 9 15 7 11 4 18 10
6
min 1 4
min 3 8
act 6 14
min 3 8
act 2 3
min 1 8
```

Restricciones: n hasta 10^6 ; hasta 10^6 operaciones.

Tu programa debe

1. Construir un árbol de segmentos donde cada nodo guarda el mínimo de su tramo.
2. Para una corrección, actualizar la hoja y recalculer los mínimos subiendo hasta la raíz.
3. Para una consulta, combinar los nodos que cubren exactamente el rango.
4. Mostrar el resultado de cada operación.

Salida esperada

```
mínima de los días 1 a 4: 7 °C
mínima de los días 3 a 8: 4 °C
corrección: día 6 = 14 °C
mínima de los días 3 a 8: 7 °C
corrección: día 2 = 3 °C
mínima de los días 1 a 8: 3 °C
```

Reglas

- Complejidad esperada: $O(n)$ de construcción y $O(\log n)$ por operación.
- Recorrer el rango ($O(n)$ por consulta) o precalcular todos los rangos ($O(n^2)$) no pasa con estos límites.
- La versión iterativa (de abajo hacia arriba) evita la recursión y es muy rápida.

Pista (leela solo si te trabás): cualquier rango se puede cubrir con $O(\log n)$ nodos del árbol; y cambiar un valor solo afecta a los $O(\log n)$ nodos que están sobre él.

Bonus (opcional)

- Guardá también el máximo y la posición del mínimo en cada nodo.
- Si no hubiera correcciones, una tabla dispersa responde en $O(1)$ (ver el ejercicio del MCD de rangos).

Cierre: El árbol de segmentos es la estructura de consultas por rango con actualizaciones que usan bases de datos de series temporales y motores de juegos.

Ejercicio 311: Palabras prohibidas en el chat

Nivel 7: Senior I · Algoritmia avanzada · Temática: Redes sociales · Dificultad 63/100

Situación: El chat de una comunidad online tiene una lista de palabras que se censuran automáticamente. Buscar cada palabra prohibida por separado en cada mensaje es lento cuando la lista tiene miles de entradas y llegan miles de mensajes por segundo. La solución es el autómata de Aho-Corasick: un trie con enlaces de fallo que encuentra todas las palabras en una sola pasada.

Datos de entrada

Formato: primera línea: cantidad de palabras prohibidas; después, una por línea (en minúsculas); luego la cantidad de mensajes y un mensaje por línea.

Ejemplo

```
3
estafa
tonto
bot
3
Che, esto parece una estafa, no seas tonto
El robot del depósito anda bien
Estafadores y tontos por todos lados
```

Restricciones: hasta 10^5 palabras prohibidas (10^6 caracteres en total) y mensajes de hasta 10^4 caracteres.

Tu programa debe

1. Construir un trie con las palabras prohibidas.
2. Calcular con un BFS el enlace de fallo de cada nodo (el sufijo propio más largo que también es un camino del trie) y heredar las palabras que terminan en ese sufijo.
3. Recorrer cada mensaje en minúsculas una sola vez siguiendo el autómata; cada vez que se llega a un nodo con palabras, marcar sus caracteres.
4. Mostrar el mensaje con los caracteres marcados reemplazados por * y las palabras encontradas en orden de aparición.

Salida esperada

```
"Che, esto parece una ***** , no seas *****" | encontradas: estafa, tonto
"El ro*** del depósito anda bien" | encontradas: bot
"*****dores y *****s por todos lados" | encontradas: estafa, tonto
```

Reglas

- Complejidad esperada: $O(\text{total de caracteres de las palabras} + \text{largo del mensaje} + \text{coincidencias})$.
- Se censuran también las palabras que aparecen dentro de otras ("bot" dentro de "robot"): es el comportamiento del autómata; el bonus lo refina.
- La búsqueda no distingue mayúsculas, pero el mensaje censurado conserva las mayúsculas originales.

Pista (leela solo si te trabás): el enlace de fallo responde "si acá se rompe la coincidencia, ¿desde dónde sigo sin volver atrás en el texto?". Es la misma idea de KMP, pero para muchas palabras a la vez.

Bonus (opcional)

- Censurá solo palabras completas (que no tengan letras pegadas antes o después).
- Detectá variantes con números en lugar de letras ("t0nt0") normalizando antes de buscar.

Cierre: Es el algoritmo de Aho-Corasick, usado en filtros de contenido, antivirus, detección de intrusos y búsqueda en genomas.

Ejercicio 312: Saldos por rango de fechas en tiempo real

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 63/100

Situación: Un banco digital permite consultar el movimiento neto de la cuenta entre dos fechas cualesquiera, y a la vez registra movimientos nuevos (incluso con fecha valor en días pasados). En el nivel 4 las sumas prefijas resolvían las consultas, pero cada movimiento nuevo obligaría a recalcularlas enteras. El árbol de Fenwick hace las dos cosas en tiempo logarítmico.

Datos de entrada

Formato: primera línea n (días); segunda, el neto inicial de cada día; después la cantidad de operaciones y cada una: "saldo a b" (neto de los días a a b) o "mov d monto" (se suma un movimiento al día d).

Ejemplo

```
10
0 15000 -3000 0 -8000 20000 0 -4500 -2000 10000
7
saldo 1 5
saldo 3 8
mov 4 -12000
saldo 1 5
mov 1 5000
saldo 1 10
saldo 6 6
```

Restricciones: n hasta 10^6 ; hasta 10^7 operaciones; montos en enteros de 64 bits.

Tu programa debe

1. Construir un árbol de Fenwick con los netos iniciales.
2. Para un movimiento, sumarlo en su día actualizando los nodos que lo cubren ($i += i \text{ AND } -i$).
3. Para una consulta, calcular $\text{prefijo}(b) - \text{prefijo}(a - 1)$, donde el prefijo recorre $i -= i \text{ AND } -i$.
4. Mostrar el resultado de cada operación con signo.

Salida esperada

```
neto días 1 a 5: +4000
neto días 3 a 8: +4500
nuevo movimiento día 4: -12000
neto días 1 a 5: -8000
nuevo movimiento día 1: +5000
neto días 1 a 10: +20500
neto días 6 a 6: +20000
```

Reglas

- Complejidad esperada: $O(\log n)$ por operación y $O(n)$ de memoria.
- $i \text{ AND } -i$ es el bit menos significativo encendido de i : define qué tramo cubre cada nodo.
- Los índices del árbol empiezan en 1.

Pista (leela solo si te trabás): cada posición del árbol guarda la suma de un tramo cuyo largo es una potencia de 2; cualquier prefijo se arma con como máximo $\log n$ de esos tramos.

Bonus (opcional)

- Soportá también "sumar un monto a todos los días de a a b" (con dos árboles de Fenwick).
- Encontrá el primer día en que el saldo acumulado supera un valor dado (búsqueda binaria sobre el árbol, ver el ejercicio de la k -ésima transacción).

Cierre: El árbol de Fenwick es la estructura liviana para sumas por rango con actualizaciones, usada en contabilidad en tiempo real y en rankings.

Ejercicio 313: Cortar la barra de chocolate en la fábrica

Nivel 7: Senior I · Algoritmia avanzada · Temática: Gastronomía · Dificultad 63/100

Situación: Una fábrica de chocolates corta barras largas en porciones en puntos fijos. La máquina de corte cobra por el largo de la pieza que tiene que sostener en cada corte (una pieza de 20 cm cuesta 20, aunque se corte cerca de la punta). El orden de los cortes cambia el costo total, y la planta quiere el orden más barato.

Datos de entrada

Formato: primera línea: largo de la barra en cm; segunda: las posiciones de corte (ordenadas, en cm desde el extremo izquierdo).

Ejemplo

```
20
4 7 12 15
```

Restricciones: barras de hasta 10^6 cm; hasta 300 cortes.

Tu programa debe

1. Agregar los extremos 0 y largo a la lista de posiciones.
2. Armar $dp[i][j]$ = costo mínimo de hacer todos los cortes que hay estrictamente entre las posiciones i y j : el mínimo sobre el primer corte k de $dp[i][k] + dp[k][j] + (pos[j] - pos[i])$.
3. Reconstruir el orden (primero el corte que parte la barra entera y después, recursivamente, cada lado).
4. Mostrar el costo mínimo, el orden y el costo de cortar de izquierda a derecha.

Salida esperada

```
Costo mínimo: 47 cm de corte
Orden de cortes: 12 (pieza de 20 cm); 7 (pieza de 12 cm); 4 (pieza de 7 cm); 15 (pieza de 8 cm)
Cortando de izquierda a derecha costaría: 57 cm
```

Reglas

- Complejidad esperada: $O(c^3)$ con c cortes ($O(c^2)$ con la optimización de Knuth).
- En empate, se elige el primer corte que da el mínimo.
- El orden se muestra en preorden: el corte, después todos los del lado izquierdo, después los del derecho.

Pista (leela solo si te trabás): el primer corte que hagas sobre un tramo siempre cuesta lo que mide el tramo; después, cada mitad es un problema independiente. Es la misma estructura que fusionar archivos de logs, al revés.

Bonus (opcional)

- Aplicá la optimización de Knuth y medí la diferencia con 300 cortes.
- Si además cada corte tiene un tiempo de preparación que depende de la posición, agregalo a la función de costo.

Cierre: Es el problema del corte óptimo de varillas por posiciones fijas, un clásico de programación dinámica sobre intervalos.

Ejercicio 314: XOR de canales en un rango

Nivel 7: Senior I · Algoritmia avanzada · Temática: Telecomunicaciones · Dificultad 63/100

Situación: Un sistema de telecomunicaciones combina señales de varios canales con la operación XOR (O exclusivo), que se usa para detectar errores y para cancelar interferencias. Los ingenieros consultan el XOR de tramos de canales consecutivos y quieren saber cuántos tramos "se cancelan" (dan 0). Con millones de canales, hay que responder en tiempo constante.

Datos de entrada

Formato: primera línea n ; segunda, los n valores (enteros de 4 bits en el ejemplo); después la cantidad de consultas y cada una "a b".

Ejemplo

```
10
5 9 12 5 3 9 14 2 7 13
4
1 4
3 7
1 10
5 5
```

Restricciones: n hasta 10^6 ; valores hasta 2^{31} ; hasta 10^6 consultas.

Tu programa debe

1. Construir el prefijo XOR: $P[0] = 0$ y $P[i] = P[i - 1] \text{ XOR } a[i]$.
2. Responder cada consulta como $P[b] \text{ XOR } P[a - 1]$ (el XOR se deshace con otro XOR, como la resta en las sumas prefijas).
3. Contar los tramos con XOR 0: son los pares $(i < j)$ con $P[i] = P[j]$, que se cuentan con un mapa de frecuencias.
4. Mostrar cada consulta (en decimal y en binario de 4 dígitos) y la cantidad de tramos que se cancelan.

Salida esperada

```
XOR de los canales 1 a 4: 5 (0101)
XOR de los canales 3 a 7: 13 (1101)
XOR de los canales 1 a 10: 9 (1001)
XOR de los canales 5 a 5: 3 (0011)
Tramos con XOR 0 (señales que se cancelan): 2
```

Reglas

- Complejidad esperada: $O(n)$ de preprocesamiento, $O(1)$ por consulta y $O(n)$ para contar los tramos.
- $x \text{ XOR } x = 0$ y $x \text{ XOR } 0 = x$: por eso el prefijo XOR funciona como las sumas prefijas.
- El binario se muestra con al menos 4 dígitos.

Pista (leela solo si te trabás): el XOR es su propia inversa: $P[b] \text{ XOR } P[a - 1]$ cancela todo lo que hay antes de a y deja solo el tramo.

Bonus (opcional)

- Contá los tramos cuyo XOR es exactamente un valor k (mismo mapa, buscando $P[j] \text{ XOR } k$).
- Encontrá el tramo con el XOR máximo (vas a necesitar un trie binario, ejercicio 339).

Cierre: El XOR es la base de la paridad, los códigos de detección de errores y el almacenamiento redundante en discos (RAID).

Ejercicio 315: Repartir premios módulo un primo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Videojuegos · Dificultad 64/100

Situación: Un servidor de juegos online calcula probabilidades exactas de los premios (como fracciones) y las comparte con los clientes en formato modular para no perder precisión: la fracción p/q se representa como $p \times q^{-1}$ módulo un primo grande. Los desarrolladores necesitan calcular esos inversos y entender cuándo no existen.

Datos de entrada

Formato: primera línea: un módulo primo; después la cantidad de fracciones y cada una "numerador denominador"; una línea "?" y pares "número módulo" con un módulo que no es primo.

Ejemplo

```
1000000007
3
1 6
7 20
999999 1000000
?
12 30
7 30
```

Restricciones: módulos hasta 10^{18} ; hasta 10^6 fracciones.

Tu programa debe

1. Con módulo primo p , calcular el inverso de q como $q^{(p-2)}$ resto p (pequeño teorema de Fermat) con exponenciación rápida.
2. Representar cada fracción como numerador $\times$ inverso(denominador) resto p y verificar que $r \times q$ resto p da el numerador.
3. Con un módulo que no es primo, usar el algoritmo de Euclides extendido: el inverso existe solo si $\text{mcd}(\text{número}, \text{módulo}) = 1$.
4. Mostrar los resultados y las verificaciones.

Salida esperada

```
Módulo primo 1000000007:
probabilidad 1/6 -> 166666668 (verificación: 166666668 × 6 resto p = 1)
probabilidad 7/20 -> 950000007 (verificación: 950000007 × 20 resto p = 7)
probabilidad 999999/1000000 -> 857143007 (verificación: 857143007 × 1000000 resto p = 999999)
Módulo compuesto 30 (inverso con Euclides extendido):
inverso de 12 módulo 30: no existe (mcd = 6)
inverso de 7 módulo 30: 13 (7 × 13 = 91 ≡ 1)
```

Reglas

- Complejidad esperada: $O(\log p)$ por inverso.
- Fermat solo sirve con módulo primo; Euclides extendido sirve siempre que el inverso exista.
- El resultado del Euclides extendido puede ser negativo: se lleva al rango $[0, m)$ sumando m .

Pista (leela solo si te trabás): Euclides extendido encuentra enteros x, y tales que $a \cdot x + m \cdot y = \text{mcd}(a, m)$. Si el mcd es 1, tomando resto m queda $a \cdot x \equiv 1$: x es el inverso.

Bonus (opcional)

- Calculá los inversos de todos los números de 1 a n en $O(n)$ con la recurrencia $\text{inv}[i] = -(p / i) \times \text{inv}[p \text{ resto } i] \text{ resto } p$.
- Sumá dos probabilidades en formato modular y verificá que coincide con sumar las fracciones y después convertir.

Cierre: El inverso modular es una pieza fundamental de la criptografía de clave pública y de los cálculos exactos en sistemas distribuidos.

Ejercicio 316: Cada empleado con su tarea ideal

Nivel 7: Senior I · Algoritmia avanzada · Temática: Recursos humanos · Dificultad 64/100

Situación: Una consultora tiene que asignar cuatro tareas de un proyecto a cuatro empleados, una por persona. Cada empleado tarda distinto en cada tarea según su experiencia. Quieren minimizar la suma de días. Con 4 personas se pueden probar las 24 combinaciones, pero la herramienta tiene que servir para equipos de hasta 20 (20! combinaciones).

Datos de entrada

Formato: primera línea n ; segunda, los nombres de las n tareas; después, una fila por empleado con su nombre y los días que tarda en cada tarea.

Ejemplo

```
4
Análisis Diseño Desarrollo Pruebas
Ana 9 2 7 8
Beto 6 4 3 7
Caro 5 8 1 8
Dani 7 6 9 4
```

Restricciones: n hasta 20 ($2^{20} \approx 10^6$ estados).

Tu programa debe

1. Definir $dp[S]$ = costo mínimo de asignar las tareas del conjunto S (bits) a los primeros $|S|$ empleados.
2. Transición: el próximo empleado (el número $|S|$) toma una tarea j que no está en S .
3. Guardar qué tarea se agregó en cada estado para reconstruir.
4. Mostrar la asignación y el total.

Salida esperada

```
Ana -> Diseño (2 días)
Beto -> Análisis (6 días)
Caro -> Desarrollo (1 día)
Dani -> Pruebas (4 días)
Total: 13 días-persona
```

Reglas

- Complejidad esperada: $O(2^n \times n)$.
- La cantidad de bits en 1 de S dice cuántos empleados ya fueron asignados: por eso no hace falta guardarlo en el estado.
- En empate, se conserva la primera transición que alcanzó el mínimo.

Pista (leela solo si te trabás): el orden de los empleados es fijo (el primero, después el segundo...): solo importa qué tareas ya se usaron. Eso reduce $n!$ órdenes a 2^n conjuntos.

Bonus (opcional)

- Resuélvelo con el algoritmo húngaro en $O(n^3)$, que sirve para $n = 1.000$.
- Permití que un empleado tome dos tareas si hay más tareas que personas.

Cierre: Es el problema de asignación, resuelto con programación dinámica sobre máscaras de bits; aparece en planificación de turnos, logística y matching.

Ejercicio 317: Inversiones en el ranking

Nivel 7: Senior I · Algoritmia avanzada · Temática: Deportes · Dificultad 64/100

Situación: Un programa deportivo premia al panelista que mejor pronosticó la tabla final del torneo. Para medir qué tan bien le fue, cuentan los pares de equipos que el panelista ordenó al revés ("inversiones"). Con rankings de miles de participantes (por ejemplo, un prode masivo), contar par por par es demasiado lento.

Datos de entrada

Formato: primera línea n; segunda, el pronóstico (del 1.º al último); tercera, la tabla real.

Ejemplo

```
8
Pumas Halcones Toros Leones Cóndores Zorros Lobos Águilas
Halcones Pumas Leones Toros Zorros Cóndores Águilas Lobos
```

Restricciones: n hasta 10^6 .

Tu programa debe

1. Reemplazar cada equipo del pronóstico por su posición real, obteniendo una secuencia de números.
2. Recorrer la secuencia con un árbol de Fenwick: para cada valor, contar cuántos valores mayores ya aparecieron antes (esas son inversiones) y registrarlo.
3. Calcular la coincidencia de Kendall: $(\text{pares totales} - 2 \times \text{inversiones}) / \text{pares totales}$.
4. Mostrar la secuencia, las inversiones y la coincidencia.

Salida esperada

```
Posiciones reales en el orden del pronóstico: 2 1 4 3 6 5 8 7
Pares invertidos: 4 de 28
Coincidencia (tau de Kendall): 0.714
```

Reglas

- Complejidad esperada: $O(n \log n)$. Contar todos los pares es $O(n^2)$.
- Mayores ya vistos = cantidad de vistos – cuántos vistos son menores o iguales (una consulta de prefijo).
- La coincidencia va de -1 (orden invertido) a 1 (pronóstico perfecto); con 3 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): una inversión es un par que está "desordenado". El árbol de Fenwick lleva la cuenta de cuántos valores de cada tamaño ya pasaron, así que contar los mayores es una consulta de prefijo.

Bonus (opcional)

- Resóvelo también con merge sort contando inversiones al mezclar, y compará.
- Pesá más los errores en los primeros puestos que en los últimos.

Cierre: Contar inversiones es la base de la distancia de Kendall, usada para comparar rankings en buscadores, sistemas de recomendación y encuestas.

Ejercicio 318: Divisores del número de lote

Nivel 7: Senior I · Algoritmia avanzada · Temática: Agricultura · Dificultad 64/100

Situación: Un sistema de trazabilidad agrícola usa números de lote enormes y, para repartir la mercadería en cajas iguales, necesita saber de cuántas formas se puede dividir un lote: la cantidad de divisores del número. Factorizar por tanteo hasta la raíz alcanza para números de hasta 10^{12} si se hace bien.

Datos de entrada

Formato: una línea con los números de lote.

Ejemplo

```
360 97 1000000007 600851475143 123456789012 1
```

Restricciones: números hasta 10^{12} (la raíz es 10^6); hasta 10^3 números.

Tu programa debe

- Factorizar cada número probando divisores hasta su raíz cuadrada (el 2 y después solo impares).
- Si al terminar queda un número mayor que 1, es un factor primo.
- Con la factorización $p_1^{e_1} \times p_2^{e_2} \times \dots$, la cantidad de divisores es $(e_1 + 1)(e_2 + 1)\dots$ y la suma es el producto de $(p^{e+1} - 1) / (p - 1)$.
- Mostrar la factorización, la cantidad y la suma de divisores.

Salida esperada

```
360 = 2^3 x 3^2 x 5 | divisores: 24 | suma de divisores: 1170
97 = 97 | divisores: 2 | suma de divisores: 98
1000000007 = 1000000007 | divisores: 2 | suma de divisores: 1000000008
600851475143 = 71 x 839 x 1471 x 6857 | divisores: 16 | suma de divisores: 610544148480
123456789012 = 2^2 x 3 x 10288065751 | divisores: 12 | suma de divisores: 288065841056
1: no tiene factores primos | divisores: 1 | suma de divisores: 1
```

Reglas

- Complejidad esperada: $O(\sqrt{n})$ por número.
- No hace falta listar los divisores para contarlos: sale de los exponentes.
- La suma de divisores puede superar 64 bits para números grandes: usá enteros de precisión arbitraria.
- El 1 no tiene factores primos y tiene un solo divisor.

Pista (leela solo si te trabás): cada divisor se arma eligiendo cuántas veces aparece cada primo (de 0 a su exponente): por eso la cantidad de divisores es un producto de (exponente + 1).

Bonus (opcional)

- Para números de hasta 10^{18} , investigá el test de Miller-Rabin y el algoritmo rho de Pollard.
- Encontrá el número menor a 10^6 con más divisores.

Cierre: La factorización es la base de la teoría de números y, por lo difícil que es con números enormes, de la seguridad de sistemas como RSA.

Ejercicio 319: Caminos en la grilla con obstáculos

Nivel 7: Senior I · Algoritmia avanzada · Temática: Logística y envíos · Dificultad 64/100

Situación: En un depósito automatizado, los robots se mueven solo hacia la derecha o hacia abajo por una grilla, y algunas celdas están ocupadas por estanterías. Para balancear el tráfico, el sistema necesita saber cuántos caminos distintos hay entre la entrada y la salida. En depósitos gigantes el número es astronómico y se informa módulo 1.000.000.007.

Datos de entrada

Formato: primera línea: filas y columnas; después la grilla (# = obstáculo); una línea "?" y grillas libres "filas columnas".

Ejemplo

```
5 6
.....
.#....
...#..
#.....
...#.
?
1000 1000
100000 100000
```

Restricciones: grillas con obstáculos de hasta 2.000×2.000 ; grillas libres de hasta $10^6 \times 10^6$.

Tu programa debe

1. Para la grilla con obstáculos: $dp[r][c] = dp[r - 1][c] + dp[r][c - 1]$ (módulo), con 0 en los obstáculos y 1 en el inicio.
2. Para una grilla libre de $F \times C$, la cantidad de caminos es el combinatorio $C(F + C - 2, F - 1)$: elegir en qué pasos se baja.
3. Calcular ese combinatorio módulo el primo con el inverso modular del denominador.
4. Mostrar los tres resultados.

Salida esperada

```
Caminos en el depósito con obstáculos: 13
Grilla libre de 1000 × 1000: C(1998, 999) resto 1000000007 = 965601742
Grilla libre de 100000 × 100000: C(199998, 99999) resto 1000000007 = 691090292
```

Reglas

- Complejidad esperada: $O(F \times C)$ con obstáculos y $O(\min(F, C) + \log \text{MOD})$ sin obstáculos.
- Reducí módulo en cada suma y cada producto para no desbordar.
- Si la entrada o la salida tienen un obstáculo, hay 0 caminos.

Pista (leela solo si te trabás): sin obstáculos, todo camino tiene exactamente $F - 1$ bajadas y $C - 1$ pasos a la derecha; lo único que cambia es el orden. Contar órdenes es un combinatorio.

Bonus (opcional)

- Con pocos obstáculos en una grilla gigante, combiná combinatoria e inclusión-exclusión en lugar de la tabla.
- Contá los caminos que pasan por una celda dada (útil para detectar cuellos de botella).

Cierre: Es el conteo de caminos en retículas, que combina programación dinámica y combinatoria, y aparece en robótica, probabilidad y física.

Ejercicio 320: Ajustes de stock por rango

Nivel 7: Senior I · Algoritmia avanzada · Temática: E-commerce · Dificultad 64/100

Situación: Un e-commerce ordena su catálogo por código y hace ajustes de stock masivos por rango ("a todos los productos del 2 al 5 sumales 10 unidades porque llegó un contenedor"). También consulta el stock total de rangos de productos. Actualizar producto por producto un rango de un millón es demasiado lento; la propagación perezosa lo resuelve.

Datos de entrada

Formato: primera línea n ; segunda, el stock inicial de cada producto; después la cantidad de operaciones: "sumar a b v" (suma v a cada producto de a a b) o "total a b".

Ejemplo

```
8
50 30 20 40 10 60 25 35
6
total 1 8
sumar 2 5 10
total 1 4
sumar 4 8 -5
total 3 6
total 1 8
```

Restricciones: n hasta 10^6 ; hasta 10^6 operaciones.

Tu programa debe

1. Construir un árbol de segmentos con la suma de cada tramo.
2. Para sumar v a un rango, cuando un nodo queda totalmente cubierto, actualizar su suma ($v \times$ tamaño del tramo) y anotar el ajuste pendiente en el nodo, sin bajar a los hijos.
3. Cuando una operación necesita entrar a los hijos de un nodo con ajuste pendiente, empujarlo a los hijos antes (propagación perezosa).
4. Mostrar el resultado de cada operación.

Salida esperada

```
stock total de productos 1 a 8: 270
ajuste +10 a los productos 2 a 5
stock total de productos 1 a 4: 170
ajuste -5 a los productos 4 a 8
stock total de productos 3 a 6: 145
stock total de productos 1 a 8: 285
```

Reglas

- Complejidad esperada: $O(\log n)$ por operación, tanto para ajustes como para consultas.
- Sin propagación perezosa, un ajuste de rango costaría $O(n \log n)$.
- Las sumas pueden superar 2^{31} : usá enteros de 64 bits.

Pista (leela solo si te trabás): la idea es "no hagas hoy lo que puedas hacer mañana": un ajuste que cubre todo un nodo se anota ahí y solo se reparte a los hijos si alguien los necesita.

Bonus (opcional)

- Agregá la operación "asignar un valor fijo a un rango" (combinar dos tipos de ajuste pendiente requiere cuidado con el orden).
- Agregá la consulta del mínimo del rango además de la suma.

Cierre: La propagación perezosa es la técnica que usan los sistemas de inventario, los editores de planillas y los juegos para aplicar cambios masivos por rango.

Ejercicio 321: La transacción huérfana

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 65/100

Situación: En un sistema de pagos, cada transacción genera dos registros: el débito y el crédito, con el mismo id. En la conciliación nocturna aparece un registro sin su pareja (una transacción "huérfana"). Hay cientos de millones de registros y la memoria es limitada: no se puede guardar un conjunto con todos los ids.

Datos de entrada

Formato: una línea con los ids de los registros (todos aparecen dos veces menos uno); una línea "?" y otra lista donde faltan las parejas de dos ids.

Ejemplo

```
T88 T12 T45 T12 T77 T45 T88
?
T10 T20 T30 T10 T40 T30
```

Restricciones: hasta 10^9 registros; ids hasta 2^{63} .

Tu programa debe

1. Aplicar XOR a todos los ids: como $x \text{ XOR } x = 0$, las parejas se cancelan y queda el huérfano.
2. Mostrar el resultado en decimal y en binario (8 dígitos).
3. Para dos huérfanos: el XOR total es $a \text{ XOR } b \neq 0$; tomar un bit encendido (el menos significativo) y repartir los ids en dos grupos según ese bit.
4. El XOR de cada grupo da uno de los dos huérfanos (se muestran de menor a mayor).

Salida esperada

```
XOR de todos los ids: 77 (01001101) -> transacción huérfana: T77
Con dos huérfanos: XOR total 60 (00111100), bit separador 4 -> T20 y T40
```

Reglas

- Complejidad esperada: $O(n)$ en tiempo y $O(1)$ de memoria extra.
- Un conjunto o un mapa resolverían el problema, pero con $O(n)$ de memoria: con 10^9 registros no entra.
- $x \text{ AND } (-x)$ aísla el bit menos significativo encendido.

Pista (leela solo si te trabás): a y b difieren en al menos un bit (el que está encendido en a XOR b). Separar por ese bit pone a cada huérfano en un grupo distinto, y dentro de cada grupo las parejas se siguen cancelando.

Bonus (opcional)

- Si todos aparecen tres veces menos uno, el XOR no alcanza: sumá cada bit módulo 3.
- Adaptá el método para detectar un registro duplicado de más en lugar de uno faltante.

Cierre: El truco del XOR es un clásico de entrevistas y se usa en verificación de integridad y en algoritmos de memoria constante.

Ejercicio 322: Formas de agrupar una expresión

Nivel 7: Senior I · Algoritmia avanzada · Temática: Educación · Dificultad 65/100

Situación: En un taller de matemática recreativa, los chicos reciben una expresión sin paréntesis y compiten por conseguir el resultado más alto poniendo paréntesis donde quieran. La profe quiere saber de antemano cuántas formas hay de agruparla y cuáles son todos los resultados posibles.

Datos de entrada

Formato: una línea con la expresión: enteros y operadores +, − y * separados por espacios.

Ejemplo

```
2 - 1 - 1 + 3 * 2
```

Restricciones: hasta 15 números (la cantidad de agrupamientos crece como los números de Catalan).

Tu programa debe

1. Guardar, para cada tramo de números i..j, el conjunto de todos los resultados posibles.
2. Un tramo se parte en el operador k en dos subtramos; se combinan todos los resultados del lado izquierdo con todos los del derecho.
3. Calcular la cantidad de agrupamientos con la fórmula de los números de Catalan.
4. Mostrar la cantidad de agrupamientos, los resultados distintos (ordenados), el mínimo y el máximo.

Salida esperada

```
Formas de poner paréntesis: 14 (número de Catalan C4)
Resultados distintos posibles: 8
Valores: -7 -6 -4 -2 6 8 9 10
Mínimo: -7 | Máximo: 10
```

Reglas

- Complejidad: los conjuntos pueden crecer mucho; con 15 números sigue siendo manejable. Si solo hace falta el mínimo y el máximo, alcanza con guardar esos dos valores por tramo (bonus).
- Catalan: $C_0 = 1$ y $C(n) = C(n-1) \times 2(2n-1) / (n+1)$.
- Dos agrupamientos distintos pueden dar el mismo resultado: por eso los "resultados distintos" son menos que los agrupamientos.

Pista (leela solo si te trabás): cualquier agrupamiento tiene una operación "última" (la de afuera de todo). Eligiendo cuál es, la expresión se parte en dos problemas iguales más chicos: programación dinámica sobre intervalos.

Bonus (opcional)

- Calculá solo el máximo y el mínimo en $O(n^3)$ guardando esos dos valores por tramo (atención: con restas y productos, el máximo puede venir de combinar mínimos).
- Mostrá la expresión con paréntesis que logra el máximo.

Cierre: Es la programación dinámica sobre intervalos que usan los compiladores para analizar expresiones y los optimizadores de consultas de bases de datos.

Ejercicio 323: Máscaras de subred de los dispositivos

Nivel 7: Senior I · Algoritmia avanzada · Temática: Telecomunicaciones · Dificultad 65/100

Situación: El equipo de redes de una operadora administra miles de subredes de clientes y dispositivos. Cuando llega tráfico de una IP, el sistema tiene que decidir en microsegundos a qué subred pertenece. Todo se reduce a operaciones de bits sobre números de 32 bits.

Datos de entrada

Formato: una línea por consulta "red/prefijo ip" (IPv4).

Ejemplo

```
192.168.10.0/24 192.168.10.77
10.20.0.0/14 10.23.255.1
172.16.5.130/26 172.16.5.200
200.45.7.1/32 200.45.7.1
```

Restricciones: hasta 10^6 subredes y 10^7 consultas por segundo.

Tu programa debe

1. Convertir cada IP a un entero de 32 bits (cada octeto son 8 bits).
2. Armar la máscara: prefijo bits en 1 seguidos de ceros.
3. Calcular la dirección de red (IP AND máscara), el broadcast (red OR NOT máscara) y la cantidad de hosts utilizables ($2^{(32 - \text{prefijo})} - 2$).
4. Decidir si la IP consultada pertenece a la red: (IP AND máscara) = red.

Salida esperada

```
192.168.10.0/24: máscara 255.255.255.0 | red 192.168.10.0 | broadcast 192.168.10.255 | hosts 254 |
192.168.10.77 pertenece
10.20.0.0/14: máscara 255.252.0.0 | red 10.20.0.0 | broadcast 10.23.255.255 | hosts 262142 |
10.23.255.1 pertenece
172.16.5.130/26: máscara 255.255.255.192 | red 172.16.5.128 | broadcast 172.16.5.191 | hosts 62 |
172.16.5.200 NO pertenece
200.45.7.1/32: máscara 255.255.255.255 | red 200.45.7.1 | broadcast 200.45.7.1 | hosts 1 |
200.45.7.1 pertenece
```

Reglas

- Complejidad esperada: $O(1)$ por consulta.
- Con prefijo /32 hay 1 host (una sola dirección); con /31, 2 (enlaces punto a punto): son excepciones a la fórmula.
- Cuidado con los desplazamientos: en muchos entornos, desplazar 32 posiciones un entero de 32 bits no da 0.

Pista (leela solo si te trabás): la máscara parte la IP en dos: la parte de red (bits en 1) y la de host (bits en 0). AND se queda con la red; OR con la negación llena la parte de host con unos.

Bonus (opcional)

- Dada una IP y muchas redes superpuestas, encontrará la de prefijo más largo que la contiene (así rutean los routers) con un trie binario.
- Dividí una red /24 en 4 subredes iguales y mostrá sus rangos.

Cierre: Es la aritmética de bits que ejecutan los routers y firewalls de internet miles de millones de veces por segundo.

Ejercicio 324: Números de serie primos entre dos billones

Nivel 7: Senior I · Algoritmia avanzada · Temática: Ciencia · Dificultad 65/100

Situación: Un fabricante de electrónica asigna números de serie primos a los chips de una línea de seguridad, y necesita todos los primos de un rango cerca de los dos billones ($2 \cdot 10^{12}$). Una criba común hasta $2 \cdot 10^{12}$ no entra en ninguna memoria, pero solo interesa un tramo angosto: la criba segmentada resuelve justo eso.

Datos de entrada

Formato: una línea "desde hasta".

Ejemplo

```
2000000000000 20000000000500
```

Restricciones: hasta $\leq 10^{14}$; hasta – desde $\leq 10^7$.

Tu programa debe

1. Generar con una criba común los primos hasta la raíz de "hasta" (unos 1,4 millones).
2. Crear un arreglo de marcas solo para el tramo [desde, hasta].
3. Para cada primo base p , tachar sus múltiplos dentro del tramo, empezando en el primer múltiplo $\geq$ desde ($y \geq p^2$).
4. Mostrar la cantidad de primos, los 3 primeros, los 3 últimos y la mayor distancia entre primos consecutivos del tramo.

Salida esperada

```
Primos entre 2000000000000 y 20000000000500: 20
Primeros: 20000000000003 20000000000123 20000000000137
Últimos: 20000000000483 20000000000489 20000000000491
Mayor distancia entre primos consecutivos del rango: 120
```

Reglas

- Complejidad esperada: $O(\sqrt{R} \log \log R + (R - L) \log \log R)$ en tiempo y $O(\sqrt{R} + (R - L))$ de memoria.
- El primer múltiplo de $p \geq L$ es $((L + p - 1) / p) \times p$ con división entera.
- Todo compuesto del tramo tiene un factor primo $\leq \sqrt{R}$: por eso alcanza con los primos base.

Pista (leela solo si te trabás): los compuestos del tramo los "delata" algún primo chico. Por eso, con los primos hasta la raíz alcanza para cribar cualquier ventana, por más lejos que esté del 0.

Bonus (opcional)

- Contá los primos gemelos (que difieren en 2) del tramo.
- Procesá un tramo de 10^9 números por bloques de 10^6 para no usar tanta memoria.

Cierre: La criba segmentada es la forma en que se generan primos en rangos enormes, algo necesario en criptografía y en investigación matemática.

Ejercicio 325: Distancias en el árbol de la red eléctrica

Nivel 7: Senior I · Algoritmia avanzada · Temática: Clima y energía · Dificultad 65/100

Situación: Una cooperativa eléctrica tiene una red en forma de árbol que une barrios con tramos de distinto largo. Quiere instalar un centro de mantenimiento en el barrio que minimice la suma de las distancias a todos los demás. Calcular esa suma desde cada barrio por separado es cuadrático; con la técnica de re-enraizado, se calculan todas en tiempo lineal.

Datos de entrada

Formato: primera línea: cantidad de nodos n ; después $n - 1$ tramos "a b km" (forman un árbol).

Ejemplo

```
7
Subestación_A Barrio_1 4
Subestación_A Barrio_2 2
Barrio_2 Barrio_3 3
Barrio_2 Barrio_4 5
Barrio_4 Barrio_5 1
Barrio_4 Barrio_6 2
```

Restricciones: n hasta 10^6 .

Tu programa debe

1. Elegir una raíz y calcular, de abajo hacia arriba, el tamaño de cada subárbol y la suma de distancias de cada nodo a su subárbol.
2. La suma total desde la raíz es la de su subárbol completo.
3. Re-enraizar de arriba hacia abajo: al mover la raíz de un padre p a un hijo h por un tramo de largo w , la suma cambia en $w \times (n - 2 \times \text{tamaño}(h))$.
4. Mostrar la suma de cada nodo y el mejor (en empate, el primero en aparecer).

Salida esperada

```
Subestación_A: 35 km
Barrio_1: 55 km
Barrio_2: 29 km
Barrio_3: 44 km
Barrio_4: 34 km
Barrio_5: 39 km
Barrio_6: 44 km
Mejor ubicación para la nueva subestación: Barrio_2 (29 km de cable total a todos)
```

Reglas

- Complejidad esperada: $O(n)$ (dos recorridos).
- Calcular un recorrido desde cada nodo sería $O(n^2)$: con 10^6 nodos no pasa.
- Por qué funciona la fórmula: al pasar de p a h , los tamaño(h) nodos del subárbol de h quedan w más cerca y los $n - \text{tamaño}(h)$ restantes, w más lejos.

Pista (leela solo si te trabás): no recalculas desde cero: la suma desde un hijo se deduce de la suma desde su padre con una sola cuenta. Ese es el truco del re-enraizado.

Bonus (opcional)

- Minimizá la distancia máxima (en lugar de la suma) a cualquier barrio: es el centro del árbol, ligado al diámetro.
- Pesá cada barrio por su consumo: la suma pasa a ser ponderada y la fórmula usa el peso del subárbol en lugar del tamaño.

Cierre: Es la programación dinámica con re-enraizado, usada en ubicación de servicios, análisis de redes y problemas de mediana en árboles.

Lote 14: ejercicios 326 a 350

Ejercicio 326: Prefijos telefónicos más comunes

Nivel 7: Senior I · Algoritmia avanzada · Temática: Telecomunicaciones · Dificultad 66/100

Situación: Una operadora de telefonía analiza los números que llaman a su centro de atención para decidir dónde abrir oficinas regionales. Quiere saber cuántos números empiezan con cada prefijo (característica de área) y cuál es el prefijo más largo que comparten muchos clientes. Son decenas de millones de números y se consulta todo el tiempo.

Datos de entrada

Formato: primera línea: cantidad de números; después, un número por línea; luego la cantidad de prefijos a consultar y uno por línea; al final, el mínimo de números K para el prefijo compartido.

Ejemplo

```
10
1145678901
1145671234
1143210000
3514567890
3514001122
3415123456
2214556677
1145000111
3514998877
1149876543
3
114
3514
2615
2
```

Restricciones: hasta 10^7 números de hasta 15 dígitos; hasta 10^6 consultas.

Tu programa debe

1. Insertar cada número en un trie de dígitos, sumando 1 al contador de cada nodo por el que pasa.
2. Para cada prefijo consultado, bajar por el trie y responder el contador del nodo final (0 si el camino no existe).
3. Recorrer el trie buscando el nodo más profundo cuyo contador sea al menos K (en empate de largo, el prefijo menor alfabéticamente).
4. Mostrar los conteos y el prefijo más largo compartido.

Salida esperada

```
Números que empiezan con 114: 5
Números que empiezan con 3514: 3
Números que empiezan con 2615: 0
Prefijo más largo compartido por al menos 2 números: 114567 (2 números)
```

Reglas

- Complejidad esperada: $O(\text{total de dígitos})$ para construir y $O(\text{largo del prefijo})$ por consulta.
- Con solo 10 hijos posibles por nodo, cada nodo puede ser un arreglo de 10 posiciones en lugar de un mapa.
- Casos borde: prefijo inexistente (0); K mayor que la cantidad de números (ningún prefijo cumple).

Pista (leela solo si te trabás): el contador de un nodo es exactamente la cantidad de números que pasan por ese prefijo: responder es solo llegar al nodo.

Bonus (opcional)

- Soportá bajas de números (restar en el camino) para mantener las estadísticas en vivo.

- Encontrá los 3 prefijos de 4 dígitos con más números.

Cierre: Los tries con contadores se usan en ruteo telefónico, en tablas de enrutamiento de internet y en análisis de prefijos de todo tipo.

Ejercicio 327: Subconjuntos de cartas que suman el objetivo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Videojuegos · Dificultad 66/100

Situación: En un juego de cartas online, el jugador tiene una mano de cartas con distintos puntajes y gana un bonus si elige un subconjunto que sume exactamente el objetivo de la ronda. El servidor tiene que calcular cuántas combinaciones ganadoras hay para ajustar el premio. Con 40 cartas hay un billón de subconjuntos: probarlos todos no es opción, pero partir la mano en dos sí.

Datos de entrada

Formato: primera línea: cantidad de cartas y objetivo; segunda: los puntajes.

Ejemplo

```
14 100
3 34 4 12 5 2 27 19 8 15 23 11 7 31
```

Restricciones: hasta 40 cartas; puntajes hasta 10^9 (el objetivo no es chico, así que la mochila por valor no sirve).

Tu programa debe

1. Partir las cartas en dos mitades y enumerar todas las sumas de subconjuntos de cada mitad (2^{20} cada una, con máscaras de bits).
2. Ordenar las sumas de la segunda mitad.
3. Para cada suma s de la primera mitad, contar con búsqueda binaria cuántas sumas de la segunda valen exactamente $\text{objetivo} - s$.
4. Con la misma búsqueda, encontrar la mejor suma estrictamente menor que el objetivo (el premio consuelo), y mostrar los resultados.

Salida esperada

```
Mitades: 128 y 128 subconjuntos (en lugar de 16384)
Subconjuntos que suman exactamente 100: 184
Mejor suma menor que 100: 99
```

Reglas

- Complejidad esperada: $O(2^{(n/2)} \times n)$ en lugar de $O(2^n)$. Con $n = 40$: unos 20 millones de operaciones contra un billón.
- El subconjunto vacío cuenta como un subconjunto (suma 0) en cada mitad.
- Memoria: $O(2^{(n/2)})$ sumas guardadas.

Pista (leela solo si te trabás): toda combinación ganadora es "algo de la mitad izquierda" más "algo de la derecha". Si fijás la parte izquierda, sabés exactamente cuánto tiene que sumar la derecha: eso se busca rápido en una lista ordenada.

Bonus (opcional)

- Mostrá una combinación ganadora concreta (guardá la máscara junto con cada suma).
- Contá los subconjuntos cuya suma cae en un rango $[\text{objetivo} - 5, \text{objetivo} + 5]$ con dos punteros.

Cierre: La técnica "encuentro a mitad de camino" transforma problemas exponenciales imposibles en exponenciales manejables; se usa en criptoanálisis y optimización.

Ejercicio 328: Contraseñas de largo N sin patrones prohibidos

Nivel 7: Senior I · Algoritmia avanzada · Temática: Seguridad informática · Dificultad 66/100

Situación: Un banco quiere prohibir en los PIN numéricos las secuencias más fáciles de adivinar ("123", "000", "987") y antes de aplicar la regla necesita medir cuánto achica el espacio de claves posibles: si descarta demasiadas, la política debilita en lugar de fortalecer. Para largos grandes (claves de sistemas internos) el número se informa módulo 1.000.000.007.

Datos de entrada

Formato: primera línea: cantidad de secuencias prohibidas; después una por línea; luego la cantidad de largos a evaluar y un largo por línea.

Ejemplo

```
3
123
000
987
4
4
6
12
1000000
```

Restricciones: hasta 50 secuencias de hasta 10 dígitos; largos hasta 10^6 .

Tu programa debe

1. Construir el autómata de Aho-Corasick con las secuencias prohibidas (como en el ejercicio del chat).
2. Definir $dp[i][s]$ = cantidad de claves de largo i que no contienen ninguna secuencia prohibida y terminan en el estado s del autómata.
3. Para cada estado y cada dígito, avanzar en el autómata; si el nuevo estado completa una secuencia prohibida, descartar.
4. Sumar todos los estados al final. Para largos de hasta 12, mostrar el valor exacto y el porcentaje descartado; para largos mayores, el resto módulo 1.000.000.007.

Salida esperada

```
Estados del autómata: 10
Largo 4: 9941 válidas de 10000 (0.59 % descartadas)
Largo 6: 988308 válidas de 1000000 (1.17 % descartadas)
Largo 12: 971132688261 válidas de 1000000000000 (2.89 % descartadas)
Largo 1000000: 300719905 válidas (resto 1000000007)
```

Reglas

- Complejidad esperada: $O(N \times \text{estados} \times 10)$. Probar las 10^N claves es imposible desde $N = 10$.
- El estado del autómata resume todo lo que importa del pasado: cuánto de alguna secuencia prohibida quedó "a medio escribir".
- Porcentaje con 2 decimales (0,5 hacia arriba). Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): para saber si agregar un dígito completa una secuencia prohibida, no necesitás la clave entera: alcanza con saber en qué nodo del autómata estás. Eso vuelve el conteo una programación dinámica sobre (largo, estado).

Bonus (opcional)

- Para $N = 10^{18}$, expresá la transición como una matriz y usá exponenciación rápida de matrices (nivel 9).
- Calculá la entropía de la política: $\log_2$ de la cantidad de claves válidas, y comparala con la de claves sin restricciones.

Cierre: Contar cadenas que evitan patrones combina autómatas y programación dinámica; se usa para medir políticas de claves y en bioinformática.

Ejercicio 329: Parcelas que no pueden sembrarse juntas

Nivel 7: Senior I · Algoritmia avanzada · Temática: Agricultura · Dificultad 66/100

Situación: Una chacra está dividida en una grilla de parcelas y quiere plantar una variedad de soja que se contagia plagas fácilmente: dos parcelas sembradas no pueden compartir un lado. Algunas parcelas son de piedra y no se pueden usar. El ingeniero agrónomo quiere saber cuántas formas de sembrar existen y cuántas parcelas como máximo puede sembrar.

Datos de entrada

Formato: primera línea: filas y columnas; después la grilla (. = parcela usable, # = piedra).

Ejemplo

```
4 5
.#...
.....
..#..
#....
```

Restricciones: hasta 1.000 filas y 10 columnas (la columna es lo angosto).

Tu programa debe

1. Precalcular las máscaras de una fila sin dos bits vecinos encendidos (mascara AND (mascara << 1) = 0).
2. Procesar fila por fila: una máscara es válida en la fila si solo usa parcelas usables y no comparte bits con la máscara de la fila anterior.
3. Llevar, para cada máscara de la última fila procesada, la cantidad de formas (módulo 1.000.000.007) y el máximo de parcelas sembradas.
4. Mostrar las máscaras válidas por fila, el total de formas y el máximo.

Salida esperada

```
Máscaras válidas por fila: 13 de 32
Formas de sembrar (incluida la de no sembrar nada): 2645
Máximo de parcelas sembradas: 9
```

Reglas

- Complejidad esperada: $O(F \times M^2)$, donde M es la cantidad de máscaras válidas (144 para 10 columnas, en lugar de 1.024).
- El perfil de la fila anterior es lo único que importa para decidir la fila actual.
- No sembrar nada también es una forma válida.

Pista (leela solo si te trabás): si el ancho es chico, una fila entera cabe en un número de bits. Entonces "no hay vecinos en la fila" y "no hay vecinos con la fila de arriba" son dos AND.

Bonus (opcional)

- Reemplazá la regla por "no pueden compartir ni lado ni esquina".
- Pesá cada parcela por su rendimiento y maximizá la producción en lugar de la cantidad.

Cierre: Es la programación dinámica por perfiles, usada en problemas de embaldosado, ubicación de antenas y diseño de circuitos.

Ejercicio 330: Clientes distintos en un rango de días

Nivel 7: Senior I · Algoritmia avanzada · Temática: E-commerce · Dificultad 66/100

Situación: Un e-commerce registra en orden cada compra con el cliente que la hizo, y el área de marketing hace consultas del tipo "¿cuántos clientes distintos compraron entre la compra 1.000 y la 50.000?". Con millones de compras y de consultas, contar con un conjunto en cada consulta es demasiado lento. La clave es responder las consultas en otro orden (fuera de línea).

Datos de entrada

Formato: primera línea n ; segunda, el cliente de cada compra en orden; después la cantidad de consultas y cada una "desde hasta".

Ejemplo

```
10
ana beto ana caro dani beto ana eli caro ana
5
1 4
2 6
5 10
3 3
1 10
```

Restricciones: n hasta 10^6 ; hasta 10^6 consultas; todas las consultas se conocen de antemano.

Tu programa debe

1. Ordenar las consultas por su extremo derecho (recordando su posición original).
2. Recorrer las compras de izquierda a derecha; para cada cliente, marcar con 1 en un árbol de Fenwick solo su aparición más reciente (y quitar la marca de la anterior).
3. Cuando se llega al extremo derecho de una consulta, la respuesta es la suma del Fenwick entre desde y hasta.
4. Mostrar las respuestas en el orden original de las consultas.

Salida esperada

```
Clientes distintos entre las compras 1 y 4: 3
Clientes distintos entre las compras 2 y 6: 4
Clientes distintos entre las compras 5 y 10: 5
Clientes distintos entre las compras 3 y 3: 1
Clientes distintos entre las compras 1 y 10: 5
```

Reglas

- Complejidad esperada: $O((n + q) \log n)$.
- Por qué funciona: con el extremo derecho fijo, cada cliente del tramo se cuenta exactamente una vez, en su última aparición.
- Es un algoritmo fuera de línea: necesita todas las consultas antes de empezar.

Pista (leela solo si te trabás): si solo contás la última aparición de cada cliente hasta el extremo derecho, el conteo de distintos en [desde, hasta] se vuelve una simple suma de rango.

Bonus (opcional)

- Resuélvelo con el algoritmo de Mo y compará los tiempos.
- Si las consultas llegan en línea (una a la vez), investigá los árboles de segmentos persistentes.

Cierre: Las consultas fuera de línea con Fenwick resuelven métricas de "usuarios únicos" por rango, una pregunta clásica de analítica.

Ejercicio 331: Qué día cae el vencimiento dentro de 10^{18} días

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 67/100

Situación: Un sistema de bonos perpetuos tiene que mostrar la fecha de un vencimiento que puede estar a una cantidad absurda de días (los tests automáticos prueban hasta 10^{18} días). Sumar día por día es imposible. Pero el calendario gregoriano se repite exactamente cada 400 años (146.097 días), y la semana cada 7: con aritmética modular sale en tiempo constante.

Datos de entrada

Formato: primera línea: fecha de emisión (AAAA-MM-DD); después, un plazo en días por línea.

Ejemplo

```
2026-09-28
30
1000
146097
1000000000000000000
```

Restricciones: plazos hasta 10^{18} días (el año resultante tiene 16 cifras; usá enteros grandes).

Tu programa debe

1. Convertir la fecha a un número de día absoluto (días desde una fecha de referencia) usando que el calendario tiene eras de 400 años.
2. Sumar el plazo al número de día.
3. Convertir de vuelta a fecha: dividir por 146.097 para las eras, y resolver el año, el mes y el día dentro de la era.
4. Calcular el día de la semana como (número de día + corrimiento) resto 7 y mostrar cada vencimiento.

Salida esperada

```
+30 días: miércoles 28/10/2026
+1000 días: domingo 24/06/2029
+146097 días: lunes 28/09/2426
+1000000000000000000 días: martes 18/05/2737907006990534
```

Reglas

- Complejidad esperada: $O(1)$ por plazo.
- Años bisiestos: divisibles por 4, salvo los divisibles por 100 que no lo son por 400.
- Un truco para simplificar: contar el año desde el 1 de marzo, así el 29 de febrero queda al final.
- Fechas con formato DD/MM/AAAA y día de la semana en minúsculas.
- Casos borde: plazo de exactamente 146.097 días (mismo día y mes, 400 años después, mismo día de la semana).

Pista (leela solo si te trabás): como 146.097 es múltiplo de 7, cada ciclo de 400 años empieza el mismo día de la semana. Dividir por el ciclo te deja un problema chiquito de menos de 400 años.

Bonus (opcional)

- Calculá la fecha que cae 10^{18} días hábiles después (sin sábados ni domingos).
- Contá cuántos viernes 13 hay en un rango de 10^{12} años.

Cierre: La aritmética de calendarios con eras de 400 años es la que usan las bibliotecas de fechas de todos los lenguajes para convertir en tiempo constante.

Ejercicio 332: La canción con más energía tras los cambios

Nivel 7: Senior I · Algoritmia avanzada · Temática: Música · Dificultad 67/100

Situación: Un DJ arma la lista de una fiesta y una app le muestra, para cualquier tramo de la lista, cuál es la canción con más energía (para decidir dónde poner el pico de la noche). Mientras tanto, el algoritmo de la app recalcula la energía de canciones sueltas con los datos de la pista de baile. La lista puede tener miles de canciones y cambiar cada segundo.

Datos de entrada

Formato: primera línea n ; segunda, los nombres; tercera, la energía de cada una; después la cantidad de operaciones: "max a b" o "cambio i v".

Ejemplo

```
8
Zamba Chacarera Cumbia Tango Cuarteto Rock Folk Reggae
62 78 91 45 91 83 50 70
6
max 1 4
max 2 6
cambio 3 40
max 2 6
cambio 8 95
max 1 8
```

Restricciones: n hasta 10^6 ; hasta 10^6 operaciones.

Tu programa debe

1. Construir un árbol de segmentos donde cada nodo guarda el par (energía máxima, posición) de su tramo.
2. Combinar dos nodos quedándose con el de mayor energía; en empate, el de menor posición.
3. Para un cambio, actualizar la hoja y subir recalculando; para una consulta, combinar los nodos que cubren el rango.
4. Mostrar cada resultado con el nombre y la posición.

Salida esperada

```
más energética entre 1 y 4: Cumbia (puesto 3, energía 91)
más energética entre 2 y 6: Cumbia (puesto 3, energía 91)
Cumbia ahora tiene energía 40
más energética entre 2 y 6: Cuarteto (puesto 5, energía 91)
Reggae ahora tiene energía 95
más energética entre 1 y 8: Reggae (puesto 8, energía 95)
```

Reglas

- Complejidad esperada: $O(\log n)$ por operación.
- Guardar la posición junto con el valor evita buscarla después.
- El desempate por menor posición tiene que estar dentro de la función de combinación, no afuera.

Pista (leela solo si te trabás): cualquier dato que se pueda "combinar" de dos mitades (máximo, mínimo, suma, el par máximo-posición) cabe en un árbol de segmentos.

Bonus (opcional)

- Devolvé las 2 canciones más energéticas del rango (cada nodo guarda un top 2).
- Agregá la consulta "primera canción del rango con energía mayor a X" bajando por el árbol.

Cierre: El árbol de segmentos con datos compuestos es la base de muchas funciones de las apps de música, las planillas y los monitores en vivo.

Ejercicio 333: La k-ésima transacción más chica

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 67/100

Situación: El área de riesgo de una billetera virtual monitorea las transacciones del día y consulta en vivo cosas como "¿cuál es la 1.000.^a transacción más chica?" (para calcular percentiles) mientras entran y se anulan transacciones. Ordenar todo en cada consulta es inviable con millones de transacciones por hora.

Datos de entrada

Formato: una operación por línea: "alta monto", "baja monto" (anula una transacción de ese monto) o "kesima k" (la k-ésima más chica de las activas).

Ejemplo

```
alta 15000
alta 3200
alta 87000
alta 3200
alta 45000
kesima 2
kesima 4
baja 3200
kesima 2
alta 500
kesima 1
kesima 5
```

Restricciones: hasta 10^6 operaciones; montos enteros hasta 10^9 (se comprimen porque se conocen de antemano).

Tu programa debe

1. Comprimir los montos: ordenar los montos distintos y reemplazar cada uno por su posición.
2. Mantener un árbol de Fenwick con la cantidad de transacciones activas de cada monto.
3. Para encontrar la k-ésima, bajar por el árbol con saltos de potencias de 2: avanzar mientras la cantidad acumulada siga siendo menor que k.
4. Mostrar cada resultado (o avisar si hay menos de k transacciones activas).

Salida esperada

```
2.a más chica: $3200
4.a más chica: $45000
2.a más chica: $15000
1.a más chica: $500
5.a más chica: $87000
```

Reglas

- Complejidad esperada: $O(\log n)$ por operación.
- La búsqueda sobre el Fenwick en un solo recorrido ($O(\log n)$) es mejor que una búsqueda binaria sobre prefijos ($O(\log^2 n)$).
- Montos repetidos cuentan una vez por transacción.

Pista (leela solo si te trabás): el Fenwick guarda sumas de bloques de tamaño potencia de 2. Bajando desde el bloque más grande podés decidir, bloque por bloque, si la k-ésima está más allá o no, sin volver a sumar desde el principio.

Bonus (opcional)

- Calculá la mediana en vivo con la misma estructura y compará con el método de los dos montículos (nivel 6).

- Soportá montos que no se conocen de antemano con un árbol de segmentos dinámico.

Cierre: Las estadísticas de orden en vivo son la base de percentiles de latencia, rankings y monitores de riesgo.

Ejercicio 334: Mezclar pociones sin que exploten

Nivel 7: Senior I · Algoritmia avanzada · Temática: Videojuegos · Dificultad 67/100

Situación: En un juego de alquimia, el jugador tiene pociones en fila y tiene que mezclarlas todas en una. Solo puede mezclar dos pociones vecinas; si tienen colores a y b , la mezcla tiene color $(a + b)$ resto 100 y larga $a \times b$ de humo. Si el humo total supera cierto límite, el caldero explota, así que el jugador quiere el orden que produzca el menor humo.

Datos de entrada

Formato: una línea con los colores de las pociones (enteros de 0 a 99), en orden.

Ejemplo

```
40 60 20 75 15
```

Restricciones: hasta 300 pociones.

Tu programa debe

1. Definir $dp[i][j]$ = humo mínimo para mezclar las pociones i a j en una.
2. Observar que el color final de un tramo es siempre la suma de sus colores resto 100, sin importar el orden (se puede precalcular con sumas prefijas).
3. Para cada tramo, probar cada punto de corte k : $dp[i][k] + dp[k + 1][j] + \text{color}(i..k) \times \text{color}(k + 1..j)$.
4. Mostrar el humo mínimo, el orden de mezcla con paréntesis (primer corte óptimo en empate) y el color final.

Salida esperada

```
Humo mínimo: 5325
Orden de mezcla: ((40 + 60) + (20 + (75 + 15)))
Color final: 10
```

Reglas

- Complejidad esperada: $O(n^3)$.
- Que el color no dependa del orden es lo que permite que cada tramo tenga un único color: sin eso, el estado sería más grande.
- La diferencia con fusionar logs: acá el costo de unir depende del producto de los colores, no de la suma.

Pista (leela solo si te trabás): $(a + b + c)$ resto 100 da lo mismo en cualquier orden, así que cada tramo tiene un color fijo. Lo único que cambia con el orden es el humo, y eso se decide eligiendo la última mezcla.

Bonus (opcional)

- Mostrá el humo que produce mezclar siempre las dos primeras, para ver cuánto ahorra la estrategia óptima.
- Resolvé la versión en la que las pociones están en círculo.

Cierre: Es otra aplicación de la programación dinámica sobre intervalos, con la misma estructura que la multiplicación óptima de cadenas de matrices.

Ejercicio 335: Boletos capicúa en un rango

Nivel 7: Senior I · Algoritmia avanzada · Temática: Turismo · Dificultad 67/100

Situación: En Argentina hay dos cábalas con los boletos de colectivo: el capicúa (se lee igual al derecho y al revés) y el "de la suerte" (la suma de las tres primeras cifras es igual a la de las tres últimas). Una empresa de transporte quiere lanzar una promo y necesita contar cuántos boletos de cada tipo hay en los talonarios que va a emitir. Los boletos tienen 6 cifras, con ceros a la izquierda.

Datos de entrada

Formato: una línea por talonario: número inicial y final (6 cifras).

Ejemplo

```
000000 999999
123456 234567
500000 500999
```

Restricciones: en la versión general, boletos de hasta 18 cifras y 10^5 rangos: recorrer cada boleto no es opción.

Tu programa debe

1. Contar los capicúa $\leq N$ por construcción: un capicúa de 6 cifras queda fijado por sus 3 primeras; son las mitades menores a la de N, más 1 si el capicúa armado con la mitad de N no se pasa.
2. Contar los "de la suerte" $\leq N$ con programación dinámica sobre dígitos: estado (posición, si todavía vas pegado a N, diferencia entre la suma de la primera mitad y la de la segunda).
3. Responder cada rango como cuenta(final) – cuenta(inicial – 1).
4. Mostrar ambos conteos por talonario.

Salida esperada

```
Boletos 000000 a 999999: capicúa 1000 | de la suerte 55252
Boletos 123456 a 234567: capicúa 111 | de la suerte 5997
Boletos 500000 a 500999: capicúa 1 | de la suerte 21
```

Reglas

- Complejidad esperada: $O(\text{cifras} \times 10 \times \text{rango de diferencias})$ por consulta con la DP; $O(\text{cifras})$ para los capicúa.
- El indicador "pegado a N" (tight) es el corazón de la DP sobre dígitos: mientras vas pegado, el próximo dígito no puede superar al de N.
- Casos borde: rango que empieza en 000000 (cuenta(-1) = 0).

Pista (leela solo si te trabás): contar en $[a, b]$ es contar hasta b menos contar hasta $a - 1$. Y contar hasta N se hace dígito por dígito, recordando solo lo necesario para saber si al final se cumple la condición.

Bonus (opcional)

- Contá los boletos que son capicúa y de la suerte a la vez.
- Generalizá a boletos de 18 cifras y verificá contra fuerza bruta en rangos chicos.

Cierre: La programación dinámica sobre dígitos cuenta números con propiedades en rangos gigantes; es un clásico de competencias y de sistemas de numeración.

Ejercicio 336: Relojes que vuelven a coincidir

Nivel 7: Senior I · Algoritmia avanzada · Temática: Ciencia · Dificultad 67/100

Situación: Un observatorio tiene tres relojes de péndulo históricos, cada uno con un ciclo distinto de marcas (se reinician cada 12, 18 y 30 minutos). Hoy cada uno muestra una marca distinta, y el museo quiere saber en qué minuto las tres señales van a coincidir para anunciar la "alineación". A veces la coincidencia no existe, y eso también hay que detectarlo.

Datos de entrada

Formato: bloques separados por "?": la cantidad de relojes y, por cada uno, "período marca_buscada" (se busca t tal que $t \bmod \text{período} = \text{marca}$).

Ejemplo

```
3
12 5
18 11
30 29
?
2
6 1
4 2
```

Restricciones: hasta 10^5 relojes; períodos hasta 10^9 (el resultado puede ser enorme: enteros grandes).

Tu programa debe

1. Plantear cada reloj como una congruencia $t \equiv \text{marca} \pmod{\text{período}}$.
2. Combinar las congruencias de a dos con el teorema chino del resto generalizado: usar Euclides extendido y verificar que $(a_2 - a_1)$ sea divisible por $\text{mcd}(n_1, n_2)$.
3. Si alguna combinación no es posible, informar que nunca coinciden.
4. Mostrar el primer $t \geq 0$ y cada cuánto se repite la coincidencia (el MCM de los períodos).

Salida esperada

```
t ≡ 5 (mód 12), t ≡ 11 (mód 18), t ≡ 29 (mód 30): primera coincidencia en t = 29, y después cada
180
t ≡ 1 (mód 6), t ≡ 2 (mód 4): nunca coinciden
```

Reglas

- Complejidad esperada: $O(k \log(\text{máx período}))$.
- El teorema chino clásico pide períodos coprimos; el generalizado no, pero puede no haber solución.
- Reducí el resultado módulo el MCM acumulado en cada paso para que no crezca de más.

Pista (leela solo si te trabás): dos relojes con períodos 12 y 18 comparten un factor 6: por eso sus marcas tienen que ser compatibles módulo 6 para que haya coincidencia. Si lo son, la solución se repite cada $\text{mcm}(12, 18) = 36$.

Bonus (opcional)

- Resolvé el sistema también por fuerza bruta (probando $t = 0, 1, 2, \dots$) y compará los tiempos con períodos grandes.
- Aplícalo a una rotación de guardias: ¿cuándo coinciden de guardia tres médicos con ciclos de 4, 6 y 9 días?

Cierre: El teorema chino del resto se usa en criptografía (para acelerar RSA), en calendarios astronómicos y en sincronización de sistemas periódicos.

Ejercicio 337: Productos de una categoría con altas y bajas

Nivel 7: Senior I · Algoritmia avanzada · Temática: E-commerce · Dificultad 68/100

Situación: Un e-commerce organiza su catálogo en un árbol de categorías (Tecnología contiene a Notebooks, que contiene a Gamers...). El stock entra y sale todo el tiempo en categorías específicas, y el tablero de gestión consulta el stock total de cualquier categoría, incluyendo todas sus subcategorías. Recorrer el subárbol en cada consulta es lento con cientos de miles de categorías.

Datos de entrada

Formato: primera parte: "categoría padre" (la raíz tiene "-"), cada una después de su padre; una línea "?" y las operaciones "alta c cantidad", "baja c cantidad" y "total c".

Ejemplo

```
Tienda -
Tecnología Tienda
Hogar Tienda
Celulares Tecnología
Notebooks Tecnología
Cocina Hogar
Gamers Notebooks
?
alta Celulares 120
alta Gamers 15
alta Notebooks 40
alta Cocina 300
total Tecnología
total Tienda
baja Gamers 5
alta Hogar 25
total Notebooks
total Hogar
```

Restricciones: hasta 10^6 categorías y 10^6 operaciones.

Tu programa debe

1. Hacer un recorrido en profundidad y numerar cada categoría al entrar (orden de Euler), guardando también el último número asignado dentro de su subárbol.
2. Así, el subárbol de cada categoría ocupa un tramo contiguo [entrada, salida] de la numeración.
3. Guardar el stock en un árbol de Fenwick indexado por el número de entrada.
4. Las altas y bajas son actualizaciones puntuales; el total de una categoría es la suma del tramo. Mostrar la numeración y cada total.

Salida esperada

```
Orden de Euler: Tienda=1 Tecnología=2 Hogar=6 Celulares=3 Notebooks=4 Cocina=7 Gamers=5
Tecnología (posiciones 2 a 5): 175 unidades
Tienda (posiciones 1 a 7): 475 unidades
Notebooks (posiciones 4 a 5): 50 unidades
Hogar (posiciones 6 a 7): 325 unidades
```

Reglas

- Complejidad esperada: $O(n)$ para numerar y $O(\log n)$ por operación.
- Los hijos se visitan en el orden en que aparecen en la entrada.
- Recorrido iterativo (con pila) para árboles profundos.

Pista (leela solo si te trabás): el truco del recorrido de Euler es aplanar el árbol: todos los descendientes de una categoría quedan numerados de corrido, así que "sumar el subárbol" se vuelve "sumar un rango".

Bonus (opcional)

- Soportá mover una categoría entera a otro padre (investigá los árboles de enlaces y cortes).
- Agregá la consulta "producto con más stock del subárbol" con un árbol de segmentos sobre la misma numeración.

Cierre: Aplanar árboles con el recorrido de Euler es una técnica muy usada en catálogos, sistemas de archivos y jerarquías organizacionales.

Ejercicio 338: Caminos posibles en la red de rutas

Nivel 7: Senior I · Algoritmia avanzada · Temática: Transporte · Dificultad 68/100

Situación: Una empresa de micros de larga distancia tiene una red de tramos que siempre van "hacia el este" (no hay ciclos). Para diseñar una promo de "ruta sorpresa", necesita saber cuántos recorridos distintos hay entre dos ciudades. En la red nacional completa ese número es astronómico, así que se informa módulo 1.000.000.007.

Datos de entrada

Formato: tramos dirigidos "origen destino" (sin ciclos); una línea "?" y consultas "origen destino".

Ejemplo

```
Córdoba Rosario
Córdoba VillaMaría
Córdoba RíoCuarto
VillaMaría Rosario
VillaMaría Venado
RíoCuarto Venado
Venado Rosario
Venado Pergamino
Rosario Pergamino
Pergamino BuenosAires
Rosario BuenosAires
?
Córdoba BuenosAires
Córdoba Pergamino
RíoCuarto Córdoba
```

Restricciones: hasta 10^5 ciudades y 10^6 tramos; hasta 10^3 consultas.

Tu programa debe

1. Calcular un orden topológico con el algoritmo de Kahn.
2. Para cada consulta, inicializar caminos[origen] = 1 y el resto en 0.
3. Recorrer las ciudades en orden topológico y sumar caminos[u] a cada vecino (módulo 1.000.000.007).
4. Mostrar el orden topológico y la cantidad de caminos de cada consulta (0 si el destino no es alcanzable).

Salida esperada

```
Orden topológico: Córdoba VillaMaría RíoCuarto Venado Rosario Pergamino BuenosAires
Caminos de Córdoba a BuenosAires: 10
Caminos de Córdoba a Pergamino: 6
Caminos de RíoCuarto a Córdoba: 0
```

Reglas

- Complejidad esperada: $O(V + E)$ por consulta.
- La cantidad de caminos crece exponencialmente con el tamaño de la red: sin módulo, desborda enseguida.
- Casos borde: destino anterior al origen en el orden topológico (0 caminos).

Pista (leela solo si te trabás): la cantidad de formas de llegar a una ciudad es la suma de las formas de llegar a cada ciudad que la precede. En orden topológico, cuando llegás a una ciudad ya tenés todos sus predecesores resueltos.

Bonus (opcional)

- Encontrá el recorrido más largo en horas (camino más largo en un DAG, también en $O(V + E)$).

- Contá los caminos que pasan obligatoriamente por Rosario (caminos hasta Rosario $\times$ caminos desde Rosario).

Cierre: La programación dinámica sobre grafos acíclicos resuelve conteos de rutas, planificación de proyectos y análisis de dependencias.

Ejercicio 339: El par de claves con XOR máximo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Seguridad informática · Dificultad 68/100

Situación: Un equipo de seguridad evalúa un generador de claves de prueba: si dos claves son muy parecidas en sus bits, una podría deducirse de la otra. Una medida de diferencia es el XOR: cuanto más alto, más bits distintos en las posiciones altas. El equipo quiere encontrar el par de claves más "distinto" (XOR máximo) entre millones, sin comparar todos los pares.

Datos de entrada

Formato: primera línea: cantidad de claves y cantidad de bits; después "nombre valor".

Ejemplo

```
6 5
llave_A 13
llave_B 25
llave_C 8
llave_D 19
llave_E 2
llave_F 30
```

Restricciones: hasta 10^6 claves de hasta 60 bits.

Tu programa debe

1. Insertar las claves, de a una, en un trie binario que recorre los bits del más significativo al menos significativo.
2. Antes de insertar cada clave, buscar en el trie la clave ya insertada que maximiza el XOR: en cada nivel, intentar ir por el bit opuesto al propio.
3. Quedarse con el mejor par (en empate, el primero encontrado).
4. Mostrar el par con sus valores en binario, el XOR y la cantidad de nodos del trie.

Salida esperada

```
Par con XOR máximo: llave_A (01101) y llave_D (10011)
XOR: 30 (11110)
Nodos del trie: 25
```

Reglas

- Complejidad esperada: $O(n \times \text{bits})$. Comparar todos los pares es $O(n^2)$.
- Ir por el bit opuesto siempre que se pueda es óptimo porque un bit alto vale más que todos los bits más bajos juntos.
- Binarios con la cantidad de bits indicada (ceros a la izquierda).

Pista (leela solo si te trabás): para maximizar un XOR conviene que el primer bit que difiera esté lo más arriba posible. Así que, bajando por el trie desde el bit más alto, siempre elegís la rama contraria si existe.

Bonus (opcional)

- Encontrá el tramo contiguo de una lista con XOR máximo combinando prefijos XOR con este trie.
- Respondé consultas "clave con XOR máximo respecto de X" solo entre las claves con valor $\leq M$ (ordenando fuera de línea).

Cierre: El trie binario resuelve problemas de XOR máximo y es la misma estructura que usan los routers para buscar el prefijo más largo.

Ejercicio 340: Tabla de posiciones con actualizaciones en vivo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Deportes · Dificultad 68/100

Situación: Una app de fútbol amateur muestra en vivo la tabla de posiciones de una liga con miles de equipos: cuando termina un partido, se suman los puntos y cada usuario quiere ver en qué puesto quedó su equipo. Reordenar la tabla completa en cada resultado es un desperdicio. Como los puntajes están acotados, un árbol de segmentos sobre los puntajes responde todo en tiempo logarítmico.

Datos de entrada

Formato: primera línea: cantidad de equipos y puntaje máximo posible; después "equipo puntos"; luego operaciones: "puntos e p" (suma p), "puesto e", "cuantos_arriba p" y "kesimo k" (puntaje del k-ésimo mejor).

Ejemplo

```
6 30
Tigres 12
Pumas 15
Cóndores 9
Leones 15
Zorros 20
Halcones 11
puesto Pumas
puntos Cóndores 6
puesto Cóndores
cuantos_arriba 14
puntos Halcones 3
puesto Halcones
kesimo 1
kesimo 4
```

Restricciones: hasta 10^6 equipos, puntajes hasta 10^6 y 10^6 operaciones.

Tu programa debe

1. Armar un árbol de segmentos indexado por puntaje, donde cada hoja cuenta cuántos equipos tienen ese puntaje.
2. Al sumar puntos, restar 1 en el puntaje viejo y sumar 1 en el nuevo.
3. El puesto de un equipo es 1 + cantidad de equipos con puntaje estrictamente mayor (una suma de rango).
4. Para el k-ésimo, bajar por el árbol priorizando la mitad de puntajes altos. Mostrar cada resultado.

Salida esperada

```
Pumas está 2.º (15 pts)
Cóndores suma 6 -> 15 pts
Cóndores está 2.º (15 pts)
Equipos con más de 14 pts: 4
Halcones suma 3 -> 14 pts
Halcones está 5.º (14 pts)
Puntaje del 1.º puesto: 20 (Zorros)
Puntaje del 4.º puesto: 15 (Pumas, Cóndores, Leones)
```

Reglas

- Complejidad esperada: $O(\log M)$ por operación, con M el puntaje máximo.
- Equipos empatados comparten puesto ("puesto de competición": 1, 2, 2, 4...).
- En "kesimo", los equipos con ese puntaje se listan en el orden de la entrada.

Pista (leela solo si te trabás): en lugar de ordenar equipos, contá cuántos hay en cada puntaje. El puesto de un equipo es cuántos tienen más puntos que él: una suma sobre los puntajes más altos.

Bonus (opcional)

- Agregá diferencia de gol como criterio de desempate (el índice pasa a ser un par).
- Si los puntajes no están acotados, comprimilos fuera de línea.

Cierre: Contar por valor en lugar de ordenar es el truco detrás de rankings en vivo de juegos, ligas y apps con millones de usuarios.

Ejercicio 341: Repartir caramelos entre chicos

Nivel 7: Senior I · Algoritmia avanzada · Temática: Educación · Dificultad 68/100

Situación: Una maestra de primer grado quiere saber de cuántas formas puede repartir los caramelos de un paquete entre los chicos del grado (los caramelos son todos iguales, los chicos no), para un juego de probabilidad. A veces exige que cada chico reciba un mínimo. Para una actividad con toda la escuela y un millón de caramelos, el número se informa módulo 1.000.000.007.

Datos de entrada

Formato: primera línea: cantidad de consultas; después "caramelos chicos mínimo_por_chico".

Ejemplo

```
4
20 4 0
20 4 1
20 4 3
1000000 1000 100
```

Restricciones: caramelos hasta 10^6 ; chicos hasta 10^5 .

Tu programa debe

1. Si hay mínimo m , dar primero m caramelos a cada chico: quedan $r = \text{caramelos} - \text{chicos} \times m$ para repartir libremente (si $r < 0$, no hay formas).
2. Repartir r caramelos iguales entre k chicos, con cero permitido, es $C(r + k - 1, k - 1)$ (estrellas y barras).
3. Calcular el combinatorio con aritmética modular y el inverso del denominador.
4. Mostrar el combinatorio y su valor (exacto si es menor que el módulo, o el resto).

Salida esperada

```
20 caramelos, 4 chicos, mínimo 0: C(23, 3) = 1771
20 caramelos, 4 chicos, mínimo 1: C(19, 3) = 969
20 caramelos, 4 chicos, mínimo 3: C(11, 3) = 165
1000000 caramelos, 1000 chicos, mínimo 100: C(900999, 999) = 645008998 (resto 1000000007)
```

Reglas

- Complejidad esperada: $O(k + \log \text{MOD})$ por consulta ($O(1)$ con factoriales precalculados).
- Estrellas y barras: r estrellas (caramelos) y $k - 1$ barras (separadores) en fila; cada orden es un reparto.
- Casos borde: mínimo tan alto que no alcanzan los caramelos (0 formas).

Pista (leela solo si te trabás): imaginate los caramelos en fila y poné $k - 1$ palitos separándolos: lo que queda entre palitos es lo de cada chico. Contar repartos es contar dónde van los palitos.

Bonus (opcional)

- Agregá un máximo por chico con inclusión-exclusión.
- Si los caramelos son de dos sabores distintos, combiná dos repartos independientes.

Cierre: Estrellas y barras es la herramienta combinatoria para contar repartos, y aparece en probabilidad, asignación de recursos y química.

Ejercicio 342: MCD de rangos de lotes

Nivel 7: Senior I · Algoritmia avanzada · Temática: Agricultura · Dificultad 69/100

Situación: Una cooperativa yerbatera tiene lotes de cosecha numerados y quiere empaquetar tramos consecutivos de lotes en cajas del mismo peso, sin sobrantes, usando la caja más grande posible: el MCD de los pesos del tramo. Los lotes no cambian durante la temporada, pero se hacen millones de consultas.

Datos de entrada

Formato: primera línea n ; segunda, el peso de cada lote en kg; después la cantidad de consultas y cada una "desde hasta".

Ejemplo

```
8
360 240 480 600 150 90 300 450
5
1 4
2 3
4 6
1 8
7 8
```

Restricciones: n hasta 10^6 ; hasta 10^7 consultas; los pesos no cambian.

Tu programa debe

1. Construir una tabla dispersa: $T[j][i] = \text{MCD de los } 2^j \text{ lotes que empiezan en } i$, con $T[j][i] = \text{mcd}(T[j-1][i], T[j-1][i + 2^{(j-1)}])$.
2. Para una consulta de largo L , tomar $k = \lfloor \log_2 L \rfloor$ y combinar dos bloques de largo 2^k que cubren el tramo (pueden superponerse).
3. Responder $\text{mcd}(T[k][\text{desde}], T[k][\text{hasta} - 2^k + 1])$.
4. Mostrar la cantidad de niveles y cada respuesta.

Salida esperada

```
Niveles de la tabla: 4
Lotes 1 a 4: cajas de 120 kg
Lotes 2 a 3: cajas de 240 kg
Lotes 4 a 6: cajas de 30 kg
Lotes 1 a 8: cajas de 30 kg
Lotes 7 a 8: cajas de 150 kg
```

Reglas

- Complejidad esperada: $O(n \log n)$ de construcción y $O(1)$ por consulta (más el costo del MCD).
- La superposición es válida porque el MCD es idempotente: $\text{mcd}(x, x) = x$. Con sumas no se podría.
- Sin actualizaciones: si los pesos cambiaran, harían falta un árbol de segmentos.

Pista (leela solo si te trabás): dos bloques de potencia de 2 alcanzan para cubrir cualquier tramo, aunque se pisen en el medio: contar dos veces un lote no cambia el MCD.

Bonus (opcional)

- Usá la misma tabla para mínimo y máximo por rango y compará con el árbol de segmentos.
- Contá cuántos tramos tienen MCD exactamente 30 (hay pocos valores distintos de MCD por extremo izquierdo).

Cierre: La tabla dispersa responde en tiempo constante consultas de operaciones idempotentes, y se usa en ancestros comunes de árboles y análisis de series.

Ejercicio 343: Fusionar lotes de pedidos al menor costo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Logística y envíos · Dificultad 69/100

Situación: Un centro de distribución consolida lotes de pedidos que están en fila en la cinta, pero la máquina de consolidación solo trabaja con exactamente 3 lotes vecinos a la vez, y cuesta tanto como la suma de los pedidos de esos tres. Hay que dejar todo en un solo lote al menor costo, o avisar si con esa máquina es imposible.

Datos de entrada

Formato: primera línea: K (lotes por fusión); segunda: los pedidos de cada lote, en orden.

Ejemplo

```
3
30 20 50 10 40 60 25
```

Restricciones: hasta 300 lotes; K entre 2 y 30.

Tu programa debe

1. Verificar que sea posible: cada fusión reduce la cantidad en $K - 1$, así que hace falta $(n - 1)$ divisible por $(K - 1)$.
2. Definir $dp[i][j]$ = costo mínimo de reducir el tramo $i..j$ a la menor cantidad posible de lotes.
3. Partir el tramo en un bloque izquierdo que se reduce a un solo lote y el resto: probar cortes de $K - 1$ en $K - 1$.
4. Si el tramo puede quedar en un solo lote ($(\text{largo} - 1)$ divisible por $K - 1$), sumar su total. Mostrar el costo mínimo.

Salida esperada

```
Costo mínimo fusionando de a 3: 440
```

Caso 2 (borde)

Con 4 lotes y fusiones de a 3:

```
3
30 20 50 10
```

Salida esperada del caso 2:

```
Imposible: con 4 lotes no se puede terminar en uno solo fusionando de a 3
```

Reglas

- Complejidad esperada: $O(n^3 / K)$.
- Con $K = 2$ es el mismo problema que fusionar logs; generalizarlo exige cuidar que los tramos queden en tamaños compatibles.
- Casos borde: combinación imposible.

Pista (leela solo si te trabás): no todo tramo puede terminar en un solo lote: solo los de largo 1, K, $2K - 1$... Por eso los cortes se prueban saltando de a $K - 1$: el bloque izquierdo siempre queda en un único lote.

Bonus (opcional)

- Reconstruí qué tramos se fusionan en cada paso.
- Permití usar también una máquina de a 2, más cara (costo $\times 1,5$), y encontrá la combinación óptima.

Cierre: La generalización de la programación dinámica sobre intervalos a fusiones de K elementos es un clásico de entrevistas técnicas y de competencias.

Ejercicio 344: Combinaciones de vacunas compatibles

Nivel 7: Senior I · Algoritmia avanzada · Temática: Salud · Dificultad 69/100

Situación: Un vacunatorio quiere armar el mejor esquema para aplicar en una sola visita, pero algunas vacunas no se deben aplicar juntas según las indicaciones que le pasó el equipo médico (en este ejercicio, las incompatibilidades son ficticias). Cada vacuna tiene un puntaje de protección para el calendario del paciente. Con hasta 20 vacunas, se pueden revisar todas las combinaciones con máscaras de bits.

Datos de entrada

Formato: primera línea n ; después "vacuna puntaje"; luego la cantidad de pares incompatibles y cada par.

Ejemplo

```
6
Gripe 8
Neumococo 9
HepatitisB 6
Tétanos 7
Sarampión 10
Varicela 8
3
Sarampión Varicela
Gripe Neumococo
HepatitisB Varicela
```

Restricciones: $n \leq 20$ ($2^{20} \approx 10^6$ combinaciones).

Tu programa debe

1. Asignar a cada vacuna un bit y guardar, para cada una, la máscara de sus incompatibles.
2. Recorrer todas las máscaras de 0 a $2^n - 1$; una combinación es compatible si ninguna vacuna incluida choca con otra incluida (máscara AND incompatibles(i) = 0).
3. Contar las compatibles y quedarse con la de mayor puntaje (en empate, la de menos vacunas).
4. Mostrar el conteo y la mejor combinación en el orden de la entrada.

Salida esperada

```
Combinaciones compatibles (incluida la vacía): 30 de 64
Mejor combinación: Neumococo, HepatitisB, Tétanos, Sarampión (protección 32)
```

Reglas

- Complejidad esperada: $O(2^n \times n)$. Con $n = 20$ son unos 20 millones de operaciones.
- Mejora: calcular el puntaje de cada máscara a partir de la máscara sin su bit más bajo, en $O(1)$.
- Las incompatibilidades son simétricas: se cargan en las dos vacunas.

Pista (leela solo si te trabás): con n chico, un número de 0 a $2^n - 1$ es un subconjunto. Verificar compatibilidad es un AND por vacuna: nada de listas ni búsquedas.

Bonus (opcional)

- Calculá el puntaje de todas las máscaras en $O(2^n)$ reutilizando la máscara sin el bit más bajo.
- Dividí las vacunas en la mínima cantidad de visitas (coloreo de grafos con máscaras, $O(3^n)$).

Cierre: Enumerar subconjuntos con bits es la herramienta para problemas chicos pero combinatorios: armado de equipos, menús y configuraciones.

Ejercicio 345: Ocupación de camas por rango de días

Nivel 7: Senior I · Algoritmia avanzada · Temática: Salud · Dificultad 69/100

Situación: Una clínica tiene una cantidad fija de camas para internaciones programadas. Cada reserva ocupa una cama durante un rango de días, y el sistema tiene que aceptarla solo si en ningún día del rango se supera la capacidad. También se cancelan reservas y se consulta la ocupación máxima de un período. El calendario cubre años y hay miles de reservas por día.

Datos de entrada

Formato: primera línea: cantidad de días y camas; después operaciones: "reserva a b", "cancelar a b" (libera una reserva confirmada de ese rango) y "ocupacion a b".

Ejemplo

```
10 3
reserva 1 4
reserva 3 6
reserva 2 5
reserva 4 8
reserva 7 10
ocupacion 1 10
cancelar 1 4
reserva 4 8
ocupacion 3 5
```

Restricciones: hasta 10^6 días y 10^6 operaciones.

Tu programa debe

1. Mantener un árbol de segmentos sobre los días que soporte "sumar v a un rango" y "máximo de un rango" (con ajuste pendiente por nodo).
2. Para una reserva, consultar el máximo del rango: si sumando 1 supera la capacidad, rechazarla; si no, sumar 1 al rango.
3. Para una cancelación, sumar -1 al rango; para una consulta, devolver el máximo.
4. Mostrar el resultado de cada operación.

Salida esperada

```
reserva días 1 a 4: confirmada
reserva días 3 a 6: confirmada
reserva días 2 a 5: confirmada
reserva días 4 a 8: RECHAZADA (sin camas)
reserva días 7 a 10: confirmada
ocupación máxima días 1 a 10: 3 de 3
cancelación días 1 a 4
reserva días 4 a 8: confirmada
ocupación máxima días 3 a 5: 3 de 3
```

Reglas

- Complejidad esperada: $O(\log n)$ por operación.
- Variante sin empujar: el ajuste pendiente de un nodo se suma al máximo de sus hijos al subir, y se acumula al bajar en la consulta.
- Casos borde: reserva de un solo día; rechazo por un único día lleno en el medio del rango.

Pista (leela solo si te trabás): aceptar una reserva equivale a preguntar "¿el día más lleno del rango tiene lugar?". Esa es una consulta de máximo, y reservar es sumar 1 a todo el rango.

Bonus (opcional)

- Devolvé el primer día del rango que está lleno (para sugerir fechas alternativas).

- Soportá camas de dos tipos (común y terapia) con capacidades separadas.

Cierre: Los árboles de segmentos con rangos son la base de los sistemas de reservas de hoteles, aerolíneas y salas.

Ejercicio 346: Cuotas coprimas con el período

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 69/100

Situación: Una fintech sortea el día de cobro de las cuotas en un ciclo de n días: arranca en el día 1 y avanza siempre de a k días (con vuelta al principio). Solo sirven los saltos k que recorren todos los días del ciclo antes de repetir, y eso pasa exactamente cuando k y n son coprimos. La cantidad de saltos válidos es la función ϕ de Euler, que además sirve para reducir exponentes gigantes.

Datos de entrada

Formato: una línea por período n ; después una línea "?" y una línea "base exponente módulo" (base y módulo coprimos).

Ejemplo

```
30
36
97
1000000
?
7 100000000000000000000 30
```

Restricciones: n hasta 10^{12} ; exponente hasta 10^{18} .

Tu programa debe

- Factorizar n y calcular $\phi(n) = n \times$ producto de $(1 - 1/p)$ para cada primo p que divide a n (con enteros: dividir por p y multiplicar por $p - 1$).
- Para períodos chicos (hasta 36), listar también los saltos válidos.
- Para la potencia: por el teorema de Euler, si $\text{mcd}(\text{base}, m) = 1$, entonces $\text{base}^e \equiv \text{base}^{(e \text{ resto } \phi(m))} \pmod{m}$.
- Mostrar ϕ de cada período y la potencia calculada por las dos vías.

Salida esperada

```
Período 30:  $\phi = 8$  pasos válidos (1 7 11 13 17 19 23 29)
Período 36:  $\phi = 12$  pasos válidos (1 5 7 11 13 17 19 23 25 29 31 35)
Período 97:  $\phi = 96$  pasos válidos
Período 1000000:  $\phi = 400000$  pasos válidos
7^100000000000000000000 resto 30: exponente reducido 100000000000000000000 resto  $\phi(30) = 8 \rightarrow 1$ 
(verificación directa: 1)
```

Reglas

- Complejidad esperada: $O(\sqrt{n})$ por período (por la factorización).
- $\phi(p) = p - 1$ si p es primo; ϕ es multiplicativa para números coprimos.
- El teorema de Euler exige base y módulo coprimos; si no lo son, la reducción puede fallar.

Pista (leela solo si te trabas): avanzar de a k en un ciclo de n recorre todo solo si k y n no comparten factores: si compartieran un d , solo visitarías los días de un "subciclo" de n/d posiciones.

Bonus (opcional)

- Calcula ϕ de todos los números hasta 10^6 con una criba (como la de Eratóstenes).
- Calcula una torre de potencias $a^{(b^c)} \text{ resto } m$ aplicando Euler de forma recursiva.

Cierre: La función ϕ de Euler es el corazón matemático de RSA y de muchos esquemas de generación de ciclos y claves.

Ejercicio 347: El juego de las fichas del recreo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Videojuegos · Dificultad 69/100

Situación: En el recreo, dos chicos juegan con pilas de fichas: por turno, cada uno elige una pila y saca las fichas que quiera (al menos una). Gana quien saca la última ficha. Una app educativa quiere decir, para cualquier posición, quién gana si los dos juegan perfecto y cuál es la jugada ganadora. El juego se llama Nim, y la respuesta está en el XOR.

Datos de entrada

Formato: posiciones separadas por "?": las cantidades de fichas de cada pila.

Ejemplo

```
3 4 5
?
1 5 9 12
?
7 7
```

Restricciones: hasta 10^6 pilas de hasta 10^{18} fichas.

Tu programa debe

1. Calcular el XOR de todas las pilas (la "suma de Nim").
2. Si es 0, el jugador que empieza pierde con juego perfecto del rival.
3. Si no es 0, gana quien empieza: para cada pila p con $p \text{ XOR total} < p$, sacar $p - (p \text{ XOR total})$ fichas deja XOR 0.
4. Mostrar el XOR y el veredicto con todas las jugadas ganadoras.

Salida esperada

```
Pilas 3 4 5: XOR = 2 -> gana quien empieza: sacar 2 de la pila 1
Pilas 1 5 9 12: XOR = 1 -> gana quien empieza: sacar 1 de la pila 1; sacar 1 de la pila 2; sacar 1
de la pila 3
Pilas 7 7: XOR = 0 -> pierde quien empieza (cualquier jugada deja una posición ganadora al rival)
```

Reglas

- Complejidad esperada: $O(n)$.
- Por qué funciona: desde XOR 0, cualquier jugada lo vuelve distinto de 0; desde $\text{XOR} \neq 0$, siempre existe una jugada que lo vuelve 0. Y la posición final (todo vacío) tiene XOR 0.
- Las pilas se numeran desde 1.

Pista (leela solo si te trabás): el XOR 0 es una posición "perdedora para quien le toca": vos querés dejarle siempre al rival XOR 0, y él nunca puede devolverte uno.

Bonus (opcional)

- Variante: por turno se sacan como máximo 3 fichas. Calculá el valor de Grundy de cada pila (p resto 4) y aplicá el XOR a esos valores.
- Programá un rival que juegue perfecto y otro al azar, y simulá 1.000 partidas.

Cierre: Nim es la base de la teoría de juegos combinatorios: el teorema de Sprague-Grundy reduce cualquier juego imparcial a pilas de Nim.

Ejercicio 348: Desafío de nivel: Libro mayor con correcciones retroactivas

Nivel 7: Senior I · Algoritmia avanzada · Temática: Finanzas y FinTech · Dificultad 70/100

Situación: El libro mayor de una fintech recibe movimientos con fecha y un id único, y a veces hay que corregir retroactivamente el monto de un movimiento viejo. El auditor externo no quiere bajarse todo el libro: pide el saldo de un período, una "huella" numérica del período para comparar con su propia copia, y un control de ids para detectar movimientos faltantes. Todo tiene que responder en tiempo logarítmico, aunque haya correcciones.

Datos de entrada

Formato: primera línea: cantidad de días; después operaciones: "mov día monto id", "corregir id nuevo_monto", "saldo a b", "huella a b" y "control a b".

Huella de un período = suma de $\text{monto} \times 31^{\text{día}}$, resto 1.000.003 (tomando el resto no negativo).
Control = XOR de los ids de los movimientos del período.

Ejemplo

```
10
mov 2 15000 101
mov 3 -4000 102
mov 5 22000 103
mov 7 -9000 104
mov 9 3000 105
saldo 1 10
huella 1 10
corregir 103 20000
saldo 4 8
huella 1 10
control 2 9
corregir 999 500
```

Restricciones: hasta 10^6 días, 10^7 movimientos y 10^6 consultas; montos en enteros de 64 bits.

Tu programa debe

1. Mantener tres árboles de Fenwick sobre los días: uno de sumas de montos, uno de huellas (módulo 1.000.003) y uno de XOR de ids.
2. Al registrar un movimiento, sumar el monto, sumar $\text{monto} \times 31^{\text{día}}$ (resto 1.000.003) y hacer XOR con el id; guardar día y monto por id.
3. Al corregir, aplicar solo la diferencia (nuevo – viejo) en la suma y en la huella; el control de ids no cambia. Si el id no existe, informar el error.
4. Responder saldo, huella y control como diferencias de prefijos (resta, resta modular y XOR), y mostrar cada resultado.

Salida esperada

```
saldo días 1 a 10: +27000
huella días 1 a 10: 650572
corrección 103 (día 5): 22000 -> 20000
saldo días 4 a 8: +11000
huella días 1 a 10: 520346
control de ids días 2 a 9: 101
corrección 999: ERROR, el movimiento no existe
```

Reglas

- Complejidad esperada: $O(\log n)$ por operación en los tres árboles.
- Las tres operaciones son "invertibles" (suma con resta, suma modular con resta modular, XOR consigo mismo): por eso los tres funcionan como prefijos.
- La huella pondera por día para que mover un monto de un día a otro también cambie la huella.

- Casos borde: corrección de un id inexistente; huella negativa antes del resto (se normaliza).

Pista (leela solo si te trabás): las correcciones retroactivas no son un problema si guardás diferencias: corregir es sumar (nuevo – viejo) en el mismo día. Los tres árboles son la misma estructura con distinta operación.

Bonus (opcional)

- Detectá cuál es el día con diferencias entre tu libro y el del auditor usando búsqueda binaria sobre huellas de prefijos ($O(\log^2 n)$ comparaciones).
- Agregá la operación "anular id" que lo quita del saldo, de la huella y del control.

Cierre: Resúmenes invertibles sobre estructuras de prefijos permiten auditar grandes libros contables comparando pocos números, como hacen los sistemas de conciliación.

Ejercicio 349: Desafío de nivel: El teclado predictivo

Nivel 7: Senior I · Algoritmia avanzada · Temática: Telecomunicaciones · Dificultad 70/100

Situación: Un teclado predictivo para celulares recibe a veces texto pegado sin espacios (por ejemplo, desde un dictado que falló) y tiene que separarlo en palabras, elegir la separación más probable según la frecuencia de uso, y calcular cuántos toques ahorraría el usuario con el autocompletado. Todo con un diccionario de cientos de miles de palabras.

Datos de entrada

Formato: primera línea: cantidad de palabras; después "palabra frecuencia"; última línea: el texto sin espacios.

Ejemplo

```
7
hola 90
hoy 70
ola 20
la 60
como 80
co 10
mo 15
holacomo
```

Restricciones: diccionario de hasta 10^6 palabras de hasta 30 letras; textos de hasta 10^5 caracteres.

Tu programa debe

1. Guardar el diccionario en un trie para encontrar, desde cada posición del texto, todas las palabras que empiezan ahí, en un solo recorrido.
2. Contar las formas de separar el texto con programación dinámica de derecha a izquierda: $\text{formas}[i] = \text{suma de formas}[j]$ para cada palabra $\text{texto}[i..j]$ (módulo 1.000.000.007).
3. Con la misma DP, encontrar la separación de frecuencia total máxima (en empate, la primera encontrada).
4. Para cada palabra elegida, calcular los toques: el menor prefijo cuya sugerencia principal es esa palabra, más 1 toque para aceptar (o el largo completo si no hay atajo más corto). Mostrar todo.

Salida esperada

```
Formas de separar "holacomo": 2
Separación más probable: hola como (frecuencia total 170)
  hola: 2 toques (escribir "h" y aceptar la sugerencia)
  como: 2 toques (escribir "c" y aceptar la sugerencia)
Total: 4 toques en lugar de 8
```

Caso 2 (borde)

Con dos separaciones posibles:

```
3
sol 50
sola 20
las 30
solasol
```

Salida esperada del caso 2:

```
Formas de separar "solasol": 1
Separación más probable: sola sol (frecuencia total 70)
  sola: 4 toques (sin atajo)
  sol: 2 toques (escribir "s" y aceptar la sugerencia)
Total: 6 toques en lugar de 7
```


Reglas

- Complejidad esperada: $O(\text{largo del texto} \times \text{largo máximo de palabra})$ para la DP.
- Sugerencia principal de un prefijo: la palabra más frecuente que empieza con él (en empate, la más corta y después la alfabéticamente menor).
- El atajo se usa solo si ahorra toques ($\text{largo del prefijo} + 1 < \text{largo de la palabra}$); se toma el prefijo más corto que funcione.
- Casos borde: texto que no se puede separar.

Pista (leela solo si te trabás): son tres problemas encadenados sobre la misma estructura: el trie encuentra candidatos, la DP combina (contar es sumar, optimizar es tomar máximo) y las sugerencias salen de guardar el mejor de cada prefijo.

Bonus (opcional)

- Tolerará una letra equivocada por palabra en la separación.
- Precalculá en cada nodo del trie la sugerencia principal para no recorrer el diccionario.

Cierre: Separar texto sin espacios es un problema real en idiomas como el chino y el japonés, y en la corrección de dictados.

Ejercicio 350: Desafío de nivel: Plan de inversión en la red de riego

Nivel 7: Senior I · Algoritmia avanzada · Temática: Agricultura · Dificultad 70/100

Situación: Una cooperativa de riego tiene una red de canales en forma de árbol que sale de la toma del río. Puede invertir en revestir canales, pero un canal solo sirve revestido si el canal del que se alimenta también lo está. Cada canal tiene un costo, una ganancia esperada y riega un tipo de cultivo; la asamblea exige que el plan beneficie a todos los cultivos de la zona, sin pasarse del presupuesto.

Datos de entrada

Formato: primera línea: cantidad de nodos, presupuesto (en millones) y cantidad de cultivos; después "canal padre costo ganancia cultivo" (la toma tiene padre "-", costo 0 y cultivo "-"), cada uno después de su padre.

Ejemplo

```
8 12 3
Toma - 0 0 -
Norte Toma 3 4 soja
Sur Toma 4 6 maíz
NorteA Norte 2 5 trigo
NorteB Norte 5 14 soja
SurA Sur 3 3 trigo
SurB Sur 2 7 maíz
SurC SurA 4 8 soja
```

Restricciones: hasta 200 canales, presupuesto hasta 100 y hasta 4 cultivos (16 máscaras).

Tu programa debe

1. Para cada canal, calcular $dp[\text{canal}]$ = mapa de (costo, máscara de cultivos) a ganancia máxima, considerando solo planes de su subárbol que incluyen al canal.
2. Combinar los hijos de a uno, como una mochila: para cada hijo, elegir no incluir nada de su subárbol o alguno de sus planes; sumar costos (sin pasar el presupuesto), unir máscaras con OR y sumar ganancias.
3. En la raíz, quedarse con los planes cuya máscara cubre todos los cultivos y elegir el de mayor ganancia (en empate, el de menor costo).
4. Mostrar los canales elegidos (en el orden de la entrada), el costo, la ganancia y, para comparar, la ganancia sin exigir todos los cultivos.

Salida esperada

```
Cultivos a cubrir: soja, maíz, trigo
Canales a mejorar: Norte, Sur, NorteA, SurB
Costo: 11 de 12 | Ganancia: 22
Sin exigir todos los cultivos, la ganancia máxima sería 24
```

Caso 2 (borde)

Con presupuesto insuficiente para cubrir los dos cultivos:

```
3 5 2
Toma - 0 0 -
A Toma 3 4 soja
B Toma 4 6 maíz
```

Salida esperada del caso 2:

```
No hay plan que cubra todos los cultivos con ese presupuesto
```

Reglas

- Complejidad esperada: $O(n \times (B \times 2^K)^2)$ en el peor caso; con los límites dados es manejable.
- Un canal cuyo costo supera el presupuesto no puede estar en ningún plan (ni su subárbol).
- Los cultivos se numeran por orden de aparición (el primero es el bit 0).
- Casos borde: ningún plan cubre todos los cultivos.

Pista (leela solo si te trabás): es una mochila sobre un árbol con una dimensión extra: además del costo, cada estado recuerda qué cultivos ya cubrió. La máscara de bits hace que "unir cultivos" sea un OR.

Bonus (opcional)

- Reducí la complejidad de la combinación acotando el costo por el tamaño del subárbol (el truco de "la suma de tamaños").
- Permití dos niveles de mejora por canal (revestido simple o completo) con costos y ganancias distintos.

Cierre: Combinar programación dinámica en árboles con máscaras de bits resuelve problemas reales de inversión en redes con restricciones de cobertura.

Lote 15: ejercicios 351 a 375

Nivel 8

Senior II · Algoritmos de sistemas

Ejercicios 351 a 400

Qué vas a practicar

Algoritmos de sistemas: cachés, limitadores, idempotencia, event sourcing, concurrencia, libros de órdenes y probabilidad.

Qué se espera al terminar

Diseñar componentes confiables con requisitos no funcionales explícitos.

Conceptos del nivel

caché LRU, rate limiter por token bucket, probabilidad, independencia, reintentos, determinismo, idempotencia, caché LFU, consistent hashing, event sourcing, motor de reglas, ventanas de tiempo, concurrencia modelada, condiciones de carrera, detección de deadlocks, grafo de espera, rate limiter de ventana deslizante, libro de órdenes, prioridad precio-tiempo, consistent hashing con réplicas virtuales, deduplicación, paradoja del cumpleaños, event sourcing con snapshots, circuit breaker, tolerancia a fallos, caché con TTL, ventanas múltiples, reglas ponderadas, lectores-escritores, sagas, compensaciones, valor esperado, libro de órdenes extendido, rebalanceo, cola de mensajes fallidos, cadenas de Markov, concurrencia, expiración, consistencia, orden de eventos....

Desafíos de nivel

- 398. Plataforma de pagos con reintentos y antifraude
- 399. La ticketera del recital masivo
- 400. Un exchange de criptomonedas en miniatura

Ejercicio 351: El catálogo que no entra en memoria

Nivel 8: Senior II · Algoritmos de sistemas · Temática: E-commerce · Dificultad 71/100

Situación: El catálogo de un e-commerce tiene 50 millones de productos y no entra en la memoria de los servidores web. Se guarda en una base de datos lenta y, delante de ella, un caché que conserva los productos consultados más recientemente. Cuando el caché se llena, se descarta el que hace más tiempo que nadie usa: la política LRU (menos usado recientemente).

Datos de entrada

Formato: primera línea: capacidad del caché; después operaciones "get id" (leer) y "put id valor" (guardar o actualizar).

Ejemplo

```
3
get P10
put P10 notebook
put P20 celular
get P10
put P30 tablet
put P40 monitor
get P20
get P30
put P10 notebook_v2
get P40
get P10
```

Restricciones: hasta 10^6 de capacidad y 10^8 operaciones por hora.

Tu programa debe

1. Combinar un mapa (id → nodo) con una lista doblemente enlazada ordenada por uso.
2. En un get con acierto, mover el nodo al final (más reciente); en un fallo, informar MISS.
3. En un put de un id existente, actualizar el valor y moverlo al final; si es nuevo y el caché está lleno, desalojar el primero de la lista.
4. Mostrar cada operación, el orden final y la tasa de aciertos de los get (1 decimal, 0,5 hacia arriba).

Salida esperada

```
get P10: MISS
put P10
put P20
get P10: HIT notebook
put P30
put P40 (desalojado P20)
get P20: MISS
get P30: HIT tablet
put P10
get P40: HIT monitor
get P10: HIT notebook_v2
Orden final (menos a más reciente): P30 P40 P10
Tasa de aciertos: 4/6 = 66.7 %
```

Reglas

- Complejidad esperada: $O(1)$ por operación. Buscar el menos usado recorriendo todo es $O(n)$.
- Un put también cuenta como uso del id.
- No funcional: memoria acotada por la capacidad (nunca más de "capacidad" elementos).
- No funcional: en un servidor con varios hilos, las operaciones deben ser atómicas; indicá en tu diseño dónde iría el bloqueo (o por qué convendría dividir el caché en segmentos con bloqueos independientes).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la lista doblemente enlazada permite sacar un nodo del medio en $O(1)$ si ya lo tenés; el mapa te da ese nodo en $O(1)$. Ninguna de las dos estructuras alcanza sola.

Bonus (opcional)

- Agregá métricas de observabilidad: aciertos, fallos y desalojos por minuto.
- Compará la tasa de aciertos de LRU con la de FIFO (desalojar el más viejo en entrar) sobre la misma secuencia.

Cierre: El caché LRU está en procesadores, navegadores, bases de datos y en casi cualquier servicio web de alto tráfico.

Ejercicio 352: Mil pedidos por segundo al procesador de pagos

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 71/100

Situación: Una fintech consume la API de un procesador de pagos que penaliza a quien le manda más pedidos de lo acordado. Para protegerse, la fintech limita su propio tráfico con un "balde de fichas": el balde tiene una capacidad máxima, se rellena a ritmo constante y cada pedido consume fichas. Si no hay fichas suficientes, el pedido se rechaza en el acto.

Datos de entrada

Formato: primera línea: capacidad del balde (fichas) y ritmo de recarga (fichas por segundo); después, un pedido por línea: "milisegundo costo_en_fichas" (en orden). El balde arranca lleno en $t = 0$.

Ejemplo

```
5 2
0 1
100 1
200 1
300 1
400 1
500 1
600 1
1500 3
1600 1
4000 5
4001 1
```

Restricciones: hasta 10^7 pedidos por segundo; tiempos en milisegundos.

Tu programa debe

1. Guardar las fichas en milésimas (enteros), para no acumular errores de decimales.
2. Ante cada pedido, recargar: $\text{fichas} = \text{mínimo}(\text{capacidad}, \text{fichas} + \text{milisegundos transcurridos} \times \text{ritmo})$ (en milésimas de ficha, $\text{ritmo fichas/s} = \text{ritmo milésimas/ms}$).
3. Si alcanzan las fichas, descontarlas y aceptar; si no, rechazar sin descontar.
4. Mostrar cada decisión con las fichas restantes (3 decimales) y el total aceptado.

Salida esperada

```
t=0 ms costo 1: ACEPTADO | fichas restantes 4.000
t=100 ms costo 1: ACEPTADO | fichas restantes 3.200
t=200 ms costo 1: ACEPTADO | fichas restantes 2.400
t=300 ms costo 1: ACEPTADO | fichas restantes 1.600
t=400 ms costo 1: ACEPTADO | fichas restantes 0.800
t=500 ms costo 1: ACEPTADO | fichas restantes 0.000
t=600 ms costo 1: RECHAZADO (429) | fichas restantes 0.200
t=1500 ms costo 3: RECHAZADO (429) | fichas restantes 2.000
t=1600 ms costo 1: ACEPTADO | fichas restantes 1.200
t=4000 ms costo 5: ACEPTADO | fichas restantes 0.000
t=4001 ms costo 1: RECHAZADO (429) | fichas restantes 0.002
Aceptados: 8 de 11
```

Reglas

- Complejidad esperada: $O(1)$ por pedido y $O(1)$ de memoria: no hace falta ningún temporizador, la recarga se calcula al llegar el pedido.
- No funcional: el cálculo debe ser determinista y exacto (enteros); con decimales binarios, después de millones de pedidos el balde "pierde" o "gana" fichas.
- No funcional: el limitador tiene que tomar la decisión en microsegundos, sin consultar servicios externos.

- Casos borde: pedido más caro que la capacidad (siempre se rechaza); ráfaga inicial (el balde lleno permite ráfagas de hasta "capacidad" pedidos).

Pista (leela solo si te trabás): no simules el paso del tiempo: al llegar un pedido, calculá cuántas fichas "se habrían recargado" desde el último, con un tope en la capacidad.

Bonus (opcional)

- Implementá el balde con goteo (leaky bucket), que suaviza el tráfico en lugar de permitir ráfagas, y compará.
- Llevá un balde por cliente con un mapa, y eliminá los baldes inactivos para no agotar la memoria.

Cierre: El balde de fichas es el algoritmo de limitación de tráfico más usado en APIs públicas, gateways y redes.

Ejercicio 353: Probabilidad de lluvia el fin de semana

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Clima y energía · Dificultad 71/100

Situación: Una app del clima muestra "60 % de probabilidad de lluvia el sábado y 50 % el domingo", y muchos usuarios concluyen que "el fin de semana llueve seguro, 110 %". El equipo quiere mostrar la probabilidad correcta de que llueva al menos un día, primero suponiendo días independientes y después con el dato real de que la lluvia de un día aumenta la del siguiente.

Datos de entrada

Formato: una línea por escenario: "p_sábado p_domingo" (independientes) o "p_sábado p_domingo p_domingo_si_llovió_el_sábado" (dependientes).

Ejemplo

```
0.6 0.5
0.3 0.3
0.6 0.5 0.8
0.1 0.1 0.1
```

Restricciones: probabilidades entre 0 y 1 con hasta 4 decimales; hasta 10^5 escenarios.

Tu programa debe

1. Días independientes: $P(\text{al menos uno}) = 1 - (1 - p_{\text{sáb}}) \times (1 - p_{\text{dom}})$; $P(\text{los dos}) = p_{\text{sáb}} \times p_{\text{dom}}$.
2. Días dependientes: $P(\text{los dos}) = p_{\text{sáb}} \times P(\text{dom} \mid \text{sáb})$ y $P(\text{al menos uno}) = p_{\text{sáb}} + p_{\text{dom}} - P(\text{los dos})$.
3. Verificar la consistencia de los datos dependientes: $P(\text{dom} \mid \text{no llovió el sáb}) = (p_{\text{dom}} - P(\text{los dos})) / (1 - p_{\text{sáb}})$ debe estar entre 0 y 1.
4. Mostrar los resultados con 4 decimales (0,5 hacia arriba) y, en el caso independiente, la "suma ingenua" para contrastar.

Salida esperada

```
Sáb 0.6, dom 0.5 (independientes): al menos un día 0.8000 | los dos 0.3000 | ninguno 0.2000 | suma
ingenua 1.1
Sáb 0.3, dom 0.3 (independientes): al menos un día 0.5100 | los dos 0.0900 | ninguno 0.4900 | suma
ingenua 0.6
Sáb 0.6, dom 0.5, dom si llovió el sáb 0.8: al menos un día 0.6200 | los dos 0.4800 | P(dom | no
llovió sáb) 0.0500
Sáb 0.1, dom 0.1, dom si llovió el sáb 0.1: al menos un día 0.1900 | los dos 0.0100 | P(dom | no
llovió sáb) 0.1000
```

Reglas

- Complejidad: $O(1)$ por escenario.
- No funcional: usá aritmética decimal exacta; al mostrar probabilidades al público, $0,1 + 0,2$ no puede dar $0,30000000000000000004$.
- No funcional: nunca mostrar una probabilidad fuera de $[0, 1]$: si los datos son inconsistentes, informarlo en lugar de calcular.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): es más fácil calcular la probabilidad de lo contrario: "no llueve ningún día". Y la suma de probabilidades solo vale para eventos que no pueden ocurrir juntos.

Bonus (opcional)

- Exténdelo a una semana de 7 días con probabilidades distintas (independientes).
- Con el modelo dependiente, calculá la probabilidad de que llueva el domingo sabiendo que no llovió el sábado y explicá cómo se lo comunicarías al usuario.

Cierre: Interpretar bien probabilidades combinadas es clave en meteorología, seguros y cualquier producto que comunique riesgos al público.

Ejercicio 354: Reintentos con espera exponencial

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Telecomunicaciones · Dificultad 72/100

Situación: Un sistema que envía notificaciones a los celulares falla a veces porque el servidor del operador está saturado. Reintentar enseguida empeora la saturación, así que se reintentan con espera exponencial (100 ms, 200 ms, 400 ms...) con un tope, y con una parte aleatoria ("jitter") para que miles de clientes no reintenten todos a la vez. Para poder testearlo, lo aleatorio tiene que ser reproducible con una semilla.

Datos de entrada

Formato: primera línea: espera base (ms), factor, tope (ms), máximo de intentos y semilla; después, una línea por envío con el resultado de cada intento (F = falla, T = éxito; si faltan resultados, se asume F).

Generador (el mismo para todos los envíos, en orden): $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$, arrancando en la semilla, un nuevo x por cada espera.

Espera antes del reintentado $k + 1$ (después de fallar el intento k): $\text{techo} = \min(\text{tope}, \text{base} \times \text{factor}^{(k - 1)})$; $\text{espera} = \text{techo} / 2 + (x \text{ resto } (\text{techo} / 2 + 1))$, con divisiones enteras.

Ejemplo

```
100 2 5000 6 42
F F F T
F F F F F F
T
```

Restricciones: hasta 10^6 envíos; máximo de intentos hasta 20.

Tu programa debe

1. Para cada envío, recorrer los intentos en orden hasta un éxito o hasta agotar el máximo.
2. Después de cada falla (que no sea la última), calcular el techo exponencial con tope, generar el siguiente x y calcular la espera con "jitter" en $[\text{techo} / 2, \text{techo}]$.
3. Acumular las esperas; si se agotan los intentos, mandar el envío a la cola de fallidos.
4. Mostrar el resultado de cada envío con sus esperas y el tiempo total esperado.

Salida esperada

```
Envío 1: éxito en el intento 4 | esperas 98 167 262 ms | total 527 ms
Envío 2: FALLÓ tras 6 intentos | esperas 93 189 383 681 1463 ms | total 2809 ms -> a la cola de fallidos
Envío 3: éxito en el intento 1 sin esperas
```

Reglas

- Complejidad: $O(\text{intentos})$ por envío.
- No funcional: determinismo; con la misma semilla y los mismos resultados, la salida tiene que ser idéntica en cualquier máquina y lenguaje (por eso el generador está especificado).
- No funcional: el tope evita esperas absurdas; el máximo de intentos evita reintentar para siempre.
- Casos borde: éxito en el primer intento (sin esperas).

Pista (leela solo si te trabás): la espera se "infla" multiplicando por el factor en cada falla, pero nunca pasa el tope; el jitter elige un valor al azar dentro de la mitad superior del techo.

Bonus (opcional)

- Compará el tráfico total si 1.000 clientes fallan a la vez con y sin jitter (contá cuántos reintentan en cada milisegundo).

- Respetá un encabezado "reintentar después de X ms" enviado por el servidor, si es mayor que la espera calculada.

Cierre: La espera exponencial con jitter es la política de reintentos recomendada por todos los grandes proveedores de servicios en la nube.

Ejercicio 355: Pagos que no se cobran dos veces

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 72/100

Situación: Un usuario toca "Pagar" en la app, la conexión se corta y la app reintenta: si el servidor procesa las dos veces, el cliente paga doble. Para evitarlo, la app manda con cada pago una clave de idempotencia; el servidor recuerda durante 24 horas el resultado de cada clave y, si la clave vuelve con los mismos datos, devuelve el mismo resultado sin cobrar otra vez.

Datos de entrada

Formato: una solicitud por línea: "hora clave tarjeta monto" (hora hh:mm:ss del día 1, o d:hh:mm:ss para el día d).

Ejemplo

```
08:00:00 K1 tarjeta_A 15000
08:00:02 K1 tarjeta_A 15000
08:01:00 K2 tarjeta_B 8000
08:01:05 K2 tarjeta_B 9000
08:02:00 K3 tarjeta_A 15000
2:08:00:03 K1 tarjeta_A 15000
```

Restricciones: hasta 10^5 solicitudes por segundo; claves únicas por cliente.

Tu programa debe

1. Guardar por clave: la hora del primer uso, los datos del pago (tarjeta y monto) y el resultado.
2. Si la clave existe y no venció (menos de 24 horas), comparar los datos: iguales → devolver el resultado guardado sin cobrar; distintos → error 422.
3. Si la clave no existe o venció, procesar el cobro, numerarlo y guardar el resultado.
4. Mostrar cada respuesta y el total efectivamente cobrado.

Salida esperada

```
K1: cobro #1 de $15000 a tarjeta_A
K1: repetido, se devuelve el resultado guardado (cobro #1 de $15000 a tarjeta_A); no se cobra de nuevo
K2: cobro #2 de $8000 a tarjeta_B
K2: ERROR 422, la clave ya se usó con otros datos
K3: cobro #3 de $15000 a tarjeta_A
K1: cobro #4 de $15000 a tarjeta_A
Total cobrado: $53000 en 4 cobros
```

Reglas

- Complejidad: $O(1)$ por solicitud con un mapa.
- No funcional: la verificación y el registro de la clave tienen que ser atómicos; si dos reintentos llegan a la vez, solo uno puede cobrar (en una base real, una restricción de unicidad sobre la clave).
- No funcional: las claves vencidas se eliminan para que la memoria no crezca sin límite.
- Una clave vence a las 24 horas exactas (86.400 segundos) de su primer uso.

Pista (leela solo si te trabás): la clave de idempotencia es un "recibo": si ya lo tenés, mostrás el mismo recibo en lugar de cobrar. Comparar los datos protege de un cliente que reutiliza una clave por error.

Bonus (opcional)

- Si el primer pedido todavía se está procesando cuando llega el reintento, respondé "en proceso" (409) en lugar de cobrar ni devolver nada.

- Guardá las claves con vencimiento en una estructura que permita limpiar las vencidas en orden (una cola por hora de uso).

Cierre: Las claves de idempotencia son un estándar de las APIs de pagos y de cualquier operación que no se puede repetir sin consecuencias.

Ejercicio 356: Los temas que siempre vuelven

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Música · Dificultad 72/100

Situación: Una app de música guarda en el celular los temas que el usuario más escucha, para reproducirlos sin conexión. A diferencia de LRU, acá conviene conservar los más escuchados en total, aunque uno no se haya escuchado ayer: la política LFU (menos frecuentemente usado). Si hay empate de frecuencia, se desaloja el que hace más tiempo que no se usa.

Datos de entrada

Formato: primera línea: capacidad; después "get tema" y "put tema" (agregar o usar).

Ejemplo

```
3
get tema1
put tema1
put tema2
put tema3
get tema1
get tema1
get tema2
put tema4
get tema3
get tema4
put tema5
get tema4
get tema2
```

Restricciones: capacidad hasta 10^5 y 10^7 operaciones.

Tu programa debe

1. Llevar la frecuencia de cada tema y, para cada frecuencia, una lista ordenada por uso reciente.
2. Mantener la frecuencia mínima actual para saber en $O(1)$ de qué lista desalojar.
3. En cada uso (get con acierto o put de un tema existente), pasar el tema de la lista f a la $f + 1$ y actualizar la mínima si esa lista quedó vacía.
4. Al insertar con el caché lleno, desalojar el más antiguo de la lista de frecuencia mínima; el nuevo arranca con frecuencia 1. Mostrar cada operación y el estado final (temas en orden alfabético con su frecuencia).

Salida esperada

```
get tema1: MISS
put tema1
put tema2
put tema3
get tema1: HIT (usos 2)
get tema1: HIT (usos 3)
get tema2: HIT (usos 2)
put tema4 (desalojado tema3)
get tema3: MISS
get tema4: HIT (usos 2)
put tema5 (desalojado tema2)
get tema4: HIT (usos 3)
get tema2: MISS
Final: tema1 (3), tema4 (3), tema5 (1)
```

Reglas

- Complejidad esperada: $O(1)$ por operación.
- Un put de un tema nuevo cuenta como su primer uso (frecuencia 1).
- No funcional: la memoria extra por tema tiene que ser constante (sin guardar el historial de usos).

- No funcional: LFU puro nunca "olvida": un tema muy escuchado hace años puede ocupar lugar para siempre; describí una estrategia de envejecimiento (por ejemplo, dividir todas las frecuencias por 2 cada semana).

Pista (leela solo si te trabás): la frecuencia mínima solo puede cambiar de dos formas: se vuelve 1 cuando entra un tema nuevo, o sube en 1 cuando su última lista queda vacía. Eso evita buscarla.

Bonus (opcional)

- Implementá el envejecimiento de frecuencias y compará los aciertos con LFU puro en una secuencia con cambios de gustos.
- Combiná LRU y LFU con una política adaptativa (investigá ARC).

Cierre: El caché LFU se usa en CDNs, cachés de consultas y sistemas donde la popularidad es estable en el tiempo.

Ejercicio 357: Jugadores que no cambian de servidor

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Videojuegos · Dificultad 72/100

Situación: Un juego online reparte a los jugadores entre servidores regionales y guarda en cada servidor el estado de las partidas de sus jugadores. Si se asigna con "hash resto cantidad de servidores", cuando un servidor se cae casi todos los jugadores cambian de servidor y pierden su sesión. Con hashing consistente (un anillo), solo se mudan los jugadores del servidor caído.

Datos de entrada

Formato: primera línea: servidores; segunda: jugadores; una línea "?" y la operación "quitar servidor".

Hash (FNV-1a con mezclado): FNV-1a de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Anillo: cada servidor ocupa la posición de su hash; un jugador va al primer servidor con posición $\geq$ su hash, y si no hay ninguno, al de menor posición (el anillo da la vuelta).

Ejemplo

```
servidor_ba servidor_cba servidor_mza servidor_ros
jugador_ana jugador_beto jugador_caro jugador_dani jugador_eli jugador_fede jugador_gabi
jugador_hugo
?
quitar servidor_cba
```

Restricciones: hasta 10^4 servidores y 10^8 jugadores.

Tu programa debe

1. Calcular el hash de cada servidor y ordenarlos por posición en el anillo.
2. Asignar cada jugador al primer servidor en sentido horario desde su hash (con búsqueda binaria sobre las posiciones).
3. Quitar el servidor indicado y reasignar.
4. Mostrar el anillo, la asignación antes y después, y cuántos jugadores se mudan con el anillo y con "hash resto n " (usando el orden de la entrada como índices).

Salida esperada

```
Anillo: servidor_mza@460904125, servidor_ba@1592788620, servidor_cba@2460365406,
servidor_ros@3033652522
jugador_ana (hash 2287367690): servidor_cba -> servidor_ros
jugador_beto (hash 814827495): servidor_ba -> servidor_ba
jugador_caro (hash 3354049126): servidor_mza -> servidor_mza
jugador_dani (hash 3853186875): servidor_mza -> servidor_mza
jugador_eli (hash 3251833503): servidor_mza -> servidor_mza
jugador_fede (hash 23944881): servidor_mza -> servidor_mza
jugador_gabi (hash 1306253260): servidor_ba -> servidor_ba
jugador_hugo (hash 4130227179): servidor_mza -> servidor_mza
Se mudan con el anillo: 1 de 8
Se mudarían con hash resto n: 7 de 8
```

Reglas

- Complejidad esperada: $O(\log S)$ por jugador, con S servidores.
- No funcional: estabilidad; al quitar un servidor, solo pueden cambiar de destino sus propios jugadores.
- No funcional: todos los nodos del sistema deben calcular la misma asignación sin coordinarse: por eso el hash está especificado al bit.
- Casos borde: hash de jugador mayor que todos los servidores (vuelta del anillo).

Pista (leela solo si te trabás): en el anillo, cada servidor es "dueño" del arco que termina en él. Si un servidor desaparece, su arco pasa entero al siguiente; los demás arcos no cambian.

Bonus (opcional)

- Con pocos servidores el reparto queda desparejo: agregá nodos virtuales (ejercicio de los servidores nuevos).
- Guardá cada jugador también en el siguiente servidor del anillo como réplica.

Cierre: El hashing consistente es la base de las bases de datos distribuidas, los cachés distribuidos y los balanceadores de carga.

Ejercicio 358: Reconstruir el carrito desde sus eventos

Nivel 8: Senior II · Algoritmos de sistemas · Temática: E-commerce · Dificultad 72/100

Situación: El carrito de un e-commerce no se guarda como "estado actual" sino como la lista de eventos que le pasaron (se agregó, se cambió la cantidad, se aplicó un cupón...). Eso permite reconstruir el carrito en cualquier momento, auditar reclamos ("yo tenía el cupón aplicado") y analizar el comportamiento de compra. El estado se obtiene reproduciendo los eventos en orden.

Datos de entrada

Formato: eventos "versión tipo datos": agregado producto cantidad precio, cantidad producto nueva_cantidad (0 = quitar), quitado producto, cupon código porcentaje; una línea "?" y las versiones a reconstruir.

Ejemplo

```
1 agregado yerba 2 3500
2 agregado mate 1 12000
3 agregado bombilla 1 4500
4 cantidad yerba 4
5 quitado bombilla
6 cupon DESC10 10
7 agregado termo 1 28000
8 cantidad mate 0
?
3
6
8
```

Restricciones: hasta 10^4 eventos por carrito y 10^7 carritos.

Tu programa debe

1. Recorrer los eventos en orden de versión hasta la versión pedida, aplicando cada uno sobre un estado vacío.
2. Mantener los productos en el orden en que se agregaron.
3. Calcular el subtotal, el descuento del cupón (porcentaje entero, redondeo hacia abajo al peso) y el total.
4. Mostrar el estado de cada versión pedida.

Salida esperada

```
Versión 3: yerba x2, mate x1, bombilla x1 | subtotal $23500 | total $23500
Versión 6: yerba x4, mate x1 | subtotal $26000 | DESC10 -$2600 | total $23400
Versión 8: yerba x4, termo x1 | subtotal $42000 | DESC10 -$4200 | total $37800
```

Reglas

- Complejidad: O(eventos hasta la versión) por reconstrucción.
- No funcional: los eventos son inmutables: nunca se modifican ni se borran, solo se agregan nuevos (una corrección es un evento nuevo).
- No funcional: reproducir la misma lista de eventos tiene que dar siempre el mismo estado (la función que aplica eventos no puede depender de la hora ni de datos externos).
- Casos borde: cambiar la cantidad a 0 equivale a quitar el producto.

Pista (leela solo si te trabás): el estado es una función pura de los eventos: $\text{estado}(v) = \text{aplicar}(\text{evento } v, \text{estado}(v - 1))$. Si esa función es determinista, podés viajar en el tiempo.

Bonus (opcional)

- Para carritos con muchos eventos, guardá una foto del estado cada 100 eventos (ver el ejercicio del saldo de diez años).
- Proyectá los eventos de todos los carritos a una métrica: productos más abandonados (agregados y después quitados).

Cierre: El event sourcing se usa en bancos, e-commerce y sistemas de auditoría, donde saber cómo se llegó al estado vale tanto como el estado.

Ejercicio 359: Reglas antifraude de tarjetas

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 72/100

Situación: Un emisor de tarjetas evalúa cada compra en tiempo real con un motor de reglas antifraude antes de autorizarla. Cada regla puede pedir "revisar" o "bloquear", y la decisión final es la más severa. Algunas reglas miran la compra sola y otras miran el historial reciente de la tarjeta (ventanas de tiempo).

Datos de entrada

Formato: una compra por línea (en orden): "hh:mm id tarjeta monto país".

Reglas: (1) monto mayor a \$500.000 → REVISAR; (2) con esta compra, 4 o más compras de la tarjeta en menos de 10 minutos (las anteriores con diferencia < 10) → BLOQUEAR; (3) alguna compra de la tarjeta en otro país hace menos de 60 minutos → REVISAR.

Ejemplo

```
10:00 T1 4111 25000 AR
10:03 T2 4111 8000 AR
10:05 T3 4111 12000 AR
10:07 T4 4111 5000 AR
10:08 T5 5500 950000 AR
10:30 T6 4111 7000 BR
11:50 T7 4111 3000 AR
12:00 T8 5500 20000 UY
```

Restricciones: hasta 10^5 compras por segundo; historial por tarjeta de hasta 24 horas.

Tu programa debe

1. Guardar por tarjeta el historial de compras (hora y país), incluidas las bloqueadas.
2. Evaluar las tres reglas para cada compra, usando solo el historial dentro de la ventana correspondiente.
3. Decidir: BLOQUEAR si alguna regla bloquea; si no, REVISAR si alguna revisa; si no, APROBAR.
4. Mostrar la decisión y las reglas activadas, en el orden en que están definidas.

Salida esperada

```
T1: APROBAR
T2: APROBAR
T3: APROBAR
T4: BLOQUEAR (4 compras en 10 min)
T5: REVISAR (monto alto)
T6: REVISAR (cambio de país en menos de 60 min)
T7: APROBAR
T8: APROBAR
```

Reglas

- Complejidad: $O(\text{compras dentro de la ventana más larga})$ por compra; con colas por tarjeta se descartan las viejas en $O(1)$ amortizado.
- No funcional: la decisión tiene que tomarse en menos de 50 ms, así que el historial vive en memoria y se poda.
- No funcional: cada decisión debe ser explicable (se registran las reglas que la causaron), por auditoría y por reclamos de clientes.
- Las ventanas son estrictas: una compra de hace exactamente 10 minutos no cuenta para la regla 2.

Pista (leela solo si te trabás): cada regla es una función independiente que recibe la compra y el historial y devuelve "nada", "revisar" o "bloquear". El motor solo las recorre y se queda con la más severa.

Bonus (opcional)

- Cargá las reglas desde un archivo de configuración (umbral, ventana y acción) sin cambiar el código.
- Agregá el modo "sombra": una regla nueva se evalúa y registra, pero todavía no afecta la decisión.

Cierre: Los motores de reglas con ventanas de tiempo son el primer filtro antifraude de bancos, billeteras y procesadores de pagos.

Ejercicio 360: Entradas vendidas de más

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Deportes · Dificultad 73/100

Situación: En la venta de entradas de un clásico, dos servidores atienden compras al mismo tiempo. Cada uno lee el stock, verifica que alcance y escribe el stock nuevo. Si las operaciones se intercalan mal, los dos leen el mismo stock y venden entradas que no existen. Para entenderlo sin depender de la suerte, se modela el intercalado paso a paso.

Datos de entrada

Formato: primera línea: stock inicial; después, un pedido por vendedor ("vendedor cantidad") y, a continuación, el orden de los pasos: "vendedor leer" y "vendedor escribir".

Sin control: escribir verifica con el valor leído y escribe (valor leído – cantidad). Con compare-and-set: escribir solo tiene éxito si el stock sigue siendo el valor leído; si no, el vendedor relea al instante y vuelve a intentar dentro del mismo paso.

Ejemplo

```
3
V1 2
V2 2
V1 leer
V2 leer
V1 escribir
V2 escribir
```

Restricciones: hasta 100 vendedores y 10^4 pasos en la simulación.

Tu programa debe

1. Simular el intercalado sin control: cada vendedor guarda lo que leyó y escribe basándose en eso.
2. Simular el mismo intercalado con compare-and-set (CAS), que detecta que otro escribió en el medio.
3. Contar las entradas vendidas y el stock final en cada modo, y marcar la sobreventa.
4. Mostrar el registro de cada modo.

Salida esperada

```
[sin control] V1 lee stock 3; V2 lee stock 3; V1 vende 2 y escribe 1; V2 vende 2 y escribe 1 |
vendas 4 de 3, stock final 1 -> SOBREVENTA
[con compare-and-set] V1 lee stock 3; V2 lee stock 3; V1 CAS(3 -> 1) ok, vende 2; V2 CAS(3 -> 1)
falla: el stock es 1, relea; V2 rechaza (stock 1) | vendas 2 de 3, stock final 1
```

Reglas

- Complejidad: $O(\text{pasos})$.
- No funcional: la simulación tiene que ser determinista: el problema de concurrencia se estudia con el intercalado explícito, no con hilos reales que "a veces" fallan.
- No funcional: en producción, la operación leer-verificar-escribir debe ser atómica (una transacción, un bloqueo o un CAS en la base de datos).
- Casos borde: un vendedor escribe sin que nadie haya escrito en el medio (el CAS tiene éxito a la primera).

Pista (leela solo si te trabas): el error no está en ningún vendedor por separado: está en el hueco entre leer y escribir. El CAS dice "escribí esto solo si nadie tocó el valor desde que lo leí".

Bonus (opcional)

- Probá todos los intercalados posibles de dos vendedores (son 6) y contá en cuántos hay sobreventa.

- Implementá la alternativa con un bloqueo y compará cuántos pasos "espera" cada vendedor.

Cierre: Las condiciones de carrera son una de las causas más comunes de errores graves en ticketeras, bancos y sistemas de stock.

Ejercicio 361: Deadlock en la reserva de camas

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Salud · Dificultad 73/100

Situación: El sistema de gestión de un hospital reserva recursos (camas, monitores, sillas de traslado) para distintos procesos. Un proceso que tiene un recurso y espera otro puede quedar trabado para siempre si otro proceso espera lo que él tiene: un deadlock. El sistema tiene que detectarlo a partir de las asignaciones y esperas, y sugerir a quién cancelar.

Datos de entrada

Formato: una línea por hecho: "asigna recurso proceso" (el proceso tiene el recurso) o "espera proceso recurso" (el proceso espera el recurso).

Ejemplo

```
asigna cama_12 admision_1
asigna monitor_3 enfermeria_2
asigna silla_7 traslados_3
espera admision_1 monitor_3
espera enfermeria_2 silla_7
espera traslados_3 cama_12
asigna cama_20 admision_4
espera admision_4 cama_12
```

Restricciones: hasta 10^5 procesos y recursos; cada recurso tiene un solo dueño y cada proceso espera como máximo un recurso.

Tu programa debe

1. Armar el grafo de espera: el proceso P apunta al proceso Q si P espera un recurso que tiene Q.
2. Como cada proceso espera un solo recurso, cada nodo tiene como máximo una flecha saliente: seguir las flechas desde cada nodo no visitado detecta los ciclos.
3. Para cada ciclo, sugerir como víctima el proceso de menor id (alfabéticamente) e informar qué recursos libera.
4. Mostrar el grafo, los ciclos y los procesos bloqueados que no son parte del ciclo.

Salida esperada

```
Grafo de espera: admision_1 -> enfermeria_2, enfermeria_2 -> traslados_3, traslados_3 ->
    admision_1, admision_4 -> admision_1
DEADLOCK: admision_1 -> enfermeria_2 -> traslados_3 -> admision_1
Víctima sugerida (menor id): admision_1, libera cama_12
Bloqueados por el ciclo (sin ser parte): admision_4
```

Reglas

- Complejidad esperada: $O(\text{procesos} + \text{recursos})$.
- Con varias esperas por proceso, el grafo general necesita DFS con colores para detectar ciclos.
- No funcional: el detector corre periódicamente en segundo plano y no puede bloquear las reservas mientras analiza (trabaja sobre una foto del estado).
- No funcional: la elección de la víctima tiene que ser determinista, para que dos corridas del detector no cancelen procesos distintos.

Pista (leela solo si te trabás): un deadlock es un ciclo en el grafo "quién espera a quién". Si cada proceso espera a lo sumo a uno, seguir las flechas es como recorrer una lista: o termina, o vuelve a un nodo ya visitado en el mismo recorrido.

Bonus (opcional)

- Prevenir los deadlocks en lugar de detectarlos: imponé un orden global de recursos (ver el ejercicio de los cocineros).

- Soportá recursos con varias unidades (el algoritmo del banquero).

Cierre: La detección de deadlocks es parte de los motores de bases de datos, los sistemas operativos y los orquestadores de recursos.

Ejercicio 362: El bot que prueba contraseñas de a poco

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Seguridad informática · Dificultad 73/100

Situación: El login de un home banking limita los intentos por usuario para frenar bots que prueban contraseñas. Un bot astuto aprendió a aprovechar el límite por "ventana fija" (3 intentos por minuto calendario): hace 3 al final de un minuto y 3 al principio del siguiente. La ventana deslizante (3 intentos en los últimos 60 segundos, en cualquier momento) cierra ese hueco.

Datos de entrada

Formato: primera línea: máximo de intentos N y ventana W en segundos; después, "usuario segundo" por intento (en orden).

Ventana deslizante: se permite si hubo menos de N intentos permitidos en $(t - W, t]$. Ventana fija: se permite si hubo menos de N intentos permitidos en el mismo bloque t / W (división entera).

Ejemplo

```
3 60
usuario_x 10
usuario_x 50
usuario_x 55
usuario_x 61
usuario_x 65
usuario_x 118
usuario_x 119
usuario_x 125
usuario_y 20
```

Restricciones: hasta 10^6 intentos por segundo y 10^7 usuarios.

Tu programa debe

1. Para la ventana deslizante, guardar por usuario una cola con los instantes de sus intentos permitidos; descartar los que quedaron fuera de la ventana.
2. Permitir si la cola tiene menos de N elementos (y agregar el instante); si no, bloquear.
3. Para la ventana fija, guardar por usuario el bloque actual y su contador.
4. Mostrar ambas decisiones para cada intento y el total de bloqueos de cada método.

Salida esperada

```
usuario_x t=10: deslizante permitido | fija permitido
usuario_x t=50: deslizante permitido | fija permitido
usuario_x t=55: deslizante permitido | fija permitido
usuario_x t=61: deslizante BLOQUEADO | fija permitido
usuario_x t=65: deslizante BLOQUEADO | fija permitido
usuario_x t=118: deslizante permitido | fija permitido
usuario_x t=119: deslizante permitido | fija BLOQUEADO
usuario_x t=125: deslizante permitido | fija permitido
usuario_y t=20: deslizante permitido | fija permitido
Bloqueos: ventana deslizante 2, ventana fija 1
```

Reglas

- Complejidad: $O(1)$ amortizado por intento en ambos métodos.
- Los intentos bloqueados no se registran en la ventana (si no, un usuario bloqueado nunca saldría).
- No funcional: memoria acotada: la cola de cada usuario tiene como máximo N elementos, y los usuarios inactivos se eliminan.
- No funcional: en un sistema con varios servidores, el conteo tiene que ser compartido (por ejemplo, en un almacenamiento en memoria centralizado); si no, el bot reparte los intentos entre servidores.

Pista (leela solo si te trabás): la ventana fija se "reinicia" en el borde del minuto sin importar lo que pasó un segundo antes; la deslizante siempre mira exactamente los últimos W segundos.

Bonus (opcional)

- Implementá la aproximación de ventana deslizante con dos contadores (bloque actual y anterior ponderado) y medí el error frente a la exacta.
- Agregá un bloqueo progresivo: después de 3 bloqueos, el usuario queda 15 minutos sin poder intentar.

Cierre: Los limitadores por ventana deslizante protegen logins, APIs y formularios contra ataques automatizados de fuerza bruta.

Ejercicio 363: Motor de matching de órdenes de compra y venta

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 73/100

Situación: Un mercado de valores electrónico recibe órdenes de compra y de venta con precio límite. El motor de matching cruza cada orden nueva con las del lado contrario que tengan mejor precio y, entre las de igual precio, con la que llegó primero (prioridad precio-tiempo). Lo que no se ejecuta queda esperando en el libro de órdenes.

Datos de entrada

Formato: una orden por línea, en orden de llegada: "id compra|venta precio cantidad" (precio límite en pesos enteros).

Ejemplo

```
01 compra 100 10
02 compra 101 5
03 venta 103 8
04 venta 102 4
05 venta 101 7
06 compra 103 10
07 venta 99 12
```

Restricciones: hasta 10^6 órdenes por segundo; hasta 10^5 niveles de precio.

Tu programa debe

1. Mantener dos colas de prioridad: compras (mayor precio primero) y ventas (menor precio primero); en empate de precio, la más antigua primero.
2. Una compra nueva se ejecuta contra las ventas mientras la mejor venta tenga precio $\leq$ su límite; una venta nueva, contra las compras con precio $\geq$ su límite.
3. Cada operación se hace al precio de la orden que ya estaba en el libro, por la cantidad mínima entre las dos; lo que queda de la orden nueva se agrega al libro.
4. Mostrar las operaciones y el libro final (cada lado ordenado por prioridad) en formato id cantidad@precio.

Salida esperada

```
Operaciones:
05 vende 5 @ 101 (contra 02)
06 compra 2 @ 101 (contra 05)
06 compra 4 @ 102 (contra 04)
06 compra 4 @ 103 (contra 03)
07 vende 10 @ 100 (contra 01)
Compras en el libro: ninguna
Ventas en el libro: 07 2@99, 03 4@103
```

Reglas

- Complejidad esperada: $O(\log n)$ por orden más $O(1)$ por operación ejecutada.
- No funcional: equidad: la prioridad precio-tiempo tiene que respetarse siempre; es un requisito regulatorio.
- No funcional: determinismo: procesar la misma secuencia de órdenes tiene que dar exactamente las mismas operaciones (es lo que permite auditar y reconstruir el mercado).
- Casos borde: una orden que barre varios niveles de precio; una orden que no cruza nada.

Pista (leela solo si te trabás): el precio de la operación lo pone la orden que estaba esperando ("el libro"): quien llega y cruza acepta el precio que ya estaba ofrecido.

Bonus (opcional)

- Agregá órdenes de mercado (sin precio límite: cruzan contra lo que haya).
- Mostrá el libro agregado por nivel de precio (cantidad total en cada precio), como lo ven los operadores.

Cierre: Los motores de matching de las bolsas y exchanges son uno de los sistemas más exigentes en latencia y correctitud del mundo.

Ejercicio 364: Servidores nuevos sin mudanza masiva

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Telecomunicaciones · Dificultad 73/100

Situación: Una operadora distribuye los datos de sus clientes entre servidores con un anillo de hashing consistente. Con un solo punto por servidor, el reparto queda muy despaseado (un servidor puede cargar el doble que otro). La solución es que cada servidor ocupe muchos puntos del anillo ("nodos virtuales"). El equipo quiere medir cuánto mejora el reparto y cuántas claves se mueven al agregar un servidor.

Datos de entrada

Formato: primera línea: servidores; segunda: cantidades de nodos virtuales a probar; tercera: cantidad de claves (se llaman clave1, clave2...); una línea "?" y el servidor nuevo.

Hash (FNV-1a con mezclado): FNV-1a de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$. El nodo virtual i del servidor s se llama " $s\#i$ " (i desde 0). Cada clave va al primer punto con $\text{hash} \geq$ al suyo (con vuelta del anillo).

Ejemplo

```
s1 s2 s3
1 10 100
10000
?
s4
```

Restricciones: hasta 10^4 servidores, 1.000 nodos virtuales por servidor y 10^8 claves.

Tu programa debe

1. Construir el anillo con V puntos por servidor, ordenados por hash.
2. Asignar cada clave con búsqueda binaria sobre el anillo y contar las claves por servidor.
3. Agregar el servidor nuevo, reasignar y contar cuántas claves cambiaron de servidor (y verificar que ninguna se mudó entre servidores viejos).
4. Mostrar, para cada V , las cargas, el cociente máximo/ideal (2 decimales) y el porcentaje de claves movidas (1 decimal).

Salida esperada

```
1 nodo(s) virtual(es) por servidor:
antes: s1=4898 s2=991 s3=4111 | máx/ideal = 1.47
después: s1=3055 s2=991 s3=4111 s4=1843 | máx/ideal = 1.64
claves movidas: 1843 (18.4 %), movidas a un servidor viejo: 0
10 nodo(s) virtual(es) por servidor:
antes: s1=2788 s2=2897 s3=4315 | máx/ideal = 1.29
después: s1=2585 s2=2255 s3=2476 s4=2684 | máx/ideal = 1.07
claves movidas: 2684 (26.8 %), movidas a un servidor viejo: 0
100 nodo(s) virtual(es) por servidor:
antes: s1=2840 s2=3393 s3=3767 | máx/ideal = 1.13
después: s1=2179 s2=2365 s3=2579 s4=2877 | máx/ideal = 1.15
claves movidas: 2877 (28.8 %), movidas a un servidor viejo: 0
```

Reglas

- Complejidad: $O(S \times V \times \log(S \times V))$ para armar el anillo y $O(\log(S \times V))$ por clave.
- No funcional: balance de carga: con suficientes nodos virtuales, el servidor más cargado debería estar cerca de 1,0 veces la carga ideal.
- No funcional: movimiento mínimo: al agregar un servidor, solo deberían moverse claves hacia el servidor nuevo (idealmente, $1 / (S + 1)$ de ellas).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): con un solo punto, el tamaño del arco de cada servidor es muy variable. Con muchos puntos al azar, cada servidor tiene muchos arcos chicos cuya suma se parece mucho al promedio.

Bonus (opcional)

- Dale más nodos virtuales a los servidores más potentes para repartir según capacidad.
- Compará con "rendezvous hashing" (ejercicio de las particiones de usuarios).

Cierre: Los nodos virtuales son el ajuste que hace práctico al hashing consistente en sistemas como bases de datos distribuidas y cachés.

Ejercicio 365: Mensajes entregados al menos una vez

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Logística y envíos · Dificultad 74/100

Situación: El sistema de tracking de una empresa de envíos recibe eventos ("paquete escaneado en sucursal") a través de una cola de mensajes que garantiza entrega "al menos una vez": ningún mensaje se pierde, pero algunos llegan repetidos. El consumidor tiene que descartar los duplicados. Recordar todos los ids para siempre no entra en memoria, así que se usa una ventana de ids recientes.

Datos de entrada

Formato: primera línea: tamaño de la ventana de deduplicación N ; segunda: los ids de los mensajes en orden de llegada.

Ejemplo

```
3
M1 M2 M1 M3 M4 M2 M5 M6 M1 M7 M3
```

Restricciones: hasta 10^6 mensajes por segundo; los duplicados llegan, como máximo, a una distancia conocida del original.

Tu programa debe

1. Deduplicar sin límite: un conjunto con todos los ids vistos.
2. Deduplicar con ventana: además del conjunto, una cola con los últimos N ids procesados; al superar N , sacar el más viejo del conjunto.
3. Contar los descartados y los procesados dos veces en cada caso.
4. Mostrar la secuencia procesada de cada estrategia.

Salida esperada

```
[sin límite] procesados: M1 M2 M3 M4 M5 M6 M7 | descartados 4 | procesados dos veces 0
[ventana de 3] procesados: M1 M2 M3 M4 M5 M6 M1 M7 M3 | descartados 2 | procesados dos veces 2
```

Reglas

- Complejidad: $O(1)$ por mensaje en ambos casos.
- No funcional: la memoria de la ventana es $O(N)$; tiene que dimensionarse según la distancia máxima entre un mensaje y su duplicado.
- No funcional: si un duplicado puede escaparse, el procesamiento tiene que ser idempotente (procesar dos veces no debe romper nada).
- Casos borde: un duplicado que llega después de salir de la ventana se procesa de nuevo.

Pista (leela solo si te trabás): la ventana es un compromiso: menos memoria a cambio de no detectar duplicados muy tardíos. Comparar las dos corridas muestra exactamente qué se pierde.

Bonus (opcional)

- Reemplazá el conjunto por un filtro de Bloom (nivel 10) y medí los falsos positivos.
- Usá una ventana por tiempo (ids de los últimos 10 minutos) en lugar de por cantidad.

Cierre: La entrega "al menos una vez" más deduplicación es el patrón estándar para lograr "exactamente una vez" en sistemas distribuidos.

Ejercicio 366: ¿Cuándo chocan dos códigos cortos?

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Seguridad informática · Dificultad 74/100

Situación: Un acertador de links genera códigos al azar (como "a7Xk") y el equipo de seguridad quiere saber cuándo empiezan a repetirse: un choque hace que un link lleve al destino equivocado. La intuición falla: con 1,6 millones de códigos posibles, el 50 % de probabilidad de choque llega con menos de 1.500 códigos. Es la paradoja del cumpleaños.

Datos de entrada

Formato: una línea por escenario: "tamaño_del_alfabeto largo cantidad_de_códigos_generados".

Ejemplo

```
36 4 1000
36 6 1000
36 6 100000
62 8 1000000
```

Restricciones: alfabetos hasta 62 símbolos, largos hasta 12, hasta 10^7 códigos.

Tu programa debe

1. Calcular la cantidad total de códigos $N = \text{alfabeto}^{\text{largo}}$.
2. La probabilidad de que n códigos sean todos distintos es el producto de $(1 - i / N)$ para i de 0 a $n - 1$; la de choque es 1 menos eso.
3. Para no perder precisión, sumar logaritmos ($\log(1 - x)$ calculado de forma precisa para x chico) y exponenciar al final.
4. Mostrar la probabilidad de choque (4 decimales) y la cantidad aproximada de códigos para llegar al 50 %: $\lceil \sqrt{2 \times N \times \ln 2} \rceil$.

Salida esperada

```
Alfabeto 36, largo 4 (1679616 códigos): con 1000 códigos P(choque) = 0.2573 | 50 % se alcanza con
unos 1526 códigos
Alfabeto 36, largo 6 (2176782336 códigos): con 1000 códigos P(choque) = 0.0002 | 50 % se alcanza
con unos 54934 códigos
Alfabeto 36, largo 6 (2176782336 códigos): con 100000 códigos P(choque) = 0.8994 | 50 % se alcanza
con unos 54934 códigos
Alfabeto 62, largo 8 (218340105584896 códigos): con 1000000 códigos P(choque) = 0.0023 | 50 % se
alcanza con unos 17397807 códigos
```

Reglas

- Complejidad: $O(n)$ por escenario (o $O(1)$ con la aproximación $1 - e^{(-n^2/2N)}$).
- No funcional: estabilidad numérica: multiplicar un millón de factores cercanos a 1 acumula error; los logaritmos lo evitan.
- No funcional: en producción, un acertador no puede confiar solo en la probabilidad: tiene que verificar unicidad al insertar.
- Las probabilidades se muestran con 4 decimales y punto decimal.

Pista (leela solo si te trabás): la cantidad de pares posibles crece con n^2 , no con n : con 1.500 códigos hay más de un millón de pares, y cada par tiene una chance de chocar.

Bonus (opcional)

- Calculá el largo mínimo del código para que, con 10 millones de links, la probabilidad de choque sea menor a 0,0001.
- Verificalo por simulación con un generador con semilla y compará con el cálculo exacto.

Cierre: La paradoja del cumpleaños define el tamaño de los identificadores, los hashes y los códigos de todos los sistemas que generan valores al azar.

Ejercicio 367: El saldo de diez años en un segundo

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 74/100

Situación: Una cuenta de inversión guarda como eventos todos sus movimientos de los últimos diez años (event sourcing). Reconstruir el saldo reproduciendo miles de eventos en cada consulta hace lenta la app. La solución es guardar cada tanto una "foto" (snapshot) del estado y reproducir solo los eventos posteriores a la última foto.

Datos de entrada

Formato: primera línea: cantidad de eventos y cada cuántos se guarda una foto; después, las versiones a consultar.

Evento i (de 1 a n): movimiento = $(i \times 7919) \text{ resto } 20001 - 10000$ pesos (positivo es depósito, negativo es extracción).

Ejemplo

```
3650 500
3650
1
1234
2000
3001
```

Restricciones: hasta 10^7 eventos por cuenta; consultas de cualquier versión.

Tu programa debe

1. Generar los eventos con la fórmula y reproducirlos una vez, guardando una foto del saldo cada K eventos.
2. Para consultar la versión v , tomar la foto de la versión $\lfloor v / K \rfloor \times K$ y reproducir solo los eventos posteriores hasta v .
3. Contar cuántos eventos se reprodujeron en cada consulta.
4. Mostrar el saldo de cada versión y los eventos reproducidos contra los que harían falta sin fotos.

Salida esperada

```
Fotos guardadas: 7 (cada 500 eventos)
Versión 3650: saldo $34639 | eventos reproducidos 150 (sin fotos serían 3650)
Versión 1: saldo -$2081 | eventos reproducidos 1 (sin fotos serían 1)
Versión 1234: saldo -$2675 | eventos reproducidos 234 (sin fotos serían 1234)
Versión 2000: saldo $27745 | eventos reproducidos 0 (sin fotos serían 2000)
Versión 3001: saldo $11452 | eventos reproducidos 1 (sin fotos serían 3001)
```

Reglas

- Complejidad: $O(K)$ por consulta en lugar de $O(v)$.
- No funcional: las fotos son una optimización: si se pierden, el estado tiene que poder reconstruirse igual desde los eventos (la fuente de verdad son los eventos).
- No funcional: elegir K es un compromiso entre espacio (más fotos) y latencia de consulta (menos eventos a reproducir).
- La versión 0 es el estado inicial (saldo 0).

Pista (leela solo si te trabás): la foto de la versión 3.000 ya "contiene" los primeros 3.000 eventos: para la versión 3.001 alcanza con uno más.

Bonus (opcional)

- Verificá la integridad: recalculá el saldo desde cero y comparalo con cada foto.

- Si cambia la forma de los eventos (una nueva versión del esquema), definí cómo migrar las fotos viejas.

Cierre: Las fotos periódicas son lo que hace viable el event sourcing en sistemas con historias largas, como bancos y plataformas de inversión.

Ejercicio 368: El proveedor de envíos que se cae

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Logística y envíos · Dificultad 74/100

Situación: Un e-commerce cotiza los envíos con un proveedor externo que a veces se cae. Mientras está caído, cada llamada tarda 30 segundos en fallar y bloquea el checkout. Un "circuit breaker" corta las llamadas después de varias fallas seguidas, responde con un valor de respaldo y, pasado un tiempo, prueba con una sola llamada si el proveedor volvió.

Datos de entrada

Formato: primera línea: fallas consecutivas para abrir (F) y segundos de espera (T); después, "segundo resultado" para cada pedido (el resultado que daría el proveedor si se lo llamara).

Ejemplo

```
3 30
0 ok
5 falla
8 falla
10 falla
12 ok
25 ok
41 falla
42 ok
75 ok
76 ok
```

Restricciones: hasta 10^4 pedidos por segundo.

Tu programa debe

1. CERRADO: llamar al proveedor; un ok reinicia el contador de fallas; al llegar a F fallas seguidas, pasar a ABIERTO y anotar la hora.
2. ABIERTO: no llamar; responder con el respaldo. Si pasaron T segundos o más desde la apertura, el próximo pedido pasa a SEMIABIERTO.
3. SEMIABIERTO: hacer una llamada de prueba; si sale bien, pasar a CERRADO con el contador en 0; si falla, volver a ABIERTO.
4. Mostrar el estado y la acción de cada pedido.

Salida esperada

```
t=0: CERRADO, ok
t=5: CERRADO, falla 1
t=8: CERRADO, falla 2
t=10: CERRADO, falla 3 -> ABIERTO
t=12: ABIERTO -> no se llama al proveedor, respuesta de respaldo
t=25: ABIERTO -> no se llama al proveedor, respuesta de respaldo
t=41: SEMIABIERTO, llamada de prueba falla -> ABIERTO
t=42: ABIERTO -> no se llama al proveedor, respuesta de respaldo
t=75: SEMIABIERTO, llamada de prueba ok -> CERRADO
t=76: CERRADO, ok
```

Reglas

- Complejidad: $O(1)$ por pedido.
- No funcional: tolerancia a fallos: la caída del proveedor no puede tumbar el checkout; con el circuito abierto, la respuesta es inmediata.
- No funcional: observabilidad: cada cambio de estado tiene que registrarse (para alertar al equipo de guardia).
- Es una máquina de estados: cada pedido depende solo del estado, del contador y de la hora de apertura.

Pista (leela solo si te trabás): son tres estados y cinco transiciones. Dibujá el diagrama antes de programar: el semiabierto es el único que decide en base a una sola llamada.

Bonus (opcional)

- Abrió el circuito por tasa de fallas (más del 50 % en los últimos 20 pedidos) en lugar de fallas consecutivas.
- Combinalo con reintentos con espera exponencial (ejercicio de los reintentos) sin que los reintentos cuenten doble.

Cierre: El circuit breaker es un patrón clave de resiliencia en arquitecturas de microservicios.

Ejercicio 369: Caché con vencimiento para el pronóstico

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Clima y energía · Dificultad 74/100

Situación: Una app del clima consulta el pronóstico a un servicio externo que cobra por consulta. Como el pronóstico cambia poco en unos minutos, cada respuesta se guarda en un caché con vencimiento (TTL): mientras no venza, se reutiliza. El caché además tiene capacidad limitada y, si se llena, desaloja la ciudad usada hace más tiempo.

Datos de entrada

Formato: primera línea: TTL en segundos y capacidad; después "segundo ciudad" por pedido (en orden).

Ejemplo

```
600 3
0 Rosario
30 Córdoba
100 Rosario
200 Mendoza
250 Salta
300 Córdoba
650 Rosario
700 Salta
900 Córdoba
```

Restricciones: hasta 10^5 ciudades y 10^6 pedidos por minuto.

Tu programa debe

1. Guardar por ciudad la hora en que se obtuvo el pronóstico, en un mapa ordenado por uso.
2. Si la ciudad está y su edad es menor que el TTL, es un acierto: moverla al final.
3. Si no está o venció, consultar al servicio; si hay que hacer lugar, desalojar la menos usada recientemente.
4. Mostrar cada decisión y la cantidad de consultas al servicio.

Salida esperada

```
t=0 Rosario: MISS -> consulta al servicio
t=30 Córdoba: MISS -> consulta al servicio
t=100 Rosario: HIT (edad 100 s)
t=200 Mendoza: MISS -> consulta al servicio
t=250 Salta: MISS -> consulta al servicio, desaloja Córdoba
t=300 Córdoba: MISS -> consulta al servicio, desaloja Rosario
t=650 Rosario: MISS -> consulta al servicio, desaloja Mendoza
t=700 Salta: HIT (edad 450 s)
t=900 Córdoba: vencido -> consulta al servicio
Consultas al servicio: 7 de 9 pedidos
```

Reglas

- Complejidad: $O(1)$ por pedido.
- Una entrada con edad igual al TTL ya está vencida.
- No funcional: fresca: ningún usuario puede ver un pronóstico de más de TTL segundos de antigüedad.
- No funcional: el vencimiento se verifica al leer ("perezoso"); describí también cómo limpiarías entradas vencidas que nadie vuelve a pedir, para que no ocupen memoria.

Pista (leela solo si te trabás): el TTL y la capacidad son dos razones distintas para sacar algo del caché: una por tiempo, otra por espacio. Chequeá primero el vencimiento, después la capacidad.

Bonus (opcional)

- Evitá la "estampida": si vence una ciudad muy pedida y llegan 1.000 pedidos a la vez, solo uno debe consultar al servicio.
- Renová en segundo plano las entradas populares un poco antes de que venzan.

Cierre: Los cachés con TTL están en DNS, CDNs, APIs públicas y en cualquier integración con servicios pagos o lentos.

Ejercicio 370: Velocidad de transacciones sospechosa

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 74/100

Situación: Una billetera virtual detecta el uso de tarjetas robadas por la "velocidad": muchas compras seguidas, montos que se acumulan rápido, comercios nunca usados. En lugar de reglas que bloquean solas, cada señal suma puntos y la decisión depende del puntaje total, lo que reduce los falsos positivos.

Datos de entrada

Formato: una transacción por línea (en orden): "hh:mm:ss tarjeta monto comercio".

Señales (contando la transacción actual): más de 3 en los últimos 60 s: +40; más de 5 en los últimos 600 s: +30; suma de montos en los últimos 3.600 s mayor a \$150.000: +50; comercio nunca usado por la tarjeta (si la tarjeta ya tiene historial): +10. Decisión: 70 o más → RECHAZAR; 40 a 69 → DESAFÍO; menos de 40 → APROBAR.

Ejemplo

```
10:00:00 C1 8000 comercio_1
10:00:20 C1 9000 comercio_2
10:00:40 C1 7000 comercio_2
10:00:50 C1 6000 comercio_3
10:05:00 C1 12000 comercio_1
10:06:00 C1 150000 comercio_4
10:30:00 C2 5000 comercio_1
10:45:00 C1 4000 comercio_1
```

Restricciones: hasta 10^5 transacciones por segundo.

Tu programa debe

1. Guardar por tarjeta el historial de (hora, monto) y el conjunto de comercios usados.
2. Calcular las cuatro señales con sus ventanas (las ventanas son estrictas: "en los últimos 60 s" excluye lo de hace exactamente 60 s).
3. Sumar los puntos y decidir según los umbrales.
4. Mostrar el puntaje, la decisión y el detalle de las señales activadas.

Salida esperada

```
C1 $8000: puntaje 0 -> APROBAR
C1 $9000: puntaje 10 -> APROBAR [comercio nuevo (+10)]
C1 $7000: puntaje 0 -> APROBAR
C1 $6000: puntaje 50 -> DESAFÍO (código por SMS) [4 en 1 min (+40); comercio nuevo (+10)]
C1 $12000: puntaje 0 -> APROBAR
C1 $150000: puntaje 90 -> RECHAZAR [6 en 10 min (+30); $192000 en 1 h (+50); comercio nuevo (+10)]
C2 $5000: puntaje 0 -> APROBAR
C1 $4000: puntaje 50 -> DESAFÍO (código por SMS) [$196000 en 1 h (+50)]
```

Reglas

- Complejidad: $O(1)$ amortizado por transacción si cada ventana se mantiene con una cola y una suma acumulada.
- Todas las transacciones se registran en el historial, incluso las rechazadas (un fraude insistente sigue sumando).
- No funcional: latencia: la evaluación completa debe tardar menos de 20 ms.
- No funcional: explicabilidad: cada decisión se registra con las señales y los puntos.

Pista (leela solo si te trabás): cada ventana es una cola: al llegar una transacción, sacás del frente las que ya quedaron afuera. La cuenta y la suma se actualizan al entrar y al salir.

Bonus (opcional)

- Mantené las tres ventanas con colas y contadores en $O(1)$ amortizado, en lugar de recorrer el historial.
- Ajustá los pesos con datos históricos etiquetados para minimizar los rechazos de clientes legítimos.

Cierre: Los puntajes de riesgo con reglas ponderadas son el paso intermedio entre las reglas fijas y los modelos de aprendizaje automático en prevención de fraude.

Ejercicio 371: Lectores y escritores en la historia clínica

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Salud · Dificultad 75/100

Situación: La historia clínica electrónica de un hospital es leída por muchos profesionales a la vez, pero cuando alguien la modifica nadie puede leerla ni escribirla (para no ver datos a medio actualizar). Si los lectores tienen prioridad, un flujo constante de lecturas puede dejar esperando para siempre a quien quiere escribir. El equipo quiere comparar dos políticas con una simulación determinista.

Datos de entrada

Formato: una solicitud por línea (en orden de llegada): "id llegada duración" (en minutos); los ids que empiezan con L son lecturas y los que empiezan con E, escrituras.

Simulación por minuto: en cada minuto t , primero terminan las operaciones cuyo fin es t , después se encolan las solicitudes que llegan en t y, por último, se recorre la cola en orden y arranca cada solicitud que pueda: una lectura puede arrancar si no hay una escritura activa (y, con preferencia de escritores, si no hay escrituras esperando); una escritura puede arrancar si no hay nada activo.

Ejemplo

```
L1 0 4
L2 1 4
E1 2 3
L3 3 4
L4 5 4
L5 7 3
```

Restricciones: hasta 10^4 solicitudes en la simulación.

Tu programa debe

1. Implementar la simulación minuto a minuto con la cola de pendientes y la lista de activos.
2. Correrla con preferencia de lectores y con preferencia de escritores.
3. Registrar el inicio y el fin de cada solicitud.
4. Mostrar, para cada política, los intervalos de cada solicitud y cuánto esperó la escritura.

Salida esperada

```
[preferencia lectores] L1:0-4 L2:1-5 E1:10-13 L3:3-7 L4:5-9 L5:7-10 | espera de E1: 8
[preferencia escritores] L1:0-4 L2:1-5 E1:5-8 L3:8-12 L4:8-12 L5:8-11 | espera de E1: 3
```

Reglas

- Complejidad: $O(\text{tiempo total} \times \text{solicitudes})$ para la simulación.
- No funcional: ausencia de inanición: ninguna solicitud debería esperar indefinidamente; la simulación muestra cuál política lo garantiza.
- No funcional: consistencia: nunca puede haber una escritura activa al mismo tiempo que otra operación.
- El intervalo "a-b" significa que la operación empieza en el minuto a y termina en el b.

Pista (leela solo si te trabás): con preferencia de lectores, mientras siga habiendo alguien leyendo y lleguen lectores nuevos, la escritura nunca encuentra el recurso libre. Con preferencia de escritores, en cuanto hay una escritura esperando, los lectores nuevos hacen fila.

Bonus (opcional)

- Implementá una política justa por orden de llegada (FIFO estricto) y compará las esperas promedio.

- Medí la espera máxima de lectores en cada política: la preferencia de escritores puede demorar lecturas urgentes.

Cierre: El problema de lectores y escritores es la base de los bloqueos de lectura/escritura en bases de datos y sistemas operativos.

Ejercicio 372: Vuelo, hotel y auto: todo o nada

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Turismo · Dificultad 75/100

Situación: Una agencia de viajes online vende paquetes de vuelo, hotel y auto, cada uno reservado en un proveedor distinto. No existe una transacción que abarque a los tres: si el auto falla después de reservar vuelo y hotel, hay que cancelar lo ya reservado. El patrón "saga" define un paso de compensación para cada reserva y las ejecuta en orden inverso.

Datos de entrada

Formato: primera línea: los pasos en orden; después, un escenario por línea: "todo_ok", "falla paso" y, opcionalmente, "; compensación paso falla k" (esa cancelación falla k veces antes de funcionar).

Ejemplo

```
vuelo hotel auto
todo_ok
falla hotel
falla auto
falla auto; compensación vuelo falla 2
```

Restricciones: hasta 20 pasos por saga y 10^4 sagas por minuto.

Tu programa debe

1. Ejecutar los pasos en orden hasta que todos salgan bien o uno falle.
2. Si uno falla, compensar los pasos ya completados en orden inverso.
3. Si una compensación falla, reintentarla hasta que funcione (las compensaciones no se pueden abandonar).
4. Mostrar el registro de cada escenario y el resultado final.

Salida esperada

```
Escenario 1: reservar vuelo: ok; reservar hotel: ok; reservar auto: ok -> VIAJE CONFIRMADO
Escenario 2: reservar vuelo: ok; reservar hotel: FALLA; cancelar vuelo: ok -> VIAJE CANCELADO, sin
cargos
Escenario 3: reservar vuelo: ok; reservar hotel: ok; reservar auto: FALLA; cancelar hotel: ok;
cancelar vuelo: ok -> VIAJE CANCELADO, sin cargos
Escenario 4: reservar vuelo: ok; reservar hotel: ok; reservar auto: FALLA; cancelar hotel: ok;
cancelar vuelo: falla (intento 1); cancelar vuelo: falla (intento 2); cancelar vuelo: ok
(intento 3) -> VIAJE CANCELADO, sin cargos
```

Reglas

- Complejidad: $O(\text{pasos} + \text{reintentos})$ por saga.
- No funcional: consistencia eventual: al terminar la saga, o están todas las reservas o no está ninguna.
- No funcional: las compensaciones tienen que ser idempotentes, porque un reintento puede ejecutarlas dos veces.
- No funcional: el estado de la saga se guarda después de cada paso, para poder retomarla si el servidor se reinicia a mitad de camino.
- El paso que falla no se compensa (no llegó a reservarse).

Pista (leela solo si te trabás): pensá la saga como una pila: cada paso exitoso apila su compensación; si algo falla, desapilás y ejecutás.

Bonus (opcional)

- Modelá un paso que tarda en responder (timeout): no sabés si se reservó o no; definí qué hacer.
- Agregá un paso "pagar" al final que no se puede compensar automáticamente, y reordená la saga para minimizar el riesgo.

Cierre: Las sagas coordinan transacciones entre microservicios en reservas de viajes, e-commerce y pagos.

Ejercicio 373: Cajas de botín y valor esperado

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Videojuegos · Dificultad 75/100

Situación: Un juego vende cajas de botín con premios de distinta rareza. Por regulación de defensa del consumidor, tiene que publicar el valor esperado de una caja, cuántas cajas hacen falta en promedio para conseguir una legendaria y, para la colección de figuritas del evento, cuántas cajas se necesitan en promedio para completarla.

Datos de entrada

Formato: primera parte: "rareza probabilidad valor" (la última es la legendaria); una línea "?" y las probabilidades de cada figurita de la colección (suman 1).

Ejemplo

```
común 0.70 100
rara 0.25 800
legendaria 0.05 5000
?
0.4 0.3 0.2 0.1
```

Restricciones: hasta 20 figuritas (se usa inclusión-exclusión sobre 2^{20} subconjuntos).

Tu programa debe

1. Valor esperado = suma de probabilidad $\times$ valor.
2. Cajas hasta la primera legendaria: $1 / p$ (distribución geométrica); probabilidad de no obtenerla en 20 cajas: $(1 - p)^{20}$.
3. Cajas esperadas para completar la colección con probabilidades distintas: suma sobre subconjuntos no vacíos S de $(-1)^{|S| + 1} / P(S)$, donde $P(S)$ es la suma de sus probabilidades.
4. Comparar con el caso de probabilidades iguales: $n \times (1 + 1/2 + \dots + 1/n)$. Mostrar todo con 2 decimales (4 para la probabilidad).

Salida esperada

```
Valor esperado por caja: 520.00 monedas
Cajas esperadas hasta una legendaria: 20.00
Probabilidad de ninguna legendaria en 20 cajas: 0.3585
Cajas esperadas para completar la colección de 4 figuritas: 12.36 (con probabilidades iguales
serían 8.33)
```

Reglas

- Complejidad: $O(2^n \times n)$ para la colección.
- No funcional: transparencia: los números publicados tienen que ser exactos y reproducibles; usá aritmética decimal.
- No funcional: los cálculos se verifican también por simulación con semilla (bonus) antes de publicarlos.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): para la colección, el truco es la inclusión-exclusión: la espera hasta ver "alguna figurita de S " es $1 / P(S)$, y combinando esas esperas con signos alternados sale la espera hasta verlas todas.

Bonus (opcional)

- Simulá 100.000 aperturas con un generador con semilla y compará con los valores teóricos.
- Calculá cuántas cajas hacen falta para tener un 90 % de probabilidad de conseguir una legendaria.

Cierre: El valor esperado y el problema del coleccionista son la base del diseño y la regulación de mecánicas aleatorias en juegos.

Ejercicio 374: Órdenes stop y cancelaciones

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 75/100

Situación: El motor de matching del mercado incorpora dos funciones que piden los operadores: cancelar órdenes que todavía no se ejecutaron y órdenes stop, que quedan ocultas hasta que el precio de la última operación cruza un precio gatillo, y entonces entran como órdenes de mercado. Hay que integrarlas sin romper la prioridad precio-tiempo.

Datos de entrada

Formato: una orden por línea, en orden: "id compra|venta precio cantidad" (límite), "id stop_compra|stop_venta gatillo cantidad" o "id cancelar id_orden".

Un stop de compra se activa cuando el último precio operado es $\geq$ gatillo; uno de venta, cuando es $\leq$ gatillo. Al activarse, se ejecuta como orden de mercado (cruza contra lo que haya, sin límite de precio; lo que no se ejecuta se descarta).

Ejemplo

```
01 compra 100 5
02 venta 102 5
03 stop_compra 101 3
04 venta 105 10
05 compra 101 2
06 venta 101 2
07 cancelar 02
08 compra 102 4
```

Restricciones: hasta 10^6 órdenes por segundo y 10^5 stops pendientes.

Tu programa debe

1. Partir del motor del ejercicio del matching (dos colas de prioridad).
2. Cancelar con "borrado perezoso": marcar la orden como cancelada y descartarla recién cuando llegue al tope de su cola.
3. Después de cada operación ejecutada, revisar los stops pendientes (en orden de llegada) y activar los que correspondan; una activación puede generar operaciones que activan otros stops.
4. Mostrar los eventos (operaciones, activaciones y cancelaciones), el libro final y los stops pendientes.

Salida esperada

```
Eventos:
 06 vende 2 @ 101 (contra 05)
 (se activa 03: último precio 101)
 03 compra 3 @ 102 (contra 02)
 02 cancelada (2 sin ejecutar)
Compras en el libro: 08 4@102, 01 5@100
Ventas en el libro: 04 10@105
Stops pendientes: ninguno
```

Reglas

- Complejidad: $O(\log n)$ por orden límite y por cancelación (amortizado); revisar los stops con dos colas ordenadas por gatillo sería $O(\log n)$ por activación (bonus).
- No funcional: determinismo: las activaciones en cascada tienen que procesarse siempre en el mismo orden.
- No funcional: una cancelación que llega después de que la orden se ejecutó por completo no hace nada (no es un error).
- Casos borde: cancelar una orden parcialmente ejecutada (se cancela solo el resto).

Pista (leela solo si te trabás): el borrado perezoso evita buscar la orden dentro del montículo: la dejás ahí marcada y la ignorás cuando aparezca arriba. Y los stops son "observadores" del último precio.

Bonus (opcional)

- Guardá los stops en dos colas de prioridad por gatillo para activarlos en $O(\log n)$.
- Agregá órdenes "llenar o matar": se ejecutan completas en el momento o no se ejecutan.

Cierre: Las órdenes stop y las cancelaciones son funciones básicas de cualquier plataforma de trading, y su correcta implementación es un requisito regulatorio.

Ejercicio 375: Repartir usuarios entre particiones

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Redes sociales · Dificultad 75/100

Situación: Una red social reparte los datos de sus usuarios en particiones. Cuando se agrega o se quita una partición, hay que mover datos, y cada usuario movido implica copiar su perfil, sus fotos y su historial. El equipo evalúa "rendezvous hashing" (el mayor puntaje gana): cada usuario calcula un puntaje con cada partición y se queda con la de mayor puntaje, sin anillo.

Datos de entrada

Formato: primera línea: particiones; segunda: cantidad de usuarios (usuario1, usuario2...); una línea "?" y operaciones, cada una aplicada sobre el reparto inicial: "agregar p" o "quitar p".

Puntaje de la partición s para el usuario u : FNV-1a de 32 bits del texto " $s:u$ " (FNV-1a de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.) En empate, gana la partición de nombre mayor.

Ejemplo

```
p1 p2 p3 p4
1000
?
agregar p5
quitar p2
```

Restricciones: hasta 10^3 particiones y 10^9 usuarios ($O(\text{particiones})$ por usuario).

Tu programa debe

1. Asignar cada usuario a la partición de mayor puntaje.
2. Para cada operación, recalculan la asignación con el nuevo conjunto de particiones.
3. Contar los usuarios movidos y compararlos con los que se moverían con "hash resto cantidad de particiones".
4. Mostrar los repartos y los porcentajes (1 decimal).

Salida esperada

```
Reparto inicial: p1=237 p2=286 p3=241 p4=236
agregar p5: p1=190 p2=237 p3=189 p4=189 p5=195 | movidos 195 (19.5 %) | con hash resto n: 806
(80.6 %)
quitar p2: p1=337 p3=336 p4=327 | movidos 286 (28.6 %) | con hash resto n: 783 (78.3 %)
```

Reglas

- Complejidad: $O(P)$ por usuario (P particiones); para P grande, el anillo con búsqueda binaria es más rápido.
- No funcional: movimiento mínimo: al agregar una partición, solo se mueven usuarios hacia ella ($\approx 1 / (P + 1)$); al quitar una, solo se mueven los suyos.
- No funcional: sin estado compartido: cualquier servidor calcula el destino de cualquier usuario sin consultar una tabla central.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): agregar una partición no cambia los puntajes de las demás: un usuario solo se muda si la partición nueva le da un puntaje más alto que su favorita actual.

Bonus (opcional)

- Agregá pesos por partición (las más grandes reciben más usuarios) con el puntaje ponderado.
- Elegí 2 particiones por usuario (las de mayor puntaje) para tener una réplica.

Cierre: El rendezvous hashing es una alternativa simple al anillo, usada en CDNs, cachés distribuidos y sistemas de almacenamiento.

Lote 16: ejercicios 376 a 400

Ejercicio 376: La nómina que se procesa de noche

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Recursos humanos · Dificultad 76/100

Situación: El sistema de recursos humanos de una empresa procesa de noche la liquidación de sueldos: cada liquidación es un mensaje en una cola. Algunos fallan por problemas temporales (el banco no responde) y conviene reintentarlos más tarde; otros fallan por errores permanentes (un CBU inválido) y reintentar no sirve. Los que no se pueden procesar van a una cola de mensajes fallidos para revisión manual, sin frenar al resto.

Datos de entrada

Formato: primera línea: máximo de reintentos y espera base en segundos; después, un mensaje por línea con los resultados de sus intentos en orden: T (error temporal), P (error permanente) u ok. Si faltan resultados, se asume T.

Ejemplo

```
3 60
liq_ana T T ok
liq_beto P
liq_caro T ok
liq_dani T T T T
liq_eli ok
```

Restricciones: hasta 10^6 mensajes por noche.

Tu programa debe

1. Poner todos los mensajes en una cola de prioridad por hora del próximo intento (todos arrancan en $t = 0$; en empate, en el orden de la entrada y después los reintentos por orden de creación).
2. Procesar el mensaje con la hora más temprana: si sale ok, terminar; si es error permanente, mandarlo a la cola de fallidos.
3. Si es error temporal y le quedan reintentos, reprogramarlo con espera base $\times 2^{(\text{número de reintento, desde } 0)}$; si no le quedan, a la cola de fallidos.
4. Mostrar la línea de tiempo y la cola de fallidos.

Salida esperada

```
t=0s liq_ana: error temporal, reintento en 60s
t=0s liq_beto: error permanente -> cola de fallidos
t=0s liq_caro: error temporal, reintento en 60s
t=0s liq_dani: error temporal, reintento en 60s
t=0s liq_eli: procesado (intento 1)
t=60s liq_ana: error temporal, reintento en 120s
t=60s liq_caro: procesado (intento 2)
t=60s liq_dani: error temporal, reintento en 120s
t=180s liq_ana: procesado (intento 3)
t=180s liq_dani: error temporal, reintento en 240s
t=420s liq_dani: error temporal, sin reintentos -> cola de fallidos
Cola de fallidos (revisión manual): liq_beto, liq_dani
```

Reglas

- Complejidad: $O(\log n)$ por intento.
- El primer intento no cuenta como reintento: con máximo 3, un mensaje puede intentarse hasta 4 veces.
- No funcional: aislamiento: un mensaje que falla nunca bloquea a los demás (no se reintenta "en el lugar").
- No funcional: la cola de fallidos conserva el mensaje completo y el motivo, para reprocesarlo después de corregir el dato.

Pista (leela solo si te trabás): la cola de prioridad por "próxima hora de intento" hace que los reintentos se intercalen solos con los mensajes nuevos, sin temporizadores.

Bonus (opcional)

- Agregá un botón de "reprocesar" que devuelve los mensajes de la cola de fallidos a la cola principal.
- Limitá la cantidad de liquidaciones en vuelo contra el banco a 10 simultáneas.

Cierre: Los reintentos con cola de mensajes fallidos son el patrón estándar de procesamiento asíncrono confiable en sistemas empresariales.

Ejercicio 377: Si hoy llueve, ¿mañana también?

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Clima y energía · Dificultad 76/100

Situación: Un servicio de pronóstico agrícola modela el tiempo de cada día según el del día anterior: si hoy llueve, mañana llueve con cierta probabilidad, se nubla con otra, etc. Es una cadena de Markov. Los productores quieren saber la probabilidad de cada estado dentro de varios días y cómo es el clima "a la larga" en la zona.

Datos de entrada

Formato: primera línea: los estados; después, la matriz de transición (fila = hoy, columna = mañana; cada fila suma 1); luego el estado de hoy y los días a consultar.

Ejemplo

```
sol nublado lluvia
0.7 0.2 0.1
0.3 0.4 0.3
0.2 0.4 0.4
lluvia
1 2 7 30
```

Restricciones: hasta 50 estados y 10^6 días (para plazos largos, exponenciación rápida de matrices).

Tu programa debe

1. Representar el estado actual como un vector de probabilidades (1 en el estado de hoy).
2. Avanzar un día multiplicando el vector por la matriz: $\text{nuevo}[j] = \text{suma de } v[i] \times P[i][j]$.
3. Informar la distribución en cada día consultado.
4. Calcular la distribución estacionaria resolviendo $\pi = \pi \times P$ con la condición de que π sume 1 (eliminación de Gauss), y mostrarla con 4 decimales y como fracción exacta.

Salida esperada

```
Dentro de 1 día(s): sol 0.2000 | nublado 0.4000 | lluvia 0.4000
Dentro de 2 día(s): sol 0.3400 | nublado 0.3600 | lluvia 0.3000
Dentro de 7 día(s): sol 0.4591 | nublado 0.3087 | lluvia 0.2321
Dentro de 30 día(s): sol 0.4615 | nublado 0.3077 | lluvia 0.2308
Distribución estacionaria (largo plazo): sol 0.4615 (6/13) | nublado 0.3077 (4/13) | lluvia 0.2308 (3/13)
```

Reglas

- Complejidad: $O(\text{días} \times \text{estados}^2)$ avanzando de a un día; $O(\text{estados}^3 \times \log \text{días})$ con exponenciación de matrices.
- No funcional: exactitud: los cálculos se hacen con fracciones exactas y se redondean solo al mostrar (4 decimales, 0,5 hacia arriba).
- No funcional: validación de entrada: rechazar matrices cuyas filas no suman 1 o tienen valores negativos.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la propiedad de Markov dice que mañana depende solo de hoy. Por eso toda la historia se resume en un vector, y cada día es una multiplicación.

Bonus (opcional)

- Calculá la cantidad esperada de días hasta el primer día de sol, partiendo de lluvia.
- Estimá la matriz a partir de un registro histórico de 10 años (contando transiciones).

Cierre: Las cadenas de Markov modelan clima, colas de atención, comportamiento de usuarios en una web y el algoritmo original de PageRank.

Ejercicio 378: Stock reservado por 10 minutos

Nivel 8: Senior II · Algoritmos de sistemas · Temática: E-commerce · Dificultad 76/100

Situación: En un e-commerce, al pasar al checkout el stock queda reservado durante 10 minutos para que nadie se lo "gane" al comprador mientras paga. Si no paga a tiempo, la reserva vence y las unidades vuelven a estar disponibles. El sistema tiene que ser consistente: nunca vender más que el stock, ni cobrar una reserva vencida.

Datos de entrada

Formato: primera línea: stock y duración de la reserva en segundos; después, eventos en orden: "segundo reservar usuario cantidad" y "segundo pagar usuario".

Ejemplo

```
10 600
0 reservar ana 3
60 reservar beto 4
120 reservar caro 4
300 pagar ana
650 reservar dani 2
700 pagar beto
720 reservar eli 3
800 pagar caro
1400 pagar eli
```

Restricciones: hasta 10^6 reservas activas y 10^4 eventos por segundo.

Tu programa debe

1. Guardar las reservas activas en un mapa y sus vencimientos en una cola de prioridad.
2. Antes de procesar cada evento, vencer todas las reservas con vencimiento $\leq$ la hora actual (devolviendo sus unidades).
3. Reservar solo si hay stock libre suficiente; pagar solo si la reserva sigue activa.
4. Mostrar cada evento, los vencimientos y el estado final.

Salida esperada

```
t=0 ana reserva 3 (vence t=600) | libres 7
t=60 beto reserva 4 (vence t=660) | libres 3
t=120 caro pide 4: sin stock suficiente (libres 3)
t=300 ana paga 3 | vendidas 3
t=650 dani reserva 2 (vence t=1250) | libres 1
(t=660: vence la reserva de beto, vuelven 4)
t=700 beto intenta pagar: la reserva no existe o venció
t=720 eli reserva 3 (vence t=1320) | libres 2
t=800 caro intenta pagar: la reserva no existe o venció
(t=1250: vence la reserva de dani, vuelven 2)
(t=1320: vence la reserva de eli, vuelven 3)
t=1400 eli intenta pagar: la reserva no existe o venció
Final: vendidas 3, libres 7, reservadas 0
```

Reglas

- Complejidad: $O(\log n)$ por reserva y por vencimiento.
- Una reserva que vence exactamente en el segundo del pago ya está vencida.
- No funcional: consistencia: libres + reservadas + vendidas = stock inicial en todo momento (verificalo como invariante).
- No funcional: el vencimiento es perezoso (se procesa al llegar un evento); describí qué pasaría con la invariante si un proceso aparte venciera reservas en paralelo sin coordinación.

Pista (leela solo si te trabás): no hace falta un temporizador por reserva: antes de cada evento, sacás de la cola de prioridad todo lo que ya venció.

Bonus (opcional)

- Permití extender una reserva una sola vez por 5 minutos más (cuidado con la entrada vieja en la cola: usá el vencimiento como versión).
- Agregá una lista de espera: cuando vence una reserva, las unidades se ofrecen al primero de la lista.

Cierre: Las reservas con vencimiento se usan en e-commerce, venta de entradas, reservas de asientos y turnos.

Ejercicio 379: Doble gasto en la billetera virtual

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 76/100

Situación: Una billetera virtual permite pagar sin conexión desde varios dispositivos del mismo usuario (el celular y la tablet), con un límite local. Cuando los dispositivos se sincronizan, el servidor recibe los eventos de cada uno por separado y tiene que ordenarlos de forma determinista, aplicar los que alcanzan y detectar los gastos que superaron el saldo real (doble gasto).

Datos de entrada

Formato: primera línea: saldo inicial; después, eventos de cualquier dispositivo en cualquier orden: "dispositivo número hh:mm:ss carga|compra monto" (el número es el contador local del dispositivo).

Orden global: por hora; en empate, por nombre de dispositivo; en empate, por número.

Ejemplo

```
10000
celular 1 09:00:05 compra 6000
tablet 1 09:00:07 compra 5000
celular 2 09:10:00 carga 3000
tablet 2 09:10:00 compra 2500
tablet 3 09:20:00 compra 1000
```

Restricciones: hasta 10^3 dispositivos por usuario y 10^6 eventos por sincronización.

Tu programa debe

1. Juntar los eventos de todos los dispositivos y ordenarlos con el criterio global.
2. Aplicarlos en ese orden: las cargas suman; una compra se aplica si el saldo alcanza.
3. Si una compra no alcanza, marcarla como doble gasto (ya se hizo sin conexión, así que queda como monto en disputa).
4. Mostrar el resultado de cada evento, el saldo final y el monto en disputa.

Salida esperada

```
celular#1 compra 6000 -> saldo 4000
tablet#1 compra 5000: DOBLE GASTO detectado (saldo 4000), se marca para cobro posterior
celular#2 carga 3000 -> saldo 7000
tablet#2 compra 2500 -> saldo 4500
tablet#3 compra 1000 -> saldo 3500
Saldo final: 3500 | monto en disputa: 5000
```

Reglas

- Complejidad: $O(n \log n)$ por el ordenamiento.
- No funcional: determinismo: todos los servidores que procesen los mismos eventos tienen que llegar al mismo resultado, sin importar el orden en que llegaron (por eso el desempate completo).
- No funcional: los relojes de los dispositivos pueden estar desfasados; describí qué riesgo introduce ordenar por la hora del dispositivo y cómo lo mitigarías (por ejemplo, con relojes lógicos).
- Casos borde: dos eventos en el mismo segundo desde distintos dispositivos.

Pista (leela solo si te trabás): el problema no es sumar y restar: es ponerse de acuerdo en qué pasó primero. Un orden total y determinista convierte eventos desordenados en una historia única.

Bonus (opcional)

- Reemplazá la hora del dispositivo por relojes de Lamport y compará el orden resultante.
- Limitá el gasto sin conexión de cada dispositivo a una fracción del saldo para acotar la disputa máxima.

Cierre: Ordenar eventos concurrentes es un problema central de los sistemas distribuidos, desde billeterías hasta bases de datos replicadas.

Ejercicio 380: Límites por usuario y por país

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Telecomunicaciones · Dificultad 76/100

Situación: Una operadora expone una API de envío de SMS con límites en varios niveles: por usuario (para que nadie acapare), por país (por los acuerdos con cada operador local) y global (la capacidad de la plataforma). Un pedido se acepta solo si ninguno de los tres niveles está agotado, y en ese caso consume cupo en los tres.

Datos de entrada

Formato: primera línea: los límites por segundo "usuario N país N global N"; después, "segundo usuario país" por pedido (en orden). Los contadores se reinician al cambiar de segundo (ventana fija).

Ejemplo

```
usuario 2 país 3 global 4
0 ana AR
0 ana AR
0 ana AR
0 beto AR
0 caro BR
0 dani AR
0 eli BR
1 ana AR
1 beto AR
1 caro BR
```

Restricciones: hasta 10^6 pedidos por segundo, 10^7 usuarios y 200 países.

Tu programa debe

1. Llevar un contador por usuario, por país y global para el segundo actual; reiniciarlos al cambiar de segundo.
2. Para cada pedido, verificar los tres niveles en orden (usuario, país, global) y encontrar el primero agotado.
3. Si hay alguno agotado, rechazar informando el nivel; si no, sumar 1 en los tres.
4. Mostrar cada decisión.

Salida esperada

```
s=0 ana (AR): aceptado
s=0 ana (AR): aceptado
s=0 ana (AR): RECHAZADO por límite de usuario
s=0 beto (AR): aceptado
s=0 caro (BR): aceptado
s=0 dani (AR): RECHAZADO por límite de país
s=0 eli (BR): RECHAZADO por límite global
s=1 ana (AR): aceptado
s=1 beto (AR): aceptado
s=1 caro (BR): aceptado
```

Reglas

- Complejidad: $O(\text{niveles})$ por pedido.
- No funcional: atomicidad: un pedido rechazado no puede consumir cupo en ningún nivel (verificar todo antes de descontar).
- No funcional: equidad: un usuario que abusa se frena en su propio nivel sin agotar el cupo de su país para los demás.
- No funcional: en un sistema distribuido, los contadores globales viven en un almacenamiento compartido; describí cómo evitar que ese almacenamiento sea un cuello de botella (por ejemplo, cupos locales que se reponen).

Pista (leela solo si te trabás): primero preguntá a todos los niveles "¿hay lugar?" y recién si todos dicen que sí, descontá en todos. Descontar a medida que verificás deja cupo "perdido" en los niveles superiores.

Bonus (opcional)

- Reemplazá la ventana fija por baldes de fichas en cada nivel.
- Dale a los usuarios premium un límite propio más alto, sin cambiar los de país y global.

Cierre: Los limitadores jerárquicos protegen plataformas de mensajería, APIs en la nube y redes de telecomunicaciones.

Ejercicio 381: Cinco cocineros y cinco sartenes

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Gastronomía · Dificultad 77/100

Situación: En la cocina de un restaurante, cinco cocineros trabajan en una mesa redonda con cinco sartenes, una entre cada par de cocineros. Para cocinar un plato, cada uno necesita las dos sartenes que tiene al lado. Si todos toman primero la de su izquierda, pueden quedar trabados para siempre. La solución clásica es tomar siempre primero la sartén de menor número.

Datos de entrada

Formato: una línea: cantidad de cocineros (y de sartenes) y platos que cocina cada uno.

El cocinero i usa las sartenes i e $(i + 1) \text{ resto } n$. Simulación por rondas: en cada ronda, cada cocinero (de 0 a $n - 1$) hace una sola acción si puede: tomar su primera sartén, tomar la segunda, o cocinar un plato y soltar ambas. Si en una ronda nadie puede hacer nada, hay deadlock.

Ejemplo

```
5 2
```

Restricciones: hasta 10^3 cocineros en la simulación.

Tu programa debe

1. Definir, para cada cocinero, el orden en que toma las sartenes según la estrategia.
2. Simular por rondas con la estrategia "izquierda primero" (sartén i y después $i + 1$).
3. Simular con "menor número primero" (el último cocinero toma la sartén 0 antes que la $n - 1$).
4. Mostrar, para cada estrategia, si hay deadlock (y en qué ronda) o en cuántas rondas terminan, y los platos de cada uno.

Salida esperada

```
[izquierda primero] DEADLOCK en la ronda 2: cada cocinero tiene una sartén y espera la otra |
  platos 0 0 0 0 0
[sartén de menor número primero] terminan en 16 rondas | platos 2 2 2 2 2
```

Reglas

- Complejidad: $O(\text{rondas} \times n)$.
- No funcional: vivacidad: la estrategia elegida tiene que garantizar que siempre alguien pueda avanzar.
- No funcional: la simulación es determinista (orden fijo de los cocineros) para que el deadlock sea reproducible.
- Por qué funciona el orden global: para que haya un ciclo de espera, alguien tendría que esperar una sartén de número menor que la que tiene, y la regla lo impide.

Pista (leela solo si te trabás): el deadlock necesita un ciclo: cada uno tiene algo que el siguiente necesita. Si todos piden los recursos en el mismo orden global, el ciclo no se puede cerrar.

Bonus (opcional)

- Probá la solución del "mozo": como máximo $n - 1$ cocineros pueden intentar cocinar a la vez.
- Medí la equidad: ¿algún cocinero cocina mucho menos que otros en la versión ordenada si los platos son infinitos?

Cierre: Es el problema de los filósofos comensales de Dijkstra, y el orden global de recursos es la regla que usan las bases de datos para evitar bloqueos mutuos.

Ejercicio 382: El feed armado desde eventos

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Redes sociales · Dificultad 77/100

Situación: Una red social guarda todo lo que pasa como eventos (seguir, dejar de seguir, publicar, borrar). El feed de cada usuario es una "proyección": una vista calculada a partir de los eventos. Si cambia la lógica del feed, se puede recalcular desde cero con los mismos eventos, sin migrar datos.

Datos de entrada

Formato: eventos "versión tipo datos": sigue u v, deja u v, publica autor texto (el id de la publicación es su versión), borra id; una línea "?" y consultas "usuario k" (las k publicaciones más recientes de quienes sigue).

Ejemplo

```
1 sigue ana beto
2 sigue ana caro
3 publica beto Llegué_a_Bariloche
4 publica caro Nuevo_trabajo!
5 publica dani Receta_de_empanadas
6 sigue ana dani
7 publica beto Nieve_por_todos_lados
8 borra 4
9 deja ana beto
10 publica dani Foto_del_asado
11 sigue beto caro
12 publica caro Primer_día_en_la_oficina
?
ana 3
beto 3
```

Restricciones: hasta 10^9 eventos; feeds de hasta 100 publicaciones.

Tu programa debe

1. Reproducir los eventos en orden para armar la proyección: a quién sigue cada usuario y qué publicaciones existen.
2. Para cada consulta, filtrar las publicaciones de las cuentas que sigue el usuario al final de los eventos.
3. Ordenarlas de la más nueva a la más vieja y quedarse con las k primeras.
4. Mostrar cada feed (los guiones bajos del texto se muestran como espacios).

Salida esperada

```
Feed de ana: caro: Primer día en la oficina | dani: Foto del asado | dani: Receta de empanadas
Feed de beto: caro: Primer día en la oficina
```

Reglas

- Complejidad: $O(\text{eventos})$ para la proyección; $O(p \log p)$ por feed con p publicaciones candidatas (con un montículo, $O(p \log k)$).
- Dejar de seguir oculta también las publicaciones viejas de esa cuenta; una publicación borrada desaparece de todos los feeds.
- No funcional: reconstruibilidad: la proyección se puede tirar y rearmar desde los eventos en cualquier momento.
- No funcional: en producción, la proyección se actualiza de forma incremental con cada evento nuevo, y se acepta que el feed tarde unos segundos en reflejarlo (consistencia eventual).

Pista (leela solo si te trabás): la proyección es un "resumen" de los eventos pensado para responder rápido una pregunta concreta. Los eventos son la verdad; la proyección es descartable.

Bonus (opcional)

- Armá una segunda proyección desde los mismos eventos: cantidad de seguidores de cada usuario.
- Materializá los feeds "al publicar" (fan-out on write) y compará con calcularlos "al leer".

Cierre: Las proyecciones sobre eventos son la forma en que redes sociales y sistemas de mensajería arman vistas personalizadas a gran escala.

Ejercicio 383: La racha que funde la caja chica

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 77/100

Situación: La caja chica de una sucursal arranca con unos pocos billetes; cada día, con cierta probabilidad entra un billete (ventas en efectivo) y si no, sale uno (gastos). Si la caja llega a cero, hay que pedir fondos a la central; si llega a la meta, se deposita. Finanzas quiere la probabilidad de fundirse antes de llegar a la meta y la de sufrir una racha larga de días malos.

Datos de entrada

Formato: primera parte, una línea por escenario: "p_día_bueno caja_inicial meta"; una línea "?" y "p_día_bueno días k" para la racha.

Ejemplo

```
0.45 5 20
0.5 5 20
0.55 5 20
?
0.5 30 5
```

Restricciones: caja y meta hasta 10^4 ; días hasta 10^6 ; racha hasta 100.

Tu programa debe

1. Ruina del jugador: con $r = q / p$ ($q = 1 - p$), $P(\text{fundirse}) = (r^i - r^N) / (1 - r^N)$; si $p = q$, $P = 1 - i / N$.
2. Para la racha, programación dinámica: estado = largo de la racha actual de días malos (0 a $k - 1$); un día bueno la reinicia, uno malo la alarga.
3. Al final, $P(\text{alguna racha de } k) = 1 - (\text{probabilidad de estar en algún estado sin haber llegado a } k)$.
4. Mostrar las probabilidades con 4 decimales.

Salida esperada

```
p=0.45 caja 5, meta 20: P(fundirse) = 0.9682
p=0.5 caja 5, meta 20: P(fundirse) = 0.7500
p=0.55 caja 5, meta 20: P(fundirse) = 0.3550
P(al menos 5 días malos seguidos en 30 días, p día malo = 0.5) = 0.3682
```

Reglas

- Complejidad: $O(\log N)$ por escenario de ruina (potencias) y $O(\text{días} \times k)$ para la racha.
- No funcional: exactitud: usó fracciones exactas o decimales de alta precisión; con p cercano a 0,5, las potencias de r generan errores grandes con decimales binarios.
- No funcional: el resultado se redondea recién al final (4 decimales, 0,5 hacia arriba).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la ruina del jugador sale de plantear $P(i) = p \times P(i + 1) + q \times P(i - 1)$ con $P(0) = 1$ y $P(N) = 0$. Una pequeña ventaja diaria (0,55 contra 0,45) cambia muchísimo el resultado.

Bonus (opcional)

- Verificá ambas probabilidades por simulación con un generador con semilla.
- Calculá la cantidad esperada de días hasta que la caja llegue a 0 o a la meta.

Cierre: La ruina del jugador y las rachas son modelos básicos de gestión de liquidez, riesgo operativo y apuestas.

Ejercicio 384: Escritura directa o diferida en el caché de stock

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Logística y envíos · Dificultad 77/100

Situación: Un sistema de depósitos guarda el stock en una base de datos y usa un caché para no consultarla todo el tiempo. Al actualizar el stock hay dos políticas: escribir en el caché y en la base a la vez (escritura directa) o escribir solo en el caché y bajar a la base cuando el dato sale del caché (escritura diferida). La diferida ahorra escrituras, pero si el servidor se cae, lo no bajado se pierde.

Datos de entrada

Formato: primera línea: capacidad del caché (LRU); después operaciones "leer producto", "escribir producto valor" y "CAIDA" (el servidor se cae y el caché se pierde). Stock inicial en la base: A = 100, B = 20, C = 30.

Ejemplo

```
2
leer A
escribir A 50
escribir A 45
leer B
escribir B 10
leer C
escribir A 40
escribir C 7
leer A
CAIDA
```

Restricciones: hasta 10^6 productos y 10^5 operaciones por segundo.

Tu programa debe

1. Simular el caché LRU: una lectura o escritura de un producto ausente lo carga desde la base (una lectura a la base).
2. Con escritura directa, cada escritura va también a la base.
3. Con escritura diferida, la escritura marca el producto como "sucio"; al desalojarlo, si está sucio, se escribe en la base.
4. Al llegar la caída, listar los productos sucios (su valor en el caché y el de la base) y mostrar los contadores de cada política.

Salida esperada

```
[escritura directa] lecturas a la base 4, escrituras a la base 5 | datos perdidos en la caída:
ninguno
[escritura diferida] lecturas a la base 4, escrituras a la base 2 | datos perdidos en la caída:
A=40 (base 45), C=7 (base 30)
```

Reglas

- Complejidad: $O(1)$ por operación.
- No funcional: durabilidad contra rendimiento: la escritura directa nunca pierde datos confirmados; la diferida reduce la carga de la base a costa de un riesgo.
- No funcional: con escritura diferida, describí un mecanismo para acotar la pérdida (por ejemplo, bajar los sucios cada N segundos o registrar las escrituras en un log).
- Los datos perdidos se listan en orden alfabético.

Pista (leela solo si te trabás): la escritura diferida junta varias escrituras del mismo producto en una sola: por eso ahorra. Pero mientras el dato está sucio, la única copia correcta vive en memoria.

Bonus (opcional)

- Agregá un volcado periódico de sucios y medí cuántos datos se perderían con distintos períodos.
- Implementá "escritura alrededor" (write-around): las escrituras van solo a la base y el caché se invalida.

Cierre: Elegir entre escritura directa y diferida es una decisión clásica en discos, bases de datos y cachés distribuidos.

Ejercicio 385: Dos recepcionistas, un mismo turno

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Salud · Dificultad 77/100

Situación: En una clínica, dos recepcionistas abren el mismo turno a la vez: una lo pasa a otra hora y la otra le cambia el médico. Si el sistema guarda "lo último que llegó", el segundo cambio pisa al primero sin que nadie se entere. Con control de concurrencia optimista, cada lectura trae un número de versión y una escritura solo se acepta si la versión no cambió desde que se leyó.

Datos de entrada

Formato: primera línea: "turno médico hora versión"; después, acciones en orden: "recepcionista leer", "recepcionista cambiar_hora hh:mm" o "recepcionista cambiar_médico nombre".

Ejemplo

```
turno_42 Dra.Paz 10:00 1
Recepción1 leer
Recepción2 leer
Recepción1 cambiar_hora 11:00
Recepción2 cambiar_médico Dr.Ríos
Recepción2 leer
Recepción2 cambiar_médico Dr.Ríos
Recepción1 cambiar_hora 11:30
```

Restricciones: hasta 10^5 turnos editados por minuto.

Tu programa debe

1. Guardar el turno con su versión actual.
2. Al leer, recordar la versión que vio cada recepcionista.
3. Al escribir, aceptar solo si la versión vista coincide con la actual: aplicar el cambio, incrementar la versión y actualizar lo que vio quien escribió.
4. Si no coincide, rechazar con un conflicto (409) y exigir releer. Mostrar cada acción.

Salida esperada

```
Recepción1 lee turno_42: Dra.Paz 10:00 (versión 1)
Recepción2 lee turno_42: Dra.Paz 10:00 (versión 1)
Recepción1 cambiar_hora -> Dra.Paz 11:00 (versión 2)
Recepción2 intenta cambiar_médico con versión 1: CONFLICTO 409 (la versión actual es 2), debe releer
Recepción2 lee turno_42: Dra.Paz 11:00 (versión 2)
Recepción2 cambiar_médico -> Dr.Ríos 11:00 (versión 3)
Recepción1 intenta cambiar_hora con versión 2: CONFLICTO 409 (la versión actual es 3), debe releer
```

Reglas

- Complejidad: $O(1)$ por acción.
- No funcional: ninguna actualización se pierde en silencio: o se aplica sobre datos frescos, o se rechaza.
- No funcional: en una base de datos, la verificación y la escritura van en una sola instrucción ("actualizar donde versión = v"), para que sea atómica.
- Quien escribe con éxito queda con la versión nueva (puede seguir editando sin releer).

Pista (leela solo si te trabás): optimista significa "no bloqueo a nadie mientras edita; verifico al guardar". La versión es la prueba de que nadie tocó el dato en el medio.

Bonus (opcional)

- En lugar de rechazar, fusioná automáticamente cambios sobre campos distintos (hora y médico no chocan).
- Compará con bloqueo pesimista: la primera recepcionista que abre el turno lo bloquea 5 minutos.

Cierre: El control optimista con versiones se usa en APIs REST (con encabezados de versión), editores colaborativos y bases de datos.

Ejercicio 386: Viajes compartidos que se emparejan solos

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Transporte · Dificultad 77/100

Situación: Una app de viajes compartidos empareja pasajeros con conductores. Los pasajeros se atienden por orden de llegada, y a cada uno se le asigna el conductor libre más cercano. Cuando no hay conductores libres, los pasajeros esperan en fila y se asignan apenas un conductor termina un viaje (queda libre donde dejó al pasajero).

Datos de entrada

Formato: eventos "conductor minuto id x y" (se conecta libre en (x, y)) y "pasajero minuto id x y duración" (el viaje dura esa cantidad de minutos después de que el conductor lo busca).

Distancia: $|x_1 - x_2| + |y_1 - y_2|$ (cuadras, 1 minuto por cuadra). En empate de distancia, el conductor libre desde hace más tiempo; en empate, el de menor id. El conductor queda libre en (x, y) del pasajero al terminar (en el ejercicio, el destino coincide con el origen).

Ejemplo

```
conductor 0 C1 0 0
conductor 0 C2 10 10
pasajero 1 P1 2 1 8
pasajero 2 P2 9 9 5
pasajero 3 P3 1 1 6
pasajero 4 P4 0 5 4
conductor 5 C3 5 5
pasajero 12 P5 3 3 2
```

Restricciones: hasta 10^5 conductores y 10^6 pedidos por hora.

Tu programa debe

1. Procesar los eventos en orden de minuto (en empate, en el orden de la entrada).
2. Antes de cada evento, liberar los conductores cuyos viajes terminaron hasta ese minuto (en orden de fin), intentando asignar pasajeros en espera en el minuto en que se liberan.
3. Al llegar un pasajero o un conductor, asignar mientras haya pasajeros en espera y conductores libres: el primer pasajero de la fila con el conductor libre más cercano.
4. El viaje termina en minuto actual + distancia + duración. Mostrar cada asignación y quién quedó sin asignar.

Salida esperada

```
t=1: P1 con C1 (distancia 3, esperó 0, libera t=12)
t=2: P2 con C2 (distancia 2, esperó 0, libera t=9)
t=5: P3 con C3 (distancia 8, esperó 2, libera t=19)
t=9: P4 con C2 (distancia 13, esperó 5, libera t=26)
t=12: P5 con C1 (distancia 3, esperó 0, libera t=17)
Sin asignar: nadie
```

Reglas

- Complejidad: $O(\text{conductores libres})$ por asignación con búsqueda lineal; con un índice espacial (grilla o árbol k-d), mucho menos.
- No funcional: equidad: los pasajeros se atienden estrictamente por orden de llegada.
- No funcional: latencia: la asignación tiene que responder en menos de 1 segundo; describí cómo dividirías la ciudad en zonas para no comparar contra todos los conductores.
- Casos borde: conductor que se libera y atiende al instante a un pasajero en espera.

Pista (leela solo si te trabás): hay dos fuentes de eventos: los de la entrada y los fines de viaje. Una cola de prioridad por hora de fin te dice cuándo vuelve a estar libre cada conductor.

Bonus (opcional)

- Minimizá la espera total emparejando en lotes cada 30 segundos (asignación óptima) en lugar de a uno.
- Agregá viajes compartidos: un conductor puede llevar a dos pasajeros si sus orígenes están a menos de 3 cuadras.

Cierre: El emparejamiento en tiempo real con colas y prioridades es el corazón de las apps de transporte, delivery y servicios a domicilio.

Ejercicio 387: ¿El botón nuevo vende más?

Nivel 8: Senior II · Algoritmos de sistemas · Temática: E-commerce · Dificultad 78/100

Situación: Un e-commerce prueba un botón de compra nuevo (versión B) contra el actual (versión A), mostrando cada versión a una mitad de los visitantes. B convirtió un poco más, pero el equipo de producto no quiere festejar una diferencia que podría ser casualidad. Un intervalo de confianza para la diferencia responde si la mejora es real.

Datos de entrada

Formato: bloques separados por "?": dos líneas "versión visitas compras".

Ejemplo

```
A 5000 250
B 5000 310
?
A 800 40
B 800 46
```

Restricciones: hasta 10^9 visitas por versión.

Tu programa debe

1. Calcular la tasa de conversión de cada versión: $\text{compras} / \text{visitas}$.
2. Calcular el error estándar de la diferencia: $\sqrt{p_A(1 - p_A)/n_A + p_B(1 - p_B)/n_B}$.
3. Armar el intervalo de confianza del 95 %: $\text{diferencia} \pm 1,96 \times \text{error estándar}$.
4. Concluir: si el intervalo no incluye el 0, la diferencia es significativa. Mostrar los porcentajes con 2 decimales.

Salida esperada

```
5000 y 5000 visitas: A 5.00 % | B 6.20 % | diferencia 1.20 puntos | IC 95 % [0.30; 2.10] -> B
vende más (el intervalo no incluye el 0)
800 y 800 visitas: A 5.00 % | B 5.75 % | diferencia 0.75 puntos | IC 95 % [-1.46; 2.96] -> no se
puede concluir (el intervalo incluye el 0)
```

Reglas

- Complejidad: $O(1)$ por prueba.
- No funcional: reproducibilidad: el análisis se define antes de mirar los datos (tamaño de muestra y umbral); mirar todos los días y cortar cuando conviene infla los falsos positivos.
- No funcional: la aproximación normal es válida con muestras grandes (más de 10 compras y 10 no compras por versión); si no, informalo.
- Porcentajes con 2 decimales y punto decimal.

Pista (leela solo si te trabás): la misma diferencia (del 5 % al 6,2 %) puede ser concluyente con 5.000 visitas y pura casualidad con 800: el error estándar se achica con la raíz del tamaño de la muestra.

Bonus (opcional)

- Calculá cuántas visitas por versión harían falta para detectar una mejora de 1 punto con un 80 % de potencia.
- Resuelvelo con una prueba de permutaciones con semilla (sin fórmulas) y compará.

Cierre: Las pruebas A/B con intervalos de confianza son la forma estándar de decidir cambios de producto en empresas digitales.

Ejercicio 388: Webhooks de pago que llegan desordenados

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 78/100

Situación: Un procesador de pagos avisa a la tienda los cambios de estado de cada pago con webhooks (creado, autorizado, capturado, reembolsado). Los webhooks pueden llegar desordenados y repetidos. La tienda tiene que aplicar los cambios en el orden correcto, ignorar los duplicados y rechazar transiciones imposibles, sin perder ningún evento.

Datos de entrada

Formato: un webhook por línea en orden de llegada: "pago número_de_secuencia evento" (la secuencia empieza en 1 para cada pago).

Transiciones válidas: (nada) → creado → autorizado → capturado → reembolsado.

Ejemplo

```
pago_1 2 autorizado
pago_1 1 creado
pago_2 1 creado
pago_1 3 capturado
pago_1 3 capturado
pago_2 3 capturado
pago_2 2 autorizado
pago_1 4 reembolsado
pago_3 1 creado
pago_3 2 capturado
```

Restricciones: hasta 10^5 webhooks por segundo y 10^8 pagos.

Tu programa debe

1. Guardar por pago el último número de secuencia aplicado y un buffer de eventos que llegaron antes de tiempo.
2. Si la secuencia ya se aplicó o ya está en el buffer, es un duplicado: ignorarlo.
3. Si no, guardarlo en el buffer y aplicar en orden todos los eventos consecutivos disponibles, validando cada transición (una inválida se rechaza, pero se consume su número).
4. Mostrar qué pasó con cada webhook y los estados finales (pagos en orden alfabético).

Salida esperada

```
pago_1 #2 autorizado: en espera (falta #1)
pago_1 #1 creado: procesa #1 creado, #2 autorizado
pago_2 #1 creado: procesa #1 creado
pago_1 #3 capturado: procesa #3 capturado
pago_1 #3 capturado: duplicado, se ignora
pago_2 #3 capturado: en espera (falta #2)
pago_2 #2 autorizado: procesa #2 autorizado, #3 capturado
pago_1 #4 reembolsado: procesa #4 reembolsado
pago_3 #1 creado: procesa #1 creado
pago_3 #2 capturado: procesa #2 capturado RECHAZADO (transición inválida desde creado)
Estados finales: pago_1=reembolsado, pago_2=capturado, pago_3=creado
```

Reglas

- Complejidad: $O(1)$ amortizado por webhook.
- No funcional: idempotencia: recibir el mismo webhook diez veces tiene el mismo efecto que recibirlo una.
- No funcional: el buffer por pago tiene que tener un límite y un vencimiento: si un evento intermedio nunca llega, hay que consultarle el estado al procesador en lugar de esperar para siempre.
- Casos borde: evento con transición inválida (se registra y se rechaza).

Pista (leela solo si te trabás): los números de secuencia convierten el desorden en un problema de "huecos": aplicás mientras el siguiente número esté disponible y guardás el resto hasta que el hueco se llene.

Bonus (opcional)

- Si un pago lleva más de 5 minutos con un hueco, generá una tarea de conciliación contra la API del procesador.
- Guardá los webhooks crudos en un registro de solo agregado para poder reprocesarlos.

Cierre: Procesar webhooks desordenados y duplicados es una tarea cotidiana de cualquier integración con pasarelas de pago.

Ejercicio 389: El caché de dos niveles del CDN

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Telecomunicaciones · Dificultad 78/100

Situación: Una red de distribución de contenidos (CDN) sirve videos desde un caché chico en cada ciudad (borde) y uno más grande por región. Si el video no está en el borde, se busca en el regional; si tampoco, en el servidor de origen, que es caro y está lejos. Al traer un video, se guarda en los niveles por los que pasó.

Datos de entrada

Formato: primera línea: capacidad del borde y del regional (en cantidad de videos, ambos LRU); después "video tamaño_MB" por pedido.

Ejemplo

```
2 4
video_a 50
video_b 30
video_a 50
video_c 80
video_d 20
video_a 50
video_b 30
video_e 60
video_c 80
video_a 50
```

Restricciones: hasta 10^9 pedidos por día y 10^7 videos.

Tu programa debe

1. Buscar en el borde: si está, acierto de borde.
2. Si no, buscar en el regional: si está, acierto regional y copiarlo al borde.
3. Si no, traerlo del origen (sumar su tamaño) y guardarlo en el regional y en el borde.
4. Mostrar el nivel que atendió cada pedido, los contadores, los MB desde el origen y la tasa de aciertos (1 decimal).

Salida esperada

```
video_a: ORIGEN
video_b: ORIGEN
video_a: borde
video_c: ORIGEN
video_d: ORIGEN
video_a: regional
video_b: regional
video_e: ORIGEN
video_c: ORIGEN
video_a: regional
Aciertos borde 1, regional 3, origen 6 de 10 | MB desde el origen: 320 | tasa de aciertos total
40.0 %
```

Reglas

- Complejidad: $O(1)$ por pedido.
- Usar un video (en cualquier nivel donde esté presente y se lo encuentre) lo vuelve el más reciente de ese nivel.
- No funcional: costo: el objetivo es minimizar los MB desde el origen, que es lo que se paga.
- No funcional: describí cómo invalidarías un video en todos los niveles si el origen lo actualiza.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): cada nivel es un caché LRU independiente; la jerarquía solo define en qué orden se consultan y dónde se guarda lo que se trae.

Bonus (opcional)

- Hacé que el borde admita un video solo si fue pedido al menos 2 veces (filtro de admisión) y medí el efecto.
- Usá capacidad en MB en lugar de cantidad de videos.

Cierre: Las jerarquías de cachés son la base de las CDNs, de los procesadores (L1, L2, L3) y de los sistemas de archivos distribuidos.

Ejercicio 390: Motor de reglas para becas

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Educación · Dificultad 78/100

Situación: Una fundación educativa otorga becas con reglas que cambian cada año, y no quiere tocar código cada vez. Las reglas se escriben en un archivo con una prioridad, condiciones y un resultado; el motor las evalúa en orden de prioridad y aplica la primera que se cumple. Cada decisión tiene que poder explicarse al postulante.

Datos de entrada

Formato: primera parte: una regla por línea "prioridad condición condición ... -> resultado" (condiciones del tipo campo>=valor, campo<=valor, campo<valor, campo>valor o campo=valor; todas deben cumplirse; sin condiciones es la regla por defecto); una línea "?" y los postulantes "nombre campo=valor ...".

Ejemplo

```
10 promedio>=9 ingreso<=500000 -> beca_completa
20 promedio>=8 ingreso<=300000 -> beca_completa
30 promedio>=7 ingreso<=600000 -> media_beca
40 promedio>=9 -> beca_excelencia
50 materias_aprobadas<3 -> rechazo
99 -> rechazo
?
Ana promedio=9.2 ingreso=450000 materias_aprobadas=10
Bruno promedio=8.1 ingreso=250000 materias_aprobadas=6
Carla promedio=9.5 ingreso=900000 materias_aprobadas=12
Diego promedio=7.4 ingreso=700000 materias_aprobadas=2
Elena promedio=6.9 ingreso=100000 materias_aprobadas=8
```

Restricciones: hasta 10^3 reglas y 10^6 postulantes por convocatoria.

Tu programa debe

1. Leer las reglas y ordenarlas por prioridad (menor número primero).
2. Para cada postulante, recorrer las reglas en orden y evaluar sus condiciones con los datos del postulante.
3. Aplicar la primera regla cuyas condiciones se cumplen todas.
4. Mostrar el resultado y la regla que lo decidió (explicación).

Salida esperada

```
Ana: beca_completa (regla 10: promedio>=9 y ingreso<=500000)
Bruno: beca_completa (regla 20: promedio>=8 y ingreso<=300000)
Carla: beca_excelencia (regla 40: promedio>=9)
Diego: rechazo (regla 50: materias_aprobadas<3)
Elena: rechazo (regla 99: por defecto)
```

Reglas

- Complejidad: $O(\text{reglas} \times \text{condiciones})$ por postulante.
- Los valores se comparan como números decimales exactos.
- No funcional: configurabilidad: cambiar las reglas no requiere cambiar el programa.
- No funcional: explicabilidad y auditoría: cada decisión se guarda con la regla aplicada y la versión del archivo de reglas.
- No funcional: tiene que existir una regla por defecto, para que ningún postulante quede sin decisión.

Pista (leela solo si te trabás): separa el "qué" (las reglas, que son datos) del "cómo" (el motor, que solo sabe evaluar condiciones y recorrer en orden).

Bonus (opcional)

- Detectá reglas que nunca pueden aplicarse porque una de mayor prioridad las tapa siempre.
- Agregá condiciones con "o" y agrupadas con paréntesis (reutilizá el analizador de expresiones del nivel 5).

Cierre: Los motores de reglas se usan en seguros, créditos, becas, promociones y cumplimiento normativo, donde las reglas las escriben personas de negocio.

Ejercicio 391: Canciones al azar de un catálogo infinito

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Música · Dificultad 78/100

Situación: Una radio online arma una selección de canciones al azar de un flujo de novedades que no para de llegar y cuyo tamaño total nadie conoce. Guardar todo el flujo para elegir después no es posible. El muestreo por reservorio elige k canciones de forma que cada canción del flujo tenga la misma probabilidad de quedar, usando solo k lugares de memoria.

Datos de entrada

Formato: primera línea: tamaño de la muestra k , semilla y cantidad de temas; segunda: los temas en orden de llegada.

Generador: $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$, un nuevo x por cada tema a partir del $k + 1$.

Ejemplo

```
3 2026 12
tema01 tema02 tema03 tema04 tema05 tema06 tema07 tema08 tema09 tema10 tema11 tema12
```

Restricciones: flujo potencialmente infinito; k hasta 10^6 .

Tu programa debe

1. Guardar los primeros k temas en el reservorio.
2. Para el tema número i (contando desde 0, con $i \geq k$), generar x y calcular $j = x \text{ resto } (i + 1)$.
3. Si $j < k$, reemplazar el tema de la posición j del reservorio por el nuevo; si no, descartarlo.
4. Mostrar los reemplazos (numerando los temas desde 1), la muestra final y la probabilidad teórica k / n .

Salida esperada

```
tema 7: j = 0 -> reemplaza a tema01 por tema07
tema 9: j = 0 -> reemplaza a tema07 por tema09
Muestra final: tema09 tema02 tema03
Probabilidad de cada tema de estar en la muestra: 3/12 = 0.2500
```

Reglas

- Complejidad: $O(1)$ por tema y $O(k)$ de memoria.
- Por qué es justo: el tema i entra con probabilidad $k / (i + 1)$, y cada tema posterior lo "saca" con probabilidad $1 / (i + 2)$, $1 / (i + 3)$...; el producto da k / n para todos.
- No funcional: memoria constante ($O(k)$) sin importar el largo del flujo.
- No funcional: reproducibilidad: con la misma semilla, la misma muestra.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): al principio cada tema entra seguro; a medida que avanza el flujo, cada tema nuevo tiene menos chance de entrar, justo en la proporción que compensa haber llegado tarde.

Bonus (opcional)

- Corré el algoritmo 10.000 veces con semillas distintas y verificá que cada tema aparece cerca de k/n de las veces.
- Hacé un muestreo ponderado: temas con más reproducciones tienen más chance de quedar.

Cierre: El muestreo por reservorio se usa en monitoreo de tráfico, análisis de logs y cualquier sistema que tenga que resumir flujos enormes.

Ejercicio 392: Órdenes iceberg en el libro

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 79/100

Situación: Un fondo quiere vender una cantidad grande de acciones sin asustar al mercado mostrando todo su volumen. Usa una orden "iceberg": solo muestra una parte (el pico) y, cuando esa parte se ejecuta, repone otra porción visible. Para ser justos con los demás, cada reposición pierde la prioridad de tiempo y va al final de la fila de su precio.

Datos de entrada

Formato: órdenes en orden de llegada: "id venta precio cantidad [pico]" (con pico, es iceberg) o "id compra precio cantidad" (las compras se ejecutan contra el libro y lo que no se ejecuta se descarta, para simplificar).

Ejemplo

```
I1 venta 100 30 10
L1 venta 100 15
C1 compra 100 12
C2 compra 100 20
C3 compra 100 25
```

Restricciones: hasta 10^6 órdenes por segundo.

Tu programa debe

1. Guardar cada orden de venta en el libro con su parte visible; para las iceberg, guardar aparte la cantidad oculta y el pico.
2. Ejecutar cada compra contra las ventas visibles por prioridad precio-tiempo.
3. Cuando la parte visible de una iceberg se agota y le queda cantidad oculta, reponer mínimo(pico, oculta) con un nuevo tiempo de llegada (al final de la fila).
4. Mostrar las operaciones, las reposiciones y el libro final (con las cantidades ocultas).

Salida esperada

```
I1 vende 30 @ 100 (iceberg, visible 10)
L1 vende 15 @ 100
C1 compra 10 @ 100 (contra I1)
I1 repone 10 visibles (quedan 10 ocultas) y pasa al final de la fila
C1 compra 2 @ 100 (contra L1)
C2 compra 13 @ 100 (contra L1)
C2 compra 7 @ 100 (contra I1)
C3 compra 3 @ 100 (contra I1)
I1 repone 10 visibles (quedan 0 ocultas) y pasa al final de la fila
C3 compra 10 @ 100 (contra I1)
C3: 12 sin ejecutar
Libro de ventas: vacío
```

Reglas

- Complejidad: $O(\log n)$ por ejecución y por reposición.
- No funcional: justicia: una reposición nunca puede adelantarse a órdenes que llegaron antes al mismo precio.
- No funcional: confidencialidad: la cantidad oculta no se muestra en los datos públicos del libro (acá se muestra solo para verificar).
- Casos borde: una compra que consume la parte visible exacta.

Pista (leela solo si te trabás): una iceberg se comporta como muchas órdenes chicas que se van creando de a una: cada "pedazo" nuevo entra al libro como si recién llegara.

Bonus (opcional)

- Hacé que el pico sea variable (al azar con semilla entre 8 y 12) para que sea más difícil detectar la iceberg.
- Mostrá el libro público agregado por precio sin revelar nada oculto.

Cierre: Las órdenes iceberg son habituales en bolsas y exchanges para operar volúmenes grandes sin mover el precio.

Ejercicio 393: Anomalías en la telemetría del satélite

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Ciencia · Dificultad 79/100

Situación: Un satélite de observación manda cada segundo la temperatura de sus paneles. El centro de control quiere detectar lecturas anómalas en tiempo real: una lectura es sospechosa si se aleja más de z desvíos de la media de las últimas W lecturas normales. Las lecturas anómalas no entran a la ventana, para no contaminar la referencia.

Datos de entrada

Formato: primera línea: tamaño de la ventana W y umbral z ; segunda: las lecturas en orden.

Ejemplo

```
5 3
20.1 20.3 19.9 20.2 20.0 20.4 27.5 20.1 19.8 20.3 20.2 12.0 20.0
```

Restricciones: hasta 10^6 lecturas por segundo por sensor; W hasta 10^4 .

Tu programa debe

1. Mantener una ventana con las últimas W lecturas normales.
2. Mientras la ventana no esté llena, aceptar las lecturas sin evaluar.
3. Con la ventana llena, calcular la media y el desvío estándar poblacional; el puntaje z es $(\text{lectura} - \text{media}) / \text{desvío}$.
4. Si $|z| > \text{umbral}$, marcar la anomalía y no agregarla; si no, agregarla y sacar la más vieja. Mostrar cada anomalía y el total.

Salida esperada

```
lectura 7 = 27.5: ANOMALÍA (media 20.16, desvío 0.185, z = 39.57)
lectura 12 = 12.0: ANOMALÍA (media 20.16, desvío 0.206, z = -39.63)
Anomalías: 2 de 13 lecturas
```

Reglas

- Complejidad: $O(W)$ por lectura recalculando; $O(1)$ manteniendo suma y suma de cuadrados (bonus).
- Redondeos: media con 2 decimales, desvío con 3 y z con 2 (0,5 hacia arriba).
- No funcional: tiempo real: la decisión se toma antes de que llegue la siguiente lectura.
- No funcional: estabilidad numérica: la fórmula "suma de cuadrados – cuadrado de la suma" puede dar varianzas negativas por redondeo; usá decimales exactos o el algoritmo de Welford.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): si las lecturas anómalas entraran a la ventana, inflarían el desvío y la siguiente anomalía pasaría inadvertida.

Bonus (opcional)

- Mantené la media y la varianza en $O(1)$ por lectura, con actualización al entrar y salir de la ventana.
- Si hay 3 anomalías seguidas en la misma dirección, tratálas como un cambio de régimen: reiniciá la ventana.

Cierre: La detección de anomalías con puntajes z móviles es la primera línea de monitoreo en telemetría espacial, industrial y de servidores.

Ejercicio 394: Registro de accesos que no se puede editar

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Seguridad informática · Dificultad 79/100

Situación: Un sistema de salud tiene que guardar quién accedió a cada historia clínica, y la ley exige que ese registro no pueda modificarse sin que se note. Cada entrada nueva guarda un hash calculado sobre el hash de la anterior y su propio contenido: una cadena de hashes. Si alguien cambia una entrada vieja, su hash deja de coincidir y la cadena "se rompe" desde ahí.

Datos de entrada

Formato: primera parte: las entradas del registro en orden; una línea "?" y una manipulación "modificar posición nuevo_contenido".

$\text{hash}(i) = H(\text{hash}(i-1) + "|" + \text{entrada}(i))$, con $\text{hash}(0) = "0"$ (los hashes se concatenan como texto decimal) y $H = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

(En producción se usa una función de hash criptográfica; acá se usa una simple para poder calcularla a mano.)

Ejemplo

```
admin login 08:00
ana ver_historia_123 08:05
beto editar_turno_77 08:10
ana ver_historia_456 08:12
admin exportar_reporte 08:30
?
modificar 3 beto ver_turno_77 08:10
```

Restricciones: hasta 10^9 entradas; verificación incremental.

Tu programa debe

1. Calcular la cadena de hashes del registro original.
2. Aplicar la manipulación sobre una copia del registro (sin tocar los hashes guardados).
3. Verificar: recorrer la copia recalculando cada hash a partir del hash guardado de la entrada anterior y compararlo con el guardado.
4. Mostrar la cadena original y la primera entrada donde falla la verificación.

Salida esperada

```
Registro original:
1. admin login 08:00 | hash 3754091715
2. ana ver_historia_123 08:05 | hash 3549476643
3. beto editar_turno_77 08:10 | hash 1299420858
4. ana ver_historia_456 08:12 | hash 3786545898
5. admin exportar_reporte 08:30 | hash 3562207894
Alguien modifica la entrada 3 -> verificación: falla en la entrada 3 (hash esperado 1299420858,
calculado 2354853788)
```

Reglas

- Complejidad: $O(n)$ para verificar todo; $O(1)$ por entrada nueva.
- No funcional: solo agregado: el sistema no ofrece ninguna operación para editar o borrar entradas.
- No funcional: el último hash se publica periódicamente fuera del sistema (por ejemplo, se envía a un tercero), para que ni siquiera un administrador pueda rehacer toda la cadena sin que se note.
- Casos borde: manipulación de la última entrada.

Pista (leela solo si te trabás): cada hash "sella" todo lo anterior, porque incluye el hash previo. Cambiar una entrada obliga a recalcular todos los hashes siguientes, y eso se detecta si el último hash está guardado en otro lado.

Bonus (opcional)

- Agrupá las entradas en un árbol de Merkle para probar que una entrada está en el registro sin revelar las demás.
- Verificá solo las entradas agregadas desde la última verificación (incremental).

Cierre: Las cadenas de hashes son la base de los registros de auditoría inalterables y de las cadenas de bloques.

Ejercicio 395: La espera en los boxes de atención

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Salud · Dificultad 79/100

Situación: La guardia de un centro de salud quiere decidir cuántos boxes de atención abrir en el turno noche. Con uno solo, la espera se dispara; con muchos, quedan vacíos. En lugar de adivinar, simulan la llegada de pacientes con un generador aleatorio con semilla (para que la simulación sea reproducible) y comparan distintas cantidades de boxes con los mismos pacientes.

Datos de entrada

Formato: primera línea: cantidad de pacientes, semilla, máximo entre llegadas y un factor de duración; segunda: las cantidades de boxes a comparar.

Para cada paciente: $x = \text{siguiente}(x)$; $\text{llegada} = \text{llegada anterior} + 1 + (x \text{ resto máximo_entre_llegadas})$; $x = \text{siguiente}(x)$; $\text{duración} = 5 + (x \text{ resto } (\text{factor} \times 3))$. Generador: $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$, arrancando en la semilla.

Ejemplo

```
20 2026 7 5
1 2 3
```

Restricciones: hasta 10^6 pacientes y 100 boxes en la simulación.

Tu programa debe

1. Generar los pacientes (llegada y duración) una sola vez, para comparar todas las configuraciones con los mismos datos.
2. Para cada cantidad de boxes, atender a los pacientes por orden de llegada en el box que quede libre primero (en empate, el de menor número).
3. Registrar la espera de cada paciente y el tiempo total ocupado.
4. Mostrar la espera promedio (1 decimal), la máxima y la ocupación = tiempo ocupado / (boxes × fin de la simulación) (1 decimal).

Salida esperada

```
Pacientes simulados: 20 (último llega en el minuto 90)
1 box(es): espera promedio 77.2 min | máxima 137 min | ocupación 99.2 %
2 box(es): espera promedio 18.2 min | máxima 29 min | ocupación 91.0 %
3 box(es): espera promedio 1.5 min | máxima 7 min | ocupación 74.6 %
```

Reglas

- Complejidad: $O(\text{pacientes} \times \text{boxes})$ ($O(\text{pacientes} \times \log \text{boxes})$ con un montículo).
- No funcional: reproducibilidad: la semilla y el generador están especificados para que cualquier implementación obtenga los mismos números.
- No funcional: validez: las configuraciones se comparan con la misma secuencia de pacientes ("números aleatorios comunes").
- El fin de la simulación es el momento en que termina el último paciente.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): primero generá todos los pacientes y guardalos; después simulá cada configuración sobre la misma lista. Si generaras pacientes distintos para cada una, la comparación mezclaría suerte con capacidad.

Bonus (opcional)

- Repetí la simulación con 100 semillas y calculá el promedio y el percentil 90 de la espera para cada configuración.
- Agregá prioridad por gravedad (triage): los pacientes graves pasan adelante.

Cierre: La simulación de eventos discretos con semilla se usa para dimensionar guardias, cajas de supermercado, call centers y servidores.

Ejercicio 396: Riego limitado por sensores

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Agricultura · Dificultad 79/100

Situación: Un sistema de riego automático abre la válvula cuando los sensores marcan poca humedad y la cierra cuando el suelo se recupera. Si usara un solo umbral, la válvula se abriría y cerraría decenas de veces por hora con cada pequeña variación y se rompería. Por eso usa dos umbrales (histéresis), un tiempo mínimo entre activaciones y un máximo de activaciones por hora.

Datos de entrada

Formato: primera línea: umbral para abrir (%), umbral para cerrar (%), segundos mínimos entre activaciones y máximo de activaciones por hora; después, "segundo humedad" por lectura.

Ejemplo

```
30 45 600 3
0 42
300 29
400 33
700 46
800 28
1000 27
1500 38
2000 47
2100 25
2500 46
3200 24
3700 23
4000 22
```

Restricciones: hasta 10^4 válvulas y una lectura por segundo por sensor.

Tu programa debe

1. Llevar el estado de la válvula (cerrada o regando), la hora de la última activación y una cola con las activaciones de la última hora.
2. Con la válvula cerrada y humedad menor al umbral de apertura, abrir solo si pasaron los segundos mínimos y no se alcanzó el máximo por hora; si no, informar el motivo.
3. Con la válvula abierta y humedad mayor al umbral de cierre, cerrar.
4. Mostrar la acción de cada lectura.

Salida esperada

```
t=0 humedad 42 %: sin cambios (cerrada)
t=300 humedad 29 %: ABRE la válvula
t=400 humedad 33 %: sin cambios (regando)
t=700 humedad 46 %: CIERRA la válvula
t=800 humedad 28 %: quiere regar, pero pasaron 500 s desde la última activación (mín 600)
t=1000 humedad 27 %: ABRE la válvula
t=1500 humedad 38 %: sin cambios (regando)
t=2000 humedad 47 %: CIERRA la válvula
t=2100 humedad 25 %: ABRE la válvula
t=2500 humedad 46 %: CIERRA la válvula
t=3200 humedad 24 %: quiere regar, pero ya hubo 3 activaciones en la última hora
t=3700 humedad 23 %: quiere regar, pero ya hubo 3 activaciones en la última hora
t=4000 humedad 22 %: ABRE la válvula
```

Reglas

- Complejidad: $O(1)$ amortizado por lectura.
- Las activaciones de la última hora son las de $(t - 3.600, t]$.
- No funcional: protección del hardware: los límites de activación tienen prioridad sobre la humedad.

- No funcional: si un sensor deja de reportar más de 10 minutos, la válvula debe cerrarse por seguridad (describí cómo lo agregarías).

Pista (leela solo si te trabás): la histéresis crea una "zona muerta" entre 30 % y 45 % donde la válvula mantiene lo que venía haciendo. Eso por sí solo elimina el parpadeo; los límites de tiempo protegen de los casos extremos.

Bonus (opcional)

- Agregá el cierre por falta de lecturas y un registro de alarmas.
- Ajustá los umbrales según la hora del día (regar de noche pierde menos agua por evaporación).

Cierre: La histéresis y la limitación de activaciones son técnicas básicas de control en termostatos, riego, bombas y sistemas industriales.

Ejercicio 397: Servidores de partidas que se caen

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Videojuegos · Dificultad 79/100

Situación: Un juego online corre cada partida en un servidor. Los servidores mandan un "latido" cada pocos segundos; si un servidor deja de latir por más de T segundos, se lo considera caído y sus partidas se mudan a los servidores vivos con menos carga (los jugadores se reconectan con el último estado guardado). Un servidor que vuelve arranca vacío.

Datos de entrada

Formato: primera línea: T en segundos; después "servidor partidas..."; una línea "?" y eventos en orden: "segundo servidor" (latido) o "segundo control" (revisión periódica).

Ejemplo

```
10
srv_a partida1 partida2 partida3
srv_b partida4 partida5
srv_c partida6
?
0 srv_a
0 srv_b
0 srv_c
5 srv_a
6 srv_c
8 srv_a
12 srv_c
15 srv_a
16 srv_c
19 srv_a
21 control
26 srv_c
30 srv_b
32 control
```

Restricciones: hasta 10^4 servidores y 10^6 partidas.

Tu programa debe

1. Guardar el último latido de cada servidor.
2. En cada control, marcar como caído a todo servidor vivo cuyo último latido sea de hace más de T segundos (revisándolos en el orden de la entrada).
3. Reasignar cada partida del servidor caído al servidor vivo con menos partidas (en empate, el de menor nombre), de a una.
4. Mostrar las caídas, las reasignaciones y la carga de todos los servidores en cada control.

Salida esperada

```
t=21: srv_b sin latido hace 21 s -> caído
partida4 -> srv_c
partida5 -> srv_c
t=21: carga srv_a=3, srv_b=0 (caído), srv_c=3
t=30: srv_b vuelve (sin partidas asignadas)
t=32: srv_a sin latido hace 13 s -> caído
partida1 -> srv_b
partida2 -> srv_b
partida3 -> srv_b
t=32: carga srv_a=0 (caído), srv_b=3, srv_c=3
```

Reglas

- Complejidad: $O(\text{servidores})$ por control y $O(\text{servidores})$ por partida reasignada ($O(\log \text{servidores})$ con un montículo).

- No funcional: disponibilidad: ninguna partida puede quedar asignada a un servidor caído después de un control.
- No funcional: un latido perdido por la red no debería tumbar un servidor sano: por eso T es varias veces el intervalo entre latidos (describí el compromiso entre detectar rápido y evitar falsas alarmas).
- Casos borde: servidor que vuelve después de caer (arranca vacío).

Pista (leela solo si te trabás): el control no necesita recibir un aviso de caída: la ausencia de latidos es la señal. Y repartir de a una partida "al menos cargado" mantiene el balance.

Bonus (opcional)

- Evitá el efecto dominó: si un servidor quedaría con más del doble del promedio, repartí también entre servidores de otra región.
- Cuando un servidor vuelve, migrá partidas nuevas hacia él de forma gradual.

Cierre: Los latidos y la reasignación automática son la base de la tolerancia a fallos en juegos online, bases de datos replicadas y orquestadores de contenedores.

Ejercicio 398: Desafío de nivel: Plataforma de pagos con reintentos y antifraude

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 80/100

Situación: Desafío de nivel. Una plataforma de pagos combina todo lo del nivel: cada pedido trae una clave de idempotencia; si la clave es nueva, pasa por el motor antifraude; si lo aprueba, se envía al procesador, que puede fallar temporalmente y se reintenta con espera exponencial y jitter determinista. El resultado final de cada clave se guarda para responder igual a los reintentos del cliente.

Datos de entrada

Formato: primera línea: espera base (ms), máximo de reintentos y semilla; después, un pedido por línea: "segundo clave tarjeta monto país respuestas_del_procesador..." (T = error temporal, ok = éxito; si faltan, T).

Antifraude: monto > \$500.000 → rechazo; compra de la misma tarjeta en otro país hace menos de 60 s → rechazo (el historial registra todos los pedidos que pasan la idempotencia). Espera antes del reintento $i + 1$ (i desde 0): $\text{techo} = \text{base} \times 2^i$; $\text{espera} = \text{techo} / 2 + (x \text{ resto } (\text{techo} / 2 + 1))$, con $x = \text{siguiente}(x)$, generador $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$ compartido entre todos los pedidos.

Ejemplo

```
100 3 7
0 K1 tarjeta_A 20000 AR T T ok
1 K2 tarjeta_B 900000 AR ok
2 K1 tarjeta_A 20000 AR ok
3 K3 tarjeta_A 5000 BR ok
4 K4 tarjeta_C 7000 AR T T T
5 K5 tarjeta_A 3000 AR ok
6 K2 tarjeta_B 900000 AR ok
```

Restricciones: hasta 10^4 pagos por segundo.

Tu programa debe

1. Idempotencia: si la clave ya existe, responder con el resultado guardado (mismos datos) o con error 422 (datos distintos), sin evaluar nada más.
2. Antifraude: evaluar las reglas; un rechazo también se guarda como resultado de la clave.
3. Procesador: intentar hasta obtener ok o agotar los reintentos, calculando las esperas con el generador compartido.
4. Guardar el resultado de la clave, mostrar cada respuesta y el total cobrado.

Salida esperada

```
s=0 K1: APROBADO en el intento 3 (esperas 65 100 ms)
s=1 K2: RECHAZADO por antifraude (monto alto)
s=2 K1: duplicado -> APROBADO en el intento 3 (esperas 65 100 ms) (sin reprocesar)
s=3 K3: RECHAZADO por antifraude (cambio de país)
s=4 K4: FALLIDO tras 4 intentos (esperas 90 185 332 ms)
s=5 K5: RECHAZADO por antifraude (cambio de país)
s=6 K2: duplicado -> RECHAZADO por antifraude (monto alto) (sin reprocesar)
Total cobrado: $20000
```

Caso 2 (borde)

Sin reintentos permitidos y con el procesador caído:

```
100 0 1
0 K9 tarjeta_Z 1000 AR T
```

Salida esperada del caso 2:

```
s=0 K9: FALLIDO tras 1 intento  
Total cobrado: $0
```

Reglas

- Complejidad: $O(1)$ amortizado por pedido más los reintentos.
- El orden de las etapas importa: idempotencia → antifraude → procesador. Un duplicado no puede volver a pasar por antifraude ni por el procesador.
- No funcional: exactamente un cobro por clave aunque el cliente reintente (idempotencia) y aunque el procesador falle (reintentos acotados).
- No funcional: determinismo total: la misma entrada produce la misma salida (semilla y orden especificados).
- No funcional: cada decisión queda registrada con su motivo (antifraude) o sus esperas (procesador).

Pista (leela solo si te trabás): armá cada etapa como una función separada y probala sola (ya lo hiciste en ejercicios anteriores); el desafío es componerlas en el orden correcto y guardar el resultado final bajo la clave.

Bonus (opcional)

- Si el procesador falla tras todos los reintentos, mandá el pago a una cola de fallidos para reintentarlo más tarde sin bloquear al cliente.
- Agregá un circuit breaker delante del procesador que corte si fallan 5 pagos seguidos.

Cierre: Las plataformas de pago reales encadenan exactamente estas capas; un error en el orden produce cobros dobles o fraudes aprobados.

Ejercicio 399: Desafío de nivel: La ticketera del recital masivo

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Música · Dificultad 80/100

Situación: Desafío de nivel. La ticketera de un recital masivo abre la venta y en el primer segundo llegan millones de pedidos, muchos de bots. El sistema combina un limitador de pedidos por usuario, reglas de negocio (una reserva activa por persona, máximo 4 entradas) y reservas con vencimiento que devuelven entradas si no se pagan a tiempo. Nunca puede venderse una entrada dos veces.

Datos de entrada

Formato: primera línea: entradas disponibles, límite de pedidos por usuario por segundo y duración de la reserva (s); después eventos en orden: "segundo usuario cantidad" (pedido) o "segundo pago usuario".

Ejemplo

```
8 2 600
0 ana 2
0 ana 1
0 ana 1
0 beto 3
1 caro 2
1 ana 1
2 dani 5
5 pago beto
300 eli 2
700 pago caro
700 fede 3
701 pago dani
1300 gabi 2
```

Restricciones: hasta 10^7 pedidos en el primer minuto y 10^5 entradas.

Tu programa debe

1. Antes de cada evento, vencer las reservas cuyo vencimiento llegó (con una cola de prioridad).
2. Para cada pedido: primero el limitador (ventana fija por segundo: cada pedido cuenta, incluso si después se rechaza por otra regla); después, las reglas de negocio; por último, el stock.
3. Si todo pasa, reservar; un pago confirma la reserva si sigue activa.
4. Mostrar cada evento y el estado final (vendidas, reservadas y libres).

Salida esperada

```
t=0 ana reserva 2 | libres 6
t=0 ana: ya tiene una reserva activa
t=0 ana: DEMASIADOS PEDIDOS (límite 2 por segundo)
t=0 beto reserva 3 | libres 3
t=1 caro reserva 2 | libres 1
t=1 ana: ya tiene una reserva activa
t=2 dani: máximo 4 entradas por persona
t=5 beto paga 3
t=300 eli pide 2: AGOTADO (libres 1)
  (t=600: vence la reserva de ana, vuelven 2)
  (t=601: vence la reserva de caro, vuelven 2)
t=700 caro: la reserva no existe o venció
t=700 fede reserva 3 | libres 2
t=701 dani: la reserva no existe o venció
  (t=1300: vence la reserva de fede, vuelven 3)
t=1300 gabi reserva 2 | libres 3
Final: vendidas 3, reservadas 2, libres 3
```

Reglas

- Complejidad: $O(\log n)$ por pedido.

- No funcional: consistencia: vendidas + reservadas + libres = entradas iniciales en todo momento.
- No funcional: protección: el limitador va primero para que los bots no consuman recursos de las etapas caras.
- No funcional: en un sistema real con muchos servidores, la reserva de stock tiene que ser atómica (describí con qué mecanismo de los vistos en el nivel lo harías).
- Una reserva vence exactamente a los "duración" segundos.

Pista (leela solo si te trabás): pensalo como un embudo: cada etapa descarta lo que puede con el menor costo posible, y solo lo que pasa todas llega a tocar el stock.

Bonus (opcional)

- Agregá una fila virtual: cuando no hay stock libre pero sí reservas pendientes, el usuario queda en espera y recibe las entradas que venzan.
- Detectá bots: usuarios que chocan contra el limitador más de 10 veces en un minuto quedan bloqueados 1 hora.

Cierre: Las ticketeras de eventos masivos son un caso extremo de sistemas con picos de tráfico, concurrencia y equidad.

Ejercicio 400: Desafío de nivel: Un exchange de criptomonedas en miniatura

Nivel 8: Senior II · Algoritmos de sistemas · Temática: Finanzas y FinTech · Dificultad 80/100

Situación: Desafío de nivel. Un exchange de criptomonedas en miniatura tiene saldos en pesos y en una crypto, un libro de órdenes con prioridad precio-tiempo, y guarda todo como eventos para poder reconstruir el estado en cualquier versión. Las pantallas consultan constantemente el "tope del libro" (mejor compra y mejor venta), que se mantiene en caché. Las órdenes bloquean los fondos necesarios al entrar.

Datos de entrada

Formato: eventos en orden (la versión es el número de línea): "usuario deposita_pesos monto", "usuario deposita_crypto cantidad", "usuario compra precio cantidad" o "usuario venta precio cantidad"; una línea "?" y una versión a reconstruir.

Una compra bloquea precio $\times$ cantidad pesos (si no alcanzan, se rechaza); una venta bloquea la crypto. Las operaciones se hacen al precio de la orden del libro; si un comprador paga menos que su límite, la diferencia vuelve a sus pesos disponibles.

Ejemplo

```
ana deposita_pesos 1000000
beto deposita_crypto 10
caro deposita_pesos 500000
beto venta 95000 4
ana compra 96000 3
caro compra 94000 2
beto venta 94000 5
ana compra 100000 2
?
5
```

Restricciones: hasta 10^6 órdenes por segundo y 10^7 usuarios.

Tu programa debe

1. Aplicar los eventos en orden: depósitos, y órdenes que primero validan y bloquean fondos y después cruzan contra el libro.
2. En cada operación, transferir la crypto al comprador y los pesos al vendedor, devolviendo la diferencia de precio al comprador.
3. Mantener el tope del libro y mostrarlo junto con los saldos disponibles de cada usuario (en orden alfabético).
4. Reconstruir el estado en la versión pedida volviendo a aplicar los eventos desde cero hasta esa versión, y mostrarlo como pesos/crypto por usuario.

Salida esperada

```
Operaciones:
#5 ana compra 3 @ 95000 a beto
#7 beto vende 2 @ 94000 a caro
#8 ana compra 2 @ 94000 a beto
Tope del libro (lo que se guarda en caché): mejor compra - | mejor venta 94000
ana: pesos disponibles 527000 | crypto 5
beto: pesos disponibles 661000 | crypto 1
caro: pesos disponibles 312000 | crypto 2
Estado reconstruido en la versión 5: ana 715000/3, beto 285000/6, caro 500000/0 | mejor compra - |
mejor venta 95000
```

Reglas

- Complejidad: $O(\log n)$ por orden más $O(1)$ por operación; la reconstrucción es $O(\text{versión})$ (con fotos periódicas sería $O(K)$).

- Los saldos "disponibles" no incluyen lo bloqueado en órdenes abiertas.
- No funcional: conservación: la suma de pesos (disponibles + bloqueados) y de cripto de todos los usuarios solo cambia con depósitos.
- No funcional: el caché del tope del libro se invalida con cada evento que modifica el libro; nunca puede mostrar un precio que ya no está.
- No funcional: determinismo: reconstruir la misma versión dos veces da exactamente el mismo estado.

Pista (leela solo si te trabás): bloquear fondos al crear la orden garantiza que toda operación futura pueda liquidarse; el resto es el libro de órdenes que ya construiste, más la contabilidad de cada operación.

Bonus (opcional)

- Agregá cancelaciones que desbloquean los fondos restantes de la orden.
- Guardá fotos del estado cada 1.000 eventos y reconstruí cualquier versión en $O(1.000)$.

Cierre: Este es, en pequeño, el núcleo de cualquier exchange: libro de órdenes, contabilidad sin errores y un historial reproducible para auditar.

Lote 17: ejercicios 401 a 425

Nivel 9

Senior III · Matemática aplicada

Ejercicios 401 a 450

Qué vas a practicar

Matemática aplicada: strings avanzados, flujo, emparejamiento, geometría, métodos numéricos y álgebra lineal.

Qué se espera al terminar

Traducir problemas reales a modelos matemáticos y resolverlos con precisión.

Conceptos del nivel

KMP, bisección, flujo máximo, matching bipartito, envolvente convexa, Bellman-Ford, ciclos negativos, Floyd-Warshall, rolling hash, Newton-Raphson, área de polígonos, función Z, corte mínimo, exponenciación de matrices, eliminación gaussiana, punto en polígono, suffix array, LCP, intersección de segmentos, derivadas, ventanas, detección de ciclos negativos, par de puntos más cercano, divide y vencerás, cadenas de Markov, flujo máximo con capacidades, funciones no lineales, KMP sobre diferencias, barrido, área de unión de rectángulos, álgebra lineal, bootstrapping de tasas, cierre transitivo, matching bipartito de máxima cardinalidad, rotación lexicográfica mínima, física, barrido con árbol de segmentos, algoritmo de Manacher, flujo con demandas, geometría....

Desafíos de nivel

- 448. La mesa de dinero
- 449. Plataforma de logística urbana
- 450. Detector de fraude documental

Ejercicio 401: Una cláusula en miles de contratos

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 81/100

Situación: Un estudio jurídico que asesora a una fintech tiene que encontrar una cláusula en decenas de miles de contratos digitalizados. La búsqueda ingenua vuelve atrás en el texto cada vez que una coincidencia parcial falla, y con textos largos y patrones repetitivos se vuelve cuadrática. El algoritmo KMP nunca retrocede en el texto: usa lo que ya sabe del patrón.

Datos de entrada

Formato: primera línea: el patrón; segunda: cantidad de textos; después "nombre: texto" por línea (se distinguen mayúsculas y espacios).

Ejemplo

```
la parte vendedora
3
contrato_A: el comprador y la parte vendedora acuerdan que la parte vendedora entrega
contrato_B: la parte compradora abona a la parte vend edora el saldo
contrato_C: la parte vendedora la parte vendedoral la parte vendedora
```

Restricciones: patrones de hasta 10^5 caracteres; hasta 10^9 caracteres de texto en total.

Tu programa debe

1. Calcular la función de prefijos del patrón: $pi[i] = \text{largo del prefijo propio más largo de patrón}[0..i]$ que también es sufijo de él.
2. Recorrer cada texto una sola vez con un puntero k al patrón: si el carácter coincide, avanzar k ; si no, retroceder k a $pi[k - 1]$ (sin mover el texto).
3. Cada vez que k llega al largo del patrón, registrar una aparición y seguir desde $pi[k - 1]$ (así se detectan superpuestas).
4. Mostrar la función de prefijos, las posiciones (desde 0) y la cantidad de comparaciones de caracteres.

Salida esperada

```
Función de prefijos de "la parte vendedora": 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
contrato_A: 2 aparición(es) en 15, 47 | comparaciones 74 (texto de 73 caracteres)
contrato_B: 0 aparición(es) en - | comparaciones 60 (texto de 56 caracteres)
contrato_C: 3 aparición(es) en 0, 19, 37 | comparaciones 55 (texto de 55 caracteres)
```

Reglas

- Complejidad esperada: $O(n + m)$; la cantidad de comparaciones es a lo sumo $2n$.
- Se cuenta una comparación por cada vez que se compara un carácter del texto con uno del patrón, incluidas las que fallan y hacen retroceder.
- No funcional: procesamiento en flujo: el texto se lee una sola vez y en orden, así que puede venir de un archivo enorme sin cargarlo entero en memoria.
- No funcional: la función de prefijos se calcula una vez por patrón y se reutiliza en todos los contratos.

Pista (leela solo si te trabás): cuando una coincidencia falla después de haber igualado k caracteres, esos k caracteres son un prefijo del patrón: ya sabés qué dicen. La función de prefijos te dice cuánto de eso sigue sirviendo.

Bonus (opcional)

- Buscá varias cláusulas a la vez con Aho-Corasick (nivel 7) y compará.
- Normalizá el texto (minúsculas, espacios múltiples) antes de buscar, para encontrar "parte Vendedora".

Cierre: KMP es uno de los algoritmos clásicos de búsqueda de texto y la base de herramientas de búsqueda, editores y análisis de documentos.

Ejercicio 402: La tasa que esconde el crédito

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 81/100

Situación: Una financiera ofrece préstamos anunciando solo la cuota ("llevate \$1.000.000 en 12 cuotas de \$100.000") y no la tasa. Una asociación de consumidores quiere calcular la tasa efectiva real para comparar ofertas. En el sistema francés, la cuota depende de la tasa con una fórmula que no se puede despejar: hay que encontrar la tasa numéricamente.

Datos de entrada

Formato: una línea por oferta: "capital cantidad_de_cuotas cuota".

Sistema francés: $\text{cuota} = \text{capital} \times i / (1 - (1 + i)^{-n})$, con i la tasa efectiva mensual (TEM).

Ejemplo

```
1000000 12 100000
500000 24 30000
200000 6 36000
```

Restricciones: hasta 10^6 ofertas; TEM entre 0 % y 100 %.

Tu programa debe

1. Definir $f(i) = \text{capital} \times i / (1 - (1 + i)^{-n}) - \text{cuota}$, que es creciente en i (y vale $\text{capital} / n - \text{cuota}$ cuando $i \rightarrow 0$).
2. Aplicar bisección en $[0, 1]$: en cada paso, si $f(\text{medio}) < 0$ la tasa está a la derecha; si no, a la izquierda. Hacer 60 iteraciones.
3. Calcular la TEA = $(1 + \text{TEM})^{12} - 1$.
4. Mostrar la TEM (4 decimales), la TEA (2 decimales), el total pagado y el recargo sobre el capital.

Salida esperada

```
Capital 1000000, 12 cuotas de 100000: TEM 2.9229 % | TEA 41.30 % | total pagado 1200000 (20.00 % sobre el capital)
Capital 500000, 24 cuotas de 30000: TEM 3.1491 % | TEA 45.07 % | total pagado 720000 (44.00 % sobre el capital)
Capital 200000, 6 cuotas de 36000: TEM 2.2442 % | TEA 30.52 % | total pagado 216000 (8.00 % sobre el capital)
```

Reglas

- Complejidad: $O(\text{iteraciones})$ por oferta; cada iteración divide el intervalo por 2 (60 iteraciones dan una precisión de 10^{-18}).
- La bisección es lenta pero no puede fallar si la función cambia de signo en el intervalo; es la opción robusta.
- No funcional: robustez: el método tiene que terminar siempre (cantidad fija de iteraciones), aunque los datos sean raros.
- No funcional: al publicar tasas a consumidores, los redondeos se hacen solo al final (0,5 hacia arriba).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): no hace falta despejar i : alcanza con saber evaluar la cuota para una tasa dada y con que la cuota crezca con la tasa. Así podés "acorrallar" la tasa partiendo el intervalo a la mitad.

Bonus (opcional)

- Incluí gastos de otorgamiento y un seguro mensual para calcular el costo financiero total (CFT).
- Resóvelo con Newton-Raphson y compará la cantidad de iteraciones para la misma precisión.

Cierre: La bisección es el método numérico más simple y seguro para encontrar raíces, y se usa para calcular tasas implícitas en todo el sistema financiero.

Ejercicio 403: Cuántos camiones soporta la red de rutas

Nivel 9: Senior III · Matemática aplicada · Temática: Logística y envíos · Dificultad 81/100

Situación: Una empresa de logística quiere saber cuántos camiones por hora puede mandar como máximo desde su depósito hasta el puerto, usando una red de rutas donde cada tramo tiene una capacidad (camiones por hora que soporta sin congestionarse). Además quiere saber qué tramos son el cuello de botella, para pedirle a vialidad que los amplíe.

Datos de entrada

Formato: primera línea: origen y destino; después, tramos dirigidos "desde hasta capacidad".

Ejemplo

```
Depósito Puerto
Depósito Rosario 12
Depósito Pergamino 8
Rosario Pergamino 3
Rosario Zárate 7
Pergamino Zárate 6
Pergamino Luján 5
Zárate Puerto 11
Luján Puerto 6
```

Restricciones: hasta 10^4 cruces y 10^5 tramos; capacidades hasta 10^6 .

Tu programa debe

1. Armar el grafo residual: cada tramo tiene capacidad restante y un tramo inverso con capacidad 0 (para poder "deshacer" flujo).
2. Buscar con BFS un camino del origen al destino con capacidad residual positiva (el más corto en cantidad de tramos: Edmonds-Karp).
3. Mandar por ese camino el mínimo de sus capacidades residuales, actualizando tramos directos e inversos; repetir hasta que no haya camino.
4. Mostrar los caminos usados, el flujo máximo y el corte mínimo: los tramos originales que van de lo alcanzable desde el origen en el último BFS a lo no alcanzable.

Salida esperada

```
Camino 1: Depósito -> Rosario -> Zárate -> Puerto (+7)
Camino 2: Depósito -> Pergamino -> Zárate -> Puerto (+4)
Camino 3: Depósito -> Pergamino -> Luján -> Puerto (+4)
Camino 4: Depósito -> Rosario -> Pergamino -> Luján -> Puerto (+1)
Flujo máximo: 16 camiones por hora
Cuello de botella (corte mínimo): Pergamino-Luján (5), Zárate-Puerto (11) = 16
```

Reglas

- Complejidad esperada: $O(V \times E^2)$ con Edmonds-Karp.
- Teorema de flujo máximo y corte mínimo: el flujo máximo es igual a la capacidad total del corte mínimo.
- BFS recorre los vecinos en el orden en que aparecen los tramos en la entrada (primero los tramos salientes y después los inversos, según su creación).
- No funcional: con capacidades enteras, el flujo resultante es entero (no hay que mandar "medio camión").
- No funcional: el resultado tiene que ser explicable: se informa el corte, no solo el número.

Pista (leela solo si te trabás): los tramos inversos permiten que un camino nuevo "redirija" flujo que un camino anterior mandó por un lugar poco conveniente. Sin ellos, el algoritmo voraz se traba en soluciones que no son óptimas.

Bonus (opcional)

- Implementá Dinic y compará la cantidad de BFS con Edmonds-Karp en redes grandes.
- Calculá cuánto aumenta el flujo máximo si se amplía en 5 un solo tramo, para cada tramo del corte.

Cierre: El flujo máximo resuelve problemas de transporte, redes de datos, asignación de recursos y planificación de evacuaciones.

Ejercicio 404: Médicos para cada guardia

Nivel 9: Senior III · Matemática aplicada · Temática: Salud · Dificultad 82/100

Situación: Un hospital tiene que cubrir las guardias nocturnas de la semana con un médico cada una, y cada médico puede hacer como máximo una guardia (por el descanso obligatorio). Cada médico indicó qué noches puede. El jefe de servicio quiere cubrir la mayor cantidad posible de guardias.

Datos de entrada

Formato: una línea por médico: "nombre guardias_que_puede..."

Ejemplo

```
Dra.Sosa lunes_noche martes_noche
Dr.Paz lunes_noche
Dra.Ruiz martes_noche miércoles_noche jueves_noche
Dr.Gil lunes_noche miércoles_noche
Dra.Luna viernes_noche
```

Restricciones: hasta 10^4 médicos y 10^4 guardias; hasta 10^6 pares posibles.

Tu programa debe

1. Modelar un grafo bipartito: médicos de un lado, guardias del otro, con una arista por cada guardia que un médico puede cubrir.
2. Para cada médico (en el orden de la entrada), buscar un camino de aumento con DFS: probar sus guardias en orden; si una está libre, tomarla; si está ocupada, intentar reubicar recursivamente a quien la tiene.
3. Cada camino de aumento exitoso suma una guardia cubierta.
4. Mostrar la asignación (guardias en orden de primera aparición), la cantidad cubierta, las guardias sin cubrir y los médicos libres.

Salida esperada

```
lunes_noche: Dr.Paz
martes_noche: Dra.Sosa
miércoles_noche: Dr.Gil
jueves_noche: Dra.Ruiz
viernes_noche: Dra.Luna
Guardias cubiertas: 5 de 5
Sin cubrir: ninguna | Médicos sin guardia: ninguno
```

Reglas

- Complejidad esperada: $O(V \times E)$ con el algoritmo de Kuhn; $O(E \sqrt{V})$ con Hopcroft-Karp.
- El resultado es un emparejamiento máximo (no solo maximal): ninguna reubicación puede cubrir más guardias.
- No funcional: determinismo: el orden de los médicos y de sus guardias define qué emparejamiento máximo se obtiene; con otro orden puede salir otro igual de bueno.
- No funcional: las restricciones de descanso son reglas duras: nunca se asignan dos guardias a un médico.

Pista (leela solo si te trabás): si la guardia que querés está ocupada, preguntale al que la tiene si puede irse a otra. Esa cadena de "correrse" es un camino de aumento.

Bonus (opcional)

- Permití que cada médico haga hasta 2 guardias (convertilo en flujo máximo con capacidades).
- Implementá Hopcroft-Karp y medí la mejora con 10^4 médicos.

Cierre: El emparejamiento bipartito se usa en asignación de turnos, residencias médicas, aulas y cualquier problema de "uno para uno" con compatibilidades.

Ejercicio 405: El alambrado más corto del campo

Nivel 9: Senior III · Matemática aplicada · Temática: Agricultura · Dificultad 82/100

Situación: Un productor quiere alambrar un monte de árboles que no se pueden dejar afuera, usando la menor cantidad de alambre posible. Los postes van en algunos árboles del borde. El alambrado más corto que encierra a todos los árboles es la envolvente convexa: el contorno que tomaría una banda elástica estirada alrededor de todos.

Datos de entrada

Formato: un árbol por línea: "x y" en metros (enteros).

Ejemplo

```
0 0
40 0
60 20
50 50
20 60
0 40
20 20
30 30
45 10
10 50
50 30
```

Restricciones: hasta 10^6 árboles; coordenadas hasta 10^6 en valor absoluto.

Tu programa debe

1. Ordenar los puntos por x (y en empate, por y).
2. Armar la cadena inferior recorriendo de izquierda a derecha: agregar cada punto y sacar del final mientras los tres últimos no doblen a la izquierda (producto cruzado ≤ 0).
3. Armar la cadena superior igual, de derecha a izquierda, y unir las (algoritmo de la cadena monótona de Andrew).
4. Mostrar los postes en sentido antihorario empezando por el de menor x, el perímetro (2 decimales), la superficie (1 decimal) y cuántos árboles quedan adentro.

Salida esperada

```
Postes del alambrado (6): (0,0) (40,0) (60,20) (50,50) (20,60) (0,40)
Perímetro: 199.81 m
Superficie encerrada: 2800.0 m²
Árboles que quedan adentro sin ser postes: 5
```

Reglas

- Complejidad esperada: $O(n \log n)$ por el ordenamiento.
- El producto cruzado de $(a - o) \times (b - o)$ dice si el giro $o \rightarrow a \rightarrow b$ es a la izquierda (> 0), a la derecha (< 0) o si están alineados ($= 0$).
- Los puntos alineados sobre un lado no son postes (se descartan con ≤ 0).
- No funcional: robustez numérica: todas las decisiones de giro se toman con enteros; los decimales aparecen recién al medir.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): recorriendo el contorno en sentido antihorario, siempre se dobla a la izquierda. Si agregar un punto obliga a doblar a la derecha, el punto anterior no era parte del contorno.

Bonus (opcional)

- Calcúlá también el rectángulo de menor área que encierra a todos (con el método de los calibres rotantes).
- Si cada metro de alambre cuesta \$2.500 y cada poste \$18.000, calculá el costo total.

Cierre: La envolvente convexa es una de las estructuras básicas de la geometría computacional, usada en mapas, robótica, colisiones y análisis de datos.

Ejercicio 406: Arbitraje de divisas

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 82/100

Situación: La mesa de cambios de un banco monitorea las cotizaciones entre monedas. Si existe una secuencia de cambios que empieza y termina en la misma moneda con más dinero que al principio, hay una oportunidad de arbitraje (y probablemente una cotización mal cargada que conviene corregir). Multiplicar tasas a lo largo de un ciclo se transforma en sumar logaritmos, y un ciclo "ganador" pasa a ser un ciclo negativo.

Datos de entrada

Formato: una cotización por línea: "desde hasta tasa" (1 unidad de desde se cambia por tasa unidades de hasta).

Ejemplo

```
ARS USD 0.00098
USD ARS 1010
USD EUR 0.93
EUR USD 1.09
EUR BRL 6.2
BRL ARS 170
USD BRL 5.6
BRL USD 0.18
```

Restricciones: hasta 200 monedas y 10^4 cotizaciones.

Tu programa debe

1. Asignar a cada cotización el peso $-\log(\text{tasa})$: un ciclo con producto > 1 tiene suma de pesos < 0 .
2. Aplicar Bellman-Ford partiendo de todas las monedas con distancia 0 (fuente virtual), relajando todas las aristas V veces y registrando la última moneda relajada en la última pasada.
3. Si en la pasada número V todavía se relajó algo, hay un ciclo negativo: retroceder V veces por los predecesores desde esa moneda para caer dentro del ciclo, y reconstruirlo.
4. Mostrar el ciclo, el producto de tasas (6 decimales) y la ganancia porcentual por vuelta (4 decimales).

Salida esperada

```
Ciclo de arbitraje: USD -> EUR -> BRL -> USD
Producto de tasas: 1.037880 (ganancia 3.7880 % por vuelta)
```

Reglas

- Complejidad esperada: $O(V \times E)$.
- Las comparaciones con logaritmos usan una tolerancia de 10^{-12} para no confundir errores de redondeo con arbitraje; el producto final se recalcula con decimales exactos.
- No funcional: alertas confiables: antes de actuar, el ciclo se verifica multiplicando las tasas reales (no los logaritmos).
- No funcional: con 200 monedas y cotizaciones que cambian cada segundo, el análisis completo tiene que correr en milisegundos.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): $\log(a \times b \times c) = \log a + \log b + \log c$. Con el signo menos, "producto mayor que 1" se convierte en "suma menor que 0": un ciclo negativo, que es justo lo que Bellman-Ford sabe detectar.

Bonus (opcional)

- Incluí una comisión de 0,1 % por cambio y verificá si el arbitraje sigue existiendo.

- Encontrá el ciclo más rentable de como máximo 4 cambios con programación dinámica.

Cierre: Detectar ciclos negativos en grafos de tasas es una técnica real de control de riesgo en mesas de cambio y exchanges.

Ejercicio 407: Distancias entre todas las ciudades del tour

Nivel 9: Senior III · Matemática aplicada · Temática: Turismo · Dificultad 82/100

Situación: Una agencia de turismo del Noroeste arma circuitos a medida y necesita la distancia más corta entre todos los pares de pueblos de la región, para cotizar cualquier combinación al instante. Como son pocos pueblos y se consultan todos contra todos, conviene calcular la tabla completa una vez con Floyd-Warshall.

Datos de entrada

Formato: rutas "pueblo pueblo km" (doble mano); una línea "?" y consultas "origen destino".

Ejemplo

```
Salta Jujuy 120
Salta Cafayate 190
Salta Cachi 160
Cachi Cafayate 160
Jujuy Humahuaca 125
Cafayate Tucumán 230
Salta Tucumán 310
?
Humahuaca Tucumán
Cachi Jujuy
Humahuaca Cafayate
```

Restricciones: hasta 500 pueblos (la tabla tiene 250.000 celdas).

Tu programa debe

1. Inicializar la tabla con 0 en la diagonal, el largo de cada ruta directa y "infinito" en el resto; guardar también el siguiente pueblo de cada camino.
2. Para cada pueblo intermedio k, y cada par (i, j): si pasar por k acorta el camino, actualizar la distancia y el siguiente pueblo.
3. Mostrar la tabla completa (con los nombres abreviados a 4 letras en el encabezado).
4. Para cada consulta, reconstruir el camino siguiendo la tabla de siguientes y mostrarlo.

Salida esperada

	Salt	Juju	Cafa	Cach	Huma	Tucu
Salta	0	120	190	160	245	310
Jujuy	120	0	310	280	125	430
Cafayate	190	310	0	160	435	230
Cachi	160	280	160	0	405	390
Humahuaca	245	125	435	405	0	555
Tucumán	310	430	230	390	555	0

Humahuaca -> Tucumán: 555 km por Humahuaca -> Jujuy -> Salta -> Tucumán
 Cachi -> Jujuy: 280 km por Cachi -> Salta -> Jujuy
 Humahuaca -> Cafayate: 435 km por Humahuaca -> Jujuy -> Salta -> Cafayate

Reglas

- Complejidad esperada: $O(n^3)$ en tiempo y $O(n^2)$ en memoria.
- El orden de los bucles importa: k tiene que ser el bucle de afuera.
- Los pueblos se numeran por orden de aparición en la entrada.
- No funcional: la tabla se precalcula de noche; las cotizaciones durante el día solo la leen ($O(1)$ por distancia).
- No funcional: si una ruta se corta, se recalcula todo; para cambios frecuentes convendría otro enfoque (bonus).

Pista (leela solo si te trabás): después de procesar el intermedio k , la tabla tiene los caminos más cortos que solo usan como escalas a los pueblos 0 a k . Agregar un pueblo más solo puede mejorar pasando por él.

Bonus (opcional)

- Actualizá la tabla en $O(n^2)$ cuando se agrega una ruta nueva, sin recalcular todo.
- Encontrá el pueblo "más central" (el que minimiza la distancia máxima a los demás) para poner la base de la agencia.

Cierre: Floyd-Warshall es la forma más simple de obtener todas las distancias entre pares, y se usa en ruteo, tablas de distancias y análisis de redes chicas.

Ejercicio 408: Plagio en los trabajos prácticos

Nivel 9: Senior III · Matemática aplicada · Temática: Educación · Dificultad 82/100

Situación: Una universidad quiere detectar trabajos prácticos copiados. Compara los textos por fragmentos de k palabras seguidas: dos trabajos que comparten muchos fragmentos probablemente se copiaron. Con miles de trabajos, comparar fragmentos como texto es lento; en su lugar, cada fragmento se representa con un hash polinomial que se calcula rápido.

Datos de entrada

Formato: primera línea: k (palabras por fragmento); después "nombre: texto" por trabajo.

Hash de un fragmento: $h = 0$; para cada carácter de cada palabra, $h = (h \times 257 + \text{código}(c)) \text{ resto } 1.000.000.007$, y después de cada palabra se suma un espacio (código 32) de la misma forma. Los códigos son los de Unicode.

Ejemplo

```
3
tp_Luciana: la programación dinámica guarda resultados parciales para no recalculiar los mismos
            subproblemas muchas veces
tp_Martín: la programación dinámica guarda resultados parciales para no recalculiar los mismos
            subproblemas y ahorra tiempo
tp_Nadia: memorizar respuestas intermedias evita repetir trabajo cuando un problema se divide en
            partes que se superponen
```

Restricciones: hasta 10^4 trabajos de hasta 10^4 palabras.

Tu programa debe

1. Partir cada texto en palabras y calcular el hash de cada fragmento de k palabras consecutivas (con hash rodante, cada fragmento nuevo se obtiene del anterior en $O(1)$; en el ejemplo alcanza con recalcularlo).
2. Guardar el conjunto de hashes de cada trabajo.
3. Para cada par, calcular la similitud de Jaccard: fragmentos comunes / fragmentos distintos entre los dos.
4. Mostrar cada par con su similitud (1 decimal) y marcar posible plagio a partir del 40 %.

Salida esperada

```
tp_Luciana vs tp_Martín: 10 fragmentos comunes de 15 | similitud 66.7 % -> POSIBLE PLAGIO
tp_Luciana vs tp_Nadia: 0 fragmentos comunes de 26 | similitud 0.0 %
tp_Martín vs tp_Nadia: 0 fragmentos comunes de 27 | similitud 0.0 %
```

Reglas

- Complejidad: $O(\text{palabras})$ por trabajo para los hashes y $O(\text{fragmentos})$ por par con conjuntos.
- Dos fragmentos distintos podrían tener el mismo hash (colisión); con módulo $10^9 + 7$ es muy improbable, pero una alerta de plagio se confirma comparando los textos reales.
- No funcional: privacidad: los trabajos se pueden comparar guardando solo los hashes, sin conservar los textos de otros años.
- No funcional: la marca de "posible plagio" es una alerta para revisión humana, nunca una sanción automática.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): un texto copiado y levemente editado conserva la mayoría de sus fragmentos de 5 palabras; un texto escrito por otra persona sobre el mismo tema casi no comparte ninguno, aunque use las mismas palabras.

Bonus (opcional)

- Implementá el hash rodante real: quitar la primera palabra y agregar la siguiente en $O(\text{largo de palabra})$.
- Usá "winnowing" (guardar solo el mínimo de cada ventana de hashes) para reducir la cantidad de fragmentos guardados.

Cierre: Los hashes polinomiales y la similitud por fragmentos son la base de los detectores de plagio y de contenido duplicado.

Ejercicio 409: La TIR de un préstamo con flujos irregulares

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 82/100

Situación: Un inversor prestó \$1.000.000 a una pyme y recibe devoluciones en fechas irregulares. Para compararlo con un plazo fijo necesita la tasa interna de retorno (TIR): la tasa anual que hace que el valor presente de todos los flujos sea cero. No hay fórmula cerrada, pero el método de Newton-Raphson converge en pocas iteraciones usando la derivada.

Datos de entrada

Formato: un flujo por línea: "día monto" (día contado desde el primer flujo; montos negativos son salidas).

Valor presente a la tasa anual r : $VP(r) = \text{suma de monto} / (1 + r)^{(\text{día} / 365)}$.

Ejemplo

```
0 -1000000
90 150000
200 300000
365 250000
500 450000
```

Restricciones: hasta 10^5 flujos.

Tu programa debe

1. Definir $VP(r)$ y su derivada $VP'(r) = \text{suma de } -(\text{día} / 365) \times \text{monto} / (1 + r)^{(\text{día} / 365 + 1)}$.
2. Partir de $r = 0,1$ y aplicar Newton: $r_{\text{nuevo}} = r - VP(r) / VP'(r)$.
3. Parar cuando $|r_{\text{nuevo}} - r| < 10^{-12}$ o a las 50 iteraciones.
4. Mostrar las primeras iteraciones (en %, 6 decimales), la TIR (4 decimales) y el valor presente a esa tasa (debería ser ≈ 0).

Salida esperada

```
Iteraciones: 16.121233 16.501005 16.502313 16.502313 ...
TIR anual: 16.5023 % (convergió en 5 iteraciones)
Valor presente a esa tasa: 0.00
```

Reglas

- Complejidad: $O(\text{flujos} \times \text{iteraciones})$; Newton suele converger en menos de 10 iteraciones (convergencia cuadrática).
- Newton puede diverger si la derivada es casi cero o el punto de partida es malo; por eso el límite de iteraciones.
- No funcional: robustez: si Newton no converge, el sistema debe caer a bisección (más lenta pero segura) en lugar de devolver un valor sin sentido.
- No funcional: el valor presente final se muestra como verificación de que la tasa encontrada es correcta.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): Newton reemplaza la curva por su recta tangente y va a donde esa recta corta el cero. Cerca de la raíz, la cantidad de dígitos correctos se duplica en cada paso.

Bonus (opcional)

- Implementá el respaldo con bisección si Newton sale del intervalo $(-0,99; 10)$.
- Calculá la TIR con flujos que cambian de signo varias veces y mostrá que puede haber más de una TIR.

Cierre: La TIR con fechas irregulares es una métrica estándar de evaluación de inversiones, y Newton-Raphson es el método que usan las planillas de cálculo para obtenerla.

Ejercicio 410: La superficie de la laguna

Nivel 9: Senior III · Matemática aplicada · Temática: Clima y energía · Dificultad 83/100

Situación: Un organismo ambiental mide lagunas con un GPS: recorre la orilla y registra puntos. Con esos vértices hay que calcular la superficie del espejo de agua para monitorear la sequía. La fórmula del "cordón de zapato" (shoelace) calcula el área de cualquier polígono simple a partir de sus vértices, sin triangular a mano.

Datos de entrada

Formato: un vértice por línea, "x y" en metros, en el orden en que se recorrió la orilla.

Ejemplo

```
0 0
120 -20
260 30
300 150
220 260
90 240
-30 130
```

Restricciones: hasta 10^6 vértices; coordenadas hasta 10^6 (la suma entra en 64 bits: usá enteros).

Tu programa debe

1. Calcular la suma de $x_i \times y_{i+1} - x_{i+1} \times y_i$ para todos los lados (el último vértice se une con el primero).
2. El área es el valor absoluto de la suma dividido 2; el signo indica el sentido del recorrido (positivo = antihorario).
3. Convertir a hectáreas (1 ha = 10.000 m²).
4. Mostrar la suma, el sentido, la superficie (1 decimal en m², 4 en ha) y el perímetro (2 decimales).

Salida esperada

```
Suma del cordón de zapato: 132100 (antihorario)
Superficie: 66050.0 m² = 6.6050 ha
Perímetro: 960.56 m
```

Reglas

- Complejidad: $O(n)$.
- La fórmula vale para polígonos simples (sin lados que se crucen), convexos o no.
- No funcional: exactitud: la suma se hace con enteros; la división por 2 y la raíz del perímetro recién al final.
- No funcional: con coordenadas geográficas (latitud y longitud) hay que proyectarlas antes a metros; describí por qué no se puede aplicar la fórmula directamente sobre grados.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): cada término $x_i \times y_{i+1} - x_{i+1} \times y_i$ es el doble del área (con signo) del triángulo formado por el origen y un lado. Las partes que quedan afuera del polígono se cancelan con signo opuesto.

Bonus (opcional)

- Calculá el centroide de la laguna con fórmulas similares.
- Detectá si el polígono tiene lados que se cruzan (ver el ejercicio de las rutas aéreas) antes de calcular.

Cierre: La fórmula del cordón de zapato es la base de los cálculos de superficie en sistemas de información geográfica y catastros.

Ejercicio 411: Patrones repetidos en el genoma

Nivel 9: Senior III · Matemática aplicada · Temática: Ciencia · Dificultad 83/100

Situación: Un laboratorio de genética busca motivos (secuencias cortas) dentro de cadenas de ADN de millones de bases y detecta secuencias que se repiten periódicamente, porque suelen indicar regiones con funciones especiales. La función Z calcula, en tiempo lineal, cuánto coincide cada posición de una cadena con su propio comienzo, y resuelve las dos tareas.

Datos de entrada

Formato: primera línea: el motivo; segunda: la cadena donde buscarlo; después, secuencias para analizar su período.

Ejemplo

```
GATC
AGATCGATCGGATCAGATC
ATGCATGCATGCATGC
ATGCATGAATGC
```

Restricciones: cadenas de hasta 10^8 bases (letras A, C, G, T).

Tu programa debe

1. Calcular la función Z : $z[i]$ = largo del prefijo más largo de la cadena que empieza también en la posición i , manteniendo la ventana $[l, r)$ más a la derecha que coincide con un prefijo.
2. Para buscar el motivo, calcular Z sobre motivo + "\$" + cadena: cada posición con $z \geq$ largo del motivo es una aparición.
3. Para el período mínimo de una secuencia, buscar el menor k tal que $k + z[k] =$ largo (el resto de la cadena coincide con su comienzo).
4. Mostrar los primeros valores de Z , las posiciones (desde 0), y para cada secuencia su período y si se repite de forma exacta.

Salida esperada

```
Z de "GATC$..." (primeros 12): 24 0 0 0 0 0 4 0 0 0 4 0
"GATC" aparece en las posiciones: 1 5 10 15
ATGCATGCATGCATGC: período mínimo 4 ("ATGC"), se repite exacto 4 veces
ATGCATGAATGC: período mínimo 8 ("ATGCATGA")
```

Reglas

- Complejidad esperada: $O(n)$ (el puntero r solo avanza).
- El separador "\$" no aparece en las cadenas: evita que una coincidencia cruce del motivo al texto.
- Por convención, $z[0]$ es el largo total de la cadena.
- No funcional: memoria: con 10^8 bases, el arreglo Z ocupa 400 MB con enteros de 4 bytes; para búsquedas puntuales conviene KMP, que usa memoria proporcional al motivo.

Pista (leela solo si te trabás): si ya sabés que $s[l..r)$ es igual a $s[0..r-l)$, entonces lo que pasa en la posición i dentro de esa ventana ya lo viste en la posición $i-l$: reutilizás $z[i-l]$ y solo comparás lo que queda afuera.

Bonus (opcional)

- Encontrá el borde más largo (prefijo que también es sufijo) de cada secuencia con la misma función.
- Contá cuántas veces aparece cada prefijo de la cadena dentro de ella.

Cierre: La función Z es una alternativa elegante a KMP para búsqueda de patrones y análisis de periodicidad en bioinformática.

Ejercicio 412: Cortar la red con el menor daño

Nivel 9: Senior III · Matemática aplicada · Temática: Telecomunicaciones · Dificultad 83/100

Situación: Una operadora de telecomunicaciones hace un análisis de resiliencia: quiere saber cuál es el conjunto de enlaces de menor capacidad total cuya caída dejaría incomunicada la central del datacenter. Esos son los enlaces críticos, donde conviene poner redundancia primero. Por el teorema de flujo máximo y corte mínimo, se obtienen corriendo un flujo máximo.

Datos de entrada

Formato: primera línea: los dos extremos; después, enlaces "a b capacidad" (bidireccionales, en Gbps).

Ejemplo

```
Central Datacenter
Central NodoA 10
Central NodoB 4
NodoA NodoB 3
NodoA NodoC 2
NodoA NodoD 5
NodoB NodoD 6
NodoC Datacenter 9
NodoD Datacenter 7
```

Restricciones: hasta 10^4 nodos y 10^5 enlaces.

Tu programa debe

1. Modelar cada enlace bidireccional como dos aristas dirigidas con la misma capacidad.
2. Calcular el flujo máximo entre los dos extremos (Edmonds-Karp, como en el ejercicio de los camiones).
3. Con el grafo residual final, marcar los nodos alcanzables desde el origen con capacidad residual positiva.
4. Los enlaces con un extremo marcado y el otro no forman el corte mínimo. Mostrar la capacidad máxima, los enlaces críticos (en el orden de la entrada), su suma y los nodos del lado del origen (en orden alfabético).

Salida esperada

```
Capacidad máxima entre Central y Datacenter: 9 Gbps
Enlaces críticos (corte mínimo): NodoA-NodoC (2), NodoD-Datacenter (7)
Suma de capacidades del corte: 9
Nodos del lado de Central: Central, NodoA, NodoB, NodoD
```

Reglas

- Complejidad: $O(V \times E^2)$ con Edmonds-Karp.
- La suma de capacidades del corte mínimo coincide siempre con el flujo máximo (sirve como verificación).
- No funcional: el análisis es defensivo: se usa para priorizar inversiones en redundancia y planes de contingencia.
- No funcional: puede haber varios cortes mínimos; el algoritmo devuelve el más cercano al origen, y hay que aclararlo en el informe.

Pista (leela solo si te trabás): cuando el flujo máximo termina, los enlaces que separan lo alcanzable de lo no alcanzable están completamente saturados: son el embudo que limita todo.

Bonus (opcional)

- Calculá el corte mínimo más cercano al destino (alcanzables hacia atrás desde el destino) y compará.
- Encontrá la cantidad mínima de enlaces (no de capacidad) cuya caída desconecta los nodos: ponele capacidad 1 a todos.

Cierre: El corte mínimo es la herramienta formal para encontrar los puntos débiles de redes de datos, energía y transporte.

Ejercicio 413: El rebaño en el año 10^{18}

Nivel 9: Senior III · Matemática aplicada · Temática: Agricultura · Dificultad 83/100

Situación: Un modelo de ganadería divide el rodeo en terneros, vaquillonas y vacas. Cada año, cada vaca tiene un ternero, los terneros pasan a vaquillonas, las vaquillonas pasan a vacas y las vacas siguen. Eso es una multiplicación por una matriz fija. Para escenarios teóricos a muy largo plazo (hasta el año 10^{18} , como prueba de estrés del sistema), se usa exponenciación rápida de matrices con resultados módulo 1.000.000.007.

Datos de entrada

Formato: tres líneas con la matriz de transición M (nuevo = $M \times$ actual, en el orden terneros, vaquillonas, vacas); una línea con el rodeo inicial; una línea con los años a consultar.

Ejemplo

```
0 0 1
1 0 0
0 1 1
0 0 10
1 2 10 1000000000000000000
```

Restricciones: matrices de hasta 50×50 ; años hasta 10^{18} . Para años ≤ 100 se muestra el valor exacto; para los demás, módulo 1.000.000.007.

Tu programa debe

1. Representar el rodeo como un vector y un año como $v_{\text{nuevo}} = M \times v$.
2. Para el año y , calcular M^y con exponenciación rápida (elevar al cuadrado y multiplicar según los bits de y), reduciendo módulo 1.000.000.007 cuando corresponde.
3. Multiplicar M^y por el vector inicial.
4. Mostrar cada categoría, el total y la cantidad de productos de matrices realizados.

Salida esperada

```
Año 1: terneros 10, vaquillonas 0, vacas 10 | total 20 | 1 producto de matrices
Año 2: terneros 10, vaquillonas 10, vacas 10 | total 30 | 2 productos de matrices
Año 10: terneros 190, vaquillonas 130, vacas 280 | total 600 | 5 productos de matrices
Año 1000000000000000000: terneros 448051778, vaquillonas 143879740, vacas 154724718 | total
746656236 (resto 1000000007) | 83 productos de matrices
```

Reglas

- Complejidad esperada: $O(k^3 \times \log y)$ para matrices de $k \times k$.
- Se cuenta un producto por cada multiplicación de matrices (cuadrados y multiplicaciones del resultado).
- No funcional: para años ≤ 100 los valores exactos pueden superar 64 bits: usá enteros de precisión arbitraria.
- No funcional: la matriz y el vector se validan (enteros no negativos) antes de calcular.

Pista (leela solo si te trabás): aplicar la matriz y veces es lo mismo que aplicar M^y una sola vez, y M^y se calcula igual que x^y con números: con cuadrados sucesivos.

Bonus (opcional)

- Calculá la tasa de crecimiento a largo plazo (el autovalor dominante) con el método de las potencias.
- Agregá mortalidad: el 10 % de las vacas se vende cada año (la matriz pasa a tener fracciones).

Cierre: La exponenciación de matrices resuelve recurrencias lineales enormes: poblaciones, Fibonacci, cadenas de Markov y conteo de caminos.

Ejercicio 414: Balancear reacciones químicas

Nivel 9: Senior III · Matemática aplicada · Temática: Ciencia · Dificultad 83/100

Situación: Una plataforma educativa de química corrige ejercicios de balanceo de ecuaciones y necesita calcular los coeficientes correctos de cualquier reacción. Balancear es resolver un sistema de ecuaciones lineales: por cada elemento, los átomos a la izquierda tienen que igualar a los de la derecha. Con eliminación gaussiana y fracciones exactas se obtiene la solución entera más chica.

Datos de entrada

Formato: una reacción por línea: "reactivo + reactivo -> producto + producto" (fórmulas sin paréntesis: símbolo del elemento seguido opcionalmente de la cantidad).

Ejemplo

```
C3H8 + O2 -> CO2 + H2O
Fe + O2 -> Fe2O3
KMnO4 + HCl -> KCl + MnCl2 + H2O + Cl2
Al + O2 -> Al2O3
```

Restricciones: hasta 20 compuestos y 20 elementos por reacción.

Tu programa debe

1. Contar los átomos de cada elemento en cada compuesto.
2. Armar la matriz elementos $\times$ compuestos, con signo positivo para reactivos y negativo para productos: se busca x con $A \times x = 0$.
3. Llevar A a forma escalonada reducida con fracciones exactas; la columna sin pivote es la variable libre: fijarla en 1 y despejar las demás.
4. Multiplicar por el mínimo común múltiplo de los denominadores y dividir por el MCD, para obtener los enteros más chicos. Mostrar la ecuación balanceada (sin escribir los coeficientes 1).

Salida esperada

```
C3H8 + 5 O2 -> 3 CO2 + 4 H2O
4 Fe + 3 O2 -> 2 Fe2O3
2 KMnO4 + 16 HCl -> 2 KCl + 2 MnCl2 + 8 H2O + 5 Cl2
4 Al + 3 O2 -> 2 Al2O3
```

Reglas

- Complejidad: $O(\text{elementos} \times \text{compuestos}^2)$ por la eliminación.
- Se asume que hay exactamente una variable libre (una única forma de balancear, a menos de un múltiplo).
- No funcional: exactitud: se usan fracciones exactas; con decimales binarios, $1/3$ se convierte en $0,3333\dots$ y los coeficientes enteros no salen.
- No funcional: validación: si el sistema no tiene solución positiva o tiene más de una variable libre, informarlo en lugar de mostrar coeficientes sin sentido.

Pista (leela solo si te trabás): cada elemento da una ecuación y cada compuesto es una incógnita. Como el lado derecho es 0, la solución es una "dirección": se elige una incógnita libre, se despeja el resto y se escala hasta que todo quede entero.

Bonus (opcional)

- Soportá paréntesis en las fórmulas, como $\text{Ca}(\text{OH})_2$, con el analizador de expresiones del nivel 5.
- Detectá reacciones con más de una forma independiente de balancearse.

Cierre: La eliminación gaussiana con fracciones exactas resuelve sistemas lineales en química, circuitos, economía y gráficos por computadora.

Ejercicio 415: ¿El repartidor está dentro de la zona?

Nivel 9: Senior III · Matemática aplicada · Temática: Logística y envíos · Dificultad 84/100

Situación: Una app de delivery define su zona de cobertura como un polígono (que puede tener forma de L por un río o una autopista). Cuando un repartidor reporta su posición, el sistema decide si está dentro de la zona para asignarle pedidos. Los puntos exactamente sobre el borde se informan aparte, porque ahí las reglas de la empresa cambian.

Datos de entrada

Formato: vértices del polígono "x y" en orden (enteros); una línea "?" y las posiciones a evaluar.

Ejemplo

```
0 0
100 0
100 60
60 60
60 100
0 100
?
50 50
80 80
100 30
60 80
30 100
-5 10
```

Restricciones: polígonos de hasta 10^4 vértices y 10^6 consultas por minuto.

Tu programa debe

1. Para cada lado, verificar primero si el punto está sobre él (producto cruzado 0 y dentro del rectángulo del lado): si es así, está en el borde.
2. Si no, lanzar un rayo horizontal hacia la derecha desde el punto y contar cuántos lados cruza (solo los lados que tienen un extremo arriba y otro abajo del punto, con la regla "estrictamente mayor" para no contar dos veces un vértice).
3. Decidir si el cruce está a la derecha del punto comparando con productos (sin divisiones).
4. Cantidad impar de cruces: dentro; par: fuera. Mostrar cada resultado.

Salida esperada

```
Repartidor en (50,50): dentro
Repartidor en (80,80): fuera
Repartidor en (100,30): en el borde
Repartidor en (60,80): en el borde
Repartidor en (30,100): en el borde
Repartidor en (-5,10): fuera
```

Reglas

- Complejidad: $O(n)$ por consulta.
- No funcional: exactitud: con coordenadas enteras, todas las decisiones se toman con enteros (sin calcular la x del cruce con decimales), así un punto en el borde nunca se clasifica mal por redondeo.
- No funcional: para millones de consultas contra el mismo polígono, conviene precalcular una grilla o descomponer el polígono (bonus).
- Casos borde: punto sobre un vértice; punto a la altura exacta de un vértice.

Pista (leela solo si te trabás): si salís caminando en línea recta desde adentro de un polígono, cruzás el borde una cantidad impar de veces. Contar cruces es todo lo que hace falta, cuidando los vértices.

Bonus (opcional)

- Para polígonos convexos, respondé en $O(\log n)$ con búsqueda binaria sobre los ángulos.
- Soportá zonas con agujeros (una plaza donde no se entrega): cuentan los cruces de todos los contornos.

Cierre: La prueba de punto en polígono es parte de todo sistema de mapas: geocercas, zonas de cobertura, catastros y videojuegos.

Ejercicio 416: Lo que tienen en común dos logs gigantes

Nivel 9: Senior III · Matemática aplicada · Temática: Seguridad informática · Dificultad 84/100

Situación: Un equipo de seguridad compara registros de dos incidentes para encontrar qué tienen en común (una misma URL de ataque, un mismo token filtrado). La herramienta busca la subcadena común más larga entre dos líneas o archivos gigantes. Con un arreglo de sufijos y el arreglo LCP, se resuelve en tiempo casi lineal.

Datos de entrada

Formato: dos líneas de registro (en el ejercicio, dos líneas; en la realidad, archivos de millones de caracteres).

Ejemplo

```
GET /api/v2/usuarios/export?formato=csv&token=abc123 403
POST /api/v2/pagos/export?formato=csv&token=abc123 200
```

Restricciones: hasta 10^7 caracteres entre los dos textos.

Tu programa debe

1. Concatenar los dos textos con un separador que no aparece en ninguno (un carácter de control).
2. Construir el arreglo de sufijos: los índices de inicio de todos los sufijos, ordenados alfabéticamente (duplicando el largo de comparación en cada ronda).
3. Calcular el arreglo LCP con el algoritmo de Kasai: el largo del prefijo común entre cada sufijo y el anterior en el orden.
4. Recorrer pares de sufijos vecinos que empiezan en textos distintos y quedarse con el mayor LCP. Mostrar el largo y la subcadena (en empate, la primera encontrada en el orden de sufijos).

Salida esperada

```
Largo de la concatenación: 111 | sufijos ordenados: 111
Subcadena común más larga (35 caracteres): "os/export?formato=csv&token=abc123 "
```

Reglas

- Complejidad: $O(n \log^2 n)$ con duplicación y ordenamiento ($O(n \log n)$ con ordenamiento por conteo) y $O(n)$ para Kasai.
- El separador evita que un prefijo común "cruce" de un texto al otro.
- No funcional: memoria: el arreglo de sufijos y el LCP ocupan dos enteros por carácter; con 10^7 caracteres son unos 80 MB.
- No funcional: los registros de seguridad pueden contener datos sensibles (tokens): el resultado se muestra solo a personal autorizado.

Pista (leela solo si te trabás): dos sufijos que comparten un prefijo largo quedan vecinos al ordenarlos. Así que la subcadena común más larga está entre sufijos consecutivos del arreglo, uno de cada texto.

Bonus (opcional)

- Encontrá todas las subcadenas comunes de largo al menos 10, sin repetir.
- Extendelo a k registros: la subcadena más larga que aparece en al menos 3 de ellos.

Cierre: Los arreglos de sufijos son la base de la búsqueda en genomas, de la compresión de datos y del análisis forense de registros.

Ejercicio 417: Habitaciones según preferencias

Nivel 9: Senior III · Matemática aplicada · Temática: Turismo · Dificultad 84/100

Situación: Un hotel boutique asigna habitaciones a los huéspedes de un contingente. Cada huésped ordenó las habitaciones según su preferencia, y el hotel tiene su propia prioridad para cada habitación (por fidelidad, por necesidades de accesibilidad). Se busca una asignación estable: que no haya un huésped y una habitación que se prefieran mutuamente por sobre lo que les tocó.

Datos de entrada

Formato: sección "huéspedes" con "nombre: habitaciones en orden de preferencia"; sección "habitaciones" con "habitación: huéspedes en orden de prioridad".

Ejemplo

```
huéspedes
Aguirre: vista_mar suite doble_1
Benítez: suite vista_mar doble_1
Castro: vista_mar doble_1 suite
habitaciones
vista_mar: Benítez Castro Aguirre
suite: Aguirre Benítez Castro
doble_1: Castro Aguirre Benítez
```

Restricciones: hasta 10^4 huéspedes y habitaciones (listas completas).

Tu programa debe

1. Poner a todos los huéspedes en una cola de libres (en el orden de la entrada).
2. Tomar el primer libre y hacer que pida la siguiente habitación de su lista que todavía no pidió.
3. Si la habitación está libre, aceptarlo; si está ocupada y la habitación prefiere al nuevo, desplazar al actual (vuelve al final de la cola); si no, rechazarlo (vuelve al final de la cola).
4. Repetir hasta que no haya libres. Mostrar el registro y la asignación final (en el orden de los huéspedes).

Salida esperada

```
Aguirre pide vista_mar: aceptado
Benítez pide suite: aceptado
Castro pide vista_mar: desplaza a Aguirre
Aguirre pide suite: desplaza a Benítez
Benítez pide vista_mar: desplaza a Castro
Castro pide doble_1: aceptado
Asignación estable: Aguirre -> suite, Benítez -> vista_mar, Castro -> doble_1
```

Reglas

- Complejidad esperada: $O(n^2)$ pedidos en el peor caso (algoritmo de Gale-Shapley).
- El resultado es el mejor emparejamiento estable posible para los huéspedes (el lado que pide).
- No funcional: transparencia: se guarda el registro de pedidos para explicar a cada huésped por qué no obtuvo su primera opción.
- No funcional: con listas incompletas, algunos huéspedes pueden quedar sin habitación; hay que informarlo explícitamente.

Pista (leela solo si te trabás): la clave es que un huésped nunca vuelve a pedir una habitación que ya lo rechazó, y una habitación solo cambia a un huésped por otro que prefiere. Por eso el proceso termina y el resultado es estable.

Bonus (opcional)

- Verificá la estabilidad: buscá cualquier par (huésped, habitación) que se prefieran mutuamente por sobre su asignación.
- Invertí los roles (las habitaciones "piden") y compará cuál lado sale favorecido.

Cierre: El algoritmo de Gale-Shapley asigna residencias médicas, vacantes escolares y más; su estudio recibió el premio Nobel de Economía en 2012.

Ejercicio 418: Corrientes del circuito eléctrico

Nivel 9: Senior III · Matemática aplicada · Temática: Clima y energía · Dificultad 84/100

Situación: Un simulador educativo de circuitos eléctricos calcula las tensiones y corrientes en redes de resistencias alimentadas por una fuente. Por la ley de Kirchhoff de las corrientes, en cada nodo la corriente que entra es igual a la que sale; con la ley de Ohm, eso da un sistema lineal que se resuelve con eliminación gaussiana.

Datos de entrada

Formato: primera línea: tensión de la fuente en volts; después, resistencias "nodo nodo ohms" ("fuente" está a esa tensión y "masa" a 0 V).

Ejemplo

```
12
fuente nodo1 2
nodo1 masa 6
nodo1 nodo2 3
nodo2 masa 4
nodo2 nodo3 2
nodo3 masa 2
```

Restricciones: hasta 1.000 nodos.

Tu programa debe

1. Plantear una ecuación por nodo desconocido: suma sobre sus resistencias de $(V_{\text{nodo}} - V_{\text{vecino}}) / R = 0$ (análisis nodal).
2. Pasar al lado derecho los términos con tensiones conocidas (fuente y masa) y armar la matriz de conductancias.
3. Resolver con eliminación de Gauss-Jordan con fracciones exactas.
4. Mostrar las tensiones, la corriente en cada resistencia (del primer nodo al segundo, positiva si circula en ese sentido) y la corriente total de la fuente, con 4 decimales.

Salida esperada

```
Tensiones: nodo1 6.9231 V | nodo2 2.7692 V | nodo3 1.3846 V
corriente fuente -> nodo1: 2.5385 A
corriente nodo1 -> masa: 1.1538 A
corriente nodo1 -> nodo2: 1.3846 A
corriente nodo2 -> masa: 0.6923 A
corriente nodo2 -> nodo3: 0.6923 A
corriente nodo3 -> masa: 0.6923 A
Corriente total entregada por la fuente: 2.5385 A
```

Reglas

- Complejidad: $O(n^3)$ por la eliminación.
- La matriz de conductancias es simétrica y, si el circuito está conectado a masa, tiene solución única.
- No funcional: exactitud: con fracciones exactas, la conservación de corriente en cada nodo se cumple exactamente (verificalo).
- No funcional: para circuitos grandes y ralos, la eliminación densa es cara; describí por qué conviene un método iterativo o una matriz rara.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): en cada nodo, todo lo que entra sale. Si expresás cada corriente como diferencia de tensiones dividida la resistencia, las incógnitas son solo las tensiones de los nodos: una ecuación por nodo.

Bonus (opcional)

- Calculá la resistencia equivalente vista por la fuente (tensión / corriente total) y verificala con reglas de serie y paralelo.
- Agregá una segunda fuente de tensión en otro nodo.

Cierre: El análisis nodal con eliminación gaussiana es lo que hacen internamente los simuladores de circuitos profesionales.

Ejercicio 419: Rutas aéreas que se cruzan

Nivel 9: Senior III · Matemática aplicada · Temática: Transporte · Dificultad 84/100

Situación: El área de planificación de una aerolínea dibuja las rutas como segmentos sobre un mapa plano de la región y necesita detectar qué rutas se cruzan, porque en esos puntos hay que separar las altitudes de vuelo. La geometría del cruce se decide con productos cruzados (orientación), sin divisiones hasta calcular el punto.

Datos de entrada

Formato: una ruta por línea: "código x1 y1 x2 y2" (coordenadas enteras en km).

Ejemplo

```
AR1201 0 0 100 80
AR1350 0 80 100 0
LA3030 20 10 90 10
AU2210 50 40 150 120
FB5010 10 70 40 70
```

Restricciones: hasta 10^5 rutas (con barrido; en el ejercicio, comparación de todos los pares).

Tu programa debe

1. Definir orientación(a, b, c) = signo del producto cruzado $(b - a) \times (c - a)$: izquierda, derecha o alineados.
2. Dos segmentos se cortan si los extremos de cada uno quedan a lados distintos del otro, o si un extremo está alineado y sobre el otro segmento.
3. Para los que se cortan y no son paralelos, calcular el punto de cruce con la fórmula paramétrica (con fracciones exactas).
4. Mostrar cada par que se cruza con su punto (2 decimales) y la cantidad total.

Salida esperada

```
AR1201 y AR1350: se cruzan en (50.00, 40.00)
AR1201 y AU2210: se superponen (colineales)
AR1350 y LA3030: se cruzan en (87.50, 10.00)
AR1350 y AU2210: se cruzan en (50.00, 40.00)
AR1350 y FB5010: se cruzan en (12.50, 70.00)
Pares que se cruzan: 5 de 10
```

Reglas

- Complejidad: $O(n^2)$ comparando todos los pares; $O((n + k) \log n)$ con el barrido de Bentley-Ottmann (k = cantidad de cruces).
- Los pares se listan en el orden de la entrada.
- No funcional: robustez: la decisión "se cruzan o no" se toma solo con enteros; el punto se calcula recién después.
- No funcional: seguridad operacional: ante la duda (segmentos que se tocan en un extremo), se informa como cruce.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): c y d quedan de lados opuestos de la recta ab si las orientaciones (a, b, c) y (a, b, d) tienen signos distintos. Para que los segmentos se corten, eso tiene que pasar en las dos direcciones.

Bonus (opcional)

- Implementá el barrido de Bentley-Ottmann para 10^5 rutas.

- Trabaja con coordenadas geográficas reales: las rutas son arcos de círculo máximo, no segmentos.

Cierre: La intersección de segmentos es un bloque básico de la geometría computacional, usado en mapas, CAD, videojuegos y control de tráfico.

Ejercicio 420: El precio de un bono según su rendimiento

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 84/100

Situación: Una sociedad de bolsa cotiza bonos corporativos: dado el rendimiento exigido por el mercado, calcula el precio (descontando cupones y amortización), y dado el precio que se opera, calcula el rendimiento implícito. Lo segundo no se despeja: se resuelve con Newton-Raphson usando la derivada del precio, que además está ligada a la duración del bono.

Datos de entrada

Formato: primera línea: valor nominal, cupón anual (%) y años al vencimiento (cupones anuales, amortización total al final); segunda: un rendimiento (%) para calcular el precio; tercera: precios para calcular el rendimiento.

Ejemplo

```
1000 8 5
10
950 1050
```

Restricciones: bonos de hasta 100 años y 10^5 cotizaciones por segundo.

Tu programa debe

1. Precio(y) = suma de cupón / $(1 + y)^k$ para k de 1 a n + nominal / $(1 + y)^n$.
2. Calcular el precio con el rendimiento dado, la duración de Macaulay (promedio de los plazos ponderado por el valor presente de cada pago) y la duración modificada (Macaulay / $(1 + y)$).
3. Para cada precio observado, aplicar Newton desde y = tasa del cupón: $y_{\text{nuevo}} = y - (\text{Precio}(y) - \text{precio_observado}) / \text{Precio}'(y)$, hasta que el cambio sea menor a 10^{-12} (máximo 50 iteraciones).
4. Mostrar el precio (2 decimales), las duraciones (4 decimales) y cada rendimiento (4 decimales, en %) con sus iteraciones.

Salida esperada

```
Precio con rendimiento 10 %: 924.18 | duración de Macaulay 4.2814 años | duración modificada
3.8922
Precio 950: rendimiento 9.2953 % (4 iteraciones)
Precio 1050: rendimiento 6.7875 % (4 iteraciones)
```

Reglas

- Complejidad: $O(n)$ por evaluación; Newton converge en pocas iteraciones porque el precio es una curva suave y convexa.
- Precio'(y) = $- \text{suma de } k \times \text{pago}_k / (1 + y)^{(k + 1)}$; la duración modificada es $-\text{Precio}'(y) / \text{Precio}(y)$.
- No funcional: consistencia: calcular el precio con el rendimiento obtenido tiene que devolver el precio observado (verificación de ida y vuelta).
- No funcional: latencia: cada cálculo debe tardar microsegundos para cotizar en tiempo real.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la derivada del precio respecto del rendimiento mide cuánto cae el precio si sube la tasa. Esa misma derivada es la que usa Newton para saber cuánto corregir en cada paso.

Bonus (opcional)

- Calculá la convexidad (segunda derivada) y estimá el precio ante una suba de 1 punto de la tasa con duración y convexidad.
- Soportá cupones semestrales y amortizaciones parciales.

Cierre: Precio, rendimiento y duración son los conceptos básicos de la renta fija, y Newton-Raphson es el método que usan todas las calculadoras de bonos.

Ejercicio 421: Evacuación del estadio

Nivel 9: Senior III · Matemática aplicada · Temática: Deportes · Dificultad 85/100

Situación: Para habilitar un estadio, defensa civil exige demostrar en cuántos minutos se puede evacuar a todo el público. Las personas salen de las tribunas, pasan por pasillos y llegan a las salidas; cada tribuna, pasillo y salida tiene un límite de personas por minuto. Como los límites están en los lugares (nodos) y no en las conexiones, cada nodo se parte en dos.

Datos de entrada

Formato: primera línea: cantidad de personas; después "lugar capacidad_por_minuto"; una línea "?" y las conexiones "desde hasta" (sin límite propio). Los lugares que empiezan con Tribuna o Platea son orígenes; los que empiezan con Salida, destinos.

Ejemplo

```
40000
Tribuna_Norte 300
Tribuna_Sur 300
Platea 200
Pasillo_A 350
Pasillo_B 250
Salida_1 400
Salida_2 200
?
Tribuna_Norte Pasillo_A
Tribuna_Sur Pasillo_A
Tribuna_Sur Pasillo_B
Platea Pasillo_B
Pasillo_A Salida_1
Pasillo_B Salida_1
Pasillo_B Salida_2
```

Restricciones: hasta 10^4 lugares y 10^5 conexiones.

Tu programa debe

1. Partir cada lugar en dos nodos (entrada y salida) unidos por una arista con su capacidad.
2. Conectar una fuente virtual a las entradas de tribunas y plateas, y las salidas de las puertas a un sumidero virtual, con capacidad ilimitada; las conexiones también son ilimitadas.
3. Calcular el flujo máximo (Edmonds-Karp).
4. Mostrar el flujo máximo, los minutos para evacuar (redondeo hacia arriba) y los lugares que forman el corte mínimo (cuellos de botella).

Salida esperada

```
Flujo máximo de evacuación: 600 personas por minuto
Tiempo para evacuar 40000 personas: 67 minutos
Cuellos de botella: Pasillo_A (350), Pasillo_B (250)
```

Reglas

- Complejidad: $O(V \times E^2)$ con Edmonds-Karp sobre el grafo con los nodos partidos.
- El cálculo supone flujo estable; en la realidad, los primeros y últimos minutos rinden menos (bonus).
- No funcional: seguridad: el resultado se presenta con un margen (por ejemplo, +20 %) y nunca como un tiempo garantizado.
- No funcional: trazabilidad: se informan los cuellos de botella para justificar obras de ampliación.

Pista (leela solo si te trabás): un límite en un lugar es lo mismo que un límite en un "pasillito" imaginario que lo atraviesa: por eso cada lugar se convierte en dos nodos unidos por una arista con su capacidad.

Bonus (opcional)

- Modelá el tiempo: un grafo expandido por minutos, donde cada minuto es una copia de la red (flujo dinámico).
- Calculá cuánto mejora la evacuación si se ensancha cada cuello de botella en 100 personas por minuto.

Cierre: El flujo máximo con capacidades en nodos se usa para planificar evacuaciones de estadios, edificios y ciudades.

Ejercicio 422: Publicaciones casi idénticas

Nivel 9: Senior III · Matemática aplicada · Temática: Redes sociales · Dificultad 85/100

Situación: Una red social detecta campañas de spam: cuentas distintas publican el mismo texto con cambios mínimos (mayúsculas, signos, una palabra). Comparar cada publicación con todas carácter por carácter es imposible con millones por hora. Se normaliza el texto, se calculan los hashes de todas las ventanas de k caracteres con un hash rodante y se comparan los conjuntos.

Datos de entrada

Formato: primera línea: largo de ventana k y umbral de similitud (%); después "id: texto".

Normalización: pasar a minúsculas, conservar solo letras, dígitos y espacios, y reducir los espacios múltiples a uno. Hash rodante: $h = h \times 131 + \text{código}(c) \text{ resto } 1.000.000.007$ sobre ventanas de k caracteres (al avanzar, restar el carácter que sale multiplicado por $131^{(k-1)}$).

Ejemplo

```
5 30
post1: Gran oferta de zapatillas en el centro, solo por hoy!!!
post2: Gran oferta de zapatillas en el centro, solo por hoy!!
post3: GRAN OFERTA de zapatillas en el centro solo por hoy
post4: Hoy juega la selección, ¿dónde lo vemos?
post5: gran oferta de zapatillas en el centro, sólo hoy
```

Restricciones: hasta 10^6 publicaciones por hora, de hasta 5.000 caracteres.

Tu programa debe

1. Normalizar cada texto.
2. Calcular el hash de la primera ventana y actualizarlo en $O(1)$ al deslizarla: quitar el carácter que sale, multiplicar por la base y sumar el que entra.
3. Guardar el conjunto de hashes de cada publicación.
4. Para cada par, calcular la similitud de Jaccard y listar los que superan el umbral (1 decimal).

Salida esperada

```
Pares casi idénticos:
post1 ~ post2: 100.0 %
post1 ~ post3: 100.0 %
post1 ~ post5: 66.7 %
post2 ~ post3: 100.0 %
post2 ~ post5: 66.7 %
post3 ~ post5: 66.7 %
```

Reglas

- Complejidad: $O(\text{largo})$ por publicación para los hashes; comparar todos los pares es $O(n^2)$, por eso en producción se usa MinHash con LSH (bonus).
- La normalización es parte del algoritmo: sin ella, "GRAN OFERTA" y "gran oferta" no comparten nada.
- No funcional: las letras con tilde se conservan como están ("sólo" y "solo" son distintas): documentará esa decisión.
- No funcional: la detección genera una alerta para moderación; la acción automática solo se toma con varias señales combinadas.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la resta del carácter que sale funciona porque el hash es un polinomio: su término más viejo está multiplicado por la base a la $k-1$. Restarlo y multiplicar por la base "corre" la ventana un lugar.

Bonus (opcional)

- Implementá MinHash: con 100 funciones de hash, estimá la similitud de Jaccard sin comparar conjuntos enteros.
- Agrupá las publicaciones similares en campañas con union-find.

Cierre: Los hashes rodantes y las firmas de similitud se usan para detectar spam, contenido duplicado y copias en buscadores y redes sociales.

Ejercicio 423: Descuentos que se pueden explotar en fletes

Nivel 9: Senior III · Matemática aplicada · Temática: Transporte · Dificultad 85/100

Situación: Una plataforma de fletes paga bonificaciones por tramos de retorno (para no volver vacío), y esas bonificaciones se cargan como costos negativos. Si una combinación de tramos forma un ciclo con costo total negativo, un transportista podría recorrerlo una y otra vez cobrando bonificaciones: una regla explotable. El equipo de producto quiere detectarlo antes de publicar la tarifa.

Datos de entrada

Formato: bloques separados por "?": primera línea, la ciudad de origen; después, tramos dirigidos "desde hasta costo" (en miles de pesos; negativos son bonificaciones).

Ejemplo

```
Córdoba
Córdoba Rosario 50
Córdoba SanLuis 40
Rosario BuenosAires 60
SanLuis Mendoza 45
Mendoza SanJuan 20
SanJuan SanLuis -70
BuenosAires MarDelPlata 35
?
Córdoba
Córdoba Rosario 50
Rosario BuenosAires 60
BuenosAires Rosario -40
Córdoba BuenosAires 120
```

Restricciones: hasta 10^3 ciudades y 10^5 tramos.

Tu programa debe

1. Aplicar Bellman-Ford desde el origen: relajar todos los tramos $V - 1$ veces.
2. Hacer una pasada más: si algún tramo todavía mejora una distancia, hay un ciclo negativo alcanzable desde el origen.
3. Para reconstruirlo, retroceder V veces por los predecesores desde el nodo mejorado (para caer dentro del ciclo) y recorrer hasta volver al mismo nodo.
4. Mostrar el ciclo con su costo por vuelta, o las distancias mínimas desde el origen si no hay ciclo.

Salida esperada

```
Desde Córdoba: CICLO EXPLOTABLE Mendoza -> SanJuan -> SanLuis -> Mendoza (costo por vuelta -5)
Desde Córdoba: sin ciclos negativos | Rosario 50, BuenosAires 110
```

Reglas

- Complejidad: $O(V \times E)$.
- Dijkstra no sirve con costos negativos: puede dar resultados incorrectos sin avisar.
- No funcional: validación previa: ninguna tarifa se publica si el análisis encuentra un ciclo explotable.
- No funcional: el informe muestra el ciclo concreto, para que producto entienda qué combinación de bonificaciones lo produce.

Pista (leela solo si te trabás): sin ciclos negativos, ningún camino más corto necesita más de $V - 1$ tramos. Si en la pasada V todavía algo mejora, es porque dar vueltas a un ciclo sigue bajando el costo.

Bonus (opcional)

- Proponé la bonificación máxima para el tramo SanJuan → SanLuis que no genera ciclos explotables.
- Usá SPFA (Bellman-Ford con cola) y compará los tiempos.

Cierre: Detectar ciclos negativos sirve para validar reglas de precios, programas de puntos y sistemas de bonificaciones antes de que alguien los explote.

Ejercicio 424: Las dos antenas más cercanas

Nivel 9: Senior III · Matemática aplicada · Temática: Telecomunicaciones · Dificultad 85/100

Situación: Una operadora revisa la ubicación de sus antenas: dos antenas demasiado cercanas generan interferencia y desperdician inversión. Con cientos de miles de antenas, comparar todas contra todas es cuadrático. El algoritmo de divide y vencerás encuentra el par más cercano en $O(n \log n)$ partiendo el mapa por la mitad.

Datos de entrada

Formato: una antena por línea: "id x y" (coordenadas enteras en km).

Ejemplo

```
A01 2 3
A02 12 30
A03 40 50
A04 5 1
A05 12 10
A06 3 4
A07 30 28
A08 41 49
```

Restricciones: hasta 10^6 antenas.

Tu programa debe

1. Ordenar las antenas por x y partir el conjunto en dos mitades por la x del medio.
2. Resolver cada mitad recursivamente (con 3 o menos antenas, comparar todas) y quedarse con la menor distancia d.
3. Revisar la franja de ancho $2d$ alrededor de la línea divisoria: ordenar esos puntos por y y comparar cada uno solo con los siguientes cuya diferencia de y sea menor que d.
4. Mostrar el par (ids en orden alfabético), la distancia (3 decimales), su cuadrado y la cantidad de comparaciones.

Salida esperada

```
Antenas más cercanas: A01 y A06
Distancia: 1.414 km (al cuadrado: 2)
Comparaciones de distancias: 5 (todos los pares serían 28)
```

Reglas

- Complejidad esperada: $O(n \log^2 n)$ ordenando la franja en cada nivel ($O(n \log n)$ si se mezcla por y durante la recursión).
- Trabajar con distancias al cuadrado evita raíces durante la búsqueda: la raíz se calcula solo al final.
- Dentro de la franja, cada punto se compara con una cantidad constante de vecinos (a lo sumo 7).
- No funcional: exactitud: todas las comparaciones son con enteros.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): si el par más cercano cruza la línea divisoria, sus dos puntos están a menos de d de la línea y a menos de d en altura: en esa cajita de $2d \times d$ no entran muchos puntos.

Bonus (opcional)

- Mezclá por y durante la recursión para llegar a $O(n \log n)$.
- Resóvelo con una grilla de celdas de tamaño d (algoritmo aleatorizado de tiempo esperado lineal).

Cierre: El par de puntos más cercano es el ejemplo clásico de divide y vencerás en geometría, y se usa en redes, control aéreo y agrupamiento de datos.

Ejercicio 425: Subdominios distintos

Nivel 9: Senior III · Matemática aplicada · Temática: Seguridad informática · Dificultad 85/100

Situación: Un equipo de seguridad analiza nombres de dominio sospechosos: los generados automáticamente por programas maliciosos suelen tener muy poca repetición interna, y una medida simple de eso es la cantidad de subcadenas distintas del nombre. Contarlas generando todas es cuadrático en memoria; con el arreglo de sufijos y el LCP sale directo.

Datos de entrada

Formato: un nombre por línea.

Ejemplo

```
abcab
banana
xn--pgo-seguro
```

Restricciones: nombres de hasta 10^6 caracteres (para textos largos, como registros completos) y hasta 10^6 nombres.

Tu programa debe

1. Construir el arreglo de sufijos de cada nombre (como en el ejercicio de los logs).
2. Calcular el arreglo LCP con Kasai.
3. Cada sufijo aporta tantas subcadenas nuevas como su largo menos el LCP con el sufijo anterior en el orden: $\text{total} = n(n + 1) / 2 - \text{suma del LCP}$.
4. Mostrar el arreglo de sufijos, el LCP y la cantidad de subcadenas distintas sobre el total.

Salida esperada

```
"abcab": sufijos ordenados 3 0 4 1 2 | LCP 0 2 0 1 0 | subcadenas distintas 12 de 15
"banana": sufijos ordenados 5 3 1 0 4 2 | LCP 0 1 3 0 0 2 | subcadenas distintas 15 de 21
"xn--pgo-seguro": sufijos ordenados 2 3 7 9 5 10 1 13 6 4 12 8 11 0 | LCP 0 1 1 0 0 1 0 0 1 0 0 0
0 0 | subcadenas distintas 101 de 105
```

Reglas

- Complejidad: $O(n \log^2 n)$ por el arreglo de sufijos y $O(n)$ por Kasai.
- Por convención, el LCP del primer sufijo es 0.
- No funcional: memoria $O(n)$: nunca se generan las subcadenas.
- No funcional: esta medida es una señal más en un sistema de puntaje de riesgo, no un veredicto por sí sola.

Pista (leela solo si te trabás): las subcadenas son los prefijos de los sufijos. Con los sufijos ordenados, los prefijos repetidos son exactamente los que un sufijo comparte con el anterior: el LCP.

Bonus (opcional)

- Contá las subcadenas distintas de largo exactamente k .
- Encontrá la subcadena que más se repite (de largo al menos 3).

Cierre: Contar subcadenas distintas con arreglos de sufijos se usa en compresión, bioinformática y análisis de la complejidad de textos.

Lote 18: ejercicios 426 a 450

Ejercicio 426: Mora que se propaga mes a mes

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 86/100

Situación: Un banco modela cada préstamo como una cadena de Markov mensual: al día, en mora de 30 días, en mora de 60, incobrable o cancelado. Los dos últimos estados son absorbentes (una vez ahí, no se sale). Con la matriz de transición, el área de riesgo proyecta la cartera mes a mes y calcula la probabilidad de que un préstamo que hoy está al día termine incobrable, para fijar las previsiones.

Datos de entrada

Formato: primera línea: los estados; después la matriz de transición mensual (fila = hoy); luego la cartera inicial (cantidad de préstamos por estado) y los meses a proyectar.

Ejemplo

```
al_día mora_30 mora_60 incobrable cancelado
0.90 0.07 0 0 0.03
0.40 0.30 0.30 0 0
0.15 0.10 0.25 0.50 0
0 0 0 1 0
0 0 0 0 1
1000 0 0 0 0
1 3 12 36
```

Restricciones: hasta 20 estados; meses hasta 10^4 .

Tu programa debe

1. Proyectar la cartera al mes m multiplicando el vector inicial por P^m (exponenciación rápida de matrices, con fracciones exactas).
2. Mostrar la cantidad esperada de préstamos en cada estado (1 decimal).
3. Para la probabilidad de absorción, tomar la submatriz Q de estados transitorios y resolver $(I - Q) \times x = R_{\text{incobrable}}$, donde $R_{\text{incobrable}}$ es la columna de probabilidades de pasar a incobrable (eliminación gaussiana).
4. Mostrar la probabilidad de terminar incobrable partiendo de "al día" (2 decimales, en %).

Salida esperada

```
Mes 1: al_día 900.0 | mora_30 70.0 | mora_60 0.0 | incobrable 0.0 | cancelado 30.0
Mes 3: al_día 791.0 | mora_30 86.0 | mora_60 30.5 | incobrable 10.5 | cancelado 82.1
Mes 12: al_día 510.1 | mora_30 58.5 | mora_60 25.0 | incobrable 147.6 | cancelado 258.8
Mes 36: al_día 161.5 | mora_30 18.5 | mora_60 7.9 | incobrable 329.8 | cancelado 482.3
Probabilidad final de terminar incobrable empezando al día: 41.42 %
```

Reglas

- Complejidad: $O(k^3 \log m)$ para la proyección y $O(t^3)$ para la absorción (t estados transitorios).
- Cada fila de la matriz suma 1; los estados absorbentes tienen un 1 en la diagonal.
- No funcional: regulatorio: los cálculos de previsiones tienen que ser reproducibles y auditables (fracciones exactas y redondeo solo al mostrar).
- No funcional: la matriz se estima con datos históricos; el informe debe indicar el período de los datos usados.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la probabilidad de terminar incobrable desde un estado es la de ir directo, más la de ir a otro estado transitorio multiplicada por la probabilidad de terminar incobrable desde ahí. Eso es un sistema lineal.

Bonus (opcional)

- Calculá la cantidad esperada de meses hasta que un préstamo se cancela o se vuelve incobrable (matriz fundamental $(I - Q)^{-1}$).
- Estimá la matriz a partir de un historial de préstamos (contando transiciones mes a mes).

Cierre: Las matrices de transición de mora son una herramienta estándar de riesgo crediticio y del cálculo de provisiones bancarias.

Ejercicio 427: Proyectos a equipos con capacidades

Nivel 9: Senior III · Matemática aplicada · Temática: Recursos humanos · Dificultad 86/100

Situación: Una software factory tiene varios proyectos para el trimestre, cada uno con horas estimadas y equipos que saben hacerlo. Cada equipo tiene horas disponibles. La gerencia quiere saber si se pueden cubrir todas las horas y cómo repartirlas: es un flujo máximo desde los proyectos hacia los equipos.

Datos de entrada

Formato: sección "proyectos": "proyecto horas equipos_capaces..."; sección "equipos": "equipo horas_disponibles".

Ejemplo

```
proyectos
App_mobile 120 Mobile Web
Portal_clientes 80 Web
Migración_datos 100 Datos Backend
API_pagos 90 Backend
equipos
Mobile 100
Web 110
Datos 60
Backend 120
```

Restricciones: hasta 10^3 proyectos y 10^2 equipos.

Tu programa debe

1. Armar la red: fuente → proyecto (capacidad = horas del proyecto), proyecto → equipo capaz (capacidad ilimitada), equipo → sumidero (capacidad = horas disponibles).
2. Calcular el flujo máximo (Edmonds-Karp).
3. Si el flujo es igual a la suma de horas pedidas, todo se puede cubrir; si no, informar cuántas faltan.
4. Mostrar el reparto (flujo en cada arista proyecto → equipo, en el orden de los equipos) y el uso de cada equipo.

Salida esperada

```
Horas pedidas 390 | horas asignables 380 | faltan 10 horas
App_mobile (120 h): Mobile 100 h, Web 20 h
Portal_clientes (80 h): Web 80 h
Migración_datos (100 h): Datos 60 h, Backend 40 h
API_pagos (90 h): Backend 80 h
Uso de equipos: Mobile 100/100, Web 100/110, Datos 60/60, Backend 120/120
```

Reglas

- Complejidad: $O(V \times E^2)$.
- El flujo en una arista es su capacidad original menos la capacidad residual final.
- Puede haber varios repartos óptimos; el algoritmo devuelve uno según el orden de exploración.
- No funcional: explicabilidad: si faltan horas, el corte mínimo indica qué equipos son el cuello de botella.
- No funcional: las estimaciones de horas son inciertas; el reparto se recalcula cada semana con los datos reales.

Pista (leela solo si te trabás): las horas son "agua" que sale de los proyectos y tiene que llegar a los equipos por los caños de compatibilidad. Si todo el agua llega, el plan es viable.

Bonus (opcional)

- Agregá un costo por hora distinto por equipo y buscá el reparto de costo mínimo (flujo de costo mínimo).
- Limitá cuántas horas puede aportar un equipo a un mismo proyecto (capacidad en la arista proyecto → equipo).

Cierre: Modelar asignaciones con capacidades como flujo es una técnica muy usada en planificación de proyectos, turnos y producción.

Ejercicio 428: El punto de equilibrio del bar

Nivel 9: Senior III · Matemática aplicada · Temática: Gastronomía · Dificultad 86/100

Situación: Un bar calcula su punto de equilibrio: cuántos clientes por mes necesita para no perder plata. Pero sus costos no son lineales: con muchos clientes hay que pagar horas extra y reforzar personal, y el costo crece más rápido. Por eso la ganancia sube, llega a un máximo y vuelve a bajar: hay dos puntos de equilibrio, que se encuentran con bisección.

Datos de entrada

Formato: primera línea: ticket promedio, costo fijo mensual, costo variable por cliente y coeficiente de saturación k ; segunda: rango de clientes a analizar.

Ganancia(q) = ticket $\times q$ – (costo fijo + variable $\times q$ + $k \times q^2$).

Ejemplo

```
4500 2800000 1800 0.35
0 20000
```

Restricciones: valores hasta 10^9 ; se usan decimales exactos con 100 iteraciones de bisección.

Tu programa debe

1. Encontrar el máximo de la ganancia: la derivada ticket – variable – $2kq$ se anula en $q^* = (\text{ticket} - \text{variable}) / (2k)$.
2. Aplicar bisección entre el inicio del rango y q^* para el primer punto de equilibrio (la ganancia cambia de signo ahí).
3. Aplicar bisección entre q^* y el final del rango para el segundo.
4. Mostrar la ganancia máxima, los dos puntos (2 decimales) y los clientes enteros a partir de los cuales hay ganancia y hasta los cuales se mantiene.

Salida esperada

```
Ganancia máxima en 3857.14 clientes por mes: $2407142.86
Primer punto de equilibrio: 1234.63 clientes (hay ganancia desde 1235)
Segundo punto de equilibrio: 6479.65 clientes (con más de 6479 vuelve a perder)
```

Reglas

- Complejidad: $O(\text{iteraciones})$ por raíz.
- La bisección necesita un intervalo con cambio de signo: por eso primero se separan las dos raíces usando el máximo.
- No funcional: robustez: si en un intervalo no hay cambio de signo (el bar nunca gana), informarlo en lugar de devolver cualquier valor.
- Aunque esta ganancia se podría resolver con la fórmula cuadrática, el método sirve igual para costos con cualquier forma.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): una función que sube y después baja cruza el cero dos veces. Partiéndola en el máximo, cada mitad tiene un solo cruce, y ahí la bisección funciona sin sorpresas.

Bonus (opcional)

- Verificá los resultados con la fórmula cuadrática y medí la diferencia.
- Agregá costos por escalones (cada 1.500 clientes hay que contratar un mozo más) y encontrá los puntos de equilibrio con la misma técnica.

Cierre: Encontrar raíces de funciones no lineales es parte del análisis de negocio, de la ingeniería y de la física.

Ejercicio 429: La melodía en otra tonalidad

Nivel 9: Senior III · Matemática aplicada · Temática: Música · Dificultad 86/100

Situación: Una app para músicos busca un motivo melódico dentro de una partitura, aunque aparezca en otra tonalidad (transpuesto): la misma melodía empezando más agudo o más grave. Comparar notas no sirve, pero sí comparar los intervalos entre notas consecutivas, que no cambian al transponer. La búsqueda se hace con KMP sobre las diferencias.

Datos de entrada

Formato: primera línea: el motivo; segunda: la melodía completa. Notas con nombre y octava (Do, Do#, Re, Re#, Mi, Fa, Fa#, Sol, Sol#, La, La#, Si), por ejemplo Sol4.

Ejemplo

```
Mi4 Mi4 Fa4 Sol4
Do4 Re4 La4 La4 La#4 Do5 Sol4 Re4 Re4 Re#4 Fa4 Mi4 Mi4 Fa4 Sol4 Sol4
```

Restricciones: melodías de hasta 10^6 notas.

Tu programa debe

1. Convertir cada nota a un número de semitonos: índice del nombre + $12 \times (\text{octava} + 1)$.
2. Calcular las diferencias entre notas consecutivas del motivo y de la melodía.
3. Buscar las diferencias del motivo dentro de las de la melodía con KMP.
4. Por cada aparición, mostrar la nota de inicio (numerada desde 1) y la transposición en semitonos respecto del motivo.

Salida esperada

```
Intervalos del motivo: +0 +1 +2
Aparece desde la nota 3 (La4), transpuesto +5 semitonos
Aparece desde la nota 8 (Re4), transpuesto -2 semitonos
Aparece desde la nota 12 (Mi4), transpuesto +0 semitonos
```

Reglas

- Complejidad: $O(n + m)$.
- Un motivo de m notas tiene $m - 1$ intervalos; una aparición en la posición i de las diferencias empieza en la nota i .
- No funcional: la conversión de nombres de notas se valida: una nota desconocida es un error de entrada, no se ignora.
- No funcional: la búsqueda tiene que ser instantánea mientras el usuario escribe el motivo.

Pista (leela solo si te trabás): transponer suma la misma cantidad a todas las notas; al restar notas consecutivas, esa cantidad se cancela. Buscás diferencias, no notas.

Bonus (opcional)

- Aceptá también el motivo invertido (intervalos con el signo cambiado) y en retrogradación (al revés).
- Tolerá ritmos distintos agregando la duración de cada nota y comparando solo las alturas.

Cierre: Buscar patrones invariantes a transformaciones (transposición, escala) es una técnica central en recuperación de información musical.

Ejercicio 430: Superficie vigilada por cámaras

Nivel 9: Senior III · Matemática aplicada · Temática: Seguridad informática · Dificultad 86/100

Situación: Una empresa de seguridad instala cámaras en un depósito; cada cámara cubre un rectángulo del plano. Para cotizar, necesita la superficie realmente vigilada: sumar las áreas cuenta dos veces las zonas superpuestas. El barrido con una línea vertical que recorre el plano calcula el área de la unión, y también la zona cubierta por dos o más cámaras (redundancia).

Datos de entrada

Formato: primera línea: el rectángulo del local "x0 y0 x1 y1"; después "cámara x0 y0 x1 y1" (enteros, en metros).

Ejemplo

```
0 0 100 60
cam1 0 0 40 30
cam2 30 20 70 50
cam3 60 0 100 25
cam4 35 25 45 35
```

Restricciones: hasta 10^5 cámaras (con árbol de segmentos); en el ejercicio, barrido simple.

Tu programa debe

1. Tomar todas las coordenadas x distintas de los bordes y ordenarlas: definen franjas verticales.
2. En cada franja, considerar los rectángulos que la atraviesan completa y barrer sus bordes en y con un contador (+1 al entrar, -1 al salir) para medir el largo cubierto al menos una vez (o al menos dos veces).
3. Sumar largo cubierto × ancho de la franja.
4. Mostrar la suma de áreas, la unión, la zona con redundancia y la cobertura del local (2 decimales).

Salida esperada

```
Suma de áreas de las cámaras: 3500 m²
Superficie vigilada (unión): 3250 m²
Superficie vigilada por 2 o más cámaras: 225 m²
Cobertura del local: 54.17 % (2750 m² sin vigilar)
```

Reglas

- Complejidad: $O(n^2 \log n)$ con este barrido; $O(n \log n)$ con un árbol de segmentos sobre las y comprimidas (ver el ejercicio de las parcelas superpuestas).
- Se asume que las cámaras están dentro del local.
- No funcional: exactitud: con coordenadas enteras, todas las áreas son enteras.
- No funcional: el informe al cliente incluye las zonas sin vigilar, no solo el porcentaje.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): entre dos coordenadas x consecutivas, el conjunto de cámaras que "tocan" la franja no cambia: el área cubierta ahí es un problema de una dimensión (largo cubierto en y) por el ancho.

Bonus (opcional)

- Implementá el árbol de segmentos que mantiene el largo cubierto al agregar y quitar intervalos.
- Encontrá el punto del local más lejano a toda cámara (para ubicar una nueva).

Cierre: El barrido es una técnica central de geometría computacional para áreas, intersecciones y eventos en el plano.

Ejercicio 431: Tasas forward a partir de la curva

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 87/100

Situación: Una mesa de dinero quiere la curva de tasas implícita en los bonos que cotizan: cuánto vale hoy \$1 a pagar dentro de 1, 2, 3 y 4 años (factores de descuento), las tasas cero y las tasas forward (la tasa implícita para prestar entre el año k y el $k + 1$). Cada bono da una ecuación lineal con los factores de descuento como incógnitas.

Datos de entrada

Formato: primera línea: valor nominal; después "años cupón_anual_% precio" por bono (cupones anuales, amortización al vencimiento; un bono por cada plazo de 1 a n).

Ejemplo

```
100
1 5 99.5
2 6 100.2
3 7 101.8
4 7 100.4
```

Restricciones: hasta 50 plazos.

Tu programa debe

1. Plantear, para cada bono, $\text{precio} = \text{suma de } \text{pago}_k \times \text{DF}_k$ (DF_k = factor de descuento del año k), con $\text{pago}_k = \text{cupón}$ y el último pago = cupón + nominal.
2. Resolver el sistema (triangular) con eliminación gaussiana y fracciones exactas (bootstrapping).
3. Tasa cero del año k : $\text{DF}_k^{(-1/k)} - 1$; forward de k a $k + 1$: $\text{DF}_k / \text{DF}_{(k+1)} - 1$.
4. Mostrar los factores (6 decimales), las tasas cero y las forward (4 decimales, en %).

Salida esperada

```
Año 1: factor de descuento 0.947619 | tasa cero 5.5276 %
Año 2: factor de descuento 0.891644 | tasa cero 5.9020 %
Año 3: factor de descuento 0.831076 | tasa cero 6.3620 %
Año 4: factor de descuento 0.763623 | tasa cero 6.9745 %
Forward 1->2: 6.2777 %
Forward 2->3: 7.2879 %
Forward 3->4: 8.8334 %
```

Reglas

- Complejidad: $O(n^2)$ por ser triangular ($O(n^3)$ con eliminación general).
- El sistema es triangular porque el bono de k años solo tiene pagos hasta el año k : se puede resolver de a un plazo ("bootstrapping").
- No funcional: validación: factores de descuento que no bajan con el plazo o tasas forward negativas son señales de precios inconsistentes y deben informarse.
- No funcional: exactitud: los factores se calculan con fracciones y las tasas se redondean solo al mostrar.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): el bono de 1 año tiene un solo pago: de ahí sale DF_1 directo. El de 2 años tiene dos pagos, y DF_1 ya lo conocés: despejás DF_2 . Y así, escalón por escalón.

Bonus (opcional)

- Interpolá la curva para plazos intermedios (1,5 años) con interpolación lineal de las tasas cero.
- Con más bonos que plazos, ajustá la curva por mínimos cuadrados (ver el ejercicio del pronóstico de demanda).

Cierre: El bootstrapping de la curva de tasas es el primer paso de la valuación de cualquier instrumento de renta fija y derivado de tasas.

Ejercicio 432: ¿Se puede llegar en tren?

Nivel 9: Senior III · Matemática aplicada · Temática: Transporte · Dificultad 87/100

Situación: Un planificador de viajes en tren quiere responder al instante si se puede ir de una estación a otra (con cualquier cantidad de combinaciones), sin calcular distancias. Algunos servicios son de una sola mano en ciertos horarios. Es el cierre transitivo del grafo, que se calcula con la variante booleana de Floyd-Warshall usando bits.

Datos de entrada

Formato: servicios dirigidos "desde hasta"; una línea "?" y consultas "desde hasta". Las estaciones se numeran por orden de aparición.

Ejemplo

```
Retiro Tigre
Tigre Retiro
Retiro LaPlata
Constitución LaPlata
LaPlata Constitución
Once Moreno
Moreno Luján
Retiro Once
?
Tigre Luján
Luján Retiro
Constitución Tigre
LaPlata Once
```

Restricciones: hasta 5.000 estaciones (con filas de bits, $n^3 / 64$ operaciones).

Tu programa debe

1. Representar la fila de alcanzables de cada estación como un número de n bits (cada estación se alcanza a sí misma).
2. Para cada intermedia k y cada estación i que alcanza a k : $\text{fila}[i] = \text{fila}[i] \text{ OR } \text{fila}[k]$ (Warshall con bits).
3. Responder cada consulta mirando un bit; mostrar la matriz (1 = alcanzable, . = no).
4. Agrupar las estaciones que se alcanzan mutuamente (ida y vuelta) y mostrar los grupos.

Salida esperada

```
Retiro: 1111111
Tigre: 1111111
LaPlata: ..11...
Constitución: ..11...
Once: ....111
Moreno: .....11
Luján: .....1
¿Tigre -> Luján? sí
¿Luján -> Retiro? no
¿Constitución -> Tigre? no
¿LaPlata -> Once? no
Grupos con ida y vuelta: Retiro, Tigre | LaPlata, Constitución | Once | Moreno | Luján
```

Reglas

- Complejidad: $O(n^3 / \text{tamaño de palabra})$ con operaciones de bits.
- Los grupos con ida y vuelta son las componentes fuertemente conexas (en el nivel 6 se calcularon con otro método).
- No funcional: memoria: n^2 bits (unos 3 MB para 5.000 estaciones).
- No funcional: la matriz se recalcula cuando cambian los servicios; las consultas del día son $O(1)$.

Pista (leela solo si te trabás): "i llega a k" y "k llega a j" implica "i llega a j". Con bits, agregar todo lo que k alcanza a la fila de i es un solo OR.

Bonus (opcional)

- Compará con un BFS desde cada estación ($O(n \times (n + m))$) según qué tan denso es el grafo.
- Agregá horarios: ¿se puede ir llegando antes de las 22:00? (ver el ejercicio de connection scan del nivel 6).

Cierre: El cierre transitivo responde preguntas de alcanzabilidad en redes de transporte, permisos heredados y dependencias de software.

Ejercicio 433: Tutores y alumnos

Nivel 9: Senior III · Matemática aplicada · Temática: Educación · Dificultad 87/100

Situación: Un programa de tutorías universitario empareja alumnos que necesitan apoyo en una materia con tutores que la dominan; cada tutor acompaña a un solo alumno por cuatrimestre. Con miles de alumnos y tutores, el algoritmo de Kuhn puede ser lento; Hopcroft-Karp busca muchos caminos de aumento a la vez, por fases.

Datos de entrada

Formato: tutores "nombre: materias..."; una línea "?" y alumnos "nombre materia".

Ejemplo

```
Ana: álgebra física
Bruno: álgebra
Carla: física química
Diego: química
Eva: álgebra física química
?
alumno1 álgebra
alumno2 física
alumno3 álgebra
alumno4 química
alumno5 física
alumno6 álgebra
```

Restricciones: hasta 10^5 alumnos y tutores; hasta 10^6 compatibilidades.

Tu programa debe

1. Armar el grafo bipartito alumno $\rightarrow$ tutores compatibles (en el orden de la entrada).
2. En cada fase, hacer un BFS desde todos los alumnos libres a la vez para calcular niveles, hasta encontrar tutores libres.
3. Hacer DFS desde cada alumno libre siguiendo solo aristas que suben un nivel, para encontrar caminos de aumento disjuntos y aplicarlos todos.
4. Repetir hasta que no haya caminos. Mostrar los emparejamientos, la cantidad, las fases y los alumnos sin tutor.

Salida esperada

```
alumno1 -> Ana
alumno2 -> Carla
alumno3 -> Bruno
alumno4 -> Diego
alumno5 -> Eva
Alumnos con tutor: 5 de 6 (fases de Hopcroft-Karp: 1)
Sin tutor: alumno6
```

Reglas

- Complejidad esperada: $O(E \sqrt{V})$.
- La cantidad de fases es $O(\sqrt{V})$: en cada una, el camino de aumento más corto se alarga.
- No funcional: escalabilidad: con 10^5 alumnos, la diferencia con Kuhn es de segundos contra minutos.
- No funcional: equidad: si hay más alumnos que tutores, el criterio de quién queda afuera tiene que ser explícito (bonus).

Pista (leela solo si te trabás): Kuhn busca un camino de aumento por vez; Hopcroft-Karp encuentra en una sola pasada muchos caminos cortos que no se pisan, y así reduce la cantidad de rondas.

Bonus (opcional)

- Dale prioridad a los alumnos de primer año (procesa los primero y verificá que ninguno pierda su tutor en fases posteriores).
- Permití que cada tutor acompañe hasta 2 alumnos (duplicá el nodo o usá flujo).

Cierre: Hopcroft-Karp es el algoritmo de referencia para emparejamiento bipartito máximo en grafos grandes.

Ejercicio 434: El candado de combinación circular

Nivel 9: Senior III · Matemática aplicada · Temática: Seguridad informática · Dificultad 87/100

Situación: Un fabricante de candados con discos circulares descubrió que dos combinaciones que son rotaciones una de otra abren el mismo candado (el mecanismo no tiene punto de inicio). Para auditar la base de combinaciones asignadas y evitar duplicados, cada combinación se lleva a una forma canónica: su rotación lexicográficamente mínima.

Datos de entrada

Formato: una combinación por línea (dígitos separados por espacios, en orden circular).

Ejemplo

```
3 1 4 1 5
1 5 3 1 4
4 1 5 3 1
2 7 1 8 2
1 8 2 2 7
5 1 3 1 4
```

Restricciones: combinaciones de hasta 10^6 posiciones y 10^6 combinaciones.

Tu programa debe

1. Para cada combinación, encontrar la rotación mínima con el algoritmo de Booth (una variante de la función de fallos de KMP sobre la cadena duplicada), en tiempo lineal.
2. Construir la forma canónica rotando desde esa posición.
3. Agrupar las combinaciones por forma canónica.
4. Mostrar la posición y la forma canónica de cada una, y la cantidad de combinaciones realmente distintas.

Salida esperada

```
3 1 4 1 5: rotación mínima desde la posición 1 -> 1 4 1 5 3
1 5 3 1 4: rotación mínima desde la posición 3 -> 1 4 1 5 3
4 1 5 3 1: rotación mínima desde la posición 4 -> 1 4 1 5 3
2 7 1 8 2: rotación mínima desde la posición 2 -> 1 8 2 2 7
1 8 2 2 7: rotación mínima desde la posición 0 -> 1 8 2 2 7
5 1 3 1 4: rotación mínima desde la posición 1 -> 1 3 1 4 5
Combinaciones realmente distintas: 3 de 6
```

Reglas

- Complejidad esperada: $O(n)$ por combinación (probar todas las rotaciones es $O(n^2)$).
- Si hay varias posiciones que dan la misma rotación mínima (combinaciones periódicas), se toma la primera.
- No funcional: seguridad: la auditoría trabaja sobre las formas canónicas; las combinaciones reales no se muestran en los informes (acá se muestran solo para verificar).
- No funcional: la forma canónica permite guardar las combinaciones en un conjunto y detectar duplicados en $O(1)$.

Pista (leela solo si te trabás): duplicar la secuencia ($s + s$) hace que todas las rotaciones sean subcadenas de largo n . Booth recorre esa cadena una vez, descartando candidatos que ya sabe que no pueden ganar.

Bonus (opcional)

- Compará con el método simple de dos punteros (algoritmo de Duval o "mínima representación") y verificá que dan lo mismo.

- Si el candado también se puede dar vuelta (leer al revés), la forma canónica es la mínima entre las rotaciones de s y de s invertida.

Cierre: La rotación lexicográfica mínima se usa para canonizar datos circulares: moléculas cíclicas, polígonos, collares y combinaciones.

Ejercicio 435: Trayectoria del proyectil del juego

Nivel 9: Senior III · Matemática aplicada · Temática: Videojuegos · Dificultad 87/100

Situación: En un juego de artillería por turnos, la inteligencia artificial del rival tiene que calcular el ángulo de disparo para acertar un objetivo a cierta distancia y altura, con una velocidad de salida fija. La ecuación de la trayectoria no se despeja fácilmente para el ángulo; con Newton-Raphson se encuentran las dos soluciones (tiro bajo y tiro alto) partiendo de dos ángulos iniciales.

Datos de entrada

Formato: primera línea: velocidad de salida (m/s) y gravedad (m/s²); después, objetivos "distancia altura" en metros.

Altura del proyectil al recorrer d metros con ángulo θ : $y(\theta) = d \times \tan \theta - g \times d^2 / (2 \times v^2 \times \cos^2 \theta)$. Se busca $y(\theta) = \text{altura del objetivo}$.

Ejemplo

```
30 9.8
60 5
80 0
100 10
```

Restricciones: hasta 10^4 cálculos por segundo (un cálculo por disparo de cada IA).

Tu programa debe

1. Definir $f(\theta) = y(\theta) - \text{altura}$ y su derivada $f'(\theta) = d / \cos^2 \theta - g \times d^2 \times \sin \theta / (v^2 \times \cos^3 \theta)$.
2. Aplicar Newton desde 20° (tiro bajo) y desde 70° (tiro alto), hasta que el cambio sea menor a 10^{-12} radianes (máximo 50 iteraciones).
3. Descartar soluciones que no convergen o quedan fuera de $(0^\circ, 90^\circ)$; si no queda ninguna, el objetivo está fuera de alcance.
4. Mostrar los ángulos en grados (4 decimales) con sus iteraciones.

Salida esperada

```
Objetivo a 60 m y 5 m de altura: ángulos 25.9984° (4 iteraciones) y 68.7653° (5 iteraciones)
Objetivo a 80 m y 0 m de altura: ángulos 30.2940° (5 iteraciones) y 59.7060° (7 iteraciones)
Objetivo a 100 m y 10 m de altura: fuera de alcance
```

Reglas

- Complejidad: $O(\text{iteraciones})$ por disparo.
- Sin rozamiento del aire (en el bonus se agrega).
- No funcional: tiempo real: el cálculo no puede demorar el cuadro del juego (menos de 1 ms).
- No funcional: robustez: un objetivo inalcanzable no puede colgar el juego ni devolver ángulos absurdos; se informa.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la función tiene forma de loma: sube con el ángulo, llega a un máximo y baja. Por eso puede cortar a la altura del objetivo dos veces, y cada punto de partida de Newton "cae" en una.

Bonus (opcional)

- Agregá rozamiento del aire: la trayectoria ya no tiene fórmula y hay que simularla por pasos; usá bisección sobre el ángulo.
- Hacé que la IA falle a propósito según la dificultad, sumando un error aleatorio con semilla al ángulo.

Cierre: Resolver ecuaciones de física con métodos numéricos es cotidiano en motores de videojuegos, simuladores y balística.

Ejercicio 436: Parcelas superpuestas

Nivel 9: Senior III · Matemática aplicada · Temática: Agricultura · Dificultad 87/100

Situación: El catastro rural de una provincia recibe declaraciones de lotes rectangulares y detecta que algunas se superponen: dos productores declaran la misma tierra. Con cientos de miles de lotes, el barrido simple es lento. Un árbol de segmentos sobre las coordenadas y mantiene, mientras una línea vertical barre el mapa, cuánto largo está cubierto al menos una vez y al menos dos veces.

Datos de entrada

Formato: un lote por línea: "nombre x0 y0 x1 y1" (enteros, en metros).

Ejemplo

```
lote_1 0 0 50 40
lote_2 30 20 80 60
lote_3 45 35 70 90
lote_4 60 0 90 30
```

Restricciones: hasta 10^5 lotes; coordenadas hasta 10^9 .

Tu programa debe

1. Comprimir las coordenadas y y crear eventos: al llegar a x_0 el lote entra (+1 en su tramo de y) y en x_1 sale (−1).
2. Ordenar los eventos por x y recorrerlos: antes de aplicar cada evento, sumar ($\text{largo cubierto} \geq 1$) $\times$ avance en x a la unión y ($\text{largo cubierto} \geq 2$) $\times$ avance a la superposición.
3. El árbol guarda por nodo cuántos lotes cubren todo su tramo y el largo cubierto al menos 1 y al menos 2 veces, combinando con los hijos.
4. Mostrar la suma declarada, la unión y la superficie en conflicto.

Salida esperada

```
Suma de superficies declaradas: 6275 m²
Superficie real (unión): 5050 m²
Superficie en conflicto (2 o más lotes): 1200 m²
```

Reglas

- Complejidad esperada: $O(n \log n)$.
- No hace falta propagación perezosa: cada intervalo que entra, sale después exactamente igual, así que el contador de un nodo nunca se "reparte".
- No funcional: exactitud: con coordenadas enteras, todo es entero (en 64 bits: $10^9 \times 10^9 = 10^{18}$).
- No funcional: el informe identifica los pares de lotes en conflicto para notificar a los productores (bonus).

Pista (leela solo si te trabás): si un nodo está cubierto 2 veces completo, todo su tramo cuenta doble; si está cubierto 1 vez, su largo "doble" es el largo cubierto al menos una vez de sus hijos (una capa más encima).

Bonus (opcional)

- Listá los pares de lotes que se superponen (detección de pares con barrido y un árbol de intervalos).
- Calculá la superficie cubierta por exactamente un lote.

Cierre: El barrido con árbol de segmentos es la técnica estándar para áreas de uniones y superposiciones en catastro, diseño de chips y mapas.

Ejercicio 437: Precios de la canasta a partir de tickets

Nivel 9: Senior III · Matemática aplicada · Temática: E-commerce · Dificultad 88/100

Situación: Una app de ahorro reconstruye los precios de un supermercado a partir de tickets que suben los usuarios: cada ticket dice qué cantidades se compraron y cuánto se pagó en total, pero no el precio de cada producto. Con tantos tickets como productos (y linealmente independientes), los precios salen de un sistema lineal. Un ticket extra sirve para detectar cambios de precio.

Datos de entrada

Formato: primera línea: los productos; después un ticket por línea "cantidades... = total"; una línea "?" y tickets de control.

Ejemplo

```
leche pan yerba
2 1 1 = 7300
1 3 0 = 5700
0 2 2 = 11400
?
1 1 1 = 6400
```

Restricciones: hasta 500 productos.

Tu programa debe

1. Armar la matriz de cantidades y el vector de totales.
2. Resolver el sistema con eliminación gaussiana y fracciones exactas.
3. Para cada ticket de control, calcular el total esperado con los precios deducidos y compararlo con el cobrado.
4. Mostrar los precios (2 decimales) y el resultado de cada control.

Salida esperada

```
Precios deducidos: leche $800.00, pan $1633.33, yerba $4066.67
Ticket de control 1 1 1: esperado $6500.00, cobrado $6400 -> DIFERENCIA de -$100.00 (¿cambió algún precio?)
```

Reglas

- Complejidad: $O(n^3)$.
- Si los tickets no son independientes (uno es combinación de otros), el sistema no tiene solución única: hay que informarlo.
- No funcional: privacidad: los tickets se usan sin datos personales de quien los subió.
- No funcional: tolerancia: en la realidad hay redondeos de centavos; un control debe aceptar diferencias menores a \$1 (acá se exige igualdad exacta).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): cada ticket es una ecuación: $2 \times \text{leche} + 1 \times \text{pan} + 1 \times \text{yerba} = 7.300$. Tres tickets, tres incógnitas: es el mismo sistema que resolviste con los circuitos.

Bonus (opcional)

- Con más tickets que productos (y pequeños errores), estimá los precios por mínimos cuadrados.
- Detectá cuál producto cambió de precio usando varios tickets de control.

Cierre: Resolver sistemas lineales permite inferir valores ocultos a partir de observaciones agregadas, desde precios hasta mediciones científicas.

Ejercicio 438: El palíndromo más largo de los posteos

Nivel 9: Senior III · Matemática aplicada · Temática: Redes sociales · Dificultad 88/100

Situación: Una red social lanza un desafío de palíndromos: el posteo que tenga el palíndromo más largo gana. Para calcularlo sobre millones de posteos, probar cada centro y expandir es cuadrático en el peor caso. El algoritmo de Manacher reutiliza los palíndromos ya encontrados (por simetría) y lo hace en tiempo lineal.

Datos de entrada

Formato: un posteo por línea.

Normalización: minúsculas y solo letras y dígitos (se quitan espacios y signos). Las letras con tilde se conservan como están.

Ejemplo

```
Anita lava la tina
Somos o no somos, reconocer es la cuestión
Hoy salimos a las 20:02 en punto
```

Restricciones: posteos de hasta 10^5 caracteres; 10^7 posteos por día.

Tu programa debe

1. Normalizar el texto e intercalar un separador entre caracteres (así los palíndromos de largo par e impar se tratan igual).
2. Recorrer las posiciones manteniendo el palíndromo que llega más a la derecha (centro c , borde r); para una posición dentro de él, arrancar con el radio de su espejo respecto de c , acotado por r .
3. Expandir desde ahí solo lo que falte y actualizar c y r .
4. Mostrar el palíndromo más largo (el primero si hay empate) y la cantidad total de subcadenas palíndromas (con repeticiones).

Salida esperada

```
"Anita lava la tina" -> normalizado 15 caracteres | palíndromo más largo: "anitalavalatina" (15) |
    subcadenas palíndromas: 24
"Somos o no somos, reconocer es la cuestión" -> normalizado 34 caracteres | palíndromo más largo:
    "somosonosomos" (13) | subcadenas palíndromas: 51
"Hoy salimos a las 20:02 en punto" -> normalizado 25 caracteres | palíndromo más largo: "salas"
    (5) | subcadenas palíndromas: 29
```

Reglas

- Complejidad esperada: $O(n)$: el borde r solo avanza.
- En la cadena con separadores, el radio en una posición es exactamente el largo del palíndromo original centrado ahí.
- La cantidad de palíndromos centrados en una posición es $(\text{radio} + 1) / 2$ con división entera.
- No funcional: la normalización tiene que ser idéntica para todos los posteos (se publica en las bases del desafío).

Pista (leela solo si te trabás): dentro de un palíndromo grande, lo que pasa a la derecha del centro es un espejo de lo que pasa a la izquierda. Si ya mediste el palíndromo del lado izquierdo, del derecho sabés al menos eso.

Bonus (opcional)

- Encontrá el palíndromo más largo que sea una palabra completa del posteo original.
- Contá los palíndromos distintos (no con repeticiones) con un árbol de palíndromos (eertree).

Cierre: El algoritmo de Manacher es un clásico de strings lineal, útil en análisis de texto y en bioinformática (secuencias palindrómicas del ADN).

Ejercicio 439: Bicicletas públicas con demandas por estación

Nivel 9: Senior III · Matemática aplicada · Temática: Transporte · Dificultad 88/100

Situación: El sistema de bicicletas públicas de una ciudad amanece desbalanceado: algunas estaciones tienen bicis de más y otras de menos. Camionetas las redistribuyen por rutas con una capacidad máxima por noche. La pregunta es si se puede cubrir toda la demanda: es un problema de flujo con demandas, que se reduce a flujo máximo con una fuente y un sumidero virtuales.

Datos de entrada

Formato: estaciones "nombre balance" (positivo = sobran, negativo = faltan); una línea "?" y rutas dirigidas "desde hasta capacidad" (bicis por noche).

Ejemplo

```
Plaza_Italia 8
Retiro -3
Congreso 5
Palermo_Soho -4
Once -6
?
Plaza_Italia Palermo_Soho 5
Plaza_Italia Retiro 4
Congreso Once 4
Congreso Retiro 2
Plaza_Italia Once 3
Retiro Once 2
```

Restricciones: hasta 10^4 estaciones y 10^5 rutas.

Tu programa debe

1. Conectar una fuente virtual a cada estación con sobrante (capacidad = sobrante) y cada estación con faltante a un sumidero virtual (capacidad = faltante).
2. Calcular el flujo máximo sobre la red con las rutas.
3. Es factible si el flujo es igual al faltante total.
4. Mostrar los totales, la factibilidad y cuántas bicis viajan por cada ruta usada (en el orden de la entrada).

Salida esperada

```
Sobrante total 13 | faltante total 13 | bicis que se pueden mover 13
FACTIBLE: se cubren todas las estaciones con faltante
Plaza_Italia -> Palermo_Soho: 4 bicis (capacidad 5)
Plaza_Italia -> Retiro: 3 bicis (capacidad 4)
Congreso -> Once: 4 bicis (capacidad 4)
Congreso -> Retiro: 1 bici (capacidad 2)
Plaza_Italia -> Once: 1 bici (capacidad 3)
Retiro -> Once: 1 bici (capacidad 2)
```

Reglas

- Complejidad: $O(V \times E^2)$ con Edmonds-Karp.
- No todo el sobrante tiene que moverse; lo que importa es cubrir los faltantes.
- Con cotas inferiores en las rutas (mínimo de bicis por ruta), el problema se transforma igual ajustando los balances (bonus).
- No funcional: el plan se calcula de madrugada y tiene que estar listo antes de que salgan las camionetas (minutos, no horas).
- No funcional: si no es factible, el informe dice cuántas bicis faltan y qué rutas están saturadas.

Pista (leela solo si te trabás): las estaciones con bicis de más son "fuentes chicas" y las que tienen de menos, "sumideros chicos". Juntarlas en una fuente y un sumidero únicos convierte el problema en el flujo máximo que ya sabés resolver.

Bonus (opcional)

- Agregá cotas inferiores: cada ruta tiene que llevar al menos 1 bici (para no desperdiciar el viaje de la camioneta).
- Minimizá los kilómetros recorridos con un flujo de costo mínimo.

Cierre: El flujo con demandas modela redistribución de inventario, logística inversa y balanceo de redes de distribución.

Ejercicio 440: Visibilidad desde la torre forestal

Nivel 9: Senior III · Matemática aplicada · Temática: Clima y energía · Dificultad 88/100

Situación: Una torre de vigilancia forestal tiene una cámara que gira en sentido horario empezando por el norte. El sistema tiene que ordenar los focos de incendio reportados según el orden en que la cámara los va a enfocar, y marcar cuáles están tapados por un cerro (representado como un segmento) y necesitan confirmación de otra torre.

Datos de entrada

Formato: primera línea: posición de la torre; después "foco x y" (x hacia el este, y hacia el norte, en metros); una línea "?" y los cerros "nombre x1 y1 x2 y2".

Ejemplo

```
0 0
fuego_1 50 80
fuego_2 -40 30
fuego_3 100 160
fuego_4 30 -60
fuego_5 -80 -20
fuego_6 120 10
?
cerro_A 20 40 60 40
cerro_B 90 -20 90 40
```

Restricciones: hasta 10^5 focos y 10^3 cerros.

Tu programa debe

1. Llevar las coordenadas a la torre como origen.
2. Ordenar los focos por rumbo horario desde el norte sin usar ángulos: separar en dos mitades (rumbos de 0° a 180° y de 180° a 360°) y, dentro de cada mitad, comparar con el producto cruzado (en el mismo rumbo, primero el más cercano).
3. Para cada foco, verificar si el segmento torre $\rightarrow$ foco cruza algún cerro (prueba de intersección de segmentos con orientaciones).
4. Mostrar los focos en orden, con su rumbo (1 decimal, solo informativo) y si están visibles o tapados.

Salida esperada

```
fuego_1 (rumbo 32.0°): tapado por cerro_A
fuego_3 (rumbo 32.0°): tapado por cerro_A
fuego_6 (rumbo 85.2°): tapado por cerro_B
fuego_4 (rumbo 153.4°): VISIBLE
fuego_5 (rumbo 256.0°): VISIBLE
fuego_2 (rumbo 306.9°): VISIBLE
```

Reglas

- Complejidad: $O(n \log n)$ para ordenar y $O(n \times m)$ para la visibilidad (con m cerros).
- El ordenamiento se hace con enteros; el rumbo en grados se calcula solo para mostrarlo.
- No funcional: seguridad: ante un caso dudoso (la visual roza el extremo de un cerro), el foco se marca como visible y además se pide confirmación (acá se cuenta como no tapado).
- No funcional: la lista ordenada se recalcula en cada reporte nuevo en milisegundos.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): los ángulos con decimales traen problemas de precisión; comparar dos direcciones con el producto cruzado dice exactamente cuál viene antes, siempre que estén en la misma mitad del giro.

Bonus (opcional)

- Con muchos cerros, usá un barrido angular para decidir la visibilidad en $O((n + m) \log(n + m))$.
- Combiná dos torres: triangulá la posición de un foco a partir de los rumbos que informa cada una.

Cierre: El ordenamiento angular y la visibilidad son la base de radares, sensores, iluminación en videojuegos y planificación de antenas.

Ejercicio 441: Sensibilidad de la cartera de bonos

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 88/100

Situación: El área de riesgo de un fondo tiene que informar cuánto perdería la cartera de bonos si las tasas suben. En lugar de fórmulas analíticas para cada tipo de bono, usa derivadas numéricas: recalcula el precio con la tasa un poquito más alta y un poquito más baja (diferencias centrales). Así obtiene la sensibilidad (DV01), la duración y la convexidad de cualquier instrumento.

Datos de entrada

Formato: un bono por línea: "nombre nominal cupón_% años rendimiento_%" (cupones anuales); una línea "?" y el paso h para las derivadas (en tasa, $0,0001 = 1$ punto básico).

Ejemplo

```
bono_A 1000 8 5 10
bono_B 1000 4 10 10
bono_C 1000 0 3 10
?
0.0001
```

Restricciones: carteras de hasta 10^5 bonos, recalculadas cada minuto.

Tu programa debe

1. Calcular el precio a la tasa y , a $y + h$ y a $y - h$.
2. Derivada $\approx (P(y + h) - P(y - h)) / (2h)$; DV01 = $-\text{derivada} \times 0,0001$ (cuánto cae el precio si la tasa sube 1 punto básico); duración modificada = $-\text{derivada} / P$.
3. Convexidad $\approx (P(y + h) - 2P(y) + P(y - h)) / (h^2 \times P)$.
4. Sumar para la cartera y estimar el cambio ante +1 punto de tasa con $-\text{DV01} \times 100 + \frac{1}{2} \times \text{convexidad} \times \text{valor} \times 0,01^2$; compararlo con la revaluación exacta.

Salida esperada

```
bono_A: precio 924.18 | DV01 0.3597 | duración modificada 3.8922 | convexidad 20.10
bono_B: precio 631.33 | DV01 0.4561 | duración modificada 7.2241 | convexidad 66.76
bono_C: precio 751.31 | DV01 0.2049 | duración modificada 2.7273 | convexidad 9.92
Cartera: valor 2306.83 | DV01 1.0207 | ante +1 punto de tasa: -98.66 (estimado con duración y
convexidad)
Revaluación exacta ante +1 punto: -98.76
```

Reglas

- Complejidad: 3 valuaciones por instrumento.
- Las diferencias centrales tienen error $O(h^2)$; un h demasiado chico amplifica los errores de redondeo (hay un h óptimo, alrededor de 10^{-4} a 10^{-5} para estos cálculos).
- No funcional: consistencia: todos los instrumentos se revalúan con el mismo h y la misma convención, para que los riesgos se puedan sumar.
- No funcional: los resultados se validan contra la revaluación completa ante escenarios grandes (por eso se muestran ambas).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): no necesitás derivar ninguna fórmula: si sabés calcular el precio, sabés estimar cuánto cambia cuando movés un poquito la tasa. Mover hacia los dos lados cancela el error principal.

Bonus (opcional)

- Probá con $h = 10^{-2}$, 10^{-4} , 10^{-8} y 10^{-12} y graficá el error contra la derivada analítica (ejercicio del precio del bono).

- Calculá la sensibilidad por plazo ("key rate duration") moviendo solo un tramo de la curva.

Cierre: Las derivadas numéricas son la forma estándar de medir sensibilidades en sistemas de riesgo con miles de instrumentos distintos.

Ejercicio 442: Un logo escondido en la imagen

Nivel 9: Senior III · Matemática aplicada · Temática: Redes sociales · Dificultad 89/100

Situación: Una red social detecta si una imagen contiene un logo registrado (para marcar publicidad no declarada). Simplificando, la imagen y el logo son grillas de píxeles. Comparar el logo en cada posición es $O(\text{tamaño de imagen} \times \text{tamaño de logo})$; con un hash rodante en dos dimensiones, cada ventana se evalúa en $O(1)$ y solo se verifican a fondo las que coinciden.

Datos de entrada

Formato: filas y columnas del logo, el logo; filas y columnas de la imagen, la imagen (cada carácter es un píxel).

Hash de filas: $h = h \times 131 + \text{código}(c) \text{ resto } 1.000.000.007$ sobre ventanas de tantas columnas como el logo; hash de la ventana: combinar los hashes de fila de arriba hacia abajo con base 137, del mismo modo.

Ejemplo

```
3 3
#.#
.#.
#.#
8 10
..#.#....
...#.....
..#.#.#.#
.....#.#
.#.#...#.#
..#.....
.#.#...#.#
.....#..
```

Restricciones: imágenes de hasta 4.000×4.000 y logos de hasta 500×500 .

Tu programa debe

1. Calcular, para cada fila de la imagen, el hash rodante de cada ventana horizontal del ancho del logo.
2. Para cada columna de inicio, deslizar verticalmente una ventana de tantas filas como el logo, combinando los hashes de fila con otro hash rodante (base 137).
3. Cuando el hash coincide con el del logo, verificar carácter por carácter para descartar colisiones.
4. Mostrar el hash del logo, las apariciones (fila, columna de la esquina superior izquierda) y la cantidad de ventanas revisadas.

Salida esperada

```
Hash del logo: 496466912
Apariciones (fila, columna desde 0): (0,2) (2,7) (4,1)
Ventanas revisadas: 48
```

Reglas

- Complejidad esperada: $O(\text{filas} \times \text{columnas})$ de la imagen, más la verificación de las coincidencias.
- Usar bases distintas para filas y columnas evita que el hash sea simétrico (una ventana transpuesta daría lo mismo).
- No funcional: sin falsos positivos: toda coincidencia de hash se verifica antes de informarse.
- No funcional: el procesamiento de una imagen de 4.000×4.000 tiene que tardar menos de un segundo.

Pista (leela solo si te trabás): primero reducís cada fila a una lista de hashes de ventanas horizontales; así la imagen se convierte en una grilla de números, y buscar el logo pasa a ser buscar una columna de números, que es un problema de una dimensión.

Bonus (opcional)

- Detectá el logo también rotado 90° , 180° y 270° (calculando el hash de cada rotación del logo).
- Tolerá hasta 2 píxeles distintos (el hash exacto ya no sirve: combiná con comparación parcial).

Cierre: El hash rodante en dos dimensiones (Rabin-Karp 2D) se usa en búsqueda de imágenes, detección de patrones y compresión.

Ejercicio 443: El mejor lugar para el hospital

Nivel 9: Senior III · Matemática aplicada · Temática: Salud · Dificultad 89/100

Situación: Una provincia va a construir un hospital regional sobre una ruta recta (el eje x) que atraviesa varios pueblos ubicados a distintas distancias de ella. Quiere ubicarlo donde se minimice la suma de distancias en línea recta ponderadas por la población. Esa función es convexa sobre la ruta, así que la búsqueda ternaria encuentra el mínimo sin derivadas.

Datos de entrada

Formato: un pueblo por línea: "nombre x y habitantes" (x = km sobre la ruta, y = distancia perpendicular a la ruta en km).

$\text{Costo}(x) = \text{suma de habitantes} \times \sqrt{(x - x_{\text{pueblo}})^2 + y_{\text{pueblo}}^2}$.

Ejemplo

```
Villa_Norte 2 8 1200
Villa_Sur 10 -6 800
Los_Pinos 15 4 1500
El_Molino 25 -3 600
```

Restricciones: hasta 10^5 pueblos.

Tu programa debe

1. Buscar en el intervalo entre la menor y la mayor x de los pueblos (el óptimo no puede estar afuera).
2. En cada paso, evaluar el costo en los dos puntos que dividen el intervalo en tercios y descartar el tercio del lado del punto más caro.
3. Parar cuando el intervalo mida menos de 10^{-9} km.
4. Mostrar la ubicación (4 decimales), el costo (2 decimales), las iteraciones y, para comparar, el costo en la mediana de las x .

Salida esperada

```
Ubicación óptima sobre la ruta: km 12.8076
Distancia ponderada total: 35810.91 habitante-km
Iteraciones de búsqueda ternaria: 59
Comparación: en el km 15 (mediana de las  $x$ ) sería 36829.59
```

Reglas

- Complejidad: $O(n \times \log(\text{rango} / \text{precisión}))$ (cada iteración achica el intervalo a $2/3$).
- La búsqueda ternaria solo sirve para funciones unimodales (un único mínimo); la suma de distancias euclídeas es convexa, así que lo es.
- No funcional: la decisión final considera también rutas reales y accesos; este cálculo es un insumo, no la decisión.
- No funcional: sensibilidad: el informe muestra cuánto empeora el costo si el hospital se corre 1 km (bonus).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): si en el punto a un tercio la función es menor que en el punto a dos tercios, el mínimo no puede estar en el último tercio (la función es convexa). Descartás un tercio por paso, sin derivadas.

Bonus (opcional)

- Mostrá el costo si el hospital se corre 1 km hacia cada lado.
- Sin la restricción de la ruta (cualquier punto del plano), resóvelo con el algoritmo de Weiszfeld.

Cierre: La búsqueda ternaria y los problemas de ubicación óptima se usan para decidir dónde poner hospitales, depósitos, antenas y escuelas.

Ejercicio 444: Vuelos con límite de escalas

Nivel 9: Senior III · Matemática aplicada · Temática: Turismo · Dificultad 89/100

Situación: Un buscador de vuelos baratos permite al usuario limitar la cantidad de escalas (nadie quiere 5 escalas para ahorrar poco). Dijkstra no respeta ese límite de forma directa. Bellman-Ford acotado hace exactamente $k + 1$ rondas de relajación (k escalas = $k + 1$ vuelos), y en cada ronda usa solo las distancias de la ronda anterior.

Datos de entrada

Formato: vuelos dirigidos "origen destino precio" (en miles de pesos); una línea "?" y consultas "origen destino máximo_de_escalas".

Ejemplo

```
Ushuaia BuenosAires 180
Ushuaia RíoGallegos 60
RíoGallegos Trelew 70
Trelew BuenosAires 60
RíoGallegos BuenosAires 140
Ushuaia Calafate 50
Calafate BuenosAires 110
Calafate Trelew 30
?
Ushuaia BuenosAires 0
Ushuaia BuenosAires 1
Ushuaia BuenosAires 2
Ushuaia BuenosAires 3
```

Restricciones: hasta 10^4 aeropuertos, 10^5 vuelos y hasta 10 escalas.

Tu programa debe

1. Inicializar el precio del origen en 0 y el resto en infinito.
2. Hacer $k + 1$ rondas: en cada una, calcular una copia nueva de los precios relajando todos los vuelos con los precios de la ronda anterior (así cada ronda agrega como máximo un vuelo).
3. Guardar el recorrido junto con cada precio.
4. Mostrar, para cada consulta, el precio mínimo y el recorrido, o informar que no hay vuelos.

Salida esperada

```
Ushuaia -> BuenosAires con hasta 0 escala(s): $180 mil por Ushuaia -> BuenosAires
Ushuaia -> BuenosAires con hasta 1 escala(s): $160 mil por Ushuaia -> Calafate -> BuenosAires
Ushuaia -> BuenosAires con hasta 2 escala(s): $140 mil por Ushuaia -> Calafate -> Trelew ->
BuenosAires
Ushuaia -> BuenosAires con hasta 3 escala(s): $140 mil por Ushuaia -> Calafate -> Trelew ->
BuenosAires
```

Reglas

- Complejidad: $O(k \times E)$ por consulta.
- Usar la copia de la ronda anterior es esencial: si se relaja sobre la misma tabla, una ronda puede encadenar varios vuelos y violar el límite.
- En empate de precio, se conserva el primer recorrido encontrado.
- No funcional: los precios cambian constantemente; la búsqueda se hace en cada consulta con los precios vigentes (sin tablas precalculadas).
- No funcional: el tiempo de respuesta al usuario tiene que ser menor a 300 ms.

Pista (leela solo si te trabás): después de la ronda r , la tabla tiene el mejor precio usando como máximo r vuelos. Si relajás sobre la misma tabla que estás modificando, te "colás" vuelos de más en una sola ronda.

Bonus (opcional)

- Resuélvelo con Dijkstra sobre el grafo de estados (aeropuerto, vuelos usados) y compará.
- Devolvé las 3 mejores opciones con distinta cantidad de escalas, para que el usuario elija.

Cierre: Las variantes acotadas de caminos mínimos se usan en buscadores de vuelos, ruteo con límite de saltos en redes y planificación con restricciones.

Ejercicio 445: Proyectos rentables que dependen de otros

Nivel 9: Senior III · Matemática aplicada · Temática: E-commerce · Dificultad 89/100

Situación: Una empresa de e-commerce evalúa proyectos que generan ganancia, pero cada uno requiere componentes técnicos que cuestan (una pasarela de pagos, un catálogo, notificaciones). Un componente se construye una vez y sirve a todos los proyectos que lo usan. Elegir el conjunto de proyectos de ganancia neta máxima es un problema de clausura de peso máximo, que se resuelve con un corte mínimo.

Datos de entrada

Formato: sección "proyectos": "proyecto ganancia componentes_requeridos..."; sección "componentes": "componente costo" (en millones).

Ejemplo

```
proyectos
Tienda_online 120 Pasarela Catálogo
App_fidelidad 60 Catálogo Push
Marketplace 150 Pasarela Catálogo Logística
Chatbot 25 Push IA
componentes
Pasarela 80
Catálogo 50
Push 20
Logística 110
IA 40
```

Restricciones: hasta 10^3 proyectos y 10^3 componentes.

Tu programa debe

1. Armar la red: fuente $\rightarrow$ proyecto (capacidad = ganancia), proyecto $\rightarrow$ componente requerido (capacidad infinita), componente $\rightarrow$ sumidero (capacidad = costo).
2. Calcular el flujo máximo = corte mínimo.
3. La ganancia neta máxima es la suma de las ganancias menos el corte mínimo.
4. Los proyectos y componentes del lado de la fuente en el corte (alcanzables en el residual) son los elegidos. Mostrar todo.

Salida esperada

```
Suma de ganancias posibles: 355 | corte mínimo: 285
Ganancia neta máxima: 70
Proyectos a hacer: Tienda_online, App_fidelidad, Marketplace
Componentes a construir: Pasarela, Catálogo, Push, Logística
```

Reglas

- Complejidad: $O(V \times E^2)$ con Edmonds-Karp.
- Por qué funciona: cortar la arista de un proyecto significa "renunciar a su ganancia"; cortar la de un componente, "pagar su costo". Las aristas infinitas impiden hacer un proyecto sin sus componentes.
- No funcional: el resultado se presenta con la lista de proyectos descartados y el motivo (sus componentes cuestan más de lo que aportan).
- No funcional: las estimaciones son inciertas: el análisis se repite con escenarios pesimista y optimista.

Pista (leela solo si te trabás): pensá cada corte como una decisión: o perdés la ganancia de un proyecto, o pagás el costo de sus componentes. El corte mínimo es la combinación de renuncias y pagos más barata.

Bonus (opcional)

- Agregá dependencias entre proyectos (el Marketplace requiere la Tienda online): también son aristas infinitas.
- Mostrá cuánto tendría que subir la ganancia del Chatbot para que convenga hacerlo.

Cierre: La clausura de peso máximo modela selección de proyectos, minería a cielo abierto y planificación de inversiones con dependencias.

Ejercicio 446: Pacientes y donantes compatibles

Nivel 9: Senior III · Matemática aplicada · Temática: Salud · Dificultad 89/100

Situación: Un banco de sangre ficticio de una simulación académica asigna unidades de donantes a pacientes según la compatibilidad de grupo sanguíneo (O da a todos, A da a A y AB, B da a B y AB, AB solo a AB). Los pacientes están ordenados por prioridad médica. Se busca atender a la mayor cantidad posible, y entre esas soluciones, respetar la prioridad.

Datos de entrada

Formato: sección "donantes": "id grupo"; sección "pacientes": "id grupo" en orden de prioridad (el primero es el más urgente).

Ejemplo

```
donantes
D1 O
D2 A
D3 B
D4 A
D5 AB
pacientes
P1 AB
P2 O
P3 A
P4 B
P5 A
P6 O
```

Restricciones: hasta 10^4 donantes y pacientes.

Tu programa debe

1. Armar la lista de donantes compatibles de cada paciente según la tabla de compatibilidad.
2. Procesar los pacientes en orden de prioridad con el algoritmo de Kuhn: cada uno busca un camino de aumento (puede reubicar a otros, pero un paciente ya atendido nunca queda sin atender).
3. El resultado es un emparejamiento máximo que además atiende a los pacientes más prioritarios posibles.
4. Mostrar el donante asignado a cada paciente (o "en espera") y el total.

Salida esperada

```
P1 (AB): donante D5
P2 (O): donante D1
P3 (A): donante D4
P4 (B): donante D3
P5 (A): donante D2
P6 (O): en espera
Pacientes atendidos: 5 de 6
```

Reglas

- Complejidad: $O(V \times E)$.
- Propiedad clave: en Kuhn, un vértice que ya está emparejado sigue emparejado después de un aumento (puede cambiar de pareja, pero no queda libre). Por eso el orden de procesamiento define la prioridad.
- No funcional: ética y trazabilidad: el orden de prioridad lo define el equipo médico, no el algoritmo; cada asignación se registra con su justificación.
- No funcional: los datos del ejercicio son ficticios y la compatibilidad está simplificada (sin factor Rh).

Pista (leela solo si te trabás): los donantes O son los más versátiles: conviene no "gastarlos" en pacientes que podrían recibir de otro grupo. Los caminos de aumento hacen esa reubicación automáticamente.

Bonus (opcional)

- Agregá el factor Rh (negativo da a positivo y negativo; positivo solo a positivo).
- Si cada paciente necesita 2 unidades, modelalo como flujo máximo.

Cierre: El emparejamiento con prioridades se usa en asignación de recursos escasos: turnos, vacantes, órganos y becas.

Ejercicio 447: Pronóstico de demanda eléctrica

Nivel 9: Senior III · Matemática aplicada · Temática: Clima y energía · Dificultad 89/100

Situación: Una distribuidora eléctrica pronostica la demanda del día siguiente según la tendencia (el número de día) y la temperatura máxima pronosticada (el aire acondicionado dispara el consumo). Ajusta un modelo lineal por mínimos cuadrados, resolviendo las ecuaciones normales: un sistema chico que se resuelve con eliminación gaussiana.

Datos de entrada

Formato: una observación por línea: "día temperatura demanda_MW"; una línea "?" y "día temperatura" a pronosticar.

Ejemplo

```
1 18 520
2 22 580
3 30 710
4 33 760
5 26 650
6 19 560
7 35 820
8 31 750
?
9 29
```

Restricciones: hasta 10^6 observaciones y hasta 50 variables explicativas.

Tu programa debe

1. Armar la matriz X con una columna de unos, el día y la temperatura, y el vector y con la demanda.
2. Calcular $X^T X$ (3×3) y $X^T y$ acumulando sobre las observaciones.
3. Resolver $(X^T X) \times \text{coeficientes} = X^T y$ con eliminación gaussiana y fracciones exactas.
4. Calcular el $R^2 = 1 - (\text{suma de errores}^2) / (\text{suma de desvíos}^2 \text{ respecto de la media})$, y mostrar el modelo (4 decimales), el R^2 y el pronóstico (1 decimal).

Salida esperada

```
Modelo: demanda = 231.3260 + 5.4761 × día + 15.4311 × temperatura
R²: 0.9968
Pronóstico para el día 9 con 29 °C: 728.1 MW
```

Reglas

- Complejidad: $O(n \times k^2 + k^3)$ con k variables.
- Las ecuaciones normales pueden estar mal condicionadas si las variables están muy correlacionadas; con decimales binarios, conviene la factorización QR (bonus).
- No funcional: trazabilidad: cada pronóstico se guarda con la versión del modelo y los datos con que se ajustó.
- No funcional: el pronóstico se publica con un margen de error, nunca como un número exacto.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): minimizar la suma de errores al cuadrado lleva a un sistema lineal: la derivada respecto de cada coeficiente igualada a cero da una ecuación. Con 3 coeficientes, son 3 ecuaciones.

Bonus (opcional)

- Agregá la temperatura al cuadrado como variable (el consumo sube más rápido con calor extremo) y compará el R^2 .
- Resóvelo con la factorización QR y compará la estabilidad con datos casi colineales.

Cierre: Los mínimos cuadrados son la base de la regresión lineal, del pronóstico de demanda y de gran parte del aprendizaje automático.

Ejercicio 448: Desafío de nivel: La mesa de dinero

Nivel 9: Senior III · Matemática aplicada · Temática: Finanzas y FinTech · Dificultad 90/100

Situación: Desafío de nivel. La mesa de dinero de un banco corre cada mañana un tablero de control con tres análisis: detecta arbitrajes en las cotizaciones cargadas (que suelen indicar errores de carga), calcula el rendimiento implícito del bono que más se opera y estresa la cartera en monedas contra escenarios de variación de tipo de cambio, con una multiplicación de matriz por vector.

Datos de entrada

Formato: tres bloques separados por "?": (1) cotizaciones "desde hasta tasa"; (2) bono "nominal cupón_% años precio"; (3) monedas de la cartera, posiciones (en cada moneda), y una fila por escenario con el factor de cada tipo de cambio contra el peso ($1,05 =$ el dólar sube 5 %).

Valuación base en pesos: USD 1.015, EUR 1.100, ARS 1. Resultado de un escenario = suma de posición $\times$ valor base $\times$ (factor – 1).

Ejemplo

```
ARS USD 0.00098
USD ARS 1015
USD EUR 0.92
EUR USD 1.08
EUR ARS 1100
ARS EUR 0.00090
?
1000 6 4 1030
?
USD EUR ARS
5000 -2000 1000000
1.05 1.02 1.00
0.95 1.00 1.00
1.00 0.97 1.00
```

Restricciones: hasta 200 monedas, 10^3 escenarios y 10^4 posiciones.

Tu programa debe

1. Arbitraje: Bellman-Ford sobre $-\log(\text{tasa})$ desde todas las monedas; si hay ciclo negativo, reconstruirlo y verificar el producto con decimales.
2. Rendimiento del bono: Newton-Raphson sobre $\text{precio}(y) - \text{precio observado}$, desde la tasa del cupón.
3. Estrés: multiplicar la matriz de escenarios (menos 1) por el vector de posiciones valuadas.
4. Mostrar los tres resultados y el peor escenario.

Salida esperada

```
1) Arbitraje: no hay
2) Rendimiento del bono a precio 1030: 5.1510 %
3) Cartera valuada en ARS: 3875000.00
   escenario 1: resultado 209750.00
   escenario 2: resultado -253750.00
   escenario 3: resultado 66000.00
   peor escenario: 2 (-253750.00)
```

Reglas

- Complejidad: $O(V \times E) + O(\text{iteraciones} \times n) + O(\text{escenarios} \times \text{monedas})$.
- Cada módulo se reutiliza de ejercicios anteriores; el desafío es integrarlos con entradas y salidas bien definidas.
- No funcional: el tablero corre antes de la apertura: tiene que terminar en segundos.
- No funcional: cada módulo falla de forma aislada (si el bono no converge, los otros dos análisis se muestran igual).

- No funcional: exactitud en el estrés: fracciones exactas y redondeo a 2 decimales al mostrar.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): separa los tres análisis en funciones independientes con entradas claras. El estrés es simplemente: para cada escenario, sumar lo que gana o pierde cada posición.

Bonus (opcional)

- Agregá el bono a la cartera y estresalo también ante subas de tasa (con su duración).
- Calculá el valor en riesgo histórico con 250 escenarios reales de variación diaria.

Cierre: Los tableros de riesgo de las mesas de dinero integran exactamente estas piezas: validación de precios, métricas de renta fija y pruebas de estrés.

Ejercicio 449: Desafío de nivel: Plataforma de logística urbana

Nivel 9: Senior III · Matemática aplicada · Temática: Logística y envíos · Dificultad 90/100

Situación: Desafío de nivel. Una plataforma de logística urbana recibe los pedidos del día con dirección escrita a mano y coordenadas. Primero tiene que limpiar direcciones duplicadas (el mismo pedido cargado dos veces con distinta escritura), después asignar cada pedido a su zona de reparto según las coordenadas, y por último decidir cuántos pedidos se pueden despachar con la capacidad de cada depósito.

Datos de entrada

Formato: secciones "zonas" ("zona x1 y1 x2 y2 ...", polígonos), "pedidos" ("dirección | x y") y "depósitos" ("depósito capacidad zonas_que_atiende...").

Normalización de direcciones: minúsculas, puntos por espacios, sin tildes ni diéresis, "av" → "avenida", sin la palabra "calle" y espacios simples.

Ejemplo

```
zonas
Centro 0 0 40 0 40 30 0 30
Norte 0 30 40 30 40 60 0 60
Este 40 0 80 0 80 60 40 60
pedidos
Av. Corrientes 1234 | 10 10
AV CORRIENTES 1234 | 10 10
Calle Florida 50 | 35 25
Av. Santa Fe 3000 | 20 45
Güemes 4500 | 45 50
Honduras 5700 | 60 20
Av. Cabildo 2000 | 90 90
depósitos
Dep_Sur 3 Centro Este
Dep_Norte 2 Norte Este
```

Restricciones: hasta 10^6 pedidos por día, 10^3 zonas y 10^2 depósitos.

Tu programa debe

1. Normalizar cada dirección y descartar las repetidas (texto normalizado ya visto).
2. Asignar cada pedido a la primera zona que lo contiene (punto en polígono); si solo está en el borde, a la primera zona en cuyo borde está; si no, fuera de cobertura.
3. Armar la red: fuente → depósito (capacidad), depósito → zona que atiende (ilimitada), zona → sumidero (pedidos de la zona), y calcular el flujo máximo.
4. Mostrar el tratamiento de cada pedido, los pedidos por zona y cuántos se despachan.

Salida esperada

```
"Av. Corrientes 1234" -> Centro
"AV CORRIENTES 1234": DUPLICADO de "Av. Corrientes 1234"
"Calle Florida 50" -> Centro
"Av. Santa Fe 3000" -> Norte
"Güemes 4500" -> Este
"Honduras 5700" -> Este
"Av. Cabildo 2000": fuera de cobertura
Pedidos por zona: Centro 2, Norte 1, Este 2
Pedidos que se pueden despachar hoy: 5 de 5
```

Reglas

- Complejidad: $O(\text{pedidos} \times \text{vértices})$ para las zonas (con índice espacial, mucho menos) y $O(V \times E^2)$ para el flujo.
- Integra los tres temas del nivel: strings (normalización), geometría (punto en polígono) y flujo.

- No funcional: calidad de datos: un pedido duplicado no despachado es mejor que uno despachado dos veces; ante la duda, se marca para revisión.
- No funcional: el proceso completo corre antes de las 7:00 y su resultado es reproducible con los mismos datos.
- No funcional: los pedidos fuera de cobertura se informan al cliente, no se descartan en silencio.

Pista (leela solo si te trabás): tres etapas encadenadas, cada una con una herramienta que ya construiste. El cuidado está en las uniones: qué pasa con un pedido en el borde entre dos zonas o con una dirección que se normaliza igual que otra.

Bonus (opcional)

- Usá distancia de edición (nivel 6) para detectar duplicados con errores de tipeo ("Corrientes").
- Agregá costo por depósito y zona, y minimizá el costo total del despacho.

Cierre: Las plataformas de logística combinan limpieza de datos, geometría y optimización en cada corrida diaria de planificación.

Ejercicio 450: Desafío de nivel: Detector de fraude documental

Nivel 9: Senior III · Matemática aplicada · Temática: Seguridad informática · Dificultad 90/100

Situación: Desafío de nivel. El área de seguridad de un banco recibe documentos y mails reportados por clientes y los compara con plantillas de fraude conocidas. Para cada par calcula la similitud por fragmentos (hash rodante), y para los sospechosos extrae el pasaje común más largo (arreglo de sufijos) como evidencia. Como cada plantilla la investiga un analista distinto que toma un caso por día, se asignan los documentos a plantillas con un emparejamiento máximo.

Datos de entrada

Formato: primera línea: k (palabras por fragmento) y umbral de similitud (%); después "plantilla: texto"; una línea "?" y "documento: texto".

Hash de fragmentos igual que en el ejercicio del plagio (base 257, módulo 1.000.000.007, espacio después de cada palabra).

Ejemplo

```
4 30
plantilla_1: pago urgente a proveedor por transferencia a la cuenta indicada en el adjunto
plantilla_2: su cuenta sera suspendida si no confirma sus datos dentro de las 24 horas
?
doc_A: estimado cliente su cuenta sera suspendida si no confirma sus datos dentro de las 48 horas
doc_B: se solicita pago urgente a proveedor por transferencia a la cuenta indicada en el mail
doc_C: recordatorio de la reunion de directorio del jueves a las 10 horas
doc_D: aviso su cuenta sera suspendida si no confirma sus datos hoy mismo
```

Restricciones: hasta 10^3 plantillas y 10^5 documentos por día.

Tu programa debe

1. Calcular los fragmentos de k palabras de cada texto y la similitud de Jaccard de cada documento con cada plantilla.
2. Para los pares que superan el umbral, construir el arreglo de sufijos de plantilla + separador + documento y encontrar la subcadena común más larga con el LCP.
3. Armar el grafo bipartito documento → plantillas similares y calcular un emparejamiento máximo (Kuhn, en el orden de la entrada).
4. Mostrar los pares sospechosos con su evidencia, la asignación, los sospechosos sin asignar y los documentos limpios.

Salida esperada

```
doc_A ~ plantilla_2: similitud 60.0 % | pasaje común más largo (65): "su cuenta sera suspendida si
no confirma sus datos dentro de las"
doc_B ~ plantilla_1: similitud 69.2 % | pasaje común más largo (70): "pago urgente a proveedor por
transferencia a la cuenta indicada en el"
doc_D ~ plantilla_2: similitud 42.9 % | pasaje común más largo (51): "su cuenta sera suspendida si
no confirma sus datos"
Asignación de casos (cada plantilla a un analista, un documento por plantilla): plantilla_1 ->
doc_B, plantilla_2 -> doc_A
Documentos sospechosos sin asignar: doc_D | limpios: doc_C
```

Caso 2 (borde)

Un documento sin relación con ninguna plantilla:

```
4 30
plantilla_1: pago urgente a proveedor por transferencia a la cuenta indicada en el adjunto
?
doc_X: hola como estas
```

Salida esperada del caso 2:

Asignación de casos (cada plantilla a un analista, un documento por plantilla): ninguna
Documentos sospechosos sin asignar: ninguno | limpios: doc_X

Reglas

- Complejidad: $O(\text{documentos} \times \text{plantillas} \times \text{largo})$ para las similitudes; el arreglo de sufijos solo se construye para los pares sospechosos.
- Integra los tres temas del nivel en strings y grafos: hashing, arreglos de sufijos y emparejamiento.
- No funcional: evidencia: toda alerta incluye el pasaje concreto que la justifica, para que el analista no dependa solo de un porcentaje.
- No funcional: privacidad: los documentos de clientes se procesan en el entorno del banco y se eliminan según la política de retención.
- No funcional: los documentos sospechosos que no se asignan hoy quedan en cola para el día siguiente (no se descartan).

Pista (leela solo si te trabás): usá la similitud por fragmentos como filtro barato y el arreglo de sufijos como lupa cara solo donde hace falta. El emparejamiento es la última etapa, sobre un grafo ya chico.

Bonus (opcional)

- Usá MinHash con LSH para no comparar cada documento contra todas las plantillas.
- Agregá prioridad: documentos con más similitud se asignan primero (procesalos ordenados en Kuhn).

Cierre: Los sistemas antifraude documental combinan exactamente estas técnicas para filtrar, explicar y priorizar alertas.

Lote 19: ejercicios 451 a 475

Nivel 10

Staff / Experto

Ejercicios 451 a 500

Qué vas a practicar

Nivel experto: estructuras probabilísticas, compresión, criptografía didáctica, sistemas distribuidos, FFT, optimización y finanzas cuantitativas.

Qué se espera al terminar

Combinar algoritmos, matemática y diseño de sistemas a escala.

Conceptos del nivel

Bloom filter, HyperLogLog, Count-Min sketch, codificación de Huffman, RSA, exponenciación modular, relojes vectoriales, CRDT de contadores, árbol de Merkle, quórums de lectura y escritura, LZ77, FFT, multiplicación de polinomios, programación lineal, simplex, VaR, simulación de Monte Carlo, recocido simulado, Black-Scholes, branch & bound, consenso simplificado, cadenas de hashes, portafolio de Markowitz, optimización, CRDT de secuencias, MinHash, LSH, análisis de frecuencias, resolución de conflictos, árboles de Merkle, anti-entropía, heavy hitters, probabilidad, Huffman, dificultad ajustable, HyperLogLog con fusión, árbol binomial, valuación de opciones, quórums, particiones de red....

Desafíos de nivel

- 498. Ledger distribuido que sobrevive a cortes de red
- 499. Antifraude a escala planetaria
- 500. La mesa de riesgo del banco

Ejercicio 451: Mil millones de sitios peligrosos en poca memoria

Nivel 10: Staff / Experto · Temática: Seguridad informática · Dificultad 91/100

Situación: Un navegador quiere avisar si un sitio está en la lista de mil millones de sitios peligrosos, pero no puede guardar esa lista en cada celular. Un filtro de Bloom la resume en un arreglo de bits: puede decir "seguro que no está" o "probablemente está" (y solo en ese caso consulta al servidor). Nunca da falsos negativos, y los falsos positivos se controlan con el tamaño.

Datos de entrada

Formato: primera línea: bits del filtro m y cantidad de funciones k ; segunda: sitios peligrosos; una línea "?" y los sitios a consultar.

Posiciones de un texto s : $h1 = H(s)$, $h2 = H(s + "\#") \text{ OR } 1$, posición $i = (h1 + i \times h2) \text{ resto } m$ para i de 0 a $k - 1$ (doble hashing). $H(\text{texto}) = \text{FNV-1a de 32 bits sobre los caracteres (todos ASCII)}$: $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
64 3
malware1.test phishing1.test estafa1.test troyano1.test spam1.test malware2.test phishing2.test
  estafa2.test troyano2.test spam2.test
?
phishing1.test estafa2.test spam2.test banco.test diario.test clima.test tienda.test juegos.test
radio.test correo.test escuela.test mapas.test
```

Restricciones: hasta 10^9 sitios; el filtro real usa unos 10 bits por sitio.

Tu programa debe

1. Insertar cada sitio peligroso encendiendo sus k bits.
2. Consultar cada sitio: si alguno de sus k bits está en 0, seguro no está; si todos están en 1, "posible peligro".
3. Contar los falsos positivos (se conocen porque el ejercicio tiene la lista real).
4. Mostrar los bits en 1, cada resultado, y comparar con la tasa teórica $(1 - e^{(-kn/m)})^k$ y el k óptimo $(m / n) \times \ln 2$.

Salida esperada

```
Bits en 1: 27 de 64
phishing1.test: POSIBLE PELIGRO
estafa2.test: POSIBLE PELIGRO
spam2.test: POSIBLE PELIGRO
banco.test: seguro
diario.test: seguro
clima.test: seguro
tienda.test: seguro
juegos.test: seguro
radio.test: POSIBLE PELIGRO (falso positivo)
correo.test: seguro
escuela.test: seguro
mapas.test: seguro
Falsos positivos: 1 de 9 sitios limpios | teórico 5.24 % | k óptimo 4.44
```

Reglas

- Complejidad: $O(k)$ por inserción y consulta, sin importar cuántos sitios haya.
- Memoria fija: m bits, sin guardar ningún sitio.
- No funcional: garantía: el filtro jamás dice "seguro" para un sitio peligroso (sin falsos negativos); los falsos positivos se verifican con el servidor.

- No funcional: los parámetros se eligen a partir de n y de la tasa de falsos positivos aceptable, no al azar.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): un bit en 1 no dice quién lo encendió. Por eso un sitio limpio puede tener sus k bits encendidos por otros sitios: más bits por elemento, menos chance de que pase.

Bonus (opcional)

- Dimensioná m y k para 10^9 sitios con 1 % de falsos positivos y calculá cuántos megabytes ocupa.
- Agregá contadores en lugar de bits para poder borrar sitios (ejercicio de los productos dados de baja).

Cierre: Los filtros de Bloom están en navegadores, bases de datos, CDNs y sistemas de almacenamiento para evitar búsquedas costosas.

Ejercicio 452: ¿Cuánta gente distinta entró hoy a la tienda?

Nivel 10: Staff / Experto · Temática: E-commerce · Dificultad 91/100

Situación: Un e-commerce quiere saber cuántos clientes distintos entraron hoy, en tiempo real, sobre cientos de millones de eventos. Guardar un conjunto con todos los ids ocupa gigas. HyperLogLog estima la cantidad de distintos con unos pocos kilobytes, mirando cuántos ceros iniciales tienen los hashes: ver un hash con muchos ceros adelante es raro y sugiere que hubo muchos elementos.

Datos de entrada

Formato: una línea: p (se usan 2^p registros), cantidad de eventos, cantidad de clientes posibles y semilla.

Eventos: $x = \text{siguiente}(x)$; cliente = "cliente" + (x resto clientes_posibles), con $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Para cada elemento: $h = H(\text{elemento})$; registro = los p bits más altos de h ; $w = \text{los } 32 - p \text{ bits restantes (corridos a la izquierda)}$; $p = \text{posición del primer 1 en } w$ (1 si empieza con 1), como máximo $32 - p + 1$; el registro guarda el máximo p visto. Estimación: $E = \alpha \times m^2 / \text{suma de } 2^{(-\text{registro})}$, con $\alpha = 0,709$ para $m = 64$; si $E \leq 2,5m$ y hay registros en 0 (V), $E = m \times \ln(m / V)$.

Ejemplo

```
6 6000 2500 7
```

Restricciones: hasta 10^{10} eventos por día; p entre 4 y 16.

Tu programa debe

1. Generar los eventos con el generador (hay clientes repetidos).
2. Para cada evento, calcular el hash, elegir el registro con los p bits altos y actualizar el máximo p .
3. Calcular la estimación con la media armónica y aplicar la corrección para rangos chicos.
4. Mostrar los primeros registros, la estimación (redondeada), el valor real y el error relativo, junto con el error típico esperado $1,04 / \sqrt{m}$.

Salida esperada

```
Eventos: 6000 | registros: 64 (memoria: 64 valores de 5 bits)
Primeros 16 registros: 8 8 9 5 8 6 7 5 5 6 11 6 4 10 5 7
Estimación: 2249 | real: 2255 | error 0.25 % (error típico esperado 13.00 %)
```

Reglas

- Complejidad: $O(1)$ por evento y memoria $O(m)$.
- El valor real se calcula solo para validar el ejercicio (con un conjunto); en producción no se tiene.
- No funcional: memoria acotada e independiente del tráfico: con $p = 14$ son 16.384 registros (12 KB) para cualquier cantidad de eventos.
- No funcional: el tablero muestra la estimación con su margen de error, no como un número exacto.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): si tirás una moneda hasta que sale cara, sacar 10 cecas seguidas es raro: pasa una vez cada 1.024 intentos. Ver en algún hash 10 ceros adelante sugiere que hubo del orden de mil elementos distintos. Repartir en m registros y promediar reduce la varianza.

Bonus (opcional)

- Fusioná dos HyperLogLog (el de hoy y el de ayer) tomando el máximo registro a registro (ejercicio de los turistas únicos).
- Grificá el error relativo para p de 4 a 14 con 20 semillas distintas.

Cierre: HyperLogLog se usa en bases de datos, sistemas de analítica y redes para contar usuarios únicos a escala planetaria.

Ejercicio 453: Las canciones más escuchadas con memoria mínima

Nivel 10: Staff / Experto · Temática: Música · Dificultad 91/100

Situación: Una plataforma de música cuenta reproducciones de millones de canciones en tiempo real en cada servidor de borde, con muy poca memoria. El Count-Min sketch es una tabla de d filas y w columnas de contadores: cada canción suma en una columna por fila (elegida con un hash distinto) y su estimación es el mínimo de sus d contadores. Nunca subestima; puede sobrestimar por colisiones.

Datos de entrada

Formato: primera línea: ancho w y profundidad d ; después "canción reproducciones" (se procesan como si llegaran de a una); una línea "?" y las canciones a consultar.

Columna de la canción x en la fila i : $H(i + ":" + x)$ resto w (con i en decimal). $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619)$ resto 2^{32} . Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507)$ resto 2^{32} ; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909)$ resto 2^{32} ; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
4 3
Zamba_de_mi_esperanza 120
Cancion_con_todos 95
La_bifurcada 60
Muchachos 150
Oncemil 8
Rapsodia_andina 3
Luna_tucumana 40
Mil_horas 70
?
Muchachos Cancion_con_todos Oncemil Rapsodia_andina Tema_inexistente
```

Restricciones: hasta 10^9 reproducciones por hora y 10^8 canciones.

Tu programa debe

1. Crear la tabla de $d \times w$ contadores en 0.
2. Por cada reproducción, sumar 1 en la columna que le toca a la canción en cada fila (en el ejemplo se suma la cantidad de una vez).
3. Estimar una canción como el mínimo de sus d contadores.
4. Mostrar estimado, real y exceso para cada consulta, y la cota teórica de error $e \times N / w$.

Salida esperada

```
Muchachos: estimado 218 | real 150 | exceso 68
Cancion_con_todos: estimado 215 | real 95 | exceso 120
Oncemil: estimado 118 | real 8 | exceso 110
Rapsodia_andina: estimado 3 | real 3 | exceso 0
Tema_inexistente: estimado 0 | real 0 | exceso 0
Total de reproducciones  $N = 546$  | cota de error  $e \times N / w = 371.0$  (con probabilidad  $1 - e^{(-3)} \approx 95.0\%$ )
```

Reglas

- Complejidad: $O(d)$ por actualización y consulta; memoria $O(d \times w)$.
- Garantía: estimado $\geq$ real siempre, y estimado $\leq$ real + $e \times N / w$ con probabilidad $1 - e^{(-d)}$.
- No funcional: memoria fija: la tabla no crece con la cantidad de canciones.
- No funcional: los contadores de distintos servidores se combinan sumando las tablas celda a celda (mismas funciones de hash).

- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): en cada fila, una canción comparte su contador con otras que cayeron en la misma columna, así que el contador solo puede "pasarse". Con varias filas independientes, alguna probablemente tenga pocas colisiones: por eso el mínimo.

Bonus (opcional)

- Mantené el top 10 de canciones con un montículo alimentado por las estimaciones ("heavy hitters").
- Usá "conservative update": al sumar, solo incrementá los contadores que están en el mínimo, y medí la mejora.

Cierre: El Count-Min sketch se usa en redes, publicidad online y monitoreo para contar frecuencias en flujos gigantes.

Ejercicio 454: Comprimir los mensajes de la flota

Nivel 10: Staff / Experto · Temática: Telecomunicaciones · Dificultad 92/100

Situación: Una flota de camiones manda mensajes de estado por un enlace satelital que se cobra por bit. Los mensajes usan pocos caracteres y algunos mucho más que otros. La codificación de Huffman asigna códigos cortos a los caracteres frecuentes y largos a los raros, sin que ningún código sea prefijo de otro, y es óptima entre los códigos de ese tipo.

Datos de entrada

Formato: una línea con el mensaje.

Desempate determinista: las hojas entran a la cola de prioridad ordenadas por carácter (código Unicode) y numeradas en ese orden; cada nodo interno nuevo recibe el número siguiente. Se extrae siempre el de menor (frecuencia, número); el primero extraído es el hijo izquierdo (bit 0).

Ejemplo

```
la casa de la cascada cae en cascada
```

Restricciones: mensajes de hasta 10^8 caracteres.

Tu programa debe

1. Contar la frecuencia de cada carácter.
2. Poner una hoja por carácter en una cola de prioridad; mientras haya más de un nodo, sacar los dos menores y unirlos en un nodo con la suma de frecuencias.
3. Recorrer el árbol: izquierda agrega 0 y derecha 1; el camino hasta cada hoja es su código.
4. Mostrar la tabla (de mayor a menor frecuencia; el espacio como `␣`), el comienzo del mensaje codificado y la comparación con un código de largo fijo.

Salida esperada

```
a: frecuencia 11, código 10
␣: frecuencia 7, código 01
c: frecuencia 6, código 110
d: frecuencia 3, código 1110
e: frecuencia 3, código 1111
s: frecuencia 3, código 000
l: frecuencia 2, código 0011
n: frecuencia 1, código 0010
Mensaje codificado (99 bits): 001110011101000010011110111101001110011101000011...
Bits con Huffman: 99 | con código fijo de 3 bits: 108 | ahorro 8.3 %
```

Reglas

- Complejidad: $O(n + \sigma \log \sigma)$ con σ caracteres distintos.
- Ningún código es prefijo de otro, así que el mensaje se decodifica sin separadores.
- No funcional: el receptor necesita la tabla de códigos (o las frecuencias) para decodificar: en mensajes cortos, su tamaño puede comerse el ahorro.
- No funcional: la reproducibilidad exige el desempate especificado; con otro desempate, los códigos cambian pero el total de bits no.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): los dos caracteres menos frecuentes pueden estar en lo más profundo del árbol, como hermanos, sin perder nada. Unirlos y repetir es la decisión voraz que resulta óptima.

Bonus (opcional)

- Decodificá el mensaje recorriendo el árbol bit a bit y verificá que es idéntico.

- Usá códigos canónicos de Huffman, que se transmiten enviando solo los largos de los códigos.

Cierre: Huffman es parte de formatos como ZIP, PNG, JPEG y MP3, casi siempre combinado con otras técnicas de compresión.

Ejercicio 455: Recibos que nadie puede falsificar

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 92/100

Situación: Una fintech quiere que sus recibos digitales no se puedan falsificar: cualquiera tiene que poder verificar que un recibo lo emitió la empresa y que nadie le cambió el monto. En este ejercicio didáctico se implementa una firma RSA de juguete con primos chicos, para entender la matemática (en producción se usan claves de 2.048 bits, bibliotecas auditadas y esquemas de relleno).

Datos de entrada

Formato: primera línea: los primos p y q y el exponente público e ; después, el recibo original y los recibos a verificar.

Hash del recibo: $H(\text{texto})$ resto n . $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
61 53 17
Recibo 0001: pago de 150000 pesos a Ferreteria El Tornillo
Recibo 0001: pago de 950000 pesos a Ferreteria El Tornillo
```

Restricciones: solo didáctico: con primos chicos, la clave se rompe en milisegundos.

Tu programa debe

1. Calcular $n = p \times q$ y $\phi(n) = (p - 1)(q - 1)$, y la clave privada $d = \text{inverso de } e \text{ módulo } \phi(n)$ (Euclides extendido).
2. Firmar: $\text{firma} = \text{hash}^d \text{ resto } n$ (exponenciación rápida).
3. Verificar cada recibo: calcular $\text{firma}^e \text{ resto } n$ y compararlo con el hash del recibo presentado.
4. Mostrar las claves, la firma y el resultado de cada verificación.

Salida esperada

```
n = 3233 | φ(n) = 3120 | e = 17 | d = 2753 (verificación e × d resto φ = 1)
Recibo original: hash resto n = 262 | firma = 3019
Verificación de "Recibo 0001: pago de 150000 pesos a Ferr...": firma^e resto n = 262, hash = 262
-> VÁLIDO
Verificación de "Recibo 0001: pago de 950000 pesos a Ferr...": firma^e resto n = 262, hash = 2744
-> FALSIFICADO
```

Reglas

- Complejidad: $O(\log d)$ multiplicaciones modulares por firma.
- Por qué funciona: $(h^d)^e = h^{de} \equiv h \pmod{n}$, porque $de \equiv 1 \pmod{\phi(n)}$ (teorema de Euler).
- No funcional: seguridad: nunca implementes criptografía propia para producción; este ejercicio es para entender el mecanismo.
- No funcional: la clave privada d nunca sale del sistema que firma; la verificación solo usa (n, e) .
- Con un n tan chico, dos recibos distintos pueden tener el mismo hash resto n ; con claves reales es inviable encontrarlos.

Pista (leela solo si te trabás): firmar es "elevar a la d " y verificar es "elevar a la e "; como uno deshace al otro, solo quien conoce d puede producir algo que, elevado a la e , dé el hash.

Bonus (opcional)

- Rompé la clave de juguete factorizando n y mostrá por qué hacen falta primos enormes.
- Cifrá (en lugar de firmar) un número con la clave pública y descifralo con la privada.

Cierre: RSA fue uno de los primeros sistemas de clave pública y sigue presente en certificados, firmas digitales y protocolos seguros.

Ejercicio 456: ¿Quién escribió primero en el grupo?

Nivel 10: Staff / Experto · Temática: Redes sociales · Dificultad 92/100

Situación: En un grupo de chat distribuido, cada dispositivo tiene su propio reloj y los relojes no coinciden, así que la hora no sirve para saber qué mensaje se escribió "antes". Los relojes vectoriales capturan la causalidad: un mensaje ocurrió antes que otro si quien escribió el segundo ya había visto el primero. Si ninguno vio al otro, son concurrentes.

Datos de entrada

Formato: primera línea: los participantes; después eventos "p envía mensaje", "p recibe mensaje" o "p local"; una línea "?" y pares de mensajes a comparar.

Ejemplo

```
A B C
A envía m1
B recibe m1
B envía m2
C envía m3
A recibe m2
C recibe m2
A local
A recibe m3
?
m1 m2
m2 m3
m1 m3
```

Restricciones: hasta 10^3 participantes por grupo y 10^9 mensajes.

Tu programa debe

1. Cada participante guarda un vector con un contador por participante, en 0.
2. En cada evento, el participante incrementa su propia posición; al enviar, adjunta una copia de su vector al mensaje.
3. Al recibir, primero toma el máximo posición por posición entre su vector y el del mensaje, y después incrementa su posición.
4. Mostrar el vector después de cada evento y comparar los pares: $V(x) < V(y)$ (todas las posiciones $\leq$ y alguna $<$) significa "x antes que y"; si ninguno es menor, son concurrentes.

Salida esperada

```
A envía m1: A = [1, 0, 0]
B recibe m1: B = [1, 1, 0]
B envía m2: B = [1, 2, 0]
C envía m3: C = [0, 0, 1]
A recibe m2: A = [2, 2, 0]
C recibe m2: C = [1, 2, 2]
A local: A = [3, 2, 0]
A recibe m3: A = [4, 2, 1]
m1 ocurrió antes que m2
m2 y m3 son concurrentes
m1 y m3 son concurrentes
```

Reglas

- Complejidad: $O(\text{participantes})$ por evento y por comparación.
- No funcional: tamaño: el vector crece con la cantidad de participantes; para grupos enormes se usan variantes compactas.
- No funcional: la interfaz muestra los mensajes concurrentes en un orden estable (por ejemplo, por id), aclarando que no hay un "antes" real.
- Un evento local (sin mensajes) también incrementa el contador.

Pista (leela solo si te trabás): la posición i del vector de un evento cuenta cuántos eventos de i "conocía" quien lo produjo. Si conocía todo lo que conocía el otro y algo más, lo vio antes; si cada uno conoce algo que el otro no, no se vieron.

Bonus (opcional)

- Ordená los mensajes para mostrarlos de forma que ningún mensaje aparezca antes de uno que lo causó (orden topológico por vectores).
- Compará con relojes de Lamport: mostrá un par que Lamport ordena aunque sea concurrente.

Cierre: Los relojes vectoriales se usan en bases de datos distribuidas, sistemas de replicación y edición colaborativa para detectar conflictos.

Ejercicio 457: Votos del jugador del partido en varios servidores

Nivel 10: Staff / Experto · Temática: Deportes · Dificultad 92/100

Situación: Una app deportiva recibe votos para el jugador del partido en servidores de distintos países, que no siempre pueden hablarse. Cada servidor tiene que aceptar votos (y arrepentimientos) localmente y, cuando se sincronizan, todos tienen que llegar al mismo total sin perder ni duplicar votos, sin importar el orden ni cuántas veces se sincronicen. Un contador CRDT lo garantiza.

Datos de entrada

Formato: primera línea: los servidores; después operaciones "servidor +k" (votos), "servidor -k" (votos retirados) o "sync s1 s2" (intercambian estado).

Ejemplo

```
S1 S2 S3
S1 +3
S2 +5
S3 +2
S1 -1
sync S1 S2
S2 +1
sync S2 S3
S3 -2
sync S3 S1
sync S1 S2
sync S2 S3
```

Restricciones: hasta 10^3 servidores y 10^9 operaciones.

Tu programa debe

1. Cada servidor guarda dos vectores: P (sumas) y N (restas), con una posición por servidor.
2. Una operación local suma solo en la posición propia de P o de N.
3. Sincronizar es tomar el máximo posición por posición en P y en N (en los dos servidores).
4. El valor es suma de P – suma de N. Mostrar el valor de cada servidor después de cada operación y si convergieron.

Salida esperada

```
S1 +3: S1=3 | S2=0 | S3=0
S2 +5: S1=3 | S2=5 | S3=0
S3 +2: S1=3 | S2=5 | S3=2
S1 -1: S1=2 | S2=5 | S3=2
sync S1 S2: S1=7 | S2=7 | S3=2
S2 +1: S1=7 | S2=8 | S3=2
sync S2 S3: S1=7 | S2=10 | S3=10
S3 -2: S1=7 | S2=10 | S3=8
sync S3 S1: S1=8 | S2=10 | S3=8
sync S1 S2: S1=8 | S2=8 | S3=8
sync S2 S3: S1=8 | S2=8 | S3=8
Convergiaron: sí (valor 8; suma real de operaciones 8)
```

Reglas

- Complejidad: $O(\text{servidores})$ por sincronización.
- El máximo es conmutativo, asociativo e idempotente: por eso sincronizar en cualquier orden, o repetir una sincronización, da lo mismo.
- No funcional: disponibilidad: cada servidor acepta votos aunque esté aislado; la consistencia llega cuando se sincronizan (consistencia eventual).
- No funcional: no hay coordinación ni bloqueos entre servidores.

- Un contador único que se suma al sincronizar contaría dos veces; uno que toma el máximo perdería votos. Por eso hay una posición por servidor.

Pista (leela solo si te trabás): cada servidor es el único que escribe su propia posición, y esa posición solo crece. Tomar el máximo nunca pierde información ni la duplica.

Bonus (opcional)

- Implementá un conjunto CRDT (con agregados y borrados) para la lista de candidatos.
- Evitá que el total quede negativo si se retiran más votos de los emitidos (y explicá por qué es difícil sin coordinación).

Cierre: Los CRDT permiten replicar datos sin coordinación en apps colaborativas, bases de datos distribuidas y sistemas offline-first.

Ejercicio 458: Probar que una transacción está en el lote

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 92/100

Situación: Un procesador de pagos agrupa las transacciones del día en lotes y publica un único hash que resume cada lote: la raíz de un árbol de Merkle. Un cliente puede comprobar que su transacción está en el lote pidiendo solo unos pocos hashes (la "prueba de inclusión"), sin descargar el lote entero ni ver las transacciones de los demás.

Datos de entrada

Formato: transacciones del lote (una por línea); una línea "?" con el número de la transacción a probar y una versión adulterada de ella.

Hoja = H("hoja:" + transacción); nodo = H("nodo:" + izquierdo + ":" + derecho) (hashes en decimal); si un nivel tiene cantidad impar, se duplica el último. H(texto) = FNV-1a de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$. (En producción se usa una función de hash criptográfica.)

Ejemplo

```
tx1 ana->beto 1000
tx2 caro->dani 2500
tx3 beto->eli 300
tx4 dani->ana 700
tx5 eli->fedo 50
tx6 fedo->caro 1200
?
4
tx4 dani->ana 7000
```

Restricciones: lotes de hasta 10^7 transacciones (la prueba tiene unos 24 hashes).

Tu programa debe

1. Calcular los hashes de las hojas y combinarlos de a pares, nivel por nivel, hasta la raíz.
2. Armar la prueba de inclusión de la transacción k : en cada nivel, el hash hermano y de qué lado está.
3. Verificar: partir del hash de la transacción y combinarlo con cada hermano de la prueba, respetando el lado; comparar con la raíz publicada.
4. Mostrar la raíz, la prueba, la verificación de la transacción real y de la adulterada.

Salida esperada

```
Raíz publicada: 2062351687 (niveles: 4)
Prueba para la transacción 4: 3928217129 (izq), 1130125356 (izq), 779048549 (der)
Verificación de "tx4 dani->ana 700": raíz calculada 2062351687 -> INCLUIDA
Verificación de "tx4 dani->ana 7000": raíz calculada 2965722109 -> NO coincide (adulterada)
Hashes en la prueba: 3 (en lugar de 6 transacciones)
```

Reglas

- Complejidad: $O(n)$ para construir y $O(\log n)$ para la prueba y su verificación.
- Los prefijos "hoja:" y "nodo:" evitan que un nodo interno pueda hacerse pasar por una hoja.
- No funcional: privacidad: la prueba no revela las otras transacciones, solo hashes.
- No funcional: la raíz se publica en un medio que el procesador no puede modificar después (por ejemplo, enviada a un tercero).

Pista (leela solo si te trabás): cada nivel reduce el problema a la mitad; para recalculer la raíz desde una hoja solo necesitás, en cada nivel, el hash de al lado. Cambiar un solo carácter de la transacción cambia todo el camino hasta la raíz.

Bonus (opcional)

- Probá que una transacción NO está en el lote, ordenando las hojas y mostrando las dos vecinas.
- Agregá transacciones al lote sin recalculer todo el árbol.

Cierre: Los árboles de Merkle están en sistemas de control de versiones, bases de datos distribuidas, certificados y cadenas de bloques.

Ejercicio 459: Historias clínicas en tres centros de datos

Nivel 10: Staff / Experto · Temática: Salud · Dificultad 92/100

Situación: Una red de clínicas replica cada historia clínica en tres centros de datos. Una escritura se confirma cuando la guardan W réplicas y una lectura consulta R réplicas y se queda con la versión más nueva. Si $W + R > N$, toda lectura se cruza con la última escritura confirmada y nunca devuelve datos viejos; si no, es más rápido y tolera más caídas, pero puede mostrar un dato desactualizado.

Datos de entrada

Formato: primera línea: las réplicas en orden de preferencia; después "escribir versión caídas réplicas_caídas" y "leer orden réplicas_en_el_orden_de_consulta caídas réplicas_caídas" ("- = ninguna).

Una escritura va a las primeras W réplicas vivas en el orden de preferencia (si hay menos de W vivas, se rechaza). Una lectura consulta las primeras R vivas en su orden y devuelve la versión máxima.

Ejemplo

```
R1 R2 R3
escribir 1 caídas -
escribir 2 caídas R1
leer orden R1 R2 R3 caídas -
escribir 3 caídas R2 R3
leer orden R3 R2 R1 caídas -
```

Restricciones: N hasta 7 réplicas; 10^5 operaciones por segundo.

Tu programa debe

1. Simular la secuencia para cada combinación (W , R) propuesta, con todas las réplicas arrancando en la versión 0.
2. En cada escritura, verificar el quórum y actualizar las réplicas elegidas.
3. En cada lectura, verificar el quórum y devolver la versión máxima entre las consultadas; marcar si es más vieja que la última confirmada.
4. Mostrar el resultado de cada operación para cada combinación.

Salida esperada

```
W=1 R=1 (W+R<=3): w1:ok w2:ok r=1(VIEJO, último 2) w3:ok r=0(VIEJO, último 3)
W=2 R=2 (W+R>3): w1:ok w2:ok r=2 w3:RECHAZADA r=2
W=3 R=1 (W+R>3): w1:ok w2:RECHAZADA r=1 w3:RECHAZADA r=1
W=1 R=3 (W+R>3): w1:ok w2:ok r=2 w3:ok r=3
```

Reglas

- Complejidad: $O(N)$ por operación.
- Si $W + R > N$, cualquier conjunto de R réplicas comparte al menos una con cualquier conjunto de W : la lectura ve la última escritura.
- No funcional: disponibilidad contra consistencia: con $W = N$, una sola caída impide escribir; con $W = 1$, se escribe siempre pero las lecturas pueden ser viejas.
- No funcional: para datos clínicos, una lectura vieja puede ser peligrosa: la elección de W y R es una decisión de negocio documentada.

Pista (leela solo si te trabás): pensá en dos grupos elegidos de 3 réplicas: uno de W y otro de R . Si $W + R$ es mayor que 3, no pueden ser disjuntos, así que alguna réplica leída tiene la última versión.

Bonus (opcional)

- Agregá "reparación en la lectura": cuando una lectura detecta réplicas viejas, les escribe la versión nueva.
- Simulá una partición de red: dos clientes, cada uno ve solo parte de las réplicas.

Cierre: Los quóruns de lectura y escritura son la base de bases de datos distribuidas como las de tipo Dynamo.

Ejercicio 460: Comprimir los logs de los sensores

Nivel 10: Staff / Experto · Temática: Ciencia · Dificultad 93/100

Situación: Los sensores de una estación científica mandan logs con muchísimas repeticiones ("TEMP21TEMP21..."). LZ77 comprime reemplazando cada repetición por una referencia hacia atrás: "copiá L caracteres empezando D posiciones atrás". Es la idea base de formatos como ZIP y PNG.

Datos de entrada

Formato: primera línea: tamaño de la ventana hacia atrás y largo máximo de coincidencia; segunda: el texto.

Salida: tripletas (desplazamiento, largo, siguiente carácter). Se busca la coincidencia más larga en la ventana (en empate, el menor desplazamiento); el último carácter del texto siempre va como "siguiente carácter"; la coincidencia puede extenderse sobre la parte que se está copiando.

Ejemplo

```
16 8
TEMP21TEMP21TEMP21TEMP22TEMP21TEMP21TEMP21TEMP21
```

Restricciones: logs de hasta 10^9 caracteres; ventana de hasta 32 KB.

Tu programa debe

1. Recorrer el texto desde la posición actual y buscar, entre los desplazamientos de 1 al tamaño de la ventana, la coincidencia más larga con lo que sigue.
2. Emitir la tripleta (desplazamiento, largo, carácter siguiente) y avanzar largo + 1.
3. Decodificar las tripletas copiando carácter por carácter desde atrás (así funcionan las copias superpuestas) y verificar.
4. Mostrar las tripletas, la verificación y el tamaño estimado (3 bytes por tripleta).

Salida esperada

```
Tripletas: (0,0,T) (0,0,E) (0,0,M) (0,0,P) (0,0,2) (0,0,1) (6,8,M) (6,8,2) (12,8,M) (6,8,1)
(6,5,1)
Decodificado igual al original: sí
Caracteres: 48 | tripletas: 11 | tamaño estimado 33 bytes contra 48 (68.8 %)
```

Reglas

- Complejidad: $O(n \times \text{ventana} \times \text{largo})$ con búsqueda directa; con tablas de hash de prefijos se acelera muchísimo.
- Si no hay coincidencia, la tripleta es (0, 0, carácter).
- No funcional: la decodificación es muy rápida (solo copias), lo que importa para leer logs frecuentemente.
- No funcional: con textos sin repeticiones, LZ77 agranda los datos; un compresor real detecta ese caso y guarda el bloque sin comprimir.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): copiar "desde 6 posiciones atrás, 12 caracteres" funciona aunque todavía no existan los últimos 6: los vas escribiendo mientras copiás. Así, una repetición larga se codifica con una sola tripleta.

Bonus (opcional)

- Combiná LZ77 con Huffman sobre las tripletas (como hace DEFLATE) y medí el tamaño total.
- Usá una tabla de hash de los 3 caracteres siguientes para encontrar candidatos rápido.

Cierre: La familia LZ77 está en ZIP, gzip, PNG y en la compresión de casi todo lo que viaja por internet.

Ejercicio 461: Todas las cosechas posibles de la cooperativa

Nivel 10: Staff / Experto · Temática: Agricultura · Dificultad 93/100

Situación: Una cooperativa agrícola quiere saber de cuántas formas distintas puede llegar a cada total de toneladas, combinando lo que puede aportar cada productor (cada productor tiene cierta cantidad de lotes posibles para cada cantidad). Eso es multiplicar polinomios: el coeficiente de x^t del producto cuenta las formas de sumar t . Con miles de productores y totales grandes, la FFT multiplica en $O(n \log n)$.

Datos de entrada

Formato: un productor por línea: la cantidad de formas de aportar 0, 1, 2... toneladas (coeficientes del polinomio).

Ejemplo

```
1 2 1
1 1 1 1
2 1
3 0 1
```

Restricciones: polinomios de grado hasta 10^6 ; coeficientes chicos (con coeficientes grandes, usá la transformada numérica módulo un primo).

Tu programa debe

1. Rellenar los dos polinomios con ceros hasta una potencia de 2 mayor o igual que el largo del producto.
2. Aplicar la FFT (Cooley-Tukey recursiva: pares e impares, combinados con las raíces de la unidad) a ambos.
3. Multiplicar punto a punto, aplicar la FFT inversa, dividir por n y redondear al entero más cercano.
4. Multiplicar los productores de a uno; mostrar el resultado, el total de combinaciones y la verificación contra la multiplicación directa.

Salida esperada

```
Formas de lograr cada total (toneladas 0 a 8): 6 21 35 43 41 27 13 5 1
Total de combinaciones: 192
Coincide con la multiplicación directa: sí | máximo error antes de redondear: menor a 1e-9
```

Reglas

- Complejidad esperada: $O(n \log n)$ por producto, contra $O(n^2)$ de la multiplicación directa.
- No funcional: precisión: la FFT con números complejos acumula error de redondeo; hay que verificar que el error antes de redondear sea chico (se muestra).
- No funcional: con coeficientes grandes o productos exactos módulo un primo, la transformada numérica (NTT) evita el problema.
- El total de combinaciones es el producto de las sumas de cada polinomio (sirve de control).

Pista (leela solo si te trabás): un polinomio de grado n queda determinado por sus valores en $n + 1$ puntos; multiplicar valores es trivial. La FFT elige puntos especiales (raíces de la unidad) que permiten pasar de coeficientes a valores y volver en $O(n \log n)$.

Bonus (opcional)

- Implementá la NTT módulo 998.244.353 y compará con la FFT compleja.
- Calculá la probabilidad de cada total si cada forma es igualmente probable.

Cierre: La FFT está en telecomunicaciones, procesamiento de audio e imágenes, y en la multiplicación rápida de números gigantes.

Ejercicio 462: Alimento balanceado al menor costo

Nivel 10: Staff / Experto · Temática: Agricultura · Dificultad 93/100

Situación: Una fábrica de alimento balanceado para ganado mezcla maíz, harina de soja y afrechillo. Cada kilo de mezcla tiene que cumplir mínimos de proteína, energía y fibra, y se busca el menor costo. Es un problema de programación lineal; en este ejercicio se resuelve con el método simplex aplicado al problema dual, que tiene una solución inicial obvia.

Datos de entrada

Formato: primera línea: ingredientes con su costo por kg; después "nutriente contenido_por_kg_de_cada_ingredientes $\geq$ mínimo_por_ración".

Primal: minimizar costo $\times x$ sujeto a $Ax \geq b$, $x \geq 0$ (x = kg de cada ingrediente por ración diaria; en este modelo simplificado la ración no tiene un peso fijo). Dual: maximizar $b \times y$ sujeto a $A^T y \leq$ costo, $y \geq 0$.

Ejemplo

```
maíz 180 soja 420 afrechillo 150
proteína 8 44 16  $\geq$  18
energía 3.3 2.9 2.0  $\geq$  2.8
fibra 2 6 10  $\geq$  5
```

Restricciones: hasta 10^3 ingredientes y 10^2 nutrientes.

Tu programa debe

1. Armar el dual: una variable y por nutriente, una restricción por ingrediente ($A^T y \leq$ costo). Como los costos son positivos, $y = 0$ es factible: el simplex arranca ahí.
2. Aplicar el simplex con fracciones exactas: elegir como entrante la primera variable con costo reducido negativo (regla de Bland) y como saliente la fila de menor cociente (en empate, la de menor índice de variable básica).
3. Al terminar, el óptimo del dual es el costo mínimo del primal, y la cantidad de cada ingrediente es el costo reducido de la holgura de su restricción en la fila objetivo.
4. Mostrar la mezcla, el costo mínimo, los precios sombra (valores de y) y la cantidad de pivoteos.

Salida esperada

```
Mezcla óptima por ración: maíz 0.2391 kg, soja 0.0000 kg, afrechillo 1.0054 kg
Costo mínimo: $193.86 por ración
Precios sombra (cuánto sube el costo por unidad extra de requisito): proteína 3.67, energía 45.65,
fibra 0.00
Pivoteos del simplex: 2
```

Reglas

- Complejidad: exponencial en el peor caso teórico, pero muy rápido en la práctica.
- La regla de Bland evita que el simplex cicle en problemas degenerados.
- No funcional: exactitud: con fracciones, el óptimo es exacto; los solvers industriales usan decimales con tolerancias cuidadosamente elegidas.
- No funcional: los precios sombra se informan porque son la información de negocio más útil: cuánto cuesta endurecer cada requisito.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): dualidad: el costo mínimo de cumplir los requisitos es igual al máximo "valor" que se le puede asignar a cada nutriente sin que ningún ingrediente resulte más barato que su contenido. Resolver el dual evita tener que buscar una mezcla inicial factible.

Bonus (opcional)

- Agregá la restricción de que los ingredientes sumen exactamente 1 kg (hace falta el simplex de dos fases).
- Hacé análisis de sensibilidad: ¿hasta qué precio de la soja la mezcla óptima no cambia?

Cierre: La programación lineal optimiza dietas, producción, transporte y carteras; el simplex es uno de los algoritmos más importantes del siglo XX.

Ejercicio 463: Cuánto podemos perder en un mal día

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 93/100

Situación: El área de riesgo de un banco informa todos los días el valor en riesgo (VaR): la pérdida que no se supera con 95 % de confianza en un día. Para una cartera de acciones y bonos con rendimientos correlacionados, simula diez mil días posibles con Monte Carlo, ordena las pérdidas y toma el percentil. La simulación usa una semilla para que el informe sea auditable.

Datos de entrada

Formato: primera línea: cantidad de simulaciones y semilla; después "activo monto media_diaria desvío_diario" (dos activos); última línea: correlación.

Para cada simulación: $x = \text{siguiente}(x)$; $u1 = (x + 0,5) / 2^{31}$; $x = \text{siguiente}(x)$; $u2 = (x + 0,5) / 2^{31}$ ($x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$). Box-Muller: $z1 = \sqrt{-2 \ln u1} \cos(2\pi u2)$, $z2 = \sqrt{-2 \ln u1} \sin(2\pi u2)$. Correlación: $e1 = z1$, $e2 = \rho z1 + \sqrt{1 - \rho^2} z2$. Resultado = suma de monto $\times$ (media + desvío $\times$ e).

Ejemplo

```
10000 2026
acciones 6000000 0.0005 0.025
bonos 4000000 0.0002 0.008
0.3
```

Restricciones: hasta 10^6 simulaciones y 10^4 activos (con la descomposición de Cholesky de la matriz de covarianzas).

Tu programa debe

1. Generar los pares de normales independientes con Box-Muller y correlacionarlos.
2. Calcular el resultado de la cartera en cada simulación y ordenar de menor a mayor.
3. VaR al 95 % = $-\text{resultado en la posición } [0,05 \times N]$ (desde 0); al 99 %, en $[0,01 \times N]$; el ES (expected shortfall) es el promedio de las pérdidas por debajo de la posición del 95 %.
4. Comparar con el VaR paramétrico $1,645 \sigma_{\text{cartera}} - \mu_{\text{cartera}}$ y mostrar todo redondeado al peso.

Salida esperada

```
VaR 95 % (Monte Carlo): $267717 | VaR 99 %: $365963
Pérdida esperada más allá del VaR 95 % (ES): $329209
VaR 95 % paramétrico ( $1,645 \sigma - \mu$ ): $263501 | peor escenario simulado: $595172
```

Reglas

- Complejidad: $O(N \log N)$ por el ordenamiento ($O(N)$ con selección).
- No funcional: reproducibilidad: generador, semilla y transformación están especificados; con aritmética de doble precisión estándar, la salida es la misma en cualquier lenguaje.
- No funcional: regulatorio: el modelo se valida con backtesting (contar cuántos días reales la pérdida superó el VaR).
- El VaR no dice cuánto se pierde en el peor 5 %: por eso se informa también el ES.

Pista (leela solo si te trabás): Monte Carlo no necesita fórmulas: generás miles de días posibles, mirás el 5 % peor y leés dónde empieza. La correlación importa porque acciones y bonos que caen juntos empeoran la cola.

Bonus (opcional)

- Hacé backtesting con un año de rendimientos históricos: contá las excepciones y evaluá si son razonables.

- Extendé a n activos con la descomposición de Cholesky (ejercicio del riesgo de crédito).

Cierre: El VaR y el ES son medidas estándar de riesgo de mercado exigidas por los reguladores bancarios.

Ejercicio 464: El recorrido de 300 entregas

Nivel 10: Staff / Experto · Temática: Logística y envíos · Dificultad 93/100

Situación: Una empresa de envíos arma cada mañana el recorrido de un camión con 300 entregas. Para esa cantidad, encontrar el óptimo exacto es inviable. El recorrido simulado busca un buen recorrido aplicando cambios al azar (invertir un tramo) y aceptando a veces cambios que empeoran, con una probabilidad que baja con la "temperatura", para escapar de óptimos locales. El ejemplo usa 12 puntos para poder verificarlo.

Datos de entrada

Formato: primera línea: cantidad de puntos y semilla; segunda: temperatura inicial, factor de enfriamiento e iteraciones.

Puntos: para cada uno, $x = \text{siguiente}(x)$, coordenada $a = x \bmod 100$; $x = \text{siguiente}(x)$, $b = x \bmod 100$ ($x = (1103515245 \times x + 12345) \bmod 2^{31}$). En cada iteración: $i = \text{siguiente}(x) \bmod n$, $j = \text{siguiente}(x) \bmod n$ (ordenados); si $j - i < 2$, solo se enfría; si no, se invierte el tramo $i..j$, se calcula Δ y $u = \text{siguiente}(x) / 2^{31}$; se acepta si $\Delta < 0$ o $u < e^{(-\Delta/T)}$. Siempre $T = T \times \text{factor}$ al final de la iteración.

Ejemplo

```
12 42
100 0.995 6000
```

Restricciones: hasta 10^4 entregas y 10^7 iteraciones.

Tu programa debe

1. Generar los puntos y partir del recorrido 0, 1, 2, ..., $n - 1$.
2. En cada iteración, proponer la inversión de un tramo (movimiento 2-opt), calcular la nueva longitud y decidir con el criterio de Metropolis.
3. Guardar el mejor recorrido visto y enfriar la temperatura.
4. Comparar con el recorrido inicial y con la heurística del vecino más cercano; mostrar el mejor recorrido empezando en el punto 0.

Salida esperada

```
Puntos: (27,64) (53,6) (35,32) (33,66) (59,52) (65,42) (3,36) (13,14) (19,0) (53,58) (59,80)
(93,34)
Recorrido inicial (en orden): 491.86 | vecino más cercano: 347.63
Recocido simulado: mejor 304.06 | movimientos aceptados 323
Mejor recorrido: 0 3 10 9 4 5 11 1 8 7 2 6
```

Reglas

- Complejidad: $O(\text{iteraciones} \times n)$ con recálculo completo; $O(\text{iteraciones})$ calculando Δ solo con las 4 aristas que cambian (bonus).
- No funcional: reproducibilidad: con la misma semilla y aritmética de doble precisión, el resultado es idéntico.
- No funcional: tiempo acotado: el algoritmo tiene que terminar antes de que salga el camión, así que se fija la cantidad de iteraciones, no una precisión.
- No se garantiza el óptimo; el valor del método está en encontrar recorridos muy buenos en tiempo controlado.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): al principio, con temperatura alta, se aceptan casi todos los cambios (exploración); al final, casi solo mejoras (refinamiento). Aceptar empeoramientos a tiempo evita quedar atrapado en un recorrido "bueno pero no tanto".

Bonus (opcional)

- Calculá Δ en $O(1)$ con las 4 aristas que cambian al invertir el tramo.
- Compará con el óptimo exacto de Held-Karp (nivel 7) en el ejemplo de 12 puntos.

Cierre: El recocido simulado es una metaheurística clásica para ruteo, cronogramas, diseño de circuitos y cualquier optimización combinatoria difícil.

Ejercicio 465: El precio justo de una opción

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 94/100

Situación: Una plataforma de inversión muestra el precio teórico de opciones europeas sobre acciones: el derecho (no la obligación) de comprar o vender a un precio fijo en una fecha. El modelo de Black-Scholes da una fórmula cerrada a partir del precio actual, el precio de ejercicio, la tasa libre de riesgo, la volatilidad y el plazo. La paridad compra-venta sirve para verificar el cálculo.

Datos de entrada

Formato: una línea por opción: "precio_actual precio_de_ejercicio tasa volatilidad plazo_en_años" (tasa y volatilidad anuales, continuas).

$d1 = (\ln(S/K) + (r + \sigma^2/2) T) / (\sigma \sqrt{T})$; $d2 = d1 - \sigma \sqrt{T}$. Compra = $S N(d1) - K e^{(-rT)} N(d2)$; venta = $K e^{(-rT)} N(-d2) - S N(-d1)$; $N(x) = (1 + \operatorname{erf}(x / \sqrt{2})) / 2$.

Ejemplo

```
1000 1050 0.05 0.30 0.5
1000 900 0.05 0.30 0.25
1000 1000 0.05 0.60 1
```

Restricciones: hasta 10^6 valuaciones por segundo.

Tu programa debe

1. Calcular $d1$ y $d2$.
2. Calcular la función de distribución normal N con la función de error (erf).
3. Calcular los precios de compra (call) y venta (put) y el delta de la compra, $N(d1)$.
4. Verificar la paridad: compra – venta = $S - K e^{(-rT)}$ (la diferencia debe ser prácticamente 0).
Mostrar con 4 decimales.

Salida esperada

```
S=1000 K=1050 σ=0.3 T=0.5: compra 73.9841 | venta 98.0595 | delta compra 0.4976 | paridad compra -
venta - (S - K e^{(-rT)}) = 0.0000000000
S=1000 K=900 σ=0.3 T=0.25: compra 128.5820 | venta 17.4020 | delta compra 0.8053 | paridad compra
- venta - (S - K e^{(-rT)}) = 0.0000000000
S=1000 K=1000 σ=0.6 T=1: compra 255.2321 | venta 206.4615 | delta compra 0.6493 | paridad compra -
venta - (S - K e^{(-rT)}) = 0.0000000000
```

Reglas

- Complejidad: $O(1)$ por opción.
- Supuestos del modelo: volatilidad constante, sin dividendos, ejercicio solo al vencimiento (europea), mercado sin fricciones.
- No funcional: validación: si la paridad no se cumple con tolerancia 10^{-10} , hay un error de implementación.
- No funcional: los precios se muestran como referencia teórica, aclarando los supuestos, no como recomendación de inversión.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): $N(d2)$ se interpreta como la probabilidad (ajustada por riesgo) de que la opción termine ejerciéndose, y $N(d1)$ como la cantidad de acciones que replican la opción (el delta).

Bonus (opcional)

- Calculá la volatilidad implícita: la σ que hace que el precio del modelo iguale un precio de mercado (Newton-Raphson con vega como derivada).

- Compará con el árbol binomial (ejercicio de la opción americana) con 1.000 pasos.

Cierre: Black-Scholes revolucionó los mercados financieros y le valió el premio Nobel de Economía a sus autores en 1997.

Ejercicio 466: La cartera óptima con presupuesto entero

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 94/100

Situación: Un asesor arma carteras para clientes minoristas: cada instrumento se compra en unidades enteras (no se puede comprar media cuotaparte de ese fondo), tiene un precio, una ganancia esperada anual y un límite de unidades por diversificación. Con presupuesto fijo, se busca la máxima ganancia esperada. Ramificación y poda explora las combinaciones descartando las ramas que no pueden superar la mejor solución encontrada.

Datos de entrada

Formato: primera línea: presupuesto; después "instrumento precio ganancia_esperada máximo_de_unidades".

Ejemplo

```
100000
bono_corto 12000 1300 5
acción_A 25000 3200 3
acción_B 18000 2100 4
fondo_C 30000 3800 2
cedear_D 9000 900 6
```

Restricciones: hasta 50 instrumentos; presupuestos grandes (la programación dinámica por presupuesto no escala).

Tu programa debe

1. Ordenar los instrumentos por ganancia por peso (en empate, por nombre).
2. Explorar en profundidad: para cada instrumento, probar cantidades de la mayor posible a 0.
3. En cada nodo, calcular una cota superior con la relajación fraccionaria (llenar el presupuesto restante voraz permitiendo fracciones): si no supera la mejor solución, podar.
4. Mostrar la cartera óptima (en el orden de exploración), la inversión, la ganancia, los nodos explorados y la verificación con programación dinámica (en miles de pesos).

Salida esperada

```
Cartera óptima: acción_A x2, fondo_C x1, acción_B x1
Inversión 98000 de 100000 | ganancia esperada 12300
Nodos explorados: 29 | verificación con programación dinámica: 12300
```

Reglas

- Complejidad: exponencial en el peor caso; la poda con buena cota reduce muchísimo los nodos en la práctica.
- La relajación fraccionaria siempre da una cota válida ($\geq$ al óptimo entero), por eso podar con ella nunca pierde el óptimo.
- No funcional: tiempo acotado: si la búsqueda supera un límite de nodos, se devuelve la mejor solución encontrada, informando que puede no ser óptima.
- No funcional: la ganancia esperada no es garantizada: el informe al cliente incluye el riesgo.

Pista (leela solo si te trabás): si incluso llenando lo que queda con "pedacitos" del mejor instrumento no superás lo que ya tenés, esa rama no vale la pena. Empezar por las cantidades grandes encuentra rápido una buena solución, que hace que la poda sea más efectiva.

Bonus (opcional)

- Agregá un límite de riesgo total (suma de riesgo por unidad) como segunda restricción.
- Compará los nodos explorados con y sin ordenar por ganancia por peso.

Cierre: Ramificación y poda es la técnica base de los solvers de programación entera, usados en finanzas, logística y planificación.

Ejercicio 467: Elección de líder entre servidores

Nivel 10: Staff / Experto · Temática: Telecomunicaciones · Dificultad 94/100

Situación: Un servicio de mensajería corre en un grupo de servidores y uno de ellos es el líder, que coordina las escrituras. Si el líder se cae, los demás tienen que ponerse de acuerdo en uno nuevo sin intervención humana. El algoritmo del "matón" (bully) elige al servidor vivo de mayor id: quien detecta la caída desafía a los mayores y, si alguno responde, este toma la posta.

Datos de entrada

Formato: primera línea: ids de los servidores; después eventos: "lider x", "cae x", "detecta x" (x nota que el líder no responde) y "vuelve x" (x vuelve e inicia una elección).

Conteo de mensajes (modelo simplificado): cada desafío a un mayor es 1 mensaje (vivo o no), cada respuesta es 1, y el anuncio del ganador son $n - 1$ mensajes. El que continúa la elección es el menor de los que respondieron.

Ejemplo

```
1 2 3 4 5
lider 5
cae 5
detecta 2
cae 4
detecta 1
vuelve 5
```

Restricciones: hasta 100 servidores.

Tu programa debe

1. Cuando un servidor detecta que el líder no responde, envía desafíos a todos los de id mayor.
2. Si nadie vivo responde, gana y avisa a todos; si responden, la elección continúa desde el menor que respondió (que a su vez desafía a sus mayores).
3. Un servidor que vuelve inicia una elección (si tiene el id más alto, recupera el liderazgo).
4. Mostrar cada evento, el desarrollo de las elecciones y la cantidad de mensajes.

Salida esperada

```
Líder inicial: 5
Cae el servidor 5
2 detecta la caída del líder: 2 inicia elección; 2 -> 3, 4, 5: responden 3, 4; 3 -> 4, 5:
    responden 4; 4 gana y avisa a todos (13 mensajes)
Cae el servidor 4
1 detecta la caída del líder: 1 inicia elección; 1 -> 2, 3, 4, 5: responden 2, 3; 2 -> 3, 4, 5:
    responden 3; 3 gana y avisa a todos (16 mensajes)
Vuelve 5: 5 inicia elección; 5 gana y avisa a todos (4 mensajes)
```

Reglas

- Complejidad: $O(n^2)$ mensajes en el peor caso.
- No funcional: vivacidad: mientras haya al menos un servidor vivo, siempre termina habiendo líder.
- No funcional: seguridad: el modelo supone que la detección de caídas es perfecta; con redes reales (particiones, demoras), pueden surgir dos líderes, y por eso se usan algoritmos de consenso con mayoría como Raft (bonus).
- Si el líder responde, no hay elección.

Pista (leela solo si te trabás): el algoritmo es simple porque el ganador está predeterminado: el mayor vivo. Todo el intercambio de mensajes sirve para descubrir quién es sin un coordinador.

Bonus (opcional)

- Implementá la elección de Raft simplificada: términos, votos por mayoría y tiempos de espera aleatorios con semilla.
- Simulá una partición de red y mostrá cómo el algoritmo del matón produce dos líderes.

Cierre: La elección de líder es un problema central de los sistemas distribuidos: bases de datos replicadas, colas de mensajes y orquestadores.

Ejercicio 468: El libro de actas inalterable

Nivel 10: Staff / Experto · Temática: Educación · Dificultad 94/100

Situación: Una universidad lleva el libro de actas del consejo en formato digital con una cadena de hashes. Pero una cadena sola no alcanza: quien administra el sistema podría borrar un acta y recalculer todos los hashes siguientes. Por eso, cada tanto, publica un "sello" (el último hash) en un medio externo que no controla. Una eliminación posterior rompe la coincidencia con algún sello.

Datos de entrada

Formato: las actas en orden (texto sin tildes, para que el hash trabaje sobre caracteres ASCII); una línea "?", la línea "sellos" con los números de acta tras los que se publicó un sello, y la manipulación "eliminar k".

$\text{hash}(i) = H(\text{hash}(i - 1) + "|" + \text{acta}(i))$, con $\text{hash}(0) = "0"$. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
Acta 1: apertura del ciclo lectivo
Acta 2: aprobacion del plan de estudios
Acta 3: designacion de jurados
Acta 4: calendario de examenes
Acta 5: sancion disciplinaria revisada
Acta 6: cierre del primer cuatrimestre
?
sellos 3 6
eliminar 5
```

Restricciones: hasta 10^6 actas; un sello por semana.

Tu programa debe

1. Calcular la cadena de hashes y los sellos publicados.
2. Simular la manipulación: eliminar el acta k y recalculer la cadena completa (como haría un atacante con acceso al sistema).
3. Comparar cada sello publicado con el hash recalculado en la misma posición.
4. Mostrar los sellos, las comparaciones y el primer sello roto (que acota dónde está la manipulación).

Salida esperada

```
Sellos publicados: tras acta 3: 1238200114, tras acta 6: 2691860363
Alguien elimina el acta 5 y recalcula toda la cadena (ahora hay 5 actas):
  sello tras acta 3: publicado 1238200114, recalculado 1238200114 -> coincide
  sello tras acta 6: publicado 2691860363, el acta 6 ya no existe -> NO COINCIDE: manipulación
    detectada
Primer sello roto: tras el acta 6 (la manipulación está entre el sello anterior y este)
```

Reglas

- Complejidad: $O(n)$ para verificar toda la cadena.
- Sin sellos externos, recalculer la cadena completa ocultaría cualquier cambio: la seguridad está en publicar afuera.
- No funcional: los sellos se publican en al menos dos medios independientes (por ejemplo, un diario y un correo a los consejeros).
- No funcional: la verificación la puede hacer cualquiera con las actas y los sellos, sin confiar en el administrador.
- Casos borde: manipulación después del último sello (no se detecta hasta el próximo sello).

Pista (leela solo si te trabás): la cadena de hashes garantiza que no se puede cambiar el pasado sin cambiar todos los hashes siguientes; el sello externo garantiza que esos hashes no se pueden cambiar.

Bonus (opcional)

- Detectá también el reordenamiento de dos actas y un cambio de texto, y mostrá qué sello rompe cada uno.
- Publicá como sello la raíz de un árbol de Merkle de las actas de la semana (ejercicio de la transacción en el lote).

Cierre: Las cadenas de hashes con anclajes externos se usan en registros de auditoría, archivos notariales digitales y sistemas de transparencia.

Ejercicio 469: Rentabilidad sin perder el sueño

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 94/100

Situación: Un robo-advisor arma carteras con tres clases de activos. Para cada rendimiento esperado objetivo, busca la combinación de menor riesgo (varianza), teniendo en cuenta que los activos se mueven de forma correlacionada: es el modelo de Markowitz. Con restricciones de igualdad, el óptimo sale de resolver un sistema lineal (condiciones de Lagrange).

Datos de entrada

Formato: primera línea: los activos; segunda: rendimientos esperados anuales; después la matriz de covarianzas (3 filas); última línea: rendimientos objetivo.

Ejemplo

```
bonos acciones_locales cedears
0.08 0.14 0.11
0.0025 0.0010 0.0005
0.0010 0.0400 0.0120
0.0005 0.0120 0.0225
0.10 0.12
```

Restricciones: hasta 10^3 activos (sistema de $n + 2$ ecuaciones).

Tu programa debe

1. Cartera de mínima varianza: minimizar $w^T \Sigma w$ con suma de pesos = 1; las condiciones de Lagrange dan el sistema $2\Sigma w + \lambda \cdot 1 = 0$, $1^T w = 1$.
2. Para cada objetivo, agregar la restricción $\mu^T w = \text{objetivo}$: $2\Sigma w + \lambda_1 \mu + \lambda_2 \cdot 1 = 0$, $\mu^T w = \text{objetivo}$, $1^T w = 1$.
3. Resolver cada sistema con eliminación gaussiana y fracciones exactas.
4. Mostrar los pesos (2 decimales, en %), el rendimiento y la volatilidad ($\sqrt{w^T \Sigma w}$).

Salida esperada

```
Mínima varianza: bonos 91.10 %, acciones_locales 1.25 %, cedears 7.66 % | rendimiento 8.30 % |
volatilidad 4.83 %
Objetivo 10.00 %: bonos 57.99 %, acciones_locales 24.65 %, cedears 17.36 % | volatilidad 7.32 %
Objetivo 12.00 %: bonos 18.93 %, acciones_locales 52.27 %, cedears 28.80 % | volatilidad 12.94 %
```

Reglas

- Complejidad: $O(n^3)$ por sistema.
- En este modelo se permiten pesos negativos (ventas en corto); con la restricción $w \geq 0$ hace falta programación cuadrática (bonus).
- No funcional: estabilidad: con covarianzas estimadas con pocos datos, los pesos óptimos cambian mucho ante pequeños cambios; se informa esa sensibilidad.
- No funcional: la recomendación final incluye el perfil de riesgo del cliente, no solo el óptimo matemático.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): minimizar una forma cuadrática con restricciones lineales de igualdad es resolver un sistema lineal: derivás, igualás a cero y agregás las restricciones como ecuaciones.

Bonus (opcional)

- Graficá la frontera eficiente: la volatilidad mínima para objetivos entre 8 % y 14 %.
- Prohibí las ventas en corto ($w \geq 0$) y resóvelo con un método de conjunto activo.

Cierre: El modelo de Markowitz es la base de la teoría moderna de carteras y de la mayoría de los robo-advisors.

Ejercicio 470: La receta colaborativa sin conflictos

Nivel 10: Staff / Experto · Temática: Gastronomía · Dificultad 94/100

Situación: Una app de recetas permite que dos personas editen la misma receta a la vez, incluso sin conexión. Si cada una inserta texto en la misma posición, al sincronizar el resultado tiene que ser idéntico en los dos celulares, sin importar el orden en que llegaron los cambios. Un CRDT de secuencias (RGA) le da a cada carácter un id único y ordena las inserciones concurrentes por id.

Datos de entrada

Formato: operaciones "réplica contador insertar_después_de carácter" ("- " = al principio; los ids se escriben réplica + contador, por ejemplo A2) y "borrar id"; una línea "?" y, para cada réplica, el orden en que le llegaron las operaciones (números de operación desde 1).

Id de un carácter: (contador, réplica); se compara primero el contador y después la réplica. Cada réplica recibe una operación solo después de las que la causaron.

Ejemplo

```
A 1 - S
A 2 A1 A
A 3 A2 A
A 4 A3 L
borrar A3
A 5 A4 +
A 6 A5 A
A 7 A6 J
A 8 A7 O
B 5 A4 +
B 6 B5 P
B 7 B6 A
B 8 B7 N
?
orden_réplica_1 1 2 3 4 5 6 7 8 9 10 11 12 13
orden_réplica_2 1 2 3 4 10 11 12 13 6 7 8 9 5
```

Restricciones: documentos de hasta 10^6 caracteres y 10^3 editores.

Tu programa debe

1. Guardar el documento como lista de elementos (id, carácter, borrado).
2. Para insertar después de un id: ubicarse justo después de ese elemento y saltar todos los elementos siguientes con id mayor al nuevo (inserciones concurrentes "más nuevas"); insertar ahí.
3. Borrar marca el elemento como borrado (lápida), sin sacarlo, para que otras inserciones lo puedan seguir usando como referencia.
4. Aplicar las operaciones en el orden de cada réplica y mostrar el texto visible, el texto con las lápidas en minúscula y si convergen.

Salida esperada

```
orden_réplica_1: "SAL+PAN+AJ0" (con borrados: SAaL+PAN+AJ0)
orden_réplica_2: "SAL+PAN+AJ0" (con borrados: SAaL+PAN+AJ0)
Convergen: sí
```

Reglas

- Complejidad: $O(n)$ por operación con una lista; con árboles balanceados, $O(\log n)$.
- No funcional: convergencia: dos réplicas que recibieron el mismo conjunto de operaciones muestran exactamente el mismo texto, sin coordinación ni servidor central.
- No funcional: intención: las inserciones de cada usuario quedan juntas y en su orden, aunque se intercalen en el tiempo con las de otro.

- No funcional: las lápidas ocupan memoria; se eliminan cuando todas las réplicas confirmaron haberlas visto (recolección de basura).

Pista (leela solo si te trabás): la regla de "saltar los ids mayores" hace que el lugar donde cae una inserción no dependa de qué otras inserciones concurrentes llegaron antes: todas las réplicas terminan con el mismo orden.

Bonus (opcional)

- Agregá deshacer: una operación "restaurar id" que quita la lápida.
- Compará con la transformación operacional (OT), que resuelve el mismo problema transformando posiciones.

Cierre: Los CRDT de secuencias son la base de editores colaborativos y apps de notas que funcionan sin conexión.

Ejercicio 471: Reseñas casi duplicadas a escala

Nivel 10: Staff / Experto · Temática: E-commerce · Dificultad 95/100

Situación: Un marketplace detecta reseñas casi duplicadas (la misma reseña con cambios mínimos, típica de campañas pagas). Con millones de reseñas no se pueden comparar todos los pares. MinHash resume cada reseña en una firma corta que estima la similitud de Jaccard, y LSH agrupa las firmas por bandas para comparar solo los pares que probablemente sean parecidos.

Datos de entrada

Formato: primera línea: largo de los fragmentos de caracteres k , cantidad de funciones de hash y cantidad de bandas; después "id: reseña".

Normalización: minúsculas, sin tildes ($\tilde{n} \rightarrow n$), solo letras, dígitos y espacios simples. Firma: para cada i de 0 a funciones $- 1$, el mínimo de $H(i + "|" + \text{fragmento})$ sobre los fragmentos de la reseña. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
5 20 5
r1: excelente producto llegó rápido y bien embalado lo recomiendo
r2: excelente producto, llegó rapido y bien embalado. lo recomiendo!!
r3: excelente producto llegó rápido bien embalado recomiendo
r4: el producto llegó roto y el vendedor no responde los mensajes
r5: el producto llegó roto y el vendedor no contesta los mensajes
```

Restricciones: hasta 10^8 reseñas.

Tu programa debe

1. Normalizar cada reseña y obtener sus fragmentos de k caracteres.
2. Calcular la firma MinHash: para cada función de hash, el mínimo sobre los fragmentos.
3. Dividir la firma en bandas; dos reseñas son candidatas si coinciden completamente en alguna banda (se agrupan en cubos por banda).
4. Para cada par candidato (en orden), mostrar la similitud estimada (fracción de posiciones iguales de la firma) y la real.

Salida esperada

```
Pares candidatos (LSH con 5 bandas de 4 filas): 3 de 10
r1 ~ r2: estimada 1.00 | real 1.00
r1 ~ r3: estimada 0.75 | real 0.76
r2 ~ r3: estimada 0.75 | real 0.76
```

Reglas

- Complejidad: $O(\text{fragmentos} \times \text{funciones})$ por reseña, y los pares candidatos son muchos menos que todos los pares.
- La probabilidad de que dos firmas coincidan en una posición es exactamente la similitud de Jaccard de los conjuntos.
- No funcional: escala: solo se comparan en detalle los candidatos; la cantidad de bandas y filas se ajusta para el umbral buscado (con b bandas de r filas, el umbral es aproximadamente $(1/b)^{(1/r)}$).
- No funcional: las reseñas marcadas pasan a revisión humana; no se borran automáticamente.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): el mínimo de un hash sobre un conjunto es como elegir un elemento al azar: dos conjuntos eligen el mismo con probabilidad igual a su similitud. Con muchas funciones, contás cuántas veces coinciden.

Bonus (opcional)

- Calculá la curva de probabilidad de ser candidato según la similitud para distintas combinaciones de bandas y filas.
- Agrupá las reseñas similares en campañas con union-find sobre los pares confirmados.

Cierre: MinHash y LSH se usan en buscadores, detección de duplicados, recomendaciones y bioinformática para encontrar similares a gran escala.

Ejercicio 472: La vibración del motor del colectivo

Nivel 10: Staff / Experto · Temática: Transporte · Dificultad 95/100

Situación: Una empresa de colectivos monitorea la vibración de los motores con acelerómetros: cada tipo de falla (un rodamiento gastado, un desbalance) vibra a una frecuencia característica. La señal medida es la suma de todas las vibraciones mezcladas; la transformada rápida de Fourier la descompone en frecuencias y muestra cuáles están presentes y con qué intensidad.

Datos de entrada

Formato: primera línea: cantidad de muestras N (potencia de 2) y frecuencia de muestreo (Hz); después, componentes de la señal simulada "amplitud frecuencia fase_en_grados".

Señal: $x[n] = \text{suma de amplitud} \times \sin(2\pi \times \text{frecuencia} \times n / f_s + \text{fase})$, para n de 0 a $N - 1$.

Ejemplo

```
64 64
3 5 0
1.5 12 0
0.5 20 90
```

Restricciones: N hasta 2^{20} muestras por segundo, por sensor.

Tu programa debe

1. Generar la señal muestreada sumando las componentes.
2. Aplicar la FFT (la misma de la multiplicación de polinomios).
3. Calcular la amplitud de cada frecuencia $k \times f_s / N$ como $|X[k]| \times 2 / N$, para k de 1 a $N/2 - 1$.
4. Mostrar las frecuencias con amplitud mayor a 0,1, la comparación de operaciones y la componente dominante.

Salida esperada

```
Frecuencias detectadas (Hz, amplitud): 5.0 Hz 3.000, 12.0 Hz 1.500, 20.0 Hz 0.500
Muestras: 64 | operaciones aproximadas FFT: 384 contra DFT directa: 4096
Componente dominante: 5.0 Hz
```

Reglas

- Complejidad: $O(N \log N)$ contra $O(N^2)$ de la transformada directa.
- Solo se pueden detectar frecuencias menores a $f_s / 2$ (límite de Nyquist); una frecuencia mayor aparece "disfrazada" de otra (aliasing).
- No funcional: tiempo real: el análisis de cada segundo de señal tiene que terminar antes de que llegue el siguiente.
- No funcional: con señales reales (frecuencias que no caen justo en un k), la energía se reparte entre k vecinos; se usan ventanas (bonus).
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): la FFT compara la señal con senos y cosenos de cada frecuencia a la vez. Si la señal tiene una componente de 12 Hz, la comparación con 12 Hz da grande y con las demás, casi cero.

Bonus (opcional)

- Agregá ruido con un generador con semilla y medí cuánto ruido tolera la detección.
- Aplicá una ventana de Hann y compará el espectro con una frecuencia que no cae justo en un k (por ejemplo 12,5 Hz).

Cierre: El análisis espectral con FFT se usa en mantenimiento predictivo, audio, telecomunicaciones, sismología y medicina.

Ejercicio 473: El cronograma de enfermería que conforma a todos

Nivel 10: Staff / Experto · Temática: Salud · Dificultad 95/100

Situación: El servicio de enfermería de un hospital arma el cronograma semanal: cada día necesita una cantidad fija de enfermeros (restricción dura), nadie puede trabajar más de cierto número de días (dura), y cada uno pidió francos en días específicos (blanda: se intenta respetar). Con planteles grandes, el recocido simulado encuentra cronogramas muy buenos minimizando una penalización.

Datos de entrada

Formato: primera línea: enfermeros, días, enfermeros por día, máximo de días por persona y semilla; después "nombre días_de_franco_pedidos..." (días 0 = lunes a 6 = domingo); última línea: iteraciones, temperatura inicial y factor de enfriamiento.

Cada día se elige un par de enfermeros (todos los pares posibles en orden: (0,1), (0,2), ...). Inicial: día d usa el par número d resto cantidad_de_pares. Movimiento: $d = \text{siguiente}(x)$ resto días; nuevo par = $\text{siguiente}(x)$ resto pares; $u = \text{siguiente}(x) / 2^{31}$. Penalización = $100 \times \text{días en exceso del máximo} + \text{cantidad de francos pedidos no respetados}$. Se acepta si no empeora o si $u < e^{(-\Delta/T)}$. $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$.

Ejemplo

```
4 7 2 4 7
Ana 0 6
Beto 2
Caro 5 6
Dani 1 3
2000 50 0.99
```

Restricciones: hasta 200 enfermeros, 31 días y 10^6 iteraciones.

Tu programa debe

1. Representar el cronograma como un par de enfermeros por día (así la cobertura siempre se cumple por construcción).
2. Calcular la penalización: las violaciones duras pesan 100 y las blandas 1.
3. Iterar: cambiar el par de un día al azar, calcular Δ y aceptar con el criterio de Metropolis; guardar el mejor y enfriar.
4. Mostrar la penalización inicial, la mejor encontrada (desglosada) y el cronograma.

Salida esperada

```
Penalización inicial: 3 (duras 0, blandas 3)
Mejor encontrada: 0 (duras 0, blandas 0)
lun: Caro+Dani | mar: Beto+Caro | mié: Ana+Caro | jue: Ana+Caro | vie: Ana+Dani | sáb: Ana+Dani |
dom: Beto+Dani
```

Reglas

- Complejidad: $O(\text{iteraciones} \times \text{días})$ con recálculo completo ($O(\text{iteraciones})$ actualizando solo el día cambiado).
- Modelar una restricción dura "por construcción" (cada día tiene exactamente un par) achica el espacio de búsqueda.
- No funcional: equidad: además del óptimo, el informe muestra cuántos francos le respetaron a cada persona.
- No funcional: reproducibilidad: con la misma semilla y aritmética de doble precisión, el mismo cronograma.

Pista (leela solo si te trabás): las restricciones duras con un peso enorme hacen que el algoritmo primero las elimine y después "pula" las blandas. Y lo que se puede garantizar desde la representación, conviene no dejarlo a la penalización.

Bonus (opcional)

- Agregá turnos (mañana, tarde, noche) y la regla de no encadenar noche con mañana.
- Resolvé el ejemplo exacto por fuerza bruta (6^7 cronogramas) y verificá que el recocido encontró el óptimo.

Cierre: Los cronogramas de personal con restricciones duras y blandas se resuelven en hospitales, aerolíneas y fábricas con metaheurísticas como esta.

Ejercicio 474: Inventario sincronizado entre depósitos

Nivel 10: Staff / Experto · Temática: Logística y envíos · Dificultad 95/100

Situación: Una empresa de logística tiene el inventario de un producto replicado en tres depósitos que a veces pierden conexión. Cada depósito puede corregir el stock (después de un conteo) y se sincronizan de a pares. Con relojes vectoriales se sabe si una versión es más nueva que otra o si hubo ediciones concurrentes (un conflicto real). Ante un conflicto, la política conservadora toma el menor stock y lo marca para recontar.

Datos de entrada

Formato: primera línea: los depósitos; después "depósito fija valor" (corrección local) o "sync origen destino" (el origen envía su estado al destino).

Al recibir: si el vector del origen es menor o igual, no cambia nada; si es mayor, el destino adopta el estado del origen; si son concurrentes, el valor es el mínimo, el vector es el máximo posición por posición y el destino incrementa su propia posición.

Ejemplo

```
D1 D2 D3
D1 fija 50
sync D1 D2
sync D1 D3
D2 fija 45
D3 fija 42
sync D2 D1
sync D3 D1
sync D1 D2
sync D1 D3
```

Restricciones: hasta 10^3 depósitos y 10^7 productos.

Tu programa debe

1. Guardar en cada depósito el valor y su reloj vectorial; una corrección local incrementa la posición propia.
2. Al sincronizar, comparar los vectores: igual, menor, mayor o concurrente.
3. Aplicar la regla correspondiente; en un conflicto, resolver con el mínimo y registrar la alerta.
4. Mostrar cada paso y el estado final.

Salida esperada

```
D1 fija 50: vc [1, 0, 0]
sync D1->D2: D2 adopta 50 | D2 = 50 vc [1, 0, 0]
sync D1->D3: D3 adopta 50 | D3 = 50 vc [1, 0, 0]
D2 fija 45: vc [1, 1, 0]
D3 fija 42: vc [1, 0, 1]
sync D2->D1: D1 adopta 45 | D1 = 45 vc [1, 1, 0]
sync D3->D1: CONFLICTO (42 vs 45): D1 resuelve con el mínimo 42 y lo marca para revisión | D1 = 42
vc [2, 1, 1]
sync D1->D2: D2 adopta 42 | D2 = 42 vc [2, 1, 1]
sync D1->D3: D3 adopta 42 | D3 = 42 vc [2, 1, 1]
Estado final: D1=42 | D2=42 | D3=42
```

Reglas

- Complejidad: $O(\text{depósitos})$ por sincronización.
- No funcional: seguridad del negocio: ante la duda, el stock mostrado nunca es mayor que el más bajo informado (evita vender lo que no hay).
- No funcional: trazabilidad: cada conflicto queda registrado con los dos valores para que un humano lo resuelva.

- No funcional: con "el último que escribe gana" por hora del reloj, una corrección se perdería en silencio; los vectores lo evitan.
- Después de suficientes sincronizaciones, todos los depósitos convergen al mismo valor.

Pista (leela solo si te trabás): el reloj vectorial responde "¿esta versión ya conocía a la otra?". Si sí, no hay conflicto: gana la que sabe más. Si ninguna conocía a la otra, dos personas corrigieron sin enterarse, y ahí se necesita una política.

Bonus (opcional)

- Guardá los dos valores en conflicto ("hermanos") en lugar de resolver con el mínimo, y dejá que el próximo conteo los reemplace.
- Modelá el stock como un contador PN (ejercicio de los votos) para las ventas y reservá los relojes para las correcciones.

Cierre: Los relojes vectoriales y la resolución explícita de conflictos son la base de las bases de datos con replicación multi-maestro.

Ejercicio 475: Dos réplicas, ¿en qué difieren?

Nivel 10: Staff / Experto · Temática: Telecomunicaciones · Dificultad 95/100

Situación: Dos réplicas de una base de datos de una operadora deberían tener los mismos datos, pero después de un corte de red pueden haber quedado diferencias. Mandar todas las claves para compararlas es carísimo. Con árboles de Merkle (anti-entropía), las réplicas comparan primero las raíces y bajan solo por las ramas cuyos hashes difieren, hasta encontrar exactamente las claves distintas.

Datos de entrada

Formato: dos líneas, una por réplica, con los pares clave=valor en el mismo orden de claves (cantidad potencia de 2).

Hoja = H("hoja:" + clave=valor); nodo = H("nodo:" + izquierdo + ":" + derecho). H(texto) = FNV-1a de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
k1=10 k2=20 k3=30 k4=40 k5=50 k6=60 k7=70 k8=80 k9=90 k10=100 k11=110 k12=120 k13=130 k14=140
k15=150 k16=160
k1=10 k2=20 k3=30 k4=40 k5=50 k6=60 k7=70 k8=80 k9=90 k10=100 k11=111 k12=120 k13=130 k14=140
k15=150 k16=160
```

Restricciones: hasta 10^9 claves por réplica; los árboles se mantienen actualizados al escribir.

Tu programa debe

1. Construir el árbol de Merkle de cada réplica sobre el mismo orden de claves.
2. Comparar las raíces: si son iguales, las réplicas coinciden.
3. Si difieren, bajar recursivamente a los dos hijos y comparar; en las hojas distintas está la diferencia.
4. Mostrar las raíces, las claves distintas y la cantidad de hashes comparados.

Salida esperada

```
Raíces: réplica 1 1906879872 | réplica 2 1406206030
Claves distintas: k11
Hashes comparados: 9 (comparar todo serían 16 valores; con  $10^9$  claves y 1 diferencia, unos 60 hashes)
```

Reglas

- Complejidad: $O(d \times \log n)$ hashes comparados para d diferencias, en lugar de $O(n)$.
- Las dos réplicas tienen que usar la misma partición de claves y la misma función de hash.
- No funcional: ancho de banda: solo viajan hashes de los subárboles que difieren, y al final, los valores de las claves distintas.
- No funcional: el proceso corre en segundo plano, periódicamente, sin bloquear lecturas ni escrituras.
- Una vez encontradas las diferencias, la resolución (qué valor gana) la decide otra política (ver el ejercicio del inventario).

Pista (leela solo si te trabás): si dos subárboles tienen el mismo hash, todo lo que hay abajo es igual: no hace falta mirarlo. Solo bajás por donde hay una diferencia.

Bonus (opcional)

- Hacé que el árbol se actualice en $O(\log n)$ con cada escritura, sin reconstruirlo.
- Estimá cuántos bytes viajan para 10^9 claves con 1.000 diferencias, contra enviar todo.

Cierre: La anti-entropía con árboles de Merkle se usa en bases de datos distribuidas como Cassandra y DynamoDB para reparar réplicas.

Lote 20: ejercicios 476 a 500

Ejercicio 476: Escaneos de puertos con memoria mínima

Nivel 10: Staff / Experto · Temática: Seguridad informática · Dificultad 96/100

Situación: El firewall de una empresa quiere detectar en tiempo real las direcciones que hacen escaneos de puertos (miles de intentos de conexión), para bloquearlas. Con millones de direcciones de origen, no puede tener un contador exacto por cada una. Un Count-Min sketch estima los intentos de cada origen con memoria fija y dispara una alerta apenas la estimación supera el umbral ("heavy hitters").

Datos de entrada

Formato: primera línea: ancho, profundidad y umbral de alerta; después "origen intentos". Los intentos llegan intercalados: en cada ronda, cada origen con intentos pendientes envía uno, en el orden de la entrada.

Columna del origen x en la fila i : $H(i + ":" + x) \bmod \text{ancho}$. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \bmod 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \bmod 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \bmod 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
32 3 400
10.0.0.1 30
10.0.0.2 25
10.0.0.3 40
10.0.0.4 12
10.0.0.5 8
10.0.0.6 20
10.0.0.7 5
10.0.0.8 18
10.0.0.9 15
10.0.0.10 22
192.168.9.9 900
172.16.4.4 650
10.0.0.11 35
10.0.0.12 10
```

Restricciones: hasta 10^8 eventos por minuto y 10^9 direcciones posibles.

Tu programa debe

1. Procesar los eventos en el orden intercalado, sumando 1 en las d celdas de cada origen.
2. Después de cada evento, estimar ese origen (mínimo de sus d celdas).
3. Si la estimación supera el umbral y el origen todavía no tiene alerta, generarla con el número de evento.
4. Mostrar las alertas y marcar los falsos positivos (se conocen porque el ejercicio tiene los totales reales).

Salida esperada

```
Eventos procesados: 1790 | memoria: 3 × 32 contadores
ALERTA 192.168.9.9: estimado 401 al evento 1041 (real total 900)
ALERTA 172.16.4.4: estimado 401 al evento 1042 (real total 650)
```

Reglas

- Complejidad: $O(d)$ por evento; memoria $O(d \times w)$.
- Como el sketch nunca subestima, ningún escaneo real que supere el umbral pasa inadvertido.
- No funcional: latencia: la alerta se emite en el mismo evento en que se cruza el umbral, no al final de la ventana.

- No funcional: una alerta del sketch dispara una verificación más precisa antes de bloquear (por los falsos positivos).

Pista (leela solo si te trabás): solo te interesan los que hacen mucho ruido. El sketch puede exagerar un poco a los chicos, pero nunca achica a los grandes: si alguien supera el umbral, seguro aparece.

Bonus (opcional)

- Agregá ventanas de tiempo: reiniciá el sketch cada minuto, o usá dos sketches alternados.
- Contá puertos distintos por origen con un HyperLogLog por candidato (un escaneo toca muchos puertos distintos).

Cierre: Detectar "heavy hitters" con sketches es una técnica estándar en firewalls, routers y sistemas de detección de intrusiones.

Ejercicio 477: Despacho eléctrico al menor costo

Nivel 10: Staff / Experto · Temática: Clima y energía · Dificultad 96/100

Situación: El operador del sistema eléctrico decide cada hora cuánto genera cada central para cubrir la demanda al menor costo. Sin más restricciones alcanzaría con encender las más baratas primero (orden de mérito), pero hay un tope de emisiones por hora que obliga a usar centrales más limpias aunque sean más caras. Es un problema de programación lineal, que se resuelve con el simplex sobre su dual.

Datos de entrada

Formato: primera línea: demanda (MW) y tope de emisiones (t por hora); después "central costo_por_MWh capacidad_MW emisiones_t_por_MWh".

Primal: minimizar suma costo_i x_i sujeto a suma x_i ≥ demanda, x_i ≤ capacidad_i, suma emisión_i x_i ≤ tope, x ≥ 0. Dual (como en el ejercicio del alimento balanceado): variables y (demanda), z_i (capacidades) y w (emisiones); maximizar demanda × y – suma capacidad_i z_i – tope × w, con y – z_i – emisión_i × w ≤ costo_i.

Ejemplo

```
900 100
hidro 20 400 0
nuclear 35 300 0
ciclo_combinado 60 500 0.4
térmica_vieja 45 400 1.1
```

Restricciones: hasta 10³ centrales; se resuelve cada 5 minutos.

Tu programa debe

1. Armar el dual, que arranca factible en el origen porque los costos son positivos.
2. Aplicar el simplex con fracciones y la regla de Bland.
3. Leer el despacho de cada central de los costos reducidos de las holguras, y los precios sombra de las variables del dual: y es el precio marginal de la energía y w el costo de la restricción ambiental.
4. Mostrar el despacho, el costo, las emisiones y, para comparar, el orden de mérito sin tope de emisiones.

Salida esperada

```
Despacho óptimo: hidro 400.0 MW, nuclear 300.0 MW, ciclo_combinado 171.4 MW, térmica_vieja 28.6 MW
Costo total: $30071.43 por hora | precio marginal de la energía: $68.57 por MWh
Emisiones: 100.0 t de 100 | precio sombra del límite de emisiones: $21.43 por t
Orden de mérito sin límite de emisiones: hidro 400, nuclear 300, térmica_vieja 200,
ciclo_combinado 0 (costo $27500)
```

Reglas

- Complejidad: el simplex es muy rápido para estos tamaños.
- El precio marginal (y) es cuánto costaría 1 MW más de demanda: es el precio que se paga en muchos mercados eléctricos.
- No funcional: seguridad de suministro: si el problema no tiene solución (no alcanza la capacidad limpia), el sistema lo informa para activar reservas, nunca despacha de menos en silencio.
- No funcional: el despacho se publica con los precios sombra, para que sea auditable por el regulador.

Pista (leela solo si te trabás): sin el tope, la solución es obvia: de la más barata a la más cara. El tope de emisiones cambia la respuesta porque encarece implícitamente a las centrales sucias: ese sobre costo implícito es justamente el precio sombra de la restricción.

Bonus (opcional)

- Agregá una restricción de transporte: el ciclo combinado está detrás de una línea que soporta como máximo 300 MW.
- Resolvé las 24 horas del día con demandas distintas y una restricción de rampa (cuánto puede cambiar cada central entre horas).

Cierre: El despacho económico con programación lineal es la operación diaria de los sistemas eléctricos del mundo.

Ejercicio 478: Cien dados sobre la mesa

Nivel 10: Staff / Experto · Temática: Videojuegos · Dificultad 96/100

Situación: Un juego de rol calcula la probabilidad de que la suma de muchos dados supere un valor (por ejemplo, 100 dados para un hechizo épico). La distribución de la suma es la potencia n -ésima del polinomio de un dado; con la FFT y exponenciación rápida se calcula en unos pocos productos. Como control, se compara con el cálculo exacto y con la aproximación normal.

Datos de entrada

Formato: primera línea: cantidad de dados y caras; segunda: valores umbral.

Polinomio de un dado: $(x + x^2 + \dots + x^{\text{caras}}) / \text{caras}$ (las probabilidades como coeficientes).

Ejemplo

```
100 6
350 400 300
```

Restricciones: hasta 10^4 dados de hasta 100 caras.

Tu programa debe

1. Elevar el polinomio de un dado a la n con exponenciación rápida, multiplicando con FFT (y truncando valores negativos por redondeo a 0).
2. Sumar las probabilidades desde el umbral para obtener $P(\text{suma} \geq \text{umbral})$.
3. Calcular el valor exacto con programación dinámica de enteros (formas / caras^n) y la aproximación normal con corrección de continuidad.
4. Mostrar la cantidad de productos, la suma más probable y las tres probabilidades con 6 decimales.

Salida esperada

```
Productos por FFT: 9 | suma más probable: 350
P(suma ≥ 350): FFT 0.511661 | exacta 0.511661 | aproximación normal 0.511678
P(suma ≥ 400): FFT 0.001823 | exacta 0.001823 | aproximación normal 0.001875
P(suma ≥ 300): FFT 0.998494 | exacta 0.998494 | aproximación normal 0.998447
```

Reglas

- Complejidad: $O(m \log m \times \log n)$, con $m = n \times \text{caras}$ el largo del polinomio final.
- Media = $n (\text{caras} + 1) / 2$; varianza = $n (\text{caras}^2 - 1) / 12$.
- No funcional: precisión: las probabilidades muy chicas (colas) pueden quedar dominadas por el error de redondeo de la FFT; por eso se compara con el valor exacto.
- No funcional: el juego muestra la probabilidad redondeada al jugador; el cálculo exacto se usa en las pruebas automáticas.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): tirar dos dados y sumar es multiplicar sus polinomios. Tirar 100 es elevar a la 100, y eso se hace con 7 cuadrados y algunas multiplicaciones, cada una rápida gracias a la FFT.

Bonus (opcional)

- Calculá la distribución de "tirar 4 dados y quedarse con los 3 mejores" (no es una potencia: hace falta otra técnica).
- Usá la NTT módulo un primo para obtener la cantidad exacta de formas módulo ese primo.

Cierre: La FFT aplicada a probabilidades se usa en juegos, en riesgo de seguros (suma de siniestros) y en estadística.

Ejercicio 479: Comprimir partituras

Nivel 10: Staff / Experto · Temática: Música · Dificultad 96/100

Situación: Una biblioteca digital de partituras quiere guardar millones de melodías ocupando poco espacio. Las melodías tienen dos tipos de redundancia: algunas notas son mucho más frecuentes (lo aprovecha Huffman) y hay frases que se repiten (lo aprovecha LZ77). El equipo compara ambos métodos sobre una melodía tradicional para decidir cuál usar.

Datos de entrada

Formato: primera línea: ventana y largo máximo de LZ77; segunda: la melodía (notas separadas por espacios).

Para LZ77, cada nota distinta se reemplaza por una letra (A, B, C... en orden alfabético de las notas); cada tripleta usa $\lceil \log_2(\text{ventana} + 1) \rceil + \lceil \log_2(\text{largo} + 1) \rceil + \lceil \log_2(\text{notas distintas}) \rceil$ bits. Huffman con el desempate del ejercicio de la flota (hojas en orden alfabético).

Ejemplo

```
16 8
Do4 Re4 Mi4 Do4 Do4 Re4 Mi4 Do4 Mi4 Fa4 Sol4 Mi4 Fa4 Sol4 Sol4 La4 Sol4 Fa4 Mi4 Do4 Sol4 La4 Sol4
Fa4 Mi4 Do4 Do4 Sol3 Do4 Do4 Sol3 Do4
```

Restricciones: partituras de hasta 10^5 notas.

Tu programa debe

1. Calcular el tamaño con un código fijo: cantidad de notas $\times \lceil \log_2(\text{notas distintas}) \rceil$.
2. Construir el código de Huffman sobre las notas y calcular el total de bits.
3. Aplicar LZ77 a la secuencia de notas y calcular los bits de las tripletas.
4. Mostrar los tres tamaños, los códigos de Huffman y el mejor método.

Salida esperada

```
Notas: 32 (7 distintas) | código fijo: 96 bits
Huffman: 84 bits | códigos: Do4=10, Fa4=011, La4=1100, Mi4=111, Re4=1101, Sol3=010, Sol4=00
LZ77: 14 tripletas  $\times$  12 bits = 168 bits
Mejor: Huffman
```

Reglas

- Complejidad: $O(n \log \sigma)$ para Huffman y $O(n \times \text{ventana} \times \text{largo})$ para LZ77.
- No funcional: los tamaños no incluyen la tabla de Huffman ni los parámetros; en melodías cortas, esos encabezados pueden pesar.
- No funcional: el formato elegido tiene que decodificar sin pérdida: se verifica decodificando.
- Los compresores reales combinan ambos (LZ77 primero y Huffman sobre sus tripletas), como DEFLATE.

Pista (leela solo si te trabás): Huffman mira cuántas veces aparece cada símbolo, sin importar dónde; LZ77 mira qué se repite en secuencia, sin importar cuán frecuente es cada símbolo. Según la melodía, gana uno u otro.

Bonus (opcional)

- Aplicá Huffman sobre las tripletas de LZ77 (como DEFLATE) y compará.
- Codificá las notas como intervalos (diferencias) antes de comprimir, como en el ejercicio de la melodía transpuesta.

Cierre: Elegir el método de compresión según la estructura de los datos es una decisión de ingeniería clave en formatos de audio, imágenes y archivos.

Ejercicio 480: Prueba de trabajo de juguete

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 96/100

Situación: En un curso de sistemas financieros, los alumnos implementan una "prueba de trabajo" de juguete para entender cómo algunas cadenas de bloques hacen costoso reescribir la historia: para agregar un bloque hay que encontrar un número (nonce) que haga que el hash del bloque quede por debajo de un objetivo. Cada bit de dificultad duplica el trabajo esperado, y cambiar un bloque viejo obliga a rehacer el trabajo de todos los siguientes.

Datos de entrada

Formato: primera línea: cantidad de bloques; después los bloques; última línea: dificultades a probar (en bits).

Hash del bloque = $H(\text{hash_anterior} + "|" + \text{bloque} + "|" + \text{nonce})$, con hash inicial "0" y nonce desde 0; se acepta si el hash es menor que $2^{(32 - \text{dificultad})}$. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR } \text{código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$. (Solo didáctico: con 32 bits, cualquier computadora resuelve esto al instante.)

Ejemplo

```
3
bloque1: ana paga 500 a beto
bloque2: beto paga 200 a caro
bloque3: caro paga 50 a dani
8 12 16
```

Restricciones: didáctico: dificultades de hasta 24 bits.

Tu programa debe

1. Para cada bloque, probar nonces 0, 1, 2... hasta encontrar un hash menor que el objetivo.
2. Encadenar: el hash encontrado es el hash anterior del bloque siguiente.
3. Contar los intentos totales y compararlos con el esperado (bloques $\times 2^{\text{dificultad}}$).
4. Mostrar, para cada dificultad, los nonces, los intentos y el último hash.

Salida esperada

```
Dificultad 8 bits: nonce 16, nonce 18, nonce 4 | intentos totales 41 (esperados 768) | último hash
13374570
Dificultad 12 bits: nonce 1598, nonce 11632, nonce 214 | intentos totales 13447 (esperados 12288)
| último hash 844220
Dificultad 16 bits: nonce 17206, nonce 11783, nonce 116889 | intentos totales 145881 (esperados
196608) | último hash 17673
```

Reglas

- Complejidad: $2^{\text{dificultad}}$ intentos esperados por bloque; verificar un bloque cuesta un solo hash.
- La asimetría es la clave: encontrar el nonce es caro, verificarlo es barato.
- No funcional: este ejercicio es didáctico; no sirve como mecanismo de seguridad real (hash de 32 bits no criptográfico).
- No funcional: el costo energético de la prueba de trabajo real es enorme; muchos sistemas usan alternativas (bonus).

Pista (leela solo si te trabás): un hash menor que $2^{(32 - d)}$ es un hash cuyos d primeros bits son 0. Como el hash se comporta como algo aleatorio, cada intento tiene probabilidad $1 / 2^d$ de lograrlo.

Bonus (opcional)

- Ajusta la dificultad automáticamente para que cada bloque tarde, en promedio, una cantidad fija de intentos.
- Modifica el bloque 1 y mide cuánto trabajo hace falta para rehacer la cadena completa.

Cierre: La prueba de trabajo es el mecanismo de consenso original de Bitcoin; entenderla ayuda a evaluar críticamente las tecnologías de cadenas de bloques.

Ejercicio 481: Turistas únicos por región

Nivel 10: Staff / Experto · Temática: Turismo · Dificultad 97/100

Situación: El ministerio de turismo cuenta turistas únicos por región con HyperLogLog, porque recibe cientos de millones de registros (tarjetas, reservas, conexiones). También quiere el total del país y cuántos visitaron dos regiones. La gran ventaja de HyperLogLog es que dos resúmenes se fusionan tomando el máximo registro a registro, sin volver a leer los datos.

Datos de entrada

Formato: primera línea: p y semilla; después "región eventos turistas_posibles desplazamiento". Cada evento: x = siguiente(x), turista = "turista" + (desplazamiento + x resto turistas_posibles) (el desplazamiento hace que las regiones compartan una parte de los turistas). Mismo HyperLogLog que en el ejercicio de la tienda, con $\alpha = 0,7213 / (1 + 1,079 / m)$ para $m = 256$.

Ejemplo

```
8 2026
Norte 30000 20000 0
Sur 25000 18000 12000
```

Restricciones: hasta 10^9 eventos por región; 24 regiones.

Tu programa debe

1. Construir un HyperLogLog por región con los eventos generados.
2. Estimar los turistas únicos de cada región.
3. Fusionar los dos resúmenes con el máximo registro a registro y estimar la unión.
4. Estimar la intersección con inclusión-exclusión: $|A| + |B| - |A \cup B|$, y mostrar todo junto a los valores reales.

Salida esperada

```
Norte: estimado 16147 | real 15508
Sur: estimado 12688 | real 13526
Unión (máximo registro a registro): estimado 26104 | real 24366 | error 7.13 %
Turistas que visitaron ambas (inclusión-exclusión): estimado 2731 | real 4668
```

Reglas

- Complejidad: $O(m)$ para fusionar, sin importar cuántos eventos hubo.
- La fusión es exacta: el HyperLogLog de la unión es idéntico al que se obtendría procesando todos los eventos juntos.
- No funcional: privacidad: los registros de HyperLogLog no permiten recuperar qué turistas se contaron.
- No funcional: la intersección por inclusión-exclusión tiene un error relativo grande cuando es chica comparada con la unión; se informa con su margen.

Pista (leela solo si te trabás): el registro j de la unión es el máximo p visto por cualquier elemento que cayó en j, venga de donde venga: por eso alcanza con tomar el máximo de los dos registros.

Bonus (opcional)

- Calculá la unión de las 24 regiones y el error relativo en función de p.
- Estimá la intersección con MinHash (ejercicio de las reseñas) y compará la precisión.

Cierre: Los sketches fusionables permiten calcular métricas globales en sistemas distribuidos sin mover los datos crudos.

Ejercicio 482: Choferes para cada servicio de larga distancia

Nivel 10: Staff / Experto · Temática: Transporte · Dificultad 97/100

Situación: Una empresa de micros de larga distancia asigna un chofer a cada servicio del día. Cada chofer tarda distinto en cada ruta (por su base de origen y su experiencia), y se busca minimizar las horas totales. Ramificación y poda asigna ruta por ruta, calculando una cota inferior optimista del costo restante para descartar ramas que no pueden mejorar la mejor asignación encontrada.

Datos de entrada

Formato: primera línea: los choferes; después "ruta horas_con_cada_chofer".

Ejemplo

```
Ramos Díaz Luna Soto Vega  
BA-Córdoba 9 7 8 12 10  
BA-Rosario 4 6 5 3 7  
BA-Mendoza 14 11 13 12 15  
BA-MarDelPlata 6 8 5 9 4  
BA-Tucumán 16 18 14 15 17
```

Restricciones: hasta 20 choferes (con el algoritmo húngaro se llega a miles).

Tu programa debe

1. Asignar las rutas en orden; en cada nivel, probar los choferes libres de menor a mayor costo.
2. Cota inferior de un nodo: costo acumulado + para cada ruta restante, el mínimo entre los choferes todavía libres (ignorando que dos rutas podrían querer al mismo).
3. Podar si la cota es mayor o igual que la mejor solución completa encontrada.
4. Mostrar la asignación, el costo, los nodos explorados frente a las permutaciones y la verificación por fuerza bruta.

Salida esperada

```
BA-Córdoba: Díaz (7 h)  
BA-Rosario: Ramos (4 h)  
BA-Mendoza: Soto (12 h)  
BA-MarDelPlata: Vega (4 h)  
BA-Tucumán: Luna (14 h)  
Costo total: 41 horas | nodos explorados: 25 (permutaciones: 120) | fuerza bruta: 41
```

Reglas

- Complejidad: exponencial en el peor caso; con buena cota y buen orden, se exploran muy pocos nodos.
- La cota es válida porque relaja una restricción: nunca es mayor que el costo real de completar.
- No funcional: reglas laborales (descanso mínimo, horas máximas) son restricciones duras que se verifican antes de aceptar una asignación.
- No funcional: si el cálculo supera un tiempo límite, se entrega la mejor asignación encontrada, indicándolo.

Pista (leela solo si te trabás): explorar primero la opción más barata encuentra enseguida una buena solución; desde ahí, cualquier rama cuya cota optimista ya iguale o supere ese costo se descarta sin mirarla.

Bonus (opcional)

- Implementá el algoritmo húngaro ($O(n^3)$) y compará con 20 choferes.
- Mejorá la cota usando la reducción de filas y columnas de la matriz.

Cierre: Ramificación y poda es la base de la planificación de tripulaciones en aerolíneas, micros y trenes.

Ejercicio 483: La opción que se puede ejercer antes

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 97/100

Situación: Una plataforma de derivados necesita valuar opciones de venta americanas, que se pueden ejercer en cualquier momento antes del vencimiento. Black-Scholes no sirve (solo vale para ejercicio al vencimiento). El árbol binomial modela el precio subiendo o bajando en cada paso y, recorriendo hacia atrás, en cada nodo compara continuar con ejercer ya.

Datos de entrada

Formato: una línea: precio actual, precio de ejercicio, tasa, volatilidad, plazo en años y cantidad de pasos N.

Modelo de Cox-Ross-Rubinstein: $\Delta t = T / N$; $u = e^{(\sigma \sqrt{\Delta t})}$; $d = 1/u$; $p = (e^{(r \Delta t)} - d) / (u - d)$. Precio en el nodo (paso i, j subidas) = $S u^j d^{(i-j)}$.

Ejemplo

```
100 100 0.06 0.30 1 200
```

Restricciones: hasta 10^4 pasos ($O(N^2)$ nodos).

Tu programa debe

1. Calcular los pagos al vencimiento: $\max(K - \text{precio}, 0)$ en cada nodo final.
2. Recorrer hacia atrás: el valor de continuar es $e^{(-r \Delta t)} \times (p \times \text{valor_arriba} + (1 - p) \times \text{valor_abajo})$.
3. Para la europea, el valor del nodo es el de continuar; para la americana, el máximo entre continuar y ejercer ya (contando los nodos donde ejercer es estrictamente mejor).
4. Mostrar los parámetros, la europea comparada con Black-Scholes, la americana, la prima por ejercicio anticipado y los nodos de ejercicio anticipado.

Salida esperada

```
Parámetros del árbol: u = 1.021440, d = 0.979010, p = 0.501768
Venta europea (árbol): 8.8788 | Black-Scholes: 8.8935
Venta americana (árbol): 9.5245 | prima por ejercicio anticipado: 0.6457
Nodos donde conviene ejercer antes: 8908
```

Reglas

- Complejidad: $O(N^2)$ en tiempo y $O(N)$ en memoria (se guarda solo un nivel).
- Con N grande, la europea del árbol converge al valor de Black-Scholes (sirve como verificación).
- No funcional: estabilidad: p tiene que estar entre 0 y 1; si no, el paso de tiempo es demasiado grande y hay que aumentar N.
- No funcional: el valor mostrado es teórico y depende de los supuestos del modelo.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): en cada nodo del árbol preguntás: "¿me conviene ejercer ahora o esperar?". Esperar vale el promedio descontado de los dos futuros posibles; ejercer vale lo que cobro hoy.

Bonus (opcional)

- Graficá la frontera de ejercicio: para cada paso, el precio por debajo del cual conviene ejercer.
- Valuá una opción de compra americana sin dividendos y verificá que nunca conviene ejercerla antes.

Cierre: Los árboles binomiales son el método estándar para valuar opciones con ejercicio anticipado y muchos derivados exóticos.

Ejercicio 484: Tabla de posiciones replicada con particiones

Nivel 10: Staff / Experto · Temática: Videojuegos · Dificultad 97/100

Situación: Un juego online guarda la tabla de posiciones replicada en cinco servidores. Un corte de red parte los servidores en dos grupos que no se ven. Con quórums de mayoría ($W = R = 3$ de 5), solo el grupo mayoritario puede escribir y leer; el minoritario rechaza las operaciones en lugar de aceptar datos que podrían contradecirse. Al resolverse la partición, las réplicas se sincronizan.

Datos de entrada

Formato: primera línea: N, W y R; después operaciones "escribe cliente jugador puntaje grupo réplicas_visibles...", "lee cliente jugador grupo réplicas_visibles..." y "particion_resuelta".

Una escritura confirmada recibe la versión siguiente y se guarda en las primeras W réplicas visibles; una lectura consulta las primeras R visibles y devuelve la de mayor versión. Al resolverse la partición, cada réplica adopta, para cada jugador, el valor de mayor versión.

Ejemplo

```
5 3 3
escribe A ana 120 grupo 1 2 3
escribe B beto 90 grupo 4 5
lee A ana grupo 1 2 3
lee B ana grupo 4 5
particion_resuelta
escribe B beto 95 grupo 1 2 3 4 5
lee B beto grupo 3 4 5
```

Restricciones: hasta 10^6 jugadores y 10^4 escrituras por segundo.

Tu programa debe

1. Llevar en cada réplica el valor y la versión de cada jugador.
2. Rechazar escrituras y lecturas del lado de la partición que no alcanza el quórum.
3. En el lado mayoritario, operar normalmente.
4. Al resolverse la partición, sincronizar y seguir operando. Mostrar cada paso.

Salida esperada

```
Cliente A escribe ana=120: confirmada en réplicas 1, 2, 3 (versión 1)
Cliente B escribe beto=90 viendo 2 réplicas: RECHAZADA (quórum 3)
Cliente A lee ana: 120 (versión 1)
Cliente B lee ana viendo 2 réplicas: RECHAZADA (quórum 3)
Partición resuelta: anti-entropía sincroniza todas las réplicas
Cliente B escribe beto=95: confirmada en réplicas 1, 2, 3 (versión 2)
Cliente B lee beto: 95 (versión 2)
```

Reglas

- Complejidad: $O(N)$ por operación.
- Con mayorías ($W, R > N/2$), dos grupos disjuntos no pueden tener ambos quórum: nunca hay dos "verdades" distintas.
- No funcional: teorema CAP: durante una partición, este diseño elige consistencia sobre disponibilidad (el lado minoritario no responde).
- No funcional: el cliente que recibe un rechazo reintenta con espera exponencial, sin perder su dato.

Pista (leela solo si te trabás): una partición que deja 3 réplicas de un lado y 2 del otro: solo el lado de 3 puede formar una mayoría. Es la misma idea de los quórums de la historia clínica, llevada a un corte de red.

Bonus (opcional)

- Cambiá a $W = 1$, $R = 1$ (disponibilidad máxima) y mostrá cómo aparecen dos puntajes distintos para el mismo jugador.
- Agregá "hinted handoff": las escrituras del lado minoritario se guardan como pendientes y se aplican al resolver la partición.

Cierre: Elegir entre consistencia y disponibilidad durante particiones es la decisión central del diseño de bases de datos distribuidas.

Ejercicio 485: Productos dados de baja en memoria mínima

Nivel 10: Staff / Experto · Temática: E-commerce · Dificultad 97/100

Situación: Un e-commerce usa un filtro de Bloom en sus servidores de borde para saber rápido si un código de producto está activo. Pero los productos se dan de baja todo el tiempo, y en un filtro de bits no se puede borrar (apagar un bit podría afectar a otro producto). El filtro de Bloom con contadores reemplaza cada bit por un contador chico: dar de alta suma, dar de baja resta.

Datos de entrada

Formato: primera línea: cantidad de contadores y de funciones; después operaciones "alta producto", "baja producto" y "consulta producto" (en orden). Posiciones con doble hashing, como en el ejercicio de los sitios peligrosos. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
32 3
alta yerba_A12 alta mate_B7 alta termo_C3 alta bombilla_D1 alta yerba_Z9
baja mate_B7
consulta mate_B7 consulta termo_C3 consulta yerba_A12 consulta pava_E5 consulta mate_B7
```

Restricciones: hasta 10^8 productos; contadores de 4 bits.

Tu programa debe

1. Al dar de alta, sumar 1 en los k contadores del producto.
2. Al dar de baja, restar 1 en los mismos k contadores.
3. Al consultar, responder "probablemente activo" si los k contadores son mayores que 0.
4. Mostrar los contadores tras cada baja, cada consulta, y el estado final.

Salida esperada

```
baja mate_B7: contadores 0 1 1
consulta mate_B7: no está (o fue dado de baja)
consulta termo_C3: probablemente activo
consulta yerba_A12: probablemente activo
consulta pava_E5: no está (o fue dado de baja)
consulta mate_B7: no está (o fue dado de baja)
Contadores no nulos: 10 de 32 | máximo contador: 2 (con 4 bits por contador alcanza hasta 15)
```

Reglas

- Complejidad: $O(k)$ por operación.
- Solo se puede dar de baja algo que se dio de alta: restar un producto inexistente rompe el filtro y puede generar falsos negativos.
- No funcional: memoria: con contadores de 4 bits, el filtro ocupa 4 veces más que uno de bits; es el precio de poder borrar.
- No funcional: un contador que llega al máximo se deja fijo (saturado) y el sistema lo registra.

Pista (leela solo si te trabás): un contador cuenta cuántos productos "pisaron" esa posición. Al dar de baja uno, los otros siguen sumando, así que su información no se pierde.

Bonus (opcional)

- Mostrá cómo una baja de un producto nunca dado de alta produce un falso negativo.
- Compará con un filtro de cuco (cuckoo filter), que también permite borrar y ocupa menos.

Cierre: Los filtros de Bloom con contadores se usan en cachés y routers donde el conjunto cambia constantemente.

Ejercicio 486: Sensores que mienten

Nivel 10: Staff / Experto · Temática: Ciencia · Dificultad 97/100

Situación: Una red de sensores de temperatura de un laboratorio tiene que acordar un valor aunque algunos sensores fallen de forma arbitraria ("bizantina"): no solo se apagan, sino que pueden reportar valores absurdos o maliciosos. Con n sensores y hasta f fallidos, si $n \geq 3f + 1$, descartar los f valores más altos y los f más bajos y promediar el resto da un valor dentro del rango de los sensores correctos.

Datos de entrada

Formato: pares de líneas: f (máximo de sensores con fallas) y las lecturas "sensor valor ...".

Ejemplo

```
1
s1 21.3 s2 21.1 s3 21.4 s4 21.2 s5 85.0
2
t1 30.1 t2 29.8 t3 30.3 t4 -40.0 t5 30.0 t6 99.9 t7 30.2
```

Restricciones: hasta 10^3 sensores por grupo.

Tu programa debe

1. Calcular el promedio simple (sensible a valores absurdos) y la mediana.
2. Verificar si $n \geq 3f + 1$.
3. Si se cumple, ordenar, descartar los f menores y f mayores y promediar el resto (promedio recortado).
4. Mostrar las tres medidas (2 decimales) y si la cantidad de sensores alcanza para tolerar las fallas.

Salida esperada

```
5 sensores, hasta 1 con fallas: promedio simple 34.00 | mediana 21.30 | promedio recortado 21.30
  (descarta 1 de cada extremo) |  $n \geq 3f + 1$ : se tolera
7 sensores, hasta 2 con fallas: promedio simple 30.04 | mediana 30.10 | promedio recortado 30.10
  (descarta 2 de cada extremo) |  $n \geq 3f + 1$ : se tolera
```

Reglas

- Complejidad: $O(n \log n)$ por el ordenamiento.
- Por qué funciona: los f valores descartados de cada extremo incluyen a todos los fallidos que se fueron a los extremos; los que quedan en el medio están acotados por valores de sensores correctos.
- No funcional: tolerancia a fallas: el sistema se dimensiona con $n \geq 3f + 1$ según cuántas fallas simultáneas se quieren tolerar.
- No funcional: los sensores descartados con frecuencia se reportan para mantenimiento.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): un sensor malicioso puede "tirar" el promedio todo lo que quiera, pero no puede mover el valor del medio si hay suficientes sensores honestos a cada lado.

Bonus (opcional)

- Simulá varias rondas de "acuerdo aproximado": cada sensor correcto recibe los valores de todos y aplica el promedio recortado; mostrará cómo convergen.
- Estudiá por qué con $n = 3f$ no alcanza (un contraejemplo con 3 sensores y 1 fallido).

Cierre: La tolerancia a fallas bizantinas es la base de sistemas críticos: aviónica, redes de sensores y protocolos de consenso distribuidos.

Ejercicio 487: Intercambio de claves de juguete

Nivel 10: Staff / Experto · Temática: Seguridad informática · Dificultad 98/100

Situación: En una clase de seguridad informática, los alumnos implementan el intercambio de claves de Diffie-Hellman con números chicos: dos personas acuerdan una clave secreta común hablando por un canal que todos escuchan, sin haberse visto nunca. Ana y Beto publican g^a y g^b ; cada uno combina lo del otro con su secreto y ambos llegan a $g^{(ab)}$. El ejercicio también muestra por qué los números tienen que ser enormes.

Datos de entrada

Formato: una línea: el primo p , el generador g , el secreto de Ana a y el secreto de Beto b .

Ejemplo

```
2027 2 385 1203
```

Restricciones: solo didáctico: en producción, p tiene 2.048 bits o más, o se usan curvas elípticas, y siempre con bibliotecas auditadas.

Tu programa debe

1. Calcular $A = g^a \text{ resto } p$ y $B = g^b \text{ resto } p$ con exponenciación rápida (lo que se publica).
2. Calcular la clave de Ana, $B^a \text{ resto } p$, y la de Beto, $A^b \text{ resto } p$: son iguales porque $(g^b)^a = (g^a)^b$.
3. Mostrar que, con p chico, alguien que solo ve A puede probar exponentes 0, 1, 2... hasta encontrar uno que dé A (logaritmo discreto por fuerza bruta).
4. Mostrar los valores públicos, las claves y la cantidad de pasos.

Salida esperada

```
Ana publica  $A = 2^{385} \text{ resto } 2027 = 139$   
Beto publica  $B = 2^{1203} \text{ resto } 2027 = 1384$   
Clave de Ana:  $B^a \text{ resto } p = 868$  | clave de Beto:  $A^b \text{ resto } p = 868$  -> coinciden  
Con  $p$  tan chico, probar exponentes hasta obtener  $A$  lleva 386 pasos (encuentra  $a = 385$ ); con  $p$  de 2.048 bits sería inviable.
```

Reglas

- Complejidad: $O(\log \text{exponente})$ para calcular; la fuerza bruta es $O(p)$, que con 2.048 bits es impracticable.
- La seguridad depende de que calcular el logaritmo discreto sea difícil para números enormes.
- No funcional: nunca implementes criptografía propia para producción; este ejercicio es para entender el mecanismo.
- No funcional: Diffie-Hellman sin autenticación es vulnerable a que un intermediario se haga pasar por cada parte; los protocolos reales lo combinan con firmas (ver el ejercicio de los recibos).

Pista (leela solo si te trabás): elevar a una potencia módulo p es fácil en un sentido y muy difícil de deshacer. Ana y Beto aprovechan que el orden de los exponentes no importa: $(g^a)^b = (g^b)^a$.

Bonus (opcional)

- Reemplazá la fuerza bruta por el algoritmo "paso de bebé, paso de gigante" ($O(\sqrt{p})$) y compará los pasos.
- Agregá una tercera persona (Caro) y acordá una clave común entre los tres.

Cierre: Diffie-Hellman fue la primera forma pública de acordar claves sobre canales inseguros y está en el corazón de los protocolos seguros de internet.

Ejercicio 488: Riesgo de crédito con incumplimientos correlacionados

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 98/100

Situación: Un banco que presta a pymes sabe que los incumplimientos no son independientes: en una crisis, muchas pymes fallan juntas. Para medir el riesgo de la cartera, simula escenarios con Monte Carlo generando variables normales correlacionadas mediante la descomposición de Cholesky de la matriz de correlaciones. Cada pyme incumple si su variable cae por debajo de su umbral.

Datos de entrada

Formato: primera línea: simulaciones y semilla; después "pyme exposición umbral" (umbral de la normal: $-1,645 \approx 5\%$ de probabilidad de incumplir); luego la matriz de correlaciones.

Por simulación: z_1, z_2 de un par Box-Muller y z_3 del siguiente par (se descarta su segundo valor). Normales: $x = \text{siguiente}(x)$; $u_1 = (x + 0,5) / 2^{31}$; $x = \text{siguiente}(x)$; $u_2 = (x + 0,5) / 2^{31}$; $z_1 = \sqrt{-2 \ln u_1} \cos(2\pi u_2)$, $z_2 = \sqrt{-2 \ln u_1} \sin(2\pi u_2)$ ($x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$). Variables correlacionadas: $e = L \times z$, con L la matriz de Cholesky.

Ejemplo

```
20000 99
pyme_A 1000000 -1.645
pyme_B 800000 -1.282
pyme_C 1200000 -2.054
1 0.5 0.3
0.5 1 0.4
0.3 0.4 1
```

Restricciones: hasta 10^5 deudores y 10^6 simulaciones.

Tu programa debe

1. Calcular la matriz triangular L de Cholesky tal que $L \times L^T = \text{matriz de correlaciones}$.
2. En cada simulación, generar tres normales independientes y correlacionarlas con L .
3. Marcar los incumplimientos ($\text{variable} < \text{umbral}$) y sumar las exposiciones perdidas.
4. Mostrar L (4 decimales), las tasas simuladas, la pérdida esperada, la probabilidad de 2 o más incumplimientos y la pérdida en el percentil 99 (posición $[0,99 N]$ de las pérdidas ordenadas).

Salida esperada

```
Cholesky: 1.0000 | 0.5000 0.8660 | 0.3000 0.2887 0.9092
Tasa de incumplimiento simulada: pyme_A 4.89 %, pyme_B 10.28 %, pyme_C 2.07 %
Pérdida esperada: $155870 | P(2 o más incumplimientos): 2.69 % | pérdida al 99 %: $1800000
```

Reglas

- Complejidad: $O(n^3)$ para Cholesky y $O(N \times n^2)$ para la simulación.
- Cholesky solo existe si la matriz de correlaciones es definida positiva; si no, los datos son inconsistentes y hay que informarlo.
- No funcional: reproducibilidad: semilla, generador y orden de uso de las normales están especificados.
- No funcional: el modelo se valida comparando las tasas simuladas con las teóricas (5 %, 10 % y 2 %).
- Las pérdidas se redondean al peso.

Pista (leela solo si te trabás): si las normales fueran independientes, que fallen dos pymes a la vez sería rarísimo. La correlación hace que "los días malos" sean malos para varias a la vez, y eso engorda la cola de pérdidas.

Bonus (opcional)

- Compará la pérdida al 99 % con correlaciones 0 y con correlaciones 0,8.
- Agregá una tasa de recupero del 40 % sobre la exposición incumplida.

Cierre: Los modelos de incumplimiento correlacionado con Monte Carlo son la base del cálculo de capital por riesgo de crédito en bancos.

Ejercicio 489: Recorrido de las cosechadoras

Nivel 10: Staff / Experto · Temática: Agricultura · Dificultad 98/100

Situación: Una cooperativa manda su cosechadora desde el galpón a varios lotes y de vuelta, y el traslado por caminos rurales es lento y caro. Para pocos lotes quiere el recorrido óptimo exacto. Ramificación y poda construye recorridos parciales y descarta los que, incluso en el mejor caso, no pueden mejorar el mejor recorrido encontrado.

Datos de entrada

Formato: primera línea: el galpón y los lotes; después la matriz de distancias en km (simétrica).

Ejemplo

```
Galpón Lote_1 Lote_2 Lote_3 Lote_4 Lote_5 Lote_6
0 12 10 19 8 20 15
12 0 3 7 2 11 13
10 3 0 6 4 9 10
19 7 6 0 11 5 7
8 2 4 11 0 13 12
20 11 9 5 13 0 4
15 13 10 7 12 4 0
```

Restricciones: hasta 20 lotes (para más, heurísticas como el recocido simulado).

Tu programa debe

1. Partir del galpón y extender el recorrido probando primero el lote más cercano.
2. Cota inferior: distancia acumulada + la menor salida del lote actual + la menor salida de cada lote no visitado (cada uno tiene que ser abandonado por alguna arista).
3. Podar si la cota es mayor o igual que el mejor recorrido completo; al completar, sumar la vuelta al galpón.
4. Mostrar el recorrido óptimo, la distancia y los nodos explorados frente a los recorridos posibles.

Salida esperada

```
Recorrido óptimo: Galpón -> Lote_4 -> Lote_1 -> Lote_2 -> Lote_3 -> Lote_5 -> Lote_6 -> Galpón
Distancia: 43 km
Nodos explorados: 578 (recorridos posibles: 720)
```

Reglas

- Complejidad: exponencial en el peor caso; en la práctica, la poda reduce mucho la exploración.
- La cota es válida porque cada lote pendiente y el actual todavía tienen que "salir" por alguna arista, y ninguna puede ser más corta que su mínima.
- No funcional: garantía de optimalidad: a diferencia de las heurísticas, el resultado es el óptimo exacto (se puede verificar con Held-Karp del nivel 7).
- No funcional: un límite de tiempo devuelve la mejor solución encontrada si el problema es demasiado grande.

Pista (leela solo si te trabás): una buena solución temprana (ir siempre al más cercano) hace que la poda funcione desde el principio; una buena cota hace que se poden ramas enteras sin recorrerlas.

Bonus (opcional)

- Mejorá la cota con el árbol de expansión mínima de los lotes no visitados.
- Verificá el óptimo con la programación dinámica de Held-Karp.

Cierre: Ramificación y poda resuelve exactamente problemas de ruteo de tamaño mediano, y es el núcleo de los solvers de optimización exacta.

Ejercicio 490: Contratos de energía entre vecinos productores

Nivel 10: Staff / Experto · Temática: Clima y energía · Dificultad 98/100

Situación: En un barrio con paneles solares, algunos vecinos producen más energía de la que consumen y otros la necesitan. Una cooperativa arma contratos para repartir el excedente minimizando el costo de transporte por la red (que depende de la distancia y las pérdidas). Es el clásico problema de transporte, un programa lineal que se resuelve con el simplex sobre el dual.

Datos de entrada

Formato: secciones "productores" ("nombre kWh_disponibles"), "consumidores" ("nombre kWh_necesarios") y "costos" ("productor costo_hacia_cada_consumidor").

Primal: minimizar $\sum c_{ij} x_{ij}$ sujeto a $\sum_j x_{ij} \leq \text{oferta}_i$, $\sum_i x_{ij} \geq \text{demanda}_j$, $x \geq 0$. Dual: maximizar $\sum \text{demanda}_j v_j - \sum \text{oferta}_i u_i$ con $v_j - u_i \leq c_{ij}$.

Ejemplo

```
productores
casa_1 6
casa_2 4
escuela 10
consumidores
panadería 7
club 5
almacén 6
costos
casa_1 3 5 4
casa_2 2 4 6
escuela 6 2 3
```

Restricciones: hasta 10^3 productores y consumidores.

Tu programa debe

1. Armar el dual (una restricción por par productor-consumidor), que arranca factible en el origen.
2. Resolverlo con el simplex (fracciones y regla de Bland).
3. Leer los flujos de los costos reducidos de las holguras y los precios locales de los consumidores (v_j).
4. Mostrar los contratos con flujo positivo, el costo total, el sobrante y los precios locales.

Salida esperada

```
casa_1 -> panadería: 3.0 kWh
casa_1 -> almacén: 1.0 kWh
casa_2 -> panadería: 4.0 kWh
escuela -> club: 5.0 kWh
escuela -> almacén: 5.0 kWh
Costo total de transporte: 46.00 (pivoteos 5)
Sobrante: 2.0 kWh | precios locales de consumidores: panadería 3.00, club 3.00, almacén 4.00
```

Reglas

- Complejidad: el problema de transporte tiene algoritmos especializados más rápidos (bonus), pero el simplex general lo resuelve bien.
- El precio local de un consumidor es cuánto costaría abastecerle 1 kWh más: sirve para fijar tarifas justas por ubicación.
- No funcional: transparencia: los contratos y precios se publican a los vecinos con la explicación del cálculo.
- No funcional: si la oferta no alcanza la demanda, el problema no tiene solución y la cooperativa compra el faltante a la red general (se informa).

Pista (leela solo si te trabás): el dual asigna un "precio" a la energía en cada punto de la red: para que nadie quiera arbitrar, el precio en el consumidor no puede superar el precio en el productor más el costo de transporte.

Bonus (opcional)

- Resuelvelo con el método de la esquina noroeste más el método de los potenciales (MODI).
- Agregá capacidades máximas en las líneas entre casas.

Cierre: El problema de transporte es uno de los modelos de programación lineal más usados: logística, energía, cadenas de suministro.

Ejercicio 491: Índice comprimido del buscador de recetas

Nivel 10: Staff / Experto · Temática: Gastronomía · Dificultad 98/100

Situación: Un buscador de recetas guarda, para cada palabra, la lista ordenada de recetas que la contienen (índice invertido). Con millones de recetas, esas listas ocupan mucho. Como están ordenadas, se guardan las diferencias entre números consecutivos (que son chicas) con una codificación de largo variable: los números chicos ocupan 1 byte en lugar de 4.

Datos de entrada

Formato: una línea por palabra: "palabra ids_de_recetas_ordenados"; una línea "?" y dos palabras a intersecar.

Codificación de bytes variables: el número se parte en grupos de 7 bits, del más significativo al menos significativo; cada grupo va en un byte y el último byte lleva el bit 128 encendido (marca de fin). Los bytes se muestran en hexadecimal.

Ejemplo

```
dulce_de_leche 3 7 8 15 120 121 5000 5003
empanada 2 3 9 15 120 700 5003 9000
?
dulce_de_leche empanada
```

Restricciones: hasta 10^8 recetas y 10^6 palabras.

Tu programa debe

1. Calcular los huecos de cada lista: el primer id y después las diferencias entre ids consecutivos.
2. Codificar cada hueco con bytes variables.
3. Comparar el tamaño total con guardar cada id en 4 bytes.
4. Intersecar dos listas con dos punteros (avanzando el menor) y mostrar el resultado y las comparaciones.

Salida esperada

```
dulce_de_leche: huecos 3 4 1 7 105 1 4879 3 | 9 bytes (83 84 81 87 E9 81 26 8F 83)
empanada: huecos 2 1 6 6 105 580 4303 3997 | 11 bytes (82 81 86 86 E9 04 C4 21 CF 1F 9D)
Total: 20 bytes contra 64 con enteros de 4 bytes (31.3 %)
Recetas con "dulce_de_leche" y "empanada": 3 15 120 5003 (11 comparaciones)
```

Reglas

- Complejidad: $O(\text{largo})$ para codificar y $O(a + b)$ para intersecar.
- La intersección se puede hacer decodificando sobre la marcha, sin descomprimir las listas completas.
- No funcional: velocidad de consulta: la codificación por bytes se decodifica muy rápido (no requiere operar bit a bit).
- No funcional: el índice se reconstruye de noche; las recetas nuevas se agregan en un índice chico aparte que se fusiona después.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): los ids pueden ser enormes, pero en una lista ordenada y densa, la distancia entre uno y el siguiente suele ser chica. Guardar distancias en lugar de valores convierte números de 4 bytes en números de 1 byte.

Bonus (opcional)

- Agregá "saltos" (cada 128 ids, guardá el id absoluto y su posición) para intersecar listas muy desparejas más rápido.
- Compará con la codificación gamma de Elias, bit a bit.

Cierre: Los índices invertidos comprimidos son el corazón de los buscadores web, los motores de búsqueda de texto y las bases de datos de documentos.

Ejercicio 492: Turnos de fábrica que respetan a todos

Nivel 10: Staff / Experto · Temática: Recursos humanos · Dificultad 99/100

Situación: Una fábrica que trabaja las 24 horas arma los turnos semanales de un equipo: cada día necesita exactamente una persona a la mañana, una a la tarde y una a la noche; nadie puede pasar de un turno noche a uno mañana al día siguiente; y nadie trabaja más de 5 días (restricciones duras). Además, cada persona tiene pedidos (no hacer noche tal día, tener franco otro): restricciones blandas.

Datos de entrada

Formato: primera línea: el equipo; después "persona pedidos", donde "N0" significa "no quiero turno noche el día 0", "M5" no quiero mañana el día 5, "T2" no quiero tarde el día 2 y "F6" quiero franco el día 6 (días 0 = lunes a 6 = domingo); última línea: iteraciones, temperatura inicial, factor de enfriamiento y semilla.

Inicial: el día d , la persona $(d + k)$ resto personas hace el turno k de "MTN" y el resto tiene franco. Movimiento: $d = \text{siguiente}(x) \text{ resto } 7$, $e1 = \text{siguiente}(x) \text{ resto personas}$, $e2 = \text{siguiente}(x) \text{ resto personas}$; se intercambian los turnos de $e1$ y $e2$ ese día; $u = \text{siguiente}(x) / 2^{31}$. Penalización = $100 \times (\text{faltantes o sobrantes por turno y día} + \text{transiciones noche} \rightarrow \text{mañana} + \text{personas con más de 5 días}) + \text{pedidos no respetados}$. $x = (1103515245 \times x + 12345) \text{ resto } 2^{31}$.

Ejemplo

```
Ana Beto Caro Dani Eli
Ana N0 N1
Beto M5 M6
Caro T2
Dani F6
Eli F0 F1
3000 20 0.995 2026
```

Restricciones: hasta 100 personas, 4 semanas y 10^6 iteraciones.

Tu programa debe

1. Calcular la penalización del cronograma inicial.
2. Iterar: intercambiar los turnos de dos personas en un día (así la cobertura nunca se rompe), calcular Δ y aceptar con el criterio de Metropolis.
3. Guardar el mejor cronograma y enfriar la temperatura.
4. Mostrar la penalización inicial y la mejor (desglosadas) y el cronograma.

Salida esperada

```
Penalización inicial 2 (duras 0, blandas 2) -> mejor 0 (duras 0, blandas 0)
Ana   F T T M F M N
Beto  M M M N N F F
Caro  N F N T F N T
Dani  T N F F M F F
Eli   F F F F T T M
(días de lunes a domingo; M mañana, T tarde, N noche, F franco)
```

Reglas

- Complejidad: $O(\text{iteraciones} \times \text{personas} \times \text{días})$ con recálculo completo.
- El movimiento de intercambio preserva la cobertura: el recocido solo tiene que resolver las demás restricciones. Igual se la verifica en la penalización, por seguridad.
- No funcional: un cronograma con penalización dura mayor que 0 no se publica: se informa que no se encontró uno válido.
- No funcional: reproducibilidad: misma semilla y aritmética de doble precisión, mismo resultado.

Pista (leela solo si te trabás): con penalizaciones de 100 para lo duro y 1 para lo blando, el recocido primero "arregla" lo obligatorio y después usa la libertad que queda para complacer pedidos.

Bonus (opcional)

- Agregá equidad: la diferencia entre la persona con más y con menos noches no puede superar 1.
- Compará el resultado con varias semillas y quedate con el mejor (reinicios).

Cierre: Los problemas de turnos con restricciones duras y blandas se resuelven en fábricas, hospitales y call centers con metaheurísticas y programación con restricciones.

Ejercicio 493: El feed que llega desordenado

Nivel 10: Staff / Experto · Temática: Redes sociales · Dificultad 99/100

Situación: Una red social ordena el feed por hora de publicación, pero los relojes de los dispositivos están desfasados: una respuesta puede tener una hora anterior a la publicación que responde. Los relojes lógicos híbridos (HLC) combinan la hora física con un contador lógico: se parecen a la hora real, pero respetan la causalidad (si algo se publicó después de ver otra cosa, queda después).

Datos de entrada

Formato: eventos en orden de procesamiento: "dispositivo reloj_físico publica|recibe publicación".

HLC = (l, c). Evento local o publicación: $l' = \max(l, \text{físico})$; $c' = c + 1$ si $l' = l$, si no 0. Recepción de un mensaje con (lm, cm): $l' = \max(l, lm, \text{físico})$; $c' = \max(c, cm) + 1$ si $l' = l = lm$; $c + 1$ si $l' = l$; $cm + 1$ si $l' = lm$; 0 si no. Las publicaciones llevan el HLC con que se crearon.

Ejemplo

```
celular 1000 publica p1
notebook 998 recibe p1
notebook 999 publica p2
tablet 1005 publica p3
celular 1001 recibe p2
tablet 1002 recibe p2
celular 1001 publica p4
```

Restricciones: hasta 10^9 dispositivos; desfasajes de hasta minutos.

Tu programa debe

1. Mantener el HLC (l, c) de cada dispositivo, empezando en (0, 0).
2. Actualizarlo en cada publicación y recepción con las reglas dadas.
3. Ordenar las publicaciones por (l, c) y, para comparar, por el reloj físico del dispositivo que las publicó.
4. Mostrar el HLC de cada evento y los dos órdenes.

Salida esperada

```
celular (reloj físico 1000) publica p1: HLC (1000, 0)
notebook (reloj físico 998) recibe p1: HLC (1000, 1)
notebook (reloj físico 999) publica p2: HLC (1000, 2)
tablet (reloj físico 1005) publica p3: HLC (1005, 0)
celular (reloj físico 1001) recibe p2: HLC (1001, 0)
tablet (reloj físico 1002) recibe p2: HLC (1005, 1)
celular (reloj físico 1001) publica p4: HLC (1001, 1)
Orden del feed por HLC: p1 < p2 < p4 < p3
Orden por reloj físico del dispositivo: p2 < p1 < p4 < p3
```

Reglas

- Complejidad: $O(1)$ por evento; el HLC ocupa dos enteros (como una marca de tiempo común).
- Propiedad: si a causó b, $HLC(a) < HLC(b)$; y l nunca se aleja de la hora física más que el desfasaje máximo.
- No funcional: compatibilidad: l se puede mostrar al usuario como hora aproximada, a diferencia de un reloj de Lamport puro.
- No funcional: un dispositivo con el reloj muy adelantado "arrastra" a los demás; el sistema rechaza mensajes con l demasiado lejos de su hora (bonus).

Pista (leela solo si te trabás): el reloj híbrido toma la hora física cuando puede y, si la hora "retrocede" (porque el otro dispositivo va adelantado), mantiene la mayor y desempata con el contador.

Bonus (opcional)

- Rechazá mensajes cuyo I supere la hora local en más de 60 unidades y registrá el dispositivo sospechoso.
- Compará con relojes vectoriales: qué pares concurrentes ordena el HLC de forma arbitraria.

Cierre: Los relojes lógicos híbridos se usan en bases de datos distribuidas modernas para ordenar transacciones sin relojes atómicos.

Ejercicio 494: Telemetría de un país entero en poca memoria

Nivel 10: Staff / Experto · Temática: Telecomunicaciones · Dificultad 99/100

Situación: Una operadora recibe la telemetría de millones de dispositivos de todo el país y quiere un tablero en vivo con dos métricas sobre los últimos minutos: cuántos dispositivos distintos reportaron (HyperLogLog) y qué región generó más eventos (Count-Min). Para la ventana deslizante, guarda un resumen por minuto y los combina: los HyperLogLog con el máximo, los Count-Min sumando.

Datos de entrada

Formato: primera línea: p del HyperLogLog, ancho y profundidad del Count-Min y semilla; segunda: tamaño de la ventana en minutos y eventos por minuto; tercera: minutos a simular, dispositivos posibles y regiones.

Cada evento: x = siguiente(x), dispositivo = "disp" + (x resto dispositivos); x = siguiente(x), región = "region" + (x resto regiones). Los sketches son los de los ejercicios anteriores (α para $m = 256$ según la fórmula general).

Ejemplo

```
8 32 3 2026
3 1200
6 4000 5
```

Restricciones: 10^9 eventos por minuto; ventanas de hasta 60 minutos.

Tu programa debe

1. Generar los eventos de cada minuto.
2. Para cada ventana de W minutos consecutivos, fusionar los HyperLogLog de sus minutos (máximo registro a registro) y estimar los dispositivos únicos.
3. Construir el Count-Min de la ventana (equivale a sumar los de cada minuto) y estimar los eventos por región; elegir la de mayor estimación (en empate, la de mayor nombre).
4. Mostrar cada ventana con las estimaciones y los valores reales.

Salida esperada

```
Ventana minutos 0-2: dispositivos únicos estimado 1613 (real 1678) | región con más eventos
region1: estimado 776 (real 776)
Ventana minutos 1-3: dispositivos únicos estimado 1635 (real 1657) | región con más eventos
region1: estimado 751 (real 751)
Ventana minutos 2-4: dispositivos únicos estimado 1590 (real 1675) | región con más eventos
region1: estimado 767 (real 767)
Ventana minutos 3-5: dispositivos únicos estimado 1562 (real 1659) | región con más eventos
region1: estimado 739 (real 739)
```

Reglas

- Complejidad: $O(\text{eventos})$ para procesar y $O(W \times m)$ por ventana para fusionar.
- Los HyperLogLog de una ventana no se pueden "restar" cuando sale un minuto: por eso se guarda uno por minuto y se fusiona la ventana.
- No funcional: memoria acotada: W resúmenes chicos por métrica, sin importar el tráfico.
- No funcional: el tablero se actualiza cada minuto en menos de un segundo.

Pista (leela solo si te trabás): los dos sketches son fusionables, pero de distinta forma: el máximo no se puede deshacer y la suma sí. Guardar un resumen por minuto resuelve la ventana para los dos.

Bonus (opcional)

- Para el Count-Min, mantené la ventana en $O(1)$ restando la tabla del minuto que sale.
- Agregá alertas: si una región supera el doble de su promedio de la última hora, avisar.

Cierre: Las métricas en vivo de plataformas masivas se calculan con sketches fusionables sobre ventanas de tiempo.

Ejercicio 495: Rendición de cuentas de un fondo verificable

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 99/100

Situación: Un fondo de inversión quiere que cada inversor pueda verificar su saldo sin que el fondo publique los saldos de todos. Cada mes arma un árbol de Merkle con los saldos, firma la raíz con su clave privada (RSA de juguete en este ejercicio) y le da a cada inversor su prueba de inclusión. El inversor comprueba dos cosas: que su saldo está en el árbol y que la raíz la firmó el fondo.

Datos de entrada

Formato: primera línea: p, q y e de la clave RSA del fondo; segunda: las hojas "inversor:saldo"; una línea "?" y la hoja a verificar.

Árbol de Merkle como en el ejercicio de la transacción en el lote (con duplicación del último en niveles impares); firma = (raíz resto n)^d resto n. H(texto) = FNV-1a de 32 bits sobre los caracteres (todos ASCII): h = 2166136261; para cada carácter c: h = h XOR código(c); h = (h × 16777619) resto 2³². Después, un mezclado final para repartir mejor los bits: h = h XOR (h >> 16); h = (h × 2246822507) resto 2³²; h = h XOR (h >> 13); h = (h × 3266489909) resto 2³²; h = h XOR (h >> 16).

Ejemplo

```
61 53 17
ana:125000 beto:98000 caro:310000 dani:45000 eli:220000
?
caro:310000
```

Restricciones: hasta 10⁷ inversores (pruebas de unos 24 hashes).

Tu programa debe

1. Construir el árbol de Merkle de los saldos y calcular la raíz.
2. Firmar la raíz con la clave privada del fondo.
3. Armar la prueba de inclusión del inversor y verificarla recalculando la raíz.
4. Verificar la firma con la clave pública sobre la raíz recalculada y mostrar el resultado.

Salida esperada

```
Raíz de saldos: 2018827305 | firma del fondo: 5
Prueba para "caro:310000": 508951365 (der), 188732357 (izq), 96167814 (der)
1) Inclusión: raíz recalculada 2018827305 -> coincide
2) Firma: firma^e resto n = 3086, raíz resto n = 3086 -> auténtica
Resultado: saldo verificado
```

Reglas

- Complejidad: O(n) para el árbol y O(log n) por verificación.
- Las dos verificaciones son necesarias: la prueba sin firma no demuestra que la raíz sea la del fondo; la firma sin prueba no demuestra que el saldo esté incluido.
- No funcional: privacidad: el inversor solo ve hashes de los demás, nunca sus saldos.
- No funcional: seguridad: el ejercicio usa hash y RSA de juguete; en producción, funciones criptográficas estándar y claves de tamaño adecuado.

Pista (leela solo si te trabás): la firma "ata" la raíz al fondo, y la prueba "ata" tu saldo a la raíz. Juntas, dicen: el fondo se comprometió a que tenés este saldo.

Bonus (opcional)

- Detectá saldos negativos escondidos: cada nodo interno guarda también la suma de saldos de su subárbol (árbol de Merkle con sumas).

- Publicó la raíz firmada cada mes y verificó que ningún saldo cambió sin un movimiento registrado.

Cierre: Las pruebas de reservas con árboles de Merkle y firmas se usan en exchanges y fondos para dar transparencia sin revelar datos de clientes.

Ejercicio 496: Radiografías más nítidas

Nivel 10: Staff / Experto · Temática: Salud · Dificultad 99/100

Situación: Un software de diagnóstico por imágenes mejora la nitidez de las radiografías para que los bordes (por ejemplo, de un hueso) se vean mejor. La operación es una convolución: cada píxel se reemplaza por una combinación de él y sus vecinos según un núcleo. Con núcleos grandes e imágenes enormes, la convolución se hace con FFT. El ejemplo usa una fila de píxeles.

Datos de entrada

Formato: primera línea: intensidades de una fila (0 a 255); segunda: el núcleo (impar, centrado).

Resultado[i] = suma de núcleo[j] × píxel[i + j – centro], repitiendo el píxel del borde fuera de la imagen, y recortando a [0, 255]. Por FFT: convolución del arreglo extendido con el núcleo invertido.

Ejemplo

```
40 42 41 43 60 120 180 200 198 202 199 150 90 50 44 42
-1 3 -1
```

Restricciones: imágenes de hasta 4.096×4.096 y núcleos de hasta 64×64 .

Tu programa debe

1. Calcular la convolución directa píxel por píxel.
2. Extender la fila repitiendo los bordes, rellenar con ceros hasta una potencia de 2 y calcular la convolución con FFT (producto punto a punto de las transformadas).
3. Tomar la parte central, redondear, recortar a [0, 255] y verificar que coincide con la directa.
4. Mostrar la fila original, la resultante y el contraste total (suma de saltos entre píxeles vecinos) antes y después.

Salida esperada

```
Original:  40  42  41  43  60 120 180 200 198 202 199 150  90  50  44  42
Nitidez:   38  45  38  28  17 120 220 222 192 209 245 161  70  16  40  40
FFT igual a la convolución directa: sí
Contraste total (suma de saltos): 328 -> 576
```

Reglas

- Complejidad: $O(n \times m)$ directa contra $O(n \log n)$ con FFT; con núcleos grandes la diferencia es enorme.
- Teorema de convolución: convolucionar es multiplicar en el dominio de frecuencias.
- No funcional: seguridad del paciente: la imagen procesada es una ayuda visual; el original siempre se conserva y se muestra junto con la procesada.
- No funcional: trazabilidad: se registra qué filtro y qué parámetros se aplicaron a cada imagen.

Pista (leela solo si te trabás): el núcleo $-1, 3, -1$ resalta un píxel respecto de sus vecinos: donde todo es parejo no cambia nada ($3 - 1 - 1 = 1$), y donde hay un salto, lo exagera.

Bonus (opcional)

- Extiéndelo a 2 dimensiones con un núcleo de 3×3 (FFT por filas y después por columnas).
- Aplicá un desenfoque gaussiano antes de la nitidez para no amplificar el ruido.

Cierre: La convolución con FFT está en el procesamiento de imágenes médicas, fotografía, audio y en las redes neuronales convolucionales.

Ejercicio 497: Qué avión vuela cada ruta

Nivel 10: Staff / Experto · Temática: Turismo · Dificultad 99/100

Situación: Una aerolínea asigna un tipo de avión a cada ruta del día. Cada tipo tiene una cantidad de unidades disponibles y una capacidad de asientos; cada combinación tiene un costo operativo, y si el avión es chico para la demanda, se pierden pasajeros (con un costo por pasajero perdido). Es un problema entero: ramificación y poda con una cota basada en la relajación (ignorar la cantidad de aviones disponibles).

Datos de entrada

Formato: sección "tipos": "tipo unidades asientos"; sección "rutas": "ruta demanda costo_operativo_con_cada_tipo" (en miles).

Costo de asignar el tipo k a la ruta = costo operativo + $\max(0, \text{demanda} - \text{asientos})$ (1 por pasajero perdido, en miles).

Ejemplo

```
tipos
jet_grande 2 180
jet_chico 2 110
turbohélice 2 70
rutas
BA-Córdoba 150 90 70 95
BA-Bariloche 170 120 99 99
BA-Iguazú 100 100 60 80
BA-Salta 120 110 75 90
Córdoba-Salta 60 99 50 40
```

Restricciones: hasta 200 rutas y 10 tipos de avión.

Tu programa debe

1. Asignar ruta por ruta, probando primero el tipo de menor costo para esa ruta, siempre que queden unidades.
2. Cota inferior: costo acumulado + para cada ruta restante, el menor costo posible ignorando la disponibilidad (una relajación, como en programación lineal).
3. Podar las ramas cuya cota iguale o supere la mejor asignación completa.
4. Mostrar la asignación con el desglose de costos, el total, los nodos explorados y la verificación exhaustiva.

Salida esperada

```
BA-Córdoba: jet_grande (costo 90 + pasajeros perdidos 0)
BA-Bariloche: jet_grande (costo 120 + pasajeros perdidos 0)
BA-Iguazú: jet_chico (costo 60 + pasajeros perdidos 0)
BA-Salta: jet_chico (costo 75 + pasajeros perdidos 10)
Córdoba-Salta: turbohélice (costo 40 + pasajeros perdidos 0)
Costo total: 395 | nodos explorados: 12 | verificación exhaustiva: 395
```

Reglas

- Complejidad: exponencial en el peor caso; la cota relajada poda la mayoría de las ramas.
- Relajar una restricción (la disponibilidad) siempre da una cota válida: el problema relajado nunca es más caro que el real.
- No funcional: operación: la asignación tiene que respetar mantenimiento y tripulaciones (en el bonus); una solución que no se puede volar no sirve aunque sea barata.
- No funcional: el plan se recalcula ante cancelaciones en minutos, partiendo de la solución anterior.

Pista (leela solo si te trabás): la cota ignora que hay solo 2 jets grandes: supone que cada ruta usa su avión ideal. Si ni así mejora la mejor solución, esa rama no tiene futuro.

Bonus (opcional)

- Usá la relajación lineal completa (con simplex) como cota y compará la cantidad de nodos.
- Agregá la restricción de que un avión que hace BA-Córdoba puede hacer después Córdoba-Salta (encadenamiento de rutas).

Cierre: La asignación de flota es un problema clásico de investigación operativa en aerolíneas, resuelto con programación entera y ramificación y poda.

Ejercicio 498: Desafío de nivel: Ledger distribuido que sobrevive a cortes de red

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 100/100

Situación: Desafío de nivel. Un ledger distribuido de una billetera corre en tres nodos que pueden quedar partidos por la red. Los depósitos nunca ponen en riesgo nada, así que se aceptan en cualquier nodo y se replican con un contador CRDT. Las extracciones sí (podrían dejar saldo negativo si dos lados extraen a la vez), así que solo se aceptan en un nodo que ve a la mayoría. Al final, los nodos comparan raíces de Merkle para confirmar que convergieron.

Datos de entrada

Formato: primera línea: los nodos; después operaciones: "nodo deposita cuenta monto", "nodo extrae cuenta monto", "sync a b" (solo si están en el mismo grupo), "particion grupo | grupo", "particion_resuelta" y "raices".

Cada cuenta es un contador PN (ejercicio de los votos). Hojas del árbol: "cuenta:saldo" en orden alfabético; árbol como en el ejercicio de la transacción en el lote. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Ejemplo

```
N1 N2 N3
N1 deposita cuenta_A 1000
sync N1 N2
sync N1 N3
particion N1 N2 | N3
N1 extrae cuenta_A 600
N3 extrae cuenta_A 700
N2 deposita cuenta_A 200
sync N1 N2
particion_resuelta
sync N1 N3
sync N3 N2
raices
```

Restricciones: hasta 100 nodos y 10^8 cuentas.

Tu programa debe

1. Aplicar los depósitos localmente en cualquier nodo (sumando en la posición propia del contador de la cuenta).
2. Aceptar una extracción solo si el nodo ve una mayoría estricta de nodos y su saldo local alcanza; si no, rechazarla con el motivo.
3. Sincronizar tomando el máximo posición por posición de los contadores, solo entre nodos conectados.
4. Al pedir "raices", calcular la raíz de Merkle de los saldos de cada nodo y compararlas. Mostrar cada paso.

Salida esperada

```
N1 deposita 1000 en cuenta_A: ok (saldo local 1000)
sync N1 N2: N1 ve cuenta_A=1000, N2 ve cuenta_A=1000
sync N1 N3: N1 ve cuenta_A=1000, N3 ve cuenta_A=1000
Partición: N1, N2 | N3
N1 extrae 600: ok con mayoría (saldo local 400)
N3 extrae 700: RECHAZADA (sin mayoría: ve 1 de 3 nodos)
```

```
N2 deposita 200 en cuenta_A: ok (saldo local 1200)
sync N1 N2: N1 ve cuenta_A=600, N2 ve cuenta_A=600
Partición resuelta
sync N1 N3: N1 ve cuenta_A=600, N3 ve cuenta_A=600
sync N3 N2: N3 ve cuenta_A=600, N2 ve cuenta_A=600
Raíces de Merkle: N1=1770350494, N2=1770350494, N3=1770350494 -> réplicas IDÉNTICAS
```

Reglas

- Complejidad: $O(\text{nodos})$ por sincronización de cada cuenta; $O(\text{cuentas})$ para la raíz.
- Integra los tres temas: CRDT (depósitos sin coordinación), quórums (extracciones con mayoría) y árboles de Merkle (verificación de convergencia).
- No funcional: invariante de negocio: ningún saldo puede quedar negativo, aunque la red se parta.
- No funcional: disponibilidad selectiva: lo que no pone en riesgo la invariante (depósitos) se acepta siempre; lo que sí (extracciones), solo con mayoría.
- No funcional: en un sistema real, el saldo que verifica una extracción debe incluir lo que la mayoría conoce, no solo lo local (bonus).

Pista (leela solo si te trabás): no todas las operaciones necesitan el mismo nivel de coordinación. Sumar nunca rompe nada; restar puede romper la invariante, y ahí se paga el costo de pedir mayoría.

Bonus (opcional)

- Antes de aprobar una extracción, sincronizá con la mayoría visible para usar el saldo más actualizado.
- Usá las raíces de Merkle para encontrar exactamente qué cuentas difieren entre dos nodos (ejercicio de las réplicas).

Cierre: Combinar CRDT, quórums y verificación con hashes es la forma en que los sistemas financieros distribuidos equilibran disponibilidad y corrección.

Ejercicio 499: Desafío de nivel: Antifraude a escala planetaria

Nivel 10: Staff / Experto · Temática: Seguridad informática · Dificultad 100/100

Situación: Desafío de nivel. Un procesador de tarjetas con centros en dos continentes evalúa cada transacción en milisegundos. Usa un filtro de Bloom para la lista negra (y confirma contra la lista real solo si el filtro da positivo), un Count-Min para la velocidad de uso por tarjeta, y relojes vectoriales para detectar que la misma tarjeta se usó "a la vez" en dos regiones (eventos concurrentes, que ninguna región conocía de la otra).

Datos de entrada

Formato: primera línea: bits y funciones del Bloom, ancho y profundidad del Count-Min, y límite de usos; segunda: la lista negra; una línea "?" y las transacciones "región [reloj_AR,reloj_EU] tarjeta monto" en orden de llegada al motor central.

Filtro y sketch como en los ejercicios anteriores. $H(\text{texto}) = \text{FNV-1a}$ de 32 bits sobre los caracteres (todos ASCII): $h = 2166136261$; para cada carácter c : $h = h \text{ XOR código}(c)$; $h = (h \times 16777619) \text{ resto } 2^{32}$. Después, un mezclado final para repartir mejor los bits: $h = h \text{ XOR } (h \gg 16)$; $h = (h \times 2246822507) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 13)$; $h = (h \times 3266489909) \text{ resto } 2^{32}$; $h = h \text{ XOR } (h \gg 16)$.

Reglas: lista negra confirmada → BLOQUEAR; uso estimado (contando la actual) mayor al límite → REVISAR; la última transacción de la tarjeta fue en otra región y es concurrente con esta → REVISAR. Se informan todos los motivos.

Ejemplo

```
128 3 16 3 3
lista_negra 4111-0001 4111-0002 4111-0003
?
AR [1,0] 4111-9001 5000
AR [2,0] 4111-9001 3000
EU [0,1] 4111-9001 4000
AR [3,0] 4111-0002 1000
EU [0,2] 4111-9002 2000
AR [4,1] 4111-9002 2500
AR [5,1] 4111-9001 1000
AR [6,1] 4111-9001 800
AR [7,1] 4111-7777 100
```

Restricciones: 10^6 transacciones por segundo en cada región.

Tu programa debe

1. Cargar la lista negra en el filtro de Bloom; ante un positivo, confirmar contra la lista exacta.
2. Actualizar el Count-Min de la tarjeta y aplicar la regla de velocidad.
3. Comparar el reloj vectorial con el de la última transacción de la tarjeta: si son concurrentes y de regiones distintas, marcar uso concurrente.
4. Decidir (bloquear tiene prioridad sobre revisar) y mostrar la decisión con los motivos.

Salida esperada

```
AR 4111-9001 $5000: APROBAR
AR 4111-9001 $3000: APROBAR
EU 4111-9001 $4000: REVISAR (uso concurrente en AR y EU)
AR 4111-0002 $1000: BLOQUEAR (lista negra (confirmada))
EU 4111-9002 $2000: APROBAR
AR 4111-9002 $2500: REVISAR (uso concurrente en EU y AR)
AR 4111-9001 $1000: REVISAR (velocidad: 4 usos estimados)
AR 4111-9001 $800: REVISAR (velocidad: 5 usos estimados)
AR 4111-7777 $100: APROBAR
```

Reglas

- Complejidad: $O(k + d + \text{regiones})$ por transacción.
- El filtro de Bloom evita consultar la lista exacta (lenta) en el 99 % de los casos; sus falsos positivos se descartan con la confirmación.
- No funcional: latencia: la decisión completa en menos de 10 ms.
- No funcional: explicabilidad: cada decisión se registra con todos sus motivos, incluidos los descartados.
- No funcional: los relojes vectoriales detectan concurrencia sin depender de relojes físicos sincronizados entre continentes.

Pista (leela solo si te trabás): cada estructura contesta una pregunta distinta en tiempo constante: "¿podría estar en la lista?", "¿cuántas veces se usó?", "¿esta región sabía del uso anterior?". El motor solo combina las respuestas.

Bonus (opcional)

- Agregá ventanas de tiempo al Count-Min (un sketch por minuto, como en el ejercicio de la telemetría).
- Cuando una tarjeta se bloquea, propagá el bloqueo a la otra región e incorporala al filtro.

Cierre: Los sistemas antifraude de escala global combinan estructuras probabilísticas, reglas y ordenamiento de eventos distribuidos.

Ejercicio 500: Desafío de nivel: La mesa de riesgo del banco

Nivel 10: Staff / Experto · Temática: Finanzas y FinTech · Dificultad 100/100

Situación: Desafío final. La mesa de riesgo de un banco prepara el informe diario integrando lo aprendido: arma la cartera de mínima varianza (Markowitz), mide su VaR al 99 % con Monte Carlo correlacionado (Cholesky), cotiza la cobertura con opciones de venta (Black-Scholes) y reparte el capital de riesgo entre las mesas de operación con programación lineal, respetando los límites de cada una.

Datos de entrada

Formato: primera línea: simulaciones y semilla; después "activo media_diaria desvío_diario" (tres activos); la matriz de covarianzas diarias; el capital de la cartera; la opción "S K tasa volatilidad plazo"; los límites "mesas cantidad límite_%..." y el rendimiento por punto de capital de cada mesa.

Monte Carlo como en el ejercicio del riesgo de crédito (tres normales por escenario, correlacionadas con la Cholesky de la matriz de correlaciones derivada de las covarianzas). Reparto: maximizar suma rendimiento_i $\times$ x_i con suma x_i $\leq$ 100 y x_i $\leq$ límite_i.

Ejemplo

```
5000 2026
acciones 0.0006 0.020
bonos 0.0003 0.008
dólar 0.0004 0.012
0.004000 0.000048 0.000120
0.000048 0.000064 0.000019
0.000120 0.000019 0.000144
10000000
1000 1000 0.05 0.25 0.5
mesas 3 50 40 30
0.9 0.6 0.4
```

Restricciones: carteras de 10^4 activos, 10^5 escenarios y decenas de mesas, recalculadas cada noche.

Tu programa debe

1. Resolver el sistema de Lagrange de la cartera de mínima varianza (fracciones exactas).
2. Simular el resultado diario de la cartera con esos pesos y tomar la pérdida en el percentil 1 (posición [0,01 N]).
3. Valuar la opción de venta con Black-Scholes y su delta ($N(d1) - 1$).
4. Resolver el reparto de capital con el simplex. Mostrar los cuatro resultados.

Salida esperada

```
1) Cartera de mínima varianza: acciones -0.39 %, bonos 73.65 %, dólar 26.74 %
2) VaR 99 % a un día de la cartera (Monte Carlo, 5000 escenarios): $166535
3) Cobertura con opciones de venta (Black-Scholes): prima 57.9101 por unidad | delta de la venta -0.4091
4) Capital de riesgo por mesa (simplex): mesa 1 50.0 %, mesa 2 40.0 %, mesa 3 10.0 % | rendimiento esperado 73.0
```

Reglas

- Complejidad: dominada por la simulación, $O(N \times n^2)$.
- Cada módulo reutiliza la implementación de su ejercicio y se prueba por separado antes de integrarlo.
- No funcional: reproducibilidad total: semillas, generador y orden de uso de los números aleatorios especificados.

- No funcional: aislamiento de fallas: si un módulo falla (por ejemplo, una matriz no definida positiva), el informe se emite con los demás módulos y el error explicado.
- No funcional: el informe es una herramienta de gestión de riesgo; no constituye una recomendación de inversión.
- Los decimales se muestran con punto y sin separador de miles en la salida.

Pista (leela solo si te trabás): a esta altura ya construiste cada pieza. El desafío es de ingeniería: interfaces claras entre módulos, datos validados en cada frontera y una salida que un comité de riesgo pueda leer y auditar.

Bonus (opcional)

- Calculá cuántas opciones de venta hacen falta para bajar el VaR a la mitad (combinando el delta con la simulación).
- Automatizá el informe para que corra cada noche y alerte si el VaR supera un límite (con un circuit breaker si faltan datos de mercado).

Cierre: Llegaste al final: combinar algoritmos, matemática y diseño de sistemas para resolver problemas reales es exactamente el trabajo de un ingeniero de nivel staff.

Apéndice A: ejercicios y conceptos

N.º	Niv.	Título	Conceptos principales
1	1	La propina de la mesa 4	variables, aritmética básica, porcentajes
2	1	¿Cuánto tarda en llegar el colectivo?	variables, aritmética, conversión de unidades
3	1	Termómetro de dos escalas	variables, fórmulas, números decimales
4	1	El kiosco que cobra con descuento	condicionales simples, porcentajes
5	1	¿Aprobé o voy a recuperatorio?	condicionales, promedio simple
6	1	Plazo fijo a 30 días	aritmética, porcentajes, redondeo
7	1	El semáforo del IMC	condicionales encadenados, fórmulas
8	1	Horas extra del turno noche	condicionales, aritmética
9	1	Cuenta regresiva para el lanzamiento del globo sonda	bucles simples, contadores
10	1	El precio del envío según el peso	condicionales por rangos
11	1	¿Me alcanza para la cuota?	condicionales, comparación de montos
12	1	Megas que quedan en el plan	aritmética, conversión de unidades, división entera y resto
13	1	Goles por partido del torneo barrial	acumuladores, promedio
14	1	La factura de luz escalonada	condicionales por tramos, acumuladores
15	1	Vidas y escudo del personaje	variables, condicionales, contadores
16	1	El metrónomo del ensayo	aritmética, conversión de tiempo
17	1	Cambio de moneda en el aeropuerto	aritmética, comisiones, redondeo
18	1	Seguidores ganados en la semana	bucles, acumuladores
19	1	Tres intentos para entrar	contadores, condicionales, bucle con corte
20	1	La alcancía con meta de ahorro	bucle mientras, acumuladores
21	1	Pasos diarios hacia los 10.000	acumuladores, condicionales
22	1	La billetera virtual que rinde todos los días	bucles, interés diario, redondeo
23	1	Kilómetros por litro del camión de reparto	división, promedio
24	1	La receta para 12 comensales	proporciones, números decimales
25	1	Tablas de multiplicar para la seño	bucles anidados simples
26	1	El año bisiesto del cumpleaños	condicionales compuestos, operador resto
27	1	Estacionamiento por fracción de hora	condicionales, redondeo hacia arriba
28	1	Préstamo entre amigos con interés simple	fórmulas, porcentajes
29	1	El vuelto con billetes y monedas	división entera, resto, bucles
30	1	Paneles solares para el techo	aritmética, redondeo hacia arriba
31	1	Racha de victorias del equipo	contadores con reinicio, bucles
32	1	Monedas de oro del cofre	acumuladores, condicionales
33	1	¿Cuotas sin interés o contado con descuento?	comparación de alternativas, aritmética
34	1	La helada de la madrugada	máximo y mínimo con bucle
35	1	Duración total de la playlist	conversión de segundos a minutos, acumuladores
36	1	Presupuesto de las vacaciones en la costa	acumuladores, condicionales
37	1	¿Es primo el número de la rifa?	bucles, divisibilidad
38	1	Del sueldo bruto al neto	porcentajes, acumulación de descuentos
39	1	Recargo por pago atrasado	condicionales, porcentajes por día de atraso
40	1	El plan de datos que conviene	comparación de alternativas, condicionales
41	1	Dosis según el peso del paciente	fórmulas, redondeo, rangos válidos
42	1	Pedidos que entran en la camioneta	acumuladores con tope, bucle con corte
43	1	Likes que se duplican cada hora	bucles, crecimiento exponencial
44	1	El dígito que delata una tarjeta mal tipeada	extracción de dígitos, resto, bucles
45	1	Cuánto debo después de cada cuota	bucles, acumuladores, interés sobre saldo
46	1	La cantina escolar y sus tickets	contadores por categoría
47	1	Rendimiento de la cosecha por hectárea	aritmética, promedio ponderado
48	1	Desafío de nivel: La caja registradora del almacén	integrador: acumuladores, descuentos, vuelto
49	1	Desafío de nivel: El cajero automático que no da monedas	integrador: condicionales, división entera, contadores
50	1	Desafío de nivel: La liga de fútbol 5 de los jueves	integrador: bucles, puntos, condicionales
51	2	Notas del curso: la mejor y la peor	listas, máximo, mínimo, promedio
52	2	Días del mes por encima del promedio	listas, promedio, filtrado

N.º	Niv.	Título	Conceptos principales
53	2	Gastos del mes día por día	listas, acumuladores, máximo
54	2	Patentes capicúa	recorrido de texto, comparación de extremos
55	2	Vocales y consonantes de la canción	recorrido de texto, contadores
56	2	Stock bajo en el depósito	listas, búsqueda lineal, filtrado
57	2	Cuentas claras en el asado	listas, saldos, emparejamiento de deudores y acreedores
58	2	Butacas libres en el teatro	matrices, recorrido, contadores
59	2	Frecuencia cardíaca en reposo	listas, promedio, rangos
60	2	El hashtag más largo del posteo	recorrido de texto, separar palabras
61	2	Conversor de monedas en funciones	funciones, parámetros, valor de retorno
62	2	Ta-te-ti: ¿quién ganó?	matrices, filas, columnas y diagonales
63	2	Nombres prolijos en la planilla de personal	recorrido de texto, funciones
64	2	Lluvia acumulada por semana en el campo	matrices, sumas por fila y columna
65	2	El producto estrella del día	listas paralelas, máximo
66	2	¿Contraseña fuerte o débil?	recorrido de texto, clasificación de caracteres
67	2	El mensaje secreto del club de ajedrez	recorrido de texto, desplazamiento de caracteres
68	2	Movimientos de la cuenta y el saldo más bajo	listas, saldo acumulado, mínimo
69	2	La vuelta más rápida del karting	listas, mínimo, conversión de tiempo
70	2	Cobertura de antenas en la grilla	matrices, vecinos de una celda
71	2	Menú del día que entra en el presupuesto	búsqueda lineal, filtrado
72	2	Vuelos demorados del día	listas, filtrado, promedio
73	2	Espejo de palabras para el cartel	recorrido de texto, inversión
74	2	El paciente con la fiebre más alta	listas de registros, máximo
75	2	Rotar la foto 90 grados	matrices, transposición
76	2	La tasa real de las cuotas	funciones, porcentajes
77	2	Consumo eléctrico por franja horaria	listas, sumas por rango
78	2	Contador de palabras para el ensayo	recorrido de texto, contadores
79	2	La parada de colectivo más cercana	búsqueda lineal, distancia absoluta
80	2	Inventario de pociones	listas, búsqueda, actualización
81	2	El CBU que no cierra	funciones, dígitos verificadores, texto
82	2	Planilla de asistencia del mes	matrices, contadores por fila y columna
83	2	Transportar acordes un tono arriba	listas, aritmética circular
84	2	Pedidos atrasados del día	listas, comparación de horas
85	2	El comentario con más reacciones	listas de registros, máximo
86	2	Glucemia de la semana	listas, promedio, alertas por umbral
87	2	Tabla de posiciones sin ordenar	listas paralelas, máximo con empates
88	2	Cuánto rinde cada alternativa de ahorro	funciones, interés compuesto, comparación
89	2	Sopa de letras de la escuela	matrices, búsqueda de palabras en filas y columnas
90	2	Calorías del plato armado	listas, acumulación, funciones
91	2	SMS sospechosos en la bandeja	recorrido de texto, palabras clave
92	2	Parcelas con sequía	matrices, umbrales, contadores
93	2	El robot del depósito sigue instrucciones	matrices, simulación simple con instrucciones
94	2	La acción que más subió en la semana	listas, variación porcentual, máximo
95	2	Horario de la medicación	funciones, horas, listas
96	2	Sensores que marcan valores imposibles	listas, validación, filtrado
97	2	Una generación del juego de la vida	matrices, vecinos, copia de matriz
98	2	Desafío de nivel: El resumen de la tarjeta de crédito	integrador: listas, funciones, totales, máximos
99	2	Desafío de nivel: Batalla naval en la terminal	integrador: matrices, texto, contadores
100	2	Desafío de nivel: Boletín de calificaciones del curso	integrador: matrices, promedios, texto
101	3	Invitados repetidos del casamiento	conjuntos, eliminación de duplicados
102	3	Palabras más repetidas en las reseñas	mapas clave-valor, parseo de texto
103	3	Gastos agrupados por categoría	mapas clave-valor, acumulación por clave
104	3	La fila de la guardia por hora de llegada	implementar un ordenamiento simple
105	3	Días hasta el vencimiento	fechas, años bisiestos
106	3	Alumnos que cursan los dos talleres	conjuntos, intersección
107	3	Leer el log del servidor	parseo de texto, mapas clave-valor
108	3	Ranking de goleadores del campeonato	mapas clave-valor, ordenamiento
109	3	Turnos médicos que se pisan	fechas y horas, comparación de intervalos

N.º	Niv.	Título	Conceptos principales
110	3	Canciones favoritas en común	conjuntos, intersección y unión
111	3	Postulantes con email inválido	validación de datos, parseo de texto
112	3	¿De cuántas formas sirvo los platos?	recursión básica, factorial
113	3	Stock consolidado de sucursales	mapas clave-valor, fusión de datos
114	3	Horas trabajadas por semana del año	fechas, agrupación
115	3	Anagramas del juego de palabras	mapas de frecuencias, ordenamiento de caracteres
116	3	Tarifas de roaming por país	mapas clave-valor, búsqueda por clave
117	3	El extracto bancario desordenado	validación, fechas, agrupación por categoría y mes
118	3	Viaje con escalas en distintos husos horarios	fechas y horas, husos horarios
119	3	Hashtags en tendencia	mapas clave-valor, conteo, top-k por ordenamiento
120	3	Descendientes de la vaca campeona	recursión básica, mapas clave-valor
121	3	Consumo por electrodoméstico y por mes	parseo de texto separado por comas, mapas anidados
122	3	Validar un CUIT	validación, dígito verificador, parseo
123	3	Reservas duplicadas del hotel	conjuntos, claves compuestas
124	3	La carta ordenada por precio y nombre	ordenamiento con criterio múltiple
125	3	Traductor de código Morse	mapas clave-valor, parseo
126	3	Pacientes por obra social	mapas clave-valor, agrupación, conteo
127	3	Suscripciones que vencen este mes	fechas, filtrado
128	3	Planilla de jugadas del partido de básquet	parseo de texto, mapas clave-valor
129	3	Capítulos en números romanos	mapas clave-valor, conversión
130	3	Los que no te siguen de vuelta	conjuntos, diferencia
131	3	Direcciones mal cargadas	validación, parseo, normalización de texto
132	3	Las torres del museo de ciencias	recursión
133	3	Puntos de fidelidad por categoría	mapas clave-valor, reglas simples
134	3	Cumpleaños del mes en la oficina	fechas, ordenamiento
135	3	Intercalar dos listas ordenadas de transacciones	ordenamiento, intercalación
136	3	Lecturas dudosas de la estación meteorológica	validación, parseo, fechas
137	3	Logros desbloqueados	conjuntos, mapas clave-valor
138	3	Setlist sin repetir	conjuntos, orden de inserción
139	3	Tareas de la chacra según el día de la semana	fechas, día de la semana
140	3	IPs con demasiados intentos	mapas clave-valor, ventanas de tiempo simples
141	3	Pagos contra facturas	mapas clave-valor, conjuntos, validación
142	3	Tiempos de entrega por zona	mapas clave-valor, promedios, ordenamiento
143	3	Buscador de recetas por ingredientes	parseo, conjuntos, intersección
144	3	Todas las formaciones posibles del equipo	recursión, generación de combinaciones
145	3	Fechas en tres formatos distintos	parseo de fechas, validación
146	3	Aguinaldo proporcional	fechas, cálculo proporcional
147	3	Calendario de vacunación	fechas, suma de días
148	3	Desafío de nivel: El extracto de dos bancos en uno	integrador: parseo, fechas, mapas, ordenamiento
149	3	Desafío de nivel: Padrón de socios del club	integrador: validación, conjuntos, ordenamiento
150	3	Desafío de nivel: Farmacia con vencimientos	integrador: fechas, mapas, validación
151	4	La calculadora que se queja de los paréntesis	pilas
152	4	Un DNI en un padrón de 10 millones	búsqueda binaria, complejidad
153	4	La cola de la sucursal bancaria	colas, simulación con estado
154	4	Dos productos que suman el cupón	dos punteros, ordenamiento
155	4	Los 7 días de más ventas en un millón de registros	ventana deslizante, sumas prefijas, Big-O
156	4	Ctrl+Z del editor de publicaciones	pilas
157	4	Saldo acumulado por rango de días	sumas prefijas
158	4	¿Cuántos días hasta que haga más calor?	pila monótona
159	4	El mejor día para comprar y vender	una sola pasada, mínimo acumulado
160	4	El lado del potrero cuadrado	búsqueda binaria sobre la respuesta
161	4	Semáforo y fila de autos	colas, simulación con estado
162	4	Reservas que se pueden fusionar	ordenamiento, fusión de intervalos
163	4	El tramo de ADN sin bases repetidas	ventana deslizante, conjuntos
164	4	Capacidad mínima del camión para cumplir el plazo	búsqueda binaria sobre la respuesta
165	4	Lluvia atrapada entre silos	dos punteros
166	4	La versión que rompió todo	búsqueda binaria

N.º	Niv.	Título	Conceptos principales
167	4	La ventana de mayor pulso promedio	ventana deslizante de tamaño fijo
168	4	Atrás y adelante en el navegador	dos pilas
169	4	Turnos de la impresora de la facultad	colas, simulación
170	4	Tramos de gastos que suman exacto	sumas prefijas con mapa
171	4	Canciones que completan justo una hora	dos punteros, ordenamiento
172	4	La papa caliente	cola circular, simulación
173	4	La calculadora de la base antártica	pilas, notación posfija
174	4	Stock mínimo en cada ventana de días	ventana deslizante con cola doble
175	4	Parejas de tenis con nivel parecido	ordenamiento, dos punteros
176	4	Logs entre dos horarios	búsqueda binaria de límites inferior y superior
177	4	La racha más larga de días en verde	una pasada, estado
178	4	El ascensor del edificio de oficinas	simulación con estado, colas
179	4	Vistas únicas en la última hora	ventana deslizante, mapas
180	4	Hornos compartidos en la pastelería	colas, simulación
181	4	La tasa que hace cerrar la cuota	búsqueda binaria sobre números reales
182	4	Ventas por región y rango de meses	sumas prefijas en dos dimensiones
183	4	Ráfagas de intentos de login	ventana deslizante con cola
184	4	El rectángulo más grande de la tribuna	pila monótona
185	4	Cola de pedidos armada con dos pilas	cola con dos pilas, costo amortizado
186	4	Pulsos del marcapasos	simulación con estado
187	4	Llamadas simultáneas en la central	ordenamiento de eventos, barrido
188	4	Dosis máxima en cualquier ventana de 24 horas	ventana deslizante sobre tiempo
189	4	El dólar más caro de cada semana	cola doble monótona
190	4	El mazo de la escoba	colas, simulación con reglas
191	4	Tres productos para el presupuesto exacto	ordenamiento, dos punteros, complejidad cuadrática
192	4	Radár de velocidad en la ruta	sumas prefijas, consultas por rango
193	4	¿Está la melodía escondida en la canción?	dos punteros, subsecuencia
194	4	Baterías de la red domiciliaria	simulación con estado
195	4	Buscar un precio en la grilla ordenada	búsqueda en matriz ordenada
196	4	El catálogo ordenado que alguien rotó	búsqueda binaria modificada
197	4	Calendario de guardias rotativas	simulación, colas
198	4	Desafío de nivel: La caja de ahorro con débitos automáticos	integrador: colas, sumas prefijas, simulación
199	4	Desafío de nivel: Monitor de tráfico de la autopista	integrador: ventana deslizante, pilas, búsqueda binaria
200	4	Desafío de nivel: La cocina del restaurante en hora pico	integrador: colas, simulación por eventos
201	5	Vuelto con la menor cantidad de piezas	greedy con justificación
202	5	El sudoku del diario del domingo	backtracking
203	5	Estados de un pedido online	máquina de estados
204	5	La calculadora del presupuesto de obra	evaluación de expresiones, precedencia de operadores
205	5	Máximo de charlas en el auditorio	greedy por hora de finalización
206	5	Archivos duplicados en el servidor	hashing
207	5	Ocho damas en el tablero del club	backtracking
208	5	El semáforo inteligente	máquina de estados
209	5	Descuentos que se acumulan (o no)	modelado de reglas de negocio
210	5	Todas las formas de pagar justo	backtracking, conteo
211	5	El barco con carga fraccionable	greedy fraccional
212	5	La marca de la tarjeta según su número	reglas de negocio, validación por prefijos
213	5	El fixture con sedes y descansos	backtracking con restricciones
214	5	La máquina expendedora	máquina de estados
215	5	Contraseñas débiles por diccionario	backtracking de generación
216	5	Turnos de las bombas de riego	greedy con justificación
217	5	Casilleros del gimnasio	hashing, resolución de colisiones
218	5	Mesas de examen sin choques	backtracking, coloreo
219	5	Estados de un turno médico	máquina de estados
220	5	Tarifa de taxi con recargo nocturno	reglas de negocio

N.º	Niv.	Título	Conceptos principales
221	5	Fotos casi iguales	hashing de características
222	5	Cartones de bingo que suman justo	backtracking con poda
223	5	Liquidación de sueldos según convenio	reglas de negocio
224	5	Comandos del chatbot de atención	máquina de estados, parseo
225	5	Cargar primero lo que más rinde	greedy con justificación
226	5	Reglas del firewall	evaluación de expresiones booleanas
227	5	Dos equipos parejos	backtracking, partición
228	5	El crucigrama de la revista escolar	backtracking
229	5	Impuesto progresivo por tramos	reglas de negocio, tramos
230	5	El mismo tema cargado dos veces	hashing, normalización de texto
231	5	El cajero automático como máquina de estados	máquina de estados
232	5	Menú de pasos sin ingredientes repetidos	backtracking con conjuntos
233	5	Paradas de carga del auto eléctrico	greedy con justificación
234	5	Aprobación de créditos por puntaje	reglas de negocio, scoring
235	5	Validar fórmulas químicas	parseo, evaluación con pilas
236	5	Tareas del sprint con fecha límite	greedy con plazos
237	5	El saludo de tres pasos entre módems	máquina de estados
238	5	Aulas mínimas para la cursada	greedy por intervalos
239	5	Política de cancelación de reservas	reglas de negocio, fechas
240	5	El ahorcado con estados	máquina de estados
241	5	Cupones incompatibles	backtracking, maximización
242	5	Filtros con comodines	backtracking sobre texto
243	5	Rotación de cultivos con restricciones	backtracking
244	5	Salto entre antenas repetidoras	greedy
245	5	Fórmulas de precios con variables	evaluación de expresiones con variables
246	5	El city tour con presupuesto	backtracking con poda
247	5	Transacciones duplicadas por reintento	hashing, claves compuestas
248	5	Desafío de nivel: Motor de promociones del supermercado	integrador: reglas, greedy, hashing
249	5	Desafío de nivel: El intérprete de la hoja de cálculo	integrador: expresiones, referencias, detección de ciclos simples
250	5	Desafío de nivel: Agenda del congreso médico	integrador: backtracking, greedy, restricciones
251	6	Niveles del organigrama	árboles, recorrido por niveles
252	6	Las 5 transacciones más grandes del día	heap, top-k
253	6	El camino más corto en el depósito	BFS en grilla
254	6	Islas de cultivo en la foto satelital	DFS, componentes conexas
255	6	Materias correlativas	orden topológico
256	6	Amigos de amigos	BFS por niveles
257	6	La mochila del mochilero	mochila 0/1
258	6	El mismo cliente escrito distinto	distancia de edición
259	6	Árbol de categorías de la tienda	árboles, sumas en subárboles
260	6	Triage en la guardia	cola de prioridad
261	6	Barrios conectados a la red eléctrica	union-find
262	6	Reparto con peajes y horarios de corte	Dijkstra con restricciones
263	6	Qué cambió en el reglamento	subsecuencia común más larga
264	6	Precios entre dos valores al instante	árbol binario de búsqueda, consultas por rango
265	6	De cuántas formas se junta un monto	programación dinámica de conteo
266	6	Orden de carga de los niveles del juego	orden topológico, detección de ciclos
267	6	El antepasado común de dos especies	ancestro común más bajo
268	6	Mediana de tiempos de respuesta en vivo	dos heaps
269	6	Fibra óptica mínima entre edificios	árbol generador mínimo, union-find
270	6	La racha creciente de precios más larga	subsecuencia creciente más larga en $O(n \log n)$
271	6	Cuánto tarda en viajar un rumor	Dijkstra
272	6	Cadena de contagio en el hospital	BFS multi-fuente
273	6	Dos cosechas del mismo peso	programación dinámica de suma de subconjuntos
274	6	Combinar vuelos de varias aerolíneas	heap, fusión de k listas
275	6	Cortes de tela en la fábrica	programación dinámica de corte
276	6	El árbol de decisión del crédito	árboles, recorrido, validación

N.º	Niv.	Título	Conceptos principales
277	6	Pagos circulares entre cuentas	DFS, detección de ciclos dirigidos
278	6	Palabras escondidas en el tablero	DFS con backtracking en grilla
279	6	Escalera de palabras	BFS en grafo implícito
280	6	El planificador del router	heap, simulación
281	6	Cuentas del mismo titular	union-find con mapas
282	6	Tendido eléctrico sobre terreno irregular	programación dinámica en grilla
283	6	La sala más profunda del museo	DFS, profundidad máxima
284	6	¿Se pueden armar dos bandos rivales?	grafo bipartito con BFS
285	6	Guardar y recuperar el menú anidado	árboles, serialización
286	6	Monedas con denominaciones raras	programación dinámica de mínimo
287	6	Trenes con horarios fijos	Dijkstra con tiempos de salida
288	6	Reposición con prioridades que cambian	heap con actualización perezosa
289	6	Mutaciones con costos distintos	distancia de edición ponderada
290	6	Estaciones críticas de la red de subte	DFS, puentes y puntos de articulación
291	6	Campañas publicitarias con cupos	mochila acotada
292	6	Árbol de géneros musicales	árboles, recorridos
293	6	El proveedor más confiable	Dijkstra con probabilidades
294	6	El camino crítico del proyecto	orden topológico, programación dinámica
295	6	La frase palíndroma más larga de la melodía	programación dinámica sobre subsecuencias
296	6	Camas disponibles en la clínica	heaps, simulación por eventos
297	6	Conversiones encadenadas de monedas	BFS/DFS con pesos multiplicativos
298	6	Desafío de nivel: Centro de distribución con prioridades y rutas	integrador: Dijkstra, heap, union-find
299	6	Desafío de nivel: El planificador de la cursada	integrador: orden topológico, programación dinámica
300	6	Desafío de nivel: Presupuesto de inversión con dependencias	integrador: mochila, orden topológico
301	7	Autocompletado del buscador de la tienda	trie
302	7	Permisos que caben en un número	manipulación de bits
303	7	Primos para rifas gigantes	criba de Eratóstenes
304	7	Colectivos que coinciden en la parada	MCD y MCM
305	7	Formas de armar el podio	combinatoria, aritmética modular
306	7	Interés compuesto a plazos astronómicos	exponenciación rápida modular
307	7	Orden óptimo para fusionar archivos de logs	programación dinámica sobre intervalos
308	7	Visitar 15 sucursales en un solo día	programación dinámica con bitmask
309	7	Guardias en el árbol de la empresa	programación dinámica en árboles
310	7	Mínimo de temperatura por rango	árbol de segmentos
311	7	Palabras prohibidas en el chat	trie, búsqueda de múltiples patrones
312	7	Saludos por rango de fechas en tiempo real	árbol de Fenwick, actualizaciones y consultas
313	7	Cortar la barra de chocolate en la fábrica	programación dinámica sobre intervalos
314	7	XOR de canales en un rango	bits, prefijos XOR
315	7	Repartir premios módulo un primo	inverso modular
316	7	Cada empleado con su tarea ideal	programación dinámica con bitmask
317	7	Inversiones en el ranking	árbol de Fenwick, conteo de inversiones
318	7	Divisores del número de lote	factorización, cantidad de divisores
319	7	Caminos en la grilla con obstáculos	combinatoria, programación dinámica modular
320	7	Ajustes de stock por rango	árbol de segmentos con propagación perezosa
321	7	La transacción huérfana	manipulación de bits, XOR
322	7	Formas de agrupar una expresión	programación dinámica sobre intervalos, números de Catalan
323	7	Máscaras de subred de los dispositivos	bits, redes
324	7	Números de serie primos entre dos billones	criba segmentada
325	7	Distancias en el árbol de la red eléctrica	programación dinámica en árboles con re-enraizado
326	7	Prefijos telefónicos más comunes	trie con contadores
327	7	Subconjuntos de cartas que suman el objetivo	enumeración por bits, meet in the middle
328	7	Contraseñas de largo N sin patrones prohibidos	combinatoria, programación dinámica modular
329	7	Parcelas que no pueden sembrarse juntas	programación dinámica con bitmask por perfil
330	7	Clientes distintos en un rango de días	árbol de Fenwick con consultas offline

N.º	Niv.	Título	Conceptos principales
331	7	Qué día cae el vencimiento dentro de 10^{18} días	aritmética modular
332	7	La canción con más energía tras los cambios	árbol de segmentos con actualización puntual
333	7	La k-ésima transacción más chica	árbol de Fenwick con búsqueda binaria
334	7	Mezclar pociones sin que exploten	programación dinámica sobre intervalos
335	7	Boletos capicúa en un rango	programación dinámica sobre dígitos
336	7	Relojes que vuelven a coincidir	teorema chino del resto
337	7	Productos de una categoría con altas y bajas	recorrido de Euler, árbol de Fenwick
338	7	Caminos posibles en la red de rutas	programación dinámica en DAG, aritmética modular
339	7	El par de claves con XOR máximo	trie binario
340	7	Tabla de posiciones con actualizaciones en vivo	árbol de segmentos
341	7	Repartir caramelos entre chicos	combinatoria, estrellas y barras, inverso modular
342	7	MCD de rangos de lotes	tabla dispersa, MCD
343	7	Fusionar lotes de pedidos al menor costo	programación dinámica sobre intervalos
344	7	Combinaciones de vacunas compatibles	bitmask, subconjuntos
345	7	Ocupación de camas por rango de días	árbol de segmentos con propagación perezosa
346	7	Cuotas coprimas con el período	función phi de Euler
347	7	El juego de las fichas del recreo	teoría de juegos, XOR
348	7	Desafío de nivel: Libro mayor con correcciones retroactivas	integrador: Fenwick, aritmética modular, bits
349	7	Desafío de nivel: El teclado predictivo	integrador: trie, programación dinámica, combinatoria
350	7	Desafío de nivel: Plan de inversión en la red de riego	integrador: programación dinámica en árboles, bitmask
351	8	El catálogo que no entra en memoria	caché LRU
352	8	Mil pedidos por segundo al procesador de pagos	rate limiter por token bucket
353	8	Probabilidad de lluvia el fin de semana	probabilidad, independencia
354	8	Reintentos con espera exponencial	reintentos, determinismo
355	8	Pagos que no se cobran dos veces	idempotencia
356	8	Los temas que siempre vuelven	caché LFU
357	8	Jugadores que no cambian de servidor	consistent hashing
358	8	Reconstruir el carrito desde sus eventos	event sourcing
359	8	Reglas antifraude de tarjetas	motor de reglas, ventanas de tiempo
360	8	Entradas vendidas de más	concurrency modelada, condiciones de carrera
361	8	Deadlock en la reserva de camas	detección de deadlocks, grafo de espera
362	8	El bot que prueba contraseñas de a poco	rate limiter de ventana deslizante
363	8	Motor de matching de órdenes de compra y venta	libro de órdenes, prioridad precio-tiempo
364	8	Servidores nuevos sin mudanza masiva	consistent hashing con réplicas virtuales
365	8	Mensajes entregados al menos una vez	idempotencia, deduplicación
366	8	¿Cuándo chocan dos códigos cortos?	probabilidad, paradoja del cumpleaños
367	8	El saldo de diez años en un segundo	event sourcing con snapshots
368	8	El proveedor de envíos que se cae	circuit breaker, tolerancia a fallos
369	8	Caché con vencimiento para el pronóstico	caché con TTL
370	8	Velocidad de transacciones sospechosa	ventanas múltiples, reglas ponderadas
371	8	Lectores y escritores en la historia clínica	concurrency modelada, lectores-escritores
372	8	Vuelo, hotel y auto: todo o nada	sagas, compensaciones
373	8	Cajas de botín y valor esperado	probabilidad, valor esperado
374	8	Órdenes stop y cancelaciones	libro de órdenes extendido
375	8	Repartir usuarios entre particiones	consistent hashing, rebalanceo
376	8	La nómina que se procesa de noche	reintentos, cola de mensajes fallidos
377	8	Si hoy llueve, ¿mañana también?	cadena de Markov
378	8	Stock reservado por 10 minutos	concurrency, expiración, consistencia
379	8	Doble gasto en la billetera virtual	concurrency modelada, orden de eventos
380	8	Límites por usuario y por país	rate limiter jerárquico
381	8	Cinco cocineros y cinco sartenes	deadlock, orden de recursos
382	8	El feed armado desde eventos	event sourcing, proyecciones
383	8	La racha que funde la caja chica	probabilidad, recurrencias
384	8	Escritura directa o diferida en el caché de stock	caché, consistencia
385	8	Dos recepcionistas, un mismo turno	control de concurrency optimista, versiones

N.º	Niv.	Título	Conceptos principales
386	8	Viajes compartidos que se emparejan solos	matching con prioridad, colas
387	8	¿El botón nuevo vende más?	probabilidad, intervalos de confianza
388	8	Webhooks de pago que llegan desordenados	idempotencia, orden de eventos
389	8	El caché de dos niveles del CDN	cachés jerárquicas
390	8	Motor de reglas para becas	motor de reglas con prioridades
391	8	Canciones al azar de un catálogo infinito	muestreo por reservorio, probabilidad
392	8	Órdenes iceberg en el libro	libro de órdenes avanzado
393	8	Anomalías en la telemetría del satélite	estadística, puntaje z móvil
394	8	Registro de accesos que no se puede editar	registro de solo agregado, verificación de integridad
395	8	La espera en los boxes de atención	probabilidad, simulación determinista con semilla
396	8	Riego limitado por sensores	rate limiting, estado
397	8	Servidores de partidas que se caen	tolerancia a fallos, reasignación
398	8	Desafío de nivel: Plataforma de pagos con reintentos y antifraude	integrador: reintentos, idempotencia, reglas antifraude
399	8	Desafío de nivel: La ticketera del recital masivo	integrador: rate limiter, concurrencia, reservas
400	8	Desafío de nivel: Un exchange de criptomonedas en miniatura	integrador: libro de órdenes, event sourcing, caché
401	9	Una cláusula en miles de contratos	KMP
402	9	La tasa que esconde el crédito	bisección
403	9	Cuántos camiones soporta la red de rutas	flujo máximo
404	9	Médicos para cada guardia	matching bipartito
405	9	El alambrado más corto del campo	envolvente convexa
406	9	Arbitraje de divisas	Bellman-Ford, ciclos negativos
407	9	Distancias entre todas las ciudades del tour	Floyd-Warshall
408	9	Plagio en los trabajos prácticos	rolling hash
409	9	La TIR de un préstamo con flujos irregulares	Newton-Raphson
410	9	La superficie de la laguna	área de polígonos
411	9	Patrones repetidos en el genoma	función Z
412	9	Cortar la red con el menor daño	corte mínimo
413	9	El rebaño en el año 10^{18}	exponenciación de matrices
414	9	Balancear reacciones químicas	eliminación gaussiana
415	9	¿El repartidor está dentro de la zona?	punto en polígono
416	9	Lo que tienen en común dos logs gigantes	suffix array, LCP
417	9	Habitaciones según preferencias	matching bipartito
418	9	Corrientes del circuito eléctrico	eliminación gaussiana
419	9	Rutas aéreas que se cruzan	intersección de segmentos
420	9	El precio de un bono según su rendimiento	Newton-Raphson, derivadas
421	9	Evacuación del estadio	flujo máximo
422	9	Publicaciones casi idénticas	rolling hash, ventanas
423	9	Descuentos que se pueden explotar en fletes	Bellman-Ford, detección de ciclos negativos
424	9	Las dos antenas más cercanas	par de puntos más cercano, divide y vencerás
425	9	Subdominios distintos	suffix array
426	9	Mora que se propaga mes a mes	exponenciación de matrices, cadenas de Markov
427	9	Proyectos a equipos con capacidades	flujo máximo con capacidades
428	9	El punto de equilibrio del bar	bisección, funciones no lineales
429	9	La melodía en otra tonalidad	KMP sobre diferencias
430	9	Superficie vigilada por cámaras	barrido, área de unión de rectángulos
431	9	Tasas forward a partir de la curva	álgebra lineal, bootstrapping de tasas
432	9	¿Se puede llegar en tren?	Floyd-Warshall, cierre transitivo
433	9	Tutores y alumnos	matching bipartito de máxima cardinalidad
434	9	El candado de combinación circular	rotación lexicográfica mínima
435	9	Trayectoria del proyectil del juego	Newton-Raphson, física
436	9	Parcelas superpuestas	barrido con árbol de segmentos
437	9	Precios de la canasta a partir de tickets	eliminación gaussiana
438	9	El palíndromo más largo de los posteos	algoritmo de Manacher
439	9	Bicicletas públicas con demandas por estación	flujo con demandas
440	9	Visibilidad desde la torre forestal	geometría, ordenamiento angular
441	9	Sensibilidad de la cartera de bonos	métodos numéricos, derivadas

N.º	Niv.	Título	Conceptos principales
442	9	Un logo escondido en la imagen	rolling hash en dos dimensiones
443	9	El mejor lugar para el hospital	búsqueda ternaria, geometría
444	9	Vuelos con límite de escalas	Bellman-Ford acotado
445	9	Proyectos rentables que dependen de otros	corte mínimo, clausura de peso máximo
446	9	Pacientes y donantes compatibles	matching bipartito
447	9	Pronóstico de demanda eléctrica	mínimos cuadrados, ecuaciones normales
448	9	Desafío de nivel: La mesa de dinero	integrador: Bellman-Ford, Newton-Raphson, matrices
449	9	Desafío de nivel: Plataforma de logística urbana	integrador: flujo, geometría, strings
450	9	Desafío de nivel: Detector de fraude documental	integrador: suffix array, rolling hash, matching
451	10	Mil millones de sitios peligrosos en poca memoria	Bloom filter
452	10	¿Cuánta gente distinta entró hoy a la tienda?	HyperLogLog
453	10	Las canciones más escuchadas con memoria mínima	Count-Min sketch
454	10	Comprimir los mensajes de la flota	codificación de Huffman
455	10	Recibos que nadie puede falsificar	RSA, exponenciación modular
456	10	¿Quién escribió primero en el grupo?	relojes vectoriales
457	10	Votos del jugador del partido en varios servidores	CRDT de contadores
458	10	Probar que una transacción está en el lote	árbol de Merkle
459	10	Historias clínicas en tres centros de datos	quórum de lectura y escritura
460	10	Comprimir los logs de los sensores	LZ77
461	10	Todas las cosechas posibles de la cooperativa	FFT, multiplicación de polinomios
462	10	Alimento balanceado al menor costo	programación lineal, simplex
463	10	Cuánto podemos perder en un mal día	VaR, simulación de Monte Carlo
464	10	El recorrido de 300 entregas	recocido simulado
465	10	El precio justo de una opción	Black-Scholes
466	10	La cartera óptima con presupuesto entero	branch & bound
467	10	Elección de líder entre servidores	consenso simplificado
468	10	El libro de actas inalterable	cadena de hashes
469	10	Rentabilidad sin perder el sueño	portafolio de Markowitz, optimización
470	10	La receta colaborativa sin conflictos	CRDT de secuencias
471	10	Reseñas casi duplicadas a escala	MinHash, LSH
472	10	La vibración del motor del colectivo	FFT, análisis de frecuencias
473	10	El cronograma de enfermería que conforma a todos	recocido simulado
474	10	Inventario sincronizado entre depósitos	relojes vectoriales, resolución de conflictos
475	10	Dos réplicas, ¿en qué difieren?	árboles de Merkle, anti-entropía
476	10	Escaneos de puertos con memoria mínima	Count-Min sketch, heavy hitters
477	10	Despacho eléctrico al menor costo	programación lineal, simplex
478	10	Cien dados sobre la mesa	FFT, probabilidad
479	10	Comprimir partituras	Huffman, LZ77
480	10	Prueba de trabajo de juguete	cadena de hashes, dificultad ajustable
481	10	Turistas únicos por región	HyperLogLog con fusión
482	10	Choferes para cada servicio de larga distancia	branch & bound
483	10	La opción que se puede ejercer antes	árbol binomial, valuación de opciones
484	10	Tabla de posiciones replicada con particiones	quórum, particiones de red
485	10	Productos dados de baja en memoria mínima	Bloom filter con contadores
486	10	Sensores que mienten	consenso con fallas bizantinas simplificado
487	10	Intercambio de claves de juguete	aritmética modular, criptografía didáctica
488	10	Riesgo de crédito con incumplimientos correlacionados	Monte Carlo correlacionado, descomposición de Cholesky
489	10	Recorrido de las cosechadoras	branch & bound, problema del viajante
490	10	Contratos de energía entre vecinos productores	programación lineal, simplex
491	10	Índice comprimido del buscador de recetas	compresión por diferencias, índices invertidos
492	10	Turnos de fábrica que respetan a todos	recocido simulado, restricciones duras y blandas
493	10	El feed que llega desordenado	relojes lógicos híbridos
494	10	Telemetría de un país entero en poca memoria	integrador: HyperLogLog, Count-Min, ventanas
495	10	Rendición de cuentas de un fondo verificable	árbol de Merkle, RSA
496	10	Radiografías más nítidas	FFT, convolución
497	10	Qué avión vuela cada ruta	branch & bound, programación lineal

N.º	Niv.	Título	Conceptos principales
498	10	Desafío de nivel: Ledger distribuido que sobrevive a cortes de red	integrador: CRDT, quórums, árbol de Merkle
499	10	Desafío de nivel: Antifraude a escala planetaria	integrador: Bloom, Count-Min, reglas, relojes vectoriales
500	10	Desafío de nivel: La mesa de riesgo del banco	integrador: VaR, Markowitz, Black-Scholes, simplex

Apéndice B: glosario

Algoritmo voraz. Estrategia que en cada paso toma la mejor decisión local, sin volver atrás. Solo es correcta cuando el problema tiene la propiedad de elección voraz.

Backtracking. Búsqueda que construye soluciones paso a paso y retrocede en cuanto una decisión parcial no puede llevar a una solución válida.

BFS. Recorrido en anchura de un grafo, por niveles; encuentra caminos con la menor cantidad de aristas.

Bisección. Método numérico que encuentra una raíz partiendo a la mitad un intervalo donde la función cambia de signo.

Bloom, filtro de. Estructura de bits que responde "seguro que no está" o "probablemente está", con memoria mínima y sin falsos negativos.

Búsqueda binaria. Búsqueda en datos ordenados (o sobre una respuesta monótona) que descarta la mitad del espacio en cada paso.

Caché. Almacenamiento rápido y limitado que guarda resultados recientes o frecuentes para evitar consultas costosas.

Cadena de hashes. Secuencia donde cada hash incluye al anterior; cambiar un elemento cambia todos los hashes siguientes.

Circuit breaker. Mecanismo que deja de llamar a un servicio que falla y prueba periódicamente si se recuperó.

Complejidad. Cómo crece el tiempo o la memoria de un algoritmo según el tamaño de la entrada; se expresa con la notación O .

Consistencia eventual. Garantía de que, si dejan de llegar cambios, todas las réplicas terminan con el mismo estado.

Corte mínimo. Conjunto de aristas de menor capacidad total cuya eliminación separa dos nodos; su valor es igual al flujo máximo.

Count-Min sketch. Tabla de contadores con varias funciones de hash que estima frecuencias en flujos enormes, sin subestimar nunca.

CRDT. Tipo de dato replicado que converge sin coordinación: sus operaciones de fusión son conmutativas, asociativas e idempotentes.

Deadlock. Situación en la que varios procesos esperan recursos que tienen otros procesos del mismo grupo, y ninguno puede avanzar.

DFS. Recorrido en profundidad de un grafo: avanza lo más posible por un camino antes de retroceder.

Dijkstra. Algoritmo de caminos mínimos desde un origen con pesos no negativos, usando una cola de prioridad.

Divide y vencerás. Estrategia que parte el problema en subproblemas independientes, los resuelve y combina los resultados.

Eliminación gaussiana. Método para resolver sistemas de ecuaciones lineales llevando la matriz a forma escalonada.

Envolverte convexa. Menor polígono convexo que contiene a un conjunto de puntos.

Event sourcing. Diseño donde el estado se guarda como la lista de eventos que lo produjeron, y se reconstruye reproduciéndolos.

Exponenciación rápida. Cálculo de potencias con $O(\log n)$ multiplicaciones, elevando al cuadrado según los bits del exponente.

FFT. Transformada rápida de Fourier: pasa de coeficientes a frecuencias en $O(n \log n)$; multiplica polinomios y analiza señales.

Flujo máximo. Mayor cantidad que puede circular de un origen a un destino respetando la capacidad de cada arista.

Función hash. Función que transforma un dato en un número de tamaño fijo, de forma determinista y bien distribuida.

Grafo. Conjunto de nodos unidos por aristas, dirigidas o no, con o sin pesos.

Hashing consistente. Forma de repartir claves entre servidores en un anillo, de modo que agregar o quitar un servidor mueva pocas claves.

Huffman. Código de largo variable óptimo por símbolos: los frecuentes reciben códigos cortos.

HyperLogLog. Estructura que estima la cantidad de elementos distintos de un flujo con unos pocos kilobytes.

Idempotencia. Propiedad de una operación que produce el mismo efecto aunque se ejecute varias veces.

Inverso modular. Número que multiplicado por otro da 1 módulo m ; existe si ambos son coprimos.

KMP. Algoritmo de búsqueda de patrones en $O(n + m)$ que nunca retrocede en el texto.

LRU / LFU. Políticas de desalojo de cachés: menos usado recientemente / menos usado en frecuencia.

LZ77. Compresión que reemplaza repeticiones por referencias hacia atrás (desplazamiento y largo).

Matching bipartito. Emparejamiento entre dos grupos donde cada elemento se asocia con a lo sumo uno del otro grupo.

MinHash / LSH. Firmas cortas que estiman la similitud de conjuntos, y agrupación por bandas para encontrar pares similares sin comparar todos.

Monte Carlo. Estimación mediante muchas simulaciones aleatorias con una semilla reproducible.

Montículo (heap). Árbol que mantiene el mínimo (o máximo) accesible en $O(1)$, con inserción y extracción en $O(\log n)$.

Máquina de estados. Modelo con un conjunto finito de estados y transiciones según los eventos que llegan.

Newton-Raphson. Método numérico que busca raíces siguiendo la recta tangente; converge muy rápido cerca de la solución.

Ordenamiento topológico. Orden de los nodos de un grafo dirigido sin ciclos en el que toda arista va hacia adelante.

Programación dinámica. Técnica que resuelve un problema combinando soluciones guardadas de subproblemas superpuestos.

Programación lineal. Optimización de una función lineal sujeta a restricciones lineales; se resuelve, por ejemplo, con el simplex.

Quórum. Cantidad mínima de réplicas que deben participar en una lectura o escritura para garantizar consistencia.

Ramificación y poda. Búsqueda exhaustiva que descarta ramas cuya cota optimista no puede mejorar la mejor solución encontrada.

Rate limiter. Mecanismo que limita la cantidad de pedidos por unidad de tiempo (balde de fichas, ventana fija o deslizante).

Recocido simulado. Metaheurística que acepta a veces soluciones peores, con una probabilidad que baja con la temperatura, para escapar de óptimos locales.

Relojes vectoriales. Vectores de contadores que capturan la causalidad entre eventos de distintos nodos.

Saga. Secuencia de pasos distribuidos donde cada paso tiene una compensación que se ejecuta si un paso posterior falla.

Semilla. Valor inicial de un generador de números pseudoaleatorios; con la misma semilla se obtiene la misma secuencia.

Suffix array. Arreglo con los sufijos de un texto ordenados; junto con el LCP resuelve problemas de subcadenas.

Sumas prefijas. Acumulados que permiten obtener la suma de cualquier tramo en $O(1)$.

Trie. Árbol de prefijos donde cada camino desde la raíz forma una palabra.

Union-find. Estructura que mantiene conjuntos disjuntos con uniones y consultas casi constantes.

VaR. Valor en riesgo: pérdida que no se supera con cierta confianza en un período dado.

Ventana deslizante. Técnica que mantiene un tramo contiguo que avanza sobre los datos, actualizando su resumen en $O(1)$.

XOR. O exclusivo bit a bit; cada valor es su propio inverso ($x \text{ XOR } x = 0$).

Árbol de Fenwick. Arreglo que guarda sumas de bloques de tamaño potencia de 2 y permite sumar por prefijo y actualizar en $O(\log n)$.

Árbol de Merkle. Árbol de hashes donde cada nodo resume a sus hijos; permite probar que un dato pertenece a un conjunto con pocos hashes.

Árbol de segmentos. Árbol binario sobre un arreglo donde cada nodo guarda un resumen (suma, mínimo, máximo) de un tramo; responde consultas por rango y actualizaciones en $O(\log n)$.